

3. Code fragments with comments

Ada (body)

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 6) - [internal]

```
Power_Of_Two

function Power_Of_Two (Exponent : Block_Size) return Phys_Addr is
begin
    return Phys_Addr (Shift_Left (1, Integer (Exponent)));
end Power_Of_Two;

function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean is
    Mask : constant Phys_Addr := Power_Of_Two (Size_Exp) - 1;
begin
    -- Udowodnienie, że (Addr mod 2^N) == 0 jest równoważne (Addr and (2^N - 1)) == 0
    return (Addr and Mask) = 0;
end Is_Aligned;

// ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 11) - [internal]

```
Is_Aligned

function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean is
    Mask : constant Phys_Addr := Power_Of_Two (Size_Exp) - 1;
begin
    -- Udowodnienie, że (Addr mod 2^N) == 0 jest równoważne (Addr and (2^N - 1)) == 0
    return (Addr and Mask) = 0;
end Is_Aligned;

function Get_Buddy_Address (
    Addr      : Phys_Addr;
    Size_Exp  : Block_Size
) return Phys_Addr is
    -- Kluczowy bit maski dla alokatora Buddy
    // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 18) - [internal]

```
Get_Buddy_Address

function Get_Buddy_Address (
    Addr      : Phys_Addr;
    Size_Exp  : Block_Size
) return Phys_Addr is
    -- Kluczowy bit maski dla alokatora Buddy
    Bit_Mask : constant Phys_Addr := Power_Of_Two (Size_Exp);
begin
    -- XOR z maską odwraca dokładnie ten jeden bit, który definiuje pozycję buddy.
    -- SPARK udowodni, że ponieważ Addr jest wyrównane (Pre condition z .ads),
    -- to wynik również będzie wyrównany, a XOR nie spowoduje przepełnienia.
    return Addr xor Bit_Mask;
end Get_Buddy_Address;
// ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 31) - [internal]

```
Get_Left_Child_Address

function Get_Left_Child_Address (
    Parent_Addr : Phys_Addr;
    Parent_Exp  : Block_Size
) return Phys_Addr is
begin
    -- Lewe dziecko zawsze zaczyna się tam, gdzie rodzic
    return Parent_Addr;
end Get_Left_Child_Address;

function Get_Right_Child_Address (
    Parent_Addr : Phys_Addr;
    Parent_Exp  : Block_Size
) return Phys_Addr is
    // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 40) - [internal]

Get_Right_Child_Address

```
function Get_Right_Child_Address (
  Parent_Addr : Phys_Addr;
  Parent_Exp  : Block_Size
) return Phys_Addr is
  -- Prawe dziecko to rodzic + połowa rozmiaru rodzica
  Half_Size : constant Phys_Addr := Power_Of_Two (Parent_Exp - 1);
begin
  return Parent_Addr + Half_Size;
end Get_Right_Child_Address;

function Safe_Calculate_DMA_End (
  Base_Addr : Phys_Addr;
  // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.adb (line 50) - [internal]

Safe_Calculate_DMA_End

```
function Safe_Calculate_DMA_End (
  Base_Addr : Phys_Addr;
  Buffer_Len : Phys_Addr
) return Phys_Addr is
begin
  -- Ponieważ Pre condition w .ads gwarantuje, że Base + Len <= Last,
  -- kompilator SPARK wie, że to dodawanie nigdy nie "owinie" (wrap-around) pamięci.
  return Base_Addr + Buffer_Len;
end Safe_Calculate_DMA_End;

end DS_Buddy_Math;
```

kernel_Workspace/kernel/src/policy/ada/policy.adb (line 5) - [internal] Evaluate

```
function Evaluate (Ring : Ring_Id;
  Class : Call_Class;
  Op    : Unsigned_8;
  Arg   : U64)
  return Decision
is
  pragma Unreferenced (Arg);
begin
  if Ring = Ring_Kernel then
    return Allow;
  end if;

  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/ada/policy_c.adb (line 5) - [internal] To_Ring

```
function To_Ring (V : Unsigned_8) return Ring_Id is
  (case V is
    when 0 => Ring_Kernel,
    when 1 => Ring_Driver,
    when others => Ring_User);

function To_Class (V : Unsigned_8) return Call_Class is
  (case V is
    when 0 => Cls_Sys,
    when 1 => Cls_Video,
    when 2 => Cls_Audio,
    when 3 => Cls_Input,
    // ... (truncated, see source file for more)
```

```

kernel_workspace/kernel/src/policy/ada/policy_c.adb (line 11) - [internal] To_Class
function To_Class (V : Unsigned_8) return Call_Class is
  (case V is
    when 0 => Cls_Sys,
    when 1 => Cls_Video,
    when 2 => Cls_Audio,
    when 3 => Cls_Input,
    when 4 => Cls_Block,
    when others => Cls_Net);

function policy_evaluate (ring : Unsigned_8;
                          cls  : Unsigned_8;
                          op   : Unsigned_8;
                          // ... (truncated, see source file for more)

kernel_workspace/kernel/src/policy/ada/policy_c.adb (line 20) - [internal] policy_evaluate
function policy_evaluate (ring : Unsigned_8;
                          cls  : Unsigned_8;
                          op   : Unsigned_8;
                          arg  : Unsigned_64)
  return Unsigned_8
is
  D : constant Decision :=
    Evaluate (To_Ring (ring), To_Class (cls), op, U64 (arg));
begin
  return (case D is
    when Allow => 0,
    when Deny  => 1);
  // ... (truncated, see source file for more)

```

Ada (spec)

```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 4) - [public]
DS_Buddy_Math
package DS_Buddy_Math with
  SPARK_Mode,
  Pure
is
  -- Typy bazowe dla fizycznej pamieci (bare-metal)
  type Phys_Addr is new Unsigned_64;
  type Block_Size is new Unsigned_64;

  -- Minimalny i maksymalny rozmiar bloku (np. 4KB do 1GB)
  MIN_BLOCK_SIZE : constant Block_Size := 12; -- 2^12 = 4096 bytes (4KB)
  MAX_BLOCK_SIZE : constant Block_Size := 30; -- 2^30 = 1GB

  // ... (truncated, see source file for more)

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 17) - [public]
Power_Of_Two
  -- Funkcja pomocnicza: Obliczanie 2^N (Shift left)

  function Power_Of_Two (Exponent : Block_Size) return Phys_Addr with
    Pre  => Exponent <= 63,
    -- Use a bit-shift to produce a value of type Unsigned_64 / Phys_Addr
    Post => Power_Of_Two'Result = Shift_Left (Phys_Addr (1), Integer (Exponent));

  -- Sprawdzenie, czy adres jest poprawnie wyrównany do rozmiaru bloku
  function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean with
    Post => Is_Aligned'Result = ((Addr mod Power_Of_Two (Size_Exp)) = 0);

  -- =====
  -- ALGORYTM 1: OBLICZANIE ADRESU BUDDY (PAROWANIE)
  -- =====
  // ... (truncated, see source file for more)

```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 23) - [public]

Is_Aligned

-- Sprawdzenie, czy adres jest poprawnie wyrównany do rozmiaru bloku

```
function Is_Aligned (Addr : Phys_Addr; Size_Exp : Block_Size) return Boolean with
  Post => Is_Aligned'Result = ((Addr mod Power_Of_Two (Size_Exp)) = 0);
```

```
-- =====
-- ALGORYTM 1: OBLICZANIE ADRESU BUDDY (PAROWANIE)
-- =====
-- W alokatorze Buddy, "buddy" bloku to blok o tym samym rozmiarze,
-- który różni się tylko jednym bitem (na pozycji odpowiadającej rozmiarowi).
```

```
function Get_Buddy_Address (
  Addr      : Phys_Addr;
  Size_Exp  : Block_Size
  // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 32) - [public]

Get_Buddy_Address

```
-- =====
-- ALGORYTM 1: OBLICZANIE ADRESU BUDDY (PAROWANIE)
-- =====
```

```
-- W alokatorze Buddy, "buddy" bloku to blok o tym samym rozmiarze,
-- który różni się tylko jednym bitem (na pozycji odpowiadającej rozmiarowi).
```

```
function Get_Buddy_Address (
  Addr      : Phys_Addr;
  Size_Exp  : Block_Size
) return Phys_Addr with
  Pre => Size_Exp >= MIN_BLOCK_SIZE and then Size_Exp <= MAX_BLOCK_SIZE,
  Post => Is_Aligned (Get_Buddy_Address'Result, Size_Exp),
  Post => Get_Buddy_Address'Result /= Addr, -- Buddy NIGDY nie jest tym samym adresem
  Post => Shift_Right (Get_Buddy_Address'Result, Integer (Size_Exp)) =
    (Shift_Right (Addr, Integer (Size_Exp)) xor Phys_Addr (1)); -- Udowodniona różnica dok
```

```
-- =====
-- ALGORYTM 2: DZIELENIE BLOKU (SPLITTING)
// ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 47) - [public]

Get_Left_Child_Address

```
-- =====
-- ALGORYTM 2: DZIELENIE BLOKU (SPLITTING)
-- =====
```

-- Dzielenie bloku o rozmiarze 2^N na dwa bloki o rozmiarze $2^{(N-1)}$.

```
function Get_Left_Child_Address (
  Parent_Addr : Phys_Addr;
  Parent_Exp  : Block_Size
) return Phys_Addr with
  Pre => Parent_Exp > MIN_BLOCK_SIZE and then Is_Aligned (Parent_Addr, Parent_Exp),
  Post => Get_Left_Child_Address'Result = Parent_Addr,
  Post => Is_Aligned (Get_Left_Child_Address'Result, Parent_Exp - 1);
```

```
function Get_Right_Child_Address (
  Parent_Addr : Phys_Addr;
  Parent_Exp  : Block_Size
) return Phys_Addr with
  // ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 55) - [public]

Get_Right_Child_Address

```
function Get_Right_Child_Address (
  Parent_Addr : Phys_Addr;
  Parent_Exp  : Block_Size
) return Phys_Addr with
  Pre  => Parent_Exp > MIN_BLOCK_SIZE and then Is_Aligned (Parent_Addr, Parent_Exp),
  Post => Get_Right_Child_Address'Result = Parent_Addr + Power_Of_Two (Parent_Exp - 1),
  Post => Is_Aligned (Get_Right_Child_Address'Result, Parent_Exp - 1),
  Post => Get_Right_Child_Address'Result > Parent_Addr; -- Prawe dziecko jest zawsze wyżej w pamięci

-- =====
-- ALGORYTM 3: BEZPIECZNE OBLICZANIE OFFSETU DMA
-- =====
// ... (truncated, see source file for more)
```

driverspace_workspace/formal/ds-spark-core/src/ds_buddy_math.ads (line 69) - [public]

Safe_Calculate_DMA_End

```
-- =====
-- ALGORYTM 3: BEZPIECZNE OBLICZANIE OFFSETU DMA
-- =====
-- Krytyczne dla sterowników: obliczanie adresu końcowego bufora bez przepełnienia (Overflow).

function Safe_Calculate_DMA_End (
  Base_Addr : Phys_Addr;
  Buffer_Len : Phys_Addr
) return Phys_Addr with
  Pre  => Base_Addr <= Phys_Addr'Last - Buffer_Len, -- Gwarancja braku przepełnienia UInt64
  Post => Safe_Calculate_DMA_End'Result = Base_Addr + Buffer_Len;

-- Eksport do C/Rust (FFI)
pragma Export (C, Get_Buddy_Address, "ds_spark_buddy_get_buddy");
pragma Export (C, Get_Left_Child_Address, "ds_spark_buddy_left_child");
pragma Export (C, Get_Right_Child_Address, "ds_spark_buddy_right_child");
pragma Export (C, Safe_Calculate_DMA_End, "ds_spark_safe_dma_end");
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/ada/policy.ads (line 3) - [public] Policy

```
package Policy
with SPARK_Mode
is

  type Ring_Id is (Ring_Kernel, Ring_Driver, Ring_User)
    with Size => 8;

  type Call_Class is (Cls_Sys, Cls_Video, Cls_Audio,
    Cls_Input, Cls_Block, Cls_Net)
    with Size => 8;

  type Decision is (Allow, Deny)
  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/ada/policy.ads (line 21) - [public] Evaluate

```
function Evaluate (Ring : Ring_Id;
  Class : Call_Class;
  Op    : Unsigned_8;
  Arg   : U64)
  return Decision

with
  Global => null,
  Post  =>
    (if Ring = Ring_Kernel then Evaluate'Result = Allow)
    and then
    (if Ring = Ring_User
      and then Class = Cls_Block
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/policy/ada/policy_c.ads (line 4) - [public] Policy_C

```
package Policy_C is

    function policy_evaluate (ring : Unsigned_8;
                              cls  : Unsigned_8;
                              op   : Unsigned_8;
                              arg  : Unsigned_64)
        return Unsigned_8

    with Convention => C,
         Export,
         External_Name => "policy_evaluate";

end Policy_C;
```

kernel_workspace/kernel/src/policy/ada/policy_c.ads (line 6) - [public] policy_evaluate

```
function policy_evaluate (ring : Unsigned_8;
                          cls  : Unsigned_8;
                          op   : Unsigned_8;
                          arg  : Unsigned_64)
    return Unsigned_8

with Convention => C,
     Export,
     External_Name => "policy_evaluate";

end Policy_C;
```

Assembly

comgrub/src/boot.asm (line 10) - [internal] mboot_header_start

```
mboot_header_start:
    dd MBOOT2_MAGIC
    dd MBOOT2_ARCH
    dd MBOOT2_LENGTH
    dd MBOOT2_CHECKSUM

    dw 0
    dw 0
    dd 8
mboot_header_end:

section .bss
    // ... (truncated, see source file for more)
```

comgrub/src/boot.asm (line 19) - [internal] mboot_header_end

```
mboot_header_end:

section .bss
align 16
stack_bottom:
    resb 65536
stack_top:

section .text
bits 32

global _start
    // ... (truncated, see source file for more)
```

comgrub/src/boot.asm (line 23) - [internal] stack_bottom

```
stack_bottom:
    resb 65536
stack_top:

section .text
bits 32

global _start
extern _rust_entry

_start:
    mov esp, stack_top
    // ... (truncated, see source file for more)
```

comgrub/src/boot.asm (line 25) - [internal] stack_top

```
stack_top:

section .text
bits 32

global _start
extern _rust_entry

_start:
    mov esp, stack_top

    push ebx                ; Multiboot2: EBX = adres struktury informacyjnej
    // ... (truncated, see source file for more)
```

comgrub/src/boot.asm (line 30) - [public] _start

```
global _start
extern _rust_entry

_start:
    mov esp, stack_top

    push ebx                ; Multiboot2: EBX = adres struktury informacyjnej
    push eax                ; Multiboot2: EAX = magic

    call _rust_entry

.hang:
    // ... (truncated, see source file for more)
```

comgrub/src/boot.asm (line 33) - [internal] _start

```
_start:
    mov esp, stack_top

    push ebx                ; Multiboot2: EBX = adres struktury informacyjnej
    push eax                ; Multiboot2: EAX = magic

    call _rust_entry

.hang:
    cli
    hlt
    jmp .hang
```

kernel_Workspace/kernel-bin/src/boot.asm (line 3) - [public] _start

```
; kernel_Workspace/kernel-bin/src/boot.asm
```

```
global _start
extern kernel_main
extern __bss_start
extern __bss_end

section .text
bits 64

_start:
; 1. Clear BSS section
mov rdi, __bss_start
mov rcx, __bss_end
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel-bin/src/boot.asm (line 11) - [internal] _start

```
_start:
; 1. Clear BSS section
mov rdi, __bss_start
mov rcx, __bss_end
sub rcx, rdi
shr rcx, 3 ; Divide by 8 (clearing 64-bit words)
xor rax, rax
rep stosq

; 2. Set up a temporary boot stack (if not provided by bootloader)
; We allocate 64KB for the initial boot stack
section .bss
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel-bin/src/boot.asm (line 24) - [internal] boot_stack_bottom

```
boot_stack_bottom:
resb 65536
boot_stack_top:

section .text
lea rsp, [boot_stack_top]

; 3. Align stack to 16 bytes (System V AMD64 ABI requirement)
and rsp, -16

; 4. Call Rust kernel_main
; rdi already contains the boot_info_ptr passed by the bootloader
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel-bin/src/boot.asm (line 26) - [internal] boot_stack_top

```
boot_stack_top:

section .text
lea rsp, [boot_stack_top]

; 3. Align stack to 16 bytes (System V AMD64 ABI requirement)
and rsp, -16

; 4. Call Rust kernel_main
; rdi already contains the boot_info_ptr passed by the bootloader
call kernel_main

// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/cpu/trampoline.s (line 5) - [public] trampoline_start

```
.global trampoline_start
.global trampoline_end
.global trampoline_cr3
.global trampoline_entry
.global trampoline_stack
.global trampoline_arg
```

```
.code16
trampoline_start:
    cli
    cld
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 6) - [public] trampoline_end

```
.global trampoline_end
.global trampoline_cr3
.global trampoline_entry
.global trampoline_stack
.global trampoline_arg
```

```
.code16
trampoline_start:
    cli
    cld
```

```
    mov %cs, %ax
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 7) - [public] trampoline_cr3

```
.global trampoline_cr3
.global trampoline_entry
.global trampoline_stack
.global trampoline_arg
```

```
.code16
trampoline_start:
    cli
    cld
```

```
    mov %cs, %ax
    mov %ax, %ds
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 8) - [public] trampoline_entry

```
.global trampoline_entry
.global trampoline_stack
.global trampoline_arg
```

```
.code16
trampoline_start:
    cli
    cld
```

```
    mov %cs, %ax
    mov %ax, %ds
    mov %ax, %es
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/trampoline.s (line 9) - [public] trampoline_stack

```
.global trampoline_stack
.global trampoline_arg

.code16
trampoline_start:
    cli
    cld

    mov %cs, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/trampoline.s (line 10) - [public] trampoline_arg

```
.global trampoline_arg

.code16
trampoline_start:
    cli
    cld

    mov %cs, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/trampoline.s (line 13) - [internal] trampoline_start

```
trampoline_start:
    cli
    cld

    mov %cs, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss

    lgdt %cs:(trampoline_gdtr - trampoline_start)

    mov %cr0, %eax
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/trampoline.s (line 31) - [internal] trampoline_32

```
trampoline_32:

    mov $0x10, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss

    mov %cr4, %eax
    or $((1 << 5) | (1 << 9) | (1 << 10)), %eax
    mov %eax, %cr4

    movl $(TRAMPOLINE_BASE + (trampoline_cr3 - trampoline_start)), %eax
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 59) - [internal] trampoline_64

```
trampoline_64:

    mov $0x20, %ax
    mov %ax, %ds
    mov %ax, %es
    mov %ax, %ss
    xor %eax, %eax
    mov %ax, %fs
    mov %ax, %gs

    movabs $(TRAMPOLINE_BASE + (trampoline_arg - trampoline_start)), %rcx
    mov (%rcx), %rdi
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 81) - [internal] trampoline_gdt

```
trampoline_gdt:
    .quad 0x0000000000000000
    .quad 0x00CF9A000000FFFF
    .quad 0x00CF92000000FFFF
    .quad 0x00AF9A000000FFFF
    .quad 0x00AF92000000FFFF
trampoline_gdt_end:

.align 4
trampoline_gdtr:
    .word (trampoline_gdt_end - trampoline_gdt - 1)
    .long (TRAMPOLINE_BASE + (trampoline_gdt - trampoline_start))
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 87) - [internal] trampoline_gdt_end

```
trampoline_gdt_end:

.align 4
trampoline_gdtr:
    .word (trampoline_gdt_end - trampoline_gdt - 1)
    .long (TRAMPOLINE_BASE + (trampoline_gdt - trampoline_start))

.align 8
trampoline_cr3:    .quad 0
trampoline_entry:  .quad 0
trampoline_stack:  .quad 0
trampoline_arg:    .quad 0
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 90) - [internal] trampoline_gdtr

```
trampoline_gdtr:
    .word (trampoline_gdt_end - trampoline_gdt - 1)
    .long (TRAMPOLINE_BASE + (trampoline_gdt - trampoline_start))

.align 8
trampoline_cr3:    .quad 0
trampoline_entry:  .quad 0
trampoline_stack:  .quad 0
trampoline_arg:    .quad 0

.global trampoline_end
trampoline_end:
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 95) - [internal] trampoline_cr3

```
trampoline_cr3:    .quad 0
trampoline_entry:  .quad 0
trampoline_stack:  .quad 0
trampoline_arg:    .quad 0

.global trampoline_end
trampoline_end:
    .quad 0
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 96) - [internal] trampoline_entry

```
trampoline_entry: .quad 0
trampoline_stack: .quad 0
trampoline_arg:   .quad 0

.global trampoline_end
trampoline_end:
    .quad 0
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 97) - [internal] trampoline_stack

```
trampoline_stack: .quad 0
trampoline_arg:   .quad 0

.global trampoline_end
trampoline_end:
    .quad 0
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 98) - [internal] trampoline_arg

```
trampoline_arg:   .quad 0

.global trampoline_end
trampoline_end:
    .quad 0
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 100) - [public] trampoline_end

```
.global trampoline_end
trampoline_end:
    .quad 0
```

kernel_Workspace/kernel/src/cpu/trampoline.s (line 101) - [internal] trampoline_end

```
trampoline_end:
    .quad 0
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 3) - [public] tlb_asm_invlpg

```
.global tlb_asm_invlpg
.type tlb_asm_invlpg, @function
tlb_asm_invlpg:
    invlpg (%rdi)
    ret

.global tlb_asm_read_cr3
.type tlb_asm_read_cr3, @function
tlb_asm_read_cr3:
    mov %cr3, %rax
    ret

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 5) - [internal] tlb_asm_invlpg

```
tlb_asm_invlpg:
    invlpg (%rdi)
    ret

.global tlb_asm_read_cr3
.type tlb_asm_read_cr3, @function
tlb_asm_read_cr3:
    mov %cr3, %rax
    ret

.global tlb_asm_write_cr3
.type tlb_asm_write_cr3, @function
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 9) - [public] tlb_asm_read_cr3

```
.global tlb_asm_read_cr3
.type tlb_asm_read_cr3, @function
tlb_asm_read_cr3:
    mov %cr3, %rax
    ret

.global tlb_asm_write_cr3
.type tlb_asm_write_cr3, @function
tlb_asm_write_cr3:
    mov %rdi, %cr3
    ret

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 11) - [internal] tlb_asm_read_cr3

```
tlb_asm_read_cr3:
    mov %cr3, %rax
    ret

.global tlb_asm_write_cr3
.type tlb_asm_write_cr3, @function
tlb_asm_write_cr3:
    mov %rdi, %cr3
    ret

.global tlb_asm_read_cr4
.type tlb_asm_read_cr4, @function
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 15) - [public] tlb_asm_write_cr3

```
.global tlb_asm_write_cr3
.type tlb_asm_write_cr3, @function
tlb_asm_write_cr3:
    mov %rdi, %cr3
    ret

.global tlb_asm_read_cr4
.type tlb_asm_read_cr4, @function
tlb_asm_read_cr4:
    mov %cr4, %rax
    ret

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 17) - [internal] tlb_asm_write_cr3

```
tlb_asm_write_cr3:
    mov %rdi, %cr3
    ret

.global tlb_asm_read_cr4
.type tlb_asm_read_cr4, @function
tlb_asm_read_cr4:
    mov %cr4, %rax
    ret

.global tlb_asm_write_cr4
.type tlb_asm_write_cr4, @function
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 21) - [public] tlb_asm_read_cr4

```
.global tlb_asm_read_cr4
.type tlb_asm_read_cr4, @function
tlb_asm_read_cr4:
    mov %cr4, %rax
    ret

.global tlb_asm_write_cr4
.type tlb_asm_write_cr4, @function
tlb_asm_write_cr4:
    mov %rdi, %cr4
    ret

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 23) - [internal] tlb_asm_read_cr4

```
tlb_asm_read_cr4:
    mov %cr4, %rax
    ret

.global tlb_asm_write_cr4
.type tlb_asm_write_cr4, @function
tlb_asm_write_cr4:
    mov %rdi, %cr4
    ret

.global tlb_asm_invpuid
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 27) - [public] tlb_asm_write_cr4

```
.global tlb_asm_write_cr4
.type tlb_asm_write_cr4, @function
tlb_asm_write_cr4:
    mov %rdi, %cr4
    ret

.global tlb_asm_invpuid
.type tlb_asm_invpuid, @function
tlb_asm_invpuid:
    .byte 0x66, 0x0F, 0x38, 0x82, 0x3E
    ret
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 29) - [internal] tlb_asm_write_cr4

```
tlb_asm_write_cr4:
    mov %rdi, %cr4
    ret

.global tlb_asm_invpuid
.type tlb_asm_invpuid, @function
tlb_asm_invpuid:
    .byte 0x66, 0x0F, 0x38, 0x82, 0x3E
    ret

.global tlb_asm_wbinvd
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 34) - [public] tlb_asm_invpcid

```
.global tlb_asm_invpcid
.type tlb_asm_invpcid, @function
tlb_asm_invpcid:
    .byte 0x66, 0x0F, 0x38, 0x82, 0x3E
    ret
```

```
.global tlb_asm_wbinvd
.type tlb_asm_wbinvd, @function
tlb_asm_wbinvd:
    wbinvd
    ret
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 36) - [internal] tlb_asm_invpcid

```
tlb_asm_invpcid:
    .byte 0x66, 0x0F, 0x38, 0x82, 0x3E
    ret
```

```
.global tlb_asm_wbinvd
.type tlb_asm_wbinvd, @function
tlb_asm_wbinvd:
    wbinvd
    ret
```

```
.global tlb_asm_clflush
.type tlb_asm_clflush, @function
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 40) - [public] tlb_asm_wbinvd

```
.global tlb_asm_wbinvd
.type tlb_asm_wbinvd, @function
tlb_asm_wbinvd:
    wbinvd
    ret
```

```
.global tlb_asm_clflush
.type tlb_asm_clflush, @function
tlb_asm_clflush:
    clflush (%rdi)
    ret
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 42) - [internal] tlb_asm_wbinvd

```
tlb_asm_wbinvd:
    wbinvd
    ret

.global tlb_asm_clflush
.type tlb_asm_clflush, @function
tlb_asm_clflush:
    clflush (%rdi)
    ret
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 46) - [public] tlb_asm_clflush

```
.global tlb_asm_clflush
.type tlb_asm_clflush, @function
tlb_asm_clflush:
    clflush (%rdi)
    ret
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.s (line 48) - [internal] tlb_asm_clflush

```
tlb_asm_clflush:
    clflush (%rdi)
    ret
```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 4) -
[internal] vector_table

```
vector_table:
    .align 7
    b vector_default
    .align 7
    b vector_default
    .align 7
    b vector_default
    .align 7
    b vector_default

    .align 7
    b vector_default
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 41) -
[internal] vector_default

```
vector_default:
    eret

vector_hyper:
    sub sp, sp, #272

    stp x0, x1,    [sp, #0]
    stp x2, x3,    [sp, #16]
    stp x4, x5,    [sp, #32]
    stp x6, x7,    [sp, #48]
    stp x8, x9,    [sp, #64]
    stp x10, x11, [sp, #80]
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 44) -
[internal] vector_hyper

```
vector_hyper:
    sub sp, sp, #272

    stp x0, x1,    [sp, #0]
    stp x2, x3,    [sp, #16]
    stp x4, x5,    [sp, #32]
    stp x6, x7,    [sp, #48]
    stp x8, x9,    [sp, #64]
    stp x10, x11, [sp, #80]
    stp x12, x13, [sp, #96]
    stp x14, x15, [sp, #112]
    stp x16, x17, [sp, #128]
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 102) -
[public] tr_init

```
.global tr_init
.type tr_init, @function
tr_init:
    adr x1, vector_table
    msr vbar_el1, x1
    isb
    ret

.global tr_restore_ctx
.type tr_restore_ctx, @function
tr_restore_ctx:
    // ... (truncated, see source file for more)
```



```

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 104) -
[internal] tr_init
    tr_init:
        adr x1, vector_table
        msr vbar_el1, x1
        isb
        ret

.global tr_restore_ctx
.type tr_restore_ctx, @function
tr_restore_ctx:
    ldr x9, [x0, #248]
    msr sp_el0, x9
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 111) -
[public] tr_restore_ctx
.global tr_restore_ctx
.type tr_restore_ctx, @function
tr_restore_ctx:
    ldr x9, [x0, #248]
    msr sp_el0, x9
    ldr x9, [x0, #256]
    msr elr_el1, x9
    ldr x9, [x0, #264]
    msr spsr_el1, x9

    ldr x30, [x0, #240]
    ldp x28, x29, [x0, #224]
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/aarch64/trampoline_rings.s (line 113) -
[internal] tr_restore_ctx
tr_restore_ctx:
    ldr x9, [x0, #248]
    msr sp_el0, x9
    ldr x9, [x0, #256]
    msr elr_el1, x9
    ldr x9, [x0, #264]
    msr spsr_el1, x9

    ldr x30, [x0, #240]
    ldp x28, x29, [x0, #224]
    ldp x26, x27, [x0, #208]
    ldp x24, x25, [x0, #192]
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 3) -
[internal] kstack_top
    kstack_top:
        .dword 0

.section .text
.align 2

.global trap_entry
trap_entry:
    csrrw sp, sscratch, sp          # sp = kernel, sscratch = world sp
    addi sp, sp, -288

    sd x1, 8(sp)
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 9) -

[public] trap_entry

```
.global trap_entry
trap_entry:
    csrrw sp, sscratch, sp        # sp = kernel, sscratch = world sp
    addi sp, sp, -288

    sd x1, 8(sp)
    sd x3, 24(sp)
    sd x4, 32(sp)
    sd x5, 40(sp)
    sd x6, 48(sp)
    sd x7, 56(sp)
    sd x8, 64(sp)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 10) -

[internal] trap_entry

```
trap_entry:
    csrrw sp, sscratch, sp        # sp = kernel, sscratch = world sp
    addi sp, sp, -288

    sd x1, 8(sp)
    sd x3, 24(sp)
    sd x4, 32(sp)
    sd x5, 40(sp)
    sd x6, 48(sp)
    sd x7, 56(sp)
    sd x8, 64(sp)
    sd x9, 72(sp)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 98) -

[public] tr_init

```
.global tr_init
.type tr_init, @function
tr_init:
    la t0, kstack_top
    sd a0, 0(t0)

    csrw sscratch, a0

    la t0, trap_entry
    csrw stvec, t0

    li t0, (1 << 18)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 100) -

[internal] tr_init

```
tr_init:
    la t0, kstack_top
    sd a0, 0(t0)

    csrw sscratch, a0

    la t0, trap_entry
    csrw stvec, t0

    li t0, (1 << 18)
    csrs sstatus, t0

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 114) -
[public] tr_restore_ctx
    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    ld t0, 264(a0)
    csrw sepc, t0
    ld t0, 272(a0)
    csrw sstatus, t0

    la t0, kstack_top
    ld t0, 0(t0)
    csrw sscratch, t0

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/risc-v/trampoline_rings.s (line 116) -
[internal] tr_restore_ctx
    tr_restore_ctx:
        ld t0, 264(a0)
        csrw sepc, t0
        ld t0, 272(a0)
        csrw sstatus, t0

        la t0, kstack_top
        ld t0, 0(t0)
        csrw sscratch, t0

        ld sp, 256(a0)

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 3) -
[public] tr_gdt
    .global tr_gdt
tr_gdt:
    .quad 0x0000000000000000
    .quad 0x00209A0000000000
    .quad 0x0000920000000000
    .quad 0x0020BA0000000000
    .quad 0x0000B20000000000
    .quad 0x0020FA0000000000
    .quad 0x0000F20000000000
    .global tr_tss_desc
tr_tss_desc:
    .space 16
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 4) -
[internal] tr_gdt
    tr_gdt:
        .quad 0x0000000000000000
        .quad 0x00209A0000000000
        .quad 0x0000920000000000
        .quad 0x0020BA0000000000
        .quad 0x0000B20000000000
        .quad 0x0020FA0000000000
        .quad 0x0000F20000000000
    .global tr_tss_desc
tr_tss_desc:
    .space 16

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 12) -
[public] tr_tss_desc
    .global tr_tss_desc
tr_tss_desc:
    .space 16

gdt_ptr:
    .short (gdt_ptr - tr_gdt) - 1
    .quad tr_gdt

    .align 16
    .global tr_tss
tr_tss:
    .space 104
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 13) -
[internal] tr_tss_desc
    tr_tss_desc:
        .space 16

gdt_ptr:
    .short (gdt_ptr - tr_gdt) - 1
    .quad tr_gdt

    .align 16
    .global tr_tss
tr_tss:
    .space 104

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 16) -
[internal] gdt_ptr
gdt_ptr:
    .short (gdt_ptr - tr_gdt) - 1
    .quad tr_gdt

    .align 16
    .global tr_tss
tr_tss:
    .space 104

.section .text

.global tr_init
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 21) -
[public] tr_tss
    .global tr_tss
tr_tss:
    .space 104

.section .text

.global tr_init
.type tr_init, @function
tr_init:
    mov %rdi, %rax
    mov %eax, (tr_tss + 4)
    shr $32, %rax
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 22) -
[internal] tr_tss
    tr_tss:
        .space 104

    .section .text

    .global tr_init
    .type tr_init, @function
tr_init:
    mov %rdi, %rax
    mov %eax, (tr_tss + 4)
    shr $32, %rax
    mov %eax, (tr_tss + 8)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 27) -
[public] tr_init
    .global tr_init
    .type tr_init, @function
tr_init:
    mov %rdi, %rax
    mov %eax, (tr_tss + 4)
    shr $32, %rax
    mov %eax, (tr_tss + 8)

    lgdt gdt_ptr

    mov $0x10, %ax
    mov %ax, %ss
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 29) -
[internal] tr_init
tr_init:
    mov %rdi, %rax
    mov %eax, (tr_tss + 4)
    shr $32, %rax
    mov %eax, (tr_tss + 8)

    lgdt gdt_ptr

    mov $0x10, %ax
    mov %ax, %ss
    mov %ax, %ds
    mov %ax, %es
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 54) -
[public] isr_hypercall
    .global isr_hypercall
    .type isr_hypercall, @function
isr_hypercall:
    push %rax
    push %rbx
    push %rcx
    push %rdx
    push %rsi
    push %rdi
    push %rbp
    push %r8
    push %r9
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 56) -
[internal] isr_hypercall
    isr_hypercall:
        push %rax
        push %rbx
        push %rcx
        push %rdx
        push %rsi
        push %rdi
        push %rbp
        push %r8
        push %r9
        push %r10
        push %r11
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 94) -
[public] isr_default
    .global isr_default
    .type isr_default, @function
isr_default:
    iretq

    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    mov %rdi, %rsp
    pop %r15
    pop %r14
    pop %r13
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 96) -
[internal] isr_default
    isr_default:
        iretq

    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    mov %rdi, %rsp
    pop %r15
    pop %r14
    pop %r13
    pop %r12
    pop %r11
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 99) -
[public] tr_restore_ctx
    .global tr_restore_ctx
    .type tr_restore_ctx, @function
tr_restore_ctx:
    mov %rdi, %rsp
    pop %r15
    pop %r14
    pop %r13
    pop %r12
    pop %r11
    pop %r10
    pop %r9
    pop %r8
    // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/trampoline_rings/arch/x86_64/trampoline_rings.s (line 101) -
[internal] tr_restore_ctx

```
tr_restore_ctx:
    mov %rdi, %rsp
    pop %r15
    pop %r14
    pop %r13
    pop %r12
    pop %r11
    pop %r10
    pop %r9
    pop %r8
    pop %rbp
    pop %rdi
    // ... (truncated, see source file for more)
```

triang-lang/out/x86_64/prog.asm (line 4) - [internal] buffer

```
buffer: resb 16

section .text
global main
main:
fn_main:
    mov rax, 16
    mov rbx, 32
    mov rcx, rax
    add rcx, rbx
    lea r14, [rel buffer]
    mov r15, 16
    // ... (truncated, see source file for more)
```

triang-lang/out/x86_64/prog.asm (line 7) - [public] main

```
global main
main:
fn_main:
    mov rax, 16
    mov rbx, 32
    mov rcx, rax
    add rcx, rbx
    lea r14, [rel buffer]
    mov r15, 16
    mov r12, 0
fill_1:
    test r15, r15
    // ... (truncated, see source file for more)
```

triang-lang/out/x86_64/prog.asm (line 8) - [internal] main

```
main:
fn_main:
    mov rax, 16
    mov rbx, 32
    mov rcx, rax
    add rcx, rbx
    lea r14, [rel buffer]
    mov r15, 16
    mov r12, 0
fill_1:
    test r15, r15
    jz fill_end_2
    // ... (truncated, see source file for more)
```

triang-lang/out/x86_64/prog.asm (line 9) - [internal] fn_main

```
fn_main:
    mov rax, 16
    mov rbx, 32
    mov rcx, rax
    add rcx, rbx
    lea r14, [rel buffer]
    mov r15, 16
    mov r12, 0
fill_1:
    test r15, r15
    jz fill_end_2
    mov byte [r14], r12b
    // ... (truncated, see source file for more)
```

triang-lang/out/x86_64/prog.asm (line 17) - [internal] fill_1

```
fill_1:
    test r15, r15
    jz fill_end_2
    mov byte [r14], r12b
    inc r14
    dec r15
    jmp fill_1
fill_end_2:
    lea r14, [rel buffer]
    mov byte [r14 + 0], 65
    lea r14, [rel buffer]
    mov byte [r14 + 1], 66
    // ... (truncated, see source file for more)
```

triang-lang/out/x86_64/prog.asm (line 24) - [internal] fill_end_2

```
fill_end_2:
    lea r14, [rel buffer]
    mov byte [r14 + 0], 65
    lea r14, [rel buffer]
    mov byte [r14 + 1], 66
    lea r14, [rel buffer]
    mov byte [r14 + 2], 67
    lea r14, [rel buffer]
    movzx rax, byte [r14 + 0]
    lea r14, [rel buffer]
    movzx rbx, byte [r14 + 1]
    mov rcx, rax
    // ... (truncated, see source file for more)
```

triang-lang/out/x86_64/prog.asm (line 41) - [internal] else_1

```
else_1:
    mov rax, 0
end_2:
loop_3:
    cmp rax, 0
    je loop_end_4
    mov rbx, rax
    sub rbx, 1
    mov rax, rbx
    jmp loop_3
loop_end_4:
    ret
```

triang-lang/out/x86_64/prog.asm (line 43) - [internal] end_2

```
end_2:
loop_3:
    cmp rax, 0
    je loop_end_4
    mov rbx, rax
    sub rbx, 1
    mov rax, rbx
    jmp loop_3
loop_end_4:
    ret
```


triang-lang/out/x86_64/prog.asm (line 44) - [internal] loop_3

```
loop_3:
    cmp rax, 0
    je loop_end_4
    mov rbx, rax
    sub rbx, 1
    mov rax, rbx
    jmp loop_3
loop_end_4:
    ret
```

triang-lang/out/x86_64/prog.asm (line 51) - [internal] loop_end_4

```
loop_end_4:
    ret
```

C

comlimine/src/main.c (line 6) - [public] kernel_main

```
extern void kernel_main(uint64_t magic, void *info);

LIMINE_BASE_REVISION(1);

static volatile struct limine_framebuffer_request framebuffer = {
    .id = LIMINE_FRAMEBUFFER_REQUEST,
    .revision = 0,
    .response = NULL,
};

static volatile struct limine_memmap_request memmap = {
    .id = LIMINE_MEMMAP_REQUEST,
    // ... (truncated, see source file for more)
```

comlimine/src/main.c (line 28) - [public] _start

```
void _start(void) {
    if (framebuffer.response == NULL ||
        memmap.response == NULL ||
        hhdm.response == NULL) {
        for (;;) { asm volatile ("hlt"); }
    }

    kernel_main(0, (void *)framebuffer.response);

    for (;;) { asm volatile ("hlt"); }
}
```

comlimine/src/main.c (line 35) - [public] kernel_main

```
kernel_main(0, (void *)framebuffer.response);

for (;;) { asm volatile ("hlt"); }
}
```

drivers/vgpu/vgpu_driver.c (line 15) - [public] ds_poll

```
ds_poll();
ds_msg_t r;
ds_take(id, &r);
}

static uint64_t pci_val(uint32_t op, uint64_t a0, uint64_t a1, uint64_t a2)
{
    uint64_t id = ds_call(SVC_PCI, op, a0, a1, a2);
    ds_poll();
    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        // ... (truncated, see source file for more)
```

```

drivers/vgpu/vgpu_driver.c (line 17) - [public] ds_take
    ds_take(id, &r);
}

static uint64_t pci_val(uint32_t op, uint64_t a0, uint64_t a1, uint64_t a2)
{
    uint64_t id = ds_call(SVC_PCI, op, a0, a1, a2);
    ds_poll();
    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        return r.arg0;
    }
    // ... (truncated, see source file for more)
}

drivers/vgpu/vgpu_driver.c (line 23) - [public] ds_poll
    ds_poll();
    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        return r.arg0;
    }

    return 0;
}

static void vbe_write(uint16_t idx, uint16_t val)
{
    // ... (truncated, see source file for more)
}

drivers/vgpu/vgpu_driver.c (line 26) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        return r.arg0;
    }

    return 0;
}

static void vbe_write(uint16_t idx, uint16_t val)
{
    pci_void(PORT_WRITE, VBE_INDEX, idx, 16);
    pci_void(PORT_WRITE, VBE_DATA, val, 16);
}
// ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 35) - [public] pci_void
    pci_void(PORT_WRITE, VBE_INDEX, idx, 16);
    pci_void(PORT_WRITE, VBE_DATA, val, 16);
}

bool vgpu_init(uint32_t w, uint32_t h)
{
    if (g_ready) {
        return true;
    }

    uint64_t id = ds_call(SVC_PCI, PCI_FIND, 0x0300, 0, 0);
    // ... (truncated, see source file for more)
}

```

drivers/vgpu/vgpu_driver.c (line 36) - [public] pci_void

```
pci_void(PORT_WRITE, VBE_DATA, val, 16);
}
```

bool vgpu_init(uint32_t w, uint32_t h)

```
{
    if (g_ready) {
        return true;
    }

    uint64_t id = ds_call(SVC_PCI, PCI_FIND, 0x0300, 0, 0);
    ds_poll();
    // ... (truncated, see source file for more)
```

drivers/vgpu/vgpu_driver.c (line 42) - [public] if

```
if (g_ready) {
    return true;
}

uint64_t id = ds_call(SVC_PCI, PCI_FIND, 0x0300, 0, 0);
ds_poll();

ds_msg_t r;

if (!ds_take(id, &r) || r.status != 0) {
    return false;
}
// ... (truncated, see source file for more)
```

drivers/vgpu/vgpu_driver.c (line 47) - [public] ds_poll

```
ds_poll();

ds_msg_t r;

if (!ds_take(id, &r) || r.status != 0) {
    return false;
}

uint64_t bdf = r.arg0;

if (r.arg1 != (0x1234 | (0x1111 << 16))) {
    return false;
}
// ... (truncated, see source file for more)
```

drivers/vgpu/vgpu_driver.c (line 51) - [public] if

```
if (!ds_take(id, &r) || r.status != 0) {
    return false;
}

uint64_t bdf = r.arg0;

if (r.arg1 != (0x1234 | (0x1111 << 16))) {
    return false;
}

uint64_t fb_phys = pci_val(PCI_BAR, bdf, 0, 0);

// ... (truncated, see source file for more)
```

```

drivers/vgpu/vgpu_driver.c (line 57) - [public] if
    if (r.arg1 != (0x1234 | (0x1111 << 16))) {
        return false;
    }

    uint64_t fb_phys = pci_val(PCI_BAR, bdf, 0, 0);

    if (fb_phys == 0) {
        return false;
    }

    pci_void(PCI_ENABLE, bdf, 0, 0);

    // ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 63) - [public] if
    if (fb_phys == 0) {
        return false;
    }

    pci_void(PCI_ENABLE, bdf, 0, 0);

    vbe_write(4, 0);
    vbe_write(0, 0xB0C4);
    vbe_write(1, w);
    vbe_write(2, h);
    vbe_write(3, 32);
    vbe_write(4, 0x01 | 0x40 | 0x80);
    // ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 67) - [public] pci_void
    pci_void(PCI_ENABLE, bdf, 0, 0);

    vbe_write(4, 0);
    vbe_write(0, 0xB0C4);
    vbe_write(1, w);
    vbe_write(2, h);
    vbe_write(3, 32);
    vbe_write(4, 0x01 | 0x40 | 0x80);

    uint64_t mid = ds_call(SVC_SYS, OP_MAPMMIO, fb_phys,
                          (uint64_t)w * h * 4, 0x45000000ULL);
    ds_poll();
    // ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 69) - [public] vbe_write
    vbe_write(4, 0);
    vbe_write(0, 0xB0C4);
    vbe_write(1, w);
    vbe_write(2, h);
    vbe_write(3, 32);
    vbe_write(4, 0x01 | 0x40 | 0x80);

    uint64_t mid = ds_call(SVC_SYS, OP_MAPMMIO, fb_phys,
                          (uint64_t)w * h * 4, 0x45000000ULL);
    ds_poll();

    ds_msg_t mr;
    // ... (truncated, see source file for more)

```

```
drivers/vgpu/vgpu_driver.c (line 70) - [public] vbe_write
    vbe_write(0, 0xB0C4);
    vbe_write(1, w);
    vbe_write(2, h);
    vbe_write(3, 32);
    vbe_write(4, 0x01 | 0x40 | 0x80);

    uint64_t mid = ds_call(SVC_SYS, OP_MAPMMIO, fb_phys,
                          (uint64_t)w * h * 4, 0x45000000ULL);
    ds_poll();

    ds_msg_t mr;

    // ... (truncated, see source file for more)
```

```
drivers/vgpu/vgpu_driver.c (line 71) - [public] vbe_write
    vbe_write(1, w);
    vbe_write(2, h);
    vbe_write(3, 32);
    vbe_write(4, 0x01 | 0x40 | 0x80);

    uint64_t mid = ds_call(SVC_SYS, OP_MAPMMIO, fb_phys,
                          (uint64_t)w * h * 4, 0x45000000ULL);
    ds_poll();

    ds_msg_t mr;

    if (!ds_take(mid, &mr) || mr.status != 0) {
        // ... (truncated, see source file for more)
```

```
drivers/vgpu/vgpu_driver.c (line 72) - [public] vbe_write
    vbe_write(2, h);
    vbe_write(3, 32);
    vbe_write(4, 0x01 | 0x40 | 0x80);

    uint64_t mid = ds_call(SVC_SYS, OP_MAPMMIO, fb_phys,
                          (uint64_t)w * h * 4, 0x45000000ULL);
    ds_poll();

    ds_msg_t mr;

    if (!ds_take(mid, &mr) || mr.status != 0) {
        return false;
    }
    // ... (truncated, see source file for more)
```

```
drivers/vgpu/vgpu_driver.c (line 73) - [public] vbe_write
    vbe_write(3, 32);
    vbe_write(4, 0x01 | 0x40 | 0x80);

    uint64_t mid = ds_call(SVC_SYS, OP_MAPMMIO, fb_phys,
                          (uint64_t)w * h * 4, 0x45000000ULL);
    ds_poll();

    ds_msg_t mr;

    if (!ds_take(mid, &mr) || mr.status != 0) {
        return false;
    }
    // ... (truncated, see source file for more)
```

```

drivers/vgpu/vgpu_driver.c (line 74) - [public] vbe_write
    vbe_write(4, 0x01 | 0x40 | 0x80);

    uint64_t mid = ds_call(SVC_SYS, OP_MAPMMIO, fb_phys,
                          (uint64_t)w * h * 4, 0x45000000ULL);
    ds_poll();

    ds_msg_t mr;

    if (!ds_take(mid, &mr) || mr.status != 0) {
        return false;
    }

    // ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 78) - [public] ds_poll
    ds_poll();

    ds_msg_t mr;

    if (!ds_take(mid, &mr) || mr.status != 0) {
        return false;
    }

    g_info.width = w;
    g_info.height = h;
    g_info.bpp = 32;
    g_info.stride = w;
    // ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 82) - [public] if
    if (!ds_take(mid, &mr) || mr.status != 0) {
        return false;
    }

    g_info.width = w;
    g_info.height = h;
    g_info.bpp = 32;
    g_info.stride = w;
    g_info.fb_phys = fb_phys;
    g_info.fb = (void *)0x45000000ULL;

    g_ready = true;
    // ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 106) - [public] if
    if (!g_ready || x >= g_info.width || y >= g_info.height) {
        return;
    }

    ((uint32_t *)g_info.fb)[y * g_info.stride + x] = color;
}

void vgpu_clear(uint32_t color)
{
    if (!g_ready) {
        return;
    }
    // ... (truncated, see source file for more)

```

```

drivers/vgpu/vgpu_driver.c (line 115) - [public] if
    if (!g_ready) {
        return;
    }

    uint32_t *fb = (uint32_t *)g_info.fb;
    uint32_t n = g_info.width * g_info.height;

    for (uint32_t i = 0; i < n; i++) {
        fb[i] = color;
    }
}

// ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 133) - [public] ds_poll
    ds_poll();

    ds_msg_t ar;

    if (!ds_take(aid, &ar) || ar.status == 0 ? ar.arg0 == 0 : 1) {
        return -1;
    }

    uint64_t va = ar.arg0;

    uint64_t pid = ds_call(SVC_SYS, OP_PAGEPHYS, va, 0, 0);
    ds_poll();
    // ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 137) - [public] if
    if (!ds_take(aid, &ar) || ar.status == 0 ? ar.arg0 == 0 : 1) {
        return -1;
    }

    uint64_t va = ar.arg0;

    uint64_t pid = ds_call(SVC_SYS, OP_PAGEPHYS, va, 0, 0);
    ds_poll();

    ds_msg_t pr;

    if (!ds_take(pid, &pr) || pr.status != 0) {
        // ... (truncated, see source file for more)
    }

drivers/vgpu/vgpu_driver.c (line 144) - [public] ds_poll
    ds_poll();

    ds_msg_t pr;

    if (!ds_take(pid, &pr) || pr.status != 0) {
        return -1;
    }

    for (int32_t i = 0; i < VGPU_MAX_SURFACES; i++) {
        if (!g_surf[i].used) {
            g_surf[i].used = true;
            g_surf[i].phys = pr.arg0;
        }
    }
    // ... (truncated, see source file for more)

```

```

drivers/vgpu/vgpu_driver.c (line 148) - [public] if
    if (!ds_take(pid, &pr) || pr.status != 0) {
        return -1;
    }

    for (int32_t i = 0; i < VGPU_MAX_SURFACES; i++) {
        if (!g_surf[i].used) {
            g_surf[i].used = true;
            g_surf[i].phys = pr.arg0;
            g_surf[i].va = (void *)va;
            g_surf[i].w = w;
            g_surf[i].h = h;

            // ... (truncated, see source file for more)
        }
    }

drivers/vgpu/vgpu_driver.c (line 153) - [public] if
    if (!g_surf[i].used) {
        g_surf[i].used = true;
        g_surf[i].phys = pr.arg0;
        g_surf[i].va = (void *)va;
        g_surf[i].w = w;
        g_surf[i].h = h;

        *phys_out = pr.arg0;
        return i;
    }
}

// ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 170) - [public] if
    if (id < 0 || id >= VGPU_MAX_SURFACES || !g_surf[id].used) {
        return false;
    }

    vgpu_surface_t *s = &g_surf[id];
    uint32_t *src = (uint32_t *)s->va;

    for (uint32_t yy = 0; yy < s->h && y + yy < g_info.height; yy++) {
        for (uint32_t xx = 0; xx < s->w && x + xx < g_info.width; xx++) {
            uint32_t c = src[yy * s->w + xx];

            if ((c & 0xFF000000) != 0) {
                // ... (truncated, see source file for more)
            }
        }
    }

drivers/vgpu/vgpu_driver.c (line 181) - [public] if
    if ((c & 0xFF000000) != 0) {
        ((uint32_t *)g_info.fb)[(y + yy) * g_info.stride + (x + xx)] = c;
    }
}

return true;
}

void vgpu_process_ring(volatile vgpu_slot_t *ring)
{
    if (ring == (void *)0) {
        // ... (truncated, see source file for more)
    }
}

```



```

drivers/vgpu/vgpu_driver.c (line 192) - [public] if
    if (ring == (void *)0) {
        return;
    }

    for (uint32_t i = 0; i < VGPU_RING_SLOTS; i++) {
        volatile vgpu_slot_t *s = &ring[i];

        if (s->id == 0 || s->done != 0) {
            continue;
        }

        switch (s->op) {
// ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 199) - [public] if
    if (s->id == 0 || s->done != 0) {
        continue;
    }

    switch (s->op) {
case VGPU_INFO:
    s->r0 = ((uint64_t)g_info.width << 32) | g_info.height;
    s->r1 = g_info.fb_phys;
    s->status = 0;
    break;

    case VGPU_SURF_CREATE: {
// ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 203) - [public] switch
    switch (s->op) {
case VGPU_INFO:
    s->r0 = ((uint64_t)g_info.width << 32) | g_info.height;
    s->r1 = g_info.fb_phys;
    s->status = 0;
    break;

    case VGPU_SURF_CREATE: {
        uint64_t phys = 0;
        int32_t sid = vgpu_surface_create((uint32_t)s->a0,
                                          (uint32_t)s->a1, &phys);

// ... (truncated, see source file for more)

drivers/vgpu/vgpu_driver.c (line 215) - [public] if
    if (sid < 0) {
        s->status = -1;
    } else {
        s->r0 = (uint64_t)(uint32_t)sid;
        s->r1 = phys;
        s->status = 0;
    }
    break;
}

case VGPU_PRESENT:
    s->status = vgpu_surface_present((int32_t)s->a0,
// ... (truncated, see source file for more)

```

```

drivers/vgpu/vgpu_driver.c (line 248) - [public] ds_init
    ds_init(params_va);

    if (!inited) {
        if (vgpu_init(1024, 768)) {
            uint64_t rid = ds_call(SVC_SYS, OP_ALLOC, 1, 0, 0);
            ds_poll();

            ds_msg_t rr;

            if (ds_take(rid, &rr) && rr.status == 0) {
                g_ring = (volatile vgpu_slot_t *)rr.arg0;

                // ... (truncated, see source file for more)
            }
        }
    }

drivers/vgpu/vgpu_driver.c (line 250) - [public] if
    if (!inited) {
        if (vgpu_init(1024, 768)) {
            uint64_t rid = ds_call(SVC_SYS, OP_ALLOC, 1, 0, 0);
            ds_poll();

            ds_msg_t rr;

            if (ds_take(rid, &rr) && rr.status == 0) {
                g_ring = (volatile vgpu_slot_t *)rr.arg0;

                uint64_t pid = ds_call(SVC_SYS, OP_PAGEPHYS, rr.arg0, 0, 0);
                ds_poll();
            }
            // ... (truncated, see source file for more)
        }
    }

drivers/vgpu/vgpu_driver.c (line 251) - [public] if
    if (vgpu_init(1024, 768)) {
        uint64_t rid = ds_call(SVC_SYS, OP_ALLOC, 1, 0, 0);
        ds_poll();

        ds_msg_t rr;

        if (ds_take(rid, &rr) && rr.status == 0) {
            g_ring = (volatile vgpu_slot_t *)rr.arg0;

            uint64_t pid = ds_call(SVC_SYS, OP_PAGEPHYS, rr.arg0, 0, 0);
            ds_poll();

            // ... (truncated, see source file for more)
        }
    }

drivers/vgpu/vgpu_driver.c (line 253) - [public] ds_poll
    ds_poll();

    ds_msg_t rr;

    if (ds_take(rid, &rr) && rr.status == 0) {
        g_ring = (volatile vgpu_slot_t *)rr.arg0;

        uint64_t pid = ds_call(SVC_SYS, OP_PAGEPHYS, rr.arg0, 0, 0);
        ds_poll();

        ds_msg_t pr;

        // ... (truncated, see source file for more)
    }

```

```

drivers/vgpu/vgpu_driver.c (line 257) - [public] if
    if (ds_take(rid, &rr) && rr.status == 0) {
        g_ring = (volatile vgpu_slot_t *)rr.arg0;

        uint64_t pid = ds_call(SVC_SYS, OP_PAGEPHYS, rr.arg0, 0, 0);
        ds_poll();

        ds_msg_t pr;

        if (ds_take(pid, &pr) && pr.status == 0) {
            ds_call(SVC_VGPU, VGPU_REGISTER, pr.arg0, 0, 0);
            ds_poll();

            // ... (truncated, see source file for more)
drivers/vgpu/vgpu_driver.c (line 261) - [public] ds_poll
    ds_poll();

    ds_msg_t pr;

    if (ds_take(pid, &pr) && pr.status == 0) {
        ds_call(SVC_VGPU, VGPU_REGISTER, pr.arg0, 0, 0);
        ds_poll();

        ds_msg_t zr;
        ds_take(0, &zr);
    }
    // ... (truncated, see source file for more)
drivers/vgpu/vgpu_driver.c (line 265) - [public] if
    if (ds_take(pid, &pr) && pr.status == 0) {
        ds_call(SVC_VGPU, VGPU_REGISTER, pr.arg0, 0, 0);
        ds_poll();

        ds_msg_t zr;
        ds_take(0, &zr);
    }
}

vgpu_clear(0xFF101020);
}

// ... (truncated, see source file for more)
drivers/vgpu/vgpu_driver.c (line 266) - [public] ds_call
    ds_call(SVC_VGPU, VGPU_REGISTER, pr.arg0, 0, 0);
    ds_poll();

    ds_msg_t zr;
    ds_take(0, &zr);
}

vgpu_clear(0xFF101020);
}

    initd = true;
// ... (truncated, see source file for more)

```

```

drivers/vgpu/vgpu_driver.c (line 267) - [public] ds_poll
    ds_poll();

    ds_msg_t zr;
    ds_take(0, &zr);
}

vgpu_clear(0xFF101020);
}

initied = true;
}
// ... (truncated, see source file for more)

```

```

drivers/vgpu/vgpu_driver.c (line 270) - [public] ds_take
    ds_take(0, &zr);
}

vgpu_clear(0xFF101020);
}

initied = true;
}

vgpu_process_ring(g_ring);

// ... (truncated, see source file for more)

```

```

drivers/vgpu/vgpu_driver.c (line 274) - [public] vgpu_clear
    vgpu_clear(0xFF101020);
}

initied = true;
}

vgpu_process_ring(g_ring);

ds_yield();
}

```

```

drivers/vgpu/vgpu_driver.c (line 280) - [public] vgpu_process_ring
    vgpu_process_ring(g_ring);

    ds_yield();
}

```

```

drivers/vgpu/vgpu_driver.c (line 282) - [public] ds_yield
    ds_yield();
}

```

```

kernel_Workspace/kernel/src/battery/init.c (line 8) - [public] if
    if (is_ready) {
        return true;
    }

    is_ready = true;

    return true;
}

```

```

kernel_Workspace/kernel/src/battery/operation.c (line 23) - [public] if
    if (!model.present) {
        return false;
    }

    *out = model;
    return true;
}

static const battery_backend_t model_backend = {
    .present = model_present,
    .status = model_status,
};
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/battery/operation.c (line 40) - [public] if
    if (b != (void *)0 && b->status != (void *)0) {
        backend = b;
    }
}

bool battery_present(void)
{
    return backend->present();
}

bool battery_status(battery_status_t *out)
{
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/battery/operation.c (line 52) - [public] if
    if (out == (void *)0) {
        return false;
    }

    return backend->status(out);
}

bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2)
{
    battery_status_t s;

    if (!battery_status(&s)) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/battery/operation.c (line 63) - [public] if
    if (!battery_status(&s)) {
        return false;
    }

    *a0 = ((uint64_t)s.present << 32)
        | ((uint64_t)s.state << 24)
        | (s.percent & 0xFF);

    *a1 = ((uint64_t)s.voltage_mv << 16) | (s.temp_c & 0xFFFF);
    *a2 = s.health;

    return true;
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/battery/operation.c (line 79) - [public] if
    if (low_pct > 50) {
        return false;
    }

    low_threshold = low_pct;
    low_pending = false;

    return true;
}

uint32_t battery_threshold(void)
{
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/battery/operation.c (line 103) - [public] if
    if (!model.present || model.state == BATT_STATE_CHARGING) {
        return;
    }

    tick_counter++;

    if (tick_counter % 2000 == 0 && model.percent > 0) {
        model.percent--;

        if (model.percent <= low_threshold && !low_pending) {
            low_pending = true;
        }
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/battery/operation.c (line 109) - [public] if
    if (tick_counter % 2000 == 0 && model.percent > 0) {
        model.percent--;

        if (model.percent <= low_threshold && !low_pending) {
            low_pending = true;
        }
    }
}

```

```

kernel_Workspace/kernel/src/battery/operation.c (line 112) - [public] if
    if (model.percent <= low_threshold && !low_pending) {
        low_pending = true;
    }
}

```

```

kernel_Workspace/kernel/src/bluetooth/init.c (line 14) - [public] if
    if (ret_len != 0 && ret == NULL) {
        return false;
    }

    for (uint8_t i = 0; i < 32; i++) {
        if (!bt_event_poll(buf, EVT_MAX, &len)) {
            continue;
        }

        if (len < 7 || buf[0] != HCI_EVT_PKT || buf[1] != EVT_CMD_COMPLETE) {
            continue;
        }
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/bluetooth/init.c (line 19) - [public] if
    if (!bt_event_poll(buf, EVT_MAX, &len)) {
        continue;
    }

    if (len < 7 || buf[0] != HCI_EVT_PKT || buf[1] != EVT_CMD_COMPLETE) {
        continue;
    }

    if (buf[4] != (uint8_t)(opcode & 0xFF) || buf[5] != (uint8_t)(opcode >> 8)) {
        continue;
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/init.c (line 23) - [public] if
    if (len < 7 || buf[0] != HCI_EVT_PKT || buf[1] != EVT_CMD_COMPLETE) {
        continue;
    }

    if (buf[4] != (uint8_t)(opcode & 0xFF) || buf[5] != (uint8_t)(opcode >> 8)) {
        continue;
    }

    if (buf[6] != 0 || len < (uint8_t)(7 + ret_len)) {
        return false;
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/init.c (line 27) - [public] if
    if (buf[4] != (uint8_t)(opcode & 0xFF) || buf[5] != (uint8_t)(opcode >> 8)) {
        continue;
    }

    if (buf[6] != 0 || len < (uint8_t)(7 + ret_len)) {
        return false;
    }

    for (uint8_t k = 0; k < ret_len; k++) {
        ret[k] = buf[7 + k];
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/init.c (line 31) - [public] if
    if (buf[6] != 0 || len < (uint8_t)(7 + ret_len)) {
        return false;
    }

    for (uint8_t k = 0; k < ret_len; k++) {
        ret[k] = buf[7 + k];
    }

    return true;
}

return false;
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/bluetooth/init.c (line 47) - [public] if

```
    if (bt_is_ready) {
        return true;
    }

    bt_op_model_init();

    if (!bt_hci_cmd(HCI_RESET, NULL, 0)) {
        return false;
    }

    uint8_t tmp[8];
    if (!wait_cmd_complete(HCI_RESET, tmp, 0)) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/bluetooth/init.c (line 51) - [public] bt_op_model_init

```
bt_op_model_init();

if (!bt_hci_cmd(HCI_RESET, NULL, 0)) {
    return false;
}

uint8_t tmp[8];
if (!wait_cmd_complete(HCI_RESET, tmp, 0)) {
    return false;
}

if (!bt_hci_cmd(HCI_READ_LOCAL_VERSION, NULL, 0)) {
    // ... (truncated, see source file for more)
}
}
```

kernel_Workspace/kernel/src/bluetooth/init.c (line 53) - [public] if

```
if (!bt_hci_cmd(HCI_RESET, NULL, 0)) {
    return false;
}

uint8_t tmp[8];
if (!wait_cmd_complete(HCI_RESET, tmp, 0)) {
    return false;
}

if (!bt_hci_cmd(HCI_READ_LOCAL_VERSION, NULL, 0)) {
    return false;
}
// ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/bluetooth/init.c (line 58) - [public] if

```
if (!wait_cmd_complete(HCI_RESET, tmp, 0)) {
    return false;
}

if (!bt_hci_cmd(HCI_READ_LOCAL_VERSION, NULL, 0)) {
    return false;
}

if (!wait_cmd_complete(HCI_READ_LOCAL_VERSION, tmp, 1)) {
    return false;
}

// ... (truncated, see source file for more)
}
```



```
kernel_Workspace/kernel/src/bluetooth/init.c (line 62) - [public] if
    if (!bt_hci_cmd(HCI_READ_LOCAL_VERSION, NULL, 0)) {
        return false;
    }

    if (!wait_cmd_complete(HCI_READ_LOCAL_VERSION, tmp, 1)) {
        return false;
    }

    saved_ver = tmp[0];

    if (!bt_hci_cmd(HCI_READ_BD_ADDR, NULL, 0)) {
        return false;
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/bluetooth/init.c (line 66) - [public] if
    if (!wait_cmd_complete(HCI_READ_LOCAL_VERSION, tmp, 1)) {
        return false;
    }

    saved_ver = tmp[0];

    if (!bt_hci_cmd(HCI_READ_BD_ADDR, NULL, 0)) {
        return false;
    }

    if (!wait_cmd_complete(HCI_READ_BD_ADDR, saved_addr, 6)) {
        return false;
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/bluetooth/init.c (line 72) - [public] if
    if (!bt_hci_cmd(HCI_READ_BD_ADDR, NULL, 0)) {
        return false;
    }

    if (!wait_cmd_complete(HCI_READ_BD_ADDR, saved_addr, 6)) {
        return false;
    }

    bt_is_ready = true;
    return true;
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/bluetooth/init.c (line 76) - [public] if
    if (!wait_cmd_complete(HCI_READ_BD_ADDR, saved_addr, 6)) {
        return false;
    }

    bt_is_ready = true;
    return true;
}

bool bt_ready(void)
{
    return bt_is_ready;
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/bluetooth/init.c (line 91) - [public] if
    if (hci_ver != NULL) {
        *hci_ver = saved_ver;
    }

    if (bdaddr != NULL) {
        for (uint8_t i = 0; i < 6; i++) {
            bdaddr[i] = saved_addr[i];
        }
    }
}
```

kernel_Workspace/kernel/src/bluetooth/init.c (line 95) - [public] if

```
    if (bdaddr != NULL) {
        for (uint8_t i = 0; i < 6; i++) {
            bdaddr[i] = saved_addr[i];
        }
    }
}
```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 18) - [public] if

```
if (data == NULL || len == 0 || len > EVT_MAX) {
    return false;
}

uint32_t next = (evt_head + 1) % EVT_Q;
if (next == evt_tail) {
    return false;
}

for (uint8_t i = 0; i < len; i++) {
    evt_q[evt_head][i] = data[i];
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 23) - [public] if

```
if (next == evt_tail) {
    return false;
}

for (uint8_t i = 0; i < len; i++) {
    evt_q[evt_head][i] = data[i];
}

evt_len[evt_head] = len;
evt_head = next;
return true;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 49) - [public] if

```
if (len != 0 && params == NULL) {
    return false;
}

switch (opcode) {
case HCI_RESET:
    ev[0] = HCI_EVT_PKT;
    ev[1] = EVT_CMD_COMPLETE;
    ev[2] = 4;
    ev[3] = 1;
    ev[4] = opcode & 0xFF;
    ev[5] = opcode >> 8;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 53) - [public] switch

```
switch (opcode) {
case HCI_RESET:
    ev[0] = HCI_EVT_PKT;
    ev[1] = EVT_CMD_COMPLETE;
    ev[2] = 4;
    ev[3] = 1;
    ev[4] = opcode & 0xFF;
    ev[5] = opcode >> 8;
    ev[6] = 0;
    return evt_push(ev, 7);

case HCI_READ_LOCAL_VERSION:
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 62) - [public] evt_push
    return evt_push(ev, 7);

    case HCI_READ_LOCAL_VERSION:
        ev[0] = HCI_EVT_PKT;
        ev[1] = EVT_CMD_COMPLETE;
        ev[2] = 12;
        ev[3] = 1;
        ev[4] = opcode & 0xFF;
        ev[5] = opcode >> 8;
        ev[6] = 0;
        ev[7] = hci_version;
        ev[8] = 0;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/operation.c (line 80) - [public] evt_push
    return evt_push(ev, 15);

    case HCI_READ_BD_ADDR:
        ev[0] = HCI_EVT_PKT;
        ev[1] = EVT_CMD_COMPLETE;
        ev[2] = 10;
        ev[3] = 1;
        ev[4] = opcode & 0xFF;
        ev[5] = opcode >> 8;
        ev[6] = 0;
        for (uint8_t i = 0; i < 6; i++) {
            ev[7 + i] = bdaddr[i];
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/operation.c (line 93) - [public] evt_push
    return evt_push(ev, 13);

    default:
        ev[0] = HCI_EVT_PKT;
        ev[1] = EVT_CMD_COMPLETE;
        ev[2] = 4;
        ev[3] = 1;
        ev[4] = opcode & 0xFF;
        ev[5] = opcode >> 8;
        ev[6] = 0;
        return evt_push(ev, 7);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/bluetooth/operation.c (line 103) - [public] evt_push
    return evt_push(ev, 7);
}

bool bt_hci_cmd(uint16_t opcode, const uint8_t *params, uint8_t len)
{
    if (!op_ready) {
        return false;
    }

    return bt_op_model_run_cmd(opcode, params, len);
}
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 109) - [public] if

```
    if (!op_ready) {
        return false;
    }

    return bt_op_model_run_cmd(opcode, params, len);
}

bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len)
{
    if (buf == NULL || len == NULL || evt_head == evt_tail) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 113) - [public] bt_op_model_run_cmd

```
    return bt_op_model_run_cmd(opcode, params, len);
}

bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len)
{
    if (buf == NULL || len == NULL || evt_head == evt_tail) {
        return false;
    }

    uint8_t n = evt_len[evt_tail];
    if (n > cap) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 118) - [public] if

```
    if (buf == NULL || len == NULL || evt_head == evt_tail) {
        return false;
    }

    uint8_t n = evt_len[evt_tail];
    if (n > cap) {
        return false;
    }

    for (uint8_t i = 0; i < n; i++) {
        buf[i] = evt_q[evt_tail][i];
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 123) - [public] if

```
    if (n > cap) {
        return false;
    }

    for (uint8_t i = 0; i < n; i++) {
        buf[i] = evt_q[evt_tail][i];
    }

    *len = n;
    evt_tail = (evt_tail + 1) % EVT_Q;
    return true;
}
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 138) - [public] if
    if (!op_ready || (len != 0 && data == NULL) || len > ACL_MAX) {
        return false;
    }

    uint32_t next = (acl_head + 1) % ACL_Q;
    if (next == acl_tail) {
        return false;
    }

    for (uint16_t i = 0; i < len; i++) {
        acl_q[acl_head][i] = data[i];
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 143) - [public] if
    if (next == acl_tail) {
        return false;
    }

    for (uint16_t i = 0; i < len; i++) {
        acl_q[acl_head][i] = data[i];
    }

    acl_len[acl_head] = len;
    acl_head = next;
    return true;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 158) - [public] if
    if (data == NULL || len == NULL || acl_head == acl_tail) {
        return false;
    }

    uint16_t n = acl_len[acl_tail];
    if (n > cap) {
        return false;
    }

    for (uint16_t i = 0; i < n; i++) {
        data[i] = acl_q[acl_tail][i];
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/bluetooth/operation.c (line 163) - [public] if
    if (n > cap) {
        return false;
    }

    for (uint16_t i = 0; i < n; i++) {
        data[i] = acl_q[acl_tail][i];
    }

    *len = n;
    acl_tail = (acl_tail + 1) % ACL_Q;
    return true;
}

```

```

kernel_Workspace/kernel/src/camera/init.c (line 7) - [public] if
    if (is_ready) {
        return true;
    }

    is_ready = true;

    return true;
}

```

```
kernel_Workspace/kernel/src/camera/operation.c (line 43) - [public] if
    if (((x + y + f * 8) & 0xFF) < 16) {
        r = g = b = 255;
    }

    uint32_t i = (y * CAM_W + x) * CAM_BPP;

    framebuf[i + 0] = r;
    framebuf[i + 1] = g;
    framebuf[i + 2] = b;
}
}
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/camera/operation.c (line 58) - [public] if
    if (!streaming) {
        return false;
    }

    gen_frame();

    uint32_t n = sizeof(framebuf);

    if (cap < n) {
        n = cap;
    }

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/camera/operation.c (line 62) - [public] gen_frame
    gen_frame();

    uint32_t n = sizeof(framebuf);

    if (cap < n) {
        n = cap;
    }

    uint8_t *dst = (uint8_t *)buf;

    for (uint32_t i = 0; i < n; i++) {
        dst[i] = framebuf[i];
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/camera/operation.c (line 66) - [public] if
    if (cap < n) {
        n = cap;
    }

    uint8_t *dst = (uint8_t *)buf;

    for (uint32_t i = 0; i < n; i++) {
        dst[i] = framebuf[i];
    }

    frame_id++;

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/camera/operation.c (line 78) - [public] if
    if (fid) {
        *fid = frame_id;
    }

    return true;
}

static const camera_backend_t model_backend = {
    .present = model_present,
    .start = model_start,
    .stop = model_stop,
    .frame = model_frame,
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/camera/operation.c (line 96) - [public] if
    if (b != (void *)0 && b->frame != (void *)0) {
        backend = b;
    }
}

bool camera_present(void)
{
    return backend->present();
}

bool camera_caps_get(camera_caps_t *out)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/camera/operation.c (line 108) - [public] if
    if (out == (void *)0 || !camera_present()) {
        return false;
    }

    out->present = 1;
    out->width = CAM_W;
    out->height = CAM_H;
    out->format = CAM_FMT_RGB888;
    out->fps = 30;

    return true;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/camera/operation.c (line 123) - [public] if
    if (!camera_present()) {
        return false;
    }

    return backend->start();
}

bool camera_stop(void)
{
    return backend->stop();
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/camera/operation.c (line 137) - [public] if
    if (buf == (void *)0) {
        return false;
    }

    return backend->frame(buf, cap, fid);
}

bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *fid)
{
    return backend->frame(arch_phys_to_virt(phys), cap, fid);
}

kernel_Workspace/kernel/src/core-lang/ast.c (line 14) - [public] if
    if (a->pos + aligned > a->cap) {
        return (void *)0;
    }

    void *p = a->buf + a->pos;

    a->pos += aligned;

    return p;
}

ast_node_t *cl_node(arena_t *ar, uint8_t kind, uint32_t line)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/ast.c (line 29) - [public] if
    if (n == (void *)0) {
        return (void *)0;
    }

    n->kind = kind;
    n->type.bits = 0;
    n->type.neg = false;
    n->line = line;
    n->op = 0;
    n->pool_kind = 0;
    n->is_array = false;
    n->value = 0;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/bridge.c (line 3) - [public] kprintf
extern void kprintf(const char *fmt, ...);
extern int32_t k_input_key(void);
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
extern uint32_t k_getpid(void);
extern uint64_t k_world_cr3(void);
extern uint64_t k_kernel_cr3(void);
extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t cr3);
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
    // ... (truncated, see source file for more)

```


kernel_Workspace/kernel/src/core-lang/bridge.c (line 4) - [public] k_input_key

```
extern int32_t k_input_key(void);
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
extern uint32_t k_getpid(void);
extern uint64_t k_world_cr3(void);
extern uint64_t k_kernel_cr3(void);
extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t cr3);
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 5) - [public] k_fs_read

```
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
extern uint32_t k_getpid(void);
extern uint64_t k_world_cr3(void);
extern uint64_t k_kernel_cr3(void);
extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t cr3);
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 6) - [public] k_getpid

```
extern uint32_t k_getpid(void);
extern uint64_t k_world_cr3(void);
extern uint64_t k_kernel_cr3(void);
extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t cr3);
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 7) - [public] k_world_cr3

```
extern uint64_t k_world_cr3(void);
extern uint64_t k_kernel_cr3(void);
extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t cr3);
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

static cl_vm_t *g_vm = (void *)0;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 8) - [public] k_kernel_cr3

```
extern uint64_t k_kernel_cr3(void);
extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t cr3);
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

static cl_vm_t *g_vm = (void *)0;
static uint8_t g_ring = 0;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 9) - [public] k_spawn

```
extern int64_t k_spawn(const char *path, uint32_t parent, uint64_t cr3);
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

static cl_vm_t *g_vm = (void *)0;
static uint8_t g_ring = 0;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 10) - [public] k_ipc_send

```
extern int32_t k_ipc_send(uint32_t dst, uint64_t a0, uint64_t a1);
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

static cl_vm_t *g_vm = (void *)0;
static uint8_t g_ring = 0;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 11) - [public] k_ipc_recv

```
extern int32_t k_ipc_recv(uint64_t *a0, uint64_t *a1);
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

static cl_vm_t *g_vm = (void *)0;
static uint8_t g_ring = 0;

static uint64_t br_put(uint64_t v, uint64_t b, uint64_t c,
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 12) - [public] k_tick

```
extern uint64_t k_tick(void);

extern uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                                uint32_t w, uint32_t h);
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

static cl_vm_t *g_vm = (void *)0;
static uint8_t g_ring = 0;

static uint64_t br_put(uint64_t v, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 16) - [public] battery_status_packed

```
extern bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);

static cl_vm_t *g_vm = (void *)0;
static uint8_t g_ring = 0;

static uint64_t br_put(uint64_t v, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
{
    (void)b; (void)c; (void)d; (void)e; (void)f;
    kprintf("%llu\n", (unsigned long long)v);
    return 0;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 26) - [public] kprintf

```
kprintf("%llu\n", (unsigned long long)v);
return 0;
}

static uint64_t br_putc(uint64_t v, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
{
    (void)b; (void)c; (void)d; (void)e; (void)f;
    kprintf("%c", (char)v);
    return 0;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 34) - [public] kprintf

```
kprintf("%c", (char)v);
return 0;
}

static uint64_t br_log(uint64_t ptr, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
{
    (void)b; (void)c; (void)d; (void)e; (void)f;

    char buf[128];

    if (g_ring == 0) {
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 45) - [public] if
    if (g_ring == 0) {
        const char *s = (const char *)ptr;
        int i = 0;

        while (s[i] && i < 127) {
            buf[i] = s[i];
            i++;
        }

        buf[i] = '\0';
    } else {
        extern bool k_user_cstr(uint64_t ptr, char *buf, uint32_t cap);
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/bridge.c (line 49) - [public] while
        while (s[i] && i < 127) {
            buf[i] = s[i];
            i++;
        }

        buf[i] = '\0';
    } else {
        extern bool k_user_cstr(uint64_t ptr, char *buf, uint32_t cap);

        if (!k_user_cstr(ptr, buf, sizeof(buf))) {
            return (uint64_t)-1;
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/bridge.c (line 56) - [public] k_user_cstr
        extern bool k_user_cstr(uint64_t ptr, char *buf, uint32_t cap);

        if (!k_user_cstr(ptr, buf, sizeof(buf))) {
            return (uint64_t)-1;
        }
    }

    kprintf("%s\n", buf);
    return 0;
}

static uint64_t br_key(uint64_t a, uint64_t b, uint64_t c,
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/bridge.c (line 58) - [public] if
        if (!k_user_cstr(ptr, buf, sizeof(buf))) {
            return (uint64_t)-1;
        }
    }

    kprintf("%s\n", buf);
    return 0;
}

static uint64_t br_key(uint64_t a, uint64_t b, uint64_t c,
    uint64_t d, uint64_t e, uint64_t f)
{
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 63) - [public] kprintf
    kprintf("%s\n", buf);
    return 0;
}

static uint64_t br_key(uint64_t a, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
{
    (void)a; (void)b; (void)c; (void)d; (void)e; (void)f;

    int32_t k = k_input_key();

    return k < 0 ? 0 : (uint64_t)k;
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/bridge.c (line 81) - [public] k_getpid
    return k_getpid();
}

static uint64_t br_tick(uint64_t a, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
{
    (void)a; (void)b; (void)c; (void)d; (void)e; (void)f;
    return k_tick();
}

static uint64_t br_exit(uint64_t code, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/bridge.c (line 88) - [public] k_tick
    return k_tick();
}

static uint64_t br_exit(uint64_t code, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
{
    (void)b; (void)c; (void)d; (void)e; (void)f;

    if (g_vm != (void *)0) {
        g_vm->halt_req = true;
    }

    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/bridge.c (line 96) - [public] if
    if (g_vm != (void *)0) {
        g_vm->halt_req = true;
    }

    return code;
}

static uint64_t br_spawn(uint64_t ptr, uint64_t b, uint64_t c,
                      uint64_t d, uint64_t e, uint64_t f)
{
    (void)b; (void)c; (void)d; (void)e; (void)f;

    // ... (truncated, see source file for more)
}

```

```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 108) - [public] if
    if (g_ring >= 3) {
        return (uint64_t)-1;
    }

    uint64_t cr3 = (g_ring == 0) ? k_kernel_cr3() : k_world_cr3();

    return (uint64_t)k_spawn((const char *)ptr, k_getpid(), cr3);
}

static uint64_t br_ipc_send(uint64_t dst, uint64_t a0, uint64_t a1,
                           uint64_t d, uint64_t e, uint64_t f)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/bridge.c (line 114) - [public] return
    return (uint64_t)k_spawn((const char *)ptr, k_getpid(), cr3);
}

static uint64_t br_ipc_send(uint64_t dst, uint64_t a0, uint64_t a1,
                           uint64_t d, uint64_t e, uint64_t f)
{
    (void)d; (void)e; (void)f;
    return (uint64_t)(int64_t)k_ipc_send((uint32_t)dst, a0, a1);
}

static uint64_t br_ipc_recv(uint64_t a, uint64_t b, uint64_t c,
                           uint64_t d, uint64_t e, uint64_t f)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/bridge.c (line 121) - [public] return
    return (uint64_t)(int64_t)k_ipc_send((uint32_t)dst, a0, a1);
}

static uint64_t br_ipc_recv(uint64_t a, uint64_t b, uint64_t c,
                           uint64_t d, uint64_t e, uint64_t f)
{
    (void)a; (void)b; (void)c; (void)d; (void)e; (void)f;

    uint64_t a0 = 0, a1 = 0;
    int32_t from = k_ipc_recv(&a0, &a1);

    return (uint64_t)(int64_t)from;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/bridge.c (line 140) - [public] if
    if (g_ring >= 3) {
        return (uint64_t)-1;
    }

    return (uint64_t)(int64_t)k_fs_read((const char *)path,
                                       (void *)buf, (uint32_t)cap);
}

static uint64_t br_vfill(uint64_t color, uint64_t xy, uint64_t wh,
                        uint64_t d, uint64_t e, uint64_t f)
{
    (void)d; (void)e; (void)f;
    // ... (truncated, see source file for more)
}

```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 153) - [public] if

```
    if (g_ring >= 3) {
        return (uint64_t)-1;
    }

    return hdmi_submit_fill((uint32_t)color,
                           (uint32_t)(xy & 0xFFFF),
                           (uint32_t)(xy >> 16),
                           (uint32_t)(wh & 0xFFFF),
                           (uint32_t)(wh >> 16));
}

static uint64_t br_bat(uint64_t a, uint64_t b, uint64_t c,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 171) - [public] if

```
    if (!battery_status_packed(&a0, &a1, &a2)) {
        return (uint64_t)-1;
    }

    return a0;
}
```

```
int cl_bridge_add(cl_vm_t *vm, const char *name, cl_ext_fn fn)
{
    return cl_vm_register_extern(vm, name, fn);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 181) - [public] cl_vm_register_extern

```
    return cl_vm_register_extern(vm, name, fn);
}
```

```
int cl_bridge_init(cl_vm_t *vm, uint8_t ring)
{
    g_vm = vm;
    g_ring = ring;

    cl_bridge_add(vm, "put", br_put);
    cl_bridge_add(vm, "putc", br_putc);
    cl_bridge_add(vm, "log", br_log);
    cl_bridge_add(vm, "key", br_key);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 189) - [public] cl_bridge_add

```
    cl_bridge_add(vm, "put", br_put);
    cl_bridge_add(vm, "putc", br_putc);
    cl_bridge_add(vm, "log", br_log);
    cl_bridge_add(vm, "key", br_key);
    cl_bridge_add(vm, "tick", br_tick);
    cl_bridge_add(vm, "bat", br_bat);
    cl_bridge_add(vm, "exit", br_exit);

    if (ring <= 1) {
        cl_bridge_add(vm, "getpid", br_getpid);
        cl_bridge_add(vm, "spawn", br_spawn);
        cl_bridge_add(vm, "ipc_send", br_ipc_send);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 190) - [public] cl_bridge_add

```
cl_bridge_add(vm, "putc", br_putc);
cl_bridge_add(vm, "log", br_log);
cl_bridge_add(vm, "key", br_key);
cl_bridge_add(vm, "tick", br_tick);
cl_bridge_add(vm, "bat", br_bat);
cl_bridge_add(vm, "exit", br_exit);

if (ring <= 1) {
    cl_bridge_add(vm, "getpid", br_getpid);
    cl_bridge_add(vm, "spawn", br_spawn);
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 191) - [public] cl_bridge_add

```
cl_bridge_add(vm, "log", br_log);
cl_bridge_add(vm, "key", br_key);
cl_bridge_add(vm, "tick", br_tick);
cl_bridge_add(vm, "bat", br_bat);
cl_bridge_add(vm, "exit", br_exit);

if (ring <= 1) {
    cl_bridge_add(vm, "getpid", br_getpid);
    cl_bridge_add(vm, "spawn", br_spawn);
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 192) - [public] cl_bridge_add

```
cl_bridge_add(vm, "key", br_key);
cl_bridge_add(vm, "tick", br_tick);
cl_bridge_add(vm, "bat", br_bat);
cl_bridge_add(vm, "exit", br_exit);

if (ring <= 1) {
    cl_bridge_add(vm, "getpid", br_getpid);
    cl_bridge_add(vm, "spawn", br_spawn);
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 193) - [public] cl_bridge_add

```
cl_bridge_add(vm, "tick", br_tick);
cl_bridge_add(vm, "bat", br_bat);
cl_bridge_add(vm, "exit", br_exit);

if (ring <= 1) {
    cl_bridge_add(vm, "getpid", br_getpid);
    cl_bridge_add(vm, "spawn", br_spawn);
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
}
// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/core-lang/bridge.c (line 194) - [public] cl_bridge_add

```
cl_bridge_add(vm, "bat", br_bat);
cl_bridge_add(vm, "exit", br_exit);

if (ring <= 1) {
    cl_bridge_add(vm, "getpid", br_getpid);
    cl_bridge_add(vm, "spawn", br_spawn);
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 195) - [public] cl_bridge_add

```
cl_bridge_add(vm, "exit", br_exit);

if (ring <= 1) {
    cl_bridge_add(vm, "getpid", br_getpid);
    cl_bridge_add(vm, "spawn", br_spawn);
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
}

return 0;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 197) - [public] if

```
if (ring <= 1) {
    cl_bridge_add(vm, "getpid", br_getpid);
    cl_bridge_add(vm, "spawn", br_spawn);
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
}

return 0;
}
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 198) - [public] cl_bridge_add

```
cl_bridge_add(vm, "getpid", br_getpid);
cl_bridge_add(vm, "spawn", br_spawn);
cl_bridge_add(vm, "ipc_send", br_ipc_send);
cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
cl_bridge_add(vm, "fsread", br_fsread);
cl_bridge_add(vm, "vfill", br_vfill);
}

return 0;
}
```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 199) - [public] cl_bridge_add

```
cl_bridge_add(vm, "spawn", br_spawn);
cl_bridge_add(vm, "ipc_send", br_ipc_send);
cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
cl_bridge_add(vm, "fsread", br_fsread);
cl_bridge_add(vm, "vfill", br_vfill);
}

return 0;
}
```

```

kernel_Workspace/kernel/src/core-lang/bridge.c (line 200) - [public] cl_bridge_add
    cl_bridge_add(vm, "ipc_send", br_ipc_send);
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
}

    return 0;
}

kernel_Workspace/kernel/src/core-lang/bridge.c (line 201) - [public] cl_bridge_add
    cl_bridge_add(vm, "ipc_recv", br_ipc_recv);
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
}

    return 0;
}

kernel_Workspace/kernel/src/core-lang/bridge.c (line 202) - [public] cl_bridge_add
    cl_bridge_add(vm, "fsread", br_fsread);
    cl_bridge_add(vm, "vfill", br_vfill);
}

    return 0;
}

kernel_Workspace/kernel/src/core-lang/bridge.c (line 203) - [public] cl_bridge_add
    cl_bridge_add(vm, "vfill", br_vfill);
}

    return 0;
}

kernel_Workspace/kernel/src/core-lang/codegen.c (line 22) - [public] if
    if (P->code_len >= P->code_cap) {
        return 0;
    }

    cl_insn_t in;

    in.op = op;
    in.bits = bits;
    in.a = a;
    in.b = b;
    in.c = c;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 45) - [public] if
    if (P->gnames[i][k] != name[k]) {
        same = false;
        break;
    }
}

    if (same) return (int)i;
}

    return -1;
}

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 63) - [public] if
    if (g->locals[i].name[k] != name[k]) {
        same = false;
        break;
    }
}

    if (same) return i;
}

return -1;
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 79) - [public] if
    if (i >= 0) {
        return g->locals[i].slot;
    }

    if (g->nlocals >= 32) {
        return 0;
    }

    for (int k = 0; name[k] && k < 63; k++) {
        g->locals[g->nlocals].name[k] = name[k];
        g->locals[g->nlocals].name[k + 1] = '\0';
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 83) - [public] if
    if (g->nlocals >= 32) {
        return 0;
    }

    for (int k = 0; name[k] && k < 63; k++) {
        g->locals[g->nlocals].name[k] = name[k];
        g->locals[g->nlocals].name[k + 1] = '\0';
    }

    g->locals[g->nlocals].bits = bits;
    g->locals[g->nlocals].slot = g->maxslots;

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 112) - [public] if
    if (neg) {
        uint8_t z[64] = { 0 };
        uint8_t sub[64];
        cl_sub(sub, z, P->consts[P->nconsts].b, w);
        for (int i = 0; i < w; i++) P->consts[P->nconsts].b[i] = sub[i];
    }

    cl_mask(P->consts[P->nconsts].b, w, bits);

    P->const_bits[P->nconsts] = bits;

    return (uint32_t)P->nconsts++;
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 115) - [public] cl_sub
    cl_sub(sub, z, P->consts[P->nconsts].b, w);
    for (int i = 0; i < w; i++) P->consts[P->nconsts].b[i] = sub[i];
}

    cl_mask(P->consts[P->nconsts].b, w, bits);

    P->const_bits[P->nconsts] = bits;

    return (uint32_t)P->nconsts++;
}

static void gen_expr(cg_t *g, ast_node_t *n, uint8_t bits);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 119) - [public] cl_mask
    cl_mask(P->consts[P->nconsts].b, w, bits);

    P->const_bits[P->nconsts] = bits;

    return (uint32_t)P->nconsts++;
}

static void gen_expr(cg_t *g, ast_node_t *n, uint8_t bits);

static void gen_expr(cg_t *g, ast_node_t *n, uint8_t bits)
{
    cl_prog_t *P = g->P;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 126) - [public] gen_expr
static void gen_expr(cg_t *g, ast_node_t *n, uint8_t bits);

static void gen_expr(cg_t *g, ast_node_t *n, uint8_t bits)
{
    cl_prog_t *P = g->P;

    if (n == (void *)0) return;

    switch (n->kind) {
    case ND_NUM:
        emit(g, OP_CONST, bits, (uint16_t)add_const(g, n->value, bits, false), 0, 0);
        break;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 134) - [public] switch
    switch (n->kind) {
    case ND_NUM:
        emit(g, OP_CONST, bits, (uint16_t)add_const(g, n->value, bits, false), 0, 0);
        break;

    case ND_VAR: {
        int li = find_local(g, n->name);

        if (li >= 0) {
            emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
        } else {
            int gi = find_global(P, n->name);
            // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 136) - [public] emit
    emit(g, OP_CONST, bits, (uint16_t)add_const(g, n->value, bits, false), 0, 0);
    break;

case ND_VAR: {
    int li = find_local(g, n->name);

    if (li >= 0) {
        emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
    } else {
        int gi = find_global(P, n->name);

        if (gi >= 0) {
            // ... (truncated, see source file for more)
        }
    }
}

kernel_Workspace/kernel/src/core-lang/codegen.c (line 142) - [public] if
    if (li >= 0) {
        emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
    } else {
        int gi = find_global(P, n->name);

        if (gi >= 0) {
            emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
        } else {
            emit(g, OP_CONST, bits, (uint16_t)add_const(g, 0, bits, false), 0, 0);
        }
    }
    break;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 143) - [public] emit
    emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
} else {
    int gi = find_global(P, n->name);

    if (gi >= 0) {
        emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
    } else {
        emit(g, OP_CONST, bits, (uint16_t)add_const(g, 0, bits, false), 0, 0);
    }
}
break;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 147) - [public] if
    if (gi >= 0) {
        emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
    } else {
        emit(g, OP_CONST, bits, (uint16_t)add_const(g, 0, bits, false), 0, 0);
    }
}
break;
}

case ND_POOL:
    if (n->pool_kind == 0) {
        emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 148) - [public] emit
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
} else {
    emit(g, OP_CONST, bits, (uint16_t)add_const(g, 0, bits, false), 0, 0);
}
}
break;
}

case ND_POOL:
    if (n->pool_kind == 0) {
        emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    } else if (n->pool_kind == 1) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 150) - [public] emit
    emit(g, OP_CONST, bits, (uint16_t)add_const(g, 0, bits, false), 0, 0);
}
}
break;
}

case ND_POOL:
    if (n->pool_kind == 0) {
        emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    } else if (n->pool_kind == 1) {
        int gi = (uint16_t)find_global(P, n->name);
        emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 157) - [public] if
    if (n->pool_kind == 0) {
        emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    } else if (n->pool_kind == 1) {
        int gi = (uint16_t)find_global(P, n->name);
        emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
        emit(g, OP_NEG, bits, 0, 0, 0);
        emit(g, OP_GSTORE, bits, (uint16_t)gi, 0, 0);
        emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
    } else {
        int gi = (uint16_t)find_global(P, n->name);
        emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
        emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 158) - [public] emit
    emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
} else if (n->pool_kind == 1) {
    int gi = (uint16_t)find_global(P, n->name);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_NEG, bits, 0, 0, 0);
    emit(g, OP_GSTORE, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
} else {
    int gi = (uint16_t)find_global(P, n->name);
    emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 161) - [public] emit
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_NEG, bits, 0, 0, 0);
    emit(g, OP_GSTORE, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
} else {
    int gi = (uint16_t)find_global(P, n->name);
    emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
}
break;

case ND_CALLIDX: {
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 162) - [public] emit
    emit(g, OP_NEG, bits, 0, 0, 0);
    emit(g, OP_GSTORE, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
} else {
    int gi = (uint16_t)find_global(P, n->name);
    emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
}
break;

case ND_CALLIDX: {
    int li = find_local(g, n->name);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 163) - [public] emit
    emit(g, OP_GSTORE, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
} else {
    int gi = (uint16_t)find_global(P, n->name);
    emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
}
break;

case ND_CALLIDX: {
    int li = find_local(g, n->name);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 164) - [public] emit
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
} else {
    int gi = (uint16_t)find_global(P, n->name);
    emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
}
break;

case ND_CALLIDX: {
    int li = find_local(g, n->name);

    if (li >= 0) {
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 167) - [public] emit
    emit(g, OP_SHARED, bits, (uint16_t)gi, 0, 0);
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
}
break;

case ND_CALLIDX: {
    int li = find_local(g, n->name);

    if (li >= 0) {
        emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
    } else {
        emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/codegen.c (line 168) - [public] emit
    emit(g, OP_GLOAD, bits, (uint16_t)gi, 0, 0);
}
break;

case ND_CALLIDX: {
    int li = find_local(g, n->name);

    if (li >= 0) {
        emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
    } else {
        emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/codegen.c (line 175) - [public] if
    if (li >= 0) {
        emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
    } else {
        emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    }
    break;
}

case ND_UN:
    gen_expr(g, n->a, bits);

    if (n->op == '-') {
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/codegen.c (line 176) - [public] emit
    emit(g, OP_LOAD, bits, g->locals[li].slot, 0, 0);
} else {
    emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
}
break;
}

case ND_UN:
    gen_expr(g, n->a, bits);

    if (n->op == '-') {
        emit(g, OP_NEG, bits, 0, 0, 0);
    }
    // ... (truncated, see source file for more)
}

```



```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 178) - [public] emit
    emit(g, OP_GLOAD, bits, (uint16_t)find_global(P, n->name), 0, 0);
    }
    break;
}

case ND_UN:
    gen_expr(g, n->a, bits);

    if (n->op == '-') {
        emit(g, OP_NEG, bits, 0, 0, 0);
    } else {
        emit(g, OP_NOT, bits, 0, 0, 0);
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 184) - [public] gen_expr
    gen_expr(g, n->a, bits);

    if (n->op == '-') {
        emit(g, OP_NEG, bits, 0, 0, 0);
    } else {
        emit(g, OP_NOT, bits, 0, 0, 0);
    }
    break;

case ND_BIN: {
    uint8_t op = 0;

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 186) - [public] if
    if (n->op == '-') {
        emit(g, OP_NEG, bits, 0, 0, 0);
    } else {
        emit(g, OP_NOT, bits, 0, 0, 0);
    }
    break;

case ND_BIN: {
    uint8_t op = 0;

    switch (n->op) {
        case '+': op = OP_ADD; break;
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 187) - [public] emit
    emit(g, OP_NEG, bits, 0, 0, 0);
    } else {
        emit(g, OP_NOT, bits, 0, 0, 0);
    }
    break;

case ND_BIN: {
    uint8_t op = 0;

    switch (n->op) {
        case '+': op = OP_ADD; break;
        case '-': op = OP_SUB; break;
    }
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 189) - [public] emit

```
    emit(g, OP_NOT, bits, 0, 0, 0);
}
break;

case ND_BIN: {
    uint8_t op = 0;

    switch (n->op) {
        case '+': op = OP_ADD; break;
        case '-': op = OP_SUB; break;
        case '*': op = OP_MUL; break;
        case '/': op = OP_DIV; break;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 196) - [public] switch

```
    switch (n->op) {
        case '+': op = OP_ADD; break;
        case '-': op = OP_SUB; break;
        case '*': op = OP_MUL; break;
        case '/': op = OP_DIV; break;
        case '%': op = OP_MOD; break;
        case '&': op = OP_AND; break;
        case '|': op = OP_OR; break;
        case '^': op = OP_XOR; break;
        case TK_SHL: op = OP_SHL; break;
        case TK_EQ: op = OP_EQ; break;
        case TK_NEQ: op = OP_NE; break;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 215) - [public] gen_expr

```
    gen_expr(g, n->a, bits);
    gen_expr(g, n->b, bits);
    emit(g, op, bits, 0, 0, 0);
    break;
}

case ND_ASSIGN: {
    gen_expr(g, n->b, bits);

    if (n->a->kind == ND_VAR) {
        int li = find_local(g, n->a->name);

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 216) - [public] gen_expr

```
    gen_expr(g, n->b, bits);
    emit(g, op, bits, 0, 0, 0);
    break;
}

case ND_ASSIGN: {
    gen_expr(g, n->b, bits);

    if (n->a->kind == ND_VAR) {
        int li = find_local(g, n->a->name);

        if (li >= 0) {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 217) - [public] emit

```
    emit(g, op, bits, 0, 0, 0);
    break;
}

case ND_ASSIGN: {
    gen_expr(g, n->b, bits);

    if (n->a->kind == ND_VAR) {
        int li = find_local(g, n->a->name);

        if (li >= 0) {
            emit(g, OP_STORE, bits, g->locals[li].slot, 0, 0);
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 222) - [public] gen_expr

```
gen_expr(g, n->b, bits);

if (n->a->kind == ND_VAR) {
    int li = find_local(g, n->a->name);

    if (li >= 0) {
        emit(g, OP_STORE, bits, g->locals[li].slot, 0, 0);
    } else {
        emit(g, OP_GSTORE, bits, (uint16_t)find_global(P, n->a->name), 0, 0);
    }
}
break;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 224) - [public] if

```
if (n->a->kind == ND_VAR) {
    int li = find_local(g, n->a->name);

    if (li >= 0) {
        emit(g, OP_STORE, bits, g->locals[li].slot, 0, 0);
    } else {
        emit(g, OP_GSTORE, bits, (uint16_t)find_global(P, n->a->name), 0, 0);
    }
}
break;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 227) - [public] if

```
if (li >= 0) {
    emit(g, OP_STORE, bits, g->locals[li].slot, 0, 0);
} else {
    emit(g, OP_GSTORE, bits, (uint16_t)find_global(P, n->a->name), 0, 0);
}
}
break;
}

case ND_MEMBER:
    gen_expr(g, n->a, bits);
    break;
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 228) - [public] emit
    emit(g, OP_STORE, bits, g->locals[li].slot, 0, 0);
    } else {
        emit(g, OP_GSTORE, bits, (uint16_t)find_global(P, n->a->name), 0, 0);
    }
    }
    break;
}

case ND_MEMBER:
    gen_expr(g, n->a, bits);
    break;

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 230) - [public] emit
    emit(g, OP_GSTORE, bits, (uint16_t)find_global(P, n->a->name), 0, 0);
    }
    }
    break;
}

case ND_MEMBER:
    gen_expr(g, n->a, bits);
    break;

case ND_CALL: {
    if (n->a != (void *)0 && n->a->kind == ND_VAR) {
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/codegen.c (line 237) - [public] gen_expr
    gen_expr(g, n->a, bits);
    break;

case ND_CALL: {
    if (n->a != (void *)0 && n->a->kind == ND_VAR) {
        int ei = -1;

        for (size_t i = 0; i < P->nexts; i++) {
            bool same = true;

            for (int k = 0; P->ext_names[i][k] || n->a->name[k]; k++) {
                if (P->ext_names[i][k] != n->a->name[k]) {
                    // ... (truncated, see source file for more)
                }
            }
        }

kernel_Workspace/kernel/src/core-lang/codegen.c (line 241) - [public] if
    if (n->a != (void *)0 && n->a->kind == ND_VAR) {
        int ei = -1;

        for (size_t i = 0; i < P->nexts; i++) {
            bool same = true;

            for (int k = 0; P->ext_names[i][k] || n->a->name[k]; k++) {
                if (P->ext_names[i][k] != n->a->name[k]) {
                    same = false;
                    break;
                }
            }
        }

        // ... (truncated, see source file for more)
    }
}

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 248) - [public] if

```
        if (P->ext_names[i][k] != n->a->name[k]) {
            same = false;
            break;
        }
    }

    if (same) { ei = (int)i; break; }
}

for (size_t i = 0; i < n->nlist; i++) {
    gen_expr(g, n->list[i], 64);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 254) - [public] if

```
    if (same) { ei = (int)i; break; }
}

for (size_t i = 0; i < n->nlist; i++) {
    gen_expr(g, n->list[i], 64);
}

if (ei >= 0) {
    emit(g, OP_EXTERN, 64, (uint16_t)ei, (uint16_t)n->nlist, 0);
} else {
    int fi = -1;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 258) - [public] gen_expr

```
    gen_expr(g, n->list[i], 64);
}

if (ei >= 0) {
    emit(g, OP_EXTERN, 64, (uint16_t)ei, (uint16_t)n->nlist, 0);
} else {
    int fi = -1;

    for (size_t i = 0; i < P->nfns; i++) {
        bool same = true;

        for (int k = 0; P->fn_names[i][k] || n->a->name[k]; k++) {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 261) - [public] if

```
    if (ei >= 0) {
        emit(g, OP_EXTERN, 64, (uint16_t)ei, (uint16_t)n->nlist, 0);
    } else {
        int fi = -1;

        for (size_t i = 0; i < P->nfns; i++) {
            bool same = true;

            for (int k = 0; P->fn_names[i][k] || n->a->name[k]; k++) {
                if (P->fn_names[i][k] != n->a->name[k]) {
                    same = false;
                    break;
                }
            }

            // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 262) - [public] emit
    emit(g, OP_EXTERN, 64, (uint16_t)ei, (uint16_t)n->nlist, 0);
} else {
    int fi = -1;

    for (size_t i = 0; i < P->nfns; i++) {
        bool same = true;

        for (int k = 0; P->fn_names[i][k] || n->a->name[k]; k++) {
            if (P->fn_names[i][k] != n->a->name[k]) {
                same = false;
                break;
            }
        }
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/codegen.c (line 270) - [public] if
    if (P->fn_names[i][k] != n->a->name[k]) {
        same = false;
        break;
    }

    if (same) { fi = (int)i; break; }
}

if (fi >= 0) {
    emit(g, OP_CALL, bits, (uint16_t)fi, (uint16_t)n->nlist, 0);
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 276) - [public] if
    if (same) { fi = (int)i; break; }
}

if (fi >= 0) {
    emit(g, OP_CALL, bits, (uint16_t)fi, (uint16_t)n->nlist, 0);
}
}
}
break;
}

default:
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 279) - [public] if
    if (fi >= 0) {
        emit(g, OP_CALL, bits, (uint16_t)fi, (uint16_t)n->nlist, 0);
    }
}
break;
}

default:
break;
}
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 280) - [public] emit
        emit(g, OP_CALL, bits, (uint16_t)fi, (uint16_t)n->nlist, 0);
    }
}
break;
}

default:
    break;
}
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 296) - [public] switch
switch (n->kind) {
case ND_LET: {
    uint16_t slot = add_local(g, n->name, n->type.bits);

    gen_expr(g, n->a, n->type.bits);
    emit(g, OP_STORE, n->type.bits, slot, 0, 0);
    break;
}

case ND_SET_FREE: {
    int li = find_local(g, n->name);

    // ... (truncated, see source file for more)
}
}

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 300) - [public] gen_expr
    gen_expr(g, n->a, n->type.bits);
    emit(g, OP_STORE, n->type.bits, slot, 0, 0);
    break;
}

case ND_SET_FREE: {
    int li = find_local(g, n->name);

    if (li >= 0) {
        emit(g, OP_FREE, 0, 0, g->locals[li].slot, 0);
    } else {
        emit(g, OP_FREE, 0, 1, (uint16_t)find_global(P, n->name), 0);
    }
    // ... (truncated, see source file for more)
}
}

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 301) - [public] emit
    emit(g, OP_STORE, n->type.bits, slot, 0, 0);
    break;
}

case ND_SET_FREE: {
    int li = find_local(g, n->name);

    if (li >= 0) {
        emit(g, OP_FREE, 0, 0, g->locals[li].slot, 0);
    } else {
        emit(g, OP_FREE, 0, 1, (uint16_t)find_global(P, n->name), 0);
    }
    // ... (truncated, see source file for more)
}
}

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 308) - [public] if
    if (li >= 0) {
        emit(g, OP_FREE, 0, 0, g->locals[li].slot, 0);
    } else {
        emit(g, OP_FREE, 0, 1, (uint16_t)find_global(P, n->name), 0);
    }
    break;
}

case ND_IF: {
    if (n->a != (void *)0) {
        gen_expr(g, n->a, 1);

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 309) - [public] emit
        emit(g, OP_FREE, 0, 0, g->locals[li].slot, 0);
    } else {
        emit(g, OP_FREE, 0, 1, (uint16_t)find_global(P, n->name), 0);
    }
    break;
}

case ND_IF: {
    if (n->a != (void *)0) {
        gen_expr(g, n->a, 1);

        uint32_t jz = emit(g, OP_JZ, 0, 0, 0, 0);
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 311) - [public] emit
        emit(g, OP_FREE, 0, 1, (uint16_t)find_global(P, n->name), 0);
    }
    break;
}

case ND_IF: {
    if (n->a != (void *)0) {
        gen_expr(g, n->a, 1);

        uint32_t jz = emit(g, OP_JZ, 0, 0, 0, 0);

        gen_stmt(g, n->b);
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 317) - [public] if
    if (n->a != (void *)0) {
        gen_expr(g, n->a, 1);

        uint32_t jz = emit(g, OP_JZ, 0, 0, 0, 0);

        gen_stmt(g, n->b);

        if (n->c != (void *)0) {
            uint32_t jmp = emit(g, OP_JMP, 0, 0, 0, 0);

            P->code[jz].c = (uint32_t)P->code_len;
            gen_stmt(g, n->c);
            // ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 318) - [public] gen_expr
    gen_expr(g, n->a, 1);

    uint32_t jz = emit(g, OP_JZ, 0, 0, 0, 0);

    gen_stmt(g, n->b);

    if (n->c != (void *)0) {
        uint32_t jmp = emit(g, OP_JMP, 0, 0, 0, 0);

        P->code[jz].c = (uint32_t)P->code_len;
        gen_stmt(g, n->c);
        P->code[jmp].c = (uint32_t)P->code_len;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 322) - [public] gen_stmt
    gen_stmt(g, n->b);

    if (n->c != (void *)0) {
        uint32_t jmp = emit(g, OP_JMP, 0, 0, 0, 0);

        P->code[jz].c = (uint32_t)P->code_len;
        gen_stmt(g, n->c);
        P->code[jmp].c = (uint32_t)P->code_len;
    } else {
        P->code[jz].c = (uint32_t)P->code_len;
    }
} else {
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 324) - [public] if
    if (n->c != (void *)0) {
        uint32_t jmp = emit(g, OP_JMP, 0, 0, 0, 0);

        P->code[jz].c = (uint32_t)P->code_len;
        gen_stmt(g, n->c);
        P->code[jmp].c = (uint32_t)P->code_len;
    } else {
        P->code[jz].c = (uint32_t)P->code_len;
    }
} else {
    gen_stmt(g, n->b);
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 328) - [public] gen_stmt
    gen_stmt(g, n->c);
    P->code[jmp].c = (uint32_t)P->code_len;
} else {
    P->code[jz].c = (uint32_t)P->code_len;
}
} else {
    gen_stmt(g, n->b);
}
break;
}

case ND_WHEN: {
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 334) - [public] gen_stmt
    gen_stmt(g, n->b);
}
break;
}

case ND_WHEN: {
    uint16_t tslot = add_local(g, n->name, 64);

    int li2 = find_local(g, n->name);

    if (li2 >= 0 && g->locals[li2].slot != tslot) {
        emit(g, OP_LOAD, 64, g->locals[li2].slot, 0, 0);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 344) - [public] if
    if (li2 >= 0 && g->locals[li2].slot != tslot) {
        emit(g, OP_LOAD, 64, g->locals[li2].slot, 0, 0);
        emit(g, OP_STORE, 64, tslot, 0, 0);
    } else if (li2 < 0) {
        emit(g, OP_GLOAD, 64, (uint16_t)find_global(P, n->name), 0, 0);
        emit(g, OP_STORE, 64, tslot, 0, 0);
    }

    for (size_t i = 0; i < n->nlist; i++) {
    }
    break;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 345) - [public] emit
    emit(g, OP_LOAD, 64, g->locals[li2].slot, 0, 0);
    emit(g, OP_STORE, 64, tslot, 0, 0);
} else if (li2 < 0) {
    emit(g, OP_GLOAD, 64, (uint16_t)find_global(P, n->name), 0, 0);
    emit(g, OP_STORE, 64, tslot, 0, 0);
}

    for (size_t i = 0; i < n->nlist; i++) {
    }
    break;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 346) - [public] emit
    emit(g, OP_STORE, 64, tslot, 0, 0);
} else if (li2 < 0) {
    emit(g, OP_GLOAD, 64, (uint16_t)find_global(P, n->name), 0, 0);
    emit(g, OP_STORE, 64, tslot, 0, 0);
}

    for (size_t i = 0; i < n->nlist; i++) {
    }
    break;
}

case ND_BLOCK:
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 348) - [public] emit
    emit(g, OP_GLOAD, 64, (uint16_t)find_global(P, n->name), 0, 0);
    emit(g, OP_STORE, 64, tslot, 0, 0);
}

    for (size_t i = 0; i < n->nlist; i++) {
    }
    break;
}

case ND_BLOCK:
    for (size_t i = 0; i < n->nlist; i++) {
        gen_stmt(g, n->list[i]);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 349) - [public] emit
    emit(g, OP_STORE, 64, tslot, 0, 0);
}

    for (size_t i = 0; i < n->nlist; i++) {
    }
    break;
}

case ND_BLOCK:
    for (size_t i = 0; i < n->nlist; i++) {
        gen_stmt(g, n->list[i]);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 359) - [public] gen_stmt
    gen_stmt(g, n->list[i]);
}
break;

default:
    gen_expr(g, n, 64);
    emit(g, OP_POP, 0, 0, 0, 0);
    break;
}
}

cl_prog_t *cl_codegen(ast_node_t *prog, arena_t *ar)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 364) - [public] gen_expr
    gen_expr(g, n, 64);
    emit(g, OP_POP, 0, 0, 0, 0);
    break;
}
}

cl_prog_t *cl_codegen(ast_node_t *prog, arena_t *ar)
{
    cl_prog_t *P = (cl_prog_t *)cl_arena_alloc(ar, sizeof(cl_prog_t));

    if (P == (void *)0) return (void *)0;

    // ... (truncated, see source file for more)
}

```

```
kernel_Workspace/kernel/src/core-lang/codegen.c (line 365) - [public] emit
    emit(g, OP_POP, 0, 0, 0, 0);
    break;
}
}
```

```
cl_prog_t *cl_codegen(ast_node_t *prog, arena_t *ar)
{
    cl_prog_t *P = (cl_prog_t *)cl_arena_alloc(ar, sizeof(cl_prog_t));

    if (P == (void *)0) return (void *)0;

    P->ar = ar;
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/codegen.c (line 387) - [public] if
    if (n->kind == ND_LET && P->nglobals < 64) {
        for (int k = 0; n->name[k] && k < 63; k++) {
            P->gnames[P->nglobals][k] = n->name[k];
            P->gnames[P->nglobals][k + 1] = '\0';
        }

        P->gbits[P->nglobals] = n->type.bits;
        P->gneg[P->nglobals] = n->type.neg;

        for (int k = 0; k < 8; k++) {
            P->ginit[P->nglobals].b[k] =
                (n->a != (void *)0 && n->a->kind == ND_NUM)
            // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/codegen.c (line 403) - [public] cl_wbytes
    cl_wbytes(n->type.bits), n->type.bits);
```

```
    P->nglobals++;
} else if (n->kind == ND_EXTERN) {
    for (size_t k = 0; k < n->nlist && P->nexts < 16; k++) {
        for (int x = 0; n->list[k]->name[x] && x < 63; x++) {
            P->ext_names[P->nexts][x] = n->list[k]->name[x];
            P->ext_names[P->nexts][x + 1] = '\0';
        }
        P->nexts++;
    }
} else if (n->kind == ND_FN && P->nfns < 16) {
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/codegen.c (line 436) - [public] if
    if (P->fn_names[k][x] != n->name[x]) {
        same = false;
        break;
    }
}

    if (same) { fi = (int)k; break; }
}

    cg_t g = { 0 };

    g.P = P;
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/codegen.c (line 442) - [public] if
    if (same) { fi = (int)k; break; }
}

cg_t g = { 0 };

g.P = P;

for (size_t k = 0; k < n->nlist; k++) {
    add_local(&g, n->list[k]->name, n->list[k]->type.bits);
}

P->fns[fi].start = (uint32_t)P->code_len;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 450) - [public] add_local
    add_local(&g, n->list[k]->name, n->list[k]->type.bits);
}

P->fns[fi].start = (uint32_t)P->code_len;

gen_stmt(&g, n->a);

emit(&g, OP_RET, 0, 0, 0, 0);

P->fns[fi].nslots = g.maxslots;

bool is_main = true;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 455) - [public] gen_stmt
    gen_stmt(&g, n->a);

    emit(&g, OP_RET, 0, 0, 0, 0);

    P->fns[fi].nslots = g.maxslots;

    bool is_main = true;
    const char *m = "main";

    for (int k = 0; m[k] || n->name[k]; k++) {
        if (m[k] != n->name[k]) { is_main = false; break; }
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/codegen.c (line 457) - [public] emit
    emit(&g, OP_RET, 0, 0, 0, 0);

    P->fns[fi].nslots = g.maxslots;

    bool is_main = true;
    const char *m = "main";

    for (int k = 0; m[k] || n->name[k]; k++) {
        if (m[k] != n->name[k]) { is_main = false; break; }
    }

    if (is_main) {
// ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/core-lang/codegen.c (line 465) - [public] if
    if (m[k] != n->name[k]) { is_main = false; break; }
    }

    if (is_main) {
        P->main_fn = fi;
        P->code[P->code_len - 1].op = OP_HALT;
    }
}

return P;
}
```

```
kernel_Workspace/kernel/src/core-lang/codegen.c (line 468) - [public] if
    if (is_main) {
        P->main_fn = fi;
        P->code[P->code_len - 1].op = OP_HALT;
    }
}

return P;
}
```

```
kernel_Workspace/kernel/src/core-lang/lexer.c (line 49) - [public] if
    if (c == '\n') {
        l->line++;
        l->col = 1;
    } else {
        l->col++;
    }
}

return c;
}
```

```
static void emit(lexer_t *l, token_t *t, tok_kind_t kind,
    const char *start, uint32_t len,
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/lexer.c (line 77) - [public] if
    if (c == ' ' || c == '\t' || c == '\r') {
        take(l);
    } else if (c == '\n') {
        take(l);
    } else if (c == '/' && peek2(l) == '/') {
        while (l->pos < l->len && peek(l) != '\n') {
            take(l);
        }
    } else if (c == '/' && peek2(l) == '*') {
        take(l);
        take(l);
    }

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/lexer.c (line 78) - [public] take
    take(l);
} else if (c == '\n') {
    take(l);
} else if (c == '/' && peek2(l) == '/') {
    while (l->pos < l->len && peek(l) != '\n') {
        take(l);
    }
} else if (c == '/' && peek2(l) == '*') {
    take(l);
    take(l);
}

while (l->pos < l->len &&
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 80) - [public] take

```
    take(l);
} else if (c == '/' && peek2(l) == '/') {
    while (l->pos < l->len && peek(l) != '\n') {
        take(l);
    }
} else if (c == '/' && peek2(l) == '*') {
    take(l);
    take(l);

    while (l->pos < l->len &&
           !(peek(l) == '*' && peek2(l) == '/')) {
        take(l);
    }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 82) - [public] while

```
    while (l->pos < l->len && peek(l) != '\n') {
        take(l);
    }
} else if (c == '/' && peek2(l) == '*') {
    take(l);
    take(l);

    while (l->pos < l->len &&
           !(peek(l) == '*' && peek2(l) == '/')) {
        take(l);
    }

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 83) - [public] take

```
    take(l);
}
} else if (c == '/' && peek2(l) == '*') {
    take(l);
    take(l);

    while (l->pos < l->len &&
           !(peek(l) == '*' && peek2(l) == '/')) {
        take(l);
    }

    if (l->pos < l->len) {
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 86) - [public] take

```
    take(l);
    take(l);

    while (l->pos < l->len &&
           !(peek(l) == '*' && peek2(l) == '/')) {
        take(l);
    }

    if (l->pos < l->len) {
        take(l);
        take(l);
    }
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/lexer.c (line 87) - [public] take
    take(l);

    while (l->pos < l->len &&
           !(peek(l) == '*' && peek2(l) == '/')) {
        take(l);
    }

    if (l->pos < l->len) {
        take(l);
        take(l);
    }
} else {
    // ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/core-lang/lexer.c (line 91) - [public] take
    take(l);
}

    if (l->pos < l->len) {
        take(l);
        take(l);
    }
} else {
    return;
}
}
// ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/core-lang/lexer.c (line 94) - [public] if
    if (l->pos < l->len) {
        take(l);
        take(l);
    }
} else {
    return;
}
}

static bool is_id_start(char c)
{
    // ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/core-lang/lexer.c (line 95) - [public] take
    take(l);
    take(l);
}
} else {
    return;
}
}

static bool is_id_start(char c)
{
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c == '_';
    // ... (truncated, see source file for more)

```


kernel_Workspace/kernel/src/core-lang/lexer.c (line 96) - [public] take

```
        take(l);
    }
} else {
    return;
}
}

static bool is_id_start(char c)
{
    return (c >= 'a' && c <= 'z') || (c >= 'A' && c <= 'Z') || c == '_';
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 111) - [public] is_id_start

```
return is_id_start(c) || (c >= '0' && c <= '9');
}
```

```
static bool is_digit(char c)
{
    return c >= '0' && c <= '9';
}
```

```
static int hex_val(char c)
{
    if (c >= '0' && c <= '9') return c - '0';
    if (c >= 'a' && c <= 'f') return c - 'a' + 10;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 133) - [public] if

```
if (peek(l) == '0' && (peek2(l) == 'x' || peek2(l) == 'X')) {
    take(l);
    take(l);

    while (hex_val(peek(l)) >= 0) {
        val = (val << 4) | (uint64_t)hex_val(take(l));
    }
} else if (peek(l) == '0' && (peek2(l) == 'b' || peek2(l) == 'B')) {
    take(l);
    take(l);

    while (peek(l) == '0' || peek(l) == '1') {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 134) - [public] take

```
    take(l);
    take(l);

    while (hex_val(peek(l)) >= 0) {
        val = (val << 4) | (uint64_t)hex_val(take(l));
    }
} else if (peek(l) == '0' && (peek2(l) == 'b' || peek2(l) == 'B')) {
    take(l);
    take(l);

    while (peek(l) == '0' || peek(l) == '1') {
        val = (val << 1) | (uint64_t)(take(l) - '0');
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 135) - [public] take
    take(l);

    while (hex_val(peek(l)) >= 0) {
        val = (val << 4) | (uint64_t)hex_val(take(l));
    }
} else if (peek(l) == '0' && (peek2(l) == 'b' || peek2(l) == 'B')) {
    take(l);
    take(l);

    while (peek(l) == '0' || peek(l) == '1') {
        val = (val << 1) | (uint64_t)(take(l) - '0');
    }
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 137) - [public] while
    while (hex_val(peek(l)) >= 0) {
        val = (val << 4) | (uint64_t)hex_val(take(l));
    }
} else if (peek(l) == '0' && (peek2(l) == 'b' || peek2(l) == 'B')) {
    take(l);
    take(l);

    while (peek(l) == '0' || peek(l) == '1') {
        val = (val << 1) | (uint64_t)(take(l) - '0');
    }
} else {
    while (is_digit(peek(l))) {
        val = (val << 1) | (uint64_t)(take(l) - '0');
    }
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 141) - [public] take
    take(l);
    take(l);

    while (peek(l) == '0' || peek(l) == '1') {
        val = (val << 1) | (uint64_t)(take(l) - '0');
    }
} else {
    while (is_digit(peek(l))) {
        val = val * 10 + (uint64_t)(take(l) - '0');
    }
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 142) - [public] take
    take(l);

    while (peek(l) == '0' || peek(l) == '1') {
        val = (val << 1) | (uint64_t)(take(l) - '0');
    }
} else {
    while (is_digit(peek(l))) {
        val = val * 10 + (uint64_t)(take(l) - '0');
    }
}

emit(l, t, TK_NUM, start,
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 144) - [public] while

```
    while (peek(l) == '0' || peek(l) == '1') {
        val = (val << 1) | (uint64_t)(take(l) - '0');
    }
} else {
    while (is_digit(peek(l))) {
        val = val * 10 + (uint64_t)(take(l) - '0');
    }
}

emit(l, t, TK_NUM, start,
      (uint32_t)(l->src + l->pos - start), line, col);
t->num = val;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 148) - [public] while

```
    while (is_digit(peek(l))) {
        val = val * 10 + (uint64_t)(take(l) - '0');
    }
}

emit(l, t, TK_NUM, start,
      (uint32_t)(l->src + l->pos - start), line, col);
t->num = val;
}
```

```
static void lex_ident(lexer_t *l, token_t *t,
                      uint32_t line, uint32_t col)
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 163) - [public] while

```
while (is_id_char(peek(l))) {
    take(l);
}

uint32_t len = (uint32_t)(l->src + l->pos - start);

for (size_t i = 0; i < sizeof(KEYWORDS) / sizeof(KEYWORDS[0]); i++) {
    const kw_t *k = &KEYWORDS[i];
    size_t kwlen = 0;

    while (k->kw[kwlen]) kwlen++;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 164) - [public] take

```
    take(l);
}

uint32_t len = (uint32_t)(l->src + l->pos - start);

for (size_t i = 0; i < sizeof(KEYWORDS) / sizeof(KEYWORDS[0]); i++) {
    const kw_t *k = &KEYWORDS[i];
    size_t kwlen = 0;

    while (k->kw[kwlen]) kwlen++;

    if (kwlen == len) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 175) - [public] if

```
    if (kwlen == len) {
        bool same = true;

        for (size_t j = 0; j < len; j++) {
            if (start[j] != k->kw[j]) {
                same = false;
                break;
            }
        }

        if (same) {
            emit(l, t, k->kind, start, len, line, col);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 179) - [public] if

```
    if (start[j] != k->kw[j]) {
        same = false;
        break;
    }

    if (same) {
        emit(l, t, k->kind, start, len, line, col);
        t->type_bits = k->bits;
        return;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 185) - [public] if

```
    if (same) {
        emit(l, t, k->kind, start, len, line, col);
        t->type_bits = k->bits;
        return;
    }

    if (len == 1 && start[0] == '_') {
        emit(l, t, TK_UNDERSCORE, start, len, line, col);
        return;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 186) - [public] emit

```
    emit(l, t, k->kind, start, len, line, col);
    t->type_bits = k->bits;
    return;
}

if (len == 1 && start[0] == '_') {
    emit(l, t, TK_UNDERSCORE, start, len, line, col);
    return;
}
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 193) - [public] if
    if (len == 1 && start[0] == '_') {
        emit(l, t, TK_UNDERSCORE, start, len, line, col);
        return;
    }

    emit(l, t, TK_IDENT, start, len, line, col);
}

static void lex_string(lexer_t *l, token_t *t,
                      uint32_t line, uint32_t col)
{
    take(l);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 194) - [public] emit
    emit(l, t, TK_UNDERSCORE, start, len, line, col);
    return;
}

    emit(l, t, TK_IDENT, start, len, line, col);
}

static void lex_string(lexer_t *l, token_t *t,
                      uint32_t line, uint32_t col)
{
    take(l);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 198) - [public] emit
    emit(l, t, TK_IDENT, start, len, line, col);
}

static void lex_string(lexer_t *l, token_t *t,
                      uint32_t line, uint32_t col)
{
    take(l);

    const char *start = l->src + l->pos;

    while (l->pos < l->len && peek(l) != '') {
        if (peek(l) == '\\') {
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 204) - [public] take
    take(l);

    const char *start = l->src + l->pos;

    while (l->pos < l->len && peek(l) != '') {
        if (peek(l) == '\\') {
            take(l);
        }

        take(l);
    }

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 208) - [public] while
    while (l->pos < l->len && peek(l) != '') {
        if (peek(l) == '\\') {
            take(l);
        }

        take(l);
    }

    uint32_t len = (uint32_t)(l->src + l->pos - start);

    if (peek(l) == '') {
        take(l);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 209) - [public] if
    if (peek(l) == '\\') {
        take(l);
    }

    take(l);
}

uint32_t len = (uint32_t)(l->src + l->pos - start);

if (peek(l) == '') {
    take(l);
} else {
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 210) - [public] take
    take(l);
}

    take(l);
}

uint32_t len = (uint32_t)(l->src + l->pos - start);

if (peek(l) == '') {
    take(l);
} else {
    emit(l, t, TK_ERR, start, len, line, col);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 213) - [public] take
    take(l);
}

uint32_t len = (uint32_t)(l->src + l->pos - start);

if (peek(l) == '') {
    take(l);
} else {
    emit(l, t, TK_ERR, start, len, line, col);
    return;
}

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 218) - [public] if
    if (peek(l) == '') {
        take(l);
    } else {
        emit(l, t, TK_ERR, start, len, line, col);
        return;
    }

    emit(l, t, TK_STR, start, len, line, col);
}

static void lex_char(lexer_t *l, token_t *t,
                    uint32_t line, uint32_t col)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 219) - [public] take
    take(l);
} else {
    emit(l, t, TK_ERR, start, len, line, col);
    return;
}

    emit(l, t, TK_STR, start, len, line, col);
}

static void lex_char(lexer_t *l, token_t *t,
                    uint32_t line, uint32_t col)
{
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 221) - [public] emit
    emit(l, t, TK_ERR, start, len, line, col);
    return;
}

    emit(l, t, TK_STR, start, len, line, col);
}

static void lex_char(lexer_t *l, token_t *t,
                    uint32_t line, uint32_t col)
{
    take(l);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 225) - [public] emit
    emit(l, t, TK_STR, start, len, line, col);
}

static void lex_char(lexer_t *l, token_t *t,
                    uint32_t line, uint32_t col)
{
    take(l);

    uint64_t v = 0;

    if (peek(l) == '\\') {
        take(l);
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 231) - [public] take

```
take(l);

uint64_t v = 0;

if (peek(l) == '\\') {
    take(l);

    char e = take(l);

    switch (e) {
        case 'n': v = '\\n'; break;
        case 't': v = '\\t'; break;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 235) - [public] if

```
if (peek(l) == '\\') {
    take(l);

    char e = take(l);

    switch (e) {
        case 'n': v = '\\n'; break;
        case 't': v = '\\t'; break;
        case '0': v = 0; break;
        case '\\': v = '\\'; break;
        case '\\': v = '\\'; break;
        default: v = (uint64_t)e; break;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 236) - [public] take

```
take(l);

char e = take(l);

switch (e) {
    case 'n': v = '\\n'; break;
    case 't': v = '\\t'; break;
    case '0': v = 0; break;
    case '\\': v = '\\'; break;
    case '\\': v = '\\'; break;
    default: v = (uint64_t)e; break;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 240) - [public] switch

```
switch (e) {
    case 'n': v = '\\n'; break;
    case 't': v = '\\t'; break;
    case '0': v = 0; break;
    case '\\': v = '\\'; break;
    case '\\': v = '\\'; break;
    default: v = (uint64_t)e; break;
}
} else {
    v = (uint64_t)(unsigned char)take(l);
}

// ... (truncated, see source file for more)
```



```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 252) - [public] if
    if (peek(l) == '\\') {
        take(l);
    } else {
        emit(l, t, TK_ERR, l->src + l->pos, 1, line, col);
        return;
    }

    emit(l, t, TK_CHAR, l->src + l->pos - 1, 1, line, col);
    t->num = v;
}

bool cl_lex_next(lexer_t *l, token_t *t)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 253) - [public] take
    take(l);
} else {
    emit(l, t, TK_ERR, l->src + l->pos, 1, line, col);
    return;
}

    emit(l, t, TK_CHAR, l->src + l->pos - 1, 1, line, col);
    t->num = v;
}

bool cl_lex_next(lexer_t *l, token_t *t)
{
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 255) - [public] emit
    emit(l, t, TK_ERR, l->src + l->pos, 1, line, col);
    return;
}

    emit(l, t, TK_CHAR, l->src + l->pos - 1, 1, line, col);
    t->num = v;
}

bool cl_lex_next(lexer_t *l, token_t *t)
{
    skip_ws(l);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 259) - [public] emit
    emit(l, t, TK_CHAR, l->src + l->pos - 1, 1, line, col);
    t->num = v;
}

bool cl_lex_next(lexer_t *l, token_t *t)
{
    skip_ws(l);

    uint32_t line = l->line;
    uint32_t col = l->col;

    if (l->pos >= l->len) {
        // ... (truncated, see source file for more)
    }
}

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 265) - [public] skip_ws

```
skip_ws(l);

uint32_t line = l->line;
uint32_t col = l->col;

if (l->pos >= l->len) {
    emit(l, t, TK_EOF, l->src + l->pos, 0, line, col);
    return true;
}

char c = peek(l);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 270) - [public] if

```
if (l->pos >= l->len) {
    emit(l, t, TK_EOF, l->src + l->pos, 0, line, col);
    return true;
}

char c = peek(l);

if (is_digit(c)) {
    lex_number(l, t, line, col);
    return true;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 271) - [public] emit

```
emit(l, t, TK_EOF, l->src + l->pos, 0, line, col);
return true;
}

char c = peek(l);

if (is_digit(c)) {
    lex_number(l, t, line, col);
    return true;
}

if (is_id_start(c)) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 277) - [public] if

```
if (is_digit(c)) {
    lex_number(l, t, line, col);
    return true;
}

if (is_id_start(c)) {
    lex_ident(l, t, line, col);
    return true;
}

if (c == '"') {
    lex_string(l, t, line, col);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 278) - [public] lex_number

```
    lex_number(l, t, line, col);
    return true;
}

if (is_id_start(c)) {
    lex_ident(l, t, line, col);
    return true;
}

if (c == '"') {
    lex_string(l, t, line, col);
    return t->kind != TK_ERR;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 282) - [public] if

```
if (is_id_start(c)) {
    lex_ident(l, t, line, col);
    return true;
}

if (c == '"') {
    lex_string(l, t, line, col);
    return t->kind != TK_ERR;
}

if (c == '\\') {
    lex_char(l, t, line, col);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 283) - [public] lex_ident

```
    lex_ident(l, t, line, col);
    return true;
}

if (c == '"') {
    lex_string(l, t, line, col);
    return t->kind != TK_ERR;
}

if (c == '\\') {
    lex_char(l, t, line, col);
    return t->kind != TK_ERR;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 287) - [public] if

```
if (c == '"') {
    lex_string(l, t, line, col);
    return t->kind != TK_ERR;
}

if (c == '\\') {
    lex_char(l, t, line, col);
    return t->kind != TK_ERR;
}

const char *start = l->src + l->pos;

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 288) - [public] lex_string
    lex_string(l, t, line, col);
    return t->kind != TK_ERR;
}

if (c == '\\') {
    lex_char(l, t, line, col);
    return t->kind != TK_ERR;
}

const char *start = l->src + l->pos;

if (c == '$') {
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/lexer.c (line 292) - [public] if
    if (c == '\\') {
        lex_char(l, t, line, col);
        return t->kind != TK_ERR;
    }

    const char *start = l->src + l->pos;

    if (c == '$') {
        take(l);

        if (peek(l) == '!') {
            take(l);
            // ... (truncated, see source file for more)
        }
    }

kernel_Workspace/kernel/src/core-lang/lexer.c (line 293) - [public] lex_char
    lex_char(l, t, line, col);
    return t->kind != TK_ERR;
}

const char *start = l->src + l->pos;

if (c == '$') {
    take(l);

    if (peek(l) == '!') {
        take(l);
        emit(l, t, TK_DOLLAR_BANG, start, 2, line, col);
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/core-lang/lexer.c (line 299) - [public] if
    if (c == '$') {
        take(l);

        if (peek(l) == '!') {
            take(l);
            emit(l, t, TK_DOLLAR_BANG, start, 2, line, col);
        } else if (peek(l) == '@') {
            take(l);
            emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
        } else {
            emit(l, t, TK_DOLLAR, start, 1, line, col);
        }
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 300) - [public] take
    take(l);

    if (peek(l) == '!') {
        take(l);
        emit(l, t, TK_DOLLAR_BANG, start, 2, line, col);
    } else if (peek(l) == '@') {
        take(l);
        emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
    } else {
        emit(l, t, TK_DOLLAR, start, 1, line, col);
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 302) - [public] if
    if (peek(l) == '!') {
        take(l);
        emit(l, t, TK_DOLLAR_BANG, start, 2, line, col);
    } else if (peek(l) == '@') {
        take(l);
        emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
    } else {
        emit(l, t, TK_DOLLAR, start, 1, line, col);
    }

    return true;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 303) - [public] take
    take(l);
    emit(l, t, TK_DOLLAR_BANG, start, 2, line, col);
} else if (peek(l) == '@') {
    take(l);
    emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
} else {
    emit(l, t, TK_DOLLAR, start, 1, line, col);
}

    return true;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/lexer.c (line 304) - [public] emit
    emit(l, t, TK_DOLLAR_BANG, start, 2, line, col);
} else if (peek(l) == '@') {
    take(l);
    emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
} else {
    emit(l, t, TK_DOLLAR, start, 1, line, col);
}

    return true;
}

char d = peek2(l);
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 306) - [public] take

```
        take(l);
        emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
    } else {
        emit(l, t, TK_DOLLAR, start, 1, line, col);
    }

    return true;
}

char d = peek2(l);

if (c == '=' && d == '=') { take(l); take(l); emit(l, t, TK_EQ, start, 2, line, col); return true; }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 307) - [public] emit

```
        emit(l, t, TK_DOLLAR_AT, start, 2, line, col);
    } else {
        emit(l, t, TK_DOLLAR, start, 1, line, col);
    }

    return true;
}

char d = peek2(l);

if (c == '=' && d == '=') { take(l); take(l); emit(l, t, TK_EQ, start, 2, line, col); return true; }
if (c == '=' && d == '>') { take(l); take(l); emit(l, t, TK_ARROW, start, 2, line, col); return true; }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 309) - [public] emit

```
        emit(l, t, TK_DOLLAR, start, 1, line, col);
    }

    return true;
}

char d = peek2(l);

if (c == '=' && d == '=') { take(l); take(l); emit(l, t, TK_EQ, start, 2, line, col); return true; }
if (c == '=' && d == '>') { take(l); take(l); emit(l, t, TK_ARROW, start, 2, line, col); return true; }
if (c == '=' && d == ':') { take(l); take(l); emit(l, t, TK_EQCOLON, start, 2, line, col); return true; }
if (c == '!' && d == '=') { take(l); take(l); emit(l, t, TK_NEQ, start, 2, line, col); return true; }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 317) - [public] if

```
if (c == '=' && d == '=') { take(l); take(l); emit(l, t, TK_EQ, start, 2, line, col); return true; }
if (c == '=' && d == '>') { take(l); take(l); emit(l, t, TK_ARROW, start, 2, line, col); return true; }
if (c == '=' && d == ':') { take(l); take(l); emit(l, t, TK_EQCOLON, start, 2, line, col); return true; }
if (c == '!' && d == '=') { take(l); take(l); emit(l, t, TK_NEQ, start, 2, line, col); return true; }
if (c == '<' && d == '=') { take(l); take(l); emit(l, t, TK_LEQ, start, 2, line, col); return true; }
if (c == '>' && d == '=') { take(l); take(l); emit(l, t, TK_GEQ, start, 2, line, col); return true; }
if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2, line, col); return true; }
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 318) - [public] if

```
if (c == '=' && d == '>') { take(l); take(l); emit(l, t, TK_ARROW, start, 2, line, col); return true; }
if (c == '=' && d == ':') { take(l); take(l); emit(l, t, TK_EQCOLON, start, 2, line, col); return true; }
if (c == '!' && d == '=') { take(l); take(l); emit(l, t, TK_NEQ, start, 2, line, col); return true; }
if (c == '<' && d == '=') { take(l); take(l); emit(l, t, TK_LEQ, start, 2, line, col); return true; }
if (c == '>' && d == '=') { take(l); take(l); emit(l, t, TK_GEQ, start, 2, line, col); return true; }
if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2, line, col); return true; }
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 319) - [public] if

```
if (c == '=' && d == ':') { take(l); take(l); emit(l, t, TK_EQCOLON, start, 2, line, col); return true; }
if (c == '!' && d == '=') { take(l); take(l); emit(l, t, TK_NEQ, start, 2, line, col); return true; }
if (c == '<' && d == '=') { take(l); take(l); emit(l, t, TK_LEQ, start, 2, line, col); return true; }
if (c == '>' && d == '=') { take(l); take(l); emit(l, t, TK_GEQ, start, 2, line, col); return true; }
if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2, line, col); return true; }
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

switch (c) {
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 320) - [public] if

```
if (c == '!' && d == '=') { take(l); take(l); emit(l, t, TK_NEQ, start, 2, line, col); return true; }
if (c == '<' && d == '=') { take(l); take(l); emit(l, t, TK_LEQ, start, 2, line, col); return true; }
if (c == '>' && d == '=') { take(l); take(l); emit(l, t, TK_GEQ, start, 2, line, col); return true; }
if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2, line, col); return true; }
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 321) - [public] if

```
if (c == '<' && d == '=') { take(l); take(l); emit(l, t, TK_LEQ, start, 2, line, col); return true; }
if (c == '>' && d == '=') { take(l); take(l); emit(l, t, TK_GEQ, start, 2, line, col); return true; }
if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2, line, col); return true; }
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
case ',': emit(l, t, TK_COMMA, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 322) - [public] if

```
if (c == '>' && d == '=') { take(l); take(l); emit(l, t, TK_GEQ, start, 2, line, col); return true; }
if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2, line, col); return true; }
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
case ',': emit(l, t, TK_COMMA, start, 1, line, col); return true;
case '.': emit(l, t, TK_DOT, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 323) - [public] if

```
if (c == '<' && d == '<') { take(l); take(l); emit(l, t, TK_SHL, start, 2, line, col); return true; }
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
case ',': emit(l, t, TK_COMMA, start, 1, line, col); return true;
case '.': emit(l, t, TK_DOT, start, 1, line, col); return true;
case ':': emit(l, t, TK_COLON, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 324) - [public] if

```
if (c == '>' && d == '>') { take(l); take(l); emit(l, t, TK_SHR, start, 2, line, col); return true; }
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
case ',': emit(l, t, TK_COMMA, start, 1, line, col); return true;
case '.': emit(l, t, TK_DOT, start, 1, line, col); return true;
case ':': emit(l, t, TK_COLON, start, 1, line, col); return true;
case '=': emit(l, t, TK_ASSIGN, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 325) - [public] if

```
if (c == ':' && d == ':') { take(l); take(l); emit(l, t, TK_DCOLON, start, 2, line, col); return true; }

/* jednoznaki */
take(l);

switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
case ',': emit(l, t, TK_COMMA, start, 1, line, col); return true;
case '.': emit(l, t, TK_DOT, start, 1, line, col); return true;
case ':': emit(l, t, TK_COLON, start, 1, line, col); return true;
case '=': emit(l, t, TK_ASSIGN, start, 1, line, col); return true;
case '<': emit(l, t, TK_LT, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/core-lang/lexer.c (line 328) - [public] take

```
/* jednoznaki */
```

```
take(l);
```

```
switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
case ',': emit(l, t, TK_COMMA, start, 1, line, col); return true;
case '.': emit(l, t, TK_DOT, start, 1, line, col); return true;
case ':': emit(l, t, TK_COLON, start, 1, line, col); return true;
case '=': emit(l, t, TK_ASSIGN, start, 1, line, col); return true;
case '<': emit(l, t, TK_LT, start, 1, line, col); return true;
case '>': emit(l, t, TK_GT, start, 1, line, col); return true;
case '+': emit(l, t, TK_PLUS, start, 1, line, col); return true;
case '-': emit(l, t, TK_MINUS, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 330) - [public] switch

```
switch (c) {
case ';': emit(l, t, TK_SEMI, start, 1, line, col); return true;
case ',': emit(l, t, TK_COMMA, start, 1, line, col); return true;
case '.': emit(l, t, TK_DOT, start, 1, line, col); return true;
case ':': emit(l, t, TK_COLON, start, 1, line, col); return true;
case '=': emit(l, t, TK_ASSIGN, start, 1, line, col); return true;
case '<': emit(l, t, TK_LT, start, 1, line, col); return true;
case '>': emit(l, t, TK_GT, start, 1, line, col); return true;
case '+': emit(l, t, TK_PLUS, start, 1, line, col); return true;
case '-': emit(l, t, TK_MINUS, start, 1, line, col); return true;
case '*': emit(l, t, TK_STAR, start, 1, line, col); return true;
case '/': emit(l, t, TK_SLASH, start, 1, line, col); return true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 355) - [public] emit

```
    emit(l, t, TK_ERR, start, 1, line, col);
    return false;
}
}
```

```
void cl_lexer_init(lexer_t *l, const char *src, size_t len)
```

```
{
    l->src = src;
    l->len = len;
    l->pos = 0;
    l->line = 1;
    l->col = 1;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 374) - [public] if

```
    if (n >= cap) {
        return -1;
    }

    bool ok = cl_lex_next(l, &buf[n]);

    if (!ok) {
        return -2;
    }

    tok_kind_t k = buf[n].kind;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 380) - [public] if

```
    if (!ok) {
        return -2;
    }

    tok_kind_t k = buf[n].kind;

    n++;

    if (k == TK_EOF) {
        break;
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 388) - [public] if

```
    if (k == TK_EOF) {
        break;
    }
}

*count = n;
return 0;
}
```

```
const char *cl_tok_name(tok_kind_t kind)
{
    switch (kind) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/core-lang/lexer.c (line 399) - [public] switch

```
switch (kind) {
case TK_EOF: return "EOF";
case TK_ERR: return "ERR";
case TK_IDENT: return "IDENT";
case TK_TYPE: return "TYPE";
case TK_NUM: return "NUM";
case TK_STR: return "STR";
case TK_CHAR: return "CHAR";
case TK_IF: return "if";
case TK_ELSE: return "else";
case TK_WHEN: return "when";
case TK_CASE: return "case";
// ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 19) - [public] if

```
if (toks == (void *)0) {
    *err_line = 0;
    *err_msg = "arena full";
    return (void *)0;
}

cl_lexer_init(&l, src, len);

if (cl_lex_all(&l, toks, tcap, &n) != 0) {
    *err_line = l.line;
    *err_msg = "lex error";
    return (void *)0;
}
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 25) - [public] cl_lexer_init
    cl_lexer_init(&l, src, len);

    if (cl_lex_all(&l, toks, tcap, &n) != 0) {
        *err_line = l.line;
        *err_msg = "lex error";
        return (void *)0;
    }

    parser_t p;
    ast_node_t *prog = cl_parse(&p, toks, n, ar);

    if (prog == (void *)0) {
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/loader.c (line 27) - [public] if
    if (cl_lex_all(&l, toks, tcap, &n) != 0) {
        *err_line = l.line;
        *err_msg = "lex error";
        return (void *)0;
    }

    parser_t p;
    ast_node_t *prog = cl_parse(&p, toks, n, ar);

    if (prog == (void *)0) {
        *err_line = p.err_line;
        *err_msg = p.err;
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/loader.c (line 36) - [public] if
    if (prog == (void *)0) {
        *err_line = p.err_line;
        *err_msg = p.err;
        return (void *)0;
    }

    sema_ctx_t *s = cl_sema_new(ar);

    if (s == (void *)0) {
        *err_line = 0;
        *err_msg = "arena full";
        return (void *)0;
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/loader.c (line 44) - [public] if
    if (s == (void *)0) {
        *err_line = 0;
        *err_msg = "arena full";
        return (void *)0;
    }

    cl_sema_run(s, prog);

    if (cl_sema_diag_count(s) > 0) {
        const cl_diag_t *d = cl_sema_diag(s, 0);

        *err_line = d->line;
        // ... (truncated, see source file for more)
    }

```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 50) - [public] cl_sema_run
    cl_sema_run(s, prog);

    if (cl_sema_diag_count(s) > 0) {
        const cl_diag_t *d = cl_sema_diag(s, 0);

        *err_line = d->line;
        *err_msg = d->msg;
        return (void *)0;
    }

    cl_prog_t *P = cl_codegen(prog, ar);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 52) - [public] if
    if (cl_sema_diag_count(s) > 0) {
        const cl_diag_t *d = cl_sema_diag(s, 0);

        *err_line = d->line;
        *err_msg = d->msg;
        return (void *)0;
    }

    cl_prog_t *P = cl_codegen(prog, ar);

    if (P == (void *)0) {
        *err_line = 0;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 62) - [public] if
    if (P == (void *)0) {
        *err_line = 0;
        *err_msg = "codegen failed";
        return (void *)0;
    }

    return P;
}

typedef struct {
    uint8_t *b;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 88) - [public] if
    if (!w->ok || w->o >= w->cap) {
        w->ok = false;
        return;
    }

    w->b[w->o++] = v;
}

static void wr_u32(wr_t *w, uint32_t v)
{
    for (int i = 0; i < 4; i++) {
        wr_u8(w, (uint8_t)(v >> (i * 8)));
        // ... (truncated, see source file for more)
    }
}

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 99) - [public] wr_u8

```
    wr_u8(w, (uint8_t)(v >> (i * 8)));
}

static void wr_bytes(wr_t *w, const void *src, size_t n)
{
    for (size_t i = 0; i < n; i++) {
        wr_u8(w, ((const uint8_t *)src)[i]);
    }
}

static uint8_t rd_u8(rd_t *r)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 106) - [public] wr_u8

```
    wr_u8(w, ((const uint8_t *)src)[i]);
}

static uint8_t rd_u8(rd_t *r)
{
    if (!r->ok || r->o >= r->len) {
        r->ok = false;
        return 0;
    }

    return r->b[r->o++];
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 112) - [public] if

```
    if (!r->ok || r->o >= r->len) {
        r->ok = false;
        return 0;
    }

    return r->b[r->o++];
}

static uint32_t rd_u32(rd_t *r)
{
    uint32_t v = 0;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 142) - [public] wr_bytes

```
    wr_bytes(&w, "CLBC1", 5);

    wr_u32(&w, (uint32_t)P->code_len);

    for (size_t i = 0; i < P->code_len; i++) {
        const cl_insn_t *in = &P->code[i];

        wr_u8(&w, in->op);
        wr_u8(&w, in->bits);
        wr_u8(&w, (uint8_t)(in->a & 0xFF));
        wr_u8(&w, (uint8_t)(in->a >> 8));
        wr_u8(&w, (uint8_t)(in->b & 0xFF));
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 144) - [public] wr_u32

```
wr_u32(&w, (uint32_t)P->code_len);

for (size_t i = 0; i < P->code_len; i++) {
    const cl_insn_t *in = &P->code[i];

    wr_u8(&w, in->op);
    wr_u8(&w, in->bits);
    wr_u8(&w, (uint8_t)(in->a & 0xFF));
    wr_u8(&w, (uint8_t)(in->a >> 8));
    wr_u8(&w, (uint8_t)(in->b & 0xFF));
    wr_u8(&w, (uint8_t)(in->b >> 8));
    wr_u32(&w, in->c);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 149) - [public] wr_u8

```
wr_u8(&w, in->op);
wr_u8(&w, in->bits);
wr_u8(&w, (uint8_t)(in->a & 0xFF));
wr_u8(&w, (uint8_t)(in->a >> 8));
wr_u8(&w, (uint8_t)(in->b & 0xFF));
wr_u8(&w, (uint8_t)(in->b >> 8));
wr_u32(&w, in->c);
}

wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 150) - [public] wr_u8

```
wr_u8(&w, in->bits);
wr_u8(&w, (uint8_t)(in->a & 0xFF));
wr_u8(&w, (uint8_t)(in->a >> 8));
wr_u8(&w, (uint8_t)(in->b & 0xFF));
wr_u8(&w, (uint8_t)(in->b >> 8));
wr_u32(&w, in->c);
}

wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    wr_bytes(&w, P->consts[i].b, 64);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 151) - [public] wr_u8

```
wr_u8(&w, (uint8_t)(in->a & 0xFF));
wr_u8(&w, (uint8_t)(in->a >> 8));
wr_u8(&w, (uint8_t)(in->b & 0xFF));
wr_u8(&w, (uint8_t)(in->b >> 8));
wr_u32(&w, in->c);
}

wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    wr_bytes(&w, P->consts[i].b, 64);
    wr_u8(&w, P->const_bits[i]);
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 152) - [public] wr_u8
    wr_u8(&w, (uint8_t)(in->a >> 8));
    wr_u8(&w, (uint8_t)(in->b & 0xFF));
    wr_u8(&w, (uint8_t)(in->b >> 8));
    wr_u32(&w, in->c);
}

wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    wr_bytes(&w, P->consts[i].b, 64);
    wr_u8(&w, P->const_bits[i]);
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 153) - [public] wr_u8
    wr_u8(&w, (uint8_t)(in->b & 0xFF));
    wr_u8(&w, (uint8_t)(in->b >> 8));
    wr_u32(&w, in->c);
}

wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    wr_bytes(&w, P->consts[i].b, 64);
    wr_u8(&w, P->const_bits[i]);
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 154) - [public] wr_u8
    wr_u8(&w, (uint8_t)(in->b >> 8));
    wr_u32(&w, in->c);
}

wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    wr_bytes(&w, P->consts[i].b, 64);
    wr_u8(&w, P->const_bits[i]);
}

wr_u32(&w, (uint32_t)P->nfns);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 155) - [public] wr_u32
    wr_u32(&w, in->c);
}

wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    wr_bytes(&w, P->consts[i].b, 64);
    wr_u8(&w, P->const_bits[i]);
}

wr_u32(&w, (uint32_t)P->nfns);

// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 158) - [public] wr_u32

```
wr_u32(&w, (uint32_t)P->nconsts);

for (size_t i = 0; i < P->nconsts; i++) {
    wr_bytes(&w, P->consts[i].b, 64);
    wr_u8(&w, P->const_bits[i]);
}

wr_u32(&w, (uint32_t)P->nfns);

for (size_t i = 0; i < P->nfns; i++) {
    wr_u32(&w, P->fns[i].start);
    wr_u8(&w, (uint8_t)P->fns[i].nslots);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 161) - [public] wr_bytes

```
    wr_bytes(&w, P->consts[i].b, 64);
    wr_u8(&w, P->const_bits[i]);
}

wr_u32(&w, (uint32_t)P->nfns);

for (size_t i = 0; i < P->nfns; i++) {
    wr_u32(&w, P->fns[i].start);
    wr_u8(&w, (uint8_t)P->fns[i].nslots);
    wr_u8(&w, (uint8_t)P->fns[i].nargs);
    wr_bytes(&w, P->fn_names[i], 64);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 162) - [public] wr_u8

```
    wr_u8(&w, P->const_bits[i]);
}

wr_u32(&w, (uint32_t)P->nfns);

for (size_t i = 0; i < P->nfns; i++) {
    wr_u32(&w, P->fns[i].start);
    wr_u8(&w, (uint8_t)P->fns[i].nslots);
    wr_u8(&w, (uint8_t)P->fns[i].nargs);
    wr_bytes(&w, P->fn_names[i], 64);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 165) - [public] wr_u32

```
wr_u32(&w, (uint32_t)P->nfns);

for (size_t i = 0; i < P->nfns; i++) {
    wr_u32(&w, P->fns[i].start);
    wr_u8(&w, (uint8_t)P->fns[i].nslots);
    wr_u8(&w, (uint8_t)P->fns[i].nargs);
    wr_bytes(&w, P->fn_names[i], 64);
}

wr_u32(&w, (uint32_t)P->nglobals);

for (size_t i = 0; i < P->nglobals; i++) {
    // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/core-lang/loader.c (line 168) - [public] wr_u32

```
    wr_u32(&w, P->fns[i].start);
    wr_u8(&w, (uint8_t)P->fns[i].nslots);
    wr_u8(&w, (uint8_t)P->fns[i].nargs);
    wr_bytes(&w, P->fn_names[i], 64);
}

wr_u32(&w, (uint32_t)P->nglobals);

for (size_t i = 0; i < P->nglobals; i++) {
    wr_bytes(&w, P->gnames[i], 64);
    wr_u8(&w, P->gbits[i]);
    wr_u8(&w, P->gneg[i]);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 169) - [public] wr_u8

```
    wr_u8(&w, (uint8_t)P->fns[i].nslots);
    wr_u8(&w, (uint8_t)P->fns[i].nargs);
    wr_bytes(&w, P->fn_names[i], 64);
}

wr_u32(&w, (uint32_t)P->nglobals);

for (size_t i = 0; i < P->nglobals; i++) {
    wr_bytes(&w, P->gnames[i], 64);
    wr_u8(&w, P->gbits[i]);
    wr_u8(&w, P->gneg[i]);
    wr_bytes(&w, P->ginit[i].b, 64);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 170) - [public] wr_u8

```
    wr_u8(&w, (uint8_t)P->fns[i].nargs);
    wr_bytes(&w, P->fn_names[i], 64);
}

wr_u32(&w, (uint32_t)P->nglobals);

for (size_t i = 0; i < P->nglobals; i++) {
    wr_bytes(&w, P->gnames[i], 64);
    wr_u8(&w, P->gbits[i]);
    wr_u8(&w, P->gneg[i]);
    wr_bytes(&w, P->ginit[i].b, 64);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 171) - [public] wr_bytes

```
    wr_bytes(&w, P->fn_names[i], 64);
}

wr_u32(&w, (uint32_t)P->nglobals);

for (size_t i = 0; i < P->nglobals; i++) {
    wr_bytes(&w, P->gnames[i], 64);
    wr_u8(&w, P->gbits[i]);
    wr_u8(&w, P->gneg[i]);
    wr_bytes(&w, P->ginit[i].b, 64);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 174) - [public] wr_u32

```
wr_u32(&w, (uint32_t)P->nglobals);

for (size_t i = 0; i < P->nglobals; i++) {
    wr_bytes(&w, P->gnames[i], 64);
    wr_u8(&w, P->gbits[i]);
    wr_u8(&w, P->gneg[i]);
    wr_bytes(&w, P->ginit[i].b, 64);
}

wr_u32(&w, (uint32_t)P->nexts);

for (size_t i = 0; i < P->nexts; i++) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 177) - [public] wr_bytes

```
    wr_bytes(&w, P->gnames[i], 64);
    wr_u8(&w, P->gbits[i]);
    wr_u8(&w, P->gneg[i]);
    wr_bytes(&w, P->ginit[i].b, 64);
}

wr_u32(&w, (uint32_t)P->nexts);

for (size_t i = 0; i < P->nexts; i++) {
    wr_bytes(&w, P->ext_names[i], 64);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 178) - [public] wr_u8

```
    wr_u8(&w, P->gbits[i]);
    wr_u8(&w, P->gneg[i]);
    wr_bytes(&w, P->ginit[i].b, 64);
}

wr_u32(&w, (uint32_t)P->nexts);

for (size_t i = 0; i < P->nexts; i++) {
    wr_bytes(&w, P->ext_names[i], 64);
}

wr_u32(&w, (uint32_t)P->main_fn);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 179) - [public] wr_u8

```
    wr_u8(&w, P->gneg[i]);
    wr_bytes(&w, P->ginit[i].b, 64);
}

wr_u32(&w, (uint32_t)P->nexts);

for (size_t i = 0; i < P->nexts; i++) {
    wr_bytes(&w, P->ext_names[i], 64);
}

wr_u32(&w, (uint32_t)P->main_fn);

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 180) - [public] wr_bytes
    wr_bytes(&w, P->ginit[i].b, 64);
}

wr_u32(&w, (uint32_t)P->nexts);

for (size_t i = 0; i < P->nexts; i++) {
    wr_bytes(&w, P->ext_names[i], 64);
}

wr_u32(&w, (uint32_t)P->main_fn);

return w.ok ? w.o : 0;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 183) - [public] wr_u32
    wr_u32(&w, (uint32_t)P->nexts);

    for (size_t i = 0; i < P->nexts; i++) {
        wr_bytes(&w, P->ext_names[i], 64);
    }

    wr_u32(&w, (uint32_t)P->main_fn);

    return w.ok ? w.o : 0;
}

cl_prog_t *cl_load_bc(const uint8_t *buf, size_t len, arena_t *ar)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 186) - [public] wr_bytes
    wr_bytes(&w, P->ext_names[i], 64);
}

wr_u32(&w, (uint32_t)P->main_fn);

return w.ok ? w.o : 0;
}

cl_prog_t *cl_load_bc(const uint8_t *buf, size_t len, arena_t *ar)
{
    rd_t r = { buf, len, 0, true };

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 189) - [public] wr_u32
    wr_u32(&w, (uint32_t)P->main_fn);

    return w.ok ? w.o : 0;
}

cl_prog_t *cl_load_bc(const uint8_t *buf, size_t len, arena_t *ar)
{
    rd_t r = { buf, len, 0, true };

    uint8_t magic[5];

    rd_bytes(&r, magic, 5);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 200) - [public] rd_bytes
    rd_bytes(&r, magic, 5);

    if (!r.ok || magic[0] != 'C' || magic[1] != 'L' ||
        magic[2] != 'B' || magic[3] != 'C' || magic[4] != '1') {
        return (void *)0;
    }

    cl_prog_t *P = (cl_prog_t *)cl_arena_alloc(ar, sizeof(cl_prog_t));

    if (P == (void *)0) {
        return (void *)0;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 209) - [public] if
    if (P == (void *)0) {
        return (void *)0;
    }

    P->ar = ar;
    P->code_cap = 4096;
    P->code = (cl_insn_t *)cl_arena_alloc(ar,
        P->code_cap * sizeof(cl_insn_t));

    if (P->code == (void *)0) {
        return (void *)0;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 218) - [public] if
    if (P->code == (void *)0) {
        return (void *)0;
    }

    uint32_t cl = rd_u32(&r);

    if (cl > P->code_cap) {
        return (void *)0;
    }

    P->code_len = cl;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 224) - [public] if
    if (cl > P->code_cap) {
        return (void *)0;
    }

    P->code_len = cl;

    for (size_t i = 0; i < P->code_len; i++) {
        cl_insn_t *in = &P->code[i];

        in->op = rd_u8(&r);
        in->bits = rd_u8(&r);
        in->a = (uint16_t)(rd_u8(&r) | ((uint16_t)rd_u8(&r) << 8));
    }
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 242) - [public] if

```
    if (P->nconsts > 64) {
        return (void *)0;
    }

    for (size_t i = 0; i < P->nconsts; i++) {
        rd_bytes(&r, P->consts[i].b, 64);
        P->const_bits[i] = rd_u8(&r);
    }

    P->nfns = rd_u32(&r);

    if (P->nfns > 16) {
        // ... (truncated, see source file for more)
    }
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 247) - [public] rd_bytes

```
        rd_bytes(&r, P->consts[i].b, 64);
        P->const_bits[i] = rd_u8(&r);
    }

    P->nfns = rd_u32(&r);

    if (P->nfns > 16) {
        return (void *)0;
    }

    for (size_t i = 0; i < P->nfns; i++) {
        P->fns[i].start = rd_u32(&r);
        // ... (truncated, see source file for more)
    }
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 253) - [public] if

```
    if (P->nfns > 16) {
        return (void *)0;
    }

    for (size_t i = 0; i < P->nfns; i++) {
        P->fns[i].start = rd_u32(&r);
        P->fns[i].nslots = rd_u8(&r);
        P->fns[i].nargs = rd_u8(&r);
        rd_bytes(&r, P->fn_names[i], 64);
    }

    P->nglobals = rd_u32(&r);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/core-lang/loader.c (line 261) - [public] rd_bytes

```
        rd_bytes(&r, P->fn_names[i], 64);
    }

    P->nglobals = rd_u32(&r);

    if (P->nglobals > 64) {
        return (void *)0;
    }

    for (size_t i = 0; i < P->nglobals; i++) {
        rd_bytes(&r, P->gnames[i], 64);
        P->gbits[i] = rd_u8(&r);
        // ... (truncated, see source file for more)
    }
```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 266) - [public] if
    if (P->nglobals > 64) {
        return (void *)0;
    }

    for (size_t i = 0; i < P->nglobals; i++) {
        rd_bytes(&r, P->gnames[i], 64);
        P->gbits[i] = rd_u8(&r);
        P->gneg[i] = rd_u8(&r);
        rd_bytes(&r, P->ginit[i].b, 64);
    }

    P->nexts = rd_u32(&r);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 271) - [public] rd_bytes
    rd_bytes(&r, P->gnames[i], 64);
    P->gbits[i] = rd_u8(&r);
    P->gneg[i] = rd_u8(&r);
    rd_bytes(&r, P->ginit[i].b, 64);
}

    P->nexts = rd_u32(&r);

    if (P->nexts > 16) {
        return (void *)0;
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 274) - [public] rd_bytes
    rd_bytes(&r, P->ginit[i].b, 64);
}

    P->nexts = rd_u32(&r);

    if (P->nexts > 16) {
        return (void *)0;
    }

    for (size_t i = 0; i < P->nexts; i++) {
        rd_bytes(&r, P->ext_names[i], 64);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 279) - [public] if
    if (P->nexts > 16) {
        return (void *)0;
    }

    for (size_t i = 0; i < P->nexts; i++) {
        rd_bytes(&r, P->ext_names[i], 64);
    }

    P->main_fn = (int)rd_u32(&r);

    if (!r.ok) {
        return (void *)0;
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 284) - [public] rd_bytes
    rd_bytes(&r, P->ext_names[i], 64);
}

    P->main_fn = (int)rd_u32(&r);

    if (!r.ok) {
        return (void *)0;
    }

    return P;
}

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 289) - [public] if
    if (!r.ok) {
        return (void *)0;
    }

    return P;
}

#ifdef CL_HOST
#include <stdio.h>

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 305) - [public] if
    if (f == (void *)0) {
        return -1;
    }

    size_t n = fread(buf, 1, cap, f);

    fclose(f);

    return (long)n;
}
#else
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 311) - [public] fclose
    fclose(f);

    return (long)n;
}
#else
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);

static long read_all(const char *path, uint8_t *buf, size_t cap)
{
    return (long)k_fs_read(path, buf, (uint32_t)cap);
}
#endif
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/loader.c (line 316) - [public] k_fs_read
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);

static long read_all(const char *path, uint8_t *buf, size_t cap)
{
    return (long)k_fs_read(path, buf, (uint32_t)cap);
}
#endif

cl_module_t *cl_load_path(const char *path, arena_t *ar, bool try_native)
{
    static uint8_t io[64 * 1024];
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 320) - [public] return
    return (long)k_fs_read(path, buf, (uint32_t)cap);
}
#endif

cl_module_t *cl_load_path(const char *path, arena_t *ar, bool try_native)
{
    static uint8_t io[64 * 1024];

    long n = read_all(path, io, sizeof(io));

    if (n <= 0) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 331) - [public] if
        if (n <= 0) {
            return (void *)0;
        }

        cl_module_t *m = (cl_module_t *)cl_arena_alloc(ar, sizeof(cl_module_t));

        if (m == (void *)0) {
            return (void *)0;
        }

        m->native = (void *)0;

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/loader.c (line 337) - [public] if
        if (m == (void *)0) {
            return (void *)0;
        }

        m->native = (void *)0;

        if (n > 5 && io[0] == 'C' && io[1] == 'L' && io[2] == 'B') {
            m->bc = cl_load_bc(io, (size_t)n, ar);
        } else {
            uint32_t el = 0;
            const char *em = (void *)0;

            // ... (truncated, see source file for more)

```



```
kernel_Workspace/kernel/src/core-lang/loader.c (line 343) - [public] if
    if (n > 5 && io[0] == 'C' && io[1] == 'L' && io[2] == 'B') {
        m->bc = cl_load_bc(io, (size_t)n, ar);
    } else {
        uint32_t el = 0;
        const char *em = (void *)0;

        m->bc = cl_compile_source((const char *)io, (size_t)n,
                                ar, &el, &em);
    }

    if (m->bc == (void *)0) {
        return (void *)0;
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/loader.c (line 353) - [public] if
    if (m->bc == (void *)0) {
        return (void *)0;
    }

    if (try_native) {
        m->native = cl_native_compile(m->bc);
    }

    return m;
}
```

```
kernel_Workspace/kernel/src/core-lang/loader.c (line 357) - [public] if
    if (try_native) {
        m->native = cl_native_compile(m->bc);
    }

    return m;
}
```

```
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 27) - [public] nb_u8
    nb_u8(d, 0x48); nb_u8(d, 0xB8); nb_u64(d, v);
}

static void push_rax(nb_t *d) { nb_u8(d, 0x50); }
static void pop_rax(nb_t *d) { nb_u8(d, 0x58); }
static void pop_rcx(nb_t *d) { nb_u8(d, 0x59); }
static void push_rcx(nb_t *d) { nb_u8(d, 0x51); }

static void load_slot(nb_t *d, uint16_t a) /* rax=[rbx+a*8]; push */
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/native_x86.c (line 30) - [internal] push_rax
static void push_rax(nb_t *d) { nb_u8(d, 0x50); }
static void pop_rax(nb_t *d) { nb_u8(d, 0x58); }
static void pop_rcx(nb_t *d) { nb_u8(d, 0x59); }
static void push_rcx(nb_t *d) { nb_u8(d, 0x51); }

static void load_slot(nb_t *d, uint16_t a) /* rax=[rbx+a*8]; push */
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_slot(nb_t *d, uint16_t a) /* pop rax; [rbx+a*8]=rax */
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 31) - [internal] pop_rax

```
static void pop_rax(nb_t *d) { nb_u8(d, 0x58); }
static void pop_rcx(nb_t *d) { nb_u8(d, 0x59); }
static void push_rcx(nb_t *d) { nb_u8(d, 0x51); }

static void load_slot(nb_t *d, uint16_t a) /* rax=[rbx+a*8]; push */
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_slot(nb_t *d, uint16_t a) /* pop rax; [rbx+a*8]=rax */
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 32) - [internal] pop_rcx

```
static void pop_rcx(nb_t *d) { nb_u8(d, 0x59); }
static void push_rcx(nb_t *d) { nb_u8(d, 0x51); }

static void load_slot(nb_t *d, uint16_t a) /* rax=[rbx+a*8]; push */
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_slot(nb_t *d, uint16_t a) /* pop rax; [rbx+a*8]=rax */
{
    pop_rax(d);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 33) - [internal] push_rcx

```
static void push_rcx(nb_t *d) { nb_u8(d, 0x51); }

static void load_slot(nb_t *d, uint16_t a) /* rax=[rbx+a*8]; push */
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_slot(nb_t *d, uint16_t a) /* pop rax; [rbx+a*8]=rax */
{
    pop_rax(d);
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 37) - [public] nb_u8

```
nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
push_rax(d);
}

static void store_slot(nb_t *d, uint16_t a) /* pop rax; [rbx+a*8]=rax */
{
    pop_rax(d);
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
}

static void load_glob(nb_t *d, uint16_t a)
{
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 38) - [public] push_rax
    push_rax(d);
}

static void store_slot(nb_t *d, uint16_t a) /* pop rax; [rbx+a*8]=rax */
{
    pop_rax(d);
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
}

static void load_glob(nb_t *d, uint16_t a)
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 43) - [public] pop_rax
    pop_rax(d);
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
}

static void load_glob(nb_t *d, uint16_t a)
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_glob(nb_t *d, uint16_t a)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 44) - [public] nb_u8
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x83); nb_u32(d, (uint32_t)a * 8);
}

static void load_glob(nb_t *d, uint16_t a)
{
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_glob(nb_t *d, uint16_t a)
{
    pop_rax(d);
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 49) - [public] nb_u8
    nb_u8(d, 0x48); nb_u8(d, 0x8B); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
    push_rax(d);
}

static void store_glob(nb_t *d, uint16_t a)
{
    pop_rax(d);
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
}

typedef struct {
    uint32_t insn_idx;
    // ... (truncated, see source file for more)
}

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 50) - [public] push_rax
    push_rax(d);
}

static void store_glob(nb_t *d, uint16_t a)
{
    pop_rax(d);
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
}

typedef struct {
    uint32_t insn_idx;
    size_t patch_off;
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 55) - [public] pop_rax
    pop_rax(d);
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
}

typedef struct {
    uint32_t insn_idx;
    size_t patch_off;
} nb_patch_t;

static uint8_t g_nb_mem[16384];

void *cl_native_compile(cl_prog_t *P)
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 56) - [public] nb_u8
    nb_u8(d, 0x48); nb_u8(d, 0x89); nb_u8(d, 0x85); nb_u32(d, (uint32_t)a * 8);
}

typedef struct {
    uint32_t insn_idx;
    size_t patch_off;
} nb_patch_t;

static uint8_t g_nb_mem[16384];

void *cl_native_compile(cl_prog_t *P)
{
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 76) - [public] nb_u8
/* prolog: rbx=slots, rbp=globals, r10=externs */

    nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xFB);
    nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xF5);
    nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xD2);

    uint32_t pc = P->fns[P->main_fn].start;

    for (;;) {
        if (pc >= P->code_len) return (void *)0;

        off_by_insn[pc] = d.len;

        cl_insn_t in = P->code[pc++];
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 77) - [public] nb_u8

```
nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xF5);
nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xD2);
```

```
uint32_t pc = P->fns[P->main_fn].start;
```

```
for (;;) {
    if (pc >= P->code_len) return (void *)0;
```

```
    off_by_insn[pc] = d.len;
```

```
    cl_insn_t in = P->code[pc++];
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 78) - [public] nb_u8

```
nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xD2);
```

```
uint32_t pc = P->fns[P->main_fn].start;
```

```
for (;;) {
    if (pc >= P->code_len) return (void *)0;
```

```
    off_by_insn[pc] = d.len;
```

```
    cl_insn_t in = P->code[pc++];
```

```
    switch (in.op) {
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 89) - [public] switch

```
switch (in.op) {
```

```
case OP_CONST: {
```

```
    uint64_t v = 0;
```

```
    for (int i = 7; i >= 0; i--) {
        v = (v << 8) | P->consts[in.a].b[i];
    }
```

```
    mov_rax_u64(&d, v);
```

```
    push_rax(&d);
```

```
    break;
```

```
}
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 97) - [public] mov_rax_u64

```
mov_rax_u64(&d, v);
```

```
push_rax(&d);
```

```
break;
```

```
}
```

```
case OP_LOAD: load_slot(&d, in.a); break;
```

```
case OP_STORE: store_slot(&d, in.a); break;
```

```
case OP_GLOAD: load_glob(&d, in.a); break;
```

```
case OP_GSTORE: store_glob(&d, in.a); break;
```

```
case OP_ADD:
```

```
    pop_rax(&d); pop_rcx(&d);
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 98) - [public] push_rax

```
    push_rax(&d);
    break;
}

case OP_LOAD: load_slot(&d, in.a); break;
case OP_STORE: store_slot(&d, in.a); break;
case OP_GLOAD: load_glob(&d, in.a); break;
case OP_GSTORE: store_glob(&d, in.a); break;

case OP_ADD:
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 102) - [public] load_slot

```
case OP_LOAD: load_slot(&d, in.a); break;
case OP_STORE: store_slot(&d, in.a); break;
case OP_GLOAD: load_glob(&d, in.a); break;
case OP_GSTORE: store_glob(&d, in.a); break;

case OP_ADD:
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
    push_rcx(&d);
    break;

case OP_SUB:
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 103) - [public] store_slot

```
case OP_STORE: store_slot(&d, in.a); break;
case OP_GLOAD: load_glob(&d, in.a); break;
case OP_GSTORE: store_glob(&d, in.a); break;

case OP_ADD:
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
    push_rcx(&d);
    break;

case OP_SUB:
    pop_rax(&d); pop_rcx(&d);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 104) - [public] load_glob

```
case OP_GLOAD: load_glob(&d, in.a); break;
case OP_GSTORE: store_glob(&d, in.a); break;

case OP_ADD:
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
    push_rcx(&d);
    break;

case OP_SUB:
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 105) - [public] store_glob
    case OP_GSTORE: store_glob(&d, in.a); break;

    case OP_ADD:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
        push_rcx(&d);
        break;

    case OP_SUB:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
        push_rcx(&d);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 108) - [public] pop_rax
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
    push_rcx(&d);
    break;

    case OP_SUB:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
        push_rcx(&d);
        break;

    case OP_MUL:
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 109) - [public] nb_u8
    nb_u8(&d, 0x48); nb_u8(&d, 0x01); nb_u8(&d, 0xC1);
    push_rcx(&d);
    break;

    case OP_SUB:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
        push_rcx(&d);
        break;

    case OP_MUL:
        pop_rax(&d); pop_rcx(&d);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 110) - [public] push_rcx
    push_rcx(&d);
    break;

    case OP_SUB:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
        push_rcx(&d);
        break;

    case OP_MUL:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x0F); nb_u8(&d, 0xAF); nb_u8(&d, 0xC8);
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 114) - [public] pop_rax
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
    push_rcx(&d);
    break;

    case OP_MUL:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x0F); nb_u8(&d, 0xAF); nb_u8(&d, 0xC8);
        push_rcx(&d);
        break;

    case OP_EXTERN: {
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 115) - [public] nb_u8
    nb_u8(&d, 0x48); nb_u8(&d, 0x29); nb_u8(&d, 0xC1);
    push_rcx(&d);
    break;

    case OP_MUL:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x0F); nb_u8(&d, 0xAF); nb_u8(&d, 0xC8);
        push_rcx(&d);
        break;

    case OP_EXTERN: {
        int n = in.b > 6 ? 6 : in.b;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 116) - [public] push_rcx
    push_rcx(&d);
    break;

    case OP_MUL:
        pop_rax(&d); pop_rcx(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x0F); nb_u8(&d, 0xAF); nb_u8(&d, 0xC8);
        push_rcx(&d);
        break;

    case OP_EXTERN: {
        int n = in.b > 6 ? 6 : in.b;

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 120) - [public] pop_rax
    pop_rax(&d); pop_rcx(&d);
    nb_u8(&d, 0x48); nb_u8(&d, 0x0F); nb_u8(&d, 0xAF); nb_u8(&d, 0xC8);
    push_rcx(&d);
    break;

    case OP_EXTERN: {
        int n = in.b > 6 ? 6 : in.b;

        for (int i = n - 1; i >= 0; i--) {
            pop_rax(&d);

            switch (i) {
// ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 121) - [public] nb_u8
    nb_u8(&d, 0x48); nb_u8(&d, 0x0F); nb_u8(&d, 0xAF); nb_u8(&d, 0xC8);
    push_rcx(&d);
    break;

    case OP_EXTERN: {
        int n = in.b > 6 ? 6 : in.b;

        for (int i = n - 1; i >= 0; i--) {
            pop_rax(&d);

            switch (i) {
                case 0: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC7); break;
            }
            // ... (truncated, see source file for more)
        }
    }

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 122) - [public] push_rcx
    push_rcx(&d);
    break;

    case OP_EXTERN: {
        int n = in.b > 6 ? 6 : in.b;

        for (int i = n - 1; i >= 0; i--) {
            pop_rax(&d);

            switch (i) {
                case 0: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC7); break;
                case 1: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC6); break;
            }
            // ... (truncated, see source file for more)
        }
    }

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 129) - [public] pop_rax
    pop_rax(&d);

    switch (i) {
        case 0: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC7); break;
        case 1: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC6); break;
        case 2: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC2); break;
        case 3: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
        case 4: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC0); break;
        case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 131) - [public] switch
    switch (i) {
        case 0: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC7); break;
        case 1: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC6); break;
        case 2: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC2); break;
        case 3: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
        case 4: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC0); break;
        case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

    /* rax = externs[a]; call rax; push rax */
    nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 132) - [public] nb_u8
    case 0: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC7); break;
    case 1: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC6); break;
    case 2: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC2); break;
    case 3: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    case 4: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC0); break;
    case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

/* rax = externs[a]; call rax; push rax */
nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
nb_u32(&d, (uint32_t)in.a * 8);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 133) - [public] nb_u8
    case 1: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC6); break;
    case 2: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC2); break;
    case 3: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    case 4: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC0); break;
    case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

/* rax = externs[a]; call rax; push rax */
nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
nb_u32(&d, (uint32_t)in.a * 8);
nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 134) - [public] nb_u8
    case 2: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC2); break;
    case 3: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    case 4: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC0); break;
    case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

/* rax = externs[a]; call rax; push rax */
nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
nb_u32(&d, (uint32_t)in.a * 8);
nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
push_rax(&d);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 135) - [public] nb_u8
    case 3: nb_u8(&d, 0x48); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    case 4: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC0); break;
    case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

/* rax = externs[a]; call rax; push rax */
nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
nb_u32(&d, (uint32_t)in.a * 8);
nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
push_rax(&d);
break;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 136) - [public] nb_u8
        case 4: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC0); break;
        case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

/* rax = externs[a]; call rax; push rax */
nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
nb_u32(&d, (uint32_t)in.a * 8);
nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
push_rax(&d);
break;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 137) - [public] nb_u8
        case 5: nb_u8(&d, 0x49); nb_u8(&d, 0x89); nb_u8(&d, 0xC1); break;
    }
}

/* rax = externs[a]; call rax; push rax */
nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
nb_u32(&d, (uint32_t)in.a * 8);
nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
push_rax(&d);
break;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 142) - [public] nb_u8
/* rax = externs[a]; call rax; push rax */
    nb_u8(&d, 0x49); nb_u8(&d, 0x8B); nb_u8(&d, 0x82);
    nb_u32(&d, (uint32_t)in.a * 8);
    nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
    push_rax(&d);
    break;
}

case OP_POP: pop_rax(&d); break;

case OP_JMP:
    nb_u8(&d, 0xE9);
    patches[npatches].insn_idx = in.c;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 143) - [public] nb_u32
    nb_u32(&d, (uint32_t)in.a * 8);
    nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
    push_rax(&d);
    break;
}

case OP_POP: pop_rax(&d); break;

case OP_JMP:
    nb_u8(&d, 0xE9);
    patches[npatches].insn_idx = in.c;
    patches[npatches].patch_off = d.len;
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 144) - [public] nb_u8

```
        nb_u8(&d, 0xFF); nb_u8(&d, 0xD0);
        push_rax(&d);
        break;
    }

    case OP_POP: pop_rax(&d); break;

    case OP_JMP:
        nb_u8(&d, 0xE9);
        patches[npatches].insn_idx = in.c;
        patches[npatches].patch_off = d.len;
        npatches++;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 145) - [public] push_rax

```
        push_rax(&d);
        break;
    }

    case OP_POP: pop_rax(&d); break;

    case OP_JMP:
        nb_u8(&d, 0xE9);
        patches[npatches].insn_idx = in.c;
        patches[npatches].patch_off = d.len;
        npatches++;
        nb_u32(&d, 0);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 149) - [public] pop_rax

```
    case OP_POP: pop_rax(&d); break;

    case OP_JMP:
        nb_u8(&d, 0xE9);
        patches[npatches].insn_idx = in.c;
        patches[npatches].patch_off = d.len;
        npatches++;
        nb_u32(&d, 0);
        break;

    case OP_JZ:
        pop_rax(&d);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 152) - [public] nb_u8

```
        nb_u8(&d, 0xE9);
        patches[npatches].insn_idx = in.c;
        patches[npatches].patch_off = d.len;
        npatches++;
        nb_u32(&d, 0);
        break;

    case OP_JZ:
        pop_rax(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x85); nb_u8(&d, 0xC0);
        nb_u8(&d, 0x0F); nb_u8(&d, 0x84);
        patches[npatches].insn_idx = in.c;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 156) - [public] nb_u32

```
        nb_u32(&d, 0);
        break;

    case OP_JZ:
        pop_rax(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x85); nb_u8(&d, 0xC0);
        nb_u8(&d, 0x0F); nb_u8(&d, 0x84);
        patches[npatches].insn_idx = in.c;
        patches[npatches].patch_off = d.len;
        npatches++;
        nb_u32(&d, 0);
        break;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 160) - [public] pop_rax

```
        pop_rax(&d);
        nb_u8(&d, 0x48); nb_u8(&d, 0x85); nb_u8(&d, 0xC0);
        nb_u8(&d, 0x0F); nb_u8(&d, 0x84);
        patches[npatches].insn_idx = in.c;
        patches[npatches].patch_off = d.len;
        npatches++;
        nb_u32(&d, 0);
        break;

    case OP_RET:
    case OP_HALT:
        nb_u8(&d, 0x48); nb_u8(&d, 0x31); nb_u8(&d, 0xC0);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 161) - [public] nb_u8

```
        nb_u8(&d, 0x48); nb_u8(&d, 0x85); nb_u8(&d, 0xC0);
        nb_u8(&d, 0x0F); nb_u8(&d, 0x84);
        patches[npatches].insn_idx = in.c;
        patches[npatches].patch_off = d.len;
        npatches++;
        nb_u32(&d, 0);
        break;

    case OP_RET:
    case OP_HALT:
        nb_u8(&d, 0x48); nb_u8(&d, 0x31); nb_u8(&d, 0xC0);
        nb_u8(&d, 0xC3);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 162) - [public] nb_u8

```
        nb_u8(&d, 0x0F); nb_u8(&d, 0x84);
        patches[npatches].insn_idx = in.c;
        patches[npatches].patch_off = d.len;
        npatches++;
        nb_u32(&d, 0);
        break;

    case OP_RET:
    case OP_HALT:
        nb_u8(&d, 0x48); nb_u8(&d, 0x31); nb_u8(&d, 0xC0);
        nb_u8(&d, 0xC3);
        goto done;
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 166) - [public] nb_u32
    nb_u32(&d, 0);
    break;

    case OP_RET:
    case OP_HALT:
        nb_u8(&d, 0x48); nb_u8(&d, 0x31); nb_u8(&d, 0xC0);
        nb_u8(&d, 0xC3);
        goto done;

    default:
        return (void *)0; /* subset v1 */
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 171) - [public] nb_u8
    nb_u8(&d, 0x48); nb_u8(&d, 0x31); nb_u8(&d, 0xC0);
    nb_u8(&d, 0xC3);
    goto done;

    default:
        return (void *)0; /* subset v1 */
    }
}

done:
for (size_t i = 0; i < npatches; i++) {
    uint32_t target = (uint32_t)off_by_insn[patches[i].insn_idx];
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 172) - [public] nb_u8
    nb_u8(&d, 0xC3);
    goto done;

    default:
        return (void *)0; /* subset v1 */
    }
}

done:
for (size_t i = 0; i < npatches; i++) {
    uint32_t target = (uint32_t)off_by_insn[patches[i].insn_idx];
    uint32_t rel = (uint32_t)(target - (patches[i].patch_off + 4));
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/native_x86.c (line 197) - [public] sizeof
    return sizeof(g_nb_mem);
}

#else /* aarch64 / riscv64 – na razie NULL, VM przejmuje */

void *cl_native_compile(cl_prog_t *P)
{
    (void)P;
    return (void *)0;
}

size_t cl_native_size(void)
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 7) - [public] if
    if (n >= cap) {
        n = cap - 1;
    }

    for (size_t i = 0; i < n; i++) {
        dst[i] = t->start[i];
    }

    dst[n] = '\\0';
}

static token_t *peek(parser_t *p)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 27) - [public] if
    if (i >= p->n) {
        i = p->n - 1;
    }

    return &p->toks[i];
}

static token_t *advance(parser_t *p)
{
    token_t *t = &p->toks[p->pos];

    if (p->pos + 1 < p->n) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 38) - [public] if
    if (p->pos + 1 < p->n) {
        p->pos++;
    }

    return t;
}

static bool check(parser_t *p, tok_kind_t k)
{
    return peek(p)->kind == k;
}

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 52) - [public] if
    if (check(p, k)) {
        advance(p);
        return true;
    }

    return false;
}

static void perr(parser_t *p, const char *m)
{
    if (!p->failed) {
        p->failed = true;
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 53) - [public] advance

```
    advance(p);
    return true;
}

return false;
}

static void perr(parser_t *p, const char *m)
{
    if (!p->failed) {
        p->failed = true;
        p->err = m;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 62) - [public] if

```
    if (!p->failed) {
        p->failed = true;
        p->err = m;
        p->err_line = peek(p)->line;
    }
}

static token_t *expect(parser_t *p, tok_kind_t k, const char *m)
{
    if (!check(p, k)) {
        perr(p, m);
        return peek(p);
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 71) - [public] if

```
    if (!check(p, k)) {
        perr(p, m);
        return peek(p);
    }

    return advance(p);
}

static ast_node_t *new_node(parser_t *p, uint8_t kind)
{
    return cl_node(p->ar, kind, peek(p)->line);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 72) - [public] perr

```
    perr(p, m);
    return peek(p);
}

return advance(p);
}

static ast_node_t *new_node(parser_t *p, uint8_t kind)
{
    return cl_node(p->ar, kind, peek(p)->line);
}

// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/core-lang/parser.c (line 73) - [public] peek

```
    return peek(p);
}

    return advance(p);
}

static ast_node_t *new_node(parser_t *p, uint8_t kind)
{
    return cl_node(p->ar, kind, peek(p)->line);
}

static ast_node_t *parse_expr(parser_t *p);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 76) - [public] advance

```
    return advance(p);
}

static ast_node_t *new_node(parser_t *p, uint8_t kind)
{
    return cl_node(p->ar, kind, peek(p)->line);
}

static ast_node_t *parse_expr(parser_t *p);
static ast_node_t *parse_block(parser_t *p);

/* ---- lista node'ów (rozmie przez arena-realloc nie jest możliwy,
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 81) - [public] cl_node

```
    return cl_node(p->ar, kind, peek(p)->line);
}

static ast_node_t *parse_expr(parser_t *p);
static ast_node_t *parse_block(parser_t *p);

/* ---- lista node'ów (rozmie przez arena-realloc nie jest możliwy,
    więc zbieramy do tymczasowej tablicy na stacku i kopiujemy) ---- */

#define LIST_MAX 128

typedef struct {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 99) - [public] if

```
    if (l->n < LIST_MAX) {
        l->v[l->n++] = x;
    } else {
        perr(p, "list too long");
    }
}

static void nl_commit(parser_t *p, nlist_t *l, ast_node_t *parent)
{
    ast_node_t **arr = (ast_node_t **)cl_arena_alloc(
        p->ar, l->n * sizeof(ast_node_t *));

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 102) - [public] perr
    perr(p, "list too long");
}
}

static void nl_commit(parser_t *p, nlist_t *l, ast_node_t *parent)
{
    ast_node_t **arr = (ast_node_t **)cl_arena_alloc(
        p->ar, l->n * sizeof(ast_node_t *));

    if (arr == (void *)0) {
        perr(p, "arena full");
        return;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 111) - [public] if
    if (arr == (void *)0) {
        perr(p, "arena full");
        return;
    }

    for (size_t i = 0; i < l->n; i++) {
        arr[i] = l->v[i];
    }

    parent->list = arr;
    parent->nlist = l->n;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 112) - [public] perr
    perr(p, "arena full");
    return;
}

    for (size_t i = 0; i < l->n; i++) {
        arr[i] = l->v[i];
    }

    parent->list = arr;
    parent->nlist = l->n;
}

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 128) - [public] if
    if (check(p, TK_TYPE)) {
        out->bits = peek(p)->type_bits;
        out->neg = false;
        advance(p);
        return true;
    }

    perr(p, "expected type (u4..u512/str)");
    return false;
}

/* ---- deklaracje ---- */
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 131) - [public] advance
    advance(p);
    return true;
}

    perr(p, "expected type (u4..u512/str)");
    return false;
}

/* ---- deklaracje ---- */

static ast_node_t *parse_let(parser_t *p)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/parser.c (line 135) - [public] perr
    perr(p, "expected type (u4..u512/str)");
    return false;
}

/* ---- deklaracje ---- */

static ast_node_t *parse_let(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_LET);

    expect(p, TK_LET, "let");

    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/parser.c (line 145) - [public] expect
    expect(p, TK_LET, "let");

    parse_type(p, &n->type);

    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "let name"));

    expect(p, TK_ASSIGN, "let =");

    n->a = parse_expr(p);

    expect(p, TK_SEMI, "let ;");

    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/parser.c (line 147) - [public] parse_type
    parse_type(p, &n->type);

    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "let name"));

    expect(p, TK_ASSIGN, "let =");

    n->a = parse_expr(p);

    expect(p, TK_SEMI, "let ;");

    /* opcjonalny zakres ujemny: _=>-1; */
    if (match(p, TK_UNDERSCORE)) {
        // ... (truncated, see source file for more)
    }
}

```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 149) - [public] copy_name
    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "let name"));

    expect(p, TK_ASSIGN, "let =");

    n->a = parse_expr(p);

    expect(p, TK_SEMI, "let ;");

    /* opcjonalny zakres ujemny: _=>-1; */
    if (match(p, TK_UNDERSCORE)) {
        expect(p, TK_ARROW, "_=>");
        ast_node_t *r = parse_expr(p);
        // ... (truncated, see source file for more)
    }
}
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 151) - [public] expect
    expect(p, TK_ASSIGN, "let =");

    n->a = parse_expr(p);

    expect(p, TK_SEMI, "let ;");

    /* opcjonalny zakres ujemny: _=>-1; */
    if (match(p, TK_UNDERSCORE)) {
        expect(p, TK_ARROW, "_=>");
        ast_node_t *r = parse_expr(p);
        expect(p, TK_SEMI, "range ;");

        // ... (truncated, see source file for more)
    }
}
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 158) - [public] if
    /* opcjonalny zakres ujemny: _=>-1; */
    if (match(p, TK_UNDERSCORE)) {
        expect(p, TK_ARROW, "_=>");
        ast_node_t *r = parse_expr(p);
        expect(p, TK_SEMI, "range ;");

        n->type.neg = true;
        n->b = r;
    }

    return n;
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 159) - [public] expect
    expect(p, TK_ARROW, "_=>");
    ast_node_t *r = parse_expr(p);
    expect(p, TK_SEMI, "range ;");

    n->type.neg = true;
    n->b = r;
}

return n;
}

static ast_node_t *parse_static(parser_t *p)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 174) - [public] expect

```
expect(p, TK_STATIC, "static");
expect(p, TK_DATA, "static data");
expect(p, TK_LBRACK, "[");

parse_type(p, &n->type);

expect(p, TK_COLON, ":");

n->arr_count = expect(p, TK_NUM, "count")->num;

expect(p, TK_SEMI, ";");

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 175) - [public] expect

```
expect(p, TK_DATA, "static data");
expect(p, TK_LBRACK, "[");

parse_type(p, &n->type);

expect(p, TK_COLON, ":");

n->arr_count = expect(p, TK_NUM, "count")->num;

expect(p, TK_SEMI, ";");

n->arr_init = expect(p, TK_NUM, "init")->num;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 176) - [public] expect

```
expect(p, TK_LBRACK, "[");

parse_type(p, &n->type);

expect(p, TK_COLON, ":");

n->arr_count = expect(p, TK_NUM, "count")->num;

expect(p, TK_SEMI, ";");

n->arr_init = expect(p, TK_NUM, "init")->num;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 178) - [public] parse_type

```
parse_type(p, &n->type);

expect(p, TK_COLON, ":");

n->arr_count = expect(p, TK_NUM, "count")->num;

expect(p, TK_SEMI, ";");

n->arr_init = expect(p, TK_NUM, "init")->num;

expect(p, TK_RBRACK, "]);
expect(p, TK_SEMI, "static ;");
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 180) - [public] expect
    expect(p, TK_COLON, ":");

    n->arr_count = expect(p, TK_NUM, "count")->num;

    expect(p, TK_SEMI, ";");

    n->arr_init = expect(p, TK_NUM, "init")->num;

    expect(p, TK_RBRACK, "]");
    expect(p, TK_SEMI, "static ;");

    n->is_array = true;
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 188) - [public] expect
    expect(p, TK_RBRACK, "]");
    expect(p, TK_SEMI, "static ;");

    n->is_array = true;

    return n;
}

static ast_node_t *parse_fn(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_FN);

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 200) - [public] expect
    expect(p, TK_FN, "fn");

    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "fn name"));

    expect(p, TK_LPAREN, "fn (");

    nlist_t params = { { 0 }, 0 };

    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        ast_node_t *arg = new_node(p, ND_LET);

        parse_type(p, &arg->type);
        // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 202) - [public] copy_name
    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "fn name"));

    expect(p, TK_LPAREN, "fn (");

    nlist_t params = { { 0 }, 0 };

    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        ast_node_t *arg = new_node(p, ND_LET);

        parse_type(p, &arg->type);
        copy_name(arg->name, sizeof(arg->name),
            expect(p, TK_IDENT, "param name"));
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 204) - [public] expect
    expect(p, TK_LPAREN, "fn (");

    nlist_t params = { { 0 }, 0 };

    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        ast_node_t *arg = new_node(p, ND_LET);

        parse_type(p, &arg->type);
        copy_name(arg->name, sizeof(arg->name),
            expect(p, TK_IDENT, "param name"));

        nl_push(p, &params, arg);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 208) - [public] while
    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        ast_node_t *arg = new_node(p, ND_LET);

        parse_type(p, &arg->type);
        copy_name(arg->name, sizeof(arg->name),
            expect(p, TK_IDENT, "param name"));

        nl_push(p, &params, arg);

        if (!match(p, TK_COMMA)) {
            break;
        }
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 211) - [public] parse_type
    parse_type(p, &arg->type);
    copy_name(arg->name, sizeof(arg->name),
        expect(p, TK_IDENT, "param name"));

    nl_push(p, &params, arg);

    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RPAREN, "fn ");
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 213) - [public] expect
    expect(p, TK_IDENT, "param name"));

    nl_push(p, &params, arg);

    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RPAREN, "fn ");

nl_commit(p, &params, n);
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 215) - [public] nl_push
    nl_push(p, &params, arg);

    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RPAREN, "fn ");

nl_commit(p, &params, n);

n->a = parse_block(p);
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 217) - [public] if
    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RPAREN, "fn ");

nl_commit(p, &params, n);

n->a = parse_block(p);

return n;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 222) - [public] expect
    expect(p, TK_RPAREN, "fn ");

    nl_commit(p, &params, n);

    n->a = parse_block(p);

    return n;
}

static ast_node_t *parse_enum(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_ENUM);
    // ... (truncated, see source file for more)
}

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 224) - [public] nl_commit
    nl_commit(p, &params, n);

    n->a = parse_block(p);

    return n;
}

static ast_node_t *parse_enum(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_ENUM);

    expect(p, TK_ENUM, "enum");
    // ... (truncated, see source file for more)
}

```



```

kernel_Workspace/kernel/src/core-lang/parser.c (line 235) - [public] expect
    expect(p, TK_ENUM, "enum");

    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "enum name"));

    expect(p, TK_LBRACE, "enum {");

    nlist_t items = { { 0 }, 0 };
    uint64_t val = 0;

    while (!check(p, TK_RBACE) && !check(p, TK_EOF)) {
        ast_node_t *it = new_node(p, ND_VAR);

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 237) - [public] copy_name
    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "enum name"));

    expect(p, TK_LBRACE, "enum {");

    nlist_t items = { { 0 }, 0 };
    uint64_t val = 0;

    while (!check(p, TK_RBACE) && !check(p, TK_EOF)) {
        ast_node_t *it = new_node(p, ND_VAR);

        copy_name(it->name, sizeof(it->name),
            expect(p, TK_IDENT, "enum item"));
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 244) - [public] while
    while (!check(p, TK_RBACE) && !check(p, TK_EOF)) {
        ast_node_t *it = new_node(p, ND_VAR);

        copy_name(it->name, sizeof(it->name),
            expect(p, TK_IDENT, "enum item"));

        it->value = val++;

        nl_push(p, &items, it);

        if (!match(p, TK_COMMA)) {
            break;
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 248) - [public] expect
        expect(p, TK_IDENT, "enum item"));

        it->value = val++;

        nl_push(p, &items, it);

        if (!match(p, TK_COMMA)) {
            break;
        }
    }

    expect(p, TK_RBACE, "enum }");
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 252) - [public] nl_push
    nl_push(p, &items, it);

    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RBRACE, "enum ");

nl_commit(p, &items, n);

return n;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 254) - [public] if
    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RBRACE, "enum ");

nl_commit(p, &items, n);

return n;
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 259) - [public] expect
    expect(p, TK_RBRACE, "enum ");

    nl_commit(p, &items, n);

    return n;
}

static ast_node_t *parse_extern(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_EXTERN);

    expect(p, TK_EXTERN, "extern");
    // ... (truncated, see source file for more)
}

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 261) - [public] nl_commit
    nl_commit(p, &items, n);

    return n;
}

static ast_node_t *parse_extern(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_EXTERN);

    expect(p, TK_EXTERN, "extern");

    token_t *abi = expect(p, TK_CHAR, "extern 'C'");
    // ... (truncated, see source file for more)
}

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 270) - [public] expect
    expect(p, TK_EXTERN, "extern");

    token_t *abi = expect(p, TK_CHAR, "extern 'C'");

    n->name[0] = (char)abi->num;
    n->name[1] = '\0';

    expect(p, TK_LBRACE, "extern {");

    nlist_t fns = { { 0 }, 0 };

    while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 281) - [public] while
    while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
        ast_node_t *f = new_node(p, ND_VAR);

        copy_name(f->name, sizeof(f->name),
            expect(p, TK_IDENT, "extern fn"));

        nl_push(p, &fns, f);

        if (!match(p, TK_COMMA)) {
            break;
        }
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 285) - [public] expect
        expect(p, TK_IDENT, "extern fn"));

        nl_push(p, &fns, f);

        if (!match(p, TK_COMMA)) {
            break;
        }
    }

    expect(p, TK_RBRACE, "extern }");

    nl_commit(p, &fns, n);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 287) - [public] nl_push
        nl_push(p, &fns, f);

        if (!match(p, TK_COMMA)) {
            break;
        }
    }

    expect(p, TK_RBRACE, "extern }");

    nl_commit(p, &fns, n);

    return n;
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 289) - [public] if
    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RBRACE, "extern ");

nl_commit(p, &fns, n);

return n;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 294) - [public] expect
expect(p, TK_RBRACE, "extern ");

nl_commit(p, &fns, n);

return n;
}

static ast_node_t *parse_dataimpl(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_DATAIMPL);

    expect(p, TK_DATAIMPL, "dataimpl");
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 296) - [public] nl_commit
    nl_commit(p, &fns, n);

    return n;
}

static ast_node_t *parse_dataimpl(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_DATAIMPL);

    expect(p, TK_DATAIMPL, "dataimpl");

    expect(p, TK_LBRACE, "dataimpl {");
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 305) - [public] expect
    expect(p, TK_DATAIMPL, "dataimpl");

    expect(p, TK_LBRACE, "dataimpl {");

    /* baza: data: ... a■ do } */
    nlist_t base = { { 0 }, 0 };

    while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
        if (match(p, TK_DATA)) {
            expect(p, TK_COLON, "data:");
            continue;
        }
    }
    // ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 312) - [public] while
    while (!check(p, TK_RBACE) && !check(p, TK_EOF)) {
        if (match(p, TK_DATA)) {
            expect(p, TK_COLON, "data:");
            continue;
        }

        nl_push(p, &base, parse_expr(p));

        if (!match(p, TK_COMMA)) {
            break;
        }
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 313) - [public] if
    if (match(p, TK_DATA)) {
        expect(p, TK_COLON, "data:");
        continue;
    }

    nl_push(p, &base, parse_expr(p));

    if (!match(p, TK_COMMA)) {
        break;
    }
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 314) - [public] expect
    expect(p, TK_COLON, "data:");
    continue;
}

    nl_push(p, &base, parse_expr(p));

    if (!match(p, TK_COMMA)) {
        break;
    }
}

    expect(p, TK_RBACE, "dataimpl }");
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 318) - [public] nl_push
    nl_push(p, &base, parse_expr(p));

    if (!match(p, TK_COMMA)) {
        break;
    }
}

    expect(p, TK_RBACE, "dataimpl }");

    expect(p, TK_ARROW, "=>");

    ast_node_t *baseblk = new_node(p, ND_BLOCK);
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 320) - [public] if
    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RBRACE, "dataimpl }");

expect(p, TK_ARROW, "=>");

ast_node_t *baseblk = new_node(p, ND_BLOCK);
nl_commit(p, &base, baseblk);

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 325) - [public] expect
expect(p, TK_RBRACE, "dataimpl }");

expect(p, TK_ARROW, "=>");

ast_node_t *baseblk = new_node(p, ND_BLOCK);
nl_commit(p, &base, baseblk);

n->a = baseblk;
n->b = parse_block(p);

return n;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 327) - [public] expect
expect(p, TK_ARROW, "=>");

ast_node_t *baseblk = new_node(p, ND_BLOCK);
nl_commit(p, &base, baseblk);

n->a = baseblk;
n->b = parse_block(p);

return n;
}

/* ---- statementy ---- */
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 330) - [public] nl_commit
    nl_commit(p, &base, baseblk);

    n->a = baseblk;
    n->b = parse_block(p);

    return n;
}

/* ---- statementy ---- */

static ast_node_t *parse_block(parser_t *p)
{
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 348) - [public] while

```
while (!check(p, TK_RBACE) && !check(p, TK_EOF)) {
    ast_node_t *s = (void *)0;

    switch (peek(p)->kind) {
    case TK_LET:      s = parse_let(p); break;
    case TK_STATIC:   s = parse_static(p); break;
    case TK_IF:       s = (void *)1; break; /* ni■ej */
    case TK_WHEN:     s = (void *)1; break;
    case TK_SET_FREE: {
        s = new_node(p, ND_SET_FREE);
        advance(p);
        expect(p, TK_LBRACE, "set_free {");
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 351) - [public] switch

```
switch (peek(p)->kind) {
case TK_LET:      s = parse_let(p); break;
case TK_STATIC:   s = parse_static(p); break;
case TK_IF:       s = (void *)1; break; /* ni■ej */
case TK_WHEN:     s = (void *)1; break;
case TK_SET_FREE: {
    s = new_node(p, ND_SET_FREE);
    advance(p);
    expect(p, TK_LBRACE, "set_free {");
    copy_name(s->name, sizeof(s->name),
               expect(p, TK_IDENT, "set_free var"));
    expect(p, TK_RBACE, "set_free }");
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 358) - [public] advance

```
advance(p);
expect(p, TK_LBRACE, "set_free {");
copy_name(s->name, sizeof(s->name),
           expect(p, TK_IDENT, "set_free var"));
expect(p, TK_RBACE, "set_free }");
expect(p, TK_SEMI, "set_free ;");
break;
}
default:
    s = (void *)1;
    break;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 361) - [public] expect

```
expect(p, TK_IDENT, "set_free var"));
expect(p, TK_RBACE, "set_free }");
expect(p, TK_SEMI, "set_free ;");
break;
}
default:
    s = (void *)1;
    break;
}

if (s == (void *)1) {
    if (check(p, TK_IF)) {
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 362) - [public] expect
    expect(p, TK_RBRACE, "set_free }");
    expect(p, TK_SEMI, "set_free ;");
    break;
}
default:
    s = (void *)1;
    break;
}

    if (s == (void *)1) {
        if (check(p, TK_IF)) {
            extern ast_node_t *cl_parse_if(parser_t *p);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 371) - [public] if
    if (s == (void *)1) {
        if (check(p, TK_IF)) {
            extern ast_node_t *cl_parse_if(parser_t *p);
            s = cl_parse_if(p);
        } else if (check(p, TK_WHEN)) {
            extern ast_node_t *cl_parse_when(parser_t *p);
            s = cl_parse_when(p);
        } else {
            s = parse_expr(p);
            expect(p, TK_SEMI, "stmt ;");
        }
    }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 372) - [public] if
    if (check(p, TK_IF)) {
        extern ast_node_t *cl_parse_if(parser_t *p);
        s = cl_parse_if(p);
    } else if (check(p, TK_WHEN)) {
        extern ast_node_t *cl_parse_when(parser_t *p);
        s = cl_parse_when(p);
    } else {
        s = parse_expr(p);
        expect(p, TK_SEMI, "stmt ;");
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 384) - [public] nl_push
    nl_push(p, &stmts, s);
}

    expect(p, TK_RBRACE, "}");

    nl_commit(p, &stmts, n);

    return n;
}

ast_node_t *cl_parse_if(parser_t *p)
{
    // ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/parser.c (line 387) - [public] expect
    expect(p, TK_RBRACE, "}");

    nl_commit(p, &stmts, n);

    return n;
}

ast_node_t *cl_parse_if(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_IF);

    expect(p, TK_IF, "if");
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 389) - [public] nl_commit
    nl_commit(p, &stmts, n);

    return n;
}

ast_node_t *cl_parse_if(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_IF);

    expect(p, TK_IF, "if");

    if (!check(p, TK_LBRACE)) {
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 398) - [public] expect
    expect(p, TK_IF, "if");

    if (!check(p, TK_LBRACE)) {
        n->a = parse_expr(p);
    }

    n->b = parse_block(p);

    /* else / else if / if else */
    if (match(p, TK_ELSE)) {
        if (check(p, TK_IF) && peek_at(p, 1)->kind != TK_ELSE) {
            advance(p);
        }
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 400) - [public] if
    if (!check(p, TK_LBRACE)) {
        n->a = parse_expr(p);
    }

    n->b = parse_block(p);

    /* else / else if / if else */
    if (match(p, TK_ELSE)) {
        if (check(p, TK_IF) && peek_at(p, 1)->kind != TK_ELSE) {
            advance(p);
        }

        ast_node_t *inner = new_node(p, ND_IF);
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 407) - [public] if

/ else / else if / if else */*

```
    if (match(p, TK_ELSE)) {
        if (check(p, TK_IF) && peek_at(p, 1)->kind != TK_ELSE) {
            advance(p);

            ast_node_t *inner = new_node(p, ND_IF);

            if (!check(p, TK_LBRACE)) {
                inner->a = parse_expr(p);
            }

            inner->b = parse_block(p);
            n->c = inner;
        }
        // ... (truncated, see source file for more)
    }
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 408) - [public] if

```
    if (check(p, TK_IF) && peek_at(p, 1)->kind != TK_ELSE) {
        advance(p);

        ast_node_t *inner = new_node(p, ND_IF);

        if (!check(p, TK_LBRACE)) {
            inner->a = parse_expr(p);
        }

        inner->b = parse_block(p);
        n->c = inner;
    } else {
        // ... (truncated, see source file for more)
    }
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 409) - [public] advance

```
    advance(p);

    ast_node_t *inner = new_node(p, ND_IF);

    if (!check(p, TK_LBRACE)) {
        inner->a = parse_expr(p);
    }

    inner->b = parse_block(p);
    n->c = inner;
} else {
    n->c = parse_block(p);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 413) - [public] if

```
    if (!check(p, TK_LBRACE)) {
        inner->a = parse_expr(p);
    }

    inner->b = parse_block(p);
    n->c = inner;
} else {
    n->c = parse_block(p);
}
} else if (check(p, TK_IF) && peek_at(p, 1)->kind == TK_ELSE) {
    advance(p);
    advance(p);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 423) - [public] advance

```
    advance(p);
    advance(p);
    n->c = parse_block(p);
}

return n;
}

ast_node_t *cl_parse_when(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_WHEN);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 424) - [public] advance

```
    advance(p);
    n->c = parse_block(p);
}

return n;
}

ast_node_t *cl_parse_when(parser_t *p)
{
    ast_node_t *n = new_node(p, ND_WHEN);

    expect(p, TK_WHEN, "when");
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 435) - [public] expect

```
    expect(p, TK_WHEN, "when");

    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "when target"));

    expect(p, TK_EQCOLON, "=:");
    expect(p, TK_LBRACE, "when {");

    nlist_t cases = { { 0 }, 0 };

    while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
        ast_node_t *c = new_node(p, ND_CASE);

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 437) - [public] copy_name

```
    copy_name(n->name, sizeof(n->name), expect(p, TK_IDENT, "when target"));

    expect(p, TK_EQCOLON, "=:");
    expect(p, TK_LBRACE, "when {");

    nlist_t cases = { { 0 }, 0 };

    while (!check(p, TK_RBRACE) && !check(p, TK_EOF)) {
        ast_node_t *c = new_node(p, ND_CASE);

        c->a = parse_expr(p);
        c->b = parse_block(p);
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 439) - [public] expect
    expect(p, TK_EQCOLON, "=:");
    expect(p, TK_LBRACE, "when {");

    nlist_t cases = { { 0 }, 0 };

    while (!check(p, TK_RBACE) && !check(p, TK_EOF)) {
        ast_node_t *c = new_node(p, ND_CASE);

        c->a = parse_expr(p);
        c->b = parse_block(p);

        nl_push(p, &cases, c);
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 444) - [public] while
    while (!check(p, TK_RBACE) && !check(p, TK_EOF)) {
        ast_node_t *c = new_node(p, ND_CASE);

        c->a = parse_expr(p);
        c->b = parse_block(p);

        nl_push(p, &cases, c);
    }

    expect(p, TK_RBACE, "when }");

    nl_commit(p, &cases, n);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 450) - [public] nl_push
    nl_push(p, &cases, c);
}

    expect(p, TK_RBACE, "when }");

    nl_commit(p, &cases, n);

    return n;
}

/* ---- wyrażenia ---- */

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 453) - [public] expect
    expect(p, TK_RBACE, "when }");

    nl_commit(p, &cases, n);

    return n;
}

/* ---- wyrażenia ---- */

static ast_node_t *parse_primary(parser_t *p)
{
    ast_node_t *n;
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 455) - [public] nl_commit
    nl_commit(p, &cases, n);

    return n;
}

/* ---- wyrażenia ---- */

static ast_node_t *parse_primary(parser_t *p)
{
    ast_node_t *n;

    if (check(p, TK_NUM)) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 466) - [public] if
        if (check(p, TK_NUM)) {
            n = new_node(p, ND_NUM);
            n->value = advance(p)->num;
            return n;
        }

        if (check(p, TK_STR)) {
            n = new_node(p, ND_STR);
            token_t *t = advance(p);
            copy_name(n->name, sizeof(n->name), t);
            return n;
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 472) - [public] if
        if (check(p, TK_STR)) {
            n = new_node(p, ND_STR);
            token_t *t = advance(p);
            copy_name(n->name, sizeof(n->name), t);
            return n;
        }

        if (check(p, TK_CHAR)) {
            n = new_node(p, ND_CHAR);
            n->value = advance(p)->num;
            return n;
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 475) - [public] copy_name
        copy_name(n->name, sizeof(n->name), t);
        return n;
    }

    if (check(p, TK_CHAR)) {
        n = new_node(p, ND_CHAR);
        n->value = advance(p)->num;
        return n;
    }

    if (check(p, TK_DOLLAR) || check(p, TK_DOLLAR_BANG) ||
        check(p, TK_DOLLAR_AT)) {
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 479) - [public] if

```
if (check(p, TK_CHAR)) {
    n = new_node(p, ND_CHAR);
    n->value = advance(p)->num;
    return n;
}

if (check(p, TK_DOLLAR) || check(p, TK_DOLLAR_BANG) ||
    check(p, TK_DOLLAR_AT)) {
    n = new_node(p, ND_POOL);
    token_t *t = advance(p);

    n->pool_kind = (t->kind == TK_DOLLAR) ? 0
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 486) - [public] check

```
check(p, TK_DOLLAR_AT)) {
    n = new_node(p, ND_POOL);
    token_t *t = advance(p);

    n->pool_kind = (t->kind == TK_DOLLAR) ? 0
                  : (t->kind == TK_DOLLAR_BANG) ? 1 : 2;

    copy_name(n->name, sizeof(n->name),
              expect(p, TK_IDENT, "$var"));
    return n;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 494) - [public] expect

```
expect(p, TK_IDENT, "$var"));
return n;
}

if (check(p, TK_CALL) && peek_at(p, 1)->kind == TK_LBRACK) {
    n = new_node(p, ND_CALLIDX);
    advance(p);
    expect(p, TK_LBRACK, "call[");
    copy_name(n->name, sizeof(n->name),
              expect(p, TK_IDENT, "call[x]"));
    expect(p, TK_RBRACK, "]);
    return n;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 498) - [public] if

```
if (check(p, TK_CALL) && peek_at(p, 1)->kind == TK_LBRACK) {
    n = new_node(p, ND_CALLIDX);
    advance(p);
    expect(p, TK_LBRACK, "call[");
    copy_name(n->name, sizeof(n->name),
              expect(p, TK_IDENT, "call[x]"));
    expect(p, TK_RBRACK, "]);
    return n;
}

if (match(p, TK_LPAREN)) {
    n = parse_expr(p);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 500) - [public] advance

```
    advance(p);
    expect(p, TK_LBRACK, "call[");
    copy_name(n->name, sizeof(n->name),
              expect(p, TK_IDENT, "call[x]"));
    expect(p, TK_RBRACK, "]);");
    return n;
}

if (match(p, TK_LPAREN)) {
    n = parse_expr(p);
    expect(p, TK_RPAREN, "));");
    return n;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 501) - [public] expect

```
    expect(p, TK_LBRACK, "call[");
    copy_name(n->name, sizeof(n->name),
              expect(p, TK_IDENT, "call[x]"));
    expect(p, TK_RBRACK, "]);");
    return n;
}

if (match(p, TK_LPAREN)) {
    n = parse_expr(p);
    expect(p, TK_RPAREN, "));");
    return n;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 503) - [public] expect

```
    expect(p, TK_IDENT, "call[x]"));
    expect(p, TK_RBRACK, "]);");
    return n;
}

if (match(p, TK_LPAREN)) {
    n = parse_expr(p);
    expect(p, TK_RPAREN, "));");
    return n;
}

if (check(p, TK_IDENT)) {
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 504) - [public] expect

```
    expect(p, TK_RBRACK, "]);");
    return n;
}

if (match(p, TK_LPAREN)) {
    n = parse_expr(p);
    expect(p, TK_RPAREN, "));");
    return n;
}

if (check(p, TK_IDENT)) {
    n = new_node(p, ND_VAR);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 508) - [public] if

```
    if (match(p, TK_LPAREN)) {
        n = parse_expr(p);
        expect(p, TK_RPAREN, "");
        return n;
    }

    if (check(p, TK_IDENT)) {
        n = new_node(p, ND_VAR);
        copy_name(n->name, sizeof(n->name), advance(p));
        return n;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 510) - [public] expect

```
    expect(p, TK_RPAREN, "");
    return n;
}

if (check(p, TK_IDENT)) {
    n = new_node(p, ND_VAR);
    copy_name(n->name, sizeof(n->name), advance(p));
    return n;
}

perr(p, "unexpected token in expr");
advance(p);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 514) - [public] if

```
    if (check(p, TK_IDENT)) {
        n = new_node(p, ND_VAR);
        copy_name(n->name, sizeof(n->name), advance(p));
        return n;
    }

    perr(p, "unexpected token in expr");
    advance(p);

    return new_node(p, ND_NUM);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 516) - [public] copy_name

```
    copy_name(n->name, sizeof(n->name), advance(p));
    return n;
}

perr(p, "unexpected token in expr");
advance(p);

return new_node(p, ND_NUM);
}

static ast_node_t *parse_postfix(parser_t *p)
{
    // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/core-lang/parser.c (line 520) - [public] perr

```
perr(p, "unexpected token in expr");
advance(p);

return new_node(p, ND_NUM);
}

static ast_node_t *parse_postfix(parser_t *p)
{
    ast_node_t *n = parse_primary(p);

    for (;;) {
        if (match(p, TK_DOT)) {
            // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 521) - [public] advance

```
advance(p);

return new_node(p, ND_NUM);
}

static ast_node_t *parse_postfix(parser_t *p)
{
    ast_node_t *n = parse_primary(p);

    for (;;) {
        if (match(p, TK_DOT)) {
            ast_node_t *m = new_node(p, ND_MEMBER);
            // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 523) - [public] new_node

```
return new_node(p, ND_NUM);
}

static ast_node_t *parse_postfix(parser_t *p)
{
    ast_node_t *n = parse_primary(p);

    for (;;) {
        if (match(p, TK_DOT)) {
            ast_node_t *m = new_node(p, ND_MEMBER);

            m->a = n;
            // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 531) - [public] if

```
if (match(p, TK_DOT)) {
    ast_node_t *m = new_node(p, ND_MEMBER);

    m->a = n;
    copy_name(m->name, sizeof(m->name),
              expect(p, TK_IDENT, "member"));
    n = m;
} else if (check(p, TK_DCOLON)) {
    advance(p);

    ast_node_t *m = new_node(p, ND_SCOPECALL);

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 536) - [public] expect
        expect(p, TK_IDENT, "member"));
        n = m;
    } else if (check(p, TK_DCOLON)) {
        advance(p);

        ast_node_t *m = new_node(p, ND_SCOPECALL);

        for (size_t i = 0; i < sizeof(m->name); i++) {
            m->name[i] = 0;
        }

        if (n->kind == ND_VAR) {
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 539) - [public] advance
        advance(p);

        ast_node_t *m = new_node(p, ND_SCOPECALL);

        for (size_t i = 0; i < sizeof(m->name); i++) {
            m->name[i] = 0;
        }

        if (n->kind == ND_VAR) {
            for (size_t i = 0; n->name[i] && i < 63; i++) {
                m->name[i] = n->name[i];
            }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 547) - [public] if
        if (n->kind == ND_VAR) {
            for (size_t i = 0; n->name[i] && i < 63; i++) {
                m->name[i] = n->name[i];
            }
        }

        copy_name(m->name2, sizeof(m->name2),
            expect(p, TK_IDENT, "scope fn"));

        expect(p, TK_LPAREN, "scope (");

        nlist_t args = { { 0 }, 0 };
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 554) - [public] expect
        expect(p, TK_IDENT, "scope fn"));

        expect(p, TK_LPAREN, "scope (");

        nlist_t args = { { 0 }, 0 };

        while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
            nl_push(p, &args, parse_expr(p));

            if (!match(p, TK_COMMA)) {
                break;
            }
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 556) - [public] expect
    expect(p, TK_LPAREN, "scope (");

    nlist_t args = { { 0 }, 0 };

    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        nl_push(p, &args, parse_expr(p));

        if (!match(p, TK_COMMA)) {
            break;
        }
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 560) - [public] while
    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        nl_push(p, &args, parse_expr(p));

        if (!match(p, TK_COMMA)) {
            break;
        }
    }

    expect(p, TK_RPAREN, "::");

    nl_commit(p, &args, m);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 561) - [public] nl_push
    nl_push(p, &args, parse_expr(p));

    if (!match(p, TK_COMMA)) {
        break;
    }
}

    expect(p, TK_RPAREN, "::");

    nl_commit(p, &args, m);

    n = m;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 563) - [public] if
    if (!match(p, TK_COMMA)) {
        break;
    }
}

    expect(p, TK_RPAREN, "::");

    nl_commit(p, &args, m);

    n = m;
} else if (check(p, TK_LPAREN)) {
    advance(p);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 568) - [public] expect
    expect(p, TK_RPAREN, "::-");

    nl_commit(p, &args, m);

    n = m;
} else if (check(p, TK_LPAREN)) {
    advance(p);

    ast_node_t *m = new_node(p, ND_CALL);

    m->a = n;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 570) - [public] nl_commit
    nl_commit(p, &args, m);

    n = m;
} else if (check(p, TK_LPAREN)) {
    advance(p);

    ast_node_t *m = new_node(p, ND_CALL);

    m->a = n;

    nlist_t args = { { 0 }, 0 };

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 574) - [public] advance
    advance(p);

    ast_node_t *m = new_node(p, ND_CALL);

    m->a = n;

    nlist_t args = { { 0 }, 0 };

    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        nl_push(p, &args, parse_expr(p));

        if (!match(p, TK_COMMA)) {
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 582) - [public] while
    while (!check(p, TK_RPAREN) && !check(p, TK_EOF)) {
        nl_push(p, &args, parse_expr(p));

        if (!match(p, TK_COMMA)) {
            break;
        }
    }

    expect(p, TK_RPAREN, "call ");

    nl_commit(p, &args, m);

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 583) - [public] nl_push
    nl_push(p, &args, parse_expr(p));

    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RPAREN, "call ");

nl_commit(p, &args, m);

n = m;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 585) - [public] if
    if (!match(p, TK_COMMA)) {
        break;
    }
}

expect(p, TK_RPAREN, "call ");

nl_commit(p, &args, m);

n = m;
} else {
    break;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 590) - [public] expect
    expect(p, TK_RPAREN, "call ");

    nl_commit(p, &args, m);

    n = m;
} else {
    break;
}

return n;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 592) - [public] nl_commit
    nl_commit(p, &args, m);

    n = m;
} else {
    break;
}

return n;
}

static ast_node_t *parse_unary(parser_t *p)
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 606) - [public] check

```
    check(p, TK_TILDE)) {
        ast_node_t *n = new_node(p, ND_UN);

        n->op = advance(p)->kind;
        n->a = parse_unary(p);

        return n;
    }

    return parse_postfix(p);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 615) - [public] parse_postfix

```
    return parse_postfix(p);
}

static ast_node_t *parse_bin(parser_t *p, int level)
{
    ast_node_t *lhs = parse_unary(p);

    for (;;) {
        tok_kind_t k = peek(p)->kind;
        int lv = 0;

        switch (k) {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 626) - [public] switch

```
        switch (k) {
            case TK_STAR: case TK_SLASH: case TK_PERCENT: lv = 1; break;
            case TK_PLUS: case TK_MINUS: lv = 2; break;
            case TK_SHL: case TK_SHR: lv = 3; break;
            case TK_LT: case TK_GT: case TK_LEQ: case TK_GEQ: lv = 4; break;
            case TK_EQ: case TK_NEQ: lv = 5; break;
            case TK_AMP: lv = 6; break;
            case TK_CARET: lv = 7; break;
            case TK_PIPE: lv = 8; break;
            default: return lhs;
        }

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 638) - [public] if

```
        if (lv > level) {
            return lhs;
        }

        advance(p);

        ast_node_t *rhs = parse_bin(p, lv + 1 > 8 ? 8 : lv + 1);

        ast_node_t *b = new_node(p, ND_BIN);

        b->op = k;
        b->a = lhs;
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 642) - [public] advance
    advance(p);

    ast_node_t *rhs = parse_bin(p, lv + 1 > 8 ? 8 : lv + 1);

    ast_node_t *b = new_node(p, ND_BIN);

    b->op = k;
    b->a = lhs;
    b->b = rhs;

    lhs = b;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 662) - [public] if
    if (match(p, TK_ASSIGN)) {
        ast_node_t *n = new_node(p, ND_ASSIGN);

        n->a = lhs;
        n->b = parse_assign(p);

        return n;
    }

    return lhs;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 676) - [public] parse_assign
    return parse_assign(p);
}

/* ---- top ---- */

ast_node_t *cl_parse(parser_t *p, token_t *toks, size_t n, arena_t *ar)
{
    p->toks = toks;
    p->n = n;
    p->pos = 0;
    p->ar = ar;
    p->failed = false;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/parser.c (line 694) - [public] while
    while (!check(p, TK_EOF)) {
        ast_node_t *d;

        switch (peek(p)->kind) {
            case TK_FN:      d = parse_fn(p); break;
            case TK_LET:     d = parse_let(p); break;
            case TK_STATIC:  d = parse_static(p); break;
            case TK_ENUM:    d = parse_enum(p); break;
            case TK_EXTERN:  d = parse_extern(p); break;
            case TK_DATAIMPL: d = parse_dataimpl(p); break;
            default:
                perr(p, "expected top-level decl");
        }
        // ... (truncated, see source file for more)
    }
}

```

kernel_Workspace/kernel/src/core-lang/parser.c (line 697) - [public] switch

```
    switch (peek(p)->kind) {
    case TK_FN:      d = parse_fn(p); break;
    case TK_LET:     d = parse_let(p); break;
    case TK_STATIC:  d = parse_static(p); break;
    case TK_ENUM:    d = parse_enum(p); break;
    case TK_EXTERN:  d = parse_extern(p); break;
    case TK_DATAIMPL: d = parse_dataimpl(p); break;
    default:
        perr(p, "expected top-level decl");
        advance(p);
        continue;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 705) - [public] perr

```
    perr(p, "expected top-level decl");
    advance(p);
    continue;
}

nl_push(p, &tops, d);

if (p->failed) {
    break;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 706) - [public] advance

```
    advance(p);
    continue;
}

nl_push(p, &tops, d);

if (p->failed) {
    break;
}

nl_commit(p, &tops, prog);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/parser.c (line 710) - [public] nl_push

```
    nl_push(p, &tops, d);

    if (p->failed) {
        break;
    }

    nl_commit(p, &tops, prog);

    if (p->failed) {
        return (void *)0;
    }
    // ... (truncated, see source file for more)
```



```
kernel_Workspace/kernel/src/core-lang/parser.c (line 712) - [public] if
    if (p->failed) {
        break;
    }
}

nl_commit(p, &tops, prog);

if (p->failed) {
    return (void *)0;
}

return prog;
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 717) - [public] nl_commit
    nl_commit(p, &tops, prog);

    if (p->failed) {
        return (void *)0;
    }

    return prog;
}
```

```
kernel_Workspace/kernel/src/core-lang/parser.c (line 719) - [public] if
    if (p->failed) {
        return (void *)0;
    }

    return prog;
}
```

```
kernel_Workspace/kernel/src/core-lang/run.c (line 4) - [public] cl_kernel_read
extern long cl_kernel_read(const char *path, uint8_t *buf, size_t cap);
extern void *cl_native_compile(cl_prog_t *P);

int cl_run_script(const char *path, uint8_t ring, arena_t *ar)
{
    static uint8_t io[64 * 1024];

    long n = cl_kernel_read(path, io, sizeof(io));

    if (n <= 0) {
        return -1;
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/core-lang/run.c (line 13) - [public] if
    if (n <= 0) {
        return -1;
    }

    uint32_t el = 0;
    const char *em = (void *)0;

    cl_prog_t *P = cl_compile_source((const char *)io, (size_t)n, ar, &el, &em);

    if (P == (void *)0) {
        extern void kprintf(const char *, ...);
        kprintf("core-lang: %s line %u: %s\n", path, el, em);
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/run.c (line 22) - [public] if

```
    if (P == (void *)0) {
        extern void kprintf(const char *, ...);
        kprintf("core-lang: %s line %u: %s\n", path, el, em);
        return -1;
    }

    cl_vm_t vm;

    cl_vm_init(&vm, P);
    cl_bridge_init(&vm, ring);

    void *code = cl_native_compile(P);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/run.c (line 23) - [public] kprintf

```
    extern void kprintf(const char *, ...);
    kprintf("core-lang: %s line %u: %s\n", path, el, em);
    return -1;
}

cl_vm_t vm;

cl_vm_init(&vm, P);
cl_bridge_init(&vm, ring);

void *code = cl_native_compile(P);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/run.c (line 24) - [public] kprintf

```
    kprintf("core-lang: %s line %u: %s\n", path, el, em);
    return -1;
}

cl_vm_t vm;

cl_vm_init(&vm, P);
cl_bridge_init(&vm, ring);

void *code = cl_native_compile(P);

if (code != (void *)0) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/run.c (line 30) - [public] cl_vm_init

```
cl_vm_init(&vm, P);
cl_bridge_init(&vm, ring);

void *code = cl_native_compile(P);

if (code != (void *)0) {
    extern void cl_make_exec(void *p, size_t len);
    cl_make_exec(code, 16384);

    static uint64_t slots[64], globs[64];
    static cl_ext_fn exts[16];

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/run.c (line 31) - [public] cl_bridge_init
    cl_bridge_init(&vm, ring);

    void *code = cl_native_compile(P);

    if (code != (void *)0) {
        extern void cl_make_exec(void *p, size_t len);
        cl_make_exec(code, 16384);

        static uint64_t slots[64], globs[64];
        static cl_ext_fn exts[16];

        for (size_t i = 0; i < 64; i++) {
            // ... (truncated, see source file for more)
        }
    }

kernel_Workspace/kernel/src/core-lang/run.c (line 35) - [public] if
    if (code != (void *)0) {
        extern void cl_make_exec(void *p, size_t len);
        cl_make_exec(code, 16384);

        static uint64_t slots[64], globs[64];
        static cl_ext_fn exts[16];

        for (size_t i = 0; i < 64; i++) {
            globs[i] = 0;

            for (int k = 0; k < 8; k++) {
                globs[i] |= (uint64_t)P->ginit[i].b[k] << (k * 8);
            }
            // ... (truncated, see source file for more)
        }
    }

kernel_Workspace/kernel/src/core-lang/run.c (line 36) - [public] cl_make_exec
    extern void cl_make_exec(void *p, size_t len);
    cl_make_exec(code, 16384);

    static uint64_t slots[64], globs[64];
    static cl_ext_fn exts[16];

    for (size_t i = 0; i < 64; i++) {
        globs[i] = 0;

        for (int k = 0; k < 8; k++) {
            globs[i] |= (uint64_t)P->ginit[i].b[k] << (k * 8);
        }
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/run.c (line 37) - [public] cl_make_exec
    cl_make_exec(code, 16384);

    static uint64_t slots[64], globs[64];
    static cl_ext_fn exts[16];

    for (size_t i = 0; i < 64; i++) {
        globs[i] = 0;

        for (int k = 0; k < 8; k++) {
            globs[i] |= (uint64_t)P->ginit[i].b[k] << (k * 8);
        }
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/run.c (line 57) - [public] cl_vm_run
    cl_vm_run(&vm);
}

return 0;
}

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 45) - [public] if

```
    if (s == (void *)0) {
        return (void *)0;
    }

    s->ar = ar;
    s->top = 0;

    return s;
}

static void diag(sema_ctx_t *s, uint32_t line, const char *msg,
                const char *name, bool warn)
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/sema.c (line 58) - [public] if

```
    if (s->ndiags < 64) {
        cl_diag_t *d = &s->diags[s->ndiags++];

        d->line = line;
        d->msg = msg;
        d->is_warn = warn;
        d->name[0] = '\0';

        if (name != (void *)0) {
            for (int i = 0; name[i] && i < 63; i++) {
                d->name[i] = name[i];
                d->name[i + 1] = '\0';
            }
        }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/sema.c (line 66) - [public] if

```
        if (name != (void *)0) {
            for (int i = 0; name[i] && i < 63; i++) {
                d->name[i] = name[i];
                d->name[i + 1] = '\0';
            }
        }
    }
}

static void push_scope(sema_ctx_t *s)
{
    if (s->top < 15) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/core-lang/sema.c (line 77) - [public] if

```
    if (s->top < 15) {
        s->top++;
        s->scopes[s->top].head = (void *)0;
    }
}

static void pop_scope(sema_ctx_t *s)
{
    for (symbol_t *sym = s->scopes[s->top].head; sym; sym = sym->next) {
        if (sym->kind == SYM_VAR && sym->live) {
            diag(s, 0, "warning: variable left alive without set_free",
                sym->name, true);
        }
// ... (truncated, see source file for more)
    }
}
```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 86) - [public] if
    if (sym->kind == SYM_VAR && sym->live) {
        diag(s, 0, "warning: variable left alive without set_free",
            sym->name, true);
    }
}

if (s->top > 0) {
    s->top--;
}

static symbol_t *declare(sema_ctx_t *s, const char *name, uint8_t kind,
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 92) - [public] if
    if (s->top > 0) {
        s->top--;
    }
}

static symbol_t *declare(sema_ctx_t *s, const char *name, uint8_t kind,
    cl_type_t type, uint32_t line)
{
    for (symbol_t *sym = s->scopes[s->top].head; sym; sym = sym->next) {
        bool same = true;

        for (int i = 0; name[i] || sym->name[i]; i++) {
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 104) - [public] if
        if (name[i] != sym->name[i]) {
            same = false;
            break;
        }
    }

    if (same) {
        diag(s, line, "redeclaration in same scope", name, false);
        return sym;
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 110) - [public] if
    if (same) {
        diag(s, line, "redeclaration in same scope", name, false);
        return sym;
    }
}

symbol_t *sym = (symbol_t *)cl_arena_alloc(s->ar, sizeof(symbol_t));

if (sym == (void *)0) {
    return (void *)0;
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 111) - [public] diag
    diag(s, line, "redeclaration in same scope", name, false);
    return sym;
}
}

symbol_t *sym = (symbol_t *)cl_arena_alloc(s->ar, sizeof(symbol_t));

if (sym == (void *)0) {
    return (void *)0;
}

for (int i = 0; name[i] && i < 63; i++) {
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 118) - [public] if
    if (sym == (void *)0) {
        return (void *)0;
    }

    for (int i = 0; name[i] && i < 63; i++) {
        sym->name[i] = name[i];
        sym->name[i + 1] = '\0';
    }

    sym->kind = kind;
    sym->type = type;
    sym->live = true;
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 149) - [public] if
    if (name[k] != sym->name[k]) {
        same = false;
        break;
    }
}

if (same) {
    return sym;
}
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 155) - [public] if
    if (same) {
        return sym;
    }
}

return (void *)0;
}

```

```

static bool value_fits(uint64_t v, bool negative, cl_type_t t)
{
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 167) - [public] if
    if (t.bits == 0) {
        return true;
    }

    if (t.bits >= 64) {
        return true;
    }

    if (negative) {
        if (!t.neg) {
            return false;
        }
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 171) - [public] if
    if (t.bits >= 64) {
        return true;
    }

    if (negative) {
        if (!t.neg) {
            return false;
        }
    }

    return v <= (1ULL << (t.bits - 1));
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 175) - [public] if
    if (negative) {
        if (!t.neg) {
            return false;
        }
    }

    return v <= (1ULL << (t.bits - 1));
}

    if (t.neg) {
        return v <= (1ULL << (t.bits - 1)) - 1;
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 176) - [public] if
    if (!t.neg) {
        return false;
    }

    return v <= (1ULL << (t.bits - 1));
}

    if (t.neg) {
        return v <= (1ULL << (t.bits - 1)) - 1;
    }

    return v < (1ULL << t.bits);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 183) - [public] if
    if (t.neg) {
        return v <= (1ULL << (t.bits - 1)) - 1;
    }

    return v < (1ULL << t.bits);
}

typedef struct {
    cl_type_t t;
    bool is_const;
    uint64_t cval;
    bool cneg;
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/sema.c (line 197) - [public] check_expr
static etype_t check_expr(sema_ctx_t *s, ast_node_t *n);

static void shared_note(sema_ctx_t *s, const char *name, cl_type_t t,
                        uint32_t line)
{
    if (s->sh_count < 2) {
        for (int i = 0; name[i] && i < 63; i++) {
            s->sh_name[s->sh_count][i] = name[i];
            s->sh_name[s->sh_count][i + 1] = '\0';
        }

        s->sh_type[s->sh_count] = t;
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/core-lang/sema.c (line 202) - [public] if
    if (s->sh_count < 2) {
        for (int i = 0; name[i] && i < 63; i++) {
            s->sh_name[s->sh_count][i] = name[i];
            s->sh_name[s->sh_count][i + 1] = '\0';
        }

        s->sh_type[s->sh_count] = t;
        s->sh_line[s->sh_count] = line;
        s->sh_count++;
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 216) - [public] if
    if (s->sh_count == 1) {
        diag(s, s->sh_line[0], "$@ requires two participating variables",
            s->sh_name[0], false);
    } else if (s->sh_count == 2) {
        if (s->sh_type[0].bits != s->sh_type[1].bits) {
            diag(s, s->sh_line[1], "$@ pair must share the same range",
                s->sh_name[1], false);
        }
    }

    s->sh_count = 0;
}

// ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/sema.c (line 220) - [public] if
    if (s->sh_type[0].bits != s->sh_type[1].bits) {
        diag(s, s->sh_line[1], "$@ pair must share the same range",
            s->sh_name[1], false);
    }
}

s->sh_count = 0;
}

static etype_t check_expr(sema_ctx_t *s, ast_node_t *n)
{
    etype_t r = { { 0, false }, false, 0, false };
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 233) - [public] if
    if (n == (void *)0) {
        return r;
    }

    switch (n->kind) {
    case ND_NUM:
        r.is_const = true;
        r.cval = n->value;
        return r;

    case ND_CHAR:
        r.is_const = true;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 237) - [public] switch
    switch (n->kind) {
    case ND_NUM:
        r.is_const = true;
        r.cval = n->value;
        return r;

    case ND_CHAR:
        r.is_const = true;
        r.cval = n->value;
        r.t.bits = 64;
        return r;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 255) - [public] if
    if (n->op == 45 ) {
        if (r.is_const) {
            r.cneg = true;
        } else if (r.t.bits != 0 && !r.t.neg) {
            diag(s, n->line, "negation needs _=> range", (void *)0, false);
        }
    }

    return r;

    case ND_VAR: {
        symbol_t *sym = lookup(s, n->name);
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 256) - [public] if
    if (r.is_const) {
        r.cneg = true;
    } else if (r.t.bits != 0 && !r.t.neg) {
        diag(s, n->line, "negation needs _=> range", (void *)0, false);
    }
}

return r;

case ND_VAR: {
    symbol_t *sym = lookup(s, n->name);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 259) - [public] diag
    diag(s, n->line, "negation needs _=> range", (void *)0, false);
}

return r;

case ND_VAR: {
    symbol_t *sym = lookup(s, n->name);

    if (sym == (void *)0) {
        diag(s, n->line, "undeclared identifier", n->name, false);
        return r;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 268) - [public] if
    if (sym == (void *)0) {
        diag(s, n->line, "undeclared identifier", n->name, false);
        return r;
    }

    if (sym->kind == SYM_VAR || sym->kind == SYM_STATIC) {
        if (!sym->live) {
            diag(s, n->line, "use after set_free", n->name, false);
        }

        r.t = sym->type;
    } else if (sym->kind == SYM_ENUM) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 269) - [public] diag
    diag(s, n->line, "undeclared identifier", n->name, false);
    return r;
}

    if (sym->kind == SYM_VAR || sym->kind == SYM_STATIC) {
        if (!sym->live) {
            diag(s, n->line, "use after set_free", n->name, false);
        }

        r.t = sym->type;
    } else if (sym->kind == SYM_ENUM) {
        r.t.bits = 64;
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 273) - [public] if
    if (sym->kind == SYM_VAR || sym->kind == SYM_STATIC) {
        if (!sym->live) {
            diag(s, n->line, "use after set_free", n->name, false);
        }

        r.t = sym->type;
    } else if (sym->kind == SYM_ENUM) {
        r.t.bits = 64;
    } else {
        diag(s, n->line, "not a value", n->name, false);
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 274) - [public] if
    if (!sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }

    r.t = sym->type;
} else if (sym->kind == SYM_ENUM) {
    r.t.bits = 64;
} else {
    diag(s, n->line, "not a value", n->name, false);
}

    return r;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 275) - [public] diag
    diag(s, n->line, "use after set_free", n->name, false);
}

    r.t = sym->type;
} else if (sym->kind == SYM_ENUM) {
    r.t.bits = 64;
} else {
    diag(s, n->line, "not a value", n->name, false);
}

    return r;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 282) - [public] diag
    diag(s, n->line, "not a value", n->name, false);
}

    return r;
}

case ND_POOL: {
    symbol_t *sym = lookup(s, n->name);

    if (sym == (void *)0) {
        diag(s, n->line, "undeclared identifier", n->name, false);
        return r;
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 291) - [public] if
    if (sym == (void *)0) {
        diag(s, n->line, "undeclared identifier", n->name, false);
        return r;
    }

    if (sym->depth != 0 || sym->kind != SYM_VAR) {
        diag(s, n->line, "$ works only on base pool variables",
            n->name, false);
    }

    if (!sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 292) - [public] diag
    diag(s, n->line, "undeclared identifier", n->name, false);
    return r;
}

    if (sym->depth != 0 || sym->kind != SYM_VAR) {
        diag(s, n->line, "$ works only on base pool variables",
            n->name, false);
    }

    if (!sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 296) - [public] if
    if (sym->depth != 0 || sym->kind != SYM_VAR) {
        diag(s, n->line, "$ works only on base pool variables",
            n->name, false);
    }

    if (!sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }

    sym->pool_ref = true;

    if (n->pool_kind == 1 && !sym->type.neg) {
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 301) - [public] if
    if (!sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }

    sym->pool_ref = true;

    if (n->pool_kind == 1 && !sym->type.neg) {
        diag(s, n->line, "$! needs neg range (=>-1)", n->name, false);
    }

    if (n->pool_kind == 2) {
        sym->live = true;
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 302) - [public] diag
    diag(s, n->line, "use after set_free", n->name, false);
}

    sym->pool_ref = true;

    if (n->pool_kind == 1 && !sym->type.neg) {
        diag(s, n->line, "$! needs neg range (=>-1)", n->name, false);
    }

    if (n->pool_kind == 2) {
        sym->live = true;
        shared_note(s, n->name, sym->type, n->line);
    }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 307) - [public] if
    if (n->pool_kind == 1 && !sym->type.neg) {
        diag(s, n->line, "$! needs neg range (=>-1)", n->name, false);
    }

    if (n->pool_kind == 2) {
        sym->live = true;
        shared_note(s, n->name, sym->type, n->line);
    }

    r.t = sym->type;
    return r;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 308) - [public] diag
    diag(s, n->line, "$! needs neg range (=>-1)", n->name, false);
}

    if (n->pool_kind == 2) {
        sym->live = true;
        shared_note(s, n->name, sym->type, n->line);
    }

    r.t = sym->type;
    return r;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 311) - [public] if
    if (n->pool_kind == 2) {
        sym->live = true;
        shared_note(s, n->name, sym->type, n->line);
    }

    r.t = sym->type;
    return r;
}

case ND_CALLIDX: {
    symbol_t *sym = lookup(s, n->name);
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 313) - [public] shared_note
    shared_note(s, n->name, sym->type, n->line);
}

    r.t = sym->type;
    return r;
}

case ND_CALLIDX: {
    symbol_t *sym = lookup(s, n->name);

    if (sym == (void *)0) {
        diag(s, n->line, "call[] on undeclared", n->name, false);
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/sema.c (line 323) - [public] if
    if (sym == (void *)0) {
        diag(s, n->line, "call[] on undeclared", n->name, false);
        return r;
    }

    if ((sym->kind == SYM_VAR || sym->kind == SYM_STATIC) && !sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }

    r.t = (sym->kind == SYM_FN || sym->kind == SYM_EXTERN)
        ? (cl_type_t){ 64, false } : sym->type;
    return r;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 324) - [public] diag
    diag(s, n->line, "call[] on undeclared", n->name, false);
    return r;
}

    if ((sym->kind == SYM_VAR || sym->kind == SYM_STATIC) && !sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }

    r.t = (sym->kind == SYM_FN || sym->kind == SYM_EXTERN)
        ? (cl_type_t){ 64, false } : sym->type;
    return r;
}
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 328) - [public] if
    if ((sym->kind == SYM_VAR || sym->kind == SYM_STATIC) && !sym->live) {
        diag(s, n->line, "use after set_free", n->name, false);
    }

    r.t = (sym->kind == SYM_FN || sym->kind == SYM_EXTERN)
        ? (cl_type_t){ 64, false } : sym->type;
    return r;
}

case ND_SCOPECALL: {
    symbol_t *sc = lookup(s, n->name);

    // ... (truncated, see source file for more)
}

```

```
kernel_Workspace/kernel/src/core-lang/sema.c (line 329) - [public] diag
    diag(s, n->line, "use after set_free", n->name, false);
}

r.t = (sym->kind == SYM_FN || sym->kind == SYM_EXTERN)
    ? (cl_type_t){ 64, false } : sym->type;
return r;
}
```

```
case ND_SCOPECALL: {
    symbol_t *sc = lookup(s, n->name);

    if (sc == (void *)0) {
// ... (truncated, see source file for more)
}
```

```
kernel_Workspace/kernel/src/core-lang/sema.c (line 340) - [public] if
    if (sc == (void *)0) {
        diag(s, n->line, "unknown scope", n->name, false);
        return r;
    }
```

```
    if (sc->kind == SYM_ENUM) {
        bool found = false;

        if (sc->node != (void *)0) {
            for (size_t i = 0; i < sc->node->nlist; i++) {
                ast_node_t *it = sc->node->list[i];
```

```
// ... (truncated, see source file for more)
}
```

```
kernel_Workspace/kernel/src/core-lang/sema.c (line 341) - [public] diag
    diag(s, n->line, "unknown scope", n->name, false);
    return r;
}
```

```
    if (sc->kind == SYM_ENUM) {
        bool found = false;

        if (sc->node != (void *)0) {
            for (size_t i = 0; i < sc->node->nlist; i++) {
                ast_node_t *it = sc->node->list[i];
```

```
                bool same = true;
// ... (truncated, see source file for more)
}
```

```
kernel_Workspace/kernel/src/core-lang/sema.c (line 345) - [public] if
```

```
    if (sc->kind == SYM_ENUM) {
        bool found = false;

        if (sc->node != (void *)0) {
            for (size_t i = 0; i < sc->node->nlist; i++) {
                ast_node_t *it = sc->node->list[i];

                bool same = true;

                for (int k = 0; it->name[k] || n->name2[k]; k++) {
                    if (it->name[k] != n->name2[k]) {
                        same = false;
                    }
                }
            }
        }
// ... (truncated, see source file for more)
}
```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 348) - [public] if
    if (sc->node != (void *)0) {
        for (size_t i = 0; i < sc->node->nlist; i++) {
            ast_node_t *it = sc->node->list[i];

            bool same = true;

            for (int k = 0; it->name[k] || n->name2[k]; k++) {
                if (it->name[k] != n->name2[k]) {
                    same = false;
                    break;
                }
            }
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 355) - [public] if
        if (it->name[k] != n->name2[k]) {
            same = false;
            break;
        }

        if (same) {
            found = true;
            r.is_const = true;
            r.cval = it->value;
            r.t.bits = 64;
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 361) - [public] if
        if (same) {
            found = true;
            r.is_const = true;
            r.cval = it->value;
            r.t.bits = 64;
        }
    }

    if (!found) {
        diag(s, n->line, "no such enum item", n->name2, false);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 370) - [public] if
    if (!found) {
        diag(s, n->line, "no such enum item", n->name2, false);
    }

    return r;
}

if (sc->kind == SYM_DATA) {
    bool found = false;

    if (sc->node != (void *)0 && sc->node->a != (void *)0) {
        ast_node_t *base = sc->node->a;
        // ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/sema.c (line 371) - [public] diag
    diag(s, n->line, "no such enum item", n->name2, false);
}

return r;
}

if (sc->kind == SYM_DATA) {
    bool found = false;

    if (sc->node != (void *)0 && sc->node->a != (void *)0) {
        ast_node_t *base = sc->node->a;

        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/sema.c (line 377) - [public] if
    if (sc->kind == SYM_DATA) {
        bool found = false;

        if (sc->node != (void *)0 && sc->node->a != (void *)0) {
            ast_node_t *base = sc->node->a;

            for (size_t i = 0; i < base->nlist; i++) {
                ast_node_t *e = base->list[i];

                if (e->kind == ND_VAR) {
                    bool same = true;

                    // ... (truncated, see source file for more)
                }

kernel_Workspace/kernel/src/core-lang/sema.c (line 380) - [public] if
        if (sc->node != (void *)0 && sc->node->a != (void *)0) {
            ast_node_t *base = sc->node->a;

            for (size_t i = 0; i < base->nlist; i++) {
                ast_node_t *e = base->list[i];

                if (e->kind == ND_VAR) {
                    bool same = true;

                    for (int k = 0; e->name[k] || n->name2[k]; k++) {
                        if (e->name[k] != n->name2[k]) {
                            same = false;
                        }
                    }

                    // ... (truncated, see source file for more)
                }

kernel_Workspace/kernel/src/core-lang/sema.c (line 386) - [public] if
        if (e->kind == ND_VAR) {
            bool same = true;

            for (int k = 0; e->name[k] || n->name2[k]; k++) {
                if (e->name[k] != n->name2[k]) {
                    same = false;
                    break;
                }
            }

            if (same) {
                found = true;
            }

            // ... (truncated, see source file for more)
        }
    }
}

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 390) - [public] if
        if (e->name[k] != n->name2[k]) {
            same = false;
            break;
        }
    }

    if (same) {
        found = true;
        r.t.bits = 64;
    }
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 396) - [public] if
        if (same) {
            found = true;
            r.t.bits = 64;
        }
    }
}

    if (!found) {
        diag(s, n->line, "no such data entry", n->name2, false);
    }

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 404) - [public] if
    if (!found) {
        diag(s, n->line, "no such data entry", n->name2, false);
    }

    return r;
}

    diag(s, n->line, "scope is not enum/data", n->name, false);
    return r;
}

case ND_CALL: {
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 405) - [public] diag
        diag(s, n->line, "no such data entry", n->name2, false);
    }

    return r;
}

    diag(s, n->line, "scope is not enum/data", n->name, false);
    return r;
}

case ND_CALL: {
    if (n->a == (void *)0 || n->a->kind != ND_VAR) {
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 411) - [public] diag
    diag(s, n->line, "scope is not enum/data", n->name, false);
    return r;
}

case ND_CALL: {
    if (n->a == (void *)0 || n->a->kind != ND_VAR) {
        diag(s, n->line, "bad call target", (void *)0, false);
        return r;
    }

    symbol_t *fn = lookup(s, n->a->name);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 416) - [public] if
    if (n->a == (void *)0 || n->a->kind != ND_VAR) {
        diag(s, n->line, "bad call target", (void *)0, false);
        return r;
    }

    symbol_t *fn = lookup(s, n->a->name);

    if (fn == (void *)0) {
        diag(s, n->line, "call of undeclared", n->a->name, false);
        return r;
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 417) - [public] diag
    diag(s, n->line, "bad call target", (void *)0, false);
    return r;
}

    symbol_t *fn = lookup(s, n->a->name);

    if (fn == (void *)0) {
        diag(s, n->line, "call of undeclared", n->a->name, false);
        return r;
    }

    if (fn->kind == SYM_FN) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 423) - [public] if
    if (fn == (void *)0) {
        diag(s, n->line, "call of undeclared", n->a->name, false);
        return r;
    }

    if (fn->kind == SYM_FN) {
        if (fn->nparams != n->nlist) {
            diag(s, n->line, "arity mismatch", fn->name, false);
        }

        for (size_t i = 0; i < n->nlist && i < fn->nparams; i++) {
            etype_t a = check_expr(s, n->list[i]);
        }
    }

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 424) - [public] diag
    diag(s, n->line, "call of undeclared", n->a->name, false);
    return r;
}

if (fn->kind == SYM_FN) {
    if (fn->nparams != n->nlist) {
        diag(s, n->line, "arity mismatch", fn->name, false);
    }

    for (size_t i = 0; i < n->nlist && i < fn->nparams; i++) {
        etype_t a = check_expr(s, n->list[i]);

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 428) - [public] if
    if (fn->kind == SYM_FN) {
        if (fn->nparams != n->nlist) {
            diag(s, n->line, "arity mismatch", fn->name, false);
        }

        for (size_t i = 0; i < n->nlist && i < fn->nparams; i++) {
            etype_t a = check_expr(s, n->list[i]);

            if (a.is_const &&
                !value_fits(a.cval, a.cneg, fn->params[i])) {
                diag(s, n->line, "arg out of range", fn->name, false);
            }
        }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 429) - [public] if
    if (fn->nparams != n->nlist) {
        diag(s, n->line, "arity mismatch", fn->name, false);
    }

    for (size_t i = 0; i < n->nlist && i < fn->nparams; i++) {
        etype_t a = check_expr(s, n->list[i]);

        if (a.is_const &&
            !value_fits(a.cval, a.cneg, fn->params[i])) {
            diag(s, n->line, "arg out of range", fn->name, false);
        }
    }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 430) - [public] diag
    diag(s, n->line, "arity mismatch", fn->name, false);
}

for (size_t i = 0; i < n->nlist && i < fn->nparams; i++) {
    etype_t a = check_expr(s, n->list[i]);

    if (a.is_const &&
        !value_fits(a.cval, a.cneg, fn->params[i])) {
        diag(s, n->line, "arg out of range", fn->name, false);
    }
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 438) - [public] diag
        diag(s, n->line, "arg out of range", fn->name, false);
    }
}

    r.t.bits = 64;
    return r;
}

    if (fn->kind == SYM_EXTERN) {
        for (size_t i = 0; i < n->nlist; i++) {
            check_expr(s, n->list[i]);
        }
    }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 446) - [public] if
    if (fn->kind == SYM_EXTERN) {
        for (size_t i = 0; i < n->nlist; i++) {
            check_expr(s, n->list[i]);
        }

        r.t.bits = 64;
        return r;
    }

    diag(s, n->line, "not callable", n->a->name, false);
    return r;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 448) - [public] check_expr
        check_expr(s, n->list[i]);
    }

    r.t.bits = 64;
    return r;
}

    diag(s, n->line, "not callable", n->a->name, false);
    return r;
}

case ND_MEMBER:
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 455) - [public] diag
    diag(s, n->line, "not callable", n->a->name, false);
    return r;
}

case ND_MEMBER:
    r = check_expr(s, n->a);
    return r;

case ND_ASSIGN: {
    etype_t l = check_expr(s, n->a);
    etype_t rhs = check_expr(s, n->b);

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 467) - [public] if
    if (l.t.bits != 0) {
        if (rhs.is_const) {
            if (!value_fits(rhs.cval, rhs.cneg, l.t)) {
                diag(s, n->line, "value out of range", (void *)0, false);
            }
        }
        } else if (rhs.t.bits != 0) {
            if (rhs.t.bits > l.t.bits) {
                diag(s, n->line, "narrowing assignment", (void *)0, false);
            } else if (rhs.t.bits == l.t.bits &&
                rhs.t.neg != l.t.neg) {
                diag(s, n->line, "signed/unsigned mismatch",
                    (void *)0, false);
            }
        }
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 468) - [public] if
    if (rhs.is_const) {
        if (!value_fits(rhs.cval, rhs.cneg, l.t)) {
            diag(s, n->line, "value out of range", (void *)0, false);
        }
    }
    } else if (rhs.t.bits != 0) {
        if (rhs.t.bits > l.t.bits) {
            diag(s, n->line, "narrowing assignment", (void *)0, false);
        } else if (rhs.t.bits == l.t.bits &&
            rhs.t.neg != l.t.neg) {
            diag(s, n->line, "signed/unsigned mismatch",
                (void *)0, false);
        }
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 469) - [public] if
    if (!value_fits(rhs.cval, rhs.cneg, l.t)) {
        diag(s, n->line, "value out of range", (void *)0, false);
    }
    } else if (rhs.t.bits != 0) {
        if (rhs.t.bits > l.t.bits) {
            diag(s, n->line, "narrowing assignment", (void *)0, false);
        } else if (rhs.t.bits == l.t.bits &&
            rhs.t.neg != l.t.neg) {
            diag(s, n->line, "signed/unsigned mismatch",
                (void *)0, false);
        }
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 470) - [public] diag
    diag(s, n->line, "value out of range", (void *)0, false);
    }
    } else if (rhs.t.bits != 0) {
        if (rhs.t.bits > l.t.bits) {
            diag(s, n->line, "narrowing assignment", (void *)0, false);
        } else if (rhs.t.bits == l.t.bits &&
            rhs.t.neg != l.t.neg) {
            diag(s, n->line, "signed/unsigned mismatch",
                (void *)0, false);
        }
    }
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 473) - [public] if
    if (rhs.t.bits > l.t.bits) {
        diag(s, n->line, "narrowing assignment", (void *)0, false);
    } else if (rhs.t.bits == l.t.bits &&
        rhs.t.neg != l.t.neg) {
        diag(s, n->line, "signed/unsigned mismatch",
            (void *)0, false);
    }
}
}
return l;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 474) - [public] diag
    diag(s, n->line, "narrowing assignment", (void *)0, false);
    } else if (rhs.t.bits == l.t.bits &&
        rhs.t.neg != l.t.neg) {
        diag(s, n->line, "signed/unsigned mismatch",
            (void *)0, false);
    }
}
}
return l;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 493) - [public] if
    if (cmp) {
        r.t.bits = 1;
        return r;
    }

    if (a.t.bits == 0 && b.t.bits != 0) {
        if (a.is_const && !value_fits(a.cval, a.cneg, b.t)) {
            diag(s, n->line, "value out of range", (void *)0, false);
        }

        r.t = b.t;
    } else if (b.t.bits == 0 && a.t.bits != 0) {
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 498) - [public] if
    if (a.t.bits == 0 && b.t.bits != 0) {
        if (a.is_const && !value_fits(a.cval, a.cneg, b.t)) {
            diag(s, n->line, "value out of range", (void *)0, false);
        }

        r.t = b.t;
    } else if (b.t.bits == 0 && a.t.bits != 0) {
        if (b.is_const && !value_fits(b.cval, b.cneg, a.t)) {
            diag(s, n->line, "value out of range", (void *)0, false);
        }

        r.t = a.t;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 499) - [public] if
    if (a.is_const && !value_fits(a.cval, a.cneg, b.t)) {
        diag(s, n->line, "value out of range", (void *)0, false);
    }

    r.t = b.t;
} else if (b.t.bits == 0 && a.t.bits != 0) {
    if (b.is_const && !value_fits(b.cval, b.cneg, a.t)) {
        diag(s, n->line, "value out of range", (void *)0, false);
    }

    r.t = a.t;
} else {
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 500) - [public] diag
    diag(s, n->line, "value out of range", (void *)0, false);
}

    r.t = b.t;
} else if (b.t.bits == 0 && a.t.bits != 0) {
    if (b.is_const && !value_fits(b.cval, b.cneg, a.t)) {
        diag(s, n->line, "value out of range", (void *)0, false);
    }

    r.t = a.t;
} else {
    r.t.bits = a.t.bits > b.t.bits ? a.t.bits : b.t.bits;
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 505) - [public] if
    if (b.is_const && !value_fits(b.cval, b.cneg, a.t)) {
        diag(s, n->line, "value out of range", (void *)0, false);
    }

    r.t = a.t;
} else {
    r.t.bits = a.t.bits > b.t.bits ? a.t.bits : b.t.bits;
    r.t.neg = a.t.neg || b.t.neg;
}

    return r;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 506) - [public] diag
    diag(s, n->line, "value out of range", (void *)0, false);
}

    r.t = a.t;
} else {
    r.t.bits = a.t.bits > b.t.bits ? a.t.bits : b.t.bits;
    r.t.neg = a.t.neg || b.t.neg;
}

    return r;
}
// ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/sema.c (line 524) - [public] check_block
static void check_block(sema_ctx_t *s, ast_node_t *n);

static void check_stmt(sema_ctx_t *s, ast_node_t *n)
{
    s->sh_count = 0;

    switch (n->kind) {
    case ND_LET: {
        etype_t init = check_expr(s, n->a);

        if (init.is_const &&
            !value_fits(init.cval, init.cneg, n->type)) {
            // ... (truncated, see source file for more)
        }
    }
}

kernel_Workspace/kernel/src/core-lang/sema.c (line 530) - [public] switch
switch (n->kind) {
case ND_LET: {
    etype_t init = check_expr(s, n->a);

    if (init.is_const &&
        !value_fits(init.cval, init.cneg, n->type)) {
        diag(s, n->line, "literal out of type range", n->name, false);
    }

    if (init.t.bits != 0 && init.t.bits > n->type.bits) {
        diag(s, n->line, "narrowing assignment", n->name, false);
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/sema.c (line 536) - [public] diag
diag(s, n->line, "literal out of type range", n->name, false);
}

if (init.t.bits != 0 && init.t.bits > n->type.bits) {
    diag(s, n->line, "narrowing assignment", n->name, false);
}

symbol_t *sym = declare(s, n->name, SYM_VAR, n->type, n->line);

if (sym != (void *)0 && n->type.neg) {
    sym->type.neg = true;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 539) - [public] if
if (init.t.bits != 0 && init.t.bits > n->type.bits) {
    diag(s, n->line, "narrowing assignment", n->name, false);
}

symbol_t *sym = declare(s, n->name, SYM_VAR, n->type, n->line);

if (sym != (void *)0 && n->type.neg) {
    sym->type.neg = true;
}

break;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 540) - [public] diag
    diag(s, n->line, "narrowing assignment", n->name, false);
}

    symbol_t *sym = declare(s, n->name, SYM_VAR, n->type, n->line);

    if (sym != (void *)0 && n->type.neg) {
        sym->type.neg = true;
    }

    break;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 545) - [public] if
    if (sym != (void *)0 && n->type.neg) {
        sym->type.neg = true;
    }

    break;
}

case ND_STATIC:
    declare(s, "data", SYM_STATIC, n->type, n->line);
    break;

case ND_SET_FREE: {
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 553) - [public] declare
    declare(s, "data", SYM_STATIC, n->type, n->line);
    break;

case ND_SET_FREE: {
    symbol_t *sym = lookup(s, n->name);

    if (sym == (void *)0) {
        diag(s, n->line, "set_free on undeclared", n->name, false);
    } else if (sym->freed) {
        diag(s, n->line, "double set_free", n->name, false);
    } else {
        sym->live = false;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 559) - [public] if
    if (sym == (void *)0) {
        diag(s, n->line, "set_free on undeclared", n->name, false);
    } else if (sym->freed) {
        diag(s, n->line, "double set_free", n->name, false);
    } else {
        sym->live = false;
        sym->freed = true;
    }

    break;
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 560) - [public] diag
    diag(s, n->line, "set_free on undeclared", n->name, false);
} else if (sym->freed) {
    diag(s, n->line, "double set_free", n->name, false);
} else {
    sym->live = false;
    sym->freed = true;
}

break;
}

case ND_IF:
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 562) - [public] diag
    diag(s, n->line, "double set_free", n->name, false);
} else {
    sym->live = false;
    sym->freed = true;
}

break;
}

case ND_IF:
    if (n->a != (void *)0) {
        check_expr(s, n->a);
    }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 572) - [public] if
    if (n->a != (void *)0) {
        check_expr(s, n->a);
    }

    check_block(s, n->b);

    if (n->c != (void *)0) {
        if (n->c->kind == ND_BLOCK) {
            check_block(s, n->c);
        } else {
            check_stmt(s, n->c);
        }
    }
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 573) - [public] check_expr
    check_expr(s, n->a);
}

check_block(s, n->b);

if (n->c != (void *)0) {
    if (n->c->kind == ND_BLOCK) {
        check_block(s, n->c);
    } else {
        check_stmt(s, n->c);
    }
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 576) - [public] check_block
    check_block(s, n->b);

    if (n->c != (void *)0) {
        if (n->c->kind == ND_BLOCK) {
            check_block(s, n->c);
        } else {
            check_stmt(s, n->c);
        }
    }

    break;

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 578) - [public] if
    if (n->c != (void *)0) {
        if (n->c->kind == ND_BLOCK) {
            check_block(s, n->c);
        } else {
            check_stmt(s, n->c);
        }
    }

    break;

case ND_WHEN: {
    symbol_t *tgt = lookup(s, n->name);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 579) - [public] if
    if (n->c->kind == ND_BLOCK) {
        check_block(s, n->c);
    } else {
        check_stmt(s, n->c);
    }
}

break;

case ND_WHEN: {
    symbol_t *tgt = lookup(s, n->name);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 580) - [public] check_block
    check_block(s, n->c);
} else {
    check_stmt(s, n->c);
}

break;

case ND_WHEN: {
    symbol_t *tgt = lookup(s, n->name);

    if (tgt == (void *)0) {
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 582) - [public] check_stmt
    check_stmt(s, n->c);
    }
}

break;

case ND_WHEN: {
    symbol_t *tgt = lookup(s, n->name);

    if (tgt == (void *)0) {
        diag(s, n->line, "when on undeclared", n->name, false);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 591) - [public] if
    if (tgt == (void *)0) {
        diag(s, n->line, "when on undeclared", n->name, false);
    }

    for (size_t i = 0; i < n->nlist; i++) {
        ast_node_t *c = n->list[i];

        if (c->a != (void *)0 && c->a->kind == ND_BIN &&
            c->a->a != (void *)0 && c->a->a->kind == ND_MEMBER &&
            c->a->a->a != (void *)0 &&
            c->a->a->a->kind == ND_VAR) {
            const char *cn = c->a->a->a->name;
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 592) - [public] diag
    diag(s, n->line, "when on undeclared", n->name, false);
}

    for (size_t i = 0; i < n->nlist; i++) {
        ast_node_t *c = n->list[i];

        if (c->a != (void *)0 && c->a->kind == ND_BIN &&
            c->a->a != (void *)0 && c->a->a->kind == ND_MEMBER &&
            c->a->a->a != (void *)0 &&
            c->a->a->a->kind == ND_VAR) {
            const char *cn = c->a->a->a->name;
            bool same = true;
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 606) - [public] if
        if (cn[k] != n->name[k]) {
            same = false;
            break;
        }
    }

    if (!same) {
        diag(s, c->line, "case target mismatch", cn, false);
    }

    if (tgt != (void *)0 && c->a->b != (void *)0 &&
        c->a->b->kind == ND_NUM &&
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 612) - [public] if
    if (!same) {
        diag(s, c->line, "case target mismatch", cn, false);
    }

    if (tgt != (void *)0 && c->a->b != (void *)0 &&
        c->a->b->kind == ND_NUM &&
        !value_fits(c->a->b->value, false, tgt->type)) {
        diag(s, c->line, "case value out of range",
            n->name, false);
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 613) - [public] diag
    diag(s, c->line, "case target mismatch", cn, false);
}

    if (tgt != (void *)0 && c->a->b != (void *)0 &&
        c->a->b->kind == ND_NUM &&
        !value_fits(c->a->b->value, false, tgt->type)) {
        diag(s, c->line, "case value out of range",
            n->name, false);
    }
}

    check_block(s, c->b);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 624) - [public] check_block
    check_block(s, c->b);
}

    break;
}

case ND_BLOCK:
    check_block(s, n);
    break;

default:
    check_expr(s, n);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 631) - [public] check_block
    check_block(s, n);
    break;

default:
    check_expr(s, n);
    shared_validate(s);
    break;
}

if (n->kind != ND_BLOCK) {
    shared_validate(s);
}
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 635) - [public] check_expr

```
        check_expr(s, n);
        shared_validate(s);
        break;
    }

    if (n->kind != ND_BLOCK) {
        shared_validate(s);
    }
}

static void check_block(sema_ctx_t *s, ast_node_t *n)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/sema.c (line 636) - [public] shared_validate

```
        shared_validate(s);
        break;
    }

    if (n->kind != ND_BLOCK) {
        shared_validate(s);
    }
}

static void check_block(sema_ctx_t *s, ast_node_t *n)
{
    push_scope(s);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/sema.c (line 640) - [public] if

```
    if (n->kind != ND_BLOCK) {
        shared_validate(s);
    }
}

static void check_block(sema_ctx_t *s, ast_node_t *n)
{
    push_scope(s);

    for (size_t i = 0; i < n->nlist; i++) {
        check_stmt(s, n->list[i]);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/sema.c (line 641) - [public] shared_validate

```
        shared_validate(s);
    }
}

static void check_block(sema_ctx_t *s, ast_node_t *n)
{
    push_scope(s);

    for (size_t i = 0; i < n->nlist; i++) {
        check_stmt(s, n->list[i]);
    }

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 647) - [public] push_scope
    push_scope(s);

    for (size_t i = 0; i < n->nlist; i++) {
        check_stmt(s, n->list[i]);
    }

    pop_scope(s);
}

static void declare_top(sema_ctx_t *s, ast_node_t *n)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/sema.c (line 650) - [public] check_stmt
    check_stmt(s, n->list[i]);
}

    pop_scope(s);
}

static void declare_top(sema_ctx_t *s, ast_node_t *n)
{
    switch (n->kind) {
    case ND_FN: {
        cl_type_t t = { 0, false };
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/sema.c (line 653) - [public] pop_scope
    pop_scope(s);
}

static void declare_top(sema_ctx_t *s, ast_node_t *n)
{
    switch (n->kind) {
    case ND_FN: {
        cl_type_t t = { 0, false };
        symbol_t *sym = declare(s, n->name, SYM_FN, t, n->line);

        if (sym == (void *)0) {
            // ... (truncated, see source file for more)
        }

kernel_Workspace/kernel/src/core-lang/sema.c (line 659) - [public] switch
    switch (n->kind) {
    case ND_FN: {
        cl_type_t t = { 0, false };
        symbol_t *sym = declare(s, n->name, SYM_FN, t, n->line);

        if (sym == (void *)0) {
            return;
        }

        for (size_t i = 0; i < n->nlist && i < 8; i++) {
            sym->params[sym->nparams++] = n->list[i]->type;
        }
        // ... (truncated, see source file for more)
    }
}

```



```

kernel_Workspace/kernel/src/core-lang/sema.c (line 664) - [public] if
    if (sym == (void *)0) {
        return;
    }

    for (size_t i = 0; i < n->nlist && i < 8; i++) {
        sym->params[sym->nparams++] = n->list[i]->type;
    }

    sym->node = n;
    break;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 677) - [public] declare
    declare(s, n->name, SYM_VAR, n->type, n->line);
    break;

case ND_STATIC:
    declare(s, "data", SYM_STATIC, n->type, n->line);
    break;

case ND_ENUM: {
    cl_type_t t = { 64, false };
    symbol_t *sym = declare(s, n->name, SYM_ENUM, t, n->line);

    if (sym != (void *)0) {
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/core-lang/sema.c (line 681) - [public] declare
    declare(s, "data", SYM_STATIC, n->type, n->line);
    break;

case ND_ENUM: {
    cl_type_t t = { 64, false };
    symbol_t *sym = declare(s, n->name, SYM_ENUM, t, n->line);

    if (sym != (void *)0) {
        sym->node = n;
    }

    break;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 688) - [public] if
    if (sym != (void *)0) {
        sym->node = n;
    }

    break;
}

case ND_EXTERN:
    for (size_t i = 0; i < n->nlist; i++) {
        cl_type_t t = { 64, false };
        declare(s, n->list[i]->name, SYM_EXTERN, t, n->line);
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 698) - [public] declare
    declare(s, n->list[i]->name, SYM_EXTERN, t, n->line);
}

    break;

case ND_DATAIMPL: {
    cl_type_t t = { 0, false };
    symbol_t *sym = declare(s, "data", SYM_DATA, t, n->line);

    if (sym != (void *)0) {
        sym->node = n;
    }
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 707) - [public] if
    if (sym != (void *)0) {
        sym->node = n;
    }

    break;
}

default:
    break;
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 721) - [public] if
    if (prog == (void *)0) {
        return -1;
    }

    for (size_t i = 0; i < prog->nlist; i++) {
        declare_top(s, prog->list[i]);
    }

    for (size_t i = 0; i < prog->nlist; i++) {
        ast_node_t *n = prog->list[i];

        if (n->kind != ND_FN) {
            // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 726) - [public] declare_top
    declare_top(s, prog->list[i]);
}

    for (size_t i = 0; i < prog->nlist; i++) {
        ast_node_t *n = prog->list[i];

        if (n->kind != ND_FN) {
            continue;
        }

        push_scope(s);

        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 732) - [public] if
    if (n->kind != ND_FN) {
        continue;
    }

    push_scope(s);

    for (size_t k = 0; k < n->nlist; k++) {
        declare(s, n->list[k]->name, SYM_VAR, n->list[k]->type, n->line);
    }

    check_block(s, n->a);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 736) - [public] push_scope
    push_scope(s);

    for (size_t k = 0; k < n->nlist; k++) {
        declare(s, n->list[k]->name, SYM_VAR, n->list[k]->type, n->line);
    }

    check_block(s, n->a);

    pop_scope(s);
}

    return (int)s->ndiags;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 739) - [public] declare
    declare(s, n->list[k]->name, SYM_VAR, n->list[k]->type, n->line);
}

    check_block(s, n->a);

    pop_scope(s);
}

    return (int)s->ndiags;
}

size_t cl_sema_diag_count(const sema_ctx_t *s)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/sema.c (line 742) - [public] check_block
    check_block(s, n->a);

    pop_scope(s);
}

    return (int)s->ndiags;
}

size_t cl_sema_diag_count(const sema_ctx_t *s)
{
    return s->ndiags;
}

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/sema.c (line 744) - [public] pop_scope
    pop_scope(s);
}

    return (int)s->ndiags;
}

size_t cl_sema_diag_count(const sema_ctx_t *s)
{
    return s->ndiags;
}

const cl_diag_t *cl_sema_diag(const sema_ctx_t *s, size_t i)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 19) - [public] for
    for (int i = full + (rem ? 1 : 0); i < w; i++) {
        r[i] = 0;
    }

    if (rem) {
        r[full] &= (uint8_t)((1 << rem) - 1);
    }
}

void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w)
{
    int c = 0;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 23) - [public] if
    if (rem) {
        r[full] &= (uint8_t)((1 << rem) - 1);
    }
}

void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w)
{
    int c = 0;

    for (int i = 0; i < w; i++) {
        int s = a[i] + b[i] + c;

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 85) - [public] if
    if (a[i] != b[i]) {
        return a[i] > b[i] ? 1 : -1;
    }
}

    return 0;
}

int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign)
{
    if (!sign) {
        return cmp_u(a, b, w);
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 95) - [public] if

```
    if (!sign) {
        return cmp_u(a, b, w);
    }

    uint8_t x[64], y[64];

    for (int i = 0; i < w; i++) {
        x[i] = a[i];
        y[i] = b[i];
    }

    int sb = w * 8 - 1;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 96) - [public] cmp_u

```
    return cmp_u(a, b, w);
}

uint8_t x[64], y[64];

for (int i = 0; i < w; i++) {
    x[i] = a[i];
    y[i] = b[i];
}

int sb = w * 8 - 1;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 113) - [public] cmp_u

```
    return cmp_u(x, y, w);
}

void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w)
{
    uint8_t t[64] = { 0 };

    for (int i = 0; i < w * 8; i++) {
        if (bit_get(a, i) && (int)(i + n) < w * 8) {
            bit_set(t, i + (int)n);
        }
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 121) - [public] if

```
    if (bit_get(a, i) && (int)(i + n) < w * 8) {
        bit_set(t, i + (int)n);
    }
}

for (int i = 0; i < w; i++) {
    r[i] = t[i];
}

void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w)
{
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 122) - [public] bit_set
    bit_set(t, i + (int)n);
    }
}

for (int i = 0; i < w; i++) {
    r[i] = t[i];
}

}

void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w)
{
    uint8_t t[64] = { 0 };
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 136) - [public] if
    if (bit_get(a, i) && i >= (int)n) {
        bit_set(t, i - (int)n);
    }
}

for (int i = 0; i < w; i++) {
    r[i] = t[i];
}

}

void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 137) - [public] bit_set
    bit_set(t, i - (int)n);
    }
}

for (int i = 0; i < w; i++) {
    r[i] = t[i];
}

}

void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m)
{
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 154) - [public] cl_shl
    cl_shl(shl, rem, 1, w);

    for (int k = 0; k < w; k++) {
        rem[k] = shl[k];
    }

    if (bit_get(n, i)) {
        rem[0] |= 1;
    }

    if (cmp_u(rem, d, w) >= 0) {
        uint8_t sub[64];
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 160) - [public] if
    if (bit_get(n, i)) {
        rem[0] |= 1;
    }

    if (cmp_u(rem, d, w) >= 0) {
        uint8_t sub[64];
        cl_sub(sub, rem, d, w);

        for (int k = 0; k < w; k++) {
            rem[k] = sub[k];
        }

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 164) - [public] if
    if (cmp_u(rem, d, w) >= 0) {
        uint8_t sub[64];
        cl_sub(sub, rem, d, w);

        for (int k = 0; k < w; k++) {
            rem[k] = sub[k];
        }

        bit_set(quo, i);
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 166) - [public] cl_sub
    cl_sub(sub, rem, d, w);

    for (int k = 0; k < w; k++) {
        rem[k] = sub[k];
    }

    bit_set(quo, i);
}

for (int i = 0; i < w; i++) {
    q[i] = quo[i];
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 172) - [public] bit_set
    bit_set(quo, i);
}

for (int i = 0; i < w; i++) {
    q[i] = quo[i];
    m[i] = rem[i];
}

/* ---- VM ---- */

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 207) - [public] if
    if (vm->prog->ext_names[i][k] != name[k]) {
        same = false;
        break;
    }
}

    if (same) {
        vm->externs[i] = fn;
        return 0;
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 213) - [public] if
    if (same) {
        vm->externs[i] = fn;
        return 0;
    }
}

    return -1;
}

static void push(cl_vm_t *vm, const cl_val_t *v)
{
    if (vm->sp < 256) {
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/core-lang/vm.c (line 224) - [public] if
    if (vm->sp < 256) {
        vm->stack[vm->sp++] = *v;
    } else {
        vm->err = CL_ERR_STACK;
    }
}

static cl_val_t pop(cl_vm_t *vm)
{
    if (vm->sp > 0) {
        return vm->stack[--vm->sp];
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/core-lang/vm.c (line 233) - [public] if
    if (vm->sp > 0) {
        return vm->stack[--vm->sp];
    }

    vm->err = CL_ERR_STACK;

    cl_val_t z = { { 0 } };
    return z;
}

static uint64_t val_u64(const cl_val_t *v)
{
    // ... (truncated, see source file for more)
}

```


kernel_Workspace/kernel/src/core-lang/vm.c (line 260) - [public] if

```
    if (P->main_fn < 0) {
        return CL_ERR_BADOP;
    }

    vm->frames[0].pc = P->fns[P->main_fn].start;
    vm->frames[0].fn_idx = (uint16_t)P->main_fn;

    for (int i = 0; i < 64; i++) {
        vm->frames[0].slive[i] = 0;
    }

    vm->depth = 1;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 273) - [public] while

```
    while (vm->depth > 0 && vm->err == CL_OK && !vm->halt_req) {
        vm_frame_t *f = &vm->frames[vm->depth - 1];

        if (f->pc >= P->code_len) {
            break;
        }

        cl_insn_t in = P->code[f->pc++];

        int w = cl_wbytes(in.bits);
        bool sign = false;

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 276) - [public] if

```
        if (f->pc >= P->code_len) {
            break;
        }

        cl_insn_t in = P->code[f->pc++];

        int w = cl_wbytes(in.bits);
        bool sign = false;

        cl_val_t a, b, r;

        switch (in.op) {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 287) - [public] switch

```
        switch (in.op) {
            case OP_NOP: break;

            case OP_CONST:
                r = P->consts[in.a];
                cl_mask(r.b, w, in.bits);
                push(vm, &r);
                break;

            case OP_LOAD:
                if (!f->slive[in.a]) {
                    vm->err = CL_ERR_USE_AFTER_FREE;
                }
                // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 292) - [public] cl_mask

```
    cl_mask(r.b, w, in.bits);
    push(vm, &r);
    break;

    case OP_LOAD:
        if (!f->slive[in.a]) {
            vm->err = CL_ERR_USE_AFTER_FREE;
            break;
        }
        push(vm, &f->slots[in.a]);
        break;
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 293) - [public] push

```
    push(vm, &r);
    break;

    case OP_LOAD:
        if (!f->slive[in.a]) {
            vm->err = CL_ERR_USE_AFTER_FREE;
            break;
        }
        push(vm, &f->slots[in.a]);
        break;
```

case OP_STORE:

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 297) - [public] if

```
    if (!f->slive[in.a]) {
        vm->err = CL_ERR_USE_AFTER_FREE;
        break;
    }
    push(vm, &f->slots[in.a]);
    break;
```

case OP_STORE:

```
    a = pop(vm);
    cl_mask(a.b, w, in.bits);
    f->slots[in.a] = a;
    f->slive[in.a] = 1;
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 301) - [public] push

```
    push(vm, &f->slots[in.a]);
    break;
```

case OP_STORE:

```
    a = pop(vm);
    cl_mask(a.b, w, in.bits);
    f->slots[in.a] = a;
    f->slive[in.a] = 1;
    break;
```

case OP_GLOAD:

```
    if (!vm->glive[in.a]) {
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 306) - [public] cl_mask

```
    cl_mask(a.b, w, in.bits);
    f->slots[in.a] = a;
    f->slive[in.a] = 1;
    break;
```

```
case OP_GLOAD:
    if (!vm->glive[in.a]) {
        vm->err = CL_ERR_USE_AFTER_FREE;
        break;
    }
    push(vm, &vm->globals[in.a]);
    break;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 312) - [public] if

```
    if (!vm->glive[in.a]) {
        vm->err = CL_ERR_USE_AFTER_FREE;
        break;
    }
    push(vm, &vm->globals[in.a]);
    break;
```

```
case OP_GSTORE:
    a = pop(vm);
    cl_mask(a.b, w, in.bits);
    vm->globals[in.a] = a;
    vm->glive[in.a] = 1;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 316) - [public] push

```
    push(vm, &vm->globals[in.a]);
    break;
```

```
case OP_GSTORE:
    a = pop(vm);
    cl_mask(a.b, w, in.bits);
    vm->globals[in.a] = a;
    vm->glive[in.a] = 1;
    break;
```

```
case OP_ADD: case OP_SUB: case OP_MUL:
case OP_DIV: case OP_MOD:
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 321) - [public] cl_mask

```
    cl_mask(a.b, w, in.bits);
    vm->globals[in.a] = a;
    vm->glive[in.a] = 1;
    break;
```

```
case OP_ADD: case OP_SUB: case OP_MUL:
case OP_DIV: case OP_MOD:
case OP_AND: case OP_OR: case OP_XOR:
case OP_SHL: case OP_SHR:
    b = pop(vm);
    a = pop(vm);
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 335) - [public] switch

```
switch (in.op) {
case OP_ADD: cl_add(r.b, a.b, b.b, w); break;
case OP_SUB: cl_sub(r.b, a.b, b.b, w); break;
case OP_MUL: cl_mul(r.b, a.b, b.b, w); break;
case OP_AND: for (int i = 0; i < w; i++) r.b[i] = a.b[i] & b.b[i]; break;
case OP_OR:  for (int i = 0; i < w; i++) r.b[i] = a.b[i] | b.b[i]; break;
case OP_XOR: for (int i = 0; i < w; i++) r.b[i] = a.b[i] ^ b.b[i]; break;
case OP_SHL: cl_shl(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_SHR: cl_shr(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_DIV: case OP_MOD: {
    uint8_t q[64], m[64];
    bool zero = true;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 336) - [public] cl_add

```
case OP_ADD: cl_add(r.b, a.b, b.b, w); break;
case OP_SUB: cl_sub(r.b, a.b, b.b, w); break;
case OP_MUL: cl_mul(r.b, a.b, b.b, w); break;
case OP_AND: for (int i = 0; i < w; i++) r.b[i] = a.b[i] & b.b[i]; break;
case OP_OR:  for (int i = 0; i < w; i++) r.b[i] = a.b[i] | b.b[i]; break;
case OP_XOR: for (int i = 0; i < w; i++) r.b[i] = a.b[i] ^ b.b[i]; break;
case OP_SHL: cl_shl(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_SHR: cl_shr(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_DIV: case OP_MOD: {
    uint8_t q[64], m[64];
    bool zero = true;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 337) - [public] cl_sub

```
case OP_SUB: cl_sub(r.b, a.b, b.b, w); break;
case OP_MUL: cl_mul(r.b, a.b, b.b, w); break;
case OP_AND: for (int i = 0; i < w; i++) r.b[i] = a.b[i] & b.b[i]; break;
case OP_OR:  for (int i = 0; i < w; i++) r.b[i] = a.b[i] | b.b[i]; break;
case OP_XOR: for (int i = 0; i < w; i++) r.b[i] = a.b[i] ^ b.b[i]; break;
case OP_SHL: cl_shl(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_SHR: cl_shr(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_DIV: case OP_MOD: {
    uint8_t q[64], m[64];
    bool zero = true;

    for (int i = 0; i < w; i++) {
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 338) - [public] cl_mul

```
case OP_MUL: cl_mul(r.b, a.b, b.b, w); break;
case OP_AND: for (int i = 0; i < w; i++) r.b[i] = a.b[i] & b.b[i]; break;
case OP_OR:  for (int i = 0; i < w; i++) r.b[i] = a.b[i] | b.b[i]; break;
case OP_XOR: for (int i = 0; i < w; i++) r.b[i] = a.b[i] ^ b.b[i]; break;
case OP_SHL: cl_shl(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_SHR: cl_shr(r.b, a.b, (uint32_t)b.b[0], w); break;
case OP_DIV: case OP_MOD: {
    uint8_t q[64], m[64];
    bool zero = true;

    for (int i = 0; i < w; i++) {
        if (b.b[i]) zero = false;
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 342) - [public] cl_shl
    case OP_SHL: cl_shl(r.b, a.b, (uint32_t)b.b[0], w); break;
    case OP_SHR: cl_shr(r.b, a.b, (uint32_t)b.b[0], w); break;
    case OP_DIV: case OP_MOD: {
        uint8_t q[64], m[64];
        bool zero = true;

        for (int i = 0; i < w; i++) {
            if (b.b[i]) zero = false;
        }

        if (zero) {
            vm->err = CL_ERR_DIV_ZERO;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 343) - [public] cl_shr
    case OP_SHR: cl_shr(r.b, a.b, (uint32_t)b.b[0], w); break;
    case OP_DIV: case OP_MOD: {
        uint8_t q[64], m[64];
        bool zero = true;

        for (int i = 0; i < w; i++) {
            if (b.b[i]) zero = false;
        }

        if (zero) {
            vm->err = CL_ERR_DIV_ZERO;
            break;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 352) - [public] if
    if (zero) {
        vm->err = CL_ERR_DIV_ZERO;
        break;
    }

    cl_divmod(a.b, b.b, w, q, m);

    for (int i = 0; i < w; i++) {
        r.b[i] = (in.op == OP_DIV) ? q[i] : m[i];
    }
    break;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 357) - [public] cl_divmod
    cl_divmod(a.b, b.b, w, q, m);

    for (int i = 0; i < w; i++) {
        r.b[i] = (in.op == OP_DIV) ? q[i] : m[i];
    }
    break;
}
}

    cl_mask(r.b, w, in.bits);
    push(vm, &r);
    break;
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 366) - [public] cl_mask

```
    cl_mask(r.b, w, in.bits);
    push(vm, &r);
    break;

    case OP_NEG:
        a = pop(vm);
        {
            uint8_t z[64] = { 0 };
            cl_sub(r.b, z, a.b, w);
            cl_mask(r.b, w, in.bits);
        }
        push(vm, &r);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 367) - [public] push

```
    push(vm, &r);
    break;

    case OP_NEG:
        a = pop(vm);
        {
            uint8_t z[64] = { 0 };
            cl_sub(r.b, z, a.b, w);
            cl_mask(r.b, w, in.bits);
        }
        push(vm, &r);
        break;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 374) - [public] cl_sub

```
    cl_sub(r.b, z, a.b, w);
    cl_mask(r.b, w, in.bits);
}
push(vm, &r);
break;

    case OP_NOT:
        a = pop(vm);
        for (int i = 0; i < w; i++) r.b[i] = ~a.b[i];
        cl_mask(r.b, w, in.bits);
        push(vm, &r);
        break;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 375) - [public] cl_mask

```
    cl_mask(r.b, w, in.bits);
}
push(vm, &r);
break;

    case OP_NOT:
        a = pop(vm);
        for (int i = 0; i < w; i++) r.b[i] = ~a.b[i];
        cl_mask(r.b, w, in.bits);
        push(vm, &r);
        break;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 377) - [public] push

```
    push(vm, &r);
    break;

    case OP_NOT:
        a = pop(vm);
        for (int i = 0; i < w; i++) r.b[i] = ~a.b[i];
        cl_mask(r.b, w, in.bits);
        push(vm, &r);
        break;

    case OP_EQ: case OP_NE: case OP_LT: case OP_GT:
    case OP_LE: case OP_GE:
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 383) - [public] cl_mask

```
    cl_mask(r.b, w, in.bits);
    push(vm, &r);
    break;

    case OP_EQ: case OP_NE: case OP_LT: case OP_GT:
    case OP_LE: case OP_GE:
        b = pop(vm);
        a = pop(vm);
        {
            int c = cl_cmp(a.b, b.b, w, sign);
            bool t = false;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 384) - [public] push

```
    push(vm, &r);
    break;

    case OP_EQ: case OP_NE: case OP_LT: case OP_GT:
    case OP_LE: case OP_GE:
        b = pop(vm);
        a = pop(vm);
        {
            int c = cl_cmp(a.b, b.b, w, sign);
            bool t = false;

            switch (in.op) {
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.c (line 395) - [public] switch

```
        switch (in.op) {
            case OP_EQ: t = (c == 0); break;
            case OP_NE: t = (c != 0); break;
            case OP_LT: t = (c < 0); break;
            case OP_GT: t = (c > 0); break;
            case OP_LE: t = (c <= 0); break;
            case OP_GE: t = (c >= 0); break;
        }

        for (int i = 0; i < 64; i++) r.b[i] = 0;
        r.b[0] = t ? 1 : 0;
    }
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/core-lang/vm.c (line 407) - [public] push
    push(vm, &r);
    break;

    case OP_JMP:
        f->pc = in.c;
        break;

    case OP_JZ:
        a = pop(vm);
        if (a.b[0] == 0) f->pc = in.c;
        break;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 425) - [public] pop
    pop(vm);
    break;

    case OP_FREE:
        if (in.a == 1) {
            vm->glive[in.b] = 0;
            for (int i = 0; i < 64; i++) vm->globals[in.b].b[i] = 0;
        } else {
            f->slive[in.b] = 0;
            for (int i = 0; i < 64; i++) f->slots[in.b].b[i] = 0;
        }
        break;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 429) - [public] if
    if (in.a == 1) {
        vm->glive[in.b] = 0;
        for (int i = 0; i < 64; i++) vm->globals[in.b].b[i] = 0;
    } else {
        f->slive[in.b] = 0;
        for (int i = 0; i < 64; i++) f->slots[in.b].b[i] = 0;
    }
    break;

    case OP_SHARED:
        if (!vm->glive[in.a]) {
            vm->err = CL_ERR_USE_AFTER_FREE;
        }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 439) - [public] if
    if (!vm->glive[in.a]) {
        vm->err = CL_ERR_USE_AFTER_FREE;
    }
    break;

    case OP_EXTERN: {
        if (in.a >= 16 || vm->externs[in.a] == (cl_ext_fn)0) {
            vm->err = CL_ERR_EXTERN;
            break;
        }

        uint64_t args[6] = { 0 };
    // ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/core-lang/vm.c (line 445) - [public] if
    if (in.a >= 16 || vm->externs[in.a] == (cl_ext_fn)0) {
        vm->err = CL_ERR_EXTERN;
        break;
    }

    uint64_t args[6] = { 0 };
    int n = in.b > 6 ? 6 : in.b;

    for (int i = n - 1; i >= 0; i--) {
        cl_val_t v = pop(vm);
        args[i] = val_u64(&v);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 464) - [public] push
    push(vm, &r);
    break;
}

case OP_CALL: {
    if (vm->depth >= 8) {
        vm->err = CL_ERR_DEPTH;
        break;
    }

    cl_val_t argv[8];
    int n = in.b > 8 ? 8 : in.b;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/core-lang/vm.c (line 469) - [public] if
    if (vm->depth >= 8) {
        vm->err = CL_ERR_DEPTH;
        break;
    }

    cl_val_t argv[8];
    int n = in.b > 8 ? 8 : in.b;

    for (int i = n - 1; i >= 0; i--) {
        argv[i] = pop(vm);
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/aux.c (line 50) - [public] if
    if (addr + len > 0x400) {
        return false;
    }

    uint8_t *out = (uint8_t *)buf;

    for (uint32_t i = 0; i < len; i++) {
        out[i] = dpd[addr + i];
    }

    return true;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/displayport/aux.c (line 65) - [public] if
    if (addr + len > 0x400) {
        return false;
    }

    const uint8_t *in = (const uint8_t *)buf;

    for (uint32_t i = 0; i < len; i++) {
        dpcd[addr + i] = in[i];
    }

    if (addr <= DPCD_TRAINING_SET &&
        DPCD_TRAINING_SET < addr + len) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/aux.c (line 79) - [public] if
    if (pat == 1) {
        dpcd[DPCD_LANE_STATUS] = 0x11;
        dpcd[DPCD_LANE_STATUS + 1] = 0x11;
    } else if (pat == 2) {
        train_step++;

        if (train_step >= 2) {
            dpcd[DPCD_LANE_STATUS] = 0x77;
            dpcd[DPCD_LANE_STATUS + 1] = 0x77;
            dpcd[DPCD_SYMBOL_LOCK] = 0x01;
        }
    } else {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/aux.c (line 85) - [public] if
    if (train_step >= 2) {
        dpcd[DPCD_LANE_STATUS] = 0x77;
        dpcd[DPCD_LANE_STATUS + 1] = 0x77;
        dpcd[DPCD_SYMBOL_LOCK] = 0x01;
    }
    } else {
        train_step = 0;
    }
}

return true;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/aux.c (line 100) - [public] if
    if (len > 128) {
        len = 128;
    }

    uint8_t *out = (uint8_t *)buf;

    for (uint32_t i = 0; i < len; i++) {
        out[i] = EDID[i];
    }

    return true;
}

```

kernel_Workspace/kernel/src/displayport/edid.c (line 8) - [public] if

```
*count = 0;

if (b[0] != 0x00 || b[1] != 0xFF || b[7] != 0x00) {
    return false;
}

for (uint32_t d = 0; d < 4; d++) {
    const uint8_t *p = &b[54 + d * 18];

    uint32_t clock = (uint32_t)p[0] | ((uint32_t)p[1] << 8);

    if (clock == 0) {
        continue;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/edid.c (line 17) - [public] if

```
if (clock == 0) {
    continue;
}

uint32_t hact = p[2] | (((uint32_t)p[4] >> 4) << 8);
uint32_t hblk = p[3] | (((uint32_t)p[4] & 0xF) << 8);
uint32_t vact = p[5] | (((uint32_t)p[7] >> 4) << 8);
uint32_t vblk = p[6] | (((uint32_t)p[7] & 0xF) << 8);

uint32_t hsync = p[9] | (((uint32_t)p[10] >> 4) & 0xF) << 8);
uint32_t vsync = (uint32_t)p[10] & 0xF;
uint32_t hoff = p[8] | (((uint32_t)p[11] >> 6) << 8);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/edid.c (line 32) - [public] if

```
if (hact == 0 || vact == 0) {
    continue;
}

if (*count >= max) {
    break;
}

uint32_t htot = hact + hblk;
uint32_t vtot = vact + vblk;

uint32_t refresh = (clock * 1000u) / (htot * vtot);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/edid.c (line 36) - [public] if

```
if (*count >= max) {
    break;
}

uint32_t htot = hact + hblk;
uint32_t vtot = vact + vblk;

uint32_t refresh = (clock * 1000u) / (htot * vtot);

hdmi_mode_t *m = &out[*count];

m->id = 100 + *count;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/init.c (line 13) - [public] if

```
    if (dp_is_ready) {
        return true;
    }

    dp_aux_sink_init();

    if (!dp_link_train()) {
        return false;
    }

    uint8_t raw[128];

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/init.c (line 17) - [public] dp_aux_sink_init

```
dp_aux_sink_init();

    if (!dp_link_train()) {
        return false;
    }

    uint8_t raw[128];

    if (!dp_aux_read_edid(raw, 128)) {
        return false;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/init.c (line 19) - [public] if

```
    if (!dp_link_train()) {
        return false;
    }

    uint8_t raw[128];

    if (!dp_aux_read_edid(raw, 128)) {
        return false;
    }

    if (!edid_parse(raw, MODES, 8, &MODE_COUNT)) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/init.c (line 25) - [public] if

```
    if (!dp_aux_read_edid(raw, 128)) {
        return false;
    }

    if (!edid_parse(raw, MODES, 8, &MODE_COUNT)) {
        return false;
    }

    dp_is_ready = true;

    return true;
}
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/displayport/init.c (line 29) - [public] if
    if (!edid_parse(raw, MODES, 8, &MODE_COUNT)) {
        return false;
    }

    dp_is_ready = true;

    return true;
}

bool dp_ready(void)
{
    return dp_is_ready;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/init.c (line 51) - [public] if
    if (i >= MODE_COUNT) {
        return false;
    }

    *id = MODES[i].id;
    *w = MODES[i].w;
    *h = MODES[i].h;
    *r = MODES[i].refresh;

    return true;
}

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/init.c (line 66) - [public] if
    if (MODES[i].id == id) {
        dp_op_set_fb(0, MODES[i].w, MODES[i].h, MODES[i].w);
        return true;
    }

    return false;
}

void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride)
{
    dp_op_state(fb_phys, w, h, stride);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/init.c (line 67) - [public] dp_op_set_fb
    dp_op_set_fb(0, MODES[i].w, MODES[i].h, MODES[i].w);
    return true;
}

    return false;
}

void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride)
{
    dp_op_state(fb_phys, w, h, stride);
}

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/displayport/init.c (line 77) - [public] dp_op_state
    dp_op_state(fb_phys, w, h, stride);
}

bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride)
{
    dp_op_set_fb(phys, w, h, stride);
    return true;
}

uint64_t dp_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                        uint32_t w, uint32_t h)
{
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/displayport/init.c (line 82) - [public] dp_op_set_fb
    dp_op_set_fb(phys, w, h, stride);
    return true;
}

uint64_t dp_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                        uint32_t w, uint32_t h)
{
    static uint64_t seq = 0;

    dp_op_fill(color, x, y, w, h);

    return ++seq;
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/displayport/init.c (line 91) - [public] dp_op_fill
    dp_op_fill(color, x, y, w, h);

    return ++seq;
}

```

```

kernel_Workspace/kernel/src/displayport/link.c (line 12) - [public] dp_aux_read
    dp_aux_read(DPCD_MAX_LINK_RATE, &max_rate, 1);
    dp_aux_read(DPCD_MAX_LANE_COUNT, &max_lanes, 1);

    uint8_t bw = max_rate < 0x14 ? max_rate : 0x14;
    uint8_t lanes = max_lanes & 0x1F;

    if (lanes > 4) {
        lanes = 4;
    }

    dp_aux_write(DPCD_LINK_BW_SET, &bw, 1);
    dp_aux_write(DPCD_LANE_COUNT_SET, &lanes, 1);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/displayport/link.c (line 13) - [public] dp_aux_read
    dp_aux_read(DPCD_MAX_LANE_COUNT, &max_lanes, 1);

    uint8_t bw = max_rate < 0x14 ? max_rate : 0x14;
    uint8_t lanes = max_lanes & 0x1F;

    if (lanes > 4) {
        lanes = 4;
    }

    dp_aux_write(DPCD_LINK_BW_SET, &bw, 1);
    dp_aux_write(DPCD_LANE_COUNT_SET, &lanes, 1);

    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/displayport/link.c (line 18) - [public] if

```
    if (lanes > 4) {
        lanes = 4;
    }

    dp_aux_write(DPCD_LINK_BW_SET, &bw, 1);
    dp_aux_write(DPCD_LANE_COUNT_SET, &lanes, 1);

    uint8_t tp = 1;
    dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

    uint8_t st[2] = { 0, 0 };

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/link.c (line 22) - [public] dp_aux_write

```
    dp_aux_write(DPCD_LINK_BW_SET, &bw, 1);
    dp_aux_write(DPCD_LANE_COUNT_SET, &lanes, 1);

    uint8_t tp = 1;
    dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

    uint8_t st[2] = { 0, 0 };

    for (int i = 0; i < 5; i++) {
        dp_aux_read(DPCD_LANE_STATUS, st, 2);

        if ((st[0] & 0x11) == 0x11 && (st[1] & 0x11) == 0x11) {
            // ... (truncated, see source file for more)
        }
    }
```

kernel_Workspace/kernel/src/displayport/link.c (line 23) - [public] dp_aux_write

```
    dp_aux_write(DPCD_LANE_COUNT_SET, &lanes, 1);

    uint8_t tp = 1;
    dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

    uint8_t st[2] = { 0, 0 };

    for (int i = 0; i < 5; i++) {
        dp_aux_read(DPCD_LANE_STATUS, st, 2);

        if ((st[0] & 0x11) == 0x11 && (st[1] & 0x11) == 0x11) {
            break;
        }
        // ... (truncated, see source file for more)
    }
```

kernel_Workspace/kernel/src/displayport/link.c (line 26) - [public] dp_aux_write

```
    dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

    uint8_t st[2] = { 0, 0 };

    for (int i = 0; i < 5; i++) {
        dp_aux_read(DPCD_LANE_STATUS, st, 2);

        if ((st[0] & 0x11) == 0x11 && (st[1] & 0x11) == 0x11) {
            break;
        }
    }

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/displayport/link.c (line 31) - [public] dp_aux_read
    dp_aux_read(DPCD_LANE_STATUS, st, 2);

    if ((st[0] & 0x11) == 0x11 && (st[1] & 0x11) == 0x11) {
        break;
    }
}

tp = 2;
dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

for (int i = 0; i < 5; i++) {
    dp_aux_read(DPCD_LANE_STATUS, st, 2);
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/displayport/link.c (line 33) - [public] if
    if ((st[0] & 0x11) == 0x11 && (st[1] & 0x11) == 0x11) {
        break;
    }
}

tp = 2;
dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

for (int i = 0; i < 5; i++) {
    dp_aux_read(DPCD_LANE_STATUS, st, 2);

    if ((st[0] & 0x66) == 0x66 && (st[1] & 0x66) == 0x66) {
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/displayport/link.c (line 39) - [public] dp_aux_write
    dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

for (int i = 0; i < 5; i++) {
    dp_aux_read(DPCD_LANE_STATUS, st, 2);

    if ((st[0] & 0x66) == 0x66 && (st[1] & 0x66) == 0x66) {
        break;
    }
}

uint8_t lock = 0;
dp_aux_read(DPCD_SYMBOL_LOCK, &lock, 1);
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/link.c (line 42) - [public] dp_aux_read
    dp_aux_read(DPCD_LANE_STATUS, st, 2);

    if ((st[0] & 0x66) == 0x66 && (st[1] & 0x66) == 0x66) {
        break;
    }
}

uint8_t lock = 0;
dp_aux_read(DPCD_SYMBOL_LOCK, &lock, 1);

if (lock != 1) {
    return false;
    // ... (truncated, see source file for more)
}

```



```

kernel_Workspace/kernel/src/displayport/link.c (line 44) - [public] if
    if ((st[0] & 0x66) == 0x66 && (st[1] & 0x66) == 0x66) {
        break;
    }
}

uint8_t lock = 0;
dp_aux_read(DPCD_SYMBOL_LOCK, &lock, 1);

if (lock != 1) {
    return false;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/link.c (line 50) - [public] dp_aux_read
dp_aux_read(DPCD_SYMBOL_LOCK, &lock, 1);

if (lock != 1) {
    return false;
}

tp = 0;
dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

switch (bw) {
case 0x0A: cur_rate = 2700; break;
case 0x14: cur_rate = 5400; break;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/link.c (line 52) - [public] if
if (lock != 1) {
    return false;
}

tp = 0;
dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

switch (bw) {
case 0x0A: cur_rate = 2700; break;
case 0x14: cur_rate = 5400; break;
default:   cur_rate = 1620; break;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/link.c (line 57) - [public] dp_aux_write
dp_aux_write(DPCD_TRAINING_SET, &tp, 1);

switch (bw) {
case 0x0A: cur_rate = 2700; break;
case 0x14: cur_rate = 5400; break;
default:   cur_rate = 1620; break;
}

cur_lanes = lanes;

return true;
}
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/displayport/link.c (line 59) - [public] switch

```
    switch (bw) {
        case 0x0A: cur_rate = 2700; break;
        case 0x14: cur_rate = 5400; break;
        default:   cur_rate = 1620; break;
    }

    cur_lanes = lanes;

    return true;
}

void dp_link_info(uint32_t *rate_mbps, uint32_t *lanes)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/displayport/operation.c (line 27) - [public] if

```
    if (fb_phys == 0) {
        return;
    }

    uint32_t *dst = (uint32_t *)arch_phys_to_virt(fb_phys);

    for (uint32_t yy = y; yy < y + fh && yy < h; yy++) {
        for (uint32_t xx = x; xx < x + fw && xx < w; xx++) {
            dst[yy * stride + xx] = color;
        }
    }
}
```

kernel_Workspace/kernel/src/editor/editor.c (line 11) - [public] console_set_enabled

```
extern void console_set_enabled(int on);
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
extern uint32_t k_input_keycode(void);
```

```
#define TOP_BAR 24
#define BOT_BAR 24
#define GUTTER 64
#define CHAR_W 16
#define CHAR_H 16
#define SCALE 2
```

```
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 12) - [public] k_fs_read

```
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
extern uint32_t k_input_keycode(void);
```

```
#define TOP_BAR 24
#define BOT_BAR 24
#define GUTTER 64
#define CHAR_W 16
#define CHAR_H 16
#define SCALE 2
```

```
#define COL_BG      0xFF1E1E1E
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/editor/editor.c (line 13) - [public] k_input_keycode
extern uint32_t k_input_keycode(void);

#define TOP_BAR 24
#define BOT_BAR 24
#define GUTTER 64
#define CHAR_W 16
#define CHAR_H 16
#define SCALE 2

#define COL_BG 0xFF1E1E1E
#define COL_FG 0xFFD4D4D4
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 88) - [public] if
    if (bits & (0x80 >> rx)) {
        uint32_t px = x + (uint32_t)rx * SCALE;
        uint32_t py = y + (uint32_t)ry * SCALE;

        ed_fill(px, py, SCALE, SCALE, color);
    }
}

static void ed_text(uint32_t x, uint32_t y, const char *s, uint32_t color)
{
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 92) - [public] ed_fill
    ed_fill(px, py, SCALE, SCALE, color);
}
}

static void ed_text(uint32_t x, uint32_t y, const char *s, uint32_t color)
{
    while (*s) {
        ed_char(x, y, *s, color);
        x += CHAR_W;
        s++;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 100) - [public] while
    while (*s) {
        ed_char(x, y, *s, color);
        x += CHAR_W;
        s++;
    }
}

static void ed_num(char *buf, int v)
{
    int i = 0;

    if (v == 0) { buf[0] = '0'; buf[1] = 0; return; }
    // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/editor/editor.c (line 101) - [public] ed_char

```
    ed_char(x, y, *s, color);
    x += CHAR_W;
    s++;
}
}
```

```
static void ed_num(char *buf, int v)
{
```

```
    int i = 0;

    if (v == 0) { buf[0] = '0'; buf[1] = 0; return; }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/editor/editor.c (line 111) - [public] if

```
    if (v == 0) { buf[0] = '0'; buf[1] = 0; return; }

    char tmp[12];
    int n = 0;

    while (v > 0 && n < 11) { tmp[n++] = (char)('0' + v % 10); v /= 10; }

    while (n > 0) buf[i++] = tmp[--n];
    buf[i] = 0;
}
```

```
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/editor/editor.c (line 116) - [public] while

```
    while (v > 0 && n < 11) { tmp[n++] = (char)('0' + v % 10); v /= 10; }

    while (n > 0) buf[i++] = tmp[--n];
    buf[i] = 0;
}
```

```
static void log_append(const char *s)
{
```

```
    for (int i = 0; i < 7; i++) {
        for (int k = 0; k < ED_LINE_LEN; k++) {
            log_lines[i][k] = log_lines[i + 1][k];
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/editor/editor.c (line 147) - [public] if

```
    if (v == 0) { tmp[n++] = '0'; }

    while (v > 0 && n < 20) { tmp[n++] = (char)('0' + v % 10); v /= 10; }

    while (n > 0) buf[i++] = tmp[--n];
    buf[i] = 0;

    log_append(buf);

    return 0;
}
```

```
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/editor/editor.c (line 149) - [public] while
    while (v > 0 && n < 20) { tmp[n++] = (char)('0' + v % 10); v /= 10; }

    while (n > 0) buf[i++] = tmp[--n];
    buf[i] = 0;

    log_append(buf);

    return 0;
}

static uint64_t ed_putc(uint64_t v, uint64_t b, uint64_t c,
                        uint64_t d, uint64_t e, uint64_t f)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 154) - [public] log_append
    log_append(buf);

    return 0;
}

static uint64_t ed_putc(uint64_t v, uint64_t b, uint64_t c,
                        uint64_t d, uint64_t e, uint64_t f)
{
    (void)b; (void)c; (void)d; (void)e; (void)f;

    char buf[2] = { (char)v, 0 };
    log_append(buf);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 165) - [public] log_append
    log_append(buf);
    return 0;
}

static uint32_t tok_color(tok_kind_t k)
{
    switch (k) {
        case TK_IF: case TK_ELSE: case TK_WHEN: case TK_CASE:
        case TK_FN: case TK_LET: case TK_STATIC: case TK_DATA:
        case TK_DATAIMPL: case TK_ENUM: case TK_EXTERN:
        case TK_CALL: case TK_SET_FREE:
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 172) - [public] switch
    switch (k) {
        case TK_IF: case TK_ELSE: case TK_WHEN: case TK_CASE:
        case TK_FN: case TK_LET: case TK_STATIC: case TK_DATA:
        case TK_DATAIMPL: case TK_ENUM: case TK_EXTERN:
        case TK_CALL: case TK_SET_FREE:
            return COL_KEY;
        case TK_TYPE: return COL_TYPE;
        case TK_NUM: return COL_NUM;
        case TK_STR:
        case TK_CHAR: return COL_STR;
        case TK_IDENT: return COL_ID;
        default: return COL_OP;
        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 201) - [public] if
    if (tmp[k] == '/' && tmp[k + 1] == '/') { com_at = k; break; }
}

lexer_t l;
token_t toks[64];
size_t n = 0;

cl_lexer_init(&l, tmp, ed_strlen(tmp));

if (com_at >= 0) {
    char cut[ED_LINE_LEN + 1];

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 208) - [public] cl_lexer_init
cl_lexer_init(&l, tmp, ed_strlen(tmp));

if (com_at >= 0) {
    char cut[ED_LINE_LEN + 1];

    for (int k = 0; k < com_at; k++) cut[k] = tmp[k];
    cut[com_at] = 0;

    cl_lexer_init(&l, cut, (size_t)com_at);
}

int pos = 0;
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 210) - [public] if
if (com_at >= 0) {
    char cut[ED_LINE_LEN + 1];

    for (int k = 0; k < com_at; k++) cut[k] = tmp[k];
    cut[com_at] = 0;

    cl_lexer_init(&l, cut, (size_t)com_at);
}

int pos = 0;

if (cl_lex_all(&l, toks, 64, &n) == 0) {
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 216) - [public] cl_lexer_init
    cl_lexer_init(&l, cut, (size_t)com_at);
}

int pos = 0;

if (cl_lex_all(&l, toks, 64, &n) == 0) {
    for (size_t t = 0; t < n && toks[t].kind != TK_EOF; t++) {
        int ts = (int)(toks[t].start - l.src);

        while (pos < ts) {
            ed_char(x + (uint32_t)pos * CHAR_W, y, ' ', COL_FG);
            pos++;
        }
        // ... (truncated, see source file for more)
    }
}

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 221) - [public] if
    if (cl_lex_all(&l, toks, 64, &n) == 0) {
        for (size_t t = 0; t < n && toks[t].kind != TK_EOF; t++) {
            int ts = (int)(toks[t].start - l.src);

            while (pos < ts) {
                ed_char(x + (uint32_t)pos * CHAR_W, y, ' ', COL_FG);
                pos++;
            }

            uint32_t col = tok_color(toks[t].kind);

            for (uint32_t k = 0; k < toks[t].len; k++) {
                // ... (truncated, see source file for more)
            }
        }
    }

kernel_Workspace/kernel/src/editor/editor.c (line 225) - [public] while
    while (pos < ts) {
        ed_char(x + (uint32_t)pos * CHAR_W, y, ' ', COL_FG);
        pos++;
    }

    uint32_t col = tok_color(toks[t].kind);

    for (uint32_t k = 0; k < toks[t].len; k++) {
        ed_char(x + (uint32_t)pos * CHAR_W, y,
            toks[t].start[k], col);
        pos++;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 226) - [public] ed_char
    ed_char(x + (uint32_t)pos * CHAR_W, y, ' ', COL_FG);
    pos++;
}

    uint32_t col = tok_color(toks[t].kind);

    for (uint32_t k = 0; k < toks[t].len; k++) {
        ed_char(x + (uint32_t)pos * CHAR_W, y,
            toks[t].start[k], col);
        pos++;
    }
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 240) - [public] while
    while (tmp[pos] && (com_at < 0 || pos < com_at)) {
        ed_char(x + (uint32_t)pos * CHAR_W, y, tmp[pos], COL_FG);
        pos++;
    }

    if (com_at >= 0) {
        for (int k = com_at; tmp[k]; k++) {
            ed_char(x + (uint32_t)k * CHAR_W, y, tmp[k], COL_COM);
        }
    }
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 241) - [public] ed_char
    ed_char(x + (uint32_t)pos * CHAR_W, y, tmp[pos], COL_FG);
    pos++;
}

    if (com_at >= 0) {
        for (int k = com_at; tmp[k]; k++) {
            ed_char(x + (uint32_t)k * CHAR_W, y, tmp[k], COL_COM);
        }
    }
}

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 245) - [public] if
    if (com_at >= 0) {
        for (int k = com_at; tmp[k]; k++) {
            ed_char(x + (uint32_t)k * CHAR_W, y, tmp[k], COL_COM);
        }
    }
}

    static const char *CURSOR_SPRITE[] = {
        "X.....",
        "XX.....",
        "XoX.....",
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 247) - [public] ed_char
        ed_char(x + (uint32_t)k * CHAR_W, y, tmp[k], COL_COM);
    }
}

    static const char *CURSOR_SPRITE[] = {
        "X.....",
        "XX.....",
        "XoX.....",
        "XooX.....",
        "XoooX.....",
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 285) - [public] if
        if (sx >= 0 && sx < (int)fw && sy >= 0 && sy < (int)fh) {
            int row = ed_flip_rows ? (int)fh - 1 - sy : sy;
            fb[row * fstride + sx] = col;
        }
    }
}

    static void render(void)
    {
        ed_fill(0, 0, fw, fh, COL_BG);
        // ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/editor/editor.c (line 296) - [public] ed_fill
    ed_fill(0, 0, fw, fh, COL_BG);

    ed_fill(0, 0, fw, TOP_BAR, COL_BAR);
    ed_text(8, 4, "TrangEdit [core-lang]", COL_TYPE);
    ed_text(GUTTER + 160, 4, msg, COL_GUTTER);

    int rows = (int)((fh - TOP_BAR - BOT_BAR) / CHAR_H);
    int cols = (int)((fw - GUTTER) / CHAR_W);

    for (int r = 0; r < rows; r++) {
        int row = top_line + r;

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 298) - [public] ed_fill
    ed_fill(0, 0, fw, TOP_BAR, COL_BAR);
    ed_text(8, 4, "TrangEdit [core-lang]", COL_TYPE);
    ed_text(GUTTER + 160, 4, msg, COL_GUTTER);

    int rows = (int)((fh - TOP_BAR - BOT_BAR) / CHAR_H);
    int cols = (int)((fw - GUTTER) / CHAR_W);

    for (int r = 0; r < rows; r++) {
        int row = top_line + r;

        if (row >= nlines) break;

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 299) - [public] ed_text
    ed_text(8, 4, "TrangEdit [core-lang]", COL_TYPE);
    ed_text(GUTTER + 160, 4, msg, COL_GUTTER);

    int rows = (int)((fh - TOP_BAR - BOT_BAR) / CHAR_H);
    int cols = (int)((fw - GUTTER) / CHAR_W);

    for (int r = 0; r < rows; r++) {
        int row = top_line + r;

        if (row >= nlines) break;

        uint32_t y = TOP_BAR + (uint32_t)r * CHAR_H;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 300) - [public] ed_text
    ed_text(GUTTER + 160, 4, msg, COL_GUTTER);

    int rows = (int)((fh - TOP_BAR - BOT_BAR) / CHAR_H);
    int cols = (int)((fw - GUTTER) / CHAR_W);

    for (int r = 0; r < rows; r++) {
        int row = top_line + r;

        if (row >= nlines) break;

        uint32_t y = TOP_BAR + (uint32_t)r * CHAR_H;

        // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 312) - [public] if
    if (row == caret_l) {
        ed_fill(GUTTER, y, (uint32_t)cols * CHAR_W, CHAR_H, 0xFF2D2D2D);
    }

    char nb[12];
    ed_num(nb, row + 1);
    ed_text(8, y, nb, COL_GUTTER);

    draw_hl(row, GUTTER + 4, y);
}

{
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 313) - [public] ed_fill
    ed_fill(GUTTER, y, (uint32_t)cols * CHAR_W, CHAR_H, 0xFF2D2D2D);
}

    char nb[12];
    ed_num(nb, row + 1);
    ed_text(8, y, nb, COL_GUTTER);

    draw_hl(row, GUTTER + 4, y);
}

{
    int r = caret_l - top_line;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 317) - [public] ed_num
    ed_num(nb, row + 1);
    ed_text(8, y, nb, COL_GUTTER);

    draw_hl(row, GUTTER + 4, y);
}

{
    int r = caret_l - top_line;

    if (r >= 0 && r < rows) {
        uint32_t cx = GUTTER + 4 + (uint32_t)caret_c * CHAR_W;
        uint32_t cy = TOP_BAR + (uint32_t)r * CHAR_H;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 318) - [public] ed_text
    ed_text(8, y, nb, COL_GUTTER);

    draw_hl(row, GUTTER + 4, y);
}

{
    int r = caret_l - top_line;

    if (r >= 0 && r < rows) {
        uint32_t cx = GUTTER + 4 + (uint32_t)caret_c * CHAR_W;
        uint32_t cy = TOP_BAR + (uint32_t)r * CHAR_H;

// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/editor/editor.c (line 320) - [public] draw_hl

```
    draw_hl(row, GUTTER + 4, y);
}

{
    int r = caret_l - top_line;

    if (r >= 0 && r < rows) {
        uint32_t cx = GUTTER + 4 + (uint32_t)caret_c * CHAR_W;
        uint32_t cy = TOP_BAR + (uint32_t)r * CHAR_H;

        ed_fill(cx, cy, 2, CHAR_H, COL_CARET);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 326) - [public] if

```
    if (r >= 0 && r < rows) {
        uint32_t cx = GUTTER + 4 + (uint32_t)caret_c * CHAR_W;
        uint32_t cy = TOP_BAR + (uint32_t)r * CHAR_H;

        ed_fill(cx, cy, 2, CHAR_H, COL_CARET);
    }
}

if (show_log) {
    uint32_t ly = fh - BOT_BAR - 8 * CHAR_H;

    ed_fill(0, ly, fw, 8 * CHAR_H, 0xFF101010);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 330) - [public] ed_fill

```
    ed_fill(cx, cy, 2, CHAR_H, COL_CARET);
}

}

if (show_log) {
    uint32_t ly = fh - BOT_BAR - 8 * CHAR_H;

    ed_fill(0, ly, fw, 8 * CHAR_H, 0xFF101010);

    for (int i = 0; i < 8; i++) {
        ed_text(8, ly + (uint32_t)i * CHAR_H, log_lines[i], COL_NUM);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 334) - [public] if

```
    if (show_log) {
        uint32_t ly = fh - BOT_BAR - 8 * CHAR_H;

        ed_fill(0, ly, fw, 8 * CHAR_H, 0xFF101010);

        for (int i = 0; i < 8; i++) {
            ed_text(8, ly + (uint32_t)i * CHAR_H, log_lines[i], COL_NUM);
        }
    }

    ed_fill(0, fh - BOT_BAR, fw, BOT_BAR, COL_BAR);

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/editor/editor.c (line 337) - [public] ed_fill
    ed_fill(0, ly, fw, 8 * CHAR_H, 0xFF101010);

    for (int i = 0; i < 8; i++) {
        ed_text(8, ly + (uint32_t)i * CHAR_H, log_lines[i], COL_NUM);
    }
}

ed_fill(0, fh - BOT_BAR, fw, BOT_BAR, COL_BAR);

char st[64];
int i = 0;
const char *p1 = "ln ";
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 340) - [public] ed_text
    ed_text(8, ly + (uint32_t)i * CHAR_H, log_lines[i], COL_NUM);
}
}

ed_fill(0, fh - BOT_BAR, fw, BOT_BAR, COL_BAR);

char st[64];
int i = 0;
const char *p1 = "ln ";
while (*p1) st[i++] = *p1++;
char tmp[12];
ed_num(tmp, caret_l + 1);
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 344) - [public] ed_fill
    ed_fill(0, fh - BOT_BAR, fw, BOT_BAR, COL_BAR);

char st[64];
int i = 0;
const char *p1 = "ln ";
while (*p1) st[i++] = *p1++;
char tmp[12];
ed_num(tmp, caret_l + 1);
for (int k = 0; tmp[k]; k++) st[i++] = tmp[k];
const char *p2 = " col ";
while (*p2) st[i++] = *p2++;
ed_num(tmp, caret_c + 1);
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 351) - [public] ed_num
    ed_num(tmp, caret_l + 1);
    for (int k = 0; tmp[k]; k++) st[i++] = tmp[k];
    const char *p2 = " col ";
    while (*p2) st[i++] = *p2++;
    ed_num(tmp, caret_c + 1);
    for (int k = 0; tmp[k]; k++) st[i++] = tmp[k];
    st[i] = 0;

    ed_text(8, fh - BOT_BAR + 4, st, COL_FG);
    ed_text(GUTTER + 200, fh - BOT_BAR + 4, msg, COL_GUTTER);

    draw_mouse();
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/editor/editor.c (line 355) - [public] ed_num

```
    ed_num(tmp, caret_c + 1);
    for (int k = 0; tmp[k]; k++) st[i++] = tmp[k];
    st[i] = 0;

    ed_text(8, fh - BOT_BAR + 4, st, COL_FG);
    ed_text(GUTTER + 200, fh - BOT_BAR + 4, msg, COL_GUTTER);

    draw_mouse();
}
```

```
static void ins_char(char ch)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 359) - [public] ed_text

```
    ed_text(8, fh - BOT_BAR + 4, st, COL_FG);
    ed_text(GUTTER + 200, fh - BOT_BAR + 4, msg, COL_GUTTER);

    draw_mouse();
}
```

```
static void ins_char(char ch)
{
    char *ln = lines[caret_l];
    int len = (int)ed_strlen(ln);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 360) - [public] ed_text

```
    ed_text(GUTTER + 200, fh - BOT_BAR + 4, msg, COL_GUTTER);

    draw_mouse();
}
```

```
static void ins_char(char ch)
{
    char *ln = lines[caret_l];
    int len = (int)ed_strlen(ln);

    if (len >= ED_LINE_LEN - 1 || caret_c > len) return;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 362) - [public] draw_mouse

```
    draw_mouse();
}
```

```
static void ins_char(char ch)
{
    char *ln = lines[caret_l];
    int len = (int)ed_strlen(ln);

    if (len >= ED_LINE_LEN - 1 || caret_c > len) return;

    for (int i = len; i >= caret_c; i--) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 413) - [public] if

```
    if (caret_c > 0) {
        int len = (int)ed_strlen(ln);

        for (int i = caret_c - 1; i < len; i++) {
            ln[i] = ln[i + 1];
        }

        caret_c--;
    } else if (caret_l > 0) {
        int plen = (int)ed_strlen(lines[caret_l - 1]);
        int slen = (int)ed_strlen(ln);

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 425) - [public] if

```
    if (plen + slen < ED_LINE_LEN - 1) {
        for (int k = 0; k <= slen; k++) {
            lines[caret_l - 1][plen + k] = ln[k];
        }

        for (int i = caret_l; i < nlines - 1; i++) {
            for (int k = 0; k < ED_LINE_LEN; k++) {
                lines[i][k] = lines[i + 1][k];
            }
        }

        nlines--;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 448) - [public] if

```
    if (caret_c < len) {
        for (int i = caret_c; i < len; i++) {
            ln[i] = ln[i + 1];
        }
    }

    static void clamp_scroll(int rows)
    {
        if (caret_l < top_line) top_line = caret_l;
        if (caret_l >= top_line + rows) top_line = caret_l - rows + 1;
        if (top_line < 0) top_line = 0;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 485) - [public] render

```
    render();

    ed_arena.pos = 0;

    int n = flatten();

    uint32_t el = 0;
    const char *em = (void *)0;

    cl_prog_t *P = cl_compile_source(srcbuf, (size_t)n, &ed_arena, &el, &em);

    if (P == (void *)0) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 496) - [public] if

```
if (P == (void *)0) {
    const char *pre = "ERR ln ";
    int i = 0;
    while (pre[i]) { msg[i] = pre[i]; i++; }
    char tmp[12];
    ed_num(tmp, (int)el);
    for (int k = 0; tmp[k]; k++) msg[i++] = tmp[k];
    msg[i++] = ' ';
    if (em) for (int k = 0; em[k] && i < ED_LINE_LEN - 1; k++) msg[i++] = em[k];
    msg[i] = 0;

    if (em) log_append(em);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 499) - [public] while

```
while (pre[i]) { msg[i] = pre[i]; i++; }
char tmp[12];
ed_num(tmp, (int)el);
for (int k = 0; tmp[k]; k++) msg[i++] = tmp[k];
msg[i++] = ' ';
if (em) for (int k = 0; em[k] && i < ED_LINE_LEN - 1; k++) msg[i++] = em[k];
msg[i] = 0;

if (em) log_append(em);
return;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 501) - [public] ed_num

```
ed_num(tmp, (int)el);
for (int k = 0; tmp[k]; k++) msg[i++] = tmp[k];
msg[i++] = ' ';
if (em) for (int k = 0; em[k] && i < ED_LINE_LEN - 1; k++) msg[i++] = em[k];
msg[i] = 0;

if (em) log_append(em);
return;
}

cl_vm_init(&ed_vm, P);
cl_bridge_init(&ed_vm, 0);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 507) - [public] if

```
if (em) log_append(em);
return;
}

cl_vm_init(&ed_vm, P);
cl_bridge_init(&ed_vm, 0);

cl_vm_register_extern(&ed_vm, "put", ed_put);
cl_vm_register_extern(&ed_vm, "putc", ed_putc);

log_append("== run ==");

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 511) - [public] cl_vm_init

```
cl_vm_init(&ed_vm, P);
cl_bridge_init(&ed_vm, 0);

cl_vm_register_extern(&ed_vm, "put", ed_put);
cl_vm_register_extern(&ed_vm, "putc", ed_putc);

log_append("== run ==");

cl_vm_err_t e = cl_vm_run(&ed_vm);

const char *ok = (e == CL_OK) ? "run: ok" : "run: vm error";
int i = 0;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 512) - [public] cl_bridge_init

```
cl_bridge_init(&ed_vm, 0);

cl_vm_register_extern(&ed_vm, "put", ed_put);
cl_vm_register_extern(&ed_vm, "putc", ed_putc);

log_append("== run ==");

cl_vm_err_t e = cl_vm_run(&ed_vm);

const char *ok = (e == CL_OK) ? "run: ok" : "run: vm error";
int i = 0;
while (ok[i]) { msg[i] = ok[i]; i++; }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 514) - [public] cl_vm_register_extern

```
cl_vm_register_extern(&ed_vm, "put", ed_put);
cl_vm_register_extern(&ed_vm, "putc", ed_putc);

log_append("== run ==");

cl_vm_err_t e = cl_vm_run(&ed_vm);

const char *ok = (e == CL_OK) ? "run: ok" : "run: vm error";
int i = 0;
while (ok[i]) { msg[i] = ok[i]; i++; }
msg[i] = 0;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 515) - [public] cl_vm_register_extern

```
cl_vm_register_extern(&ed_vm, "putc", ed_putc);

log_append("== run ==");

cl_vm_err_t e = cl_vm_run(&ed_vm);

const char *ok = (e == CL_OK) ? "run: ok" : "run: vm error";
int i = 0;
while (ok[i]) { msg[i] = ok[i]; i++; }
msg[i] = 0;

show_log = 1;
// ... (truncated, see source file for more)
```



```

kernel_Workspace/kernel/src/editor/editor.c (line 517) - [public] log_append
    log_append("== run ==");

    cl_vm_err_t e = cl_vm_run(&ed_vm);

    const char *ok = (e == CL_OK) ? "run: ok" : "run: vm error";
    int i = 0;
    while (ok[i]) { msg[i] = ok[i]; i++; }
    msg[i] = 0;

    show_log = 1;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 523) - [public] while
    while (ok[i]) { msg[i] = ok[i]; i++; }
    msg[i] = 0;

    show_log = 1;
}

int editor_run(const char *path)
{
    uint32_t w = 0, h = 0, s = 0;
    uint64_t base = 0;
    int32_t fliprows = 0;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 536) - [public] if
    if (gfx_fb_info_raw(&w, &h, &s, &base, &fliprows) != 0 || base == 0) {
        return -1;
    }

    fb = (uint32_t *) (uintptr_t) base;
    fw = w; fh = h; fstride = s;
    ed_flip_rows = fliprows;

    nlines = 1;
    lines[0][0] = 0;
    caret_l = caret_c = 0;
    top_line = 0;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 549) - [public] if
    if (path != (void *) 0) {
        static char rbuf[ED_MAX_LINES * ED_LINE_LEN];

        int32_t n = k_fs_read(path, rbuf, sizeof(rbuf) - 1);

        if (n > 0) {
            rbuf[n] = 0;

            nlines = 0;
            int col = 0;

            for (int i = 0; i < n && nlines < ED_MAX_LINES; i++) {
                // ... (truncated, see source file for more)
            }
        }
    }
}

```

kernel_Workspace/kernel/src/editor/editor.c (line 554) - [public] if

```
    if (n > 0) {
        rbuf[n] = 0;

        nlines = 0;
        int col = 0;

        for (int i = 0; i < n && nlines < ED_MAX_LINES; i++) {
            char c = rbuf[i];

            if (c == '\n') {
                lines[nlines][col] = 0;
                nlines++;
            }
        }
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 563) - [public] if

```
        if (c == '\n') {
            lines[nlines][col] = 0;
            nlines++;
            col = 0;
        } else if (col < ED_LINE_LEN - 1) {
            lines[nlines][col++] = c;
        }
    }

    if (col > 0 || nlines == 0) {
        lines[nlines][col] = 0;
        nlines++;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 572) - [public] if

```
        if (col > 0 || nlines == 0) {
            lines[nlines][col] = 0;
            nlines++;
        }
    }
}
```

```
console_set_enabled(0);
mouse_init();
```

```
int running = 1;
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 579) - [public] console_set_enabled

```
console_set_enabled(0);
mouse_init();
```

```
int running = 1;
```

```
while (running) {
    int dirty = 0;
    int rows = (int)((fh - TOP_BAR - BOT_BAR) / CHAR_H);
```

```
    int dx = 0, dy = 0, dz = 0, btn = 0;
```

```
    if (mouse_poll(&dx, &dy, &dz, &btn)) {
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/editor/editor.c (line 580) - [public] mouse_init
    mouse_init();

    int running = 1;

    while (running) {
        int dirty = 0;
        int rows = (int)((fh - TOP_BAR - BOT_BAR) / CHAR_H);

        int dx = 0, dy = 0, dz = 0, btn = 0;

        if (mouse_poll(&dx, &dy, &dz, &btn)) {
            mx += dx;
            // ... (truncated, see source file for more)
        }

kernel_Workspace/kernel/src/editor/editor.c (line 584) - [public] while
    while (running) {
        int dirty = 0;
        int rows = (int)((fh - TOP_BAR - BOT_BAR) / CHAR_H);

        int dx = 0, dy = 0, dz = 0, btn = 0;

        if (mouse_poll(&dx, &dy, &dz, &btn)) {
            mx += dx;
            my += dy;

            if (mx < 0) mx = 0;
            if (my < 0) my = 0;
            // ... (truncated, see source file for more)
        }

kernel_Workspace/kernel/src/editor/editor.c (line 590) - [public] if
        if (mouse_poll(&dx, &dy, &dz, &btn)) {
            mx += dx;
            my += dy;

            if (mx < 0) mx = 0;
            if (my < 0) my = 0;
            if (mx >= (int)fw) mx = (int)fw - 1;
            if (my >= (int)fh) my = (int)fh - 1;

            if (dz != 0) {
                top_line += dz;
                clamp_scroll(rows);
                // ... (truncated, see source file for more)
            }

kernel_Workspace/kernel/src/editor/editor.c (line 599) - [public] if
            if (dz != 0) {
                top_line += dz;
                clamp_scroll(rows);
                dirty = 1;
            }

            if (btn & 1 && !(mbuttons & 1)) {
                if (my >= (int)TOP_BAR && my < (int)(fh - BOT_BAR)) {
                    int row = top_line + (my - (int)TOP_BAR) / CHAR_H;
                    int col = (mx - (int)GUTTER - 4) / CHAR_W;

                    if (row >= 0 && row < nlines) caret_l = row;
                    // ... (truncated, see source file for more)
                }
            }
        }
    }
}

```

```

kernel_Workspace/kernel/src/editor/editor.c (line 601) - [public] clamp_scroll
    clamp_scroll(rows);
    dirty = 1;
}

    if (btn & 1 && !(mbuttons & 1)) {
        if (my >= (int)TOP_BAR && my < (int)(fh - BOT_BAR)) {
            int row = top_line + (my - (int)TOP_BAR) / CHAR_H;
            int col = (mx - (int)GUTTER - 4) / CHAR_W;

            if (row >= 0 && row < nlines) caret_l = row;

            int len = (int)ed_strlen(lines[caret_l]);
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 605) - [public] if
    if (btn & 1 && !(mbuttons & 1)) {
        if (my >= (int)TOP_BAR && my < (int)(fh - BOT_BAR)) {
            int row = top_line + (my - (int)TOP_BAR) / CHAR_H;
            int col = (mx - (int)GUTTER - 4) / CHAR_W;

            if (row >= 0 && row < nlines) caret_l = row;

            int len = (int)ed_strlen(lines[caret_l]);

            if (col < 0) col = 0;
            if (col > len) col = len;

            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 606) - [public] if
    if (my >= (int)TOP_BAR && my < (int)(fh - BOT_BAR)) {
        int row = top_line + (my - (int)TOP_BAR) / CHAR_H;
        int col = (mx - (int)GUTTER - 4) / CHAR_W;

        if (row >= 0 && row < nlines) caret_l = row;

        int len = (int)ed_strlen(lines[caret_l]);

        if (col < 0) col = 0;
        if (col > len) col = len;

        caret_c = col;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/editor.c (line 618) - [public] clamp_scroll
    clamp_scroll(rows);
    dirty = 1;
    }
}

    mbuttons = btn;
    dirty = 1;
}

    uint32_t k = k_input_keycode();

    if (k != 0) {
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/editor/editor.c (line 629) - [public] if

```
if (k != 0) {
    switch (k) {
        case EDK_ESC:      running = 0; break;
        case EDK_ENTER:    do_enter(); dirty = 1; break;
        case EDK_BACKSPACE: do_backspace(); dirty = 1; break;
        case EDK_DELETE:   do_delete(); dirty = 1; break;
        case EDK_TAB:      ins_char(' '); ins_char(' ');
                          ins_char(' '); ins_char(' '); dirty = 1; break;
        case EDK_LEFT:     if (caret_c > 0) caret_c--; dirty = 1; break;
        case EDK_RIGHT:    caret_c++; dirty = 1; break;
        case EDK_UP:       if (caret_l > 0) caret_l--; dirty = 1; break;
        case EDK_DOWN:     if (caret_l < nlines - 1) caret_l++; dirty = 1; break;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 630) - [public] switch

```
switch (k) {
    case EDK_ESC:      running = 0; break;
    case EDK_ENTER:    do_enter(); dirty = 1; break;
    case EDK_BACKSPACE: do_backspace(); dirty = 1; break;
    case EDK_DELETE:   do_delete(); dirty = 1; break;
    case EDK_TAB:      ins_char(' '); ins_char(' ');
                      ins_char(' '); ins_char(' '); dirty = 1; break;
    case EDK_LEFT:     if (caret_c > 0) caret_c--; dirty = 1; break;
    case EDK_RIGHT:    caret_c++; dirty = 1; break;
    case EDK_UP:       if (caret_l > 0) caret_l--; dirty = 1; break;
    case EDK_DOWN:     if (caret_l < nlines - 1) caret_l++; dirty = 1; break;
    case EDK_HOME:     caret_c = 0; dirty = 1; break;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 632) - [public] do_enter

```
case EDK_ENTER:    do_enter(); dirty = 1; break;
case EDK_BACKSPACE: do_backspace(); dirty = 1; break;
case EDK_DELETE:   do_delete(); dirty = 1; break;
case EDK_TAB:      ins_char(' '); ins_char(' ');
                  ins_char(' '); ins_char(' '); dirty = 1; break;
case EDK_LEFT:     if (caret_c > 0) caret_c--; dirty = 1; break;
case EDK_RIGHT:    caret_c++; dirty = 1; break;
case EDK_UP:       if (caret_l > 0) caret_l--; dirty = 1; break;
case EDK_DOWN:     if (caret_l < nlines - 1) caret_l++; dirty = 1; break;
case EDK_HOME:     caret_c = 0; dirty = 1; break;
case EDK_END:      caret_c = (int)ed_strlen(lines[caret_l]); dirty = 1; break;
case EDK_F5:       ed_compile_run(); dirty = 1; break;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 633) - [public] do_backspace

```
case EDK_BACKSPACE: do_backspace(); dirty = 1; break;
case EDK_DELETE:   do_delete(); dirty = 1; break;
case EDK_TAB:      ins_char(' '); ins_char(' ');
                  ins_char(' '); ins_char(' '); dirty = 1; break;
case EDK_LEFT:     if (caret_c > 0) caret_c--; dirty = 1; break;
case EDK_RIGHT:    caret_c++; dirty = 1; break;
case EDK_UP:       if (caret_l > 0) caret_l--; dirty = 1; break;
case EDK_DOWN:     if (caret_l < nlines - 1) caret_l++; dirty = 1; break;
case EDK_HOME:     caret_c = 0; dirty = 1; break;
case EDK_END:      caret_c = (int)ed_strlen(lines[caret_l]); dirty = 1; break;
case EDK_F5:       ed_compile_run(); dirty = 1; break;
case EDK_F8:       show_log = !show_log; dirty = 1; break;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 634) - [public] do_delete

```
case EDK_DELETE:    do_delete(); dirty = 1; break;
case EDK_TAB:       ins_char(' '); ins_char(' ');
                    ins_char(' '); ins_char(' '); dirty = 1; break;
case EDK_LEFT:      if (caret_c > 0) caret_c--; dirty = 1; break;
case EDK_RIGHT:     caret_c++; dirty = 1; break;
case EDK_UP:        if (caret_l > 0) caret_l--; dirty = 1; break;
case EDK_DOWN:      if (caret_l < nlines - 1) caret_l++; dirty = 1; break;
case EDK_HOME:      caret_c = 0; dirty = 1; break;
case EDK_END:       caret_c = (int)ed_strlen(lines[caret_l]); dirty = 1; break;
case EDK_F5:        ed_compile_run(); dirty = 1; break;
case EDK_F8:        show_log = !show_log; dirty = 1; break;
default:
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 635) - [public] ins_char

```
case EDK_TAB:       ins_char(' '); ins_char(' ');
                    ins_char(' '); ins_char(' '); dirty = 1; break;
case EDK_LEFT:      if (caret_c > 0) caret_c--; dirty = 1; break;
case EDK_RIGHT:     caret_c++; dirty = 1; break;
case EDK_UP:        if (caret_l > 0) caret_l--; dirty = 1; break;
case EDK_DOWN:      if (caret_l < nlines - 1) caret_l++; dirty = 1; break;
case EDK_HOME:      caret_c = 0; dirty = 1; break;
case EDK_END:       caret_c = (int)ed_strlen(lines[caret_l]); dirty = 1; break;
case EDK_F5:        ed_compile_run(); dirty = 1; break;
case EDK_F8:        show_log = !show_log; dirty = 1; break;
default:
    if (k >= 32 && k < 0x100) {
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 636) - [public] ins_char

```
                    ins_char(' '); ins_char(' '); dirty = 1; break;
case EDK_LEFT:      if (caret_c > 0) caret_c--; dirty = 1; break;
case EDK_RIGHT:     caret_c++; dirty = 1; break;
case EDK_UP:        if (caret_l > 0) caret_l--; dirty = 1; break;
case EDK_DOWN:      if (caret_l < nlines - 1) caret_l++; dirty = 1; break;
case EDK_HOME:      caret_c = 0; dirty = 1; break;
case EDK_END:       caret_c = (int)ed_strlen(lines[caret_l]); dirty = 1; break;
case EDK_F5:        ed_compile_run(); dirty = 1; break;
case EDK_F8:        show_log = !show_log; dirty = 1; break;
default:
    if (k >= 32 && k < 0x100) {
        ins_char((char)k);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 643) - [public] ed_compile_run

```
case EDK_F5:        ed_compile_run(); dirty = 1; break;
case EDK_F8:        show_log = !show_log; dirty = 1; break;
default:
    if (k >= 32 && k < 0x100) {
        ins_char((char)k);
        dirty = 1;
    }
    break;
}

clamp_scroll(rows);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 646) - [public] if

```
        if (k >= 32 && k < 0x100) {
            ins_char((char)k);
            dirty = 1;
        }
        break;
    }

    clamp_scroll(rows);
}

if (dirty) {
    render();
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 647) - [public] ins_char

```
        ins_char((char)k);
        dirty = 1;
    }
    break;
}

    clamp_scroll(rows);
}

if (dirty) {
    render();
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 653) - [public] clamp_scroll

```
    clamp_scroll(rows);
}

if (dirty) {
    render();
}

    __asm__ volatile("pause");
}

console_set_enabled(1);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/editor.c (line 656) - [public] if

```
    if (dirty) {
        render();
    }

    __asm__ volatile("pause");
}

console_set_enabled(1);

return 0;
}
```

kernel_Workspace/kernel/src/editor/editor.c (line 657) - [public] render

```
    render();
}

    __asm__ volatile("pause");
}

console_set_enabled(1);

return 0;
}
```

kernel_Workspace/kernel/src/editor/editor.c (line 660) - [public] volatile

```
    __asm__ volatile("pause");
}

console_set_enabled(1);

return 0;
}
```

kernel_Workspace/kernel/src/editor/editor.c (line 663) - [public] console_set_enabled

```
    console_set_enabled(1);

    return 0;
}
```

kernel_Workspace/kernel/src/editor/mouse.c (line 5) - [public] volatile

```
    __asm__ volatile("outb %0, %1" :: "a"(val), "Nd"(port));
}

static inline uint8_t inb(uint16_t port)
{
    uint8_t v;
    __asm__ volatile("inb %1, %0" : "=a"(v) : "Nd"(port));
    return v;
}
```

```
static void wait_write(void)
{
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/editor/mouse.c (line 11) - [public] volatile

```
    __asm__ volatile("inb %1, %0" : "=a"(v) : "Nd"(port));
    return v;
}
```

```
static void wait_write(void)
{
    while (inb(0x64) & 0x02) { }
}
```

```
static void wait_read(void)
{
    while (!(inb(0x64) & 0x01)) { }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/editor/mouse.c (line 17) - [public] while

```
    while (inb(0x64) & 0x02) { }
}
```

```
static void wait_read(void)
{
    while (!(inb(0x64) & 0x01)) { }
}
```

```
static void mouse_cmd(uint8_t cmd)
{
    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, cmd);
    // ... (truncated, see source file for more)
}
```


kernel_Workspace/kernel/src/editor/mouse.c (line 22) - [public] while

```
    while (!(inb(0x64) & 0x01)) { }
}

static void mouse_cmd(uint8_t cmd)
{
    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, cmd);
    wait_read();  inb(0x60);
}

static void mouse_rate(uint8_t r)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/mouse.c (line 27) - [public] wait_write

```
    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, cmd);
    wait_read();  inb(0x60);
}

static void mouse_rate(uint8_t r)
{
    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, 0xF3);
    wait_read();  inb(0x60);

    wait_write(); outb(0x64, 0xD4);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/mouse.c (line 28) - [public] wait_write

```
    wait_write(); outb(0x60, cmd);
    wait_read();  inb(0x60);
}

static void mouse_rate(uint8_t r)
{
    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, 0xF3);
    wait_read();  inb(0x60);

    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, r);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/mouse.c (line 29) - [public] wait_read

```
    wait_read();  inb(0x60);
}

static void mouse_rate(uint8_t r)
{
    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, 0xF3);
    wait_read();  inb(0x60);

    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, r);
    wait_read();  inb(0x60);
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/editor/mouse.c (line 34) - [public] wait_write
    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, 0xF3);
    wait_read();  inb(0x60);

    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, r);
    wait_read();  inb(0x60);
}

void mouse_init(void)
{
    wait_write(); outb(0x64, 0xA8);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 35) - [public] wait_write
    wait_write(); outb(0x60, 0xF3);
    wait_read();  inb(0x60);

    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, r);
    wait_read();  inb(0x60);
}

void mouse_init(void)
{
    wait_write(); outb(0x64, 0xA8);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 36) - [public] wait_read
    wait_read();  inb(0x60);

    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, r);
    wait_read();  inb(0x60);
}

void mouse_init(void)
{
    wait_write(); outb(0x64, 0xA8);

    mouse_cmd(0xF6);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 38) - [public] wait_write
    wait_write(); outb(0x64, 0xD4);
    wait_write(); outb(0x60, r);
    wait_read();  inb(0x60);
}

void mouse_init(void)
{
    wait_write(); outb(0x64, 0xA8);

    mouse_cmd(0xF6);

    mouse_rate(200);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/editor/mouse.c (line 39) - [public] wait_write
    wait_write(); outb(0x60, r);
    wait_read(); inb(0x60);
}

void mouse_init(void)
{
    wait_write(); outb(0x64, 0xA8);

    mouse_cmd(0xF6);

    mouse_rate(200);
    mouse_rate(100);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 40) - [public] wait_read
    wait_read(); inb(0x60);
}

void mouse_init(void)
{
    wait_write(); outb(0x64, 0xA8);

    mouse_cmd(0xF6);

    mouse_rate(200);
    mouse_rate(100);
    mouse_rate(80);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 45) - [public] wait_write
    wait_write(); outb(0x64, 0xA8);

    mouse_cmd(0xF6);

    mouse_rate(200);
    mouse_rate(100);
    mouse_rate(80);

    mouse_cmd(0xF4);
}

int mouse_poll(int *dx, int *dy, int *dz, int *buttons)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 47) - [public] mouse_cmd
    mouse_cmd(0xF6);

    mouse_rate(200);
    mouse_rate(100);
    mouse_rate(80);

    mouse_cmd(0xF4);
}

int mouse_poll(int *dx, int *dy, int *dz, int *buttons)
{
    static uint8_t pkt[4];
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/editor/mouse.c (line 49) - [public] mouse_rate

```
    mouse_rate(200);
    mouse_rate(100);
    mouse_rate(80);

    mouse_cmd(0xF4);
}

int mouse_poll(int *dx, int *dy, int *dz, int *buttons)
{
    static uint8_t pkt[4];
    static int idx = 0;
    int got = 0;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/mouse.c (line 50) - [public] mouse_rate

```
    mouse_rate(100);
    mouse_rate(80);

    mouse_cmd(0xF4);
}

int mouse_poll(int *dx, int *dy, int *dz, int *buttons)
{
    static uint8_t pkt[4];
    static int idx = 0;
    int got = 0;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/mouse.c (line 51) - [public] mouse_rate

```
    mouse_rate(80);

    mouse_cmd(0xF4);
}

int mouse_poll(int *dx, int *dy, int *dz, int *buttons)
{
    static uint8_t pkt[4];
    static int idx = 0;
    int got = 0;

    while (inb(0x64) & 0x01) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/editor/mouse.c (line 53) - [public] mouse_cmd

```
    mouse_cmd(0xF4);
}

int mouse_poll(int *dx, int *dy, int *dz, int *buttons)
{
    static uint8_t pkt[4];
    static int idx = 0;
    int got = 0;

    while (inb(0x64) & 0x01) {
        uint8_t st = inb(0x64);
        uint8_t d = inb(0x60);
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/editor/mouse.c (line 62) - [public] while
    while (inb(0x64) & 0x01) {
        uint8_t st = inb(0x64);
        uint8_t d = inb(0x60);

        if (!(st & 0x20)) {
            continue;
        }

        if (idx == 0 && !(d & 0x08)) {
            continue;
        }

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 66) - [public] if
    if (!(st & 0x20)) {
        continue;
    }

    if (idx == 0 && !(d & 0x08)) {
        continue;
    }

    pkt[idx++] = d;

    if (idx == 4) {
        idx = 0;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 70) - [public] if
    if (idx == 0 && !(d & 0x08)) {
        continue;
    }

    pkt[idx++] = d;

    if (idx == 4) {
        idx = 0;

        *buttons = pkt[0] & 0x07;

        *dx = (int)pkt[1] - (int)((pkt[0] << 4) & 0x100);
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/editor/mouse.c (line 76) - [public] if
    if (idx == 4) {
        idx = 0;

        *buttons = pkt[0] & 0x07;

        *dx = (int)pkt[1] - (int)((pkt[0] << 4) & 0x100);
        *dy = (int)pkt[2] - (int)((pkt[0] << 3) & 0x100);
        *dz = (int)(int8_t)pkt[3];

        got = 1;
    }
}
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/hdmi/init.c (line 8) - [public] if

```
    if (hdmi_is_ready) {
        return true;
    }

    if (fb_phys == 0 || w == 0 || h == 0 || stride < w) {
        return false;
    }

    hdmi_op_set_fb(fb_phys, w, h, stride);
    if (!hdmi_op_ready()) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/init.c (line 12) - [public] if

```
    if (fb_phys == 0 || w == 0 || h == 0 || stride < w) {
        return false;
    }

    hdmi_op_set_fb(fb_phys, w, h, stride);
    if (!hdmi_op_ready()) {
        return false;
    }

    const hdmi_mode_t *mode = hdmi_mode_find(w, h, 60);
    if (mode != NULL && !hdmi_mode_apply(mode)) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/init.c (line 16) - [public] hdmi_op_set_fb

```
    hdmi_op_set_fb(fb_phys, w, h, stride);
    if (!hdmi_op_ready()) {
        return false;
    }

    const hdmi_mode_t *mode = hdmi_mode_find(w, h, 60);
    if (mode != NULL && !hdmi_mode_apply(mode)) {
        return false;
    }

    hdmi_is_ready = true;
    return true;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/init.c (line 17) - [public] if

```
    if (!hdmi_op_ready()) {
        return false;
    }

    const hdmi_mode_t *mode = hdmi_mode_find(w, h, 60);
    if (mode != NULL && !hdmi_mode_apply(mode)) {
        return false;
    }

    hdmi_is_ready = true;
    return true;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/init.c (line 22) - [public] if

```
    if (mode != NULL && !hdmi_mode_apply(mode)) {
        return false;
    }

    hdmi_is_ready = true;
    return true;
}

bool hdmi_ready(void)
{
    return hdmi_is_ready && hdmi_op_ready();
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/init.c (line 32) - [public] hdmi_op_ready

```
    return hdmi_is_ready && hdmi_op_ready();
}
```

```
void hdmi_caps(hdmi_caps_t *out)
{
    hdmi_op_state(out);
}
```

```
bool hdmi_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride)
{
    if (phys == 0 || w == 0 || h == 0 || stride < w) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/init.c (line 37) - [public] hdmi_op_state

```
    hdmi_op_state(out);
}

bool hdmi_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride)
{
    if (phys == 0 || w == 0 || h == 0 || stride < w) {
        return false;
    }

    hdmi_op_set_fb(phys, w, h, stride);
    hdmi_is_ready = hdmi_op_ready();
    return hdmi_is_ready;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/init.c (line 42) - [public] if

```
    if (phys == 0 || w == 0 || h == 0 || stride < w) {
        return false;
    }

    hdmi_op_set_fb(phys, w, h, stride);
    hdmi_is_ready = hdmi_op_ready();
    return hdmi_is_ready;
}

bool hdmi_mode_set_by_id(uint32_t id)
{
    if (!hdmi_ready()) {
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/hdmi/init.c (line 46) - [public] hdmi_op_set_fb
    hdmi_op_set_fb(phys, w, h, stride);
    hdmi_is_ready = hdmi_op_ready();
    return hdmi_is_ready;
}

bool hdmi_mode_set_by_id(uint32_t id)
{
    if (!hdmi_ready()) {
        return false;
    }

    const hdmi_mode_t *mode = hdmi_mode_by_id(id);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/init.c (line 53) - [public] if
    if (!hdmi_ready()) {
        return false;
    }

    const hdmi_mode_t *mode = hdmi_mode_by_id(id);
    hdmi_caps_t caps;
    hdmi_op_state(&caps);
    if (mode == NULL || mode->w != caps.w || mode->h != caps.h) {
        return false;
    }

    return hdmi_mode_apply(mode);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/init.c (line 59) - [public] hdmi_op_state
    hdmi_op_state(&caps);
    if (mode == NULL || mode->w != caps.w || mode->h != caps.h) {
        return false;
    }

    return hdmi_mode_apply(mode);
}

bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    const hdmi_mode_t *mode = hdmi_mode_current();
    if (mode == NULL || id == NULL || w == NULL || h == NULL || r == NULL) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/init.c (line 60) - [public] if
    if (mode == NULL || mode->w != caps.w || mode->h != caps.h) {
        return false;
    }

    return hdmi_mode_apply(mode);
}

bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    const hdmi_mode_t *mode = hdmi_mode_current();
    if (mode == NULL || id == NULL || w == NULL || h == NULL || r == NULL) {
        return false;
    }
    // ... (truncated, see source file for more)

```


kernel_Workspace/kernel/src/hdmi/init.c (line 64) - [public] hdmi_mode_apply

```
    return hdmi_mode_apply(mode);
}

bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    const hdmi_mode_t *mode = hdmi_mode_current();
    if (mode == NULL || id == NULL || w == NULL || h == NULL || r == NULL) {
        return false;
    }

    *id = mode->id;
    *w = mode->w;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/init.c (line 70) - [public] if

```
    if (mode == NULL || id == NULL || w == NULL || h == NULL || r == NULL) {
        return false;
    }

    *id = mode->id;
    *w = mode->w;
    *h = mode->h;
    *r = mode->refresh;
    return true;
}
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 9) - [public] if

```
    if (owner == 0) {
        return false;
    }

    uint32_t expected = 0;
    if (atomic_compare_exchange_strong(&port_owner, &expected, owner)) {
        return true;
    }

    return expected == owner;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 14) - [public] if

```
    if (atomic_compare_exchange_strong(&port_owner, &expected, owner)) {
        return true;
    }

    return expected == owner;
}

bool hdmi_iface_release(uint32_t owner)
{
    uint32_t expected = owner;
    return owner != 0 && atomic_compare_exchange_strong(&port_owner, &expected, 0);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 29) - [public] atomic_load

```
    return atomic_load(&port_owner);
}

bool hdmi_iface_check(uint32_t owner)
{
    return owner != 0 && atomic_load(&port_owner) == owner;
}

bool hdmi_iface_init(uint32_t owner, uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride)
{
    if (owner != 0 && !hdmi_iface_check(owner)) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 39) - [public] if

```
    if (owner != 0 && !hdmi_iface_check(owner)) {
        return false;
    }

    return hdmi_init_with(fb_phys, w, h, stride);
}

bool hdmi_iface_ready(void)
{
    return hdmi_ready();
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 43) - [public] hdmi_init_with

```
    return hdmi_init_with(fb_phys, w, h, stride);
}

bool hdmi_iface_ready(void)
{
    return hdmi_ready();
}

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id)
{
    if (!hdmi_iface_check(owner)) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 48) - [public] hdmi_ready

```
    return hdmi_ready();
}

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id)
{
    if (!hdmi_iface_check(owner)) {
        return false;
    }

    return hdmi_mode_set_by_id(id);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 53) - [public] if

```
    if (!hdmi_iface_check(owner)) {
        return false;
    }

    return hdmi_mode_set_by_id(id);
}

bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    return hdmi_mode_current_raw(id, w, h, r);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 57) - [public] hdmi_mode_set_by_id

```
    return hdmi_mode_set_by_id(id);
}

bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    return hdmi_mode_current_raw(id, w, h, r);
}

bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    return hdmi_mode_at_raw(i, id, w, h, r);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 62) - [public] hdmi_mode_current_raw

```
    return hdmi_mode_current_raw(id, w, h, r);
}

bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    return hdmi_mode_at_raw(i, id, w, h, r);
}

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h)
{
    if (!hdmi_iface_check(owner)) {
        return 0;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 67) - [public] hdmi_mode_at_raw

```
    return hdmi_mode_at_raw(i, id, w, h, r);
}

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h)
{
    if (!hdmi_iface_check(owner)) {
        return 0;
    }

    return hdmi_submit_fill(color, x, y, w, h);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.c (line 72) - [public] if

```
    if (!hdmi_iface_check(owner)) {
        return 0;
    }

    return hdmi_submit_fill(color, x, y, w, h);
}

bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq)
{
    if (!hdmi_iface_check(owner)) {
        return false;
    }
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/hdmi/interface.c (line 76) - [public] hdmi_submit_fill
    return hdmi_submit_fill(color, x, y, w, h);
}

bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq)
{
    if (!hdmi_iface_check(owner)) {
        return false;
    }

    return hdmi_poll(out_seq);
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/interface.c (line 81) - [public] if
    if (!hdmi_iface_check(owner)) {
        return false;
    }

    return hdmi_poll(out_seq);
}

void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys)
{
    hdmi_caps_raw(w, h, s, phys);
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/interface.c (line 85) - [public] hdmi_poll
    return hdmi_poll(out_seq);
}

void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys)
{
    hdmi_caps_raw(w, h, s, phys);
}

bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s)
{
    if (!hdmi_iface_check(owner) || phys == NULL || w == NULL || h == NULL || s == NULL) {
        return false;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/interface.c (line 90) - [public] hdmi_caps_raw
    hdmi_caps_raw(w, h, s, phys);
}

bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s)
{
    if (!hdmi_iface_check(owner) || phys == NULL || w == NULL || h == NULL || s == NULL) {
        return false;
    }

    uint64_t p;
    uint32_t ww, hh, ss;
    hdmi_caps_raw(&ww, &hh, &ss, &p);
    // ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/hdmi/interface.c (line 95) - [public] if
    if (!hdmi_iface_check(owner) || phys == NULL || w == NULL || h == NULL || s == NULL) {
        return false;
    }

    uint64_t p;
    uint32_t ww, hh, ss;
    hdmi_caps_raw(&ww, &hh, &ss, &p);

    if (p == 0 || !hdmi_fb_grant()) {
        return false;
    }

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/hdmi/interface.c (line 101) - [public] hdmi_caps_raw
    hdmi_caps_raw(&ww, &hh, &ss, &p);

    if (p == 0 || !hdmi_fb_grant()) {
        return false;
    }

    *phys = p;
    *w = ww;
    *h = hh;
    *s = ss;
    return true;
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/hdmi/interface.c (line 103) - [public] if
    if (p == 0 || !hdmi_fb_grant()) {
        return false;
    }

    *phys = p;
    *w = ww;
    *h = hh;
    *s = ss;
    return true;
}

bool hdmi_iface_fb_revoke(uint32_t owner)
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/hdmi/interface.c (line 116) - [public] if
    if (!hdmi_iface_check(owner)) {
        return false;
    }

    hdmi_fb_revoke();
    return true;
}
```

```
kernel_Workspace/kernel/src/hdmi/interface.c (line 120) - [public] hdmi_fb_revoke
    hdmi_fb_revoke();
    return true;
}
```

kernel_Workspace/kernel/src/hdmi/mode.c (line 17) - [public] return

```
    return (uint32_t)(sizeof(MODES) / sizeof(MODES[0]));
}

const hdmi_mode_t *hdmi_mode_at(uint32_t i)
{
    if (i >= hdmi_mode_count()) {
        return NULL;
    }

    return &MODES[i];
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/mode.c (line 22) - [public] if

```
    if (i >= hdmi_mode_count()) {
        return NULL;
    }

    return &MODES[i];
}

const hdmi_mode_t *hdmi_mode_find(uint32_t w, uint32_t h, uint32_t refresh)
{
    for (uint32_t i = 0; i < hdmi_mode_count(); i++) {
        if (MODES[i].w == w && MODES[i].h == h && MODES[i].refresh == refresh) {
            return &MODES[i];
        }
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/mode.c (line 32) - [public] if

```
    if (MODES[i].w == w && MODES[i].h == h && MODES[i].refresh == refresh) {
        return &MODES[i];
    }

    return NULL;
}

const hdmi_mode_t *hdmi_mode_by_id(uint32_t id)
{
    for (uint32_t i = 0; i < hdmi_mode_count(); i++) {
        if (MODES[i].id == id) {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/mode.c (line 43) - [public] if

```
    if (MODES[i].id == id) {
        return &MODES[i];
    }

    return NULL;
}

bool hdmi_mode_valid(const hdmi_mode_t *m)
{
    if (m == NULL || m->w == 0 || m->h == 0 || m->refresh == 0 || m->clock_khz == 0) {
        return false;
    }

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/hdmi/mode.c (line 53) - [public] if
    if (m == NULL || m->w == 0 || m->h == 0 || m->refresh == 0 || m->clock_khz == 0) {
        return false;
    }

    hdmi_caps_t caps;
    hdmi_op_state(&caps);
    return caps.ready != 0 && caps.fb_phys != 0;
}

bool hdmi_mode_apply(const hdmi_mode_t *m)
{
    if (!hdmi_mode_valid(m)) {
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/hdmi/mode.c (line 58) - [public] hdmi_op_state
    hdmi_op_state(&caps);
    return caps.ready != 0 && caps.fb_phys != 0;
}

bool hdmi_mode_apply(const hdmi_mode_t *m)
{
    if (!hdmi_mode_valid(m)) {
        return false;
    }

    hdmi_op_set_mode(m);
    current_mode = m;
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/hdmi/mode.c (line 64) - [public] if
    if (!hdmi_mode_valid(m)) {
        return false;
    }

    hdmi_op_set_mode(m);
    current_mode = m;
    return true;
}

const hdmi_mode_t *hdmi_mode_current(void)
{
    return current_mode;
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/hdmi/mode.c (line 68) - [public] hdmi_op_set_mode
    hdmi_op_set_mode(m);
    current_mode = m;
    return true;
}

const hdmi_mode_t *hdmi_mode_current(void)
{
    return current_mode;
}

bool hdmi_mode_at_raw(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/hdmi/mode.c (line 81) - [public] if
    if (m == NULL || id == NULL || w == NULL || h == NULL || r == NULL) {
        return false;
    }

    *id = m->id;
    *w = m->w;
    *h = m->h;
    *r = m->refresh;
    return true;
}

```

kernel_Workspace/kernel/src/hdmi/operation.c (line 7) - [public] return

```
    return (void *)(DIRECT_BASE + phys);
}

static hdmi_caps_t cur;
static hdmi_mode_t cur_mode;
static bool op_ready;
static hdmi_transfer_t slots[HDMI_MAX_TRANSFERS];
static uint64_t slot_seq[HDMI_MAX_TRANSFERS];
static bool slot_used[HDMI_MAX_TRANSFERS];
static bool slot_done[HDMI_MAX_TRANSFERS];
static uint64_t next_seq = 1;
static uint32_t pending;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 22) - [public] if

```
    if (out != NULL) {
        *out = cur;
    }
}

bool hdmi_op_ready(void)
{
    return op_ready;
}

void hdmi_op_set_fb(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 44) - [public] if

```
    if (t == NULL || !op_ready) {
        return;
    }

    uint32_t stride = t->stride != 0 ? t->stride : cur.stride;
    uint64_t dst_phys = t->dst_phys != 0 ? t->dst_phys : cur.fb_phys;
    if (stride < cur.w || dst_phys == 0) {
        return;
    }

    uint32_t *dst = (uint32_t *)phys_to_virt(dst_phys);
    if (dst == NULL) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 50) - [public] if

```
    if (stride < cur.w || dst_phys == 0) {
        return;
    }

    uint32_t *dst = (uint32_t *)phys_to_virt(dst_phys);
    if (dst == NULL) {
        return;
    }

    if (t->kind == HDMI_TR_FILL) {
        if (t->x >= cur.w || t->y >= cur.h) {
            return;
        }
        // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/hdmi/operation.c (line 55) - [public] if

```
if (dst == NULL) {
    return;
}

if (t->kind == HDMI_TR_FILL) {
    if (t->x >= cur.w || t->y >= cur.h) {
        return;
    }
    uint32_t width = t->w < cur.w - t->x ? t->w : cur.w - t->x;
    uint32_t height = t->h < cur.h - t->y ? t->h : cur.h - t->y;
    for (uint32_t y = 0; y < height; y++) {
        for (uint32_t x = 0; x < width; x++) {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 59) - [public] if

```
if (t->kind == HDMI_TR_FILL) {
    if (t->x >= cur.w || t->y >= cur.h) {
        return;
    }
    uint32_t width = t->w < cur.w - t->x ? t->w : cur.w - t->x;
    uint32_t height = t->h < cur.h - t->y ? t->h : cur.h - t->y;
    for (uint32_t y = 0; y < height; y++) {
        for (uint32_t x = 0; x < width; x++) {
            dst[(t->y + y) * stride + t->x + x] = t->color;
        }
    }
    return;
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 60) - [public] if

```
if (t->x >= cur.w || t->y >= cur.h) {
    return;
}
uint32_t width = t->w < cur.w - t->x ? t->w : cur.w - t->x;
uint32_t height = t->h < cur.h - t->y ? t->h : cur.h - t->y;
for (uint32_t y = 0; y < height; y++) {
    for (uint32_t x = 0; x < width; x++) {
        dst[(t->y + y) * stride + t->x + x] = t->color;
    }
}
return;
// ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 73) - [public] if

```
if (t->kind == HDMI_TR_BLIT) {
    if (t->src_phys == 0 || t->stride == 0 || t->x >= cur.w || t->y >= cur.h) {
        return;
    }
    uint32_t *src = (uint32_t *)phys_to_virt(t->src_phys);
    if (src == NULL) {
        return;
    }
    uint32_t width = t->w < cur.w - t->x ? t->w : cur.w - t->x;
    uint32_t height = t->h < cur.h - t->y ? t->h : cur.h - t->y;
    for (uint32_t y = 0; y < height; y++) {
        for (uint32_t x = 0; x < width; x++) {
            // ... (truncated, see source file for more)
        }
    }
}
```

```
kernel_Workspace/kernel/src/hdmi/operation.c (line 74) - [public] if
    if (t->src_phys == 0 || t->stride == 0 || t->x >= cur.w || t->y >= cur.h) {
        return;
    }
    uint32_t *src = (uint32_t *)phys_to_virt(t->src_phys);
    if (src == NULL) {
        return;
    }
    uint32_t width = t->w < cur.w - t->x ? t->w : cur.w - t->x;
    uint32_t height = t->h < cur.h - t->y ? t->h : cur.h - t->y;
    for (uint32_t y = 0; y < height; y++) {
        for (uint32_t x = 0; x < width; x++) {
            dst[(t->y + y) * stride + t->x + x] = src[y * t->stride + x];
        }
    }
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/hdmi/operation.c (line 78) - [public] if
    if (src == NULL) {
        return;
    }
    uint32_t width = t->w < cur.w - t->x ? t->w : cur.w - t->x;
    uint32_t height = t->h < cur.h - t->y ? t->h : cur.h - t->y;
    for (uint32_t y = 0; y < height; y++) {
        for (uint32_t x = 0; x < width; x++) {
            dst[(t->y + y) * stride + t->x + x] = src[y * t->stride + x];
        }
    }
    return;
}
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/hdmi/operation.c (line 91) - [public] if
    if (t->kind == HDMI_TR_FLIP && t->dst_phys != 0) {
        cur.fb_phys = t->dst_phys;
    }
}

uint64_t hdmi_submit(const hdmi_transfer_t *t)
{
    if (t == NULL || !op_ready || pending >= HDMI_MAX_TRANSFERS) {
        return 0;
    }

    if (t->kind != HDMI_TR_FILL && t->kind != HDMI_TR_BLIT && t->kind != HDMI_TR_FLIP) {
        // ... (truncated, see source file for more)
    }
}
```

```
kernel_Workspace/kernel/src/hdmi/operation.c (line 98) - [public] if
    if (t == NULL || !op_ready || pending >= HDMI_MAX_TRANSFERS) {
        return 0;
    }

    if (t->kind != HDMI_TR_FILL && t->kind != HDMI_TR_BLIT && t->kind != HDMI_TR_FLIP) {
        return 0;
    }

    for (uint32_t i = 0; i < HDMI_MAX_TRANSFERS; i++) {
        if (!slot_used[i]) {
            slots[i] = *t;
            slot_seq[i] = next_seq++;
        }
    }
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/hdmi/operation.c (line 102) - [public] if
    if (t->kind != HDMI_TR_FILL && t->kind != HDMI_TR_BLIT && t->kind != HDMI_TR_FLIP) {
        return 0;
    }

    for (uint32_t i = 0; i < HDMI_MAX_TRANSFERS; i++) {
        if (!slot_used[i]) {
            slots[i] = *t;
            slot_seq[i] = next_seq++;
            if (next_seq == 0) {
                next_seq = 1;
            }
            slot_used[i] = true;
        }
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/hdmi/operation.c (line 107) - [public] if
    if (!slot_used[i]) {
        slots[i] = *t;
        slot_seq[i] = next_seq++;
        if (next_seq == 0) {
            next_seq = 1;
        }
        slot_used[i] = true;
        slot_done[i] = false;
        pending++;
        hdmi_op_exec(&slots[i]);
        slot_done[i] = true;
        return slot_seq[i];
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/operation.c (line 110) - [public] if
    if (next_seq == 0) {
        next_seq = 1;
    }
    slot_used[i] = true;
    slot_done[i] = false;
    pending++;
    hdmi_op_exec(&slots[i]);
    slot_done[i] = true;
    return slot_seq[i];
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/operation.c (line 116) - [public] hdmi_op_exec
    hdmi_op_exec(&slots[i]);
    slot_done[i] = true;
    return slot_seq[i];
}

return 0;
}

bool hdmi_poll(uint64_t *out_seq)
{
    if (out_seq == NULL) {
        // ... (truncated, see source file for more)
    }
}

```

kernel_Workspace/kernel/src/hdmi/operation.c (line 127) - [public] if

```
    if (out_seq == NULL) {
        return false;
    }

    for (uint32_t i = 0; i < HDMI_MAX_TRANSFERS; i++) {
        if (slot_used[i] && slot_done[i]) {
            *out_seq = slot_seq[i];
            slot_used[i] = false;
            slot_done[i] = false;
            pending--;
            return true;
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 132) - [public] if

```
        if (slot_used[i] && slot_done[i]) {
            *out_seq = slot_seq[i];
            slot_used[i] = false;
            slot_done[i] = false;
            pending--;
            return true;
        }
    }

    return false;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 159) - [public] hdmi_submit

```
    return hdmi_submit(&t);
}

void hdmi_op_set_mode(const hdmi_mode_t *m)
{
    if (m == NULL) {
        return;
    }
    cur_mode = *m;
    cur.w = m->w;
    cur.h = m->h;
    cur.ready = (cur.fb_phys != 0 && cur.w != 0 && cur.h != 0 && cur.stride >= cur.w) ? 1 : 0;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 164) - [public] if

```
    if (m == NULL) {
        return;
    }
    cur_mode = *m;
    cur.w = m->w;
    cur.h = m->h;
    cur.ready = (cur.fb_phys != 0 && cur.w != 0 && cur.h != 0 && cur.stride >= cur.w) ? 1 : 0;
    op_ready = cur.ready != 0;
}

static bool fb_granted = false;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 178) - [public] if

```
    if (cur.fb_phys == 0) {
        return false;
    }

    fb_granted = true;
    return true;
}
```

```
void hdmi_fb_revoke(void)
{
    fb_granted = false;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/operation.c (line 198) - [public] if

```
    if (w == NULL || h == NULL || s == NULL || phys == NULL) {
        return;
    }
    *w = cur.w;
    *h = cur.h;
    *s = cur.stride;
    *phys = cur.fb_phys;
}
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 8) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

static void k_memset(const void *dst, const uint8 value, const size_t n) {
    ■uint8_t *p = (uint8_t *)dst;
    ■for (size_t i = 0; i < n; i++) p[i] = value;
}

static void k_memcpy(const void *dst, const void *src, const size_t n) {
    ■uint8_t *d = (uint8_t *)dst;
    ■const uint8_t *s = (const uint8_t *)src;
    ■for (size_t i = 0; i < n; i++) d[i] = s[i];
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 10) - [internal] k_memset

```
static void k_memset(const void *dst, const uint8 value, const size_t n) {
    ■uint8_t *p = (uint8_t *)dst;
    ■for (size_t i = 0; i < n; i++) p[i] = value;
}

static void k_memcpy(const void *dst, const void *src, const size_t n) {
    ■uint8_t *d = (uint8_t *)dst;
    ■const uint8_t *s = (const uint8_t *)src;
    ■for (size_t i = 0; i < n; i++) d[i] = s[i];
}

static size_t k_strlen(const char *s) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 15) - [internal] k_memcpy

```
static void k_memcpy(const void *dst, const void *src, const size_t n) {
    ■uint8_t *d = (uint8_t *)dst;
    ■const uint8_t *s = (const uint8_t *)src;
    ■for (size_t i = 0; i < n; i++) d[i] = s[i];
}

static size_t k_strlen(const char *s) {
    ■size_t n = 0;
    ■while (s[n] != '\0') n++;
    ■return n;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 21) - [internal] k_strlen

```
static size_t k_strlen(const char *s) {
    size_t n = 0;
    while (s[n] != '\0') n++;
    return n;
}

static size_t k_usable_size(void *ptr) {
#ifdef ALLOC_DEBUG
    size_t s = dbg_usable_size(ptr);
    if (s != 0) return s;
#endif
    return heap_usable_size(ptr);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 27) - [internal] k_usable_size

```
static size_t k_usable_size(void *ptr) {
#ifdef ALLOC_DEBUG
    size_t s = dbg_usable_size(ptr);
    if (s != 0) return s;
#endif
    return heap_usable_size(ptr);
}

void *kmalloc(const size_t size) {
    if (size == 0) return NULL;
#ifdef ALLOC_DEBUG
    return dbg_alloc(size);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 32) - [public] heap_usable_size

```
return heap_usable_size(ptr);
}

void *kmalloc(const size_t size) {
    if (size == 0) return NULL;
#ifdef ALLOC_DEBUG
    return dbg_alloc(size);
#else
    return heap_alloc(size);
#endif
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 38) - [public] dbg_alloc

```
return dbg_alloc(size);
#else
return heap_alloc(size);
#endif
}

void kfree(void *ptr) {
    if (ptr == NULL) return;
#ifdef ALLOC_DEBUG
    dbg_free(ptr);
#else
    heap_free(ptr);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/api/alloc.c (line 40) - [public] heap_alloc

```
    return heap_alloc(size);
#endif
}

void kfree(void *ptr) {
    if (ptr == NULL) return;
#ifdef ALLOC_DEBUG
    dbg_free(ptr);
#else
    heap_free(ptr);
#endif
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/api/alloc.c (line 44) - [public] kfree

```
void kfree(void *ptr) {
    if (ptr == NULL) return;
#ifdef ALLOC_DEBUG
    dbg_free(ptr);
#else
    heap_free(ptr);
#endif
}

void *kzalloc(const size_t size) {
    void *ptr = kmalloc(size);
    if (ptr == NULL) return NULL;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/api/alloc.c (line 63) - [public] kzalloc

```
    return kzalloc(count * size);
}

void *kmalloc_aligned(const size_t size, const size_t align) {
    if (size == 0) return NULL;

    if (!size_is_pow2(align)) return NULL;
    return heap_alloc_aligned(size, align);
}

size_t kmalloc_usable_size(void *ptr) { return k_usable_size(ptr); }
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/api/alloc.c (line 70) - [public] heap_alloc_aligned

```
    return heap_alloc_aligned(size, align);
}

size_t kmalloc_usable_size(void *ptr) { return k_usable_size(ptr); }

void *krealloc(void *ptr, const size_t new_size) {
    if (ptr == NULL) return kmalloc(new_size);
    if (new_size == 0) {
        kfree(ptr);
        return NULL;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 74) - [public] kmalloc_usable_size

```
size_t kmalloc_usable_size(void *ptr) { return k_usable_size(ptr); }

void *krealloc(void *ptr, const size_t new_size) {
    if (ptr == NULL) return kmalloc(new_size);
    if (new_size == 0) {
        kfree(ptr);
        return NULL;
    }
    const size_t old_size = k_usable_size(ptr);
    if (old_size == 0) return NULL;
    if (old_size >= new_size) return ptr;
    void *new_ptr = kmalloc(new_size);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 79) - [public] kfree

```
kfree(ptr);
return NULL;
}
const size_t old_size = k_usable_size(ptr);
if (old_size == 0) return NULL;
if (old_size >= new_size) return ptr;
void *new_ptr = kmalloc(new_size);
if (new_ptr == NULL) return NULL;
k_memcpy(new_ptr, ptr, old_size);
kfree(ptr);
return new_ptr;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 95) - [public] heap_alloc

```
return heap_alloc(pages * ARCH_PAGE_SIZE);
}

void kfree_pages(void *ptr, const size_t pages) {
    if (ptr == NULL) return;
    size_t requested_bytes = 0;
    const size_t usable = heap_usable_size(ptr);
    if (!kfree_pages_validate(pages, ARCH_PAGE_SIZE, usable, &requested_bytes)) {
        kprintf("kfree_pages: nieprawidlowy parametr pages=%zu (pojemnosc bloku=%zu B) dla ptr=%p – odmowa zwrotu");
        return;
    }
    heap_free(ptr);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 98) - [public] kfree_pages

```
void kfree_pages(void *ptr, const size_t pages) {
    if (ptr == NULL) return;
    size_t requested_bytes = 0;
    const size_t usable = heap_usable_size(ptr);
    if (!kfree_pages_validate(pages, ARCH_PAGE_SIZE, usable, &requested_bytes)) {
        kprintf("kfree_pages: nieprawidlowy parametr pages=%zu (pojemnosc bloku=%zu B) dla ptr=%p – odmowa zwrotu");
        return;
    }
    heap_free(ptr);
}

char *kstrdup(const char *s) {
    // ... (truncated, see source file for more)
}
```


kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 103) - [public] kprintf

```
■kprintf("kfree_pages: nieprawidlowy parametr pages=%zu (pojemnosc bloku=%zu B) dla ptr=%p – odmowa zwo  
■return;  
■}  
■heap_free(ptr);  
}  
  
char *kstrdup(const char *s) {  
■if (s == NULL) return NULL;  
■const size_t n = k_strlen(s) + 1;  
■char *ptr = (char *)kmalloc(n);  
■if (ptr == NULL) return NULL;  
■k_memcpy(ptr, s, n);  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 118) - [public] kvirt_to_phys

```
uint64_t kvirt_to_phys(void *ptr) {  
■if (ptr == NULL) return UINT64_MAX;  
■const uint64_t va = (uint64_t)(uintptr_t)ptr,  
■phys = mapping_translate(MAPPING_KERNEL, va);  
■if (phys != UINT64_MAX) return phys;  
■return arch_virt_to_phys(ptr);  
}  
  
void kalloc_dump(void) {  
#ifdef ALLOC_DEBUG  
■mm_debug_dump();  
#else  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 123) - [public] arch_virt_to_phys

```
■return arch_virt_to_phys(ptr);  
}  
  
void kalloc_dump(void) {  
#ifdef ALLOC_DEBUG  
■mm_debug_dump();  
#else  
■heap_dump();  
#endif  
}
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.c (line 126) - [public] kalloc_dump

```
void kalloc_dump(void) {  
#ifdef ALLOC_DEBUG  
■mm_debug_dump();  
#else  
■heap_dump();  
#endif  
}
```

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 11) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);  
  
#define DBG_MAGIC 0x444247414C4C4F43ULL  
#define DBG_POISON_ALLOC 0xAA  
#define DBG_POISON_FREE 0xFF  
  
typedef struct dbg_header {  
■uint64_t magic;  
■uint64_t size;  
■uint64_t caller;  
■uint64_t tracked;  
■uint64_t alloc_id;  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 25) - [internal]

dbg_tail_canary

```
static uint64_t dbg_tail_canary(size_t size) { return ~((uint64_t)size) ^ DBG_MAGIC; }
static uint64_t *dbg_tail_ptr(void *user, size_t size) { return (uint64_t *)((uint8_t *)user + size); }

void *dbg_alloc(size_t size) {
    if (size == 0) return NULL;

    uint64_t caller = (uint64_t)(uintptr_t)__builtin_return_address(0);
    uint8_t *real = (uint8_t *)heap_alloc(sizeof(dbg_header_t) + size + sizeof(uint64_t));

    if (real == NULL) return NULL;

    dbg_header_t *h = (dbg_header_t *)real;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 56) - [public] dbg_free

```
void dbg_free(void *ptr) {
    if (ptr == NULL) return;

    dbg_header_t *h = (dbg_header_t *)((uint8_t *)ptr - sizeof(dbg_header_t));
    if (h->magic != DBG_MAGIC) {
        kprintf("dbg_free: bad magic at %p\n", ptr);
        return;
    }

    size_t size = h->size;
    if (*dbg_tail_ptr(ptr, size) != dbg_tail_canary(size)) kprintf("dbg_free: buffer overflow at %p caller 0\n", ptr);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 61) - [public] kprintf

```
kprintf("dbg_free: bad magic at %p\n", ptr);
return;
}

size_t size = h->size;
if (*dbg_tail_ptr(ptr, size) != dbg_tail_canary(size)) kprintf("dbg_free: buffer overflow at %p caller 0\n", ptr);

if (h->tracked) {
    size_t leaked_size = 0;
    uint64_t alloc_caller = 0;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 73) - [public] if

```
if (!leak_untrack(ptr, &leaked_size, &alloc_caller)) {
    kprintf("dbg_free: double free or unknown ptr %p\n", ptr);
    return;
}

alloc_stats_note_free(size);

h->magic = 0;

uint8_t *real = (uint8_t *)h;
size_t total = sizeof(dbg_header_t) + size + sizeof(uint64_t);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 74) - [public] kprintf

```
    kprintf("dbg_free: double free or unknown ptr %p\n", ptr);
    return;
}

alloc_stats_note_free(size);

h->magic = 0;

uint8_t *real = (uint8_t *)h;
size_t total = sizeof(dbg_header_t) + size + sizeof(uint64_t);
for (size_t i = 0; i < total; i++) real[i] = DBG_POISON_FREE;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 90) - [public] dbg_usable_size

```
size_t dbg_usable_size(void *ptr) {
    if (ptr == NULL) return 0;
    dbg_header_t *h = (dbg_header_t *)((uint8_t *)ptr - sizeof(dbg_header_t));
    if (h->magic != DBG_MAGIC) return 0;
    return h->size;
}

bool dbg_verify(void *ptr) {
    if (ptr == NULL) return false;
    dbg_header_t *h = (dbg_header_t *)((uint8_t *)ptr - sizeof(dbg_header_t));
    if (h->magic != DBG_MAGIC) return false;
    return *dbg_tail_ptr(ptr, h->size) == dbg_tail_canary(h->size);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 97) - [public] dbg_verify

```
bool dbg_verify(void *ptr) {
    if (ptr == NULL) return false;
    dbg_header_t *h = (dbg_header_t *)((uint8_t *)ptr - sizeof(dbg_header_t));
    if (h->magic != DBG_MAGIC) return false;
    return *dbg_tail_ptr(ptr, h->size) == dbg_tail_canary(h->size);
}

void mm_debug_dump(void) {
    alloc_stats_dump();
    leak_dump();
    pmm_dump();
    vmm_dump();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.c (line 104) - [public] mm_debug_dump

```
void mm_debug_dump(void) {
    alloc_stats_dump();
    leak_dump();
    pmm_dump();
    vmm_dump();
    buddy_dump();
    slab_dump();
    cache_dump();
}
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 4) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

#define LEAK_MAX 2048

typedef struct leak_entry {
    void *ptr;
    size_t size;
    uint64_t caller, alloc_id;
    int64_t cpu_id;
    bool used;
} leak_entry_t;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 22) - [public] leak_track_id

```
bool leak_track_id(void *ptr, size_t size, uint64_t caller, uint64_t *out_id) {
    smp_lock_acquire(&leak_smp_lock);
    for (size_t i = 0; i < LEAK_MAX; i++) {
        if (!leak_table[i].used) {
            uint64_t id = next_alloc_id++;
            leak_table[i].ptr = ptr;
            leak_table[i].size = size;
            leak_table[i].caller = caller;
            leak_table[i].alloc_id = id;
            leak_table[i].cpu_id = smp_current_cpu_id();
            leak_table[i].used = true;
            smp_lock_release(&leak_smp_lock);
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 25) - [public] if

```
    if (!leak_table[i].used) {
        uint64_t id = next_alloc_id++;
        leak_table[i].ptr = ptr;
        leak_table[i].size = size;
        leak_table[i].caller = caller;
        leak_table[i].alloc_id = id;
        leak_table[i].cpu_id = smp_current_cpu_id();
        leak_table[i].used = true;
        smp_lock_release(&leak_smp_lock);
        if (out_id != NULL) *out_id = id;
        return true;
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 33) - [public] smp_lock_release

```
    smp_lock_release(&leak_smp_lock);
    if (out_id != NULL) *out_id = id;
    return true;
}
}
smp_lock_release(&leak_smp_lock);
return false;
}
bool leak_track(void *ptr, size_t size, uint64_t caller) { return leak_track_id(ptr, size, caller, NULL); }
bool leak_untrack(void *ptr, size_t *out_size, uint64_t *out_caller) {
    smp_lock_acquire(&leak_smp_lock);
    for (size_t i = 0; i < LEAK_MAX; i++) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 41) - [public] leak_track

```
bool leak_track(void *ptr, size_t size, uint64_t caller) { return leak_track_id(ptr, size, caller, NULL); }
bool leak_untrack(void *ptr, size_t *out_size, uint64_t *out_caller) {
    smp_lock_acquire(&leak_smp_lock);
    for (size_t i = 0; i < LEAK_MAX; i++) {
        if (leak_table[i].used && leak_table[i].ptr == ptr) {
            if (out_size != NULL) *out_size = leak_table[i].size;
            if (out_caller != NULL) *out_caller = leak_table[i].caller;
            leak_table[i].used = false;
            smp_lock_release(&leak_smp_lock);
            return true;
        }
    }
}
// ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 42) - [public] leak_untrack

```
bool leak_untrack(void *ptr, size_t *out_size, uint64_t *out_caller) {
    ■smp_lock_acquire(&leak_smp_lock);
    ■for (size_t i = 0; i < LEAK_MAX; i++) {
    ■    if (leak_table[i].used && leak_table[i].ptr == ptr) {
    ■        if (out_size != NULL) *out_size = leak_table[i].size;
    ■        if (out_caller != NULL) *out_caller = leak_table[i].caller;
    ■        leak_table[i].used = false;
    ■        smp_lock_release(&leak_smp_lock);
    ■        return true;
    ■    }
    ■}
    ■smp_lock_release(&leak_smp_lock);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 45) - [public] if

```
    ■    if (leak_table[i].used && leak_table[i].ptr == ptr) {
    ■        if (out_size != NULL) *out_size = leak_table[i].size;
    ■        if (out_caller != NULL) *out_caller = leak_table[i].caller;
    ■        leak_table[i].used = false;
    ■        smp_lock_release(&leak_smp_lock);
    ■        return true;
    ■    }
    ■}
    ■smp_lock_release(&leak_smp_lock);
    ■return false;
    }
    bool leak_contains(void *ptr) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 49) - [public] smp_lock_release

```
    ■        smp_lock_release(&leak_smp_lock);
    ■        return true;
    ■    }
    ■}
    ■smp_lock_release(&leak_smp_lock);
    ■return false;
    }
    bool leak_contains(void *ptr) {
    ■smp_lock_acquire(&leak_smp_lock);
    ■for (size_t i = 0; i < LEAK_MAX; i++) {
    ■    if (leak_table[i].used && leak_table[i].ptr == ptr) {
    ■        smp_lock_release(&leak_smp_lock);
    ■        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 56) - [public] leak_contains

```
    bool leak_contains(void *ptr) {
    ■smp_lock_acquire(&leak_smp_lock);
    ■for (size_t i = 0; i < LEAK_MAX; i++) {
    ■    if (leak_table[i].used && leak_table[i].ptr == ptr) {
    ■        smp_lock_release(&leak_smp_lock);
    ■        return true;
    ■    }
    ■}
    ■smp_lock_release(&leak_smp_lock);
    ■return false;
    }
    size_t leak_count(void) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 59) - [public] if

```
■    if (leak_table[i].used && leak_table[i].ptr == ptr) {
■        smp_lock_release(&leak_smp_lock);
■        return true;
■    }
■}
■smp_lock_release(&leak_smp_lock);
■return false;
}
size_t leak_count(void) {
■smp_lock_acquire(&leak_smp_lock);
■size_t n = 0;
■for (size_t i = 0; i < LEAK_MAX; i++) if (leak_table[i].used) n++;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 60) - [public] smp_lock_release

```
■        smp_lock_release(&leak_smp_lock);
■        return true;
■    }
■}
■smp_lock_release(&leak_smp_lock);
■return false;
}
size_t leak_count(void) {
■smp_lock_acquire(&leak_smp_lock);
■size_t n = 0;
■for (size_t i = 0; i < LEAK_MAX; i++) if (leak_table[i].used) n++;
■smp_lock_release(&leak_smp_lock);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 67) - [public] leak_count

```
size_t leak_count(void) {
■smp_lock_acquire(&leak_smp_lock);
■size_t n = 0;
■for (size_t i = 0; i < LEAK_MAX; i++) if (leak_table[i].used) n++;
■smp_lock_release(&leak_smp_lock);
■return n;
}
void leak_dump(void) {
■smp_lock_acquire(&leak_smp_lock);
■size_t n = 0;
■for (size_t i = 0; i < LEAK_MAX; i++) if (leak_table[i].used) n++;
■kprintf("LEAKS: %llu live\n", (unsigned long long)n);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/leak.c (line 74) - [public] leak_dump

```
void leak_dump(void) {
■smp_lock_acquire(&leak_smp_lock);
■size_t n = 0;
■for (size_t i = 0; i < LEAK_MAX; i++) if (leak_table[i].used) n++;
■kprintf("LEAKS: %llu live\n", (unsigned long long)n);
■for (size_t i = 0; i < LEAK_MAX; i++) {
■    if (!leak_table[i].used) continue;
■    kprintf("    leak: %p size %llu caller 0x%llx id %llu cpu %lld\n",
■        leak_table[i].ptr,
■        (unsigned long long)leak_table[i].size,
■        (unsigned long long)leak_table[i].caller,
■        (unsigned long long)leak_table[i].alloc_id,
■        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 4) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

static size_t
■stat_live_count = 0,
■stat_live_bytes = 0,
■stat_peak_bytes = 0,
■stat_total_allocs = 0,
■stat_total_frees = 0;

static smp_ticket_lock_t stats_smp_lock = SMP_TICKET_LOCK_INIT;

void alloc_stats_note_alloc(size_t size) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 15) - [public] alloc_stats_note_alloc

```
void alloc_stats_note_alloc(size_t size) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■stat_live_count++;
    ■stat_live_bytes += size;
    ■stat_total_allocs++;
    ■if (stat_live_bytes > stat_peak_bytes) stat_peak_bytes = stat_live_bytes;
    ■smp_lock_release(&stats_smp_lock);
}

void alloc_stats_note_free(size_t size) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■if (stat_live_count > 0) stat_live_count--;
    ■if (stat_live_bytes >= size) stat_live_bytes -= size;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 23) - [public] alloc_stats_note_free

```
void alloc_stats_note_free(size_t size) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■if (stat_live_count > 0) stat_live_count--;
    ■if (stat_live_bytes >= size) stat_live_bytes -= size;
    ■else stat_live_bytes = 0;
    ■stat_total_frees++;
    ■smp_lock_release(&stats_smp_lock);
}

size_t alloc_stats_live_count(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_live_count;
    ■smp_lock_release(&stats_smp_lock);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 31) - [public] alloc_stats_live_count

```
size_t alloc_stats_live_count(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_live_count;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}

size_t alloc_stats_live_bytes(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_live_bytes;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 37) - [public] alloc_stats_live_bytes

```
size_t alloc_stats_live_bytes(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_live_bytes;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}
size_t alloc_stats_peak_bytes(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_peak_bytes;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 43) - [public] alloc_stats_peak_bytes

```
size_t alloc_stats_peak_bytes(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_peak_bytes;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}
size_t alloc_stats_total_allocs(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_total_allocs;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 49) - [public]
alloc_stats_total_allocs

```
size_t alloc_stats_total_allocs(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_total_allocs;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}
size_t alloc_stats_total_frees(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_total_frees;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 55) - [public]
alloc_stats_total_frees

```
size_t alloc_stats_total_frees(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t v = stat_total_frees;
    ■smp_lock_release(&stats_smp_lock);
    ■return v;
}
void alloc_stats_dump(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t
    ■■live_count = stat_live_count,
    ■■live_bytes = stat_live_bytes,
    ■■peak_bytes = stat_peak_bytes,
    // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/alloc/debug/stats.c (line 61) - [public] alloc_stats_dump

```
void alloc_stats_dump(void) {
    ■smp_lock_acquire(&stats_smp_lock);
    ■size_t
    ■■live_count = stat_live_count,
    ■■live_bytes = stat_live_bytes,
    ■■peak_bytes = stat_peak_bytes,
    ■■total_allocs = stat_total_allocs,
    ■■total_frees = stat_total_frees;
    ■smp_lock_release(&stats_smp_lock);
    ■kprintf("ALLOC STATS:\n");
    ■kprintf("  live: %llu objs, %llu B\n",
    ■■■(unsigned long long)live_count,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 6) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

#define BUDDY_STATE_FREE 0
#define BUDDY_STATE_USED 1
#define BUDDY_STATE_TAIL 2

static uint64_t buddy_base = 0;
static size_t buddy_pages = 0, buddy_max_order = 0;

static uint8_t *buddy_order = NULL, *buddy_state = NULL;
static int32_t *buddy_next = NULL, *buddy_heads = NULL;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 25) - [internal] order_size

```
static inline uint64_t order_size(size_t o) { return ARCH_PAGE_SIZE << o; }
static size_t page_of(uint64_t va) { return (size_t)((va - buddy_base) / ARCH_PAGE_SIZE); }
static uint64_t va_of(size_t page) { return buddy_base + (uint64_t)page * ARCH_PAGE_SIZE; }
static size_t size_to_order(size_t size) {
    ■size_t o = 0, s = ARCH_PAGE_SIZE;
    ■while (s < size && o < buddy_max_order) {
    ■    s <= 1;
    ■    o++;
    ■}
    ■return o;
}
static void fl_push(size_t o, size_t idx) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 26) - [internal] page_of

```
static size_t page_of(uint64_t va) { return (size_t)((va - buddy_base) / ARCH_PAGE_SIZE); }
static uint64_t va_of(size_t page) { return buddy_base + (uint64_t)page * ARCH_PAGE_SIZE; }
static size_t size_to_order(size_t size) {
    ■size_t o = 0, s = ARCH_PAGE_SIZE;
    ■while (s < size && o < buddy_max_order) {
    ■    s <= 1;
    ■    o++;
    ■}
    ■return o;
}
static void fl_push(size_t o, size_t idx) {
    ■buddy_next[idx] = buddy_heads[o];
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 27) - [internal] va_of

```
static uint64_t va_of(size_t page) { return buddy_base + (uint64_t)page * ARCH_PAGE_SIZE; }
static size_t size_to_order(size_t size) {
    size_t o = 0, s = ARCH_PAGE_SIZE;
    while (s < size && o < buddy_max_order) {
        s <= 1;
        o++;
    }
    return o;
}
static void fl_push(size_t o, size_t idx) {
    buddy_next[idx] = buddy_heads[o];
    buddy_heads[o] = (int32_t)idx;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 28) - [internal] size_to_order

```
static size_t size_to_order(size_t size) {
    size_t o = 0, s = ARCH_PAGE_SIZE;
    while (s < size && o < buddy_max_order) {
        s <= 1;
        o++;
    }
    return o;
}
static void fl_push(size_t o, size_t idx) {
    buddy_next[idx] = buddy_heads[o];
    buddy_heads[o] = (int32_t)idx;
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 36) - [internal] fl_push

```
static void fl_push(size_t o, size_t idx) {
    buddy_next[idx] = buddy_heads[o];
    buddy_heads[o] = (int32_t)idx;
}
static size_t fl_pop(size_t o) {
    int32_t i = buddy_heads[o];
    if (i < 0) return (size_t)-1;
    buddy_heads[o] = buddy_next[i];
    return (size_t)i;
}
static void fl_remove(size_t o, size_t idx) {
    int32_t *p = &buddy_heads[o];
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 40) - [internal] fl_pop

```
static size_t fl_pop(size_t o) {
    int32_t i = buddy_heads[o];
    if (i < 0) return (size_t)-1;
    buddy_heads[o] = buddy_next[i];
    return (size_t)i;
}
static void fl_remove(size_t o, size_t idx) {
    int32_t *p = &buddy_heads[o];
    while (*p >= 0) {
        if ((size_t)*p == idx) {
            *p = buddy_next[*p];
            return;
        }
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 46) - [internal] fl_remove

```
static void fl_remove(size_t o, size_t idx) {
    int32_t *p = &buddy_heads[o];
    while (*p >= 0) {
        if ((size_t)*p == idx) {
            *p = buddy_next[*p];
            return;
        }
        p = &buddy_next[*p];
    }
}

bool buddy_init(uint64_t base, size_t size, buddy_map_cb map_cb, buddy_unmap_cb unmap_cb) {
    if (buddy_initialized) return true;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 49) - [public] if

```
    if ((size_t)*p == idx) {
        *p = buddy_next[*p];
        return;
    }
    p = &buddy_next[*p];
}

bool buddy_init(uint64_t base, size_t size, buddy_map_cb map_cb, buddy_unmap_cb unmap_cb) {
    if (buddy_initialized) return true;
    if (map_cb == NULL || unmap_cb == NULL) return false;
    if (size == 0 || (size & (size - 1)) != 0) return false;
    if (size < ARCH_PAGE_SIZE) return false;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 56) - [public] buddy_init

```
bool buddy_init(uint64_t base, size_t size, buddy_map_cb map_cb, buddy_unmap_cb unmap_cb) {
    if (buddy_initialized) return true;
    if (map_cb == NULL || unmap_cb == NULL) return false;
    if (size == 0 || (size & (size - 1)) != 0) return false;
    if (size < ARCH_PAGE_SIZE) return false;
    if (!arch_is_page_aligned(base)) return false;
    if ((base & (size - 1)) != 0) return false;
    buddy_base = base;
    buddy_pages = size / ARCH_PAGE_SIZE;
    buddy_max_order = 0;
    while (order_size(buddy_max_order) < size) buddy_max_order++;
    size_t
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 95) - [public] buddy_ready

```
bool buddy_ready(void) { return buddy_initialized; }

void *buddy_alloc(size_t size) {
    if (!buddy_initialized || size == 0) return NULL;
    size_t o = size_to_order(size);
    if (order_size(o) < size || o > buddy_max_order) return NULL;
    size_t cur = 0;
    while (buddy_heads[cur] < 0 && cur < buddy_max_order) cur++;
    if (buddy_heads[cur] < 0) return NULL;
    size_t idx = fl_pop(cur);
    while (cur > 0) {
        cur--;
        size_t half_pages = (size_t)1 << cur, buddy_idx = idx + half_pages;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 109) - [public] fl_push

```
    fl_push(cur, buddy_idx);
}
buddy_order[idx] = (uint8_t)o;
buddy_state[idx] = BUDDY_STATE_USED;
size_t bytes = order_size(o);
if (!buddy_map(va_of(idx), bytes)) {
    buddy_state[idx] = BUDDY_STATE_FREE;
    fl_push(o, idx);
    return NULL;
}
buddy_used_bytes += bytes;
buddy_free_bytes -= bytes;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 116) - [public] fl_push

```
    fl_push(o, idx);
    return NULL;
}
buddy_used_bytes += bytes;
buddy_free_bytes -= bytes;
return (void *) (uintptr_t) va_of(idx);
}
```

```
void *buddy_alloc_aligned(size_t size, size_t align) {
    if (!buddy_initialized || size == 0) return NULL;
    if (!size_is_pow2(align)) return NULL;
    if (align <= ARCH_PAGE_SIZE) return buddy_alloc(size);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 133) - [public] buddy_alloc

```
return buddy_alloc(p);
}
```

```
void buddy_free(void *ptr) {
    if (!buddy_initialized || ptr == NULL) return;
    uint64_t va = (uint64_t)(uintptr_t)ptr;
    if (va < buddy_base || va >= buddy_base + (uint64_t)buddy_pages * ARCH_PAGE_SIZE) return;
    size_t idx = page_of(va);
    if (buddy_state[idx] != BUDDY_STATE_USED) return;
    size_t o = buddy_order[idx], bytes = order_size(o);
    buddy_unmap(va, bytes);
    buddy_state[idx] = BUDDY_STATE_FREE;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 136) - [public] buddy_free

```
void buddy_free(void *ptr) {
    if (!buddy_initialized || ptr == NULL) return;
    uint64_t va = (uint64_t)(uintptr_t)ptr;
    if (va < buddy_base || va >= buddy_base + (uint64_t)buddy_pages * ARCH_PAGE_SIZE) return;
    size_t idx = page_of(va);
    if (buddy_state[idx] != BUDDY_STATE_USED) return;
    size_t o = buddy_order[idx], bytes = order_size(o);
    buddy_unmap(va, bytes);
    buddy_state[idx] = BUDDY_STATE_FREE;
    buddy_order[idx] = (uint8_t)o;
    size_t cur = o;
    while (cur < buddy_max_order) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 153) - [public] fl_remove

```
■ fl_remove(cur, buddy_idx);
■ buddy_state[buddy_idx] = BUDDY_STATE_TAIL;
■ if (buddy_idx < idx) idx = buddy_idx;
■ cur++;
■ buddy_order[idx] = (uint8_t)cur;
■ }
■ fl_push(cur, idx);
■ buddy_used_bytes -= bytes;
■ buddy_free_bytes += bytes;
}
size_t buddy_block_size(void *ptr) {
■ if (!buddy_initialized || ptr == NULL) return 0;
  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 163) - [public] buddy_block_size

```
size_t buddy_block_size(void *ptr) {
■ if (!buddy_initialized || ptr == NULL) return 0;
■ uint64_t va = (uint64_t)(uintptr_t)ptr;
■ if (va < buddy_base || va >= buddy_base + (uint64_t)buddy_pages * ARCH_PAGE_SIZE) return 0;
■ size_t idx = page_of(va);
■ if (buddy_state[idx] != BUDDY_STATE_USED) return 0;
■ return (size_t)order_size(buddy_order[idx]);
}
size_t buddy_stat_used_bytes(void) { return buddy_used_bytes; }
size_t buddy_stat_free_bytes(void) { return buddy_free_bytes; }
void buddy_dump(void) {
■ if (!buddy_initialized) {
  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 171) - [public] buddy_stat_used_bytes

```
size_t buddy_stat_used_bytes(void) { return buddy_used_bytes; }
size_t buddy_stat_free_bytes(void) { return buddy_free_bytes; }
void buddy_dump(void) {
■ if (!buddy_initialized) {
  kprintf("BUDDY: not initialized\n");
  return;
■ }
■ kprintf("BUDDY:\n");
■ kprintf(" base: 0x%llx\n", (unsigned long long)buddy_base);
■ kprintf(" max order: %llu\n", (unsigned long long)buddy_max_order);
■ kprintf(" used: %llu KiB\n", (unsigned long long)(buddy_used_bytes >> 10));
■ kprintf(" free: %llu KiB\n", (unsigned long long)(buddy_free_bytes >> 10));
  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 172) - [public] buddy_stat_free_bytes

```
size_t buddy_stat_free_bytes(void) { return buddy_free_bytes; }
void buddy_dump(void) {
■ if (!buddy_initialized) {
  kprintf("BUDDY: not initialized\n");
  return;
■ }
■ kprintf("BUDDY:\n");
■ kprintf(" base: 0x%llx\n", (unsigned long long)buddy_base);
■ kprintf(" max order: %llu\n", (unsigned long long)buddy_max_order);
■ kprintf(" used: %llu KiB\n", (unsigned long long)(buddy_used_bytes >> 10));
■ kprintf(" free: %llu KiB\n", (unsigned long long)(buddy_free_bytes >> 10));
}
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.c (line 173) - [public] buddy_dump

```
void buddy_dump(void) {
■ if (!buddy_initialized) {
  kprintf("BUDDY: not initialized\n");
  return;
■ }
■ kprintf("BUDDY:\n");
■ kprintf(" base: 0x%llx\n", (unsigned long long)buddy_base);
■ kprintf(" max order: %llu\n", (unsigned long long)buddy_max_order);
■ kprintf(" used: %llu KiB\n", (unsigned long long)(buddy_used_bytes >> 10));
■ kprintf(" free: %llu KiB\n", (unsigned long long)(buddy_free_bytes >> 10));
}
```

kernel_workspace/kernel/src/mm/alloc/heap/buddy.c (line 175) - [public] kprintf

```
■ kprintf("BUDDY: not initialized\n");
■ return;
■ }
■ kprintf("BUDDY:\n");
■ kprintf("  base: 0x%llx\n", (unsigned long long)buddy_base);
■ kprintf("  max order: %llu\n", (unsigned long long)buddy_max_order);
■ kprintf("  used: %llu KiB\n", (unsigned long long)(buddy_used_bytes >> 10));
■ kprintf("  free: %llu KiB\n", (unsigned long long)(buddy_free_bytes >> 10));
■ }
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 32) - [public] if

```
■ if (!pmm_alloc_frame(&phys)) {
■     for (size_t j = 0; j < i; j++) {
■         uint64_t v = virt + j * ARCH_PAGE_SIZE;
■         uint64_t p = mapping_translate(MAPPING_KERNEL, v);

■         mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);

■         if (p != UINT64_MAX) {
■             pmm_free_frame(p);
■         }
■     }

■     // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 37) - [public] mapping_unmap_range

```
■         mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);

■         if (p != UINT64_MAX) {
■             pmm_free_frame(p);
■         }
■     }

■     return false;
■ }

■ if (!mapping_map_range(MAPPING_KERNEL,
■                         virt + i * ARCH_PAGE_SIZE,
■                         // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 39) - [public] if

```
■         if (p != UINT64_MAX) {
■             pmm_free_frame(p);
■         }
■     }

■     return false;
■ }

■ if (!mapping_map_range(MAPPING_KERNEL,
■                         virt + i * ARCH_PAGE_SIZE,
■                         phys,
■                         ARCH_PAGE_SIZE,
■                         // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 40) - [public] pmm_free_frame

```
■         pmm_free_frame(p);
■     }
■ }

■     return false;
■ }

■ if (!mapping_map_range(MAPPING_KERNEL,
■                         virt + i * ARCH_PAGE_SIZE,
■                         phys,
■                         ARCH_PAGE_SIZE,
■                         PTE_PRESENT | PTE_WRITABLE | PTE_NX)) {
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 52) - [public] pmm_free_frame

```
■      pmm_free_frame(phys);

■      for (size_t j = 0; j < i; j++) {
■          uint64_t v = virt + j * ARCH_PAGE_SIZE;
■          uint64_t p = mapping_translate(MAPPING_KERNEL, v);

■          mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);

■          if (p != UINT64_MAX) {
■              pmm_free_frame(p);
■          }
■      }
      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 58) - [public] mapping_unmap_range

```
■      mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);

■      if (p != UINT64_MAX) {
■          pmm_free_frame(p);
■      }

■      return false;
■  }
■}

■return true;
      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 60) - [public] if

```
■      if (p != UINT64_MAX) {
■          pmm_free_frame(p);
■      }

■      return false;
■  }
■}

■return true;
}

      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 61) - [public] pmm_free_frame

```
■          pmm_free_frame(p);
■      }

■      return false;
■  }
■}

■return true;
}

static void heap_unmap_cb(uint64_t virt, size_t size)
      // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 80) - [public] if

```
■ if (p != UINT64_MAX) {
■     mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);
■     pmm_free_frame(p);
■ }
■}
}
```

```
static void heap_memset(void *dst, uint8_t value, size_t n)
{
■uint8_t *p = (uint8_t *)dst;

■for (size_t i = 0; i < n; i++) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 81) - [public] mapping_unmap_range

```
■     mapping_unmap_range(MAPPING_KERNEL, v, ARCH_PAGE_SIZE);
■     pmm_free_frame(p);
■ }
■}
}
```

```
static void heap_memset(void *dst, uint8_t value, size_t n)
{
■uint8_t *p = (uint8_t *)dst;

■for (size_t i = 0; i < n; i++) {
■    p[i] = value;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 82) - [public] pmm_free_frame

```
■     pmm_free_frame(p);
■ }
■}
}
```

```
static void heap_memset(void *dst, uint8_t value, size_t n)
{
■uint8_t *p = (uint8_t *)dst;

■for (size_t i = 0; i < n; i++) {
■    p[i] = value;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/heap.c (line 147) - [public] slab_alloc

```
■     return slab_alloc(size);
■}
#endif
```

```
■return buddy_alloc(size);
}
```

```
void *heap_alloc_aligned(size_t size, size_t align)
{
■if (!heap_initialized || size == 0) {
■    return NULL;
    // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 151) - [public] buddy_alloc

```
    return buddy_alloc(size);
}

void *heap_alloc_aligned(size_t size, size_t align)
{
    if (!heap_initialized || size == 0) {
        return NULL;
    }

    if (!size_is_pow2(align)) {
        return NULL;
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 167) - [public] slab_alloc

```
    return slab_alloc(size);
}
#endif

return buddy_alloc_aligned(size, align);
}

void *heap_zalloc(size_t size)
{
    void *ptr = heap_alloc(size);

    if (ptr == NULL) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 171) - [public] buddy_alloc_aligned

```
return buddy_alloc_aligned(size, align);
}

void *heap_zalloc(size_t size)
{
    void *ptr = heap_alloc(size);

    if (ptr == NULL) {
        return NULL;
    }

    heap_memset(ptr, 0, size);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 197) - [public] slab_free

```
    slab_free(ptr);
    return;
}
#endif

if (heap_in_buddy(va)) {
    buddy_free(ptr);
}

size_t heap_usable_size(void *ptr)
{
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 203) - [public] buddy_free

```
    buddy_free(ptr);
}

size_t heap_usable_size(void *ptr)
{
    if (!heap_initialized || ptr == NULL) {
        return 0;
    }

    uint64_t va = (uint64_t)(uintptr_t)ptr;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 217) - [public] slab_usable_size

```
    return slab_usable_size(ptr);
}
#endif

if (heap_in_buddy(va)) {
    return buddy_block_size(ptr);
}

return 0;
}

void heap_dump(void)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 222) - [public] buddy_block_size

```
    return buddy_block_size(ptr);
}

return 0;
}

void heap_dump(void)
{
    if (!heap_initialized) {
        return;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.c (line 238) - [public] slab_dump

```
    slab_dump();
}
#endif
}
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 9) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

#define SLAB_CACHE_COUNT 8

static const size_t slab_classes[SLAB_CACHE_COUNT] = {
    16, 32, 64, 128, 256, 512, 1024, 2048
};

#define SLAB_MAX_OBJECTS_PER_SLAB 256
#define SLAB_BITMAP_WORDS (SLAB_MAX_OBJECTS_PER_SLAB / 32)

typedef struct slab_desc {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 92) - [public] if

```
■   if ((uint32_t)*p == page) {
■       *p = slab_descs[*p].next_partial;
■       return;
■   }

■   p = &slab_descs[*p].next_partial;
■ }
}

static bool slab_grow(uint32_t cache_id)
{
■ slab_cache_t *c = &slab_caches[cache_id];
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 116) - [public] bitmap_free

```
■   bitmap_free(&slab_page_bitmap, page);
■   return false;
■ }

■ if (!mapping_map_range(MAPPING_KERNEL,
■                         va,
■                         phys,
■                         ARCH_PAGE_SIZE,
■                         PTE_PRESENT | PTE_WRITABLE | PTE_NX)) {
■     pmm_free_frame(phys);
■     bitmap_free(&slab_page_bitmap, page);
■     return false;
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 125) - [public] pmm_free_frame

```
■   pmm_free_frame(phys);
■   bitmap_free(&slab_page_bitmap, page);
■   return false;
■ }

■ slab_desc_t *d = &slab_descs[page];

■ d->cache_id = cache_id;
■ d->state = 1;
■ d->free_count = c->objects_per_slab;
■ d->free_head = 0;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 126) - [public] bitmap_free

```
■   bitmap_free(&slab_page_bitmap, page);
■   return false;
■ }

■ slab_desc_t *d = &slab_descs[page];

■ d->cache_id = cache_id;
■ d->state = 1;
■ d->free_count = c->objects_per_slab;
■ d->free_head = 0;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 207) - [public] slab_bitmap_clear_all

```
■    slab_bitmap_clear_all(&slab_descs[i]);
■}

■for (size_t i = 0; i < SLAB_CACHE_COUNT; i++) {
■    slab_caches[i].object_size = slab_classes[i];
■    slab_caches[i].objects_per_slab =
■        (uint32_t)(ARCH_PAGE_SIZE / slab_classes[i]);
■    slab_caches[i].partial_head = -1;
■    slab_caches[i].total_slabs = 0;

■    if (slab_caches[i].objects_per_slab > SLAB_MAX_OBJECTS_PER_SLAB) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 218) - [public] if

```
■    if (slab_caches[i].objects_per_slab > SLAB_MAX_OBJECTS_PER_SLAB) {
■        return false;
■    }
■}
```

```
■slab_initialized = true;
```

```
■return true;
■}
```

```
bool slab_ready(void)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 255) - [public] if

```
■    if (!slab_grow(cache_id)) {
■        slab_unlock();
■        return NULL;
■    }
■}
```

```
■uint32_t page = (uint32_t)c->partial_head;
■slab_desc_t *d = &slab_descs[page];
```

```
■uint32_t idx = d->free_head;
```

```
■uint64_t va = slab_page_va(page);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 256) - [public] slab_unlock

```
■    slab_unlock();
■    return NULL;
■    }
■}
```

```
■uint32_t page = (uint32_t)c->partial_head;
■slab_desc_t *d = &slab_descs[page];
```

```
■uint32_t idx = d->free_head;
```

```
■uint64_t va = slab_page_va(page);
■uint8_t *obj =
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/slab.c (line 328) - [public] slab_unlock

```
■    slab_unlock();
■    return;
■}

■slab_bit_clear(d, idx);

■bool was_full = (d->free_count == 0);

■*(uint32_t *) (uintptr_t)va = d->free_head;
■d->free_head = idx;
■d->free_count++;

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/slab.c (line 346) - [public] partial_remove

```
■    partial_remove(c, page);

■    uint64_t phys = mapping_translate(MAPPING_KERNEL, slab_va);

■    mapping_unmap_range(MAPPING_KERNEL, slab_va, ARCH_PAGE_SIZE);

■    if (phys != UINT64_MAX) {
■        pmm_free_frame(phys);
■    }

■    bitmap_free(&slab_page_bitmap, page);

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/slab.c (line 350) - [public] mapping_unmap_range

```
■    mapping_unmap_range(MAPPING_KERNEL, slab_va, ARCH_PAGE_SIZE);

■    if (phys != UINT64_MAX) {
■        pmm_free_frame(phys);
■    }

■    bitmap_free(&slab_page_bitmap, page);

■    d->state = 0;
■    c->total_slabs--;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/slab.c (line 352) - [public] if

```
■    if (phys != UINT64_MAX) {
■        pmm_free_frame(phys);
■    }

■    bitmap_free(&slab_page_bitmap, page);

■    d->state = 0;
■    c->total_slabs--;
■}

■slab_unlock();
}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 353) - [public] pmm_free_frame

```
■    pmm_free_frame(phys);
■    }

■    bitmap_free(&slab_page_bitmap, page);

■    d->state = 0;
■    c->total_slabs--;
■}

■slab_unlock();
}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 356) - [public] bitmap_free

```
■    bitmap_free(&slab_page_bitmap, page);

■    d->state = 0;
■    c->total_slabs--;
■}

■slab_unlock();
}

size_t slab_usable_size(void *ptr)
{
■if (!slab_initialized || ptr == NULL) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 400) - [public] if

```
■    if (d->state != 1) {
■        continue;
■    }

■    const slab_cache_t *c = &slab_caches[d->cache_id];

■    used += (size_t)(c->objects_per_slab - d->free_count) *
■        c->object_size;
■}

■return used;
}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.c (line 424) - [public] if

```
■    if (d->state != 1) {
■        continue;
■    }

■    const slab_cache_t *c = &slab_caches[d->cache_id];

■    free_bytes += (size_t)d->free_count * c->object_size;
■}

■return free_bytes;
}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/heap/slab.c (line 439) - [public] kprintf

```
■ kprintf("SLAB: not initialized\n");
■ return;
■}

■kprintf("SLAB:\n");

■for (size_t i = 0; i < SLAB_CACHE_COUNT; i++) {
■ kprintf(" size %4u: slabs %u\n",
■         (unsigned int)slab_caches[i].object_size,
■         (unsigned int)slab_caches[i].total_slabs);
■}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 65) - [public]
bitmap_words_for_bits

```
■return bitmap_words_for_bits(bit_count) * sizeof(uint64_t);
}

static void bitmap_mark_tail_used(bitmap_t *bm)
{
■if (bm->word_count == 0) {
■ return;
■}

■size_t rem = bm->bit_count & 63;

■if (rem == 0) {
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 132) - [public]
bitmap_mark_tail_used

```
■ bitmap_mark_tail_used(bm);
■}
}

void bitmap_set(bitmap_t *bm, size_t bit)
{
■if (bm == NULL || bm->bits == NULL || bit >= bm->bit_count) {
■ return;
■}

■size_t word = bit / 64;
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 298) - [public] if

```
■ if (bm->bits[i] != 0) {
■ return false;
■ }
■}

■if ((bm->bits[last_word] & last_mask) != 0) {
■ return false;
■}

■return true;
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 329) - [public] if

```
■   if (idx < bm->bit_count) {
■       return idx;
■   }

■   return BITMAP_INVALID;
■}

■for (size_t i = word + 1; i < bm->word_count; i++) {
■   w = bm->bits[i];

■   if (~w != 0) {
■       size_t found_bit = (size_t)__builtin_ctzll(~w);
■       // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 339) - [public] if

```
■   if (~w != 0) {
■       size_t found_bit = (size_t)__builtin_ctzll(~w);
■       size_t idx = i * 64 + found_bit;

■       if (idx < bm->bit_count) {
■           return idx;
■       }

■       return BITMAP_INVALID;
■   }
■}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 343) - [public] if

```
■       if (idx < bm->bit_count) {
■           return idx;
■       }

■       return BITMAP_INVALID;
■   }
■}

■return BITMAP_INVALID;
■}

static size_t find_next_used_bit(const bitmap_t *bm, size_t start, size_t limit)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 377) - [public] if

```
■   if (i == start_word && start_bit != 0) {
■       w &= ~lower_bits_mask(start_bit);
■   }

■   if (i == end_word) {
■       w &= mask_to_bit(end_bit);
■   }

■   if (w != 0) {
■       size_t found_bit = (size_t)__builtin_ctzll(w);
■       size_t idx = i * 64 + found_bit;

■       // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 381) - [public] if

```
■   if (i == end_word) {
■       w &= mask_to_bit(end_bit);
■   }

■   if (w != 0) {
■       size_t found_bit = (size_t)__builtin_ctzll(w);
■       size_t idx = i * 64 + found_bit;

■       if (idx >= start && idx < limit) {
■           return idx;
■       }
■   }
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 385) - [public] if

```
■   if (w != 0) {
■       size_t found_bit = (size_t)__builtin_ctzll(w);
■       size_t idx = i * 64 + found_bit;

■       if (idx >= start && idx < limit) {
■           return idx;
■       }
■   }

■}

■return BITMAP_INVALID;
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 389) - [public] if

```
■       if (idx >= start && idx < limit) {
■           return idx;
■       }
■   }

■}

■return BITMAP_INVALID;
}

size_t bitmap_alloc(bitmap_t *bm)
{
■if (bm == NULL || bm->bits == NULL || bm->bit_count == 0) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/bitmap.c (line 474) - [public] bitmap_alloc

```
■   return bitmap_alloc(bm);
■}

■size_t candidate = 0;

■while (candidate <= bm->bit_count - count) {
■   candidate = align_up_size(candidate, align);

■   if (candidate == BITMAP_INVALID ||
■       candidate > bm->bit_count - count) {
■       break;
■   }
■}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 489) - [public] if

```
■   if (free_bit == BITMAP_INVALID) {  
■       break;  
■   }  
  
■   candidate = align_up_size(free_bit, align);  
  
■   if (candidate == BITMAP_INVALID ||  
■       candidate > bm->bit_count - count) {  
■       break;  
■   }  
  
■   if (bitmap_test_range_free(bm, candidate, count)) {  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 500) - [public] if

```
■   if (bitmap_test_range_free(bm, candidate, count)) {  
■       bitmap_set_range(bm, candidate, count);  
■       bm->alloc_hint = candidate / 64;  
■       return candidate;  
■   }  
  
■   size_t used = find_next_used_bit(bm, candidate, candidate + count);  
  
■   if (used == BITMAP_INVALID) {  
■       candidate++;  
■   } else {  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 501) - [public] bitmap_set_range

```
■       bitmap_set_range(bm, candidate, count);  
■       bm->alloc_hint = candidate / 64;  
■       return candidate;  
■   }  
  
■   size_t used = find_next_used_bit(bm, candidate, candidate + count);  
  
■   if (used == BITMAP_INVALID) {  
■       candidate++;  
■   } else {  
■       candidate = used + 1;  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.c (line 509) - [public] if

```
■   if (used == BITMAP_INVALID) {  
■       candidate++;  
■   } else {  
■       candidate = used + 1;  
■   }  
■}  
  
■return BITMAP_INVALID;  
}  
  
void bitmap_free(bitmap_t *bm, size_t bit)  
{  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 64) - [public] arch_phys_to_virt

```
■return arch_phys_to_virt(frame);
}

bool frame_is_valid(frame_t frame)
{
■if (!frame_initialized) {
■    return false;
■}

■if (!arch_is_page_aligned(frame)) {
■    return false;
■}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 159) - [public] if

```
■    if (r->type != ARCH_MEM_TYPE_USABLE) {
■        continue;
■    }

■    if (!arch_is_page_aligned(r->base) ||
■        !arch_is_page_aligned(r->len)) {
■        continue;
■    }

■    size_t pfn_start = (size_t)(r->base / ARCH_PAGE_SIZE);
■    size_t frame_count = (size_t)(r->len / ARCH_PAGE_SIZE);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 171) - [public] if

```
■    if (frame_count == 0) {
■        continue;
■    }

■    bitmap_clear_range(&frame_bitmap, pfn_start, frame_count);

■    if (allocated_frames >= frame_count) {
■        allocated_frames -= frame_count;
■    } else {
■        allocated_frames = 0;
■    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 175) - [public] bitmap_clear_range

```
■    bitmap_clear_range(&frame_bitmap, pfn_start, frame_count);

■    if (allocated_frames >= frame_count) {
■        allocated_frames -= frame_count;
■    } else {
■        allocated_frames = 0;
■    }

■    dma32_account_free(pfn_start, frame_count);
■}

■return true;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/frame.c (line 177) - [public] if

```
■    if (allocated_frames >= frame_count) {
■        allocated_frames -= frame_count;
■    } else {
■        allocated_frames = 0;
■    }

■    dma32_account_free(pfn_start, frame_count);
■}

■return true;
}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/frame.c (line 183) - [public] dma32_account_free

```
■    dma32_account_free(pfn_start, frame_count);
■}

■return true;
}

bool frame_ready(void)
{
■return frame_initialized;
}

bool frame_alloc(frame_t *out)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/frame.c (line 230) - [public] frame_free

```
■    frame_free(frame);
■    return false;
■}

■*out = frame;

■return true;
}

bool frame_alloc_contiguous(size_t count,
■                            size_t align_frames,
■                            frame_t *out)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/frame.c (line 345) - [public] volatile

```
■__asm__ volatile("" ::: "memory");

■return true;
}

bool frame_alloc_below(size_t count,
■                        size_t align_frames,
■                        uint64_t max_phys,
■                        frame_t *out)
{
■if (!frame_initialized || count == 0 || out == NULL) {
■    return false;
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 377) - [public] if

```
■   if (aligned > limit - count) {
■       break;
■   }

■   if (bitmap_test_range_free(&frame_bitmap, aligned, count)) {
■       bitmap_set_range(&frame_bitmap, aligned, count);

■       allocated_frames += count;
■       dma32_account_alloc(aligned, count);

■       *out = frame_from_pfn(aligned);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 381) - [public] if

```
■   if (bitmap_test_range_free(&frame_bitmap, aligned, count)) {
■       bitmap_set_range(&frame_bitmap, aligned, count);

■       allocated_frames += count;
■       dma32_account_alloc(aligned, count);

■       *out = frame_from_pfn(aligned);

■       return true;
■   }

■   pfn = aligned + 1;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 382) - [public] bitmap_set_range

```
■       bitmap_set_range(&frame_bitmap, aligned, count);

■       allocated_frames += count;
■       dma32_account_alloc(aligned, count);

■       *out = frame_from_pfn(aligned);

■       return true;
■   }

■   pfn = aligned + 1;
■}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.c (line 385) - [public]

dma32_account_alloc

```
■       dma32_account_alloc(aligned, count);

■       *out = frame_from_pfn(aligned);

■       return true;
■   }

■   pfn = aligned + 1;
■}

■return false;
}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 6) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

static void pmm_panic(const char *msg) __attribute__((noreturn));
static void pmm_panic(const char *msg)
{
    kprintf("pmm panic: %s\n", msg);

    for (;;) {
        __asm__ volatile("cli; hlt");
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 8) - [public] pmm_panic

```
static void pmm_panic(const char *msg) __attribute__((noreturn));
static void pmm_panic(const char *msg)
{
    kprintf("pmm panic: %s\n", msg);

    for (;;) {
        __asm__ volatile("cli; hlt");
    }

    static bool pmm_initialized = false;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 31) - [public] pmm_panic

```
    pmm_panic("pmm_unlock without lock");
}

#define PMM_SIZE_MAX ((size_t)-1)

static size_t pmm_bytes_to_frames(size_t bytes)
{
    if (bytes == 0) {
        return 0;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 80) - [public] if

```
    if (!frame_zero(phys)) {
        return false;
    }

    return true;
}

bool pmm_init(void)
{
    pmm_lock();

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 93) - [public] pmm_unlock

```
■ pmm_unlock();
■ return true;
■}

■if (!arch_memory_ready()) {
■ pmm_unlock();
■ kprintf("pmm_init: arch memory not initialized\n");
■ return false;
■}

■if (!frame_init_from_memory()) {
■ pmm_unlock();
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 98) - [public] pmm_unlock

```
■ pmm_unlock();
■ kprintf("pmm_init: arch memory not initialized\n");
■ return false;
■}

■if (!frame_init_from_memory()) {
■ pmm_unlock();
■ kprintf("pmm_init: frame allocator initialization failed\n");
■ return false;
■}

■pmm_initialized = true;
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 99) - [public] kprintf

```
■ kprintf("pmm_init: arch memory not initialized\n");
■ return false;
■}

■if (!frame_init_from_memory()) {
■ pmm_unlock();
■ kprintf("pmm_init: frame allocator initialization failed\n");
■ return false;
■}

■pmm_initialized = true;

■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 104) - [public] pmm_unlock

```
■ pmm_unlock();
■ kprintf("pmm_init: frame allocator initialization failed\n");
■ return false;
■}

■pmm_initialized = true;

■pmm_unlock();

■return true;
}

■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 105) - [public] kprintf

```
■    kprintf("pmm_init: frame allocator initialization failed\n");  
■    return false;  
■}
```

```
■pmm_initialized = true;
```

```
■pmm_unlock();
```

```
■return true;
```

```
}
```

```
bool pmm_ready(void)  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 220) - [public] pmm_alloc_frames

```
■return pmm_alloc_frames(frames, out_phys);  
}
```

```
bool pmm_alloc_zero_bytes(size_t bytes, uint64_t *out_phys)
```

```
{
```

```
■if (!pmm_initialized || out_phys == NULL || bytes == 0) {
```

```
■    return false;
```

```
■}
```

```
■size_t frames = pmm_bytes_to_frames(bytes);
```

```
■if (frames == 0) {
```

```
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 236) - [public] pmm_alloc_zero_frame

```
■    return pmm_alloc_zero_frame(out_phys);
```

```
■}
```

```
■pmm_lock();
```

```
■frame_t frame = FRAME_INVALID;
```

```
■bool ok = frame_alloc_contiguous(frames, 1, &frame);
```

```
■pmm_unlock();
```

```
■if (!ok) {
```

```
■    return false;
```

```
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 251) - [public] pmm_free_frames

```
■    pmm_free_frames(frame, frames);
```

```
■    return false;
```

```
■}
```

```
■*out_phys = frame;
```

```
■return true;
```

```
}
```

```
bool pmm_alloc_contiguous_bytes(size_t bytes,
```

```
■                                size_t align_bytes,
```

```
■                                uint64_t *out_phys)
```

```
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 275) - [public]

pmm_alloc_frames_aligned

```
■return pmm_alloc_frames_aligned(frames, align_frames, out_phys);
}
```

```
bool pmm_free_frame(uint64_t phys)
```

```
{
■if (!pmm_initialized) {
■    return false;
■}
}
```

```
■pmm_lock();
```

```
■bool ok = frame_free(phys);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 328) - [public] pmm_free_frames

```
■return pmm_free_frames(phys, frames);
}
```

```
size_t pmm_stat_total_frames(void)
```

```
{
■if (!pmm_initialized) {
■    return 0;
■}
}
```

```
■return frame_total();
}
```

```
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 337) - [public] frame_total

```
■return frame_total();
}
```

```
size_t pmm_stat_free_frames(void)
```

```
{
■if (!pmm_initialized) {
■    return 0;
■}
}
```

```
■return frame_free_count();
}
```

```
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 346) - [public] frame_free_count

```
■return frame_free_count();
}
```

```
size_t pmm_stat_allocated_frames(void)
```

```
{
■if (!pmm_initialized) {
■    return 0;
■}
}
```

```
■return frame_allocated();
}
```

```
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 355) - [public] frame_allocated

```
■return frame_allocated();
}

uint64_t pmm_stat_total_bytes(void)
{
■if (!pmm_initialized) {
■    return 0;
■}

■uint64_t frames = frame_total();

■if (frames > UINT64_MAX / ARCH_PAGE_SIZE) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 394) - [public]

frame_zone_dma32_total

```
■return frame_zone_dma32_total();
}

size_t pmm_stat_dma32_free_frames(void)
{
■if (!pmm_initialized) {
■    return 0;
■}

■return frame_zone_dma32_free();
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 403) - [public]

frame_zone_dma32_free

```
■return frame_zone_dma32_free();
}

size_t pmm_stat_normal_total_frames(void)
{
■if (!pmm_initialized) {
■    return 0;
■}

■return frame_zone_normal_total();
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/physical/pmm.c (line 412) - [public]

frame_zone_normal_total

```
■return frame_zone_normal_total();
}

size_t pmm_stat_normal_free_frames(void)
{
■if (!pmm_initialized) {
■    return 0;
■}

■return frame_zone_normal_free();
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 421) - [public]

frame_zone_normal_free

```
    return frame_zone_normal_free();
}

bool pmm_alloc_frames_below(size_t count,
    size_t align_frames,
    uint64_t max_phys,
    uint64_t *out_phys)
{
    if (!pmm_initialized || out_phys == NULL || count == 0) {
        return false;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 452) - [public] kprintf

```
    kprintf("PMM: not initialized\n");
    return;
}

uint64_t total_frames = pmm_stat_total_frames();
uint64_t free_frames = pmm_stat_free_frames();
uint64_t allocated_frames = pmm_stat_allocated_frames();

uint64_t total_mb = pmm_stat_total_bytes() >> 20;
uint64_t free_mb = pmm_stat_free_bytes() >> 20;

kprintf("PMM:\n");
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 490) - [public] if

```
    if (!pmm_alloc_zero_frame(&frames[i])) {
        kprintf("pmm_self_test: single frame alloc failed\n");
        return false;
    }

    if (!arch_is_page_aligned(frames[i])) {
        kprintf("pmm_self_test: frame not aligned\n");
        return false;
    }

    volatile uint64_t *p =
        (volatile uint64_t *)arch_phys_to_virt(frames[i]);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 491) - [public] kprintf

```
    kprintf("pmm_self_test: single frame alloc failed\n");
    return false;
}

    if (!arch_is_page_aligned(frames[i])) {
        kprintf("pmm_self_test: frame not aligned\n");
        return false;
    }

    volatile uint64_t *p =
        (volatile uint64_t *)arch_phys_to_virt(frames[i]);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 495) - [public] if

```
■   if (!arch_is_page_aligned(frames[i])) {
■       kprintf("pmm_self_test: frame not aligned\n");
■       return false;
■   }

■   volatile uint64_t *p =
■       (volatile uint64_t *)arch_phys_to_virt(frames[i]);

■   for (size_t j = 0; j < (ARCH_PAGE_SIZE / sizeof(uint64_t)); j++) {
■       if (p[j] != 0) {
■           kprintf("pmm_self_test: zero frame not zero\n");
■           return false;
■       }
■   }
■   // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 496) - [public] kprintf

```
■       kprintf("pmm_self_test: frame not aligned\n");
■       return false;
■   }

■   volatile uint64_t *p =
■       (volatile uint64_t *)arch_phys_to_virt(frames[i]);

■   for (size_t j = 0; j < (ARCH_PAGE_SIZE / sizeof(uint64_t)); j++) {
■       if (p[j] != 0) {
■           kprintf("pmm_self_test: zero frame not zero\n");
■           return false;
■       }
■   }
■   // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 504) - [public] if

```
■       if (p[j] != 0) {
■           kprintf("pmm_self_test: zero frame not zero\n");
■           return false;
■       }
■   }
■ }

■ for (size_t i = 0; i < 8; i++) {
■     if (!pmm_free_frame(frames[i])) {
■         kprintf("pmm_self_test: single frame free failed\n");
■         return false;
■     }
■ }
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 505) - [public] kprintf

```
■       kprintf("pmm_self_test: zero frame not zero\n");
■       return false;
■   }
■ }

■ for (size_t i = 0; i < 8; i++) {
■     if (!pmm_free_frame(frames[i])) {
■         kprintf("pmm_self_test: single frame free failed\n");
■         return false;
■     }
■ }
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 512) - [public] if

```
■    if (!pmm_free_frame(frames[i])) {  
■        kprintf("pmm_self_test: single frame free failed\n");  
■        return false;  
■    }  
■}
```

```
■uint64_t contiguous = PMM_INVALID_FRAME;
```

```
■if (!pmm_alloc_frames(16, &contiguous)) {  
■    kprintf("pmm_self_test: contiguous alloc failed\n");  
■    return false;  
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 513) - [public] kprintf

```
■        kprintf("pmm_self_test: single frame free failed\n");  
■        return false;  
■    }  
■}
```

```
■uint64_t contiguous = PMM_INVALID_FRAME;
```

```
■if (!pmm_alloc_frames(16, &contiguous)) {  
■    kprintf("pmm_self_test: contiguous alloc failed\n");  
■    return false;  
■}  
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 522) - [public] kprintf

```
■    kprintf("pmm_self_test: contiguous alloc failed\n");  
■    return false;  
■}
```

```
■if (!arch_is_page_aligned(contiguous)) {  
■    kprintf("pmm_self_test: contiguous not aligned\n");  
■    return false;  
■}
```

```
■if (!pmm_free_frames(contiguous, 16)) {  
■    kprintf("pmm_self_test: contiguous free failed\n");  
■    return false;  
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 527) - [public] kprintf

```
■    kprintf("pmm_self_test: contiguous not aligned\n");  
■    return false;  
■}
```

```
■if (!pmm_free_frames(contiguous, 16)) {  
■    kprintf("pmm_self_test: contiguous free failed\n");  
■    return false;  
■}
```

```
■uint64_t aligned2m = PMM_INVALID_FRAME;
```

```
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 532) - [public] kprintf

```
■ kprintf("pmm_self_test: contiguous free failed\n");
■ return false;
■ }
```

```
■ uint64_t aligned2m = PMM_INVALID_FRAME;
```

```
■ if (!pmm_alloc_frames_aligned(512, 512, &aligned2m)) {
■ kprintf("pmm_self_test: 2M aligned alloc failed\n");
■ return false;
■ }
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 540) - [public] kprintf

```
■ kprintf("pmm_self_test: 2M aligned alloc failed\n");
■ return false;
■ }
```

```
■ uint64_t align2m_mask = (512ULL * ARCH_PAGE_SIZE) - 1;
```

```
■ if ((aligned2m & align2m_mask) != 0) {
■ kprintf("pmm_self_test: 2M alignment failed\n");
■ return false;
■ }
```

```
■ if (!pmm_free_frames(aligned2m, 512)) {
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 547) - [public] kprintf

```
■ kprintf("pmm_self_test: 2M alignment failed\n");
■ return false;
■ }
```

```
■ if (!pmm_free_frames(aligned2m, 512)) {
■ kprintf("pmm_self_test: 2M aligned free failed\n");
■ return false;
■ }
```

```
■ uint64_t bytes_alloc = PMM_INVALID_FRAME;
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 552) - [public] kprintf

```
■ kprintf("pmm_self_test: 2M aligned free failed\n");
■ return false;
■ }
```

```
■ uint64_t bytes_alloc = PMM_INVALID_FRAME;
```

```
■ if (!pmm_alloc_zero_bytes(10000, &bytes_alloc)) {
■ kprintf("pmm_self_test: bytes alloc failed\n");
■ return false;
■ }
```

```
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 560) - [public] kprintf
    ■ kprintf("pmm_self_test: bytes alloc failed\n");
    ■ return false;
    ■}

    ■size_t bytes_frames = pmm_bytes_to_frames(10000);

    ■volatile uint64_t *p =
    ■ (volatile uint64_t *)arch_phys_to_virt(bytes_alloc);

    ■size_t words = bytes_frames * (ARCH_PAGE_SIZE / sizeof(uint64_t));

    ■for (size_t i = 0; i < words; i++) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 572) - [public] if
    ■ if (p[i] != 0) {
    ■ kprintf("pmm_self_test: bytes alloc not zero\n");
    ■ return false;
    ■ }
    ■}

    ■if (!pmm_free_bytes(bytes_alloc, 10000)) {
    ■ kprintf("pmm_self_test: bytes free failed\n");
    ■ return false;
    ■}

    ■kprintf("pmm_self_test: OK\n");
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 573) - [public] kprintf
    ■ kprintf("pmm_self_test: bytes alloc not zero\n");
    ■ return false;
    ■ }
    ■}

    ■if (!pmm_free_bytes(bytes_alloc, 10000)) {
    ■ kprintf("pmm_self_test: bytes free failed\n");
    ■ return false;
    ■}

    ■kprintf("pmm_self_test: OK\n");

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.c (line 579) - [public] kprintf
    ■ kprintf("pmm_self_test: bytes free failed\n");
    ■ return false;
    ■}

    ■kprintf("pmm_self_test: OK\n");

    ■return true;
    }

#endif

```

kernel_workspace/kernel/src/mm/alloc/special/contiguous.c (line 9) - [public]

size_bytes_to_pages_checked

```
■return size_bytes_to_pages_checked(bytes, ARCH_PAGE_SIZE, out_frames);
}

bool contig_alloc(size_t bytes,
■          size_t align,
■          uint64_t *out_phys,
■          void **out_virt)
{
■if (bytes == 0) {
■    return false;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/special/dma.c (line 10) - [public]

size_bytes_to_pages_checked

```
■return size_bytes_to_pages_checked(bytes, ARCH_PAGE_SIZE, out_frames);
}

bool dma_alloc_coherent(size_t bytes,
■          uint64_t zone_max,
■          uint64_t *out_phys,
■          void **out_virt)
{
■if (bytes == 0 || out_phys == NULL || out_virt == NULL) {
■    return false;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/special/dma.c (line 47) - [public] pmm_free_frames

```
■    pmm_free_frames(phys, frames);
■    return false;
■}

■*out_phys = phys;
■*out_virt = (void *) (uintptr_t) virt;

■return true;
}

void dma_free_coherent(uint64_t phys, void *virt, size_t bytes)
{
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/special/dma.c (line 76) - [public] vmm_unmap_device

```
■    vmm_unmap_device((uint64_t) (uintptr_t) virt, len);
■}

■pmm_free_frames(phys, frames);
}

static void clflush_range(void *addr, size_t len)
{
■uintptr_t start = (uintptr_t) addr & ~(uintptr_t) 63;
■uintptr_t end = (uintptr_t) addr + len;

■for (uintptr_t a = start; a < end; a += 64) {
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/mm/alloc/special/dma.c (line 88) - [public] volatile

```
■ __asm__ volatile("clflush (%0)" :: "r"(a) : "memory");
■}

■__asm__ volatile("" ::: "memory");
}

void dma_sync_for_device(void *virt, size_t len)
{
■if (virt == NULL || len == 0) {
■    return;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/special/dma.c (line 91) - [public] volatile

```
■__asm__ volatile("" ::: "memory");
}

void dma_sync_for_device(void *virt, size_t len)
{
■if (virt == NULL || len == 0) {
■    return;
■}

■clflush_range(virt, len);
}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 27) - [public] mapping_init

```
■    mapping_init();
■}

■return mapping_kernel_pml4;
}

static bool page_range_ok(uint64_t addr, size_t len)
{
■uint64_t start, end;

■if (!range_from_addr_len(addr, (uint64_t)len, ARCH_PAGE_SIZE,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 70) - [public] if

```
■    if (!paging_map_page_in(pml4, v, p, pte_flags)) {
■        for (size_t j = 0; j < mapped; j++) {
■            paging_unmap_page_in(pml4,
■                virt + (uint64_t)j * ARCH_PAGE_SIZE);
■        }

■        return false;
■    }

■    mapped++;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 102) - [public]
paging_unmap_page_in

```
■    paging_unmap_page_in(pml4, virt + (uint64_t)i * ARCH_PAGE_SIZE);
■}

■return true;
}

bool mapping_protect_range(mapping_space_t space,
■    uint64_t virt,
■    size_t len,
■    uint64_t pte_flags)
{
■if (len == 0) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 128) - [public] if

```
■    if (!paging_is_mapped_in(pml4, v)) {
■        return false;
■    }
■}

■for (size_t i = 0; i < pages; i++) {
■    uint64_t v = virt + (uint64_t)i * ARCH_PAGE_SIZE;

■    if (!paging_set_flags_in(pml4, v, pte_flags)) {

■        return false;
■    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 136) - [public] if

```
■    if (!paging_set_flags_in(pml4, v, pte_flags)) {

■        return false;
■    }
■}

■return true;
}

uint64_t mapping_translate(mapping_space_t space, uint64_t virt)
{
■return paging_translate_in(resolve_space(space), virt);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 147) - [public]
paging_translate_in

```
■return paging_translate_in(resolve_space(space), virt);
}

bool mapping_is_mapped(mapping_space_t space, uint64_t virt)
{
■return paging_is_mapped_in(resolve_space(space), virt);
}

static void copy_page_by_phys(uint64_t dst_phys, uint64_t src_phys)
{
■uint64_t *dst = (uint64_t *)arch_phys_to_virt(dst_phys);
■const uint64_t *src = (const uint64_t *)arch_phys_to_virt(src_phys);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 152) - [public]
paging_is_mapped_in

```
    return paging_is_mapped_in(resolve_space(space), virt);
}

static void copy_page_by_phys(uint64_t dst_phys, uint64_t src_phys)
{
    uint64_t *dst = (uint64_t *)arch_phys_to_virt(dst_phys);
    const uint64_t *src = (const uint64_t *)arch_phys_to_virt(src_phys);

    for (size_t i = 0; i < ARCH_PAGE_SIZE / sizeof(uint64_t); i++) {
        dst[i] = src[i];
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 164) - [public] volatile

```
    __asm__ volatile("" ::: "memory");
}

bool mapping_copy_range(mapping_space_t dst,
    uint64_t dst_virt,
    mapping_space_t src,
    uint64_t src_virt,
    size_t len)
{
    if (len == 0) {
        return true;
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/mapping.c (line 199) - [public] copy_page_by_phys

```
    copy_page_by_phys(dp, sp);
}

return true;
}
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 5) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

static page_t *page_descs = NULL;
static size_t page_desc_count = 0;
static bool page_initialized = false;

static size_t page_lock_depth = 0;
static uint64_t page_lock_flags = 0;

static void page_lock(void)
{
    uint64_t flags;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 78) - [public] kprintf

```
    kprintf("page_init: pmm not ready\n");
    return false;
}

const arch_mem_info_t *info = arch_memory_get();

if (info == NULL) {
    kprintf("page_init: no arch mem info\n");
    return false;
}

uint64_t max_pfn = info->max_usable_address / ARCH_PAGE_SIZE;
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 85) - [public] kprintf

```
■ kprintf("page_init: no arch mem info\n");
■ return false;
■}

■uint64_t max_pfn = info->max_usable_address / ARCH_PAGE_SIZE;

■page_desc_count = (size_t)(max_pfn + 1);

■size_t bytes = page_desc_count * sizeof(page_t);

■kprintf("page_init: max_usable=0x%llx max_addr=0x%llx desc_count=%llu bytes=%llu\n",
■ (unsigned long long)info->max_usable_address,
■ // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 141) - [public] page_unlock

```
■ page_unlock();
■ return NULL;
■}

■page_t *p = &page_descs[page_phys_to_pfn(phys)];

■p->refcount = 1;
■p->type = type;
■p->flags = 0;
■p->reserved = 0;

■page_unlock();
■ // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 168) - [public] page_unlock

```
■ page_unlock();
■ return NULL;
■}

■page_t *p = &page_descs[page_phys_to_pfn(phys)];

■p->refcount = 1;
■p->type = type;
■p->flags = 0;
■p->reserved = 0;

■page_unlock();
■ // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 205) - [public] page_unlock

```
■ page_unlock();
■ return false;
■}

■p->refcount = 1;
■p->type = type;

■page_unlock();

■return true;
}

■ // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 226) - [public] page_unlock

```
■ page_unlock();  
■ return false;  
■}
```

```
■page->refcount++;
```

```
■page_unlock();
```

```
■return true;  
}
```

```
bool page_put(page_t *page)  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 246) - [public] page_unlock

```
■ page_unlock();  
■ return false;  
■}
```

```
■page->refcount--;
```

```
■if (page->refcount != 0) {  
■    page_unlock();  
■    return false;  
■}
```

```
■size_t idx = (size_t)(page - page_descs);  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 253) - [public] page_unlock

```
■ page_unlock();  
■ return false;  
■}
```

```
■size_t idx = (size_t)(page - page_descs);  
■uint64_t phys = (uint64_t)idx * ARCH_PAGE_SIZE;
```

```
■page->type = PAGE_TYPE_FREE;  
■page->flags = 0;
```

```
■page_unlock();
```

```
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/page.c (line 289) - [public] arch_phys_to_virt

```
■return arch_phys_to_virt(phys);  
}
```

```
size_t page_pfn(const page_t *page)
```

```
{  
■if (!page_ptr_valid(page)) {  
■    return (size_t)-1;  
■}
```

```
■return (size_t)(page - page_descs);  
}
```

```
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 331) - [public] page_from_phys

```
■return page_from_phys(arch_virt_to_phys(va));  
}
```

```
uint32_t page_refcount(const page_t *page)  
{  
■if (!page_ptr_valid(page)) {  
■    return 0;  
■}  
  
■return page->refcount;  
}
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 379) - [public] if

```
■    if (page_descs[i].refcount != 0) {  
■        count++;  
■    }  
■}
```

```
■return count;  
}
```

```
void page_dump(void)  
{  
■if (!page_initialized) {  
■    kprintf("PAGE: not initialized\n");  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.c (line 390) - [public] kprintf

```
■    kprintf("PAGE: not initialized\n");  
■    return;  
■}  
  
■kprintf("PAGE:\n");  
■kprintf("  total pages: %llu\n", (unsigned long long)page_stat_total());  
■kprintf("  referenced: %llu\n", (unsigned long long)page_stat_referenced());  
}
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 7) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);  
  
#define VMM_VMALLOC_BASE 0xFFFFFA0000000000ULL  
#define VMM_VMALLOC_SIZE (128ULL << 30)  
#define VMM_SIZE_MAX ((size_t)-1)  
  
static bitmap_t vmm_bitmap;  
static bool vmm_initialized = false;  
static size_t vmm_total_pages = 0;  
static size_t vmm_allocated_pages = 0;  
  
static size_t vmm_lock_depth = 0;  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 201) - [public] vmm_unlock

```
■    vmm_unlock();  
■    return false;  
■}  
  
■size_t mapped = 0;  
■bool ok = true;  
  
■for (size_t i = 0; i < pages; i++) {  
■    uint64_t phys = 0;  
  
■    bool got = (flags & VMM_FLAG_ZERO)  
■        ? pmm_alloc_zero_frame(&phys)  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 213) - [public] pmm_alloc_frame

```
■          : pmm_alloc_frame(&phys);

■  if (!got) {
■      ok = false;
■      break;
■  }

■  uint64_t virt = vmm_page_virt(start + i);

■  if (!paging_map_page(virt, phys, pte)) {
■      pmm_free_frame(phys);
■      ok = false;
■      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 215) - [public] if

```
■  if (!got) {
■      ok = false;
■      break;
■  }

■  uint64_t virt = vmm_page_virt(start + i);

■  if (!paging_map_page(virt, phys, pte)) {
■      pmm_free_frame(phys);
■      ok = false;
■      break;
■  }
■  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 222) - [public] if

```
■  if (!paging_map_page(virt, phys, pte)) {
■      pmm_free_frame(phys);
■      ok = false;
■      break;
■  }

■  mapped++;
■}

■if (!ok) {
■  for (size_t i = 0; i < mapped; i++) {
■      uint64_t virt = vmm_page_virt(start + i);
■      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 223) - [public] pmm_free_frame

```
■      pmm_free_frame(phys);
■      ok = false;
■      break;
■  }

■  mapped++;
■}

■if (!ok) {
■  for (size_t i = 0; i < mapped; i++) {
■      uint64_t virt = vmm_page_virt(start + i);

■      // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/vmm.c (line 235) - [public] if

```
■    if (paging_is_mapped(virt)) {  
■        uint64_t phys = paging_translate(virt);  
■        paging_unmap_page(virt);  
■        pmm_free_frame(phys);  
■    }  
■  
■    bitmap_free_range(&vmm_bitmap, start, pages);  
■    vmm_unlock();  
■    return false;  
■}  
  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/vmm.c (line 237) - [public] paging_unmap_page

```
■        paging_unmap_page(virt);  
■        pmm_free_frame(phys);  
■    }  
■  
■    bitmap_free_range(&vmm_bitmap, start, pages);  
■    vmm_unlock();  
■    return false;  
■}  
  
vmm_allocated_pages += pages;  
  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/vmm.c (line 238) - [public] pmm_free_frame

```
■        pmm_free_frame(phys);  
■    }  
■  
■    bitmap_free_range(&vmm_bitmap, start, pages);  
■    vmm_unlock();  
■    return false;  
■}  
  
vmm_allocated_pages += pages;  
  
vmm_unlock();  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/vmm.c (line 242) - [public] bitmap_free_range

```
■    bitmap_free_range(&vmm_bitmap, start, pages);  
■    vmm_unlock();  
■    return false;  
■}  
  
vmm_allocated_pages += pages;  
  
vmm_unlock();  
  
*out_virt = vmm_page_virt(start);  
  
return true;  
    // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 243) - [public] vmm_unlock

```
■    vmm_unlock();
■    return false;
■}

■vmm_allocated_pages += pages;

■vmm_unlock();

■*out_virt = vmm_page_virt(start);

■return true;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 258) - [public] vmm_alloc_aligned

```
■return vmm_alloc_aligned(bytes, ARCH_PAGE_SIZE, flags, out_virt);
}

bool vmm_map_device(uint64_t phys, size_t len, uint64_t *out_virt)
{
■if (!vmm_initialized || out_virt == NULL || len == 0) {
■    return false;
■}

■if (!arch_is_page_aligned(phys) || !arch_is_page_aligned(len)) {
■    return false;
■}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 284) - [public] vmm_unlock

```
■    vmm_unlock();
■    return false;
■}

■size_t mapped = 0;
■bool ok = true;

■for (size_t i = 0; i < pages; i++) {
■    uint64_t virt = vmm_page_virt(start + i);
■    uint64_t frame_phys = phys + (uint64_t)i * ARCH_PAGE_SIZE;

■    if (!paging_map_page(virt, frame_phys, pte)) {
■        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 295) - [public] if

```
■    if (!paging_map_page(virt, frame_phys, pte)) {
■        ok = false;
■        break;
■    }

■    mapped++;
■}

■if (!ok) {
■    for (size_t i = 0; i < mapped; i++) {
■        paging_unmap_page(vmm_page_virt(start + i));
■    }
■    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 305) - [public] paging_unmap_page

```
■    paging_unmap_page(vmm_page_virt(start + i));
■    }

■    bitmap_free_range(&vmm_bitmap, start, pages);
■    vmm_unlock();
■    return false;
■}

■vmm_allocated_pages += pages;

■vmm_unlock();

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 308) - [public] bitmap_free_range

```
■    bitmap_free_range(&vmm_bitmap, start, pages);
■    vmm_unlock();
■    return false;
■}

■vmm_allocated_pages += pages;

■vmm_unlock();

■*out_virt = vmm_page_virt(start);

■return true;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 309) - [public] vmm_unlock

```
■    vmm_unlock();
■    return false;
■}

■vmm_allocated_pages += pages;

■vmm_unlock();

■*out_virt = vmm_page_virt(start);

■return true;
}
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 349) - [public] if

```
■    if (paging_is_mapped(v)) {
■        uint64_t phys = paging_translate(v);
■        paging_unmap_page(v);
■        pmm_free_frame(phys);
■    }
■}

■bitmap_free_range(&vmm_bitmap, start, pages);

■if (vmm_allocated_pages >= pages) {
■    vmm_allocated_pages -= pages;
■} else {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 351) - [public] paging_unmap_page

```
■    paging_unmap_page(v);
■    pmm_free_frame(phys);
■    }
■}

■bitmap_free_range(&vmm_bitmap, start, pages);

■if (vmm_allocated_pages >= pages) {
■    vmm_allocated_pages -= pages;
■} else {
■    vmm_allocated_pages = 0;
■}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 352) - [public] pmm_free_frame

```
■    pmm_free_frame(phys);
■    }
■}

■bitmap_free_range(&vmm_bitmap, start, pages);

■if (vmm_allocated_pages >= pages) {
■    vmm_allocated_pages -= pages;
■} else {
■    vmm_allocated_pages = 0;
■}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 395) - [public] if

```
■    if (paging_is_mapped(v)) {
■        paging_unmap_page(v);
■    }
■}

■bitmap_free_range(&vmm_bitmap, start, pages);

■if (vmm_allocated_pages >= pages) {
■    vmm_allocated_pages -= pages;
■} else {
■    vmm_allocated_pages = 0;
■}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.c (line 396) - [public] paging_unmap_page

```
■    paging_unmap_page(v);
■    }
■}

■bitmap_free_range(&vmm_bitmap, start, pages);

■if (vmm_allocated_pages >= pages) {
■    vmm_allocated_pages -= pages;
■} else {
■    vmm_allocated_pages = 0;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/alloc/virtual/vmm.c (line 419) - [public] paging_translate

```
■return paging_translate(virt);
}

size_t vmm_stat_total_pages(void)
{
■if (!vmm_initialized) {
■    return 0;
■}

■return vmm_total_pages;
}
```

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/mm/alloc/virtual/vmm.c (line 466) - [public] kprintf

```
■    kprintf("VMM: not initialized\n");
■    return;
■}

■kprintf("VMM:\n");
■kprintf("  base: 0x%llx\n", (unsigned long long)VMM_VMALLOC_BASE);
■kprintf("  total pages: %llu\n", (unsigned long long)vmm_stat_total_pages());
■kprintf("  free pages: %llu\n", (unsigned long long)vmm_stat_free_pages());
■kprintf("  allocated pages: %llu\n", (unsigned long long)vmm_stat_allocated_pages());
■kprintf("  total: %llu MiB\n", (unsigned long long)(vmm_stat_total_bytes() >> 20));
■kprintf("  free: %llu MiB\n", (unsigned long long)(vmm_stat_free_bytes() >> 20));
}
```

kernel_workspace/kernel/src/mm/arch/aarch64/paging.c (line 115) - [public]

arch_memory_boot_alloc

```
■    arch_memory_boot_alloc(4096, 4096, &phys);

■    uint64_t *t = tbl(phys);

■    for (int i = 0; i < 512; i++) {
■        t[i] = 0;
■    }
■}

■*entry = phys | HW_TABLE;

■return phys;
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/aarch64/paging.c (line 139) - [public] tlb_flush_page_addr

```
■    tlb_flush_page_addr(virt);
■    return true;
■}

■uint64_t l2 = ensure_table(&l1t[IDX(virt, L1_SHIFT)], false);
■uint64_t *l2t = tbl(l2);

■uint64_t l3 = ensure_table(&l2t[IDX(virt, L2_SHIFT)], false);
■uint64_t *l3t = tbl(l3);

■l3t[IDX(virt, L3_SHIFT)] = (phys & PAGING_ADDR_MASK) | hw_bits(flags, 3);

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 172) - [public] tlb_flush_page_addr
    ■ tlb_flush_page_addr(virt);
    ■ return true;
    ■}

    ■uint64_t *l2t = tbl(l1t[IDX(virt, L1_SHIFT)] & PAGING_ADDR_MASK);

    ■if (!(l2t[IDX(virt, L2_SHIFT)] & HW_VALID)) {
    ■    return false;
    ■}

    ■if ((l2t[IDX(virt, L2_SHIFT)] & 3) == HW_BLOCK) {
    ■    l2t[IDX(virt, L2_SHIFT)] = 0;
    ■    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 184) - [public] tlb_flush_page_addr
    ■ tlb_flush_page_addr(virt);
    ■ return true;
    ■}

    ■uint64_t *l3t = tbl(l2t[IDX(virt, L2_SHIFT)] & PAGING_ADDR_MASK);

    ■if (!(l3t[IDX(virt, L3_SHIFT)] & HW_VALID)) {
    ■    return false;
    ■}

    ■l3t[IDX(virt, L3_SHIFT)] = 0;
    ■tlb_flush_page_addr(virt);
    ■    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 216) - [public] return
    ■    return (e1 & 0x0000FFFFFFFFE0000ULL) | (virt & 0x1FFFFFFULL);
    ■}

    ■uint64_t *l2t = tbl(e1 & PAGING_ADDR_MASK);
    ■uint64_t e2 = l2t[IDX(virt, L2_SHIFT)];

    ■if (!(e2 & HW_VALID)) {
    ■    return UINT64_MAX;
    ■}

    ■if ((e2 & 3) == HW_BLOCK) {
    ■    return (e2 & 0x0000FFFFFFFFF0000ULL) | (virt & 0xFFFFFULL);
    ■    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 227) - [public] return
    ■    return (e2 & 0x0000FFFFFFFFF0000ULL) | (virt & 0xFFFFFULL);
    ■}

    ■uint64_t *l3t = tbl(e2 & PAGING_ADDR_MASK);
    ■uint64_t e3 = l3t[IDX(virt, L3_SHIFT)];

    ■if (!(e3 & HW_VALID)) {
    ■    return UINT64_MAX;
    ■}

    ■return (e3 & PAGING_ADDR_MASK) | (virt & 0xFFFFFULL);
    }
    // ... (truncated, see source file for more)

```

```

kernel_workspace/kernel/src/mm/arch/aarch64/paging.c (line 273) - [public] volatile
    ■__asm__ volatile("mrs %0, ttbr1_el1" : "=r"(cur));

    ■uint64_t *boot_l0 = tbl_early(cur & PAGING_ADDR_MASK);

    ■kernel_l0 = alloc_table_early();

    ■uint64_t *l0 = tbl_early(kernel_l0);

    ■for (int i = 0; i < 512; i++) {
    ■    l0[i] = boot_l0[i];
    ■}

    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/mm/arch/aarch64/paging.c (line 297) - [public] if
    ■    if (idx == 0 && pa >= 0x200000 * 512) {
    ■        break;
    ■    }

    ■    l2t[idx] = pa | HW_BLOCK | HW_AF | HW_SH | ATTR_WB | HW_PXN;
    ■}

    ■uint64_t mair = 0x00ULL | (0xFFULL << 8) | (0x44ULL << 16);

    ■__asm__ volatile("msr mair_el1, %0" :: "r"(mair));

    ■uint64_t tcr = (16ULL << 0)
    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/mm/arch/aarch64/paging.c (line 306) - [public] volatile
    ■__asm__ volatile("msr mair_el1, %0" :: "r"(mair));

    ■uint64_t tcr = (16ULL << 0)
    ■    | (16ULL << 16)
    ■    | (0b000ULL << 14)
    ■    | (0b100ULL << 30)
    ■    | (0b010ULL << 8)
    ■    | (0b010ULL << 10)
    ■    | (0b110ULL << 12)
    ■    | (0b010ULL << 24)
    ■    | (0b010ULL << 26)
    ■    | (0b110ULL << 28)
    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/mm/arch/aarch64/paging.c (line 320) - [public] volatile
    ■__asm__ volatile("msr tcr_el1, %0" :: "r"(tcr));

    ■__asm__ volatile("msr ttbr0_el1, %0" :: "r"(kernel_l0));
    ■__asm__ volatile("msr ttbr1_el1, %0" :: "r"(kernel_l0));

    ■__asm__ volatile("isb");
    ■__asm__ volatile("tlbi vmalle1");
    ■__asm__ volatile("dsb sy");
    ■__asm__ volatile("isb");
    ■}

    bool paging_enable_nx(void)
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 322) - [public] volatile

```
    __asm__ volatile("msr ttbr0_el1, %0" :: "r"(kernel_l0));
    __asm__ volatile("msr ttbr1_el1, %0" :: "r"(kernel_l0));

    __asm__ volatile("isb");
    __asm__ volatile("tlbi vmalle1");
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

bool paging_enable_nx(void)
{
    return true;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 323) - [public] volatile

```
    __asm__ volatile("msr ttbr1_el1, %0" :: "r"(kernel_l0));

    __asm__ volatile("isb");
    __asm__ volatile("tlbi vmalle1");
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

bool paging_enable_nx(void)
{
    return true;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 325) - [public] volatile

```
    __asm__ volatile("isb");
    __asm__ volatile("tlbi vmalle1");
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

bool paging_enable_nx(void)
{
    return true;
}

uint64_t paging_read_cr3(void)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 326) - [public] volatile

```
    __asm__ volatile("tlbi vmalle1");
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

bool paging_enable_nx(void)
{
    return true;
}

uint64_t paging_read_cr3(void)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 327) - [public] volatile

```
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

bool paging_enable_nx(void)
{
    return true;
}

uint64_t paging_read_cr3(void)
{
    uint64_t v;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 328) - [public] volatile

```
    __asm__ volatile("isb");
}

bool paging_enable_nx(void)
{
    return true;
}

uint64_t paging_read_cr3(void)
{
    uint64_t v;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 340) - [public] volatile

```
    __asm__ volatile("mrs %0, ttbr1_el1" : "=r"(v));

    return v & PAGING_ADDR_MASK;
}

void paging_write_cr3(uint64_t pml4_phys)
{
    __asm__ volatile("msr ttbr0_el1, %0" :: "r"(pml4_phys));
    __asm__ volatile("msr ttbr1_el1, %0" :: "r"(pml4_phys));
    __asm__ volatile("isb");

    tlb_flush_all();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 347) - [public] volatile

```
    __asm__ volatile("msr ttbr0_el1, %0" :: "r"(pml4_phys));
    __asm__ volatile("msr ttbr1_el1, %0" :: "r"(pml4_phys));
    __asm__ volatile("isb");

    tlb_flush_all();
}
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 348) - [public] volatile

```
    __asm__ volatile("msr ttbr1_el1, %0" :: "r"(pml4_phys));
    __asm__ volatile("isb");

    tlb_flush_all();
}
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.c (line 349) - [public] volatile

```
    __asm__ volatile("isb");

    tlb_flush_all();
}
```


kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 18) - [public] volatile

```
    __asm__ volatile("dsb sy");
    __asm__ volatile("tlbi vmalle1");
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

void tlb_flush_all_including_global(void)
{
    tlb_flush_all();
}

void tlb_flush_page_addr(uint64_t addr)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 19) - [public] volatile

```
    __asm__ volatile("tlbi vmalle1");
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

void tlb_flush_all_including_global(void)
{
    tlb_flush_all();
}

void tlb_flush_page_addr(uint64_t addr)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 20) - [public] volatile

```
    __asm__ volatile("dsb sy");
    __asm__ volatile("isb");
}

void tlb_flush_all_including_global(void)
{
    tlb_flush_all();
}

void tlb_flush_page_addr(uint64_t addr)
{
    __asm__ volatile("dsb ishst");
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 21) - [public] volatile

```
    __asm__ volatile("isb");
}

void tlb_flush_all_including_global(void)
{
    tlb_flush_all();
}

void tlb_flush_page_addr(uint64_t addr)
{
    __asm__ volatile("dsb ishst");
    __asm__ volatile("tlbi vaae1is, %0" :: "r"(addr >> 12));
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 31) - [public] volatile

```
    __asm__ volatile("dsb ishst");
    __asm__ volatile("tlbi vaae1is, %0" :: "r"(addr >> 12));
    __asm__ volatile("dsb ish");
    __asm__ volatile("isb");
}

void tlb_flush_page(const void *addr)
{
    __tlb_flush_page_addr((uint64_t)(uintptr_t)addr);
}

void tlb_flush_range_addr(uint64_t addr, size_t pages)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 32) - [public] volatile

```
    __asm__ volatile("tlbi vaae1is, %0" :: "r"(addr >> 12));
    __asm__ volatile("dsb ish");
    __asm__ volatile("isb");
}

void tlb_flush_page(const void *addr)
{
    __tlb_flush_page_addr((uint64_t)(uintptr_t)addr);
}

void tlb_flush_range_addr(uint64_t addr, size_t pages)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 33) - [public] volatile

```
    __asm__ volatile("dsb ish");
    __asm__ volatile("isb");
}

void tlb_flush_page(const void *addr)
{
    __tlb_flush_page_addr((uint64_t)(uintptr_t)addr);
}

void tlb_flush_range_addr(uint64_t addr, size_t pages)
{
    if (pages > 32) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.c (line 34) - [public] volatile

```
    __asm__ volatile("isb");
}

void tlb_flush_page(const void *addr)
{
    __tlb_flush_page_addr((uint64_t)(uintptr_t)addr);
}

void tlb_flush_range_addr(uint64_t addr, size_t pages)
{
    if (pages > 32) {
        tlb_flush_all();
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/aarch64/tlb.c (line 45) - [public] tlb_flush_all

```
■ tlb_flush_all();
■ return;
■}

■for (size_t i = 0; i < pages; i++) {
■ tlb_flush_page_addr(addr + i * 4096);
■}
}

void tlb_flush_range(const void *addr, size_t pages)
{
■tlb_flush_range_addr((uint64_t)(uintptr_t)addr, pages);
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/aarch64/tlb.c (line 50) - [public] tlb_flush_page_addr

```
■ tlb_flush_page_addr(addr + i * 4096);
■}
}

void tlb_flush_range(const void *addr, size_t pages)
{
■tlb_flush_range_addr((uint64_t)(uintptr_t)addr, pages);
}
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 37) - [public] arch_mem_panic

```
■ arch_mem_panic("bad multiboot2 magic");
■}

■if (mb_info < 8) {
■ arch_mem_panic("bad multiboot2 info address");
■}

■static arch_raw_mem_entry_t raw[ARCH_MAX_MEM_REGIONS];
■size_t n = 0;

■uint32_t total_size = *(const uint32_t *)mb_info;

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 41) - [public] arch_mem_panic

```
■ arch_mem_panic("bad multiboot2 info address");
■}

■static arch_raw_mem_entry_t raw[ARCH_MAX_MEM_REGIONS];
■size_t n = 0;

■uint32_t total_size = *(const uint32_t *)mb_info;

■if (total_size < 8) {
■ arch_mem_panic("multiboot2 info too small");
■}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 50) - [public] arch_mem_panic

```
■ arch_mem_panic("multiboot2 info too small");
■}

■uintptr_t end_addr = mb_info + total_size;
■uintptr_t tag_addr = (mb_info + 8 + 7) & ~(uintptr_t)7;

■while (tag_addr + 8 <= end_addr) {
■ const struct multiboot_tag *tag =
■ (const struct multiboot_tag *)tag_addr;

■ if (tag->size < 8) {
■ break;
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 60) - [public] if

```
■   if (tag->size < 8) {
■       break;
■   }

■   if (tag->type == MULTIBOOT_TAG_TYPE_END) {
■       break;
■   }

■   if (tag->type == MULTIBOOT_TAG_TYPE_MMAP) {
■       const struct multiboot_tag_mmap *mmap =
■           (const struct multiboot_tag_mmap *)tag;

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 64) - [public] if

```
■   if (tag->type == MULTIBOOT_TAG_TYPE_END) {
■       break;
■   }

■   if (tag->type == MULTIBOOT_TAG_TYPE_MMAP) {
■       const struct multiboot_tag_mmap *mmap =
■           (const struct multiboot_tag_mmap *)tag;

        uintptr_t entry_addr = (uintptr_t)mmap->entries;
        uintptr_t mmap_end = tag_addr + tag->size;

■       if (mmap->entry_size < sizeof(struct multiboot_mmap_entry)) {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 68) - [public] if

```
■   if (tag->type == MULTIBOOT_TAG_TYPE_MMAP) {
■       const struct multiboot_tag_mmap *mmap =
■           (const struct multiboot_tag_mmap *)tag;

        uintptr_t entry_addr = (uintptr_t)mmap->entries;
        uintptr_t mmap_end = tag_addr + tag->size;

■       if (mmap->entry_size < sizeof(struct multiboot_mmap_entry)) {
■           break;
■       }

■       while (entry_addr + sizeof(struct multiboot_mmap_entry) <= mmap_end) {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 75) - [public] if

```
■       if (mmap->entry_size < sizeof(struct multiboot_mmap_entry)) {
■           break;
■       }

■       while (entry_addr + sizeof(struct multiboot_mmap_entry) <= mmap_end) {
■           const struct multiboot_mmap_entry *e =
■               (const struct multiboot_mmap_entry *)entry_addr;

■           if (n < ARCH_MAX_MEM_REGIONS) {
■               raw[n].base = e->addr;
■               raw[n].len = e->len;
■               raw[n].type = e->type;
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 79) - [public] while

```
■ while (entry_addr + sizeof(struct multiboot_mmap_entry) <= mmap_end) {
■     const struct multiboot_mmap_entry *e =
■         (const struct multiboot_mmap_entry *)entry_addr;

■     if (n < ARCH_MAX_MEM_REGIONS) {
■         raw[n].base = e->addr;
■         raw[n].len = e->len;
■         raw[n].type = e->type;
■         n++;
■     }

■     entry_addr += mmap->entry_size;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 83) - [public] if

```
■     if (n < ARCH_MAX_MEM_REGIONS) {
■         raw[n].base = e->addr;
■         raw[n].len = e->len;
■         raw[n].type = e->type;
■         n++;
■     }

■     entry_addr += mmap->entry_size;
■ }
■ }

■ tag_addr = (tag_addr + tag->size + 7) & ~(uintptr_t)7;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 106) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

static void arch_mem_panic(const char *msg) __attribute__((noreturn));
static void arch_mem_panic(const char *msg)
{
■ kprintf("arch/x86_64/memory.c panic: %s\n", msg);

■ for (;;) {
■     __asm__ volatile("cli; hlt");
■ }
}
#define ARCH_U64_MAX        UINT64_MAX
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 108) - [public] arch_mem_panic

```
static void arch_mem_panic(const char *msg) __attribute__((noreturn));
static void arch_mem_panic(const char *msg)
{
■ kprintf("arch/x86_64/memory.c panic: %s\n", msg);

■ for (;;) {
■     __asm__ volatile("cli; hlt");
■ }
}
#define ARCH_U64_MAX        UINT64_MAX
#define ARCH_TMP_REGIONS    (ARCH_MAX_MEM_REGIONS * 3)

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 221) - [public] while

```
■ while (j > 0 && entries[j - 1].base > key.base) {
■     entries[j] = entries[j - 1];
■     j--;
■ }

■ entries[j] = key;
■ }
■ }
```

```
static void map_add(arch_mem_region_t *list,
■     size_t *count,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 244) - [public] if

```
■ if (r.base < last_end) {
■     uint64_t delta = last_end - r.base;

■     if (r.len <= delta) {
■         return;
■     }

■     r.base = last_end;
■     r.len -= delta;
■ }

■
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 247) - [public] if

```
■ if (r.len <= delta) {
■     return;
■ }

■ r.base = last_end;
■ r.len -= delta;
■ }

■
■ if (safe_end(last->base, last->len) == r.base &&
■     last->type == r.type) {
■     last->len += r.len;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 264) - [public] arch_mem_panic

```
■ arch_mem_panic("too many memory regions");
■ }

■ list[(*count)++] = r;
■ }

static void map_reserve_range(uint64_t base, uint64_t len)
{
■ if (len == 0) {
■     return;
■ }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 293) - [public] if

```
■   if (r_end <= base || r.base >= end) {
■       map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, r);
■       continue;
■   }

■
■   if (r.base < base) {
■       arch_mem_region_t before;
■       before.base = r.base;
■       before.len = base - r.base;
■       before.type = r.type;

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 294) - [public] map_add

```
■       map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, r);
■       continue;
■   }

■
■   if (r.base < base) {
■       arch_mem_region_t before;
■       before.base = r.base;
■       before.len = base - r.base;
■       before.type = r.type;

■       map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, before);
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 299) - [public] if

```
■   if (r.base < base) {
■       arch_mem_region_t before;
■       before.base = r.base;
■       before.len = base - r.base;
■       before.type = r.type;

■       map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, before);
■   }

■
■   uint64_t ov_base = u64_max(r.base, base);
■   uint64_t ov_end = u64_min(r_end, end);
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 305) - [public] map_add

```
■       map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, before);
■   }

■
■   uint64_t ov_base = u64_max(r.base, base);
■   uint64_t ov_end = u64_min(r_end, end);

■   if (ov_end > ov_base) {
■       arch_mem_region_t reserved;
■       reserved.base = ov_base;
■       reserved.len = ov_end - ov_base;
■       reserved.type = ARCH_MEM_TYPE_RESERVED;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 312) - [public] if

```
■ if (ov_end > ov_base) {
■     arch_mem_region_t reserved;
■     reserved.base = ov_base;
■     reserved.len = ov_end - ov_base;
■     reserved.type = ARCH_MEM_TYPE_RESERVED;

■     map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, reserved);
■ }

■
■ if (r_end > end) {
■     arch_mem_region_t after;
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 318) - [public] map_add

```
■     map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, reserved);
■ }

■
■ if (r_end > end) {
■     arch_mem_region_t after;
■     after.base = end;
■     after.len = r_end - end;
■     after.type = r.type;

■     map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, after);
■ }
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 322) - [public] if

```
■ if (r_end > end) {
■     arch_mem_region_t after;
■     after.base = end;
■     after.len = r_end - end;
■     after.type = r.type;

■     map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, after);
■ }
■ }

■ if (tcount > ARCH_MAX_MEM_REGIONS) {
■     arch_mem_panic("memory reserve produced too many regions");
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 328) - [public] map_add

```
■     map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, after);
■ }
■ }

■ if (tcount > ARCH_MAX_MEM_REGIONS) {
■     arch_mem_panic("memory reserve produced too many regions");
■ }

■ mem.count = tcount;

■ for (size_t i = 0; i < tcount; i++) {
■     mem.regions[i] = tmp_regions[i];
■     // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 333) - [public] arch_mem_panic

```
■ arch_mem_panic("memory reserve produced too many regions");
■ }

■ mem.count = tcount;

■ for (size_t i = 0; i < tcount; i++) {
■     mem.regions[i] = tmp_regions[i];
■ }
}

static void map_align_usable_regions(void)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 351) - [public] if

```
■ if (r.type != ARCH_MEM_TYPE_USABLE) {
■     map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, r);
■     continue;
■ }

■ uint64_t aligned_base = align_up_page_safe(r.base);
■ uint64_t aligned_end = align_down_page(r_end);

■ if (aligned_end <= aligned_base) {
■     arch_mem_region_t tiny;
■     tiny.base = r.base;
■     tiny.len = r.len;
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 352) - [public] map_add

```
■     map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, r);
■     continue;
■ }

■ uint64_t aligned_base = align_up_page_safe(r.base);
■ uint64_t aligned_end = align_down_page(r_end);

■ if (aligned_end <= aligned_base) {
■     arch_mem_region_t tiny;
■     tiny.base = r.base;
■     tiny.len = r.len;
■     tiny.type = ARCH_MEM_TYPE_RESERVED;
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 359) - [public] if

```
■ if (aligned_end <= aligned_base) {
■     arch_mem_region_t tiny;
■     tiny.base = r.base;
■     tiny.len = r.len;
■     tiny.type = ARCH_MEM_TYPE_RESERVED;

■     map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, tiny);
■     continue;
■ }

■ if (r.base < aligned_base) {
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 365) - [public] map_add

```
■    map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, tiny);
■    continue;
■ }

■
■    if (r.base < aligned_base) {
■        arch_mem_region_t before;
■        before.base = r.base;
■        before.len = aligned_base - r.base;
■        before.type = ARCH_MEM_TYPE_RESERVED;

■        map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, before);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 370) - [public] if

```
■    if (r.base < aligned_base) {
■        arch_mem_region_t before;
■        before.base = r.base;
■        before.len = aligned_base - r.base;
■        before.type = ARCH_MEM_TYPE_RESERVED;

■        map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, before);
■    }

■
■    {
■        arch_mem_region_t usable;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 376) - [public] map_add

```
■    map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, before);
■ }

■
■    {
■        arch_mem_region_t usable;
■        usable.base = aligned_base;
■        usable.len = aligned_end - aligned_base;
■        usable.type = ARCH_MEM_TYPE_USABLE;

■        map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, usable);
■    }
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 386) - [public] map_add

```
■    map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, usable);
■ }

■
■    if (aligned_end < r_end) {
■        arch_mem_region_t after;
■        after.base = aligned_end;
■        after.len = r_end - aligned_end;
■        after.type = ARCH_MEM_TYPE_RESERVED;

■        map_add(tmp_regions, &tcnt, ARCH_TMP_REGIONS, after);
■    }
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 390) - [public] if

```
■    if (aligned_end < r_end) {
■        arch_mem_region_t after;
■        after.base = aligned_end;
■        after.len = r_end - aligned_end;
■        after.type = ARCH_MEM_TYPE_RESERVED;

■        map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, after);
■    }
■}

■if (tcount > ARCH_MAX_MEM_REGIONS) {
■    arch_mem_panic("memory alignment produced too many regions");
■    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 396) - [public] map_add

```
■        map_add(tmp_regions, &tcount, ARCH_TMP_REGIONS, after);
■    }
■}

■if (tcount > ARCH_MAX_MEM_REGIONS) {
■    arch_mem_panic("memory alignment produced too many regions");
■}

■mem.count = tcount;

■for (size_t i = 0; i < tcount; i++) {
■    mem.regions[i] = tmp_regions[i];
■    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 401) - [public] arch_mem_panic

```
■    arch_mem_panic("memory alignment produced too many regions");
■}

■mem.count = tcount;

■for (size_t i = 0; i < tcount; i++) {
■    mem.regions[i] = tmp_regions[i];
■}
}

static void recalc_stats(void)
{
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 421) - [public] if

```
■    if (end > max_address) {
■        max_address = end;
■    }

■    if (r->type == ARCH_MEM_TYPE_USABLE) {
■        if (total_usable > ARCH_U64_MAX - r->len) {
■            total_usable = ARCH_U64_MAX;
■        } else {
■            total_usable += r->len;
■        }
■    }

■    if (end > max_usable_address) {
■        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 425) - [public] if

```
■ if (r->type == ARCH_MEM_TYPE_USABLE) {
■     if (total_usable > ARCH_U64_MAX - r->len) {
■         total_usable = ARCH_U64_MAX;
■     } else {
■         total_usable += r->len;
■     }

■     if (end > max_usable_address) {
■         max_usable_address = end;
■     }
■ }
■ // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 426) - [public] if

```
■ if (total_usable > ARCH_U64_MAX - r->len) {
■     total_usable = ARCH_U64_MAX;
■ } else {
■     total_usable += r->len;
■ }

■ if (end > max_usable_address) {
■     max_usable_address = end;
■ }
■ }
■ // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 432) - [public] if

```
■ if (end > max_usable_address) {
■     max_usable_address = end;
■ }
■ }
■ mem.total_usable = total_usable;
■ mem.max_address = max_address;
■ mem.max_usable_address = max_usable_address;
■ mem.direct_map_base = ARCH_DIRECT_MAP_BASE;
■ }

■ // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 452) - [public] arch_mem_panic

```
■ arch_mem_panic("arch_memory_init() called twice");
■ }

■ if (entries == NULL || count == 0) {
■     arch_mem_panic("empty memory map");
■ }

■ if (count > ARCH_MAX_MEM_REGIONS) {
■     arch_mem_panic("memory map too large");
■ }

■ mem.count = 0;
■ // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 456) - [public] arch_mem_panic

```
■ arch_mem_panic("empty memory map");
■}

■if (count > ARCH_MAX_MEM_REGIONS) {
■ arch_mem_panic("memory map too large");
■}

■mem.count = 0;

■for (size_t i = 0; i < count; i++) {
■ raw_sorted[i] = entries[i];
■}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 460) - [public] arch_mem_panic

```
■ arch_mem_panic("memory map too large");
■}

■mem.count = 0;

■for (size_t i = 0; i < count; i++) {
■ raw_sorted[i] = entries[i];
■}

■sort_raw_entries(raw_sorted, count);

■for (size_t i = 0; i < count; i++) {
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 475) - [public] if

```
■ if (base >= end) {
■ continue;
■ }

■ if (mem.count > 0) {
■ const arch_mem_region_t *last = &mem.regions[mem.count - 1];
■ uint64_t last_end = safe_end(last->base, last->len);

■ if (base < last_end) {
■ base = last_end;
■ }

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 479) - [public] if

```
■ if (mem.count > 0) {
■ const arch_mem_region_t *last = &mem.regions[mem.count - 1];
■ uint64_t last_end = safe_end(last->base, last->len);

■ if (base < last_end) {
■ base = last_end;
■ }

■ if (base >= end) {
■ continue;
■ }

■ }

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 483) - [public] if

```
■      if (base < last_end) {
■          base = last_end;
■      }

■      if (base >= end) {
■          continue;
■      }
■  }

■      arch_mem_region_t r;
■      r.base = base;
■      r.len = end - base;
■      // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 487) - [public] if

```
■      if (base >= end) {
■          continue;
■      }
■  }

■      arch_mem_region_t r;
■      r.base = base;
■      r.len = end - base;
■      r.type = raw_type_to_arch(raw_sorted[i].type);

■      map_add(mem.regions, &mem.count, ARCH_MAX_MEM_REGIONS, r);
■  }
■  // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 497) - [public] map_add

```
■      map_add(mem.regions, &mem.count, ARCH_MAX_MEM_REGIONS, r);
■  }

■  if (mem.count == 0) {
■      arch_mem_panic("no valid memory regions after parsing");
■  }

■  map_reserve_range(0, 0x1000000);

■  if (kernel_phys_end > kernel_phys_start) {
■      map_reserve_range(kernel_phys_start,
■          // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/arch/x86_64/memory.c (line 501) - [public] arch_mem_panic

```
■      arch_mem_panic("no valid memory regions after parsing");
■  }

■  map_reserve_range(0, 0x1000000);

■  if (kernel_phys_end > kernel_phys_start) {
■      map_reserve_range(kernel_phys_start,
■          kernel_phys_end - kernel_phys_start);
■  }

■  if (initrd_phys_end > initrd_phys_start) {
■      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 571) - [public] if

```
■ if (r->type != ARCH_MEM_TYPE_USABLE) {
■     continue;
■ }

■ uint64_t r_end = safe_end(r->base, r->len);

■ if (base >= r->base && end <= r_end) {
■     return true;
■ }
■}

■return false;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 577) - [public] if

```
■ if (base >= r->base && end <= r_end) {
■     return true;
■ }
■}

■return false;
}

void arch_memory_reserve_range(uint64_t base, uint64_t len)
{
■map_reserve_range(base, len);
■recalc_stats();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 619) - [public] if

```
■ if (r->type != ARCH_MEM_TYPE_USABLE) {
■     continue;
■ }

■ uint64_t r_end = safe_end(r->base, r->len);
■ uint64_t candidate = align_up_generic(r->base, align);

■ if (candidate < r->base) {
■     continue;
■ }

■ if (candidate <= r_end && (r_end - candidate) >= len) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 626) - [public] if

```
■ if (candidate < r->base) {
■     continue;
■ }

■ if (candidate <= r_end && (r_end - candidate) >= len) {
■     *out_base = candidate;
■     return true;
■ }
■}

■return false;
}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.c (line 630) - [public] if

```
    if (candidate <= r_end && (r_end - candidate) >= len) {
        *out_base = candidate;
        return true;
    }

return false;
}
```

```
static const char *region_type_name(arch_mem_type_t type)
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 5) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);

static void paging_panic(const char *msg) __attribute__((noreturn));
static void paging_panic(const char *msg)
{
    kprintf("arch/x86_64/paging.c panic: %s\n", msg);

    for (;;) {
        __asm__ volatile("cli; hlt");
    }

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 7) - [public] paging_panic

```
static void paging_panic(const char *msg) __attribute__((noreturn));
static void paging_panic(const char *msg)
{
    kprintf("arch/x86_64/paging.c panic: %s\n", msg);

    for (;;) {
        __asm__ volatile("cli; hlt");
    }

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 10) - [public] kprintf

```
kprintf("arch/x86_64/paging.c panic: %s\n", msg);

for (;;) {
    __asm__ volatile("cli; hlt");
}

}
```

```
static uint64_t boot_phys_offset = 0;
static bool boot_phys_offset_valid = false;

// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 24) - [public] if

```
    if (!boot_phys_offset_valid) {
        paging_panic("boot physical offset not set");
    }

    return (void *)(uintptr_t)(phys + boot_phys_offset);
}

static void zero_page_phys(uint64_t phys)
{
    uint64_t *p = (uint64_t *)phys_to_ptr(phys);

    for (size_t i = 0; i < 512; i++) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 25) - [public] paging_panic

```
    paging_panic("boot physical offset not set");
}

    return (void *)(uintptr_t)(phys + boot_phys_offset);
}

static void zero_page_phys(uint64_t phys)
{
    uint64_t *p = (uint64_t *)phys_to_ptr(phys);

    for (size_t i = 0; i < 512; i++) {
        p[i] = 0;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 28) - [public] return

```
    return (void *)(uintptr_t)(phys + boot_phys_offset);
}

static void zero_page_phys(uint64_t phys)
{
    uint64_t *p = (uint64_t *)phys_to_ptr(phys);

    for (size_t i = 0; i < 512; i++) {
        p[i] = 0;
    }

    __asm__ volatile("" ::: "memory");
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 39) - [public] volatile

```
    __asm__ volatile("" ::: "memory");
}

static inline size_t pml4_index(uint64_t virt)
{
    return (size_t)((virt >> 39) & 0x1FF);
}

static inline size_t pdpt_index(uint64_t virt)
{
    return (size_t)((virt >> 30) & 0x1FF);
}

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 44) - [public] return
    return (size_t)((virt >> 39) & 0x1FF);
}

static inline size_t pdpt_index(uint64_t virt)
{
    return (size_t)((virt >> 30) & 0x1FF);
}

static inline size_t pd_index(uint64_t virt)
{
    return (size_t)((virt >> 21) & 0x1FF);
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 49) - [public] return
    return (size_t)((virt >> 30) & 0x1FF);
}

static inline size_t pd_index(uint64_t virt)
{
    return (size_t)((virt >> 21) & 0x1FF);
}

static inline size_t pt_index(uint64_t virt)
{
    return (size_t)((virt >> 12) & 0x1FF);
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 54) - [public] return
    return (size_t)((virt >> 21) & 0x1FF);
}

static inline size_t pt_index(uint64_t virt)
{
    return (size_t)((virt >> 12) & 0x1FF);
}

uint64_t paging_read_cr3(void)
{
    uint64_t cr3;

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 59) - [public] return
    return (size_t)((virt >> 12) & 0x1FF);
}

uint64_t paging_read_cr3(void)
{
    uint64_t cr3;

    __asm__ volatile("mov %%cr3, %0" : "=r"(cr3));

    return cr3 & PAGING_ADDR_MASK;
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 66) - [public] volatile
    __asm__ volatile("mov %%cr3, %0" : "=r"(cr3));

    return cr3 & PAGING_ADDR_MASK;
}

void paging_flush_tlb_all(void)
{
    tlb_flush_all();
}

void paging_flush_page(uint64_t addr)
{
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 73) - [public] tlb_flush_all
    tlb_flush_all();
}

void paging_flush_page(uint64_t addr)
{
    tlb_flush_page_addr(addr);
}

static uint64_t alloc_zeroed_table_page(void)
{
    uint64_t phys = 0;

    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 78) - [public] tlb_flush_page_addr
    tlb_flush_page_addr(addr);
}

static uint64_t alloc_zeroed_table_page(void)
{
    uint64_t phys = 0;

    if (!arch_memory_boot_alloc(PAGING_PAGE_SIZE,
                                PAGING_PAGE_SIZE,
                                &phys)) {
        paging_panic("cannot allocate page table page");
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 88) - [public] paging_panic
    paging_panic("cannot allocate page table page");
}

zero_page_phys(phys);

return phys;
}

static uint64_t ensure_table_entry(uint64_t *table,
                                   size_t index,
                                   uint64_t flags)
{
    // ... (truncated, see source file for more)
}

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 91) - [public] zero_page_phys
    zero_page_phys(phys);

    return phys;
}

static uint64_t ensure_table_entry(uint64_t *table,
                                   size_t index,
                                   uint64_t flags)
{
    uint64_t entry = table[index];

    if (entry & PTE_PRESENT) {
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 102) - [public] if
    if (entry & PTE_PRESENT) {
        if (entry & PTE_PAGE_SIZE) {
            /*
             * Large page (2MB or 1GB) found where we need a page table.
             * Split it into smaller 4KB page entries.
             */
            uint64_t large_phys = entry & PAGING_ADDR_MASK;
            uint64_t large_flags = entry & (PAGING_ADDR_MASK - 1);

            /* Allocate a new page table */
            uint64_t table_phys = alloc_zeroed_table_page();
            uint64_t *new_table = (uint64_t *)phys_to_ptr(table_phys);
            // ... (truncated, see source file for more)
        }
    }

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 103) - [public] if
    if (entry & PTE_PAGE_SIZE) {
        /*
         * Large page (2MB or 1GB) found where we need a page table.
         * Split it into smaller 4KB page entries.
         */
        uint64_t large_phys = entry & PAGING_ADDR_MASK;
        uint64_t large_flags = entry & (PAGING_ADDR_MASK - 1);

        /* Allocate a new page table */
        uint64_t table_phys = alloc_zeroed_table_page();
        uint64_t *new_table = (uint64_t *)phys_to_ptr(table_phys);

        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 121) - [public] if
    if (i < pages) {
        uint64_t page_phys = large_phys + (uint64_t)i * PAGING_PAGE_SIZE;
        new_table[i] = (page_phys & PAGING_ADDR_MASK) |
            (large_flags & ~(PTE_PAGE_SIZE));
    } else {
        new_table[i] = 0;
    }
}

/*
 * Replace the large page entry with the new page table entry.
 * Preserve the access flags from the large page.
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 135) - [public] if
    if (large_flags & PTE_USER) {
        table_flags |= PTE_USER;
    }
    table[index] = (table_phys & PAGING_ADDR_MASK) | table_flags;

    return table_phys;
}

return entry & PAGING_ADDR_MASK;
}

uint64_t table_phys = alloc_zeroed_table_page();
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 154) - [public] if
    if (flags & PTE_USER) {
        table_flags |= PTE_USER;
    }

    table[index] = (table_phys & PAGING_ADDR_MASK) | table_flags;

    return table_phys;
}

bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags)
{
    if (!boot_phys_offset_valid) {
        // ... (truncated, see source file for more)
    }

    if (!boot_phys_offset_valid) {
        return false;
    }

    if ((virt & PAGING_PAGE_MASK) != 0 ||
        (phys & PAGING_PAGE_MASK) != 0) {
        return false;
    }

    /*
     * Tymczasowo nie używamy NX, dopóki EFER.NXE nie jest włączony.
     */
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 197) - [public] paging_flush_page
    paging_flush_page(virt);

    return true;
}

static bool map_page_2m(uint64_t virt, uint64_t phys, uint64_t flags)
{
    if (!boot_phys_offset_valid) {
        return false;
    }

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 205) - [public] if
    if (!boot_phys_offset_valid) {
        return false;
    }

    if ((virt & PAGING_2M_PAGE_MASK) != 0 ||
        (phys & PAGING_2M_PAGE_MASK) != 0) {
        return false;
    }

    flags &= ~PTE_NX;
    flags &= ~PTE_PAGE_SIZE;

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 233) - [public] if
/*
 * Jeżeli istnieje już tablica 4 KiB w tym miejscu,
 * nie nadpisujemy jej 2 MiB stron.
 */

    if ((old & PTE_PRESENT) && !(old & PTE_PAGE_SIZE)) {
        paging_panic("cannot map 2M page over existing 4K table");
    }

    pd[idx] =
        (phys & PAGING_ADDR_MASK) |
        flags |
        PTE_PRESENT |
        PTE_PAGE_SIZE;

    return true;
}
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 234) - [public] paging_panic
    paging_panic("cannot map 2M page over existing 4K table");
}

    pd[idx] =
        (phys & PAGING_ADDR_MASK) |
        flags |
        PTE_PRESENT |
        PTE_PAGE_SIZE;

    return true;
}

// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 248) - [public] if
    if (!boot_phys_offset_valid) {
        return false;
    }

    if (len == 0) {
        return true;
    }

    if ((virt & PAGING_PAGE_MASK) != 0 ||
        (phys & PAGING_PAGE_MASK) != 0 ||
        (len & PAGING_PAGE_MASK) != 0) {
        return false;
    }
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 252) - [public] if

```
    if (len == 0) {
        return true;
    }

    if ((virt & PAGING_PAGE_MASK) != 0 ||
        (phys & PAGING_PAGE_MASK) != 0 ||
        (len & PAGING_PAGE_MASK) != 0) {
        return false;
    }

    if (len > UINT64_MAX - virt || len > UINT64_MAX - phys) {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 262) - [public] if

```
    if (len > UINT64_MAX - virt || len > UINT64_MAX - phys) {
        return false;
    }

    uint64_t virt_end = virt + len;
    uint64_t phys_end = phys + len;

    while (virt < virt_end) {
        uint64_t remaining = virt_end - virt;

        bool can_use_2m =
            (remaining >= PAGING_2M_PAGE_SIZE) &&
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 269) - [public] while

```
    while (virt < virt_end) {
        uint64_t remaining = virt_end - virt;

        bool can_use_2m =
            (remaining >= PAGING_2M_PAGE_SIZE) &&
            ((virt & PAGING_2M_PAGE_MASK) == 0) &&
            ((phys & PAGING_2M_PAGE_MASK) == 0);

        if (can_use_2m) {
            if (!map_page_2m(virt, phys, flags)) {
                return false;
            }
        }
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 277) - [public] if

```
    if (can_use_2m) {
        if (!map_page_2m(virt, phys, flags)) {
            return false;
        }

        virt += PAGING_2M_PAGE_SIZE;
        phys += PAGING_2M_PAGE_SIZE;
    } else {
        if (!paging_map_page(virt, phys, flags)) {
            return false;
        }
    }

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 278) - [public] if
    if (!map_page_2m(virt, phys, flags)) {
        return false;
    }

    virt += PAGING_2M_PAGE_SIZE;
    phys += PAGING_2M_PAGE_SIZE;
} else {
    if (!paging_map_page(virt, phys, flags)) {
        return false;
    }

    virt += PAGING_PAGE_SIZE;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 285) - [public] if
    if (!paging_map_page(virt, phys, flags)) {
        return false;
    }

    virt += PAGING_PAGE_SIZE;
    phys += PAGING_PAGE_SIZE;
}

    paging_flush_tlb_all();

    return true;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 294) - [public] paging_flush_tlb_all
    paging_flush_tlb_all();

    return true;
}

void paging_set_boot_phys_offset(uint64_t phys_offset)
{
    if ((phys_offset & PAGING_PAGE_MASK) != 0) {
        paging_panic("boot physical offset must be page aligned");
    }

    boot_phys_offset = phys_offset;
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 301) - [public] if
    if ((phys_offset & PAGING_PAGE_MASK) != 0) {
        paging_panic("boot physical offset must be page aligned");
    }

    boot_phys_offset = phys_offset;
    boot_phys_offset_valid = true;
}

bool paging_boot_phys_offset_valid(void)
{
    return boot_phys_offset_valid;
}

// ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 302) - [public] paging_panic
    paging_panic("boot physical offset must be page aligned");
}

    boot_phys_offset = phys_offset;
    boot_phys_offset_valid = true;
}

bool paging_boot_phys_offset_valid(void)
{
    return boot_phys_offset_valid;
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 316) - [public] if
    if (!arch_memory_ready()) {
        paging_panic("arch memory not initialized");
    }

    if (!boot_phys_offset_valid) {
        paging_panic("boot physical offset not set");
    }

    /*
     * Jeeli bootloader ju mapuje fizyczn pamie dokadnie pod
     * ARCH_DIRECT_MAP_BASE, nie musimy nic robi.
     */
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 317) - [public] paging_panic
    paging_panic("arch memory not initialized");
}

    if (!boot_phys_offset_valid) {
        paging_panic("boot physical offset not set");
    }

    /*
     * Jeeli bootloader ju mapuje fizyczn pamie dokadnie pod
     * ARCH_DIRECT_MAP_BASE, nie musimy nic robi.
     */
    if (boot_phys_offset == ARCH_DIRECT_MAP_BASE) {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 320) - [public] if
    if (!boot_phys_offset_valid) {
        paging_panic("boot physical offset not set");
    }

    /*
     * Jeeli bootloader ju mapuje fizyczn pamie dokadnie pod
     * ARCH_DIRECT_MAP_BASE, nie musimy nic robi.
     */
    if (boot_phys_offset == ARCH_DIRECT_MAP_BASE) {
        return;
    }

    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 321) - [public] paging_panic

```
    paging_panic("boot physical offset not set");
}

/*
 * Je■li bootloader ju■ mapuje fizyczn■ pamie■ dok■adnie pod
 * ARCH_DIRECT_MAP_BASE, nie musimy nic robi■.
 */
if (boot_phys_offset == ARCH_DIRECT_MAP_BASE) {
    return;
}

const arch_mem_info_t *info = arch_memory_get();
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 328) - [public] if

```
/*
 * Je■li bootloader ju■ mapuje fizyczn■ pamie■ dok■adnie pod
 * ARCH_DIRECT_MAP_BASE, nie musimy nic robi■.
 */

if (boot_phys_offset == ARCH_DIRECT_MAP_BASE) {
    return;
}

const arch_mem_info_t *info = arch_memory_get();

if (info == NULL) {
    paging_panic("no arch memory info");
}

uint64_t phys_limit = arch_page_align_up(info->max_address);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 334) - [public] if

```
if (info == NULL) {
    paging_panic("no arch memory info");
}

uint64_t phys_limit = arch_page_align_up(info->max_address);

if (phys_limit == 0 || phys_limit == UINT64_MAX) {
    paging_panic("invalid physical memory limit");
}

if (!paging_map_range(ARCH_DIRECT_MAP_BASE,
                      0,
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 335) - [public] paging_panic

```
    paging_panic("no arch memory info");
}

uint64_t phys_limit = arch_page_align_up(info->max_address);

if (phys_limit == 0 || phys_limit == UINT64_MAX) {
    paging_panic("invalid physical memory limit");
}

if (!paging_map_range(ARCH_DIRECT_MAP_BASE,
                      0,
                      phys_limit,
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 340) - [public] if

```
if (phys_limit == 0 || phys_limit == UINT64_MAX) {
    paging_panic("invalid physical memory limit");
}

if (!paging_map_range(ARCH_DIRECT_MAP_BASE,
                      0,
                      phys_limit,
                      PAGING_KERNEL_RW)) {
    paging_panic("failed to create direct map");
}

paging_flush_tlb_all();
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 341) - [public] paging_panic

```
    paging_panic("invalid physical memory limit");
}

if (!paging_map_range(ARCH_DIRECT_MAP_BASE,
                      0,
                      phys_limit,
                      PAGING_KERNEL_RW)) {
    paging_panic("failed to create direct map");
}

paging_flush_tlb_all();
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 348) - [public] paging_panic

```
    paging_panic("failed to create direct map");
}

paging_flush_tlb_all();
}

void paging_init(uint64_t phys_offset)
{
    paging_set_boot_phys_offset(phys_offset);
    paging_init_direct_map();
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 351) - [public] paging_flush_tlb_all

```
    paging_flush_tlb_all();
}

void paging_init(uint64_t phys_offset)
{
    paging_set_boot_phys_offset(phys_offset);
    paging_init_direct_map();
}

/* ----- */
/* Page-table walk helper */
/* ----- */
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 356) - [public]

paging_set_boot_phys_offset

```
    paging_set_boot_phys_offset(phys_offset);
    paging_init_direct_map();
}

/* ----- */
/* Page-table walk helper */
/* ----- */

/*
 * Walk the 4-level page table rooted at `pml4_phys` and return a pointer
 * to the PTE for `virt`. When `create` is true, missing intermediate
 * tables are allocated (with kernel RW flags); otherwise the walk stops
 * // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 357) - [public]

paging_init_direct_map

```
    paging_init_direct_map();
}

/* ----- */
/* Page-table walk helper */
/* ----- */

/*
 * Walk the 4-level page table rooted at `pml4_phys` and return a pointer
 * to the PTE for `virt`. When `create` is true, missing intermediate
 * tables are allocated (with kernel RW flags); otherwise the walk stops
 * and returns NULL on the first absent entry.
 * // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 367) - [public] allocated

```
/*
 * Walk the 4-level page table rooted at `pml4_phys` and return a pointer
 * to the PTE for `virt`. When `create` is true, missing intermediate
 *
 * tables are allocated (with kernel RW flags); otherwise the walk stops
 * and returns NULL on the first absent entry.
 *
 * Returns NULL on absent entries (create == false), allocation failure,
 * or unexpected large pages.
 */
static uint64_t *resolve_pte(uint64_t pml4_phys, uint64_t virt, bool create)
{
    uint64_t *pml4 = phys_to_ptr(pml4_phys);
    uint64_t *pdpt, *pd, *pt;
    uint64_t entry;
    size_t idx;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 382) - [public] if

```
    if (!(entry & PTE_PRESENT)) {
        if (!create) {
            return NULL;
        }
        ensure_table_entry(pml4, idx, PTE_PRESENT | PTE_WRITABLE);
        entry = pml4[idx];
    }
    pdpt = phys_to_ptr(entry & PAGING_ADDR_MASK);

    idx = pdpt_index(virt);
    entry = pdpt[idx];
    if (!(entry & PTE_PRESENT)) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 383) - [public] if

```
    if (!create) {
        return NULL;
    }
    ensure_table_entry(pml4, idx, PTE_PRESENT | PTE_WRITABLE);
    entry = pml4[idx];
}
pdpt = phys_to_ptr(entry & PAGING_ADDR_MASK);

idx = pdpt_index(virt);
entry = pdpt[idx];
if (!(entry & PTE_PRESENT)) {
    if (!create) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 386) - [public] ensure_table_entry

```
    ensure_table_entry(pml4, idx, PTE_PRESENT | PTE_WRITABLE);
    entry = pml4[idx];
}
pdpt = phys_to_ptr(entry & PAGING_ADDR_MASK);

idx = pdpt_index(virt);
entry = pdpt[idx];
if (!(entry & PTE_PRESENT)) {
    if (!create) {
        return NULL;
    }
    ensure_table_entry(pdpt, idx, PTE_PRESENT | PTE_WRITABLE);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 393) - [public] if

```
if (!(entry & PTE_PRESENT)) {
    if (!create) {
        return NULL;
    }
    ensure_table_entry(pdpt, idx, PTE_PRESENT | PTE_WRITABLE);
    entry = pdpt[idx];
}
if (entry & PTE_PAGE_SIZE) {
    return NULL; /* 1G large page - not handled here */
}
pd = phys_to_ptr(entry & PAGING_ADDR_MASK);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 394) - [public] if

```
    if (!create) {
        return NULL;
    }
    ensure_table_entry(pdpt, idx, PTE_PRESENT | PTE_WRITABLE);
    entry = pdpt[idx];
}
if (entry & PTE_PAGE_SIZE) {
    return NULL; /* 1G large page - not handled here */
}
pd = phys_to_ptr(entry & PAGING_ADDR_MASK);

idx = pd_index(virt);
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 397) - [public] ensure_table_entry
    ensure_table_entry(pdpt, idx, PTE_PRESENT | PTE_WRITABLE);
    entry = pdpt[idx];
}
if (entry & PTE_PAGE_SIZE) {
    return NULL; /* 1G large page - not handled here */
}
pd = phys_to_ptr(entry & PAGING_ADDR_MASK);

idx = pd_index(virt);
entry = pd[idx];
if (!(entry & PTE_PRESENT)) {
    if (!create) {
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 400) - [public] if
    if (entry & PTE_PAGE_SIZE) {
        return NULL; /* 1G large page - not handled here */
    }
    pd = phys_to_ptr(entry & PAGING_ADDR_MASK);

    idx = pd_index(virt);
    entry = pd[idx];
    if (!(entry & PTE_PRESENT)) {
        if (!create) {
            return NULL;
        }
        ensure_table_entry(pd, idx, PTE_PRESENT | PTE_WRITABLE);
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 407) - [public] if
    if (!(entry & PTE_PRESENT)) {
        if (!create) {
            return NULL;
        }
        ensure_table_entry(pd, idx, PTE_PRESENT | PTE_WRITABLE);
        entry = pd[idx];
    }
    if (entry & PTE_PAGE_SIZE) {
        return NULL; /* 2M large page - not handled here */
    }
    pt = phys_to_ptr(entry & PAGING_ADDR_MASK);

    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 408) - [public] if
    if (!create) {
        return NULL;
    }
    ensure_table_entry(pd, idx, PTE_PRESENT | PTE_WRITABLE);
    entry = pd[idx];
}
if (entry & PTE_PAGE_SIZE) {
    return NULL; /* 2M large page - not handled here */
}
pt = phys_to_ptr(entry & PAGING_ADDR_MASK);

return &pt[pt_index(virt)];
// ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 411) - [public] ensure_table_entry
    ensure_table_entry(pd, idx, PTE_PRESENT | PTE_WRITABLE);
    entry = pd[idx];
}
if (entry & PTE_PAGE_SIZE) {
    return NULL; /* 2M large page - not handled here */
}
pt = phys_to_ptr(entry & PAGING_ADDR_MASK);

return &pt[pt_index(virt)];
}

/* ----- */
/* ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 414) - [public] if
    if (entry & PTE_PAGE_SIZE) {
        return NULL; /* 2M large page - not handled here */
    }
    pt = phys_to_ptr(entry & PAGING_ADDR_MASK);

    return &pt[pt_index(virt)];
}

/* ----- */
/* Arch-specific operations used by the generic paging layer */
/* ----- */

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 430) - [public] if
    if (!boot_phys_offset_valid) {
        return false;
    }
    if ((virt & PAGING_PAGE_MASK) != 0 || (phys & PAGING_PAGE_MASK) != 0) {
        return false;
    }
    flags &= ~PTE_PAGE_SIZE;

    pte = resolve_pte(pml4, virt, true);
    if (pte == NULL) {
        return false;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 433) - [public] if
    if ((virt & PAGING_PAGE_MASK) != 0 || (phys & PAGING_PAGE_MASK) != 0) {
        return false;
    }
    flags &= ~PTE_PAGE_SIZE;

    pte = resolve_pte(pml4, virt, true);
    if (pte == NULL) {
        return false;
    }
    *pte = (phys & PAGING_ADDR_MASK) | flags | PTE_PRESENT;

    paging_flush_page(virt);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 439) - [public] if
    if (pte == NULL) {
        return false;
    }
    *pte = (phys & PAGING_ADDR_MASK) | flags | PTE_PRESENT;

    paging_flush_page(virt);
    return true;
}

bool paging_unmap_page_in(uint64_t pml4, uint64_t virt)
{
    uint64_t *pte = resolve_pte(pml4, virt, false);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 444) - [public] paging_flush_page
    *pte = (phys & PAGING_ADDR_MASK) | flags | PTE_PRESENT;

    paging_flush_page(virt);
    return true;
}

bool paging_unmap_page_in(uint64_t pml4, uint64_t virt)
{
    uint64_t *pte = resolve_pte(pml4, virt, false);

    if (pte == NULL || !(*pte & PTE_PRESENT)) {
        return false;
    }
    *pte = 0;
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 452) - [public] if
    if (pte == NULL || !(*pte & PTE_PRESENT)) {
        return false;
    }
    *pte = 0;
    paging_flush_page(virt);
    return true;
}

uint64_t paging_translate_in(uint64_t pml4, uint64_t virt)
{
    uint64_t *pte = resolve_pte(pml4, virt, false);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 456) - [public] paging_flush_page
    *pte = 0;

    paging_flush_page(virt);
    return true;
}

uint64_t paging_translate_in(uint64_t pml4, uint64_t virt)
{
    uint64_t *pte = resolve_pte(pml4, virt, false);

    if (pte == NULL || !(*pte & PTE_PRESENT)) {
        return 0;
    }
    return (*pte & PAGING_ADDR_MASK) | (virt & PAGING_PAGE_MASK);
    // ... (truncated, see source file for more)

```



```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 464) - [public] if
    if (pte == NULL || !(*pte & PTE_PRESENT)) {
        return 0;
    }
    return (*pte & PAGING_ADDR_MASK) | (virt & PAGING_PAGE_MASK);
}

bool paging_is_mapped_in(uint64_t pml4, uint64_t virt)
{
    const uint64_t *pte = resolve_pte(pml4, virt, false);

    return pte != NULL && (*pte & PTE_PRESENT) != 0;
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 467) - [public] return
    return (*pte & PAGING_ADDR_MASK) | (virt & PAGING_PAGE_MASK);
}

bool paging_is_mapped_in(uint64_t pml4, uint64_t virt)
{
    const uint64_t *pte = resolve_pte(pml4, virt, false);

    return pte != NULL && (*pte & PTE_PRESENT) != 0;
}

bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags)
{
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 481) - [public] if
    if (pte == NULL || !(*pte & PTE_PRESENT)) {
        return false;
    }
    flags &= ~PTE_PAGE_SIZE;
    *pte = (*pte & PAGING_ADDR_MASK) | flags | PTE_PRESENT;
    paging_flush_page(virt);
    return true;
}

bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags)
{
    const uint64_t *pte = resolve_pte(pml4, virt, false);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 486) - [public] paging_flush_page
    *pte = (*pte & PAGING_ADDR_MASK) | flags | PTE_PRESENT;
    paging_flush_page(virt);
    return true;
}

bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags)
{
    const uint64_t *pte = resolve_pte(pml4, virt, false);

    if (pte == NULL || !(*pte & PTE_PRESENT)) {
        return false;
    }
    if (out_flags != NULL) {
        // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 494) - [public] if

```
    if (pte == NULL || !(*pte & PTE_PRESENT)) {
        return false;
    }
    if (out_flags != NULL) {
        *out_flags = *pte & ~PAGING_ADDR_MASK;
    }
    return true;
}

bool paging_is_mapped(uint64_t virt)
{
    return paging_is_mapped_in(paging_read_cr3(), virt);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 497) - [public] if

```
    if (out_flags != NULL) {
        *out_flags = *pte & ~PAGING_ADDR_MASK;
    }
    return true;
}

bool paging_is_mapped(uint64_t virt)
{
    return paging_is_mapped_in(paging_read_cr3(), virt);
}

uint64_t paging_translate(uint64_t virt)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 505) - [public] paging_is_mapped_in

```
    return paging_is_mapped_in(paging_read_cr3(), virt);
}

uint64_t paging_translate(uint64_t virt)
{
    return paging_translate_in(paging_read_cr3(), virt);
}

bool paging_unmap_page(uint64_t virt)
{
    return paging_unmap_page_in(paging_read_cr3(), virt);
}
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 510) - [public] paging_translate_in

```
    return paging_translate_in(paging_read_cr3(), virt);
}

bool paging_unmap_page(uint64_t virt)
{
    return paging_unmap_page_in(paging_read_cr3(), virt);
}

bool paging_set_flags(uint64_t virt, uint64_t flags)
{
    return paging_set_flags_in(paging_read_cr3(), virt, flags);
}
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 515) - [public] paging_unmap_page_in
    return paging_unmap_page_in(paging_read_cr3(), virt);
}

bool paging_set_flags(uint64_t virt, uint64_t flags)
{
    return paging_set_flags_in(paging_read_cr3(), virt, flags);
}

bool paging_get_flags(uint64_t virt, uint64_t *out_flags)
{
    return paging_get_flags_in(paging_read_cr3(), virt, out_flags);
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 520) - [public] paging_set_flags_in
    return paging_set_flags_in(paging_read_cr3(), virt, flags);
}

bool paging_get_flags(uint64_t virt, uint64_t *out_flags)
{
    return paging_get_flags_in(paging_read_cr3(), virt, out_flags);
}

/* NX is enabled once via the EFER MSR; this is idempotent. */
static bool nx_enabled = false;

void paging_enable_nx(void)
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 525) - [public] paging_get_flags_in
    return paging_get_flags_in(paging_read_cr3(), virt, out_flags);
}

/* NX is enabled once via the EFER MSR; this is idempotent. */
static bool nx_enabled = false;

void paging_enable_nx(void)
{
    uint64_t efer;

    if (nx_enabled) {
        return;
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 535) - [public] if
    if (nx_enabled) {
        return;
    }
    __asm__ volatile("rdmsr" : "=A"(efer) : "c"(0xC0000080));
    efer |= (1ULL << 11); /* NXE bit */
    __asm__ volatile("wrmsr" : : "A"(efer), "c"(0xC0000080));
    nx_enabled = true;
}

bool paging_nx_enabled(void)
{
    return nx_enabled;
}
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 538) - [public] volatile

```
    __asm__ volatile("rdmsr" : "=A"(efer) : "c"(0xC0000080));
    efer |= (1ULL << 11); /* NXE bit */
    __asm__ volatile("wrmsr" : : "A"(efer), "c"(0xC0000080));
    nx_enabled = true;
}

bool paging_nx_enabled(void)
{
    return nx_enabled;
}

void paging_write_cr3(uint64_t pml4_phys)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 540) - [public] volatile

```
if (nx_enabled){
    return;
}

__asm__ volatile("rdmsr" : "=A"(efer) : "c"(0xC0000080));
efer |= (1ULL << 11); /* NXE bit */

__asm__ volatile("wrmsr" : : "A"(efer), "c"(0xC0000080));
nx_enabled = true;
}

bool paging_nx_enabled(void)
{
    return nx_enabled;
}

void paging_write_cr3(uint64_t pml4_phys)
{
    __asm__ volatile("mov %0, %%cr3" : : "r"(pml4_phys) : "memory");
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.c (line 551) - [public] volatile

```
__asm__ volatile("mov %0, %%cr3" : : "r"(pml4_phys) : "memory");
}
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 77) - [public] tlb_asm_invpcid

```
■ tlb_asm_invpcid(INVPCID_ALL, &desc);
■ return;
■}

■tlb_asm_write_cr3(tlb_asm_read_cr3());
}

void tlb_flush_all_including_global(void)
{
    ■if (has_invpcid) {
    ■ tlb_flush_all();
    ■ return;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 87) - [public] tlb_flush_all

```
■ tlb_flush_all();
■ return;
■}

■uint64_t cr4 = tlb_asm_read_cr4();

■if (cr4 & CR4_PGE) {
    ■ tlb_asm_write_cr4(cr4 & ~CR4_PGE);
    ■ tlb_asm_write_cr4(cr4);
    ■} else {
    ■ tlb_flush_all();
    ■}
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 94) - [public] tlb_asm_write_cr4

```
■    tlb_asm_write_cr4(cr4 & ~CR4_PGE);
■    tlb_asm_write_cr4(cr4);
■} else {
■    tlb_flush_all();
■}
}
```

```
void tlb_flush_page_addr(uint64_t addr)
{
■tlb_asm_invlpg(addr & ~TLB_PAGE_MASK);
}
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 95) - [public] tlb_asm_write_cr4

```
■    tlb_asm_write_cr4(cr4);
■} else {
■    tlb_flush_all();
■}
}
```

```
void tlb_flush_page_addr(uint64_t addr)
{
■tlb_asm_invlpg(addr & ~TLB_PAGE_MASK);
}
```

```
void tlb_flush_page(const void *addr)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 97) - [public] tlb_flush_all

```
■    tlb_flush_all();
■}
}
```

```
void tlb_flush_page_addr(uint64_t addr)
{
■tlb_asm_invlpg(addr & ~TLB_PAGE_MASK);
}
```

```
void tlb_flush_page(const void *addr)
{
■if (addr == NULL) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 109) - [public] tlb_flush_all

```
■    tlb_flush_all();
■    return;
■}
}
```

```
■tlb_flush_page_addr((uint64_t)(uintptr_t)addr);
}
```

```
void tlb_flush_range_addr(uint64_t addr, size_t pages)
{
■if (pages == 0) {
■    return;
■}
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 123) - [public] tlb_flush_all

```
■ tlb_flush_all();
■ return;
■}

■uint64_t a = addr & ~TLB_PAGE_MASK;

■if (pages > (UINT64_MAX - a) / TLB_PAGE_SIZE) {
■ tlb_flush_all();
■ return;
■}

■for (size_t i = 0; i < pages; i++) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 130) - [public] tlb_flush_all

```
■ tlb_flush_all();
■ return;
■}

■for (size_t i = 0; i < pages; i++) {
■ tlb_asm_invlpg(a);
■ a += TLB_PAGE_SIZE;
■}
}

void tlb_flush_range(const void *addr, size_t pages)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 135) - [public] tlb_asm_invlpg

```
■ tlb_asm_invlpg(a);
■ a += TLB_PAGE_SIZE;
■}
}

void tlb_flush_range(const void *addr, size_t pages)
{
■if (addr == NULL) {
■ tlb_flush_all();
■ return;
■}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 143) - [public] tlb_flush_all

```
■ tlb_flush_all();
■ return;
■}

■tlb_flush_range_addr((uint64_t)(uintptr_t)addr, pages);
}

void tlb_flush_pcid(uint16_t pcid)
{
■if (has_invpcid) {
■ invpcid_desc_t desc = { pcid, 0 };
■ tlb_asm_invpcid(INVPCID_SINGLE_PCID, &desc);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 154) - [public] tlb_asm_invpcid

```
■ tlb_asm_invpcid(INVPCID_SINGLE_PCID, &desc);  
■ return;  
■ }
```

```
■ if (pcid == TLB_PCID_KERNEL) {  
■ tlb_asm_write_cr3(tlb_asm_read_cr3());  
■ }  
}
```

```
void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr)  
{  
■ if (has_invpcid) {  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 159) - [public] tlb_asm_write_cr3

```
■ tlb_asm_write_cr3(tlb_asm_read_cr3());  
■ }  
}
```

```
void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr)  
{  
■ if (has_invpcid) {  
■ invpcid_desc_t desc = { pcid, addr & ~TLB_PAGE_MASK };  
■ tlb_asm_invpcid(INVPCID_ADDR_IN_PCID, &desc);  
■ return;  
■ }
```

```
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.c (line 167) - [public] tlb_asm_invpcid

```
■ tlb_asm_invpcid(INVPCID_ADDR_IN_PCID, &desc);  
■ return;  
■ }
```

```
■ tlb_asm_invlpg(addr & ~TLB_PAGE_MASK);  
}
```

```
void tlb_wbinvd(void)  
{  
■ tlb_asm_wbinvd();  
}
```

```
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/cache/cache.c (line 3) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);
```

```
static kcache_t *cache_registry = NULL;  
static bool cache_initialized = false;
```

```
bool cache_init(void)  
{  
    if (cache_initialized) {  
        return true;  
    }
```

```
    per_cpu_init();  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/cache/cache.c (line 10) - [public] if

```
    if (cache_initialized) {
        return true;
    }

    per_cpu_init();

    cache_registry = NULL;
    cache_initialized = true;

    return true;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/cache/cache.c (line 14) - [public] per_cpu_init

```
per_cpu_init();

cache_registry = NULL;
cache_initialized = true;

return true;
}

bool cache_ready(void)
{
    return cache_initialized;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/cache/cache.c (line 29) - [public] if

```
    if (c == NULL) {
        return;
    }

    c->next = cache_registry;
    cache_registry = c;
}

kcache_t *cache_find(const char *name)
{
    kcache_t *c = cache_registry;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/cache/cache.c (line 41) - [public] while

```
while (c != NULL) {
    const char *a = c->name;
    const char *b = name;

    while (*a != '\0' && *a == *b) {
        a++;
        b++;
    }

    if (*a == '\0' && *b == '\0') {
        return c;
    }
}

// ... (truncated, see source file for more)
```



```
kernel_Workspace/kernel/src/mm/cache/cache.c (line 45) - [public] while
    while (*a != '\0' && *a == *b) {
        a++;
        b++;
    }

    if (*a == '\0' && *b == '\0') {
        return c;
    }

    c = c->next;
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/cache/cache.c (line 50) - [public] if
    if (*a == '\0' && *b == '\0') {
        return c;
    }

    c = c->next;
}

return NULL;
}

void cache_dump(void)
{
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/cache/cache.c (line 62) - [public] if
    if (!cache_initialized) {
        kprintf("CACHE: not initialized\n");
        return;
    }

    kprintf("CACHE:\n");

    kcache_t *c = cache_registry;

    while (c != NULL) {
        kprintf("  %-16s obj=%u free=%u total=%u\n",
            c->name,
            // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/cache/cache.c (line 63) - [public] kprintf
    kprintf("CACHE: not initialized\n");
    return;
}

kprintf("CACHE:\n");

kcache_t *c = cache_registry;

while (c != NULL) {
    kprintf("  %-16s obj=%u free=%u total=%u\n",
        c->name,
        (unsigned int)c->object_size,
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/cache/cache.c (line 67) - [public] kprintf

```
kprintf("CACHE:\n");

kcache_t *c = cache_registry;

while (c != NULL) {
    kprintf(" %16s obj=%u free=%u total=%u\n",
        c->name,
        (unsigned int)c->object_size,
        (unsigned int)c->free_count,
        (unsigned int)c->total_objects);

    c = c->next;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/cache/cache.c (line 71) - [public] while

```
while (c != NULL) {
    kprintf(" %16s obj=%u free=%u total=%u\n",
        c->name,
        (unsigned int)c->object_size,
        (unsigned int)c->free_count,
        (unsigned int)c->total_objects);

    c = c->next;
}
}
```

kernel_Workspace/kernel/src/mm/cache/object_cache.c (line 82) - [public] buddy_free

```
■ buddy_free(va);
■ return false;
■}

■ch->va = (uint64_t)(uintptr_t)va;
■ch->next = c->chunks;
■c->chunks = ch;

■size_t n = c->chunk_size / c->object_size;

■for (size_t i = 0; i < n; i++) {
■    uint8_t *obj =
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/cache/object_cache.c (line 96) - [public] if

```
■ if (c->ctor != NULL) {
■     c->ctor(obj);
■ }

■ *(void **)obj = c->free_head;
■ c->free_head = obj;
■}

■c->free_count += n;
■c->total_objects += n;

■return true;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/cache/object_cache.c (line 180) - [public] buddy_free

```
■ buddy_free((void *)(uintptr_t)ch->va);
■ heap_free(ch);

■ ch = next;
■}

■c->chunks = NULL;
■c->free_head = NULL;
■c->free_count = 0;
■c->total_objects = 0;

■cache_unlock();
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/cache/object_cache.c (line 181) - [public] heap_free

```
■ heap_free(ch);  
■ ch = next;  
■}  
  
■c->chunks = NULL;  
■c->free_head = NULL;  
■c->free_count = 0;  
■c->total_objects = 0;  
  
■cache_unlock();  
  
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/cache/object_cache.c (line 208) - [public] cache_unlock

```
■ cache_unlock();  
■ return obj;  
■}  
  
■if (c->free_head == NULL) {  
■ if (!kcache_grow(c)) {  
■ cache_unlock();  
■ return NULL;  
■ }  
■}
```

```
■void *obj = c->free_head;  
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/cache/object_cache.c (line 213) - [public] if

```
■ if (!kcache_grow(c)) {  
■ cache_unlock();  
■ return NULL;  
■ }  
■}
```

```
■void *obj = c->free_head;  
■c->free_head = *(void **)obj;  
■c->free_count--;
```

```
■cache_unlock();  
  
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/cache/object_cache.c (line 214) - [public] cache_unlock

```
■ cache_unlock();  
■ return NULL;  
■ }  
■}
```

```
■void *obj = c->free_head;  
■c->free_head = *(void **)obj;  
■c->free_count--;
```

```
■cache_unlock();  
  
■return obj;  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/cache/object_cache.c (line 253) - [public] cache_unlock

```
■ cache_unlock();
■ return;
■}

■*(void **)ptr = c->free_head;
■c->free_head = ptr;
■c->free_count++;

■cache_unlock();
}
```

```
size_t kcache_object_size(kcache_t *c)
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 15) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);
```

```
static bool mm_initialized = false;

bool mm_init(const mm_boot_params_t *params)
{
■if (mm_initialized) {
■    return true;
■}

■if (params == NULL ||
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 29) - [public] kprintf

```
■    kprintf("mm_init: bad params\n");
■    return false;
■}

■kprintf("mm_init: arch_memory_init...\n");
■arch_memory_init(params->memmap,
■                params->memmap_count,
■                params->kernel_phys_start,
■                params->kernel_phys_end,
■                params->initrd_phys_start,
■                params->initrd_phys_end);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 46) - [public] kprintf

```
■    kprintf("mm_init: pmm_init FAILED\n");
■    return false;
■}

■kprintf("mm_init: vmm_init...\n");
■if (!vmm_init()) {
■    kprintf("mm_init: vmm_init FAILED\n");
■    return false;
■}

■kprintf("mm_init: page_init...\n");
■if (!page_init()) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 52) - [public] kprintf

```
■ kprintf("mm_init: vmm_init FAILED\n");  
■ return false;  
■ }
```

```
■ kprintf("mm_init: page_init...\n");  
■ if (!page_init()) {  
■ kprintf("mm_init: page_init FAILED\n");  
■ return false;  
■ }
```

```
■ mapping_init();
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 58) - [public] kprintf

```
■ kprintf("mm_init: page_init FAILED\n");  
■ return false;  
■ }
```

```
■ mapping_init();
```

```
■ kprintf("mm_init: heap_init...\n");  
■ if (!heap_init()) {  
■ kprintf("mm_init: heap_init FAILED\n");  
■ return false;  
■ }
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 66) - [public] kprintf

```
■ kprintf("mm_init: heap_init FAILED\n");  
■ return false;  
■ }
```

```
■ kprintf("mm_init: cache_init...\n");  
■ if (!cache_init()) {  
■ kprintf("mm_init: cache_init FAILED\n");  
■ return false;  
■ }
```

```
■ kprintf("mm_init: paging_subsystem_init...\n");  
■ if (!paging_subsystem_init()) {  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 72) - [public] kprintf

```
■ kprintf("mm_init: cache_init FAILED\n");  
■ return false;  
■ }
```

```
■ kprintf("mm_init: paging_subsystem_init...\n");  
■ if (!paging_subsystem_init()) {  
■ kprintf("mm_init: paging_subsystem_init FAILED\n");  
■ return false;  
■ }
```

```
■ isolation_init();
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 78) - [public] kprintf

```
■ kprintf("mm_init: paging_subsystem_init FAILED\n");
■ return false;
■}

■isolation_init();

■kprintf("mm_init: aspace_subsystem_init...\n");
■if (!aspace_subsystem_init()) {
■ kprintf("mm_init: aspace_subsystem_init FAILED\n");
■ return false;
■}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 86) - [public] kprintf

```
■ kprintf("mm_init: aspace_subsystem_init FAILED\n");
■ return false;
■}

■mm_initialized = true;

■kprintf("mm_init: OK\n");

■return true;
}

bool mm_ready(void)
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 104) - [public] pmm_stat_total_bytes

```
■return pmm_stat_total_bytes();
}

uint64_t mm_free_ram(void)
{
■return pmm_stat_free_bytes();
}

void mm_dump(void)
{
■kprintf("MM: ready=%d total=%llu MiB free=%llu MiB\n",
■ (int)mm_initialized,
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/mm.c (line 109) - [public] pmm_stat_free_bytes

```
■return pmm_stat_free_bytes();
}

void mm_dump(void)
{
■kprintf("MM: ready=%d total=%llu MiB free=%llu MiB\n",
■ (int)mm_initialized,
■ (unsigned long long)(mm_total_ram() >> 20),
■ (unsigned long long)(mm_free_ram() >> 20));

■kalloc_dump();
}
```

kernel_Workspace/kernel/src/mm/core/range.c (line 53) - [public] if

```
■    if (raw_end > UINT64_MAX - pad) {
■        return false;
■    }

■    end = raw_end + pad;
■}

■if (end <= start) {
■    return false;
■}

■if (require_canonical) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/range.c (line 65) - [public] if

```
■    if (!range_is_canonical(start) || !range_is_canonical(end - 1)) {
■        return false;
■    }
■}

■if (start < limit_min) {
■    return false;
■}

■if (end > limit_max) {
■    return false;
■}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/region.c (line 64) - [public] if

```
■    if (region_valid(before)) {
■        out[n++] = before;
■    }
■}

■if (cut.end < r.end && n < max_out) {
■    region_t after;

■    after.start = cut.end > r.start ? cut.end : r.start;
■    after.end = r.end;

■    if (region_valid(after)) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/region.c (line 75) - [public] if

```
■    if (region_valid(after)) {
■        out[n++] = after;
■    }
■}

■return n;
}
```

kernel_Workspace/kernel/src/mm/core/smp_lock.c (line 5) - [public] volatile

```
■__asm__ volatile("pause" ::: "memory");
}

static inline uint64_t irq_save_disable(void)
{
■uint64_t flags;

■__asm__ volatile(
■    "pushfq\n"
■    "popq %0\n"
■    "cli"
■    : "=r"(flags)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/smp_lock.c (line 64) - [public] __atomic_load_n

```
■ __atomic_load_n(&lock->owner_cpu, __ATOMIC_ACQUIRE) == me) {
■     lock->depth++;
■     return;
■ }

■ uint32_t ticket = __atomic_fetch_add(&lock->next_ticket, 1,
■                                     __ATOMIC_RELAXED);

■ while (__atomic_load_n(&lock->now_serving, __ATOMIC_ACQUIRE) != ticket) {
■     cpu_pause();
■ }
■ lock->owner_cpu = me;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/smp_lock.c (line 73) - [public] cpu_pause

```
■     cpu_pause();
■ }
■ lock->owner_cpu = me;
■ lock->depth = 1;
■ lock->saved_flags = flags;
}

bool smp_lock_release(smp_ticket_lock_t *lock)
{
■ if (lock->depth == 0) {
■     return false;
■ }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 18) - [public] volatile

```
■ __asm__ volatile("inb %dx, %0" : "=a"(v) : "d"(port));

■ return v;
}

static inline void outb(uint16_t port, uint8_t val)
{
■ __asm__ volatile("outb %0, %%dx" : : "a"(val), "d"(port));
}

static bool uart_ready = false;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 25) - [public] volatile

```
■ __asm__ volatile("outb %0, %%dx" : : "a"(val), "d"(port));
}

static bool uart_ready = false;

static void serial_init(void)
{
■ if (uart_ready) {
■     return;
■ }

■ outb(COM1 + 1, 0x00);
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/mm/kprintf.c (line 58) - [public] serial_putc

```
■    serial_putc(*s++);
■}
}

static void serial_put_uint(uint64_t v, unsigned base, bool upper)
{
■char buf[32];
■int i = 32;

■if (v == 0) {
■    serial_putc('0');
■    return;
■    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/kprintf.c (line 68) - [public] serial_putc

```
■    serial_putc('0');
■    return;
■}

■while (v > 0) {
■    unsigned d = (unsigned)(v % base);

■    buf[--i] = (char)(d < 10 ? '0' + d : (upper ? 'A' : 'a') + d - 10);
■    v /= base;
■}

■while (i < 32) {
■    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/kprintf.c (line 80) - [public] serial_putc

```
■    serial_putc(buf[i++]);
■}
}

void kprintf(const char *fmt, ...)
{
■serial_init();

■va_list ap;

■va_start(ap, fmt);

■    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/kprintf.c (line 93) - [public] if

```
■    if (*p != '%') {
■        serial_putc(*p);
■        continue;
■    }

■    p++;
■    if (*p == '\\0') {
■        break;
■    }

■    bool ll = false;
■    bool l = false;
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 94) - [public] serial_putc

```
■ serial_putc(*p);
■ continue;
■ }

■ p++;
■ if (*p == '\0') {
■     break;
■ }

■ bool ll = false;
■ bool l = false;
■ bool z = false;
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 99) - [public] if

```
■ if (*p == '\0') {
■     break;
■ }

■ bool ll = false;
■ bool l = false;
■ bool z = false;

■ while (*p == 'l' || *p == 'z' || *p == 'h') {
■     if (*p == 'l') {
■         if (p[1] == 'l') {
■             ll = true;
■         }
■         // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 107) - [public] while

```
■ while (*p == 'l' || *p == 'z' || *p == 'h') {
■     if (*p == 'l') {
■         if (p[1] == 'l') {
■             ll = true;
■             p++;
■         } else {
■             l = true;
■         }
■     } else if (*p == 'z') {
■         z = true;
■     }
■     p++;
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 108) - [public] if

```
■ if (*p == 'l') {
■     if (p[1] == 'l') {
■         ll = true;
■         p++;
■     } else {
■         l = true;
■     }
■ } else if (*p == 'z') {
■     z = true;
■ }
■ p++;
■ }
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 109) - [public] if

```
■         if (p[1] == 'l') {
■             ll = true;
■             p++;
■         } else {
■             l = true;
■         }
■     } else if (*p == 'z') {
■         z = true;
■     }
■     p++;
■ }
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/kprintf.c (line 121) - [public] switch

```
■     switch (*p) {
■     case 's': {
■         const char *s = va_arg(ap, const char *);

■         serial_puts(s ? s : "(null)");
■         break;
■     }
■     case 'c':
■         serial_putc((char)va_arg(ap, int));
■         break;
■     case 'd':
■     case 'i': {
■         // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 125) - [public] serial_puts

```
■         serial_puts(s ? s : "(null)");
■         break;
■     }
■     case 'c':
■         serial_putc((char)va_arg(ap, int));
■         break;
■     case 'd':
■     case 'i': {
■         int64_t v = ll ? (int64_t)va_arg(ap, long long)
■             : (l ? (int64_t)va_arg(ap, long)
■                 : (int64_t)va_arg(ap, int));
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/kprintf.c (line 129) - [public] serial_putc

```
■         serial_putc((char)va_arg(ap, int));
■         break;
■     case 'd':
■     case 'i': {
■         int64_t v = ll ? (int64_t)va_arg(ap, long long)
■             : (l ? (int64_t)va_arg(ap, long)
■                 : (int64_t)va_arg(ap, int));

■         if (v < 0) {
■             serial_putc('-');
■             serial_put_uint((uint64_t)(-v), 10, false);
■         } else {
■             // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 137) - [public] if

```
■   if (v < 0) {
■       serial_putc('-');
■       serial_put_uint((uint64_t)(-v), 10, false);
■   } else {
■       serial_put_uint((uint64_t)v, 10, false);
■   }
■   break;
■ }
■ case 'u': {
■     uint64_t v = ll ? (uint64_t)va_arg(ap, unsigned long long)
■                   : (z ? (uint64_t)va_arg(ap, size_t)
■                     : (l ? (uint64_t)va_arg(ap, unsigned long)
■                       : (uint64_t)va_arg(ap, unsigned int)));
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 138) - [public] serial_putc

```
■       serial_putc('-');
■       serial_put_uint((uint64_t)(-v), 10, false);
■   } else {
■       serial_put_uint((uint64_t)v, 10, false);
■   }
■   break;
■ }
■ case 'u': {
■     uint64_t v = ll ? (uint64_t)va_arg(ap, unsigned long long)
■                   : (z ? (uint64_t)va_arg(ap, size_t)
■                     : (l ? (uint64_t)va_arg(ap, unsigned long)
■                       : (uint64_t)va_arg(ap, unsigned int)));
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 139) - [public] serial_put_uint

```
■       serial_put_uint((uint64_t)(-v), 10, false);
■   } else {
■       serial_put_uint((uint64_t)v, 10, false);
■   }
■   break;
■ }
■ case 'u': {
■     uint64_t v = ll ? (uint64_t)va_arg(ap, unsigned long long)
■                   : (z ? (uint64_t)va_arg(ap, size_t)
■                     : (l ? (uint64_t)va_arg(ap, unsigned long)
■                       : (uint64_t)va_arg(ap, unsigned int)));
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 141) - [public] serial_put_uint

```
■       serial_put_uint((uint64_t)v, 10, false);
■   }
■   break;
■ }
■ case 'u': {
■     uint64_t v = ll ? (uint64_t)va_arg(ap, unsigned long long)
■                   : (z ? (uint64_t)va_arg(ap, size_t)
■                     : (l ? (uint64_t)va_arg(ap, unsigned long)
■                       : (uint64_t)va_arg(ap, unsigned int)));
■     serial_put_uint(v, 10, false);
■     break;
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 151) - [public] serial_put_uint

```
■      serial_put_uint(v, 10, false);
■      break;
■  }
■  case 'x':
■  case 'X': {
■      uint64_t v = ll ? (uint64_t)va_arg(ap, unsigned long long)
■                      : (z ? (uint64_t)va_arg(ap, size_t)
■                      : (l ? (uint64_t)va_arg(ap, unsigned long)
■                      : (uint64_t)va_arg(ap, unsigned int)));
■
■      serial_put_uint(v, 16, *p == 'X');
■      break;
■  }
■  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 161) - [public] serial_put_uint

```
■      serial_put_uint(v, 16, *p == 'X');
■      break;
■  }
■  case 'p': {
■      void *ptr = va_arg(ap, void *);
■
■      serial_puts("0x");
■      serial_put_uint((uint64_t)(uintptr_t)ptr, 16, false);
■      break;
■  }
■  case '%':
■      serial_putc('%');
■  }
■  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 167) - [public] serial_puts

```
■      serial_puts("0x");
■      serial_put_uint((uint64_t)(uintptr_t)ptr, 16, false);
■      break;
■  }
■  case '%':
■      serial_putc('%');
■      break;
■  default:
■      serial_putc('%');
■      serial_putc(*p);
■      break;
■  }
■  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 168) - [public] serial_put_uint

```
■      serial_put_uint((uint64_t)(uintptr_t)ptr, 16, false);
■      break;
■  }
■  case '%':
■      serial_putc('%');
■      break;
■  default:
■      serial_putc('%');
■      serial_putc(*p);
■      break;
■  }
■  }
■  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 172) - [public] serial_putc

```
■    serial_putc('%');
■    break;
■    default:
■        serial_putc('%');
■        serial_putc(*p);
■        break;
■    }
■}

■va_end(ap);
}
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/kprintf.c (line 175) - [public] serial_putc

```
■    serial_putc('%');
■    serial_putc(*p);
■    break;
■    }
■}

■va_end(ap);
}
```

```
void serial_write_str(const char *s)
{
■serial_init();
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/kprintf.c (line 176) - [public] serial_putc

```
■    serial_putc(*p);
■    break;
■    }
■}

■va_end(ap);
}
```

```
void serial_write_str(const char *s)
{
■serial_init();
■serial_puts(s);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 68) - [public] if

```
■    if (pml_entry_present(e) && !pml_entry_large(e)) {
■        free_tables(pml_entry_addr(e), level - 1);
■    }
■}

■pmm_free_frame(table_phys);
}
```

```
address_space_t *paging_aspace_create(void)
{
■if (!paging_subsystem_ready) {
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 69) - [public] free_tables

```
■         free_tables(pml_entry_addr(e), level - 1);
■     }
■ }
■}

pmm_free_frame(table_phys);
}

address_space_t *paging_aspace_create(void)
{
■if (!paging_subsystem_ready) {
■    return NULL;
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 99) - [public] pmm_free_frame

```
■    pmm_free_frame(pml4_phys);
■    return NULL;
■}
```

```
■as->pml4_phys = pml4_phys;
■as->kernel = false;
```

```
■return as;
}
```

```
void paging_aspace_destroy(address_space_t *as)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 120) - [public] if

```
■    if (pml_entry_present(e) && !pml_entry_large(e)) {
■        free_tables(pml_entry_addr(e), PML_LEVEL_PDPT);
■    }
■}
```

```
■pmm_free_frame(as->pml4_phys);
```

```
■heap_free(as);
}
```

```
void paging_aspace_switch(address_space_t *as)
{
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 121) - [public] free_tables

```
■        free_tables(pml_entry_addr(e), PML_LEVEL_PDPT);
■    }
■}
```

```
■pmm_free_frame(as->pml4_phys);
```

```
■heap_free(as);
}
```

```
void paging_aspace_switch(address_space_t *as)
{
■if (as == NULL) {
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 173) - [public] if

```
■ if (!paging_map_page_in(pml4, v, p, pte)) {
■     for (size_t j = 0; j < mapped; j++) {
■         paging_unmap_page_in(pml4, virt + (uint64_t)j * ARCH_PAGE_SIZE);
■     }

■     return false;
■ }

■ mapped++;
■}

■return true;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 175) - [public] paging_unmap_page_in

```
■         paging_unmap_page_in(pml4, virt + (uint64_t)j * ARCH_PAGE_SIZE);
■     }

■     return false;
■ }

■ mapped++;
■}

■return true;
}

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 207) - [public] if

```
■ if (paging_unmap_page_in(pml4, v)) {
■     tlb_batch_add(&batch, v);
■ }
■}

■tlb_batch_commit(&batch);

■return true;
}

bool paging_aspace_protect(address_space_t *as,
■     uint64_t virt,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 208) - [public] tlb_batch_add

```
■     tlb_batch_add(&batch, v);
■ }
■}

■tlb_batch_commit(&batch);

■return true;
}

bool paging_aspace_protect(address_space_t *as,
■     uint64_t virt,
■     size_t len,
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/mm/paging/paging.c (line 238) - [public] if

```
■ if (!paging_is_mapped_in(pml4, v)) {
■     return false;
■ }
■}

■tlb_batch_t batch;
■tlb_batch_begin(&batch);

■for (size_t i = 0; i < pages; i++) {
■     uint64_t v = virt + (uint64_t)i * ARCH_PAGE_SIZE;

■     if (!paging_set_flags_in(pml4, v, pte)) {
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/paging/paging.c (line 249) - [public] if

```
■ if (!paging_set_flags_in(pml4, v, pte)) {
■     tlb_batch_commit(&batch);
■     return false;
■ }

■ tlb_batch_add(&batch, v);
■}

■tlb_batch_commit(&batch);

■return true;
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/paging/paging.c (line 250) - [public] tlb_batch_commit

```
■ tlb_batch_commit(&batch);
■ return false;
■ }

■ tlb_batch_add(&batch, v);
■}

■tlb_batch_commit(&batch);

■return true;
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/paging/paging.c (line 254) - [public] tlb_batch_add

```
■ tlb_batch_add(&batch, v);
■}

■tlb_batch_commit(&batch);

■return true;
}

uint64_t paging_aspace_translate(address_space_t *as, uint64_t virt)
{
■if (as == NULL) {
■     return paging_translate_in(paging_kernel_pml4, virt);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 265) - [public] paging_translate_in

```
    return paging_translate_in(paging_kernel_pml4, virt);
}

return paging_translate_in(as->pml4_phys, virt);
}

bool paging_kernel_map(uint64_t virt, uint64_t phys, size_t len, uint32_t prot)
{
    if (!paging_subsystem_ready) {
        return false;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 268) - [public] paging_translate_in

```
return paging_translate_in(as->pml4_phys, virt);
}

bool paging_kernel_map(uint64_t virt, uint64_t phys, size_t len, uint32_t prot)
{
    if (!paging_subsystem_ready) {
        return false;
    }

    return paging_aspace_map((address_space_t *)&(address_space_t){
        .pml4_phys = paging_kernel_pml4,
        .kernel = true,
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 277) - [public] paging_aspace_map

```
return paging_aspace_map((address_space_t *)&(address_space_t){
    .pml4_phys = paging_kernel_pml4,
    .kernel = true,
    },
    virt, phys, len, prot);
}

bool paging_kernel_unmap(uint64_t virt, size_t len)
{
    if (!paging_subsystem_ready) {
        return false;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/paging/paging.c (line 290) - [public] paging_aspace_unmap

```
return paging_aspace_unmap((address_space_t *)&(address_space_t){
    .pml4_phys = paging_kernel_pml4,
    .kernel = true,
    },
    virt, len);
}
```

kernel_Workspace/kernel/src/mm/paging/tlb.c (line 33) - [public] tlb_flush_all

```
tlb_flush_all();
batch->count = 0;
batch->full_flush = false;
return;
}

for (size_t i = 0; i < batch->count; i++) {
    tlb_flush_page_addr(batch->pages[i]);
}

batch->count = 0;
}
```

kernel_workspace/kernel/src/mm/paging/tlb.c (line 40) - [public] tlb_flush_page_addr

```
■ tlb_flush_page_addr(batch->pages[i]);  
■}  
  
■batch->count = 0;  
}
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 63) - [public] if

```
■ if (!pmm_alloc_zero_frame(&phys)) {  
■     goto rollback;  
■ }  
  
■ if (!paging_aspace_map(pa->as, v, phys, ARCH_PAGE_SIZE,  
■                             prot | PROT_USER)) {  
■     pmm_free_frame(phys);  
■     goto rollback;  
■ }  
  
■ mapped_end = v + ARCH_PAGE_SIZE;  
■}  
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 69) - [public] pmm_free_frame

```
■ pmm_free_frame(phys);  
■     goto rollback;  
■ }  
  
■ mapped_end = v + ARCH_PAGE_SIZE;  
■}  
  
■return true;  
  
rollback:  
■for (uint64_t v = start; v < mapped_end; v += ARCH_PAGE_SIZE) {  
■     uint64_t phys = paging_aspace_translate(pa->as, v);  
■     // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 82) - [public]
paging_aspace_unmap

```
■ paging_aspace_unmap(pa->as, v, ARCH_PAGE_SIZE);  
  
■ if (phys != UINT64_MAX) {  
■     pmm_free_frame(phys);  
■ }  
■}  
  
■return false;  
}  
  
static void unmap_pages_free(proc_aspace_t *pa, uint64_t start, uint64_t end)  
{  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 84) - [public] if

```
■ if (phys != UINT64_MAX) {  
■     pmm_free_frame(phys);  
■ }  
■}  
  
■return false;  
}  
  
static void unmap_pages_free(proc_aspace_t *pa, uint64_t start, uint64_t end)  
{  
■for (uint64_t v = start; v < end; v += ARCH_PAGE_SIZE) {  
■     uint64_t phys = paging_aspace_translate(pa->as, v);  
■     // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 85) - [public] pmm_free_frame

```
■    pmm_free_frame(phys);
■    }
■}

■return false;
}

static void unmap_pages_free(proc_aspace_t *pa, uint64_t start, uint64_t end)
{
■for (uint64_t v = start; v < end; v += ARCH_PAGE_SIZE) {
■    uint64_t phys = paging_aspace_translate(pa->as, v);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 97) - [public]
paging_aspace_unmap

```
■    paging_aspace_unmap(pa->as, v, ARCH_PAGE_SIZE);

■    if (phys != UINT64_MAX) {
■        pmm_free_frame(phys);
■    }
■}

static void vma_insert(proc_aspace_t *pa, vma_t *v)
{
■vma_t **p = &pa->vmass;

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 99) - [public] if

```
■    if (phys != UINT64_MAX) {
■        pmm_free_frame(phys);
■    }
■}

static void vma_insert(proc_aspace_t *pa, vma_t *v)
{
■vma_t **p = &pa->vmass;

■while (*p != NULL && (*p)->start < v->start) {
■    p = &(*p)->next;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 100) - [public] pmm_free_frame

```
■    pmm_free_frame(phys);
■    }
■}

static void vma_insert(proc_aspace_t *pa, vma_t *v)
{
■vma_t **p = &pa->vmass;

■while (*p != NULL && (*p)->start < v->start) {
■    p = &(*p)->next;
■}

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 122) - [public] if

```
■   if (*p == v) {  
■       *p = v->next;  
■       return;  
■   }  
  
■   p = &(*p)->next;  
■}  
}
```

```
vma_t *aspace_vma_find(proc_aspace_t *pa, uint64_t addr)  
{  
■vma_t *v = pa->vmas;  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 136) - [public] if

```
■   if (addr >= v->start && addr < v->end) {  
■       return v;  
■   }  
  
■   v = v->next;  
■}  
  
■return NULL;  
}
```

```
static uint64_t find_gap(proc_aspace_t *pa, uint64_t len, uint64_t hint)  
{  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 163) - [public] if

```
■   if (v->end <= cand) {  
■       v = v->next;  
■       continue;  
■   }  
  
■   if (cand <= USER_MMAP_MAX - len && cand + len <= v->start) {  
■       return cand;  
■   }  
  
■   cand = arch_page_align_up(v->end);  
  
■   if (cand > USER_MMAP_MAX - len) {  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 168) - [public] if

```
■   if (cand <= USER_MMAP_MAX - len && cand + len <= v->start) {  
■       return cand;  
■   }  
  
■   cand = arch_page_align_up(v->end);  
  
■   if (cand > USER_MMAP_MAX - len) {  
■       return 0;  
■   }  
  
■   v = v->next;  
■}  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 174) - [public] if

```
■    if (cand > USER_MMAP_MAX - len) {  
■        return 0;  
■    }  
  
■    v = v->next;  
■}  
  
■return cand;  
}
```

```
proc_aspace_t *aspace_create(void)  
{  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 199) - [public] heap_free

```
■    heap_free(pa);  
■    return NULL;  
■}  
  
■pa->brk_base = USER_BRK_BASE;  
■pa->brk = USER_BRK_BASE;  
■pa->brk_max = USER_BRK_MAX;  
  
■uint64_t stack_start = USER_STACK_TOP - USER_STACK_SIZE;  
  
■vma_t *sv = (vma_t *)heap_alloc(sizeof(vma_t));  
  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 212) - [public]
paging_aspace_destroy

```
■    paging_aspace_destroy(pa->as);  
■    heap_free(pa);  
■    return NULL;  
■}  
  
■sv->start = stack_start;  
■sv->end = USER_STACK_TOP;  
■sv->prot = PROT_READ | PROT_WRITE;  
■sv->flags = VMA_FLAG_ANON | VMA_FLAG_PRIVATE;  
■sv->next = NULL;  
  
■pa->vmass = sv;  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 213) - [public] heap_free

```
■    heap_free(pa);  
■    return NULL;  
■}  
  
■sv->start = stack_start;  
■sv->end = USER_STACK_TOP;  
■sv->prot = PROT_READ | PROT_WRITE;  
■sv->flags = VMA_FLAG_ANON | VMA_FLAG_PRIVATE;  
■sv->next = NULL;  
  
■pa->vmass = sv;  
  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 226) - [public] heap_free

```
■ heap_free(sv);
■ paging_aspace_destroy(pa->as);
■ heap_free(pa);
■ return NULL;
■}

■return pa;
}

void aspace_destroy(proc_aspace_t *pa)
{
■if (pa == NULL) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 227) - [public]
paging_aspace_destroy

```
■ paging_aspace_destroy(pa->as);
■ heap_free(pa);
■ return NULL;
■}

■return pa;
}

void aspace_destroy(proc_aspace_t *pa)
{
■if (pa == NULL) {
■    return;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 228) - [public] heap_free

```
■ heap_free(pa);
■ return NULL;
■}

■return pa;
}

void aspace_destroy(proc_aspace_t *pa)
{
■if (pa == NULL) {
■    return;
■}
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 246) - [public] unmap_pages_free

```
■ unmap_pages_free(pa, v->start, v->end);

■ pa->vmass = v->next;
■ heap_free(v);
■}

■paging_aspace_destroy(pa->as);

■heap_free(pa);

■as_unlock();
}
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 249) - [public] heap_free

```
    heap_free(v);
}

paging_aspace_destroy(pa->as);

heap_free(pa);

as_unlock();
}

address_space_t *aspace_paging_handle(proc_aspace_t *pa)
{
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 281) - [public] as_unlock

```
    as_unlock();
    return 0;
}

vma_t *v = (vma_t *)heap_alloc(sizeof(vma_t));

if (v == NULL) {
    as_unlock();
    return 0;
}

v->start = at;
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 288) - [public] as_unlock

```
    as_unlock();
    return 0;
}

v->start = at;
v->end = at + bytes;
v->prot = prot;
v->flags = VMA_FLAG_ANON | VMA_FLAG_PRIVATE;

vma_insert(pa, v);

if (!map_anon_pages(pa, at, at + bytes, prot)) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 300) - [public] vma_remove

```
    vma_remove(pa, v);
    heap_free(v);
    as_unlock();
    return 0;
}

as_unlock();

return at;
}

uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot)
// ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/mm/process/address_space.c (line 301) - [public] heap_free

```
■ heap_free(v);
■ as_unlock();
■ return 0;
■}

■as_unlock();

■return at;
}

uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot)
{
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 302) - [public] as_unlock

```
■ as_unlock();
■ return 0;
■}

■as_unlock();

■return at;
}

uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot)
{
    ■if (pa == NULL || len == 0) {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 330) - [public] as_unlock

```
■ as_unlock();
■ return 0;
■}

■v->start = a;
■v->end = b;
■v->prot = prot;
■v->flags = VMA_FLAG_ANON | VMA_FLAG_PRIVATE;

■vma_insert(pa, v);

■if (!map_anon_pages(pa, a, b, prot)) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 342) - [public] vma_remove

```
■ vma_remove(pa, v);
■ heap_free(v);
■ as_unlock();
■ return 0;
■}

■as_unlock();

■return a;
}

bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 343) - [public] heap_free

```
■ heap_free(v);
■ as_unlock();
■ return 0;
■ }
```

```
■ as_unlock();
```

```
■ return a;
}
```

```
bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len)
{
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 344) - [public] as_unlock

```
■ as_unlock();
■ return 0;
■ }
```

```
■ as_unlock();
```

```
■ return a;
}
```

```
bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len)
{
    if (pa == NULL || len == 0) {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 371) - [public] while

```
■ while (v != NULL) {
■     if (v->start < b && v->end > a) {
■         hit = v;
■         break;
■     }

■     v = v->next;
■ }

■ if (hit == NULL) {
■     break;
■ }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 372) - [public] if

```
■ if (v->start < b && v->end > a) {
■     hit = v;
■     break;
■ }

■ v = v->next;
■ }

■ if (hit == NULL) {
■     break;
■ }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 380) - [public] if

```
■   if (hit == NULL) {
■       break;
■   }

■   uint64_t s = hit->start > a ? hit->start : a;
■   uint64_t e = hit->end < b ? hit->end : b;

■   bool needs_split = (hit->start < s && hit->end > e);
■   vma_t *tail = NULL;

■   if (needs_split) {
■       tail = (vma_t *)heap_alloc(sizeof(vma_t));
■       // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 390) - [public] if

```
■   if (needs_split) {
■       tail = (vma_t *)heap_alloc(sizeof(vma_t));

■       if (tail == NULL) {
■           as_unlock();
■           return false;
■       }
■   }

■   unmap_pages_free(pa, s, e);

■   if (needs_split) {
■       // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 393) - [public] if

```
■       if (tail == NULL) {
■           as_unlock();
■           return false;
■       }
■   }

■   unmap_pages_free(pa, s, e);

■   if (needs_split) {
■       tail->start = e;
■       tail->end = hit->end;
■       tail->prot = hit->prot;
■       // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 394) - [public] as_unlock

```
■       as_unlock();
■       return false;
■   }
■   }

■   unmap_pages_free(pa, s, e);

■   if (needs_split) {
■       tail->start = e;
■       tail->end = hit->end;
■       tail->prot = hit->prot;
■       tail->flags = hit->flags;
■       // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 399) - [public] unmap_pages_free

```
■ unmap_pages_free(pa, s, e);

■ if (needs_split) {
■     tail->start = e;
■     tail->end = hit->end;
■     tail->prot = hit->prot;
■     tail->flags = hit->flags;
■     tail->next = hit->next;

■     hit->next = tail;
■     hit->end = s;
■ } else if (hit->start < s) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 401) - [public] if

```
■ if (needs_split) {
■     tail->start = e;
■     tail->end = hit->end;
■     tail->prot = hit->prot;
■     tail->flags = hit->flags;
■     tail->next = hit->next;

■     hit->next = tail;
■     hit->end = s;
■ } else if (hit->start < s) {
■     hit->end = s;
■ } else if (hit->end > e) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 415) - [public] vma_remove

```
■ vma_remove(pa, hit);
■ heap_free(hit);
■ }
■}

■as_unlock();

■return true;
}

bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
■     uint32_t checked_prot,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 416) - [public] heap_free

```
■ heap_free(hit);
■ }
■}

■as_unlock();

■return true;
}

bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
■     uint32_t checked_prot,
■     uint32_t apply_prot,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 445) - [public] as_unlock

```
■ as_unlock();
■ return false;
■}

■if (allowed != NULL && !allowed(v->prot, checked_prot)) {
■ as_unlock();
■ return false;
■}

■if (!paging_aspace_protect(pa->as, a, b - a, apply_prot)) {
■ as_unlock();
■ return false;
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 450) - [public] as_unlock

```
■ as_unlock();
■ return false;
■}

■if (!paging_aspace_protect(pa->as, a, b - a, apply_prot)) {
■ as_unlock();
■ return false;
■}

■v->prot = apply_prot;

■as_unlock();
■ // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 455) - [public] as_unlock

```
■ as_unlock();
■ return false;
■}

■v->prot = apply_prot;

■as_unlock();

■return true;
}

bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot)
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 468) - [public]

aspace_protect_checked

```
■return aspace_protect_checked(pa, addr, len, prot, prot, NULL);
}

uint64_t aspace_stack_base(void)
{
■return USER_STACK_TOP - USER_STACK_SIZE;
}

uint64_t aspace_reserve_at(proc_aspace_t *pa,
■ uint64_t addr,
■ size_t len,
■ uint32_t flags)
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 496) - [public] if

```
■    if (v->start < b && v->end > a) {  
■        as_unlock();  
■        return 0;  
■    }  
  
■    v = v->next;  
■}  
  
■vma_t *nv = (vma_t *)heap_alloc(sizeof(vma_t));  
  
■if (nv == NULL) {  
■    as_unlock();  
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 497) - [public] as_unlock

```
■        as_unlock();  
■        return 0;  
■    }  
  
■    v = v->next;  
■}  
  
■vma_t *nv = (vma_t *)heap_alloc(sizeof(vma_t));  
  
■if (nv == NULL) {  
■    as_unlock();  
■    return 0;  
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 507) - [public] as_unlock

```
■    as_unlock();  
■    return 0;  
■}  
  
■nv->start = a;  
■nv->end = b;  
■nv->prot = 0;  
■nv->flags = flags;  
  
■vma_insert(pa, nv);  
  
■as_unlock();  
■    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 533) - [public] as_unlock

```
■    as_unlock();  
■    return cur;  
■}  
  
■if (new_brk < pa->brk_base || new_brk > pa->brk_max) {  
■    as_unlock();  
■    return pa->brk;  
■}  
  
■uint64_t nb = arch_page_align_up(new_brk);  
  
■if (nb > pa->brk) {  
■    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 538) - [public] as_unlock

```
■ as_unlock();
■ return pa->brk;
■ }

■ uint64_t nb = arch_page_align_up(new_brk);

■ if (nb > pa->brk) {
■     if (!map_anon_pages(pa, pa->brk, nb, PROT_READ | PROT_WRITE)) {
■         as_unlock();
■         return pa->brk;
■     }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 545) - [public] if

```
■ if (!map_anon_pages(pa, pa->brk, nb, PROT_READ | PROT_WRITE)) {
■     as_unlock();
■     return pa->brk;
■ }

■ vma_t *v = aspace_vma_find(pa, pa->brk_base);

■ if (v == NULL) {
■     v = (vma_t *)heap_alloc(sizeof(vma_t));

    if (v == NULL) {
        unmap_pages_free(pa, pa->brk, nb);
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 546) - [public] as_unlock

```
■ as_unlock();
■ return pa->brk;
■ }

■ vma_t *v = aspace_vma_find(pa, pa->brk_base);

■ if (v == NULL) {
■     v = (vma_t *)heap_alloc(sizeof(vma_t));

    if (v == NULL) {
        unmap_pages_free(pa, pa->brk, nb);
        as_unlock();
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/process/address_space.c (line 552) - [public] if

```
■ if (v == NULL) {
■     v = (vma_t *)heap_alloc(sizeof(vma_t));

    if (v == NULL) {
        unmap_pages_free(pa, pa->brk, nb);
        as_unlock();
        return pa->brk;
    }

    v->start = pa->brk_base;
    v->end = nb;
    v->prot = PROT_READ | PROT_WRITE;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 555) - [public] if

```
■      if (v == NULL) {
■          unmap_pages_free(pa, pa->brk, nb);
■          as_unlock();
■          return pa->brk;
■      }

■      v->start = pa->brk_base;
■      v->end = nb;
■      v->prot = PROT_READ | PROT_WRITE;
■      v->flags = VMA_FLAG_ANON | VMA_FLAG_PRIVATE;

■      vma_insert(pa, v);
      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 556) - [public] unmap_pages_free

```
■      unmap_pages_free(pa, pa->brk, nb);
■      as_unlock();
■      return pa->brk;
■  }

■      v->start = pa->brk_base;
■      v->end = nb;
■      v->prot = PROT_READ | PROT_WRITE;
■      v->flags = VMA_FLAG_ANON | VMA_FLAG_PRIVATE;

■      vma_insert(pa, v);
■  } else {
      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 557) - [public] as_unlock

```
■      as_unlock();
■      return pa->brk;
■  }

■      v->start = pa->brk_base;
■      v->end = nb;
■      v->prot = PROT_READ | PROT_WRITE;
■      v->flags = VMA_FLAG_ANON | VMA_FLAG_PRIVATE;

■      vma_insert(pa, v);
■  } else {
■      v->end = nb;
      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 566) - [public] vma_insert

```
■      vma_insert(pa, v);
■  } else {
■      v->end = nb;
■  }

■      pa->brk = nb;
■  } else if (nb < pa->brk) {
■      aspace_unmap(pa, nb, pa->brk - nb);
■      pa->brk = nb;
■  }

■  uint64_t cur = pa->brk;
      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/address_space.c (line 573) - [public] aspace_unmap

```
■      aspace_unmap(pa, nb, pa->brk - nb);
■      pa->brk = nb;
■  }

■  uint64_t cur = pa->brk;

■  as_unlock();

■  return cur;
}
```


kernel_Workspace/kernel/src/mm/process/mmap.c (line 17) - [public] aspace_map_at

```
    return aspace_map_at(pa, addr, len, prot);
}
```

```
return aspace_map_anon(pa, addr, len, prot);
}
```

```
bool munmap(proc_aspace_t *pa, uint64_t addr, size_t len)
{
    return aspace_unmap(pa, addr, len);
}
```

```
bool mprotect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/mmap.c (line 20) - [public] aspace_map_anon

```
return aspace_map_anon(pa, addr, len, prot);
}
```

```
bool munmap(proc_aspace_t *pa, uint64_t addr, size_t len)
{
    return aspace_unmap(pa, addr, len);
}
```

```
bool mprotect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot)
{
    return aspace_protect_checked(pa, addr, len,
                                   prot, perm_sanitizet(prot),
                                   // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/process/mmap.c (line 25) - [public] aspace_unmap

```
return aspace_unmap(pa, addr, len);
}
```

```
bool mprotect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot)
{
    return aspace_protect_checked(pa, addr, len,
                                   prot, perm_sanitizet(prot),
                                   perm_mprotect_allowed);
}
```

kernel_Workspace/kernel/src/mm/protection/guard.c (line 6) - [public] aspace_reserve_at

```
return aspace_reserve_at(pa, addr, len, VMA_FLAG_GUARD);
}
```

```
bool guard_user_stack(proc_aspace_t *pa)
{
    uint64_t stack_base = aspace_stack_base();
}
```

```
if (stack_base < ARCH_PAGE_SIZE) {
    return false;
}
```

```
return aspace_reserve_at(pa,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/protection/isolation.c (line 5) - [public] kprintf

```
extern void kprintf(const char *fmt, ...);
```

```
static void cpuid(uint32_t leaf,
    uint32_t subleaf,
    uint32_t *a,
    uint32_t *b,
    uint32_t *c,
    uint32_t *d)
{
    __asm__ volatile(
        "cpuid"
        : "=a"(*a), "=b"(*b), "=c"(*c), "=d"(*d)
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/protection/isolation.c (line 24) - [public] volatile
    ■__asm__ volatile("mov %%cr4, %0" : "=r"(v));

    ■return v;
    }

    static void write_cr4(uint64_t v)
    {
    ■__asm__ volatile("mov %0, %%cr4" :: "r"(v) : "memory");
    }

    bool isolation_has_smep(void)
    {
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/protection/isolation.c (line 31) - [public] volatile
    ■__asm__ volatile("mov %0, %%cr4" :: "r"(v) : "memory");
    }

    bool isolation_has_smep(void)
    {
    ■uint32_t a, b, c, d;

    ■cpuid(7, 0, &a, &b, &c, &d);

    ■return (b >> 7) & 1;
    }

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/protection/isolation.c (line 81) - [public] if
    ■    if (!pml_entry_present(e)) {
    ■        continue;
    ■    }

    ■    if (e & PTE_USER) {
    ■        violations++;
    ■    }

    ■    if (!pml_entry_large(e) && level > PML_LEVEL_PT) {
    ■        violations =
    ■            audit_tables(pml_entry_addr(e), level - 1, violations);
    ■    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/protection/isolation.c (line 85) - [public] if
    ■    if (e & PTE_USER) {
    ■        violations++;
    ■    }

    ■    if (!pml_entry_large(e) && level > PML_LEVEL_PT) {
    ■        violations =
    ■            audit_tables(pml_entry_addr(e), level - 1, violations);
    ■    }
    ■}

    ■return violations;
    }
    // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/mm/protection/isolation.c (line 89) - [public] if

```
■    if (!pml_entry_large(e) && level > PML_LEVEL_PT) {  
■        violations =  
■            audit_tables(pml_entry_addr(e), level - 1, violations);  
■    }  
■}
```

```
■return violations;  
}
```

```
size_t isolation_audit_kernel(void)
```

```
{  
■uint64_t pml4 = paging_read_cr3();  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/protection/isolation.c (line 91) - [public] audit_tables

```
■        audit_tables(pml_entry_addr(e), level - 1, violations);  
■    }  
■}
```

```
■return violations;  
}
```

```
size_t isolation_audit_kernel(void)
```

```
{  
■uint64_t pml4 = paging_read_cr3();  
■uint64_t *t = pml_table_ptr(pml4);  
  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/protection/isolation.c (line 108) - [public] if

```
■    if (!pml_entry_present(e)) {  
■        continue;  
■    }  
  
■    if (e & PTE_USER) {  
■        violations++;  
■    }  
  
■    if (!pml_entry_large(e)) {  
■        violations =  
■            audit_tables(pml_entry_addr(e), PML_LEVEL_PDPT, violations);  
■    }  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/protection/isolation.c (line 112) - [public] if

```
■    if (e & PTE_USER) {  
■        violations++;  
■    }  
  
■    if (!pml_entry_large(e)) {  
■        violations =  
■            audit_tables(pml_entry_addr(e), PML_LEVEL_PDPT, violations);  
■    }  
■}
```

```
■return violations;  
}  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/protection/isolation.c (line 116) - [public] if

```
■    if (!pml_entry_large(e)) {
■        violations =
■            audit_tables(pml_entry_addr(e), PML_LEVEL_PDPT, violations);
■    }
■}

■return violations;
}

bool isolation_init(void)
{
■paging_enable_nx();
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/mm/protection/isolation.c (line 118) - [public] audit_tables

```
■        audit_tables(pml_entry_addr(e), PML_LEVEL_PDPT, violations);
■    }
■}

■return violations;
}

bool isolation_init(void)
{
■paging_enable_nx();
■isolation_enable_smep();
■isolation_enable_smap();
    // ... (truncated, see source file for more)
```

kstd/src/kernel/audio.c (line 3) - [public] k_audio_play

```
extern int32_t k_audio_play(uint64_t phys, uint32_t len);
extern int32_t k_audio_stop(void);
extern int32_t k_audio_jack(void);
extern int32_t k_audio_amp(int32_t on);
extern uint64_t kvirt_to_phys(void *ptr);

tr_status_t tr_audio_play(const void *data, uint32_t len)
{
    uint64_t phys = kvirt_to_phys((void *)data);

    if (phys == (uint64_t)-1) {
        return TR_ERR_INVALID;
        // ... (truncated, see source file for more)
```

kstd/src/kernel/audio.c (line 4) - [public] k_audio_stop

```
extern int32_t k_audio_stop(void);
extern int32_t k_audio_jack(void);
extern int32_t k_audio_amp(int32_t on);
extern uint64_t kvirt_to_phys(void *ptr);

tr_status_t tr_audio_play(const void *data, uint32_t len)
{
    uint64_t phys = kvirt_to_phys((void *)data);

    if (phys == (uint64_t)-1) {
        return TR_ERR_INVALID;
    }
    // ... (truncated, see source file for more)
```

```

kstd/src/kernel/audio.c (line 5) - [public] k_audio_jack
extern int32_t k_audio_jack(void);
extern int32_t k_audio_amp(int32_t on);
extern uint64_t kvirt_to_phys(void *ptr);

tr_status_t tr_audio_play(const void *data, uint32_t len)
{
    uint64_t phys = kvirt_to_phys((void *)data);

    if (phys == (uint64_t)-1) {
        return TR_ERR_INVALID;
    }

    // ... (truncated, see source file for more)
}

kstd/src/kernel/audio.c (line 6) - [public] k_audio_amp
extern int32_t k_audio_amp(int32_t on);
extern uint64_t kvirt_to_phys(void *ptr);

tr_status_t tr_audio_play(const void *data, uint32_t len)
{
    uint64_t phys = kvirt_to_phys((void *)data);

    if (phys == (uint64_t)-1) {
        return TR_ERR_INVALID;
    }

    return k_audio_play(phys, len) == 0 ? TR_OK : TR_ERR_IO;
    // ... (truncated, see source file for more)
}

kstd/src/kernel/audio.c (line 7) - [public] kvirt_to_phys
extern uint64_t kvirt_to_phys(void *ptr);

tr_status_t tr_audio_play(const void *data, uint32_t len)
{
    uint64_t phys = kvirt_to_phys((void *)data);

    if (phys == (uint64_t)-1) {
        return TR_ERR_INVALID;
    }

    return k_audio_play(phys, len) == 0 ? TR_OK : TR_ERR_IO;
}
// ... (truncated, see source file for more)

kstd/src/kernel/audio.c (line 13) - [public] if
    if (phys == (uint64_t)-1) {
        return TR_ERR_INVALID;
    }

    return k_audio_play(phys, len) == 0 ? TR_OK : TR_ERR_IO;
}

tr_status_t tr_audio_stop(void)
{
    k_audio_stop();
    return TR_OK;
}
// ... (truncated, see source file for more)

```

kstd/src/kernel/audio.c (line 22) - [public] k_audio_stop

```
    k_audio_stop();
    return TR_OK;
}

tr_status_t tr_audio_jack(bool *present)
{
    if (!present) {
        return TR_ERR_INVALID;
    }

    *present = k_audio_jack() == 1;
    return TR_OK;
    // ... (truncated, see source file for more)
```

kstd/src/kernel/audio.c (line 28) - [public] if

```
    if (!present) {
        return TR_ERR_INVALID;
    }

    *present = k_audio_jack() == 1;
    return TR_OK;
}

tr_status_t tr_audio_amp(bool on)
{
    k_audio_amp(on ? 1 : 0);
    return TR_OK;
    // ... (truncated, see source file for more)
```

kstd/src/kernel/audio.c (line 38) - [public] k_audio_amp

```
    k_audio_amp(on ? 1 : 0);
    return TR_OK;
}
```

kstd/src/kernel/bt.c (line 3) - [public] bt_ready

```
extern bool bt_ready(void);
extern void bt_info(uint8_t *ver, uint8_t *addr);
extern bool bt_hci_cmd(uint16_t op, const uint8_t *p, uint8_t len);
extern bool bt_event_poll(uint8_t *buf, uint8_t *len);
extern bool bt_acl_send(const uint8_t *d, uint16_t len);
extern bool bt_acl_recv(uint8_t *d, uint16_t *len);

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    if (!bt_ready()) {
        return TR_ERR_IO;
    }
    // ... (truncated, see source file for more)
```

kstd/src/kernel/bt.c (line 4) - [public] bt_info

```
extern void bt_info(uint8_t *ver, uint8_t *addr);
extern bool bt_hci_cmd(uint16_t op, const uint8_t *p, uint8_t len);
extern bool bt_event_poll(uint8_t *buf, uint8_t *len);
extern bool bt_acl_send(const uint8_t *d, uint16_t len);
extern bool bt_acl_recv(uint8_t *d, uint16_t *len);

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    if (!bt_ready()) {
        return TR_ERR_IO;
    }

    // ... (truncated, see source file for more)
```

```
kstd/src/kernel/bt.c (line 5) - [public] bt_hci_cmd
extern bool bt_hci_cmd(uint16_t op, const uint8_t *p, uint8_t len);
extern bool bt_event_poll(uint8_t *buf, uint8_t *len);
extern bool bt_acl_send(const uint8_t *d, uint16_t len);
extern bool bt_acl_recv(uint8_t *d, uint16_t *len);

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    if (!bt_ready()) {
        return TR_ERR_IO;
    }

    bt_info(hci_ver, addr6);
    // ... (truncated, see source file for more)
}
```

```
kstd/src/kernel/bt.c (line 6) - [public] bt_event_poll
extern bool bt_event_poll(uint8_t *buf, uint8_t *len);
extern bool bt_acl_send(const uint8_t *d, uint16_t len);
extern bool bt_acl_recv(uint8_t *d, uint16_t *len);

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    if (!bt_ready()) {
        return TR_ERR_IO;
    }

    bt_info(hci_ver, addr6);
    return TR_OK;
    // ... (truncated, see source file for more)
}
```

```
kstd/src/kernel/bt.c (line 7) - [public] bt_acl_send
extern bool bt_acl_send(const uint8_t *d, uint16_t len);
extern bool bt_acl_recv(uint8_t *d, uint16_t *len);

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    if (!bt_ready()) {
        return TR_ERR_IO;
    }

    bt_info(hci_ver, addr6);
    return TR_OK;
}
// ... (truncated, see source file for more)
```

```
kstd/src/kernel/bt.c (line 8) - [public] bt_acl_recv
extern bool bt_acl_recv(uint8_t *d, uint16_t *len);

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    if (!bt_ready()) {
        return TR_ERR_IO;
    }

    bt_info(hci_ver, addr6);
    return TR_OK;
}
// ... (truncated, see source file for more)
```

kstd/src/kernel/bt.c (line 12) - [public] if

```
    if (!bt_ready()) {
        return TR_ERR_IO;
    }

    bt_info(hci_ver, addr6);
    return TR_OK;
}

tr_status_t tr_bt_cmd(uint16_t opcode, const void *params, uint8_t len)
{
    return bt_hci_cmd(opcode, params, len) ? TR_OK : TR_ERR_IO;
}

// ... (truncated, see source file for more)
```

kstd/src/kernel/bt.c (line 16) - [public] bt_info

```
    bt_info(hci_ver, addr6);
    return TR_OK;
}

tr_status_t tr_bt_cmd(uint16_t opcode, const void *params, uint8_t len)
{
    return bt_hci_cmd(opcode, params, len) ? TR_OK : TR_ERR_IO;
}

tr_status_t tr_bt_evt(void *buf, uint8_t *len)
{
    return bt_event_poll(buf, len) ? TR_OK : TR_ERR_TIMEOUT;
}

// ... (truncated, see source file for more)
```

kstd/src/kernel/fs.c (line 3) - [public] k_fs_read

```
extern int32_t k_fs_read(const char *path, void *buf, uint32_t cap);
extern int32_t k_fs_exists(const char *path);

tr_status_t tr_fs_read(const char *path, void *buf, uint32_t cap, uint32_t *out)
{
    int32_t n = k_fs_read(path, buf, cap);

    if (n < 0) {
        return TR_ERR_NOTFOUND;
    }

    if (out) {
        // ... (truncated, see source file for more)
    }
}
```

kstd/src/kernel/fs.c (line 4) - [public] k_fs_exists

```
extern int32_t k_fs_exists(const char *path);

tr_status_t tr_fs_read(const char *path, void *buf, uint32_t cap, uint32_t *out)
{
    int32_t n = k_fs_read(path, buf, cap);

    if (n < 0) {
        return TR_ERR_NOTFOUND;
    }

    if (out) {
        *out = (uint32_t)n;
        // ... (truncated, see source file for more)
    }
}
```



```

kstd/src/kernel/fs.c (line 10) - [public] if
    if (n < 0) {
        return TR_ERR_NOTFOUND;
    }

    if (out) {
        *out = (uint32_t)n;
    }

    return TR_OK;
}

tr_status_t tr_fs_exists(const char *path)
    // ... (truncated, see source file for more)

kstd/src/kernel/fs.c (line 14) - [public] if
    if (out) {
        *out = (uint32_t)n;
    }

    return TR_OK;
}

tr_status_t tr_fs_exists(const char *path)
{
    return k_fs_exists(path) == 1 ? TR_OK : TR_ERR_NOTFOUND;
}

kstd/src/kernel/input.c (line 3) - [public] k_input_key
extern int32_t k_input_key(void);

int32_t tr_input_key(void)
{
    return k_input_key();
}

kstd/src/kernel/input.c (line 7) - [public] k_input_key
    return k_input_key();
}

kstd/src/kernel/log.c (line 3) - [public] kprintf
extern void kprintf(const char *fmt, ...);

void tr_log(const char *s)
{
    kprintf("%s\n", s);
}

kstd/src/kernel/log.c (line 7) - [public] kprintf
    kprintf("%s\n", s);
}

kstd/src/kernel/mem.c (line 4) - [public] kfree
extern void kfree(void *ptr);
extern bool vmm_map_device(uint64_t phys, unsigned long len, uint64_t *virt);
extern uint64_t kvirt_to_phys(void *ptr);

void *tr_alloc(uint32_t bytes)
{
    return kmalloc(bytes);
}

void tr_free(void *ptr, uint32_t bytes)
{
    (void)bytes;
    // ... (truncated, see source file for more)
}

```

```

kstd/src/kernel/mem.c (line 5) - [public] vmm_map_device
extern bool vmm_map_device(uint64_t phys, unsigned long len, uint64_t *virt);
extern uint64_t kvirt_to_phys(void *ptr);

void *tr_alloc(uint32_t bytes)
{
    return kmalloc(bytes);
}

void tr_free(void *ptr, uint32_t bytes)
{
    (void)bytes;
    kfree(ptr);
    // ... (truncated, see source file for more)
}

kstd/src/kernel/mem.c (line 6) - [public] kvirt_to_phys
extern uint64_t kvirt_to_phys(void *ptr);

void *tr_alloc(uint32_t bytes)
{
    return kmalloc(bytes);
}

void tr_free(void *ptr, uint32_t bytes)
{
    (void)bytes;
    kfree(ptr);
}
// ... (truncated, see source file for more)

kstd/src/kernel/mem.c (line 10) - [public] kmalloc
return kmalloc(bytes);
}

void tr_free(void *ptr, uint32_t bytes)
{
    (void)bytes;
    kfree(ptr);
}

tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va)
{
    uint64_t v = 0;
    // ... (truncated, see source file for more)
}

kstd/src/kernel/mem.c (line 16) - [public] kfree
kfree(ptr);
}

tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va)
{
    uint64_t v = 0;

    if (vmm_map_device(phys, len, &v)) {
        *out_va = (void *)v;
        return TR_OK;
    }

    // ... (truncated, see source file for more)
}

kstd/src/kernel/mem.c (line 23) - [public] if
if (vmm_map_device(phys, len, &v)) {
    *out_va = (void *)v;
    return TR_OK;
}

return TR_ERR_IO;
}

```

kstd/src/kernel/video.c (line 8) - [public] tr_video_init

```
tr_status_t tr_video_init(void) {
    if (dp_ready()) {
        use_dp = true;
        return TR_OK;
    }
    if (hdmi_ready()) {
        use_dp = false;
        return TR_OK;
    }
    return TR_ERR_IO;
}
```

// ... (truncated, see source file for more)

kstd/src/kernel/video.c (line 9) - [public] if

```
if (dp_ready()) {
    use_dp = true;
    return TR_OK;
}
if (hdmi_ready()) {
    use_dp = false;
    return TR_OK;
}
return TR_ERR_IO;
}
```

```
bool tr_video_is_ready(void) {
    // ... (truncated, see source file for more)
```

kstd/src/kernel/video.c (line 13) - [public] if

```
if (hdmi_ready()) {
    use_dp = false;
    return TR_OK;
}
return TR_ERR_IO;
}
```

```
bool tr_video_is_ready(void) {
    return use_dp ? dp_ready() : hdmi_ready();
}
```

```
tr_status_t tr_video_get_caps(tr_video_caps_t *caps) {
    // ... (truncated, see source file for more)
```

kstd/src/kernel/video.c (line 20) - [public] tr_video_is_ready

```
bool tr_video_is_ready(void) {
    return use_dp ? dp_ready() : hdmi_ready();
}
```

```
tr_status_t tr_video_get_caps(tr_video_caps_t *caps) {
    if (!caps) return TR_ERR_INVALID;

    if (use_dp) {
        dp_caps(&caps->fb_phys, &caps->width, &caps->height, &caps->stride);
        caps->ready = dp_ready();
    } else {
        hdmi_caps_raw(&caps->width, &caps->height, &caps->stride, &caps->fb_phys);
        // ... (truncated, see source file for more)
```

kstd/src/kernel/video.c (line 24) - [public] tr_video_get_caps

```
tr_status_t tr_video_get_caps(tr_video_caps_t *caps) {
    if (!caps) return TR_ERR_INVALID;

    if (use_dp) {
        dp_caps(&caps->fb_phys, &caps->width, &caps->height, &caps->stride);
        caps->ready = dp_ready();
    } else {
        hdmi_caps_raw(&caps->width, &caps->height, &caps->stride, &caps->fb_phys);
        caps->ready = hdmi_ready();
    }
    return TR_OK;
}

// ... (truncated, see source file for more)
```

kstd/src/kernel/video.c (line 27) - [public] if

```
if (use_dp) {
    dp_caps(&caps->fb_phys, &caps->width, &caps->height, &caps->stride);
    caps->ready = dp_ready();
} else {
    hdmi_caps_raw(&caps->width, &caps->height, &caps->stride, &caps->fb_phys);
    caps->ready = hdmi_ready();
}
return TR_OK;
}

tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
    if (!tr_video_is_ready()) return TR_ERR_IO;
    // ... (truncated, see source file for more)
```

kstd/src/kernel/video.c (line 28) - [public] dp_caps

```
dp_caps(&caps->fb_phys, &caps->width, &caps->height, &caps->stride);
caps->ready = dp_ready();
} else {
    hdmi_caps_raw(&caps->width, &caps->height, &caps->stride, &caps->fb_phys);
    caps->ready = hdmi_ready();
}
return TR_OK;
}

tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
    if (!tr_video_is_ready()) return TR_ERR_IO;

    // ... (truncated, see source file for more)
```

kstd/src/kernel/video.c (line 31) - [public] hdmi_caps_raw

```
hdmi_caps_raw(&caps->width, &caps->height, &caps->stride, &caps->fb_phys);
caps->ready = hdmi_ready();
}
return TR_OK;
}

tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
    if (!tr_video_is_ready()) return TR_ERR_IO;

    uint64_t seq = 0;
    if (use_dp) {
        seq = dp_submit_fill(color, x, y, w, h);
    }
    // ... (truncated, see source file for more)
```

```

kstd/src/kernel/video.c (line 37) - [public] tr_video_fill
    tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
        if (!tr_video_is_ready()) return TR_ERR_IO;

        uint64_t seq = 0;
        if (use_dp) {
            seq = dp_submit_fill(color, x, y, w, h);
        } else {
            seq = hdmi_submit_fill(color, x, y, w, h);
        }

        return (seq > 0) ? TR_OK : TR_ERR_BUSY;
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/kernel/video.c (line 41) - [public] if
    if (use_dp) {
        seq = dp_submit_fill(color, x, y, w, h);
    } else {
        seq = hdmi_submit_fill(color, x, y, w, h);
    }

    return (seq > 0) ? TR_OK : TR_ERR_BUSY;
}

/* ... reszta funkcji mapuje 1:1 na hdmi/dp ... */

```

```

kstd/src/user/audio.c (line 7) - [public] ds_poll
extern void ds_poll(void);
extern int ds_take(uint64_t id, ds_msg_t *out);

static void *bounce = TR_NULL;
static uint64_t bounce_phys = 0;

tr_status_t tr_audio_play(const void *data, uint32_t len)
{
    if (bounce == TR_NULL) {
        bounce = tr_alloc(4096 * 4);

        if (bounce == TR_NULL) {
            // ... (truncated, see source file for more)
        }
    }
}

```

```

kstd/src/user/audio.c (line 8) - [public] ds_take
extern int ds_take(uint64_t id, ds_msg_t *out);

static void *bounce = TR_NULL;
static uint64_t bounce_phys = 0;

tr_status_t tr_audio_play(const void *data, uint32_t len)
{
    if (bounce == TR_NULL) {
        bounce = tr_alloc(4096 * 4);

        if (bounce == TR_NULL) {
            return TR_ERR_NOMEM;
        }
        // ... (truncated, see source file for more)
    }
}

```

```

kstd/src/user/audio.c (line 15) - [public] if
    if (bounce == TR_NULL) {
        bounce = tr_alloc(4096 * 4);

        if (bounce == TR_NULL) {
            return TR_ERR_NOMEM;
        }
    }

    uint64_t id = ds_call(SVC_SYS, OP_PAGEPHYS, (uint64_t)bounce, 0, 0);
    ds_poll();

    ds_msg_t r;

    // ... (truncated, see source file for more)

```

```

kstd/src/user/audio.c (line 18) - [public] if
    if (bounce == TR_NULL) {
        return TR_ERR_NOMEM;
    }

    uint64_t id = ds_call(SVC_SYS, OP_PAGEPHYS, (uint64_t)bounce, 0, 0);
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_IO;
    }
    // ... (truncated, see source file for more)

kstd/src/user/audio.c (line 23) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_IO;
    }

    bounce_phys = r.arg0;
}

if (len > 4096 * 4) {
    // ... (truncated, see source file for more)

kstd/src/user/audio.c (line 27) - [public] if
    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_IO;
    }

    bounce_phys = r.arg0;
}

if (len > 4096 * 4) {
    len = 4096 * 4;
}

uint8_t *dst = (uint8_t *)bounce;
// ... (truncated, see source file for more)

kstd/src/user/audio.c (line 34) - [public] if
    if (len > 4096 * 4) {
        len = 4096 * 4;
    }

    uint8_t *dst = (uint8_t *)bounce;
    const uint8_t *src = (const uint8_t *)data;

    for (uint32_t i = 0; i < len; i++) {
        dst[i] = src[i];
    }

    uint64_t id = ds_call(SVC_AUDIO, AUD_PLAY, bounce_phys, len, 0);
    // ... (truncated, see source file for more)

```

```

kstd/src/user/audio.c (line 46) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        return TR_OK;
    }

    return TR_ERR_DENIED;
}

tr_status_t tr_audio_stop(void)
    // ... (truncated, see source file for more)

kstd/src/user/audio.c (line 50) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        return TR_OK;
    }

    return TR_ERR_DENIED;
}

tr_status_t tr_audio_stop(void)
{
    ds_call(SVC_AUDIO, AUD_STOP, 0, 0, 0);
    return TR_OK;
}
    // ... (truncated, see source file for more)

kstd/src/user/audio.c (line 59) - [public] ds_call
    ds_call(SVC_AUDIO, AUD_STOP, 0, 0, 0);
    return TR_OK;
}

tr_status_t tr_audio_jack(bool *present)
{
    if (!present) {
        return TR_ERR_INVALID;
    }

    uint64_t id = ds_call(SVC_AUDIO, AUD_JACK, 0, 0, 0);
    ds_poll();
    // ... (truncated, see source file for more)

kstd/src/user/audio.c (line 65) - [public] if
    if (!present) {
        return TR_ERR_INVALID;
    }

    uint64_t id = ds_call(SVC_AUDIO, AUD_JACK, 0, 0, 0);
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        *present = (r.arg0 & 1) != 0;
        return TR_OK;
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/user/audio.c (line 70) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        *present = (r.arg0 & 1) != 0;
        return TR_OK;
    }

    return TR_ERR_IO;
}

// ... (truncated, see source file for more)

kstd/src/user/audio.c (line 74) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        *present = (r.arg0 & 1) != 0;
        return TR_OK;
    }

    return TR_ERR_IO;
}

tr_status_t tr_audio_amp(bool on)
{
    ds_call(SVC_AUDIO, AUD_AMP, on ? 1 : 0, 0, 0);
    return TR_OK;
    // ... (truncated, see source file for more)

kstd/src/user/audio.c (line 84) - [public] ds_call
    ds_call(SVC_AUDIO, AUD_AMP, on ? 1 : 0, 0, 0);
    return TR_OK;
}

kstd/src/user/bt.c (line 6) - [public] ds_poll
extern void ds_poll(void);
extern int ds_take(uint64_t id, ds_msg_t *out);

static volatile uint8_t *scratch = (volatile uint8_t *)DS_SCRATCH_VA;

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    uint64_t id = ds_call(SVC_BT, BT_INFO, 0, 0, 0);
    ds_poll();

    ds_msg_t r;

    // ... (truncated, see source file for more)

kstd/src/user/bt.c (line 7) - [public] ds_take
extern int ds_take(uint64_t id, ds_msg_t *out);

static volatile uint8_t *scratch = (volatile uint8_t *)DS_SCRATCH_VA;

tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6)
{
    uint64_t id = ds_call(SVC_BT, BT_INFO, 0, 0, 0);
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        // ... (truncated, see source file for more)
    }
}

```



```

kstd/src/user/bt.c (line 14) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_IO;
    }

    *hci_ver = (uint8_t)r.arg0;

    for (int i = 0; i < 6; i++) {
        addr6[i] = (uint8_t)(r.arg1 >> (i * 8));
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/user/bt.c (line 18) - [public] if
    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_IO;
    }

    *hci_ver = (uint8_t)r.arg0;

    for (int i = 0; i < 6; i++) {
        addr6[i] = (uint8_t)(r.arg1 >> (i * 8));
    }

    return TR_OK;
}
// ... (truncated, see source file for more)

```

```

kstd/src/user/bt.c (line 40) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        return TR_OK;
    }

    return TR_ERR_DENIED;
}

tr_status_t tr_bt_evt(void *buf, uint8_t *len)
    // ... (truncated, see source file for more)

```

```

kstd/src/user/bt.c (line 44) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        return TR_OK;
    }

    return TR_ERR_DENIED;
}

tr_status_t tr_bt_evt(void *buf, uint8_t *len)
{
    uint64_t id = ds_call(SVC_BT, BT_EVT, 0, 0, 0);
    ds_poll();

    // ... (truncated, see source file for more)

```

```

kstd/src/user/bt.c (line 54) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_TIMEOUT;
    }

    uint8_t n = (uint8_t)r.arg0;

    uint8_t *dst = (uint8_t *)buf;

    // ... (truncated, see source file for more)

kstd/src/user/bt.c (line 58) - [public] if
    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_TIMEOUT;
    }

    uint8_t n = (uint8_t)r.arg0;

    uint8_t *dst = (uint8_t *)buf;

    for (uint8_t i = 0; i < n; i++) {
        dst[i] = scratch[i];
    }

    // ... (truncated, see source file for more)

kstd/src/user/bt.c (line 83) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        return TR_OK;
    }

    return TR_ERR_BUSY;
}

tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len)
    // ... (truncated, see source file for more)

kstd/src/user/bt.c (line 87) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        return TR_OK;
    }

    return TR_ERR_BUSY;
}

tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len)
{
    uint64_t id = ds_call(SVC_BT, BT_ACL_IN, 0, 0, 0);
    ds_poll();

    // ... (truncated, see source file for more)

```

```

kstd/src/user/bt.c (line 97) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_TIMEOUT;
    }

    uint16_t n = (uint16_t)r.arg0;

    uint8_t *dst = (uint8_t *)buf;

    // ... (truncated, see source file for more)

```

```

kstd/src/user/bt.c (line 101) - [public] if
    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_TIMEOUT;
    }

    uint16_t n = (uint16_t)r.arg0;

    uint8_t *dst = (uint8_t *)buf;

    for (uint16_t i = 0; i < n; i++) {
        dst[i] = scratch[i];
    }

    // ... (truncated, see source file for more)

```

```

kstd/src/user/fs.c (line 6) - [public] ds_poll
extern void ds_poll(void);
extern int ds_take(uint64_t id, ds_msg_t *out);

static volatile uint8_t *scratch = (volatile uint8_t *)DS_SCRATCH_VA;

tr_status_t tr_fs_read(const char *path, void *buf, uint32_t cap, uint32_t *out)
{
    uint32_t n = 0;

    while (path[n] && n < 255) {
        scratch[n] = (uint8_t)path[n];
        n++;
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/user/fs.c (line 7) - [public] ds_take
extern int ds_take(uint64_t id, ds_msg_t *out);

static volatile uint8_t *scratch = (volatile uint8_t *)DS_SCRATCH_VA;

tr_status_t tr_fs_read(const char *path, void *buf, uint32_t cap, uint32_t *out)
{
    uint32_t n = 0;

    while (path[n] && n < 255) {
        scratch[n] = (uint8_t)path[n];
        n++;
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/user/fs.c (line 15) - [public] while
    while (path[n] && n < 255) {
        scratch[n] = (uint8_t)path[n];
        n++;
    }

    scratch[n] = 0;

    uint64_t id = ds_call(SVC_FS, FS_READ, n, cap, 0);
    ds_poll();

    ds_msg_t r;

    // ... (truncated, see source file for more)

```

```

kstd/src/user/fs.c (line 23) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_NOTFOUND;
    }

    uint32_t got = (uint32_t)r.arg0;

    if (got > cap) {
        got = cap;
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/user/fs.c (line 27) - [public] if
    if (!ds_take(id, &r) || r.status != 0) {
        return TR_ERR_NOTFOUND;
    }

    uint32_t got = (uint32_t)r.arg0;

    if (got > cap) {
        got = cap;
    }

    uint8_t *dst = (uint8_t *)buf;

    // ... (truncated, see source file for more)

```

```

kstd/src/user/fs.c (line 33) - [public] if
    if (got > cap) {
        got = cap;
    }

    uint8_t *dst = (uint8_t *)buf;

    for (uint32_t i = 0; i < got; i++) {
        dst[i] = scratch[256 + i];
    }

    if (out) {
        *out = got;
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/user/fs.c (line 43) - [public] if
    if (out) {
        *out = got;
    }

    return TR_OK;
}

tr_status_t tr_fs_exists(const char *path)
{
    uint32_t n = 0;

    while (path[n] && n < 255) {
        // ... (truncated, see source file for more)

kstd/src/user/fs.c (line 54) - [public] while
    while (path[n] && n < 255) {
        scratch[n] = (uint8_t)path[n];
        n++;
    }

    scratch[n] = 0;

    uint64_t id = ds_call(SVC_FS, FS_EXISTS, n, 0, 0);
    ds_poll();

    ds_msg_t r;

    // ... (truncated, see source file for more)

kstd/src/user/fs.c (line 62) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0 && r.arg0 == 1) {
        return TR_OK;
    }

    return TR_ERR_NOTFOUND;
}

kstd/src/user/fs.c (line 66) - [public] if
    if (ds_take(id, &r) && r.status == 0 && r.arg0 == 1) {
        return TR_OK;
    }

    return TR_ERR_NOTFOUND;
}

kstd/src/user/input.c (line 6) - [public] ds_poll
extern void ds_poll(void);
extern int ds_take(uint64_t id, ds_msg_t *out);

int32_t tr_input_key(void)
{
    uint64_t id = ds_call(SVC_INPUT, IN_KEY_POLL, 0, 0, 0);
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0 && r.arg0 != 0) {
        return (int32_t)r.arg0;
        // ... (truncated, see source file for more)
    }
}

```

```

kstd/src/user/input.c (line 7) - [public] ds_take
extern int ds_take(uint64_t id, ds_msg_t *out);

int32_t tr_input_key(void)
{
    uint64_t id = ds_call(SVC_INPUT, IN_KEY_POLL, 0, 0, 0);
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0 && r.arg0 != 0) {
        return (int32_t)r.arg0;
    }
    // ... (truncated, see source file for more)

kstd/src/user/input.c (line 12) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0 && r.arg0 != 0) {
        return (int32_t)r.arg0;
    }

    return -1;
}

kstd/src/user/input.c (line 16) - [public] if
    if (ds_take(id, &r) && r.status == 0 && r.arg0 != 0) {
        return (int32_t)r.arg0;
    }

    return -1;
}

kstd/src/user/log.c (line 10) - [public] while
    while (s[n] && n < DS_SCRATCH_SIZE - 1) {
        scratch[n] = s[n];
        n++;
    }

    ds_call(SVC_SYS, OP_LOG, n, 0, 0);
}

kstd/src/user/log.c (line 15) - [public] ds_call
    ds_call(SVC_SYS, OP_LOG, n, 0, 0);
}

kstd/src/user/mem.c (line 6) - [public] ds_poll
extern void ds_poll(void);
extern int ds_take(uint64_t id, ds_msg_t *out);

static uint64_t next_va = 0x44000000ULL;

void *tr_alloc(uint32_t bytes)
{
    uint32_t pages = (bytes + 4095) / 4096;

    uint64_t id = ds_call(SVC_SYS, OP_ALLOC, pages, 0, 0);
    ds_poll();

    // ... (truncated, see source file for more)

```

```

kstd/src/user/mem.c (line 7) - [public] ds_take
extern int ds_take(uint64_t id, ds_msg_t *out);

static uint64_t next_va = 0x44000000ULL;

void *tr_alloc(uint32_t bytes)
{
    uint32_t pages = (bytes + 4095) / 4096;

    uint64_t id = ds_call(SVC_SYS, OP_ALLOC, pages, 0, 0);
    ds_poll();

    ds_msg_t r;
    // ... (truncated, see source file for more)

kstd/src/user/mem.c (line 16) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        return (void *)r.arg0;
    }

    return TR_NULL;
}

void tr_free(void *ptr, uint32_t bytes)
    // ... (truncated, see source file for more)

kstd/src/user/mem.c (line 20) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        return (void *)r.arg0;
    }

    return TR_NULL;
}

void tr_free(void *ptr, uint32_t bytes)
{
    (void)bytes;
    ds_call(SVC_SYS, OP_FREE, (uint64_t)ptr, 0, 0);
}
    // ... (truncated, see source file for more)

kstd/src/user/mem.c (line 30) - [public] ds_call
    ds_call(SVC_SYS, OP_FREE, (uint64_t)ptr, 0, 0);
}

tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va)
{
    uint64_t va = next_va;
    next_va += (len + 4095) & ~4095ULL;

    uint64_t id = ds_call(SVC_SYS, OP_MAPMMIO, phys, len, va);
    ds_poll();

    ds_msg_t r;
    // ... (truncated, see source file for more)

```

```

kstd/src/user/mem.c (line 39) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (ds_take(id, &r) && r.status == 0) {
        *out_va = (void *)va;
        return TR_OK;
    }

    return TR_ERR_DENIED;
}

```

```

kstd/src/user/mem.c (line 43) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        *out_va = (void *)va;
        return TR_OK;
    }

    return TR_ERR_DENIED;
}

```

```

kstd/src/user/video.c (line 6) - [public] tr_video_init
/* Zakładamy, że ds_call i ds_take są dostępne z dsclient.c */

tr_status_t tr_video_init(void) {
    /* W user-space init robi kernel, tu tylko sprawdzamy stan */
    return tr_video_is_ready() ? TR_OK : TR_ERR_IO;
}

bool tr_video_is_ready(void) {
    uint64_t id = ds_call(SVC_VIDEO, VID_HDMI_CAPS, 0, 0, 0);
    ds_poll();
    ds_msg_t r;
    if (ds_take(id, &r) && r.status == 0) {
        return true;
    }
    // ... (truncated, see source file for more)
}

```

```

kstd/src/user/video.c (line 11) - [public] tr_video_is_ready

bool tr_video_is_ready(void) {
    uint64_t id = ds_call(SVC_VIDEO, VID_HDMI_CAPS, 0, 0, 0);
    ds_poll();
    ds_msg_t r;
    if (ds_take(id, &r) && r.status == 0) {
        return true;
    }
    return false;
}

tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
    uint64_t arg1 = ((uint64_t)x) | ((uint64_t)y << 16);
    // ... (truncated, see source file for more)
}

```

```

kstd/src/user/video.c (line 13) - [public] ds_poll

    ds_poll();
    ds_msg_t r;
    if (ds_take(id, &r) && r.status == 0) {
        return true;
    }
    return false;
}

tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
    uint64_t arg1 = ((uint64_t)x) | ((uint64_t)y << 16);
    uint64_t arg2 = ((uint64_t)w) | ((uint64_t)h << 16);

    // ... (truncated, see source file for more)
}

```



```

kstd/src/user/video.c (line 15) - [public] if
    if (ds_take(id, &r) && r.status == 0) {
        return true;
    }
    return false;
}

tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
    uint64_t arg1 = ((uint64_t)x) | ((uint64_t)y << 16);
    uint64_t arg2 = ((uint64_t)w) | ((uint64_t)h << 16);

    uint64_t id = ds_call(SVC_VIDEO, VID_HDMI_FILL, color, arg1, arg2);
    ds_poll();
    // ... (truncated, see source file for more)

```

```

kstd/src/user/video.c (line 21) - [public] tr_video_fill
tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h) {
    uint64_t arg1 = ((uint64_t)x) | ((uint64_t)y << 16);
    uint64_t arg2 = ((uint64_t)w) | ((uint64_t)h << 16);

    uint64_t id = ds_call(SVC_VIDEO, VID_HDMI_FILL, color, arg1, arg2);
    ds_poll();

    ds_msg_t r;
    if (ds_take(id, &r)) {
        if (r.status == 0) return TR_OK;
        if (r.status == -1) return TR_ERR_DENIED; // Autoryzacja/Budzet
        return TR_ERR_IO;
    }
    // ... (truncated, see source file for more)

```

```

kstd/src/user/video.c (line 26) - [public] ds_poll
ds_poll();

ds_msg_t r;
if (ds_take(id, &r)) {
    if (r.status == 0) return TR_OK;
    if (r.status == -1) return TR_ERR_DENIED; // Autoryzacja/Budzet
    return TR_ERR_IO;
}
return TR_ERR_TIMEOUT;
}

/* ... reszta funkcji serializuje argumenty do DsMsg ... */

```

```

kstd/src/user/video.c (line 29) - [public] if
    if (ds_take(id, &r)) {
        if (r.status == 0) return TR_OK;
        if (r.status == -1) return TR_ERR_DENIED; // Autoryzacja/Budzet
        return TR_ERR_IO;
    }
    return TR_ERR_TIMEOUT;
}

/* ... reszta funkcji serializuje argumenty do DsMsg ... */

```

```

libs/dsclient.c (line 30) - [public] if
    if (next == d2k->tail) {
        return 0;
    }

    volatile ds_msg_t *m = &d2k_msgs[head];

    m->id = id;
    m->cmd = svc_cmd(class, op);
    m->flags = 1;
    m->arg0 = a0;
    m->arg1 = a1;
    m->arg2 = a2;
    // ... (truncated, see source file for more)

```

```

libs/dsclient.c (line 51) - [public] while
    while (k2d->tail != k2d->head) {
        volatile ds_msg_t *m = &k2d_msgs[k2d->tail];
        ds_msg_t copy = *m;

        k2d->tail = (k2d->tail + 1) % k2d->cap;

        for (int i = 0; i < 16; i++) {
            if (!cache_used[i]) {
                cache[i] = copy;
                cache_used[i] = 1;
                break;
            }
        }
        // ... (truncated, see source file for more)

libs/dsclient.c (line 58) - [public] if
        if (!cache_used[i]) {
            cache[i] = copy;
            cache_used[i] = 1;
            break;
        }
    }
}

int ds_take(uint64_t id, ds_msg_t *out)
{
    for (int i = 0; i < 16; i++) {
        // ... (truncated, see source file for more)

libs/dsclient.c (line 70) - [public] if
        if (cache_used[i] && cache[i].id == id) {
            *out = cache[i];
            cache_used[i] = 0;
            return 1;
        }
    }

    return 0;
}

// ds_init(params_va);
// uint64_t id = ds_call(SVC_VIDEO, VID_FB_INFO, 0, 0, 0);
// ... (truncated, see source file for more)

triang-lang/out/x86_64/prog.c (line 15) - [public] memset
    memset(buffer, (int)0ull, 16);
    buffer[0] = (uint8_t)65ull;
    buffer[1] = (uint8_t)66ull;
    buffer[2] = (uint8_t)67ull;
    r0 = buffer[0];
    r1 = buffer[1];
    r2 = r0 + r1;
    if (r2 != 131ull) goto else_1;
    r0 = 1ull;
    goto end_2;
else_1:
    r0 = 0ull;
    // ... (truncated, see source file for more)

```

C (header)

```

drivers/vgpu/vgpu.h (line 22) - [public] vgpu_init
    bool vgpu_init(uint32_t w, uint32_t h);
    vgpu_info_t vgpu_get_info(void);

    void vgpu_clear(uint32_t color);
    void vgpu_pixel(uint32_t x, uint32_t y, uint32_t color);

    int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t *phys_out);
    bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);

    void vgpu_process_ring(volatile void *ring);

#endif

drivers/vgpu/vgpu.h (line 23) - [public] vgpu_get_info
    vgpu_info_t vgpu_get_info(void);

    void vgpu_clear(uint32_t color);
    void vgpu_pixel(uint32_t x, uint32_t y, uint32_t color);

    int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t *phys_out);
    bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);

    void vgpu_process_ring(volatile void *ring);

#endif

drivers/vgpu/vgpu.h (line 25) - [public] vgpu_clear
    void vgpu_clear(uint32_t color);
    void vgpu_pixel(uint32_t x, uint32_t y, uint32_t color);

    int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t *phys_out);
    bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);

    void vgpu_process_ring(volatile void *ring);

#endif

drivers/vgpu/vgpu.h (line 26) - [public] vgpu_pixel
    void vgpu_pixel(uint32_t x, uint32_t y, uint32_t color);

    int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t *phys_out);
    bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);

    void vgpu_process_ring(volatile void *ring);

#endif

drivers/vgpu/vgpu.h (line 28) - [public] vgpu_surface_create
    int32_t vgpu_surface_create(uint32_t w, uint32_t h, uint64_t *phys_out);
    bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);

    void vgpu_process_ring(volatile void *ring);

#endif

drivers/vgpu/vgpu.h (line 29) - [public] vgpu_surface_present
    bool vgpu_surface_present(int32_t id, uint32_t x, uint32_t y);

    void vgpu_process_ring(volatile void *ring);

#endif

drivers/vgpu/vgpu.h (line 31) - [public] vgpu_process_ring
    void vgpu_process_ring(volatile void *ring);

#endif

```

kernel_Workspace/kernel/src/battery/battery.h (line 21) - [public] battery_init

```
bool battery_init(void);
bool battery_present(void);
bool battery_status(battery_status_t *out);
bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);
bool battery_set_threshold(uint32_t low_pct);
uint32_t battery_threshold(void);
bool battery_low_pending(void);
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/battery.h (line 22) - [public] battery_present

```
bool battery_present(void);
bool battery_status(battery_status_t *out);
bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);
bool battery_set_threshold(uint32_t low_pct);
uint32_t battery_threshold(void);
bool battery_low_pending(void);
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/battery.h (line 23) - [public] battery_status

```
bool battery_status(battery_status_t *out);
bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);
bool battery_set_threshold(uint32_t low_pct);
uint32_t battery_threshold(void);
bool battery_low_pending(void);
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/battery.h (line 24) - [public] battery_status_packed

```
bool battery_status_packed(uint64_t *a0, uint64_t *a1, uint64_t *a2);
bool battery_set_threshold(uint32_t low_pct);
uint32_t battery_threshold(void);
bool battery_low_pending(void);
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/battery.h (line 25) - [public] battery_set_threshold

```
bool battery_set_threshold(uint32_t low_pct);
uint32_t battery_threshold(void);
bool battery_low_pending(void);
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/battery.h (line 26) - [public] battery_threshold

```
uint32_t battery_threshold(void);
bool battery_low_pending(void);
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/battery.h (line 27) - [public] battery_low_pending

```
bool battery_low_pending(void);
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/battery.h (line 28) - [public] battery_sim_tick

```
void battery_sim_tick(void);

#endif
```

kernel_Workspace/kernel/src/battery/operation.h (line 7) - [public] bool

```
    bool (*present)(void);
    bool (*status)(battery_status_t *out);
} battery_backend_t;

void battery_register_backend(const battery_backend_t *b);

#endif
```

kernel_Workspace/kernel/src/battery/operation.h (line 8) - [public] bool

```
    bool (*status)(battery_status_t *out);
} battery_backend_t;

void battery_register_backend(const battery_backend_t *b);

#endif
```

kernel_Workspace/kernel/src/battery/operation.h (line 11) - [public] battery_register_backend

```
void battery_register_backend(const battery_backend_t *b);

#endif
```

kernel_Workspace/kernel/src/bluetooth/bt.h (line 8) - [public] bt_init

```
bool bt_init(void);
bool bt_ready(void);
void bt_info(uint8_t *hci_ver, uint8_t *bdaddr);
bool bt_hci_cmd(uint16_t opcode, const uint8_t *params, uint8_t len);
bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len);
bool bt_acl_send(const uint8_t *data, uint16_t len);
bool bt_acl_recv(uint8_t *data, uint16_t cap, uint16_t *len);

#endif
```

kernel_Workspace/kernel/src/bluetooth/bt.h (line 9) - [public] bt_ready

```
bool bt_ready(void);
void bt_info(uint8_t *hci_ver, uint8_t *bdaddr);
bool bt_hci_cmd(uint16_t opcode, const uint8_t *params, uint8_t len);
bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len);
bool bt_acl_send(const uint8_t *data, uint16_t len);
bool bt_acl_recv(uint8_t *data, uint16_t cap, uint16_t *len);

#endif
```

kernel_Workspace/kernel/src/bluetooth/bt.h (line 10) - [public] bt_info

```
void bt_info(uint8_t *hci_ver, uint8_t *bdaddr);
bool bt_hci_cmd(uint16_t opcode, const uint8_t *params, uint8_t len);
bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len);
bool bt_acl_send(const uint8_t *data, uint16_t len);
bool bt_acl_recv(uint8_t *data, uint16_t cap, uint16_t *len);

#endif
```

kernel_Workspace/kernel/src/bluetooth/bt.h (line 11) - [public] bt_hci_cmd

```
bool bt_hci_cmd(uint16_t opcode, const uint8_t *params, uint8_t len);
bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len);
bool bt_acl_send(const uint8_t *data, uint16_t len);
bool bt_acl_recv(uint8_t *data, uint16_t cap, uint16_t *len);

#endif
```

kernel_Workspace/kernel/src/bluetooth/bt.h (line 12) - [public] bt_event_poll

```
bool bt_event_poll(uint8_t *buf, uint8_t cap, uint8_t *len);
bool bt_acl_send(const uint8_t *data, uint16_t len);
bool bt_acl_recv(uint8_t *data, uint16_t cap, uint16_t *len);

#endif
```

```

kernel_Workspace/kernel/src/bluetooth/bt.h (line 13) - [public] bt_acl_send
    bool bt_acl_send(const uint8_t *data, uint16_t len);
    bool bt_acl_rcv(uint8_t *data, uint16_t cap, uint16_t *len);

    #endif

kernel_Workspace/kernel/src/bluetooth/bt.h (line 14) - [public] bt_acl_rcv
    bool bt_acl_rcv(uint8_t *data, uint16_t cap, uint16_t *len);

    #endif

kernel_Workspace/kernel/src/bluetooth/operation.h (line 11) - [public] bt_op_model_init
    void bt_op_model_init(void);
    bool bt_op_model_run_cmd(uint16_t opcode, const uint8_t *params, uint8_t len);

    #endif

kernel_Workspace/kernel/src/bluetooth/operation.h (line 12) - [public] bt_op_model_run_cmd
    bool bt_op_model_run_cmd(uint16_t opcode, const uint8_t *params, uint8_t len);

    #endif

kernel_Workspace/kernel/src/camera/camera.h (line 19) - [public] camera_init
    bool camera_init(void);
    bool camera_present(void);
    bool camera_caps_get(camera_caps_t *out);
    bool camera_start(void);
    bool camera_stop(void);
    bool camera_frame(void *buf, uint32_t cap, uint64_t *frame_id);
    bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

    #endif

kernel_Workspace/kernel/src/camera/camera.h (line 20) - [public] camera_present
    bool camera_present(void);
    bool camera_caps_get(camera_caps_t *out);
    bool camera_start(void);
    bool camera_stop(void);
    bool camera_frame(void *buf, uint32_t cap, uint64_t *frame_id);
    bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

    #endif

kernel_Workspace/kernel/src/camera/camera.h (line 21) - [public] camera_caps_get
    bool camera_caps_get(camera_caps_t *out);
    bool camera_start(void);
    bool camera_stop(void);
    bool camera_frame(void *buf, uint32_t cap, uint64_t *frame_id);
    bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

    #endif

kernel_Workspace/kernel/src/camera/camera.h (line 22) - [public] camera_start
    bool camera_start(void);
    bool camera_stop(void);
    bool camera_frame(void *buf, uint32_t cap, uint64_t *frame_id);
    bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

    #endif

kernel_Workspace/kernel/src/camera/camera.h (line 23) - [public] camera_stop
    bool camera_stop(void);
    bool camera_frame(void *buf, uint32_t cap, uint64_t *frame_id);
    bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

    #endif

```

```

kernel_Workspace/kernel/src/camera/camera.h (line 24) - [public] camera_frame
    bool camera_frame(void *buf, uint32_t cap, uint64_t *frame_id);
    bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

    #endif

kernel_Workspace/kernel/src/camera/camera.h (line 25) - [public] camera_frame_to_phys
    bool camera_frame_to_phys(uint64_t phys, uint32_t cap, uint64_t *frame_id);

    #endif

kernel_Workspace/kernel/src/camera/operation.h (line 7) - [public] bool
    bool (*present)(void);
    bool (*start)(void);
    bool (*stop)(void);
    bool (*frame)(void *buf, uint32_t cap, uint64_t *fid);
} camera_backend_t;

void camera_register_backend(const camera_backend_t *b);

#endif

kernel_Workspace/kernel/src/camera/operation.h (line 8) - [public] bool
    bool (*start)(void);
    bool (*stop)(void);
    bool (*frame)(void *buf, uint32_t cap, uint64_t *fid);
} camera_backend_t;

void camera_register_backend(const camera_backend_t *b);

#endif

kernel_Workspace/kernel/src/camera/operation.h (line 9) - [public] bool
    bool (*stop)(void);
    bool (*frame)(void *buf, uint32_t cap, uint64_t *fid);
} camera_backend_t;

void camera_register_backend(const camera_backend_t *b);

#endif

kernel_Workspace/kernel/src/camera/operation.h (line 10) - [public] bool
    bool (*frame)(void *buf, uint32_t cap, uint64_t *fid);
} camera_backend_t;

void camera_register_backend(const camera_backend_t *b);

#endif

kernel_Workspace/kernel/src/camera/operation.h (line 13) - [public] camera_register_backend
void camera_register_backend(const camera_backend_t *b);

#endif

kernel_Workspace/kernel/src/caps/caps.h (line 10) - [public] caps_self_bits
unsigned int caps_self_bits(void);

unsigned int caps_world_bits(unsigned int world_id);

int caps_has(unsigned int world_id, unsigned char cap_id);

int caps_name(unsigned char cap_id, unsigned char *buf, unsigned int len);

int caps_request(unsigned int target, unsigned char cap_id);

int caps_release(unsigned int world_id, unsigned char cap_id);

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/caps/caps.h (line 12) - [public] caps_world_bits
    unsigned int caps_world_bits(unsigned int world_id);

    int caps_has(unsigned int world_id, unsigned char cap_id);

    int caps_name(unsigned char cap_id, unsigned char *buf, unsigned int len);

    int caps_request(unsigned int target, unsigned char cap_id);

    int caps_release(unsigned int world_id, unsigned char cap_id);

    unsigned int caps_world_count(void);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/caps/caps.h (line 14) - [public] caps_has
    int caps_has(unsigned int world_id, unsigned char cap_id);

    int caps_name(unsigned char cap_id, unsigned char *buf, unsigned int len);

    int caps_request(unsigned int target, unsigned char cap_id);

    int caps_release(unsigned int world_id, unsigned char cap_id);

    unsigned int caps_world_count(void);

    unsigned long long caps_audit_count(void);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/caps/caps.h (line 16) - [public] caps_name
    int caps_name(unsigned char cap_id, unsigned char *buf, unsigned int len);

    int caps_request(unsigned int target, unsigned char cap_id);

    int caps_release(unsigned int world_id, unsigned char cap_id);

    unsigned int caps_world_count(void);

    unsigned long long caps_audit_count(void);

#endif

kernel_Workspace/kernel/src/caps/caps.h (line 18) - [public] caps_request
    int caps_request(unsigned int target, unsigned char cap_id);

    int caps_release(unsigned int world_id, unsigned char cap_id);

    unsigned int caps_world_count(void);

    unsigned long long caps_audit_count(void);

#endif

kernel_Workspace/kernel/src/caps/caps.h (line 20) - [public] caps_release
    int caps_release(unsigned int world_id, unsigned char cap_id);

    unsigned int caps_world_count(void);

    unsigned long long caps_audit_count(void);

#endif

kernel_Workspace/kernel/src/caps/caps.h (line 22) - [public] caps_world_count
    unsigned int caps_world_count(void);

    unsigned long long caps_audit_count(void);

#endif

```



```

kernel_Workspace/kernel/src/caps/caps.h (line 24) - [public] caps_audit_count
    unsigned long long caps_audit_count(void);

    #endif

kernel_Workspace/kernel/src/core-lang/ast.h (line 50) - [public] cl_arena_init
    void cl_arena_init(arena_t *a, uint8_t *buf, size_t cap);
    void *cl_arena_alloc(arena_t *a, size_t n);
    ast_node_t *cl_node(arena_t *ar, uint8_t kind, uint32_t line);

    #endif

kernel_Workspace/kernel/src/core-lang/bridge.h (line 6) - [public] cl_bridge_init
    int cl_bridge_init(cl_vm_t *vm, uint8_t ring);

    int cl_bridge_add(cl_vm_t *vm, const char *name, cl_ext_fn fn);

    #endif

kernel_Workspace/kernel/src/core-lang/bridge.h (line 8) - [public] cl_bridge_add
    int cl_bridge_add(cl_vm_t *vm, const char *name, cl_ext_fn fn);

    #endif

kernel_Workspace/kernel/src/core-lang/lexer.h (line 16) - [public] cl_lexer_init
    void cl_lexer_init(lexer_t *l, const char *src, size_t len);

    bool cl_lex_next(lexer_t *l, token_t *out);

    int cl_lex_all(lexer_t *l, token_t *buf, size_t cap, size_t *count);

    #endif

kernel_Workspace/kernel/src/core-lang/lexer.h (line 18) - [public] cl_lex_next
    bool cl_lex_next(lexer_t *l, token_t *out);

    int cl_lex_all(lexer_t *l, token_t *buf, size_t cap, size_t *count);

    #endif

kernel_Workspace/kernel/src/core-lang/lexer.h (line 20) - [public] cl_lex_all
    int cl_lex_all(lexer_t *l, token_t *buf, size_t cap, size_t *count);

    #endif

kernel_Workspace/kernel/src/core-lang/loader.h (line 15) - [public] cl_save_bc
    size_t cl_save_bc(cl_prog_t *P, uint8_t *buf, size_t cap);
    cl_prog_t *cl_load_bc(const uint8_t *buf, size_t len, arena_t *ar);

    cl_module_t *cl_load_path(const char *path, arena_t *ar, bool try_native);

    #endif

kernel_Workspace/kernel/src/core-lang/native.h (line 9) - [public] uint64_t
    /* signature wygenerowanej funkcji:
    uint64_t fn(uint64_t *slots, uint64_t *globals, cl_ext_fn *externs) */

    typedef uint64_t (*cl_native_fn)(uint64_t *, uint64_t *, cl_ext_fn *);

    /* Zwraca bufor z kodem (RW) albo NULL gdy arch nie wspiera subsetu.
    Wywołujący robi go executable (mprotect / mapowanie WX w kernelu). */
    void *cl_native_compile(cl_prog_t *P);
    size_t cl_native_size(void);

    #endif

kernel_Workspace/kernel/src/core-lang/native.h (line 14) - [public] cl_native_size
    size_t cl_native_size(void);

    #endif

```

kernel_Workspace/kernel/src/core-lang/sema.h (line 16) - [public] cl_sema_run

```
int cl_sema_run(sema_ctx_t *s, ast_node_t *prog);

size_t cl_sema_diag_count(const sema_ctx_t *s);
const cl_diag_t *cl_sema_diag(const sema_ctx_t *s, size_t i);

#endif
```

kernel_Workspace/kernel/src/core-lang/sema.h (line 18) - [public] cl_sema_diag_count

```
size_t cl_sema_diag_count(const sema_ctx_t *s);
const cl_diag_t *cl_sema_diag(const sema_ctx_t *s, size_t i);

#endif
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 44) - [public] cl_vm_init

```
void cl_vm_init(cl_vm_t *vm, cl_prog_t *prog);
int cl_vm_register_extern(cl_vm_t *vm, const char *name, cl_ext_fn fn);
cl_vm_err_t cl_vm_run(cl_vm_t *vm);

int cl_wbytes(int bits);
void cl_mask(uint8_t *r, int w, int bits);
void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 45) - [public] cl_vm_register_extern

```
int cl_vm_register_extern(cl_vm_t *vm, const char *name, cl_ext_fn fn);
cl_vm_err_t cl_vm_run(cl_vm_t *vm);

int cl_wbytes(int bits);
void cl_mask(uint8_t *r, int w, int bits);
void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 46) - [public] cl_vm_run

```
cl_vm_err_t cl_vm_run(cl_vm_t *vm);

int cl_wbytes(int bits);
void cl_mask(uint8_t *r, int w, int bits);
void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
uint8_t *q, uint8_t *m);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 48) - [public] cl_wbytes

```
int cl_wbytes(int bits);
void cl_mask(uint8_t *r, int w, int bits);
void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m);

#endif
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 49) - [public] cl_mask

```
void cl_mask(uint8_t *r, int w, int bits);
void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m);

#endif
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 50) - [public] cl_add

```
void cl_add(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m);

#endif
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 51) - [public] cl_sub

```
void cl_sub(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m);

#endif
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 52) - [public] cl_mul

```
void cl_mul(uint8_t *r, const uint8_t *a, const uint8_t *b, int w);
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m);

#endif
```

kernel_Workspace/kernel/src/core-lang/vm.h (line 53) - [public] cl_cmp

```
int cl_cmp(const uint8_t *a, const uint8_t *b, int w, bool sign);
void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
               uint8_t *q, uint8_t *m);

#endif
```

```

kernel_Workspace/kernel/src/core-lang/vm.h (line 54) - [public] cl_shl
    void cl_shl(uint8_t *r, const uint8_t *a, uint32_t n, int w);
    void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
    void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
                   uint8_t *q, uint8_t *m);

    #endif

kernel_Workspace/kernel/src/core-lang/vm.h (line 55) - [public] cl_shr
    void cl_shr(uint8_t *r, const uint8_t *a, uint32_t n, int w);
    void cl_divmod(const uint8_t *n, const uint8_t *d, int w,
                   uint8_t *q, uint8_t *m);

    #endif

kernel_Workspace/kernel/src/displayport/aux.h (line 17) - [public] dp_aux_sink_init
    void dp_aux_sink_init(void);
    bool dp_aux_read(uint32_t addr, void *buf, uint32_t len);
    bool dp_aux_write(uint32_t addr, const void *buf, uint32_t len);
    bool dp_aux_read_edid(void *buf, uint32_t len);

    #endif

kernel_Workspace/kernel/src/displayport/aux.h (line 18) - [public] dp_aux_read
    bool dp_aux_read(uint32_t addr, void *buf, uint32_t len);
    bool dp_aux_write(uint32_t addr, const void *buf, uint32_t len);
    bool dp_aux_read_edid(void *buf, uint32_t len);

    #endif

kernel_Workspace/kernel/src/displayport/aux.h (line 19) - [public] dp_aux_write
    bool dp_aux_write(uint32_t addr, const void *buf, uint32_t len);
    bool dp_aux_read_edid(void *buf, uint32_t len);

    #endif

kernel_Workspace/kernel/src/displayport/aux.h (line 20) - [public] dp_aux_read_edid
    bool dp_aux_read_edid(void *buf, uint32_t len);

    #endif

kernel_Workspace/kernel/src/displayport/dp.h (line 10) - [public] dp_init
    bool dp_init(void);
    bool dp_ready(void);

    void dp_link_info(uint32_t *rate_mbps, uint32_t *lanes);

    uint32_t dp_mode_count(void);
    bool dp_mode_at(uint32_t i, uint32_t *id, uint32_t *w,
                   uint32_t *h, uint32_t *r);
    bool dp_mode_set_by_id(uint32_t id);

    void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride);
    bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/displayport/dp.h (line 11) - [public] dp_ready
    bool dp_ready(void);

    void dp_link_info(uint32_t *rate_mbps, uint32_t *lanes);

    uint32_t dp_mode_count(void);
    bool dp_mode_at(uint32_t i, uint32_t *id, uint32_t *w,
                   uint32_t *h, uint32_t *r);
    bool dp_mode_set_by_id(uint32_t id);

    void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride);
    bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

    // ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/displayport/dp.h (line 13) - [public] dp_link_info
void dp_link_info(uint32_t *rate_mbps, uint32_t *lanes);

uint32_t dp_mode_count(void);
bool dp_mode_at(uint32_t i, uint32_t *id, uint32_t *w,
                uint32_t *h, uint32_t *r);
bool dp_mode_set_by_id(uint32_t id);

void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride);
bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

uint64_t dp_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                        uint32_t w, uint32_t h);
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/displayport/dp.h (line 15) - [public] dp_mode_count
uint32_t dp_mode_count(void);
bool dp_mode_at(uint32_t i, uint32_t *id, uint32_t *w,
                uint32_t *h, uint32_t *r);
bool dp_mode_set_by_id(uint32_t id);

void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride);
bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

uint64_t dp_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                        uint32_t w, uint32_t h);

#endif
```

```
kernel_Workspace/kernel/src/displayport/dp.h (line 18) - [public] dp_mode_set_by_id
bool dp_mode_set_by_id(uint32_t id);

void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride);
bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

uint64_t dp_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                        uint32_t w, uint32_t h);

#endif
```

```
kernel_Workspace/kernel/src/displayport/dp.h (line 20) - [public] dp_caps
void dp_caps(uint64_t *fb_phys, uint32_t *w, uint32_t *h, uint32_t *stride);
bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

uint64_t dp_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                        uint32_t w, uint32_t h);

#endif
```

```
kernel_Workspace/kernel/src/displayport/dp.h (line 21) - [public] dp_fb_set
bool dp_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);

uint64_t dp_submit_fill(uint32_t color, uint32_t x, uint32_t y,
                        uint32_t w, uint32_t h);

#endif
```

```
kernel_Workspace/kernel/src/displayport/link.h (line 7) - [public] dp_link_train
bool dp_link_train(void);
void dp_link_info(uint32_t *rate_mbps, uint32_t *lanes);

#endif
```

```
kernel_Workspace/kernel/src/displayport/link.h (line 8) - [public] dp_link_info
void dp_link_info(uint32_t *rate_mbps, uint32_t *lanes);

#endif
```

```

kernel_Workspace/kernel/src/displayport/operation.h (line 6) - [public] dp_op_set_fb
    void dp_op_set_fb(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
    void dp_op_state(uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *stride);
    void dp_op_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);

    #endif

kernel_Workspace/kernel/src/displayport/operation.h (line 7) - [public] dp_op_state
    void dp_op_state(uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *stride);
    void dp_op_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);

    #endif

kernel_Workspace/kernel/src/displayport/operation.h (line 8) - [public] dp_op_fill
    void dp_op_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);

    #endif

kernel_Workspace/kernel/src/editor/editor.h (line 21) - [public] editor_run
    int editor_run(const char *path);

    #endif

kernel_Workspace/kernel/src/editor/mouse.h (line 6) - [public] mouse_init
    void mouse_init(void);

    int mouse_poll(int *dx, int *dy, int *dz, int *buttons);

    #endif

kernel_Workspace/kernel/src/editor/mouse.h (line 8) - [public] mouse_poll
    int mouse_poll(int *dx, int *dy, int *dz, int *buttons);

    #endif

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 41) - [public] hdmi_init_with
    bool hdmi_init_with(uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride);
    bool hdmi_ready(void);
    void hdmi_caps(hdmi_caps_t *out);
    bool hdmi_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
    bool hdmi_mode_set_by_id(uint32_t id);
    bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
    uint64_t hdmi_submit(const hdmi_transfer_t *t);
    bool hdmi_poll(uint64_t *out_seq);
    uint32_t hdmi_pending(void);
    uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
    void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 42) - [public] hdmi_ready
    bool hdmi_ready(void);
    void hdmi_caps(hdmi_caps_t *out);
    bool hdmi_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
    bool hdmi_mode_set_by_id(uint32_t id);
    bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
    uint64_t hdmi_submit(const hdmi_transfer_t *t);
    bool hdmi_poll(uint64_t *out_seq);
    uint32_t hdmi_pending(void);
    uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
    void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

    bool hdmi_fb_grant(void);
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 43) - [public] hdmi_caps

```
void hdmi_caps(hdmi_caps_t *out);
bool hdmi_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_mode_set_by_id(uint32_t id);
bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
uint64_t hdmi_submit(const hdmi_transfer_t *t);
bool hdmi_poll(uint64_t *out_seq);
uint32_t hdmi_pending(void);
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 44) - [public] hdmi_fb_set

```
bool hdmi_fb_set(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_mode_set_by_id(uint32_t id);
bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
uint64_t hdmi_submit(const hdmi_transfer_t *t);
bool hdmi_poll(uint64_t *out_seq);
uint32_t hdmi_pending(void);
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 45) - [public] hdmi_mode_set_by_id

```
bool hdmi_mode_set_by_id(uint32_t id);
bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
uint64_t hdmi_submit(const hdmi_transfer_t *t);
bool hdmi_poll(uint64_t *out_seq);
uint32_t hdmi_pending(void);
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 46) - [public] hdmi_mode_current_raw

```
bool hdmi_mode_current_raw(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
uint64_t hdmi_submit(const hdmi_transfer_t *t);
bool hdmi_poll(uint64_t *out_seq);
uint32_t hdmi_pending(void);
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 47) - [public] hdmi_submit

```
uint64_t hdmi_submit(const hdmi_transfer_t *t);
bool hdmi_poll(uint64_t *out_seq);
uint32_t hdmi_pending(void);
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 48) - [public] hdmi_poll

```
bool hdmi_poll(uint64_t *out_seq);
uint32_t hdmi_pending(void);
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 49) - [public] hdmi_pending

```
uint32_t hdmi_pending(void);
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 50) - [public] hdmi_submit_fill

```
uint64_t hdmi_submit_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 51) - [public] hdmi_caps_raw

```
void hdmi_caps_raw(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 53) - [public] hdmi_fb_grant

```
bool hdmi_fb_grant(void);
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 54) - [public] hdmi_fb_revoke

```
void hdmi_fb_revoke(void);
bool hdmi_fb_granted(void);

#endif
```

kernel_Workspace/kernel/src/hdmi/hdmi.h (line 55) - [public] hdmi_fb_granted

```
bool hdmi_fb_granted(void);

#endif
```


kernel_Workspace/kernel/src/hdmi/interface.h (line 9) - [public] hdmi_iface_acquire

```
bool hdmi_iface_acquire(uint32_t owner);
bool hdmi_iface_release(uint32_t owner);
uint32_t hdmi_iface_owner(void);
bool hdmi_iface_check(uint32_t owner);

bool hdmi_iface_init(uint32_t owner, uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_iface_ready(void);

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 10) - [public] hdmi_iface_release

```
bool hdmi_iface_release(uint32_t owner);
uint32_t hdmi_iface_owner(void);
bool hdmi_iface_check(uint32_t owner);

bool hdmi_iface_init(uint32_t owner, uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_iface_ready(void);

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 11) - [public] hdmi_iface_owner

```
uint32_t hdmi_iface_owner(void);
bool hdmi_iface_check(uint32_t owner);

bool hdmi_iface_init(uint32_t owner, uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_iface_ready(void);

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 12) - [public] hdmi_iface_check

```
bool hdmi_iface_check(uint32_t owner);

bool hdmi_iface_init(uint32_t owner, uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_iface_ready(void);

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 14) - [public] hdmi_iface_init

```
bool hdmi_iface_init(uint32_t owner, uint64_t fb_phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_iface_ready(void);

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h, uint32_t *out_seq);
bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 15) - [public] hdmi_iface_ready

```
bool hdmi_iface_ready(void);

bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h, uint32_t *out_seq);
bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 17) - [public] hdmi_iface_mode_set

```
bool hdmi_iface_mode_set(uint32_t owner, uint32_t id);
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h, uint32_t *out_seq);
bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
bool hdmi_iface_fb_revoke(uint32_t owner);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 18) - [public] hdmi_iface_mode_current

```
bool hdmi_iface_mode_current(uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h, uint32_t *out_seq);
bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
bool hdmi_iface_fb_revoke(uint32_t owner);

#endif
```

kernel_Workspace/kernel/src/hdmi/interface.h (line 19) - [public] hdmi_iface_mode_at

```
bool hdmi_iface_mode_at(uint32_t i, uint32_t *id, uint32_t *w, uint32_t *h, uint32_t *r);

uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h, uint32_t *out_seq);
bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
bool hdmi_iface_fb_revoke(uint32_t owner);

#endif
```

```

kernel_Workspace/kernel/src/hdmi/interface.h (line 21) - [public] hdmi_iface_submit_fill
    uint64_t hdmi_iface_submit_fill(uint32_t owner, uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h,
    bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

    void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

    bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
    bool hdmi_iface_fb_revoke(uint32_t owner);

#endif

kernel_Workspace/kernel/src/hdmi/interface.h (line 22) - [public] hdmi_iface_poll
    bool hdmi_iface_poll(uint32_t owner, uint64_t *out_seq);

    void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

    bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
    bool hdmi_iface_fb_revoke(uint32_t owner);

#endif

kernel_Workspace/kernel/src/hdmi/interface.h (line 24) - [public] hdmi_iface_caps
    void hdmi_iface_caps(uint32_t *w, uint32_t *h, uint32_t *s, uint64_t *phys);

    bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
    bool hdmi_iface_fb_revoke(uint32_t owner);

#endif

kernel_Workspace/kernel/src/hdmi/interface.h (line 26) - [public] hdmi_iface_fb_grant
    bool hdmi_iface_fb_grant(uint32_t owner, uint64_t *phys, uint32_t *w, uint32_t *h, uint32_t *s);
    bool hdmi_iface_fb_revoke(uint32_t owner);

#endif

kernel_Workspace/kernel/src/hdmi/interface.h (line 27) - [public] hdmi_iface_fb_revoke
    bool hdmi_iface_fb_revoke(uint32_t owner);

#endif

kernel_Workspace/kernel/src/hdmi/mode.h (line 23) - [public] hdmi_mode_count
    uint32_t hdmi_mode_count(void);
    const hdmi_mode_t *hdmi_mode_at(uint32_t i);
    const hdmi_mode_t *hdmi_mode_find(uint32_t w, uint32_t h, uint32_t refresh);
    const hdmi_mode_t *hdmi_mode_by_id(uint32_t id);
    bool hdmi_mode_valid(const hdmi_mode_t *m);
    bool hdmi_mode_apply(const hdmi_mode_t *m);
    const hdmi_mode_t *hdmi_mode_current(void);
    bool hdmi_mode_at_raw(uint32_t i, uint32_t *id, uint32_t *w,
        uint32_t *h, uint32_t *r);

#endif

kernel_Workspace/kernel/src/hdmi/mode.h (line 27) - [public] hdmi_mode_valid
    bool hdmi_mode_valid(const hdmi_mode_t *m);
    bool hdmi_mode_apply(const hdmi_mode_t *m);
    const hdmi_mode_t *hdmi_mode_current(void);
    bool hdmi_mode_at_raw(uint32_t i, uint32_t *id, uint32_t *w,
        uint32_t *h, uint32_t *r);

#endif

kernel_Workspace/kernel/src/hdmi/mode.h (line 28) - [public] hdmi_mode_apply
    bool hdmi_mode_apply(const hdmi_mode_t *m);
    const hdmi_mode_t *hdmi_mode_current(void);
    bool hdmi_mode_at_raw(uint32_t i, uint32_t *id, uint32_t *w,
        uint32_t *h, uint32_t *r);

#endif

```

kernel_Workspace/kernel/src/hdmi/operation.h (line 7) - [public] hdmi_op_set_mode

```
void hdmi_op_set_mode(const hdmi_mode_t *m);
void hdmi_op_exec(hdmi_transfer_t *t);
void hdmi_op_state(hdmi_caps_t *out);
void hdmi_op_set_fb(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_op_ready(void);
```

#endif

kernel_Workspace/kernel/src/hdmi/operation.h (line 8) - [public] hdmi_op_exec

```
void hdmi_op_exec(hdmi_transfer_t *t);
void hdmi_op_state(hdmi_caps_t *out);
void hdmi_op_set_fb(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_op_ready(void);
```

#endif

kernel_Workspace/kernel/src/hdmi/operation.h (line 9) - [public] hdmi_op_state

```
void hdmi_op_state(hdmi_caps_t *out);
void hdmi_op_set_fb(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_op_ready(void);
```

#endif

kernel_Workspace/kernel/src/hdmi/operation.h (line 10) - [public] hdmi_op_set_fb

```
void hdmi_op_set_fb(uint64_t phys, uint32_t w, uint32_t h, uint32_t stride);
bool hdmi_op_ready(void);
```

#endif

kernel_Workspace/kernel/src/hdmi/operation.h (line 11) - [public] hdmi_op_ready

```
bool hdmi_op_ready(void);
```

#endif

kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 13) - [public] kfree

```
void kfree(void *ptr);

size_t kmalloc_usable_size(void *ptr);

void *kalloc_pages(size_t pages);
void kfree_pages(void *ptr, size_t pages);

char *kstrdup(const char *s);

uint64_t kvirt_to_phys(void *ptr);

void kalloc_dump(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 15) - [public] kmalloc_usable_size

```
size_t kmalloc_usable_size(void *ptr);

void *kalloc_pages(size_t pages);
void kfree_pages(void *ptr, size_t pages);

char *kstrdup(const char *s);

uint64_t kvirt_to_phys(void *ptr);

void kalloc_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 18) - [public] kfree_pages
void kfree_pages(void *ptr, size_t pages);

char *kstrdup(const char *s);

uint64_t kvirt_to_phys(void *ptr);

void kalloc_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 22) - [public] kvirt_to_phys
uint64_t kvirt_to_phys(void *ptr);

void kalloc_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/api/alloc.h (line 24) - [public] kalloc_dump
void kalloc_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.h (line 9) - [public] dbg_free
void dbg_free(void *ptr);

bool dbg_verify(void *ptr);
size_t dbg_usable_size(void *ptr);
void mm_debug_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.h (line 11) - [public] dbg_verify
bool dbg_verify(void *ptr);
size_t dbg_usable_size(void *ptr);
void mm_debug_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.h (line 12) - [public] dbg_usable_size
size_t dbg_usable_size(void *ptr);
void mm_debug_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/alloc_debug.h (line 13) - [public] mm_debug_dump
void mm_debug_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 8) - [public] leak_track
bool leak_track(void *ptr, size_t size, uint64_t caller);

bool leak_track_id(void *ptr, size_t size, uint64_t caller, uint64_t *out_id);

bool leak_untrack(void *ptr, size_t *out_size, uint64_t *out_caller);
bool leak_contains(void *ptr);
size_t leak_count(void);

void leak_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 10) - [public] leak_track_id
    bool leak_track_id(void *ptr, size_t size, uint64_t caller, uint64_t *out_id);

    bool leak_untrack(void *ptr, size_t *out_size, uint64_t *out_caller);
    bool leak_contains(void *ptr);
    size_t leak_count(void);

    void leak_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 12) - [public] leak_untrack
    bool leak_untrack(void *ptr, size_t *out_size, uint64_t *out_caller);
    bool leak_contains(void *ptr);
    size_t leak_count(void);

    void leak_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 13) - [public] leak_contains
    bool leak_contains(void *ptr);
    size_t leak_count(void);

    void leak_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 14) - [public] leak_count
    size_t leak_count(void);

    void leak_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/leak.h (line 16) - [public] leak_dump
    void leak_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 8) - [public] alloc_stats_note_alloc
    void alloc_stats_note_alloc(size_t size);
    void alloc_stats_note_free(size_t size);

    size_t alloc_stats_live_count(void);
    size_t alloc_stats_live_bytes(void);
    size_t alloc_stats_peak_bytes(void);
    size_t alloc_stats_total_allocs(void);
    size_t alloc_stats_total_frees(void);

    void alloc_stats_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 9) - [public] alloc_stats_note_free
    void alloc_stats_note_free(size_t size);

    size_t alloc_stats_live_count(void);
    size_t alloc_stats_live_bytes(void);
    size_t alloc_stats_peak_bytes(void);
    size_t alloc_stats_total_allocs(void);
    size_t alloc_stats_total_frees(void);

    void alloc_stats_dump(void);

#endif
```

```

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 11) - [public] alloc_stats_live_count
    size_t alloc_stats_live_count(void);
    size_t alloc_stats_live_bytes(void);
    size_t alloc_stats_peak_bytes(void);
    size_t alloc_stats_total_allocs(void);
    size_t alloc_stats_total_frees(void);

    void alloc_stats_dump(void);

#endif

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 12) - [public] alloc_stats_live_bytes
    size_t alloc_stats_live_bytes(void);
    size_t alloc_stats_peak_bytes(void);
    size_t alloc_stats_total_allocs(void);
    size_t alloc_stats_total_frees(void);

    void alloc_stats_dump(void);

#endif

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 13) - [public] alloc_stats_peak_bytes
    size_t alloc_stats_peak_bytes(void);
    size_t alloc_stats_total_allocs(void);
    size_t alloc_stats_total_frees(void);

    void alloc_stats_dump(void);

#endif

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 14) - [public]
alloc_stats_total_allocs
    size_t alloc_stats_total_allocs(void);
    size_t alloc_stats_total_frees(void);

    void alloc_stats_dump(void);

#endif

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 15) - [public]
alloc_stats_total_frees
    size_t alloc_stats_total_frees(void);

    void alloc_stats_dump(void);

#endif

kernel_Workspace/kernel/src/mm/alloc/debug/stats.h (line 17) - [public] alloc_stats_dump
    void alloc_stats_dump(void);

#endif

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 8) - [public] bool
typedef bool (*buddy_map_cb)(uint64_t virt, size_t size);
typedef void (*buddy_unmap_cb)(uint64_t virt, size_t size);

bool buddy_init(uint64_t base,
                size_t size,
                buddy_map_cb map_cb,
                buddy_unmap_cb unmap_cb);

bool buddy_ready(void);

void *buddy_alloc(size_t size);
void *buddy_alloc_aligned(size_t size, size_t align);
// ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 9) - [public] void

```
typedef void (*buddy_unmap_cb)(uint64_t virt, size_t size);
```

```
bool buddy_init(uint64_t base,  
                size_t size,  
                buddy_map_cb map_cb,  
                buddy_unmap_cb unmap_cb);
```

```
bool buddy_ready(void);
```

```
void *buddy_alloc(size_t size);  
void *buddy_alloc_aligned(size_t size, size_t align);  
void buddy_free(void *ptr);  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 16) - [public] buddy_ready

```
bool buddy_ready(void);
```

```
void *buddy_alloc(size_t size);  
void *buddy_alloc_aligned(size_t size, size_t align);  
void buddy_free(void *ptr);
```

```
size_t buddy_block_size(void *ptr);
```

```
size_t buddy_stat_used_bytes(void);  
size_t buddy_stat_free_bytes(void);
```

```
void buddy_dump(void);  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 20) - [public] buddy_free

```
void buddy_free(void *ptr);
```

```
size_t buddy_block_size(void *ptr);
```

```
size_t buddy_stat_used_bytes(void);  
size_t buddy_stat_free_bytes(void);
```

```
void buddy_dump(void);
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 22) - [public] buddy_block_size

```
size_t buddy_block_size(void *ptr);
```

```
size_t buddy_stat_used_bytes(void);  
size_t buddy_stat_free_bytes(void);
```

```
void buddy_dump(void);
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 24) - [public] buddy_stat_used_bytes

```
size_t buddy_stat_used_bytes(void);  
size_t buddy_stat_free_bytes(void);
```

```
void buddy_dump(void);
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 25) - [public] buddy_stat_free_bytes

```
size_t buddy_stat_free_bytes(void);
```

```
void buddy_dump(void);
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/buddy.h (line 27) - [public] buddy_dump

```
void buddy_dump(void);
```

```
#endif
```


kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 8) - [public] heap_init

```
bool heap_init(void);
bool heap_ready(void);

void *heap_alloc(size_t size);
void *heap_alloc_aligned(size_t size, size_t align);
void *heap_zalloc(size_t size);

void heap_free(void *ptr);

size_t heap_usable_size(void *ptr);

void heap_dump(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 9) - [public] heap_ready

```
bool heap_ready(void);

void *heap_alloc(size_t size);
void *heap_alloc_aligned(size_t size, size_t align);
void *heap_zalloc(size_t size);

void heap_free(void *ptr);

size_t heap_usable_size(void *ptr);

void heap_dump(void);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 15) - [public] heap_free

```
void heap_free(void *ptr);

size_t heap_usable_size(void *ptr);

void heap_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 17) - [public] heap_usable_size

```
size_t heap_usable_size(void *ptr);

void heap_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/heap.h (line 19) - [public] heap_dump

```
void heap_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 10) - [public] slab_init

```
bool slab_init(uint64_t base, size_t size);
bool slab_ready(void);

void *slab_alloc(size_t size);
void slab_free(void *ptr);

size_t slab_usable_size(void *ptr);

size_t slab_stat_used_bytes(void);
size_t slab_stat_free_bytes(void);

void slab_dump(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 11) - [public] slab_ready

```
bool slab_ready(void);

void *slab_alloc(size_t size);
void slab_free(void *ptr);

size_t slab_usable_size(void *ptr);

size_t slab_stat_used_bytes(void);
size_t slab_stat_free_bytes(void);

void slab_dump(void);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 14) - [public] slab_free

```
void slab_free(void *ptr);

size_t slab_usable_size(void *ptr);

size_t slab_stat_used_bytes(void);
size_t slab_stat_free_bytes(void);

void slab_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 16) - [public] slab_usable_size

```
size_t slab_usable_size(void *ptr);

size_t slab_stat_used_bytes(void);
size_t slab_stat_free_bytes(void);

void slab_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 18) - [public] slab_stat_used_bytes

```
size_t slab_stat_used_bytes(void);
size_t slab_stat_free_bytes(void);

void slab_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 19) - [public] slab_stat_free_bytes

```
size_t slab_stat_free_bytes(void);

void slab_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/heap/slab.h (line 21) - [public] slab_dump

```
void slab_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 30) - [public]

bitmap_words_for_bits

```
size_t bitmap_words_for_bits(size_t bit_count);
size_t bitmap_bytes_for_bits(size_t bit_count);

void bitmap_init_virt(bitmap_t *bm, void *storage, size_t bit_count);

bool bitmap_init_phys(bitmap_t *bm, uint64_t storage_phys, size_t bit_count);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 31) - [public]

bitmap_bytes_for_bits

```
size_t bitmap_bytes_for_bits(size_t bit_count);

void bitmap_init_virt(bitmap_t *bm, void *storage, size_t bit_count);

bool bitmap_init_phys(bitmap_t *bm, uint64_t storage_phys, size_t bit_count);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 33) - [public] bitmap_init_virt

```
void bitmap_init_virt(bitmap_t *bm, void *storage, size_t bit_count);

bool bitmap_init_phys(bitmap_t *bm, uint64_t storage_phys, size_t bit_count);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_fill(bitmap_t *bm, bool value);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 35) - [public] bitmap_init_phys

```
bool bitmap_init_phys(bitmap_t *bm, uint64_t storage_phys, size_t bit_count);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 37) - [public] bitmap_fill

```
void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 39) - [public] bitmap_set

```
void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 40) - [public] bitmap_clear

```
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 41) - [public] bitmap_test

```
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_fill(bitmap_t *bm, bool value);

void bitmap_set(bitmap_t *bm, size_t bit);
void bitmap_clear(bitmap_t *bm, size_t bit);
bool bitmap_test(const bitmap_t *bm, size_t bit);

void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 43) - [public] bitmap_fill
    void bitmap_fill(bitmap_t *bm, bool value);

    void bitmap_set(bitmap_t *bm, size_t bit);
    void bitmap_clear(bitmap_t *bm, size_t bit);
    bool bitmap_test(const bitmap_t *bm, size_t bit);

    void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
    void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

    bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_alloc(bitmap_t *bm);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 45) - [public] bitmap_set
    void bitmap_set(bitmap_t *bm, size_t bit);
    void bitmap_clear(bitmap_t *bm, size_t bit);
    bool bitmap_test(const bitmap_t *bm, size_t bit);

    void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
    void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

    bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_alloc(bitmap_t *bm);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 46) - [public] bitmap_clear
    void bitmap_clear(bitmap_t *bm, size_t bit);
    bool bitmap_test(const bitmap_t *bm, size_t bit);

    void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
    void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

    bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_alloc(bitmap_t *bm);

    size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 47) - [public] bitmap_test
    bool bitmap_test(const bitmap_t *bm, size_t bit);

    void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
    void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

    bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_alloc(bitmap_t *bm);

    size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 49) - [public] bitmap_set_range
    void bitmap_set_range(bitmap_t *bm, size_t start, size_t count);
    void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

    bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_alloc(bitmap_t *bm);

    size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);

    size_t bitmap_alloc_range(bitmap_t *bm, size_t count, size_t align);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 50) - [public] bitmap_clear_range
    void bitmap_clear_range(bitmap_t *bm, size_t start, size_t count);

    bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_alloc(bitmap_t *bm);

    size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);

    size_t bitmap_alloc_range(bitmap_t *bm, size_t count, size_t align);

    void bitmap_free(bitmap_t *bm, size_t bit);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 52) - [public]
bitmap_test_range_free
    bool bitmap_test_range_free(const bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_alloc(bitmap_t *bm);

    size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);

    size_t bitmap_alloc_range(bitmap_t *bm, size_t count, size_t align);

    void bitmap_free(bitmap_t *bm, size_t bit);
    void bitmap_free_range(bitmap_t *bm, size_t start, size_t count);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 54) - [public] bitmap_alloc
    size_t bitmap_alloc(bitmap_t *bm);

    size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);

    size_t bitmap_alloc_range(bitmap_t *bm, size_t count, size_t align);

    void bitmap_free(bitmap_t *bm, size_t bit);
    void bitmap_free_range(bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_count_free(const bitmap_t *bm);

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 57) - [public] bitmap_alloc_from
    size_t bitmap_alloc_from(bitmap_t *bm, size_t min_bit);

    size_t bitmap_alloc_range(bitmap_t *bm, size_t count, size_t align);

    void bitmap_free(bitmap_t *bm, size_t bit);
    void bitmap_free_range(bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_count_free(const bitmap_t *bm);

#endif

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 59) - [public] bitmap_alloc_range
    size_t bitmap_alloc_range(bitmap_t *bm, size_t count, size_t align);

    void bitmap_free(bitmap_t *bm, size_t bit);
    void bitmap_free_range(bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_count_free(const bitmap_t *bm);

#endif

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 61) - [public] bitmap_free
    void bitmap_free(bitmap_t *bm, size_t bit);
    void bitmap_free_range(bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_count_free(const bitmap_t *bm);

#endif

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 62) - [public] bitmap_free_range
    void bitmap_free_range(bitmap_t *bm, size_t start, size_t count);

    size_t bitmap_count_free(const bitmap_t *bm);

#endif

kernel_Workspace/kernel/src/mm/alloc/physical/bitmap.h (line 64) - [public] bitmap_count_free
    size_t bitmap_count_free(const bitmap_t *bm);

#endif

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 12) - [public] frame_init
    bool frame_init(uint64_t bitmap_phys, size_t bit_count);

    bool frame_init_from_memory(void);

    bool frame_ready(void);

    bool frame_alloc(frame_t *out);
    bool frame_alloc_zero(frame_t *out);

    bool frame_alloc_contiguous(size_t count,
                                size_t align_frames,
                                frame_t *out);
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 14) - [public]

frame_init_from_memory

```
bool frame_init_from_memory(void);

bool frame_ready(void);

bool frame_alloc(frame_t *out);
bool frame_alloc_zero(frame_t *out);

bool frame_alloc_contiguous(size_t count,
                             size_t align_frames,
                             frame_t *out);
bool frame_alloc_below(size_t count,
                        size_t align_frames,
                        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 16) - [public] frame_ready

```
bool frame_ready(void);

bool frame_alloc(frame_t *out);
bool frame_alloc_zero(frame_t *out);

bool frame_alloc_contiguous(size_t count,
                             size_t align_frames,
                             frame_t *out);
bool frame_alloc_below(size_t count,
                        size_t align_frames,
                        uint64_t max_phys,
                        frame_t *out);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 18) - [public] frame_alloc

```
bool frame_alloc(frame_t *out);
bool frame_alloc_zero(frame_t *out);

bool frame_alloc_contiguous(size_t count,
                             size_t align_frames,
                             frame_t *out);
bool frame_alloc_below(size_t count,
                        size_t align_frames,
                        uint64_t max_phys,
                        frame_t *out);

bool frame_free(frame_t frame);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 19) - [public] frame_alloc_zero

```
bool frame_alloc_zero(frame_t *out);

bool frame_alloc_contiguous(size_t count,
                             size_t align_frames,
                             frame_t *out);
bool frame_alloc_below(size_t count,
                        size_t align_frames,
                        uint64_t max_phys,
                        frame_t *out);

bool frame_free(frame_t frame);
bool frame_free_contiguous(frame_t start, size_t count);
// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 29) - [public] frame_free

```
bool frame_free(frame_t frame);
bool frame_free_contiguous(frame_t start, size_t count);

bool frame_is_valid(frame_t frame);
bool frame_zero(frame_t frame);

void *frame_virt(frame_t frame);
uint64_t frame_phys(frame_t frame);

size_t frame_to_pfn(frame_t frame);
frame_t frame_from_pfn(size_t pfn);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 30) - [public]
frame_free_contiguous

```
bool frame_free_contiguous(frame_t start, size_t count);

bool frame_is_valid(frame_t frame);
bool frame_zero(frame_t frame);

void *frame_virt(frame_t frame);
uint64_t frame_phys(frame_t frame);

size_t frame_to_pfn(frame_t frame);
frame_t frame_from_pfn(size_t pfn);

size_t frame_total(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 32) - [public] frame_is_valid

```
bool frame_is_valid(frame_t frame);
bool frame_zero(frame_t frame);

void *frame_virt(frame_t frame);
uint64_t frame_phys(frame_t frame);

size_t frame_to_pfn(frame_t frame);
frame_t frame_from_pfn(size_t pfn);

size_t frame_total(void);
size_t frame_allocated(void);
size_t frame_free_count(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 33) - [public] frame_zero

```
bool frame_zero(frame_t frame);

void *frame_virt(frame_t frame);
uint64_t frame_phys(frame_t frame);

size_t frame_to_pfn(frame_t frame);
frame_t frame_from_pfn(size_t pfn);

size_t frame_total(void);
size_t frame_allocated(void);
size_t frame_free_count(void);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 36) - [public] frame_phys

```
uint64_t frame_phys(frame_t frame);

size_t frame_to_pfn(frame_t frame);
frame_t frame_from_pfn(size_t pfn);

size_t frame_total(void);
size_t frame_allocated(void);
size_t frame_free_count(void);
```

```
size_t frame_zone_dma32_total(void);
size_t frame_zone_dma32_free(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 38) - [public] frame_to_pfn

```
size_t frame_to_pfn(frame_t frame);
frame_t frame_from_pfn(size_t pfn);

size_t frame_total(void);
size_t frame_allocated(void);
size_t frame_free_count(void);
```

```
size_t frame_zone_dma32_total(void);
size_t frame_zone_dma32_free(void);
size_t frame_zone_normal_total(void);
size_t frame_zone_normal_free(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 39) - [public] frame_from_pfn

```
frame_t frame_from_pfn(size_t pfn);

size_t frame_total(void);
size_t frame_allocated(void);
size_t frame_free_count(void);
```

```
size_t frame_zone_dma32_total(void);
size_t frame_zone_dma32_free(void);
size_t frame_zone_normal_total(void);
size_t frame_zone_normal_free(void);
```

```
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 41) - [public] frame_total

```
size_t frame_total(void);
size_t frame_allocated(void);
size_t frame_free_count(void);

size_t frame_zone_dma32_total(void);
size_t frame_zone_dma32_free(void);
size_t frame_zone_normal_total(void);
size_t frame_zone_normal_free(void);
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 42) - [public] frame_allocated

```
size_t frame_allocated(void);
size_t frame_free_count(void);

size_t frame_zone_dma32_total(void);
size_t frame_zone_dma32_free(void);
size_t frame_zone_normal_total(void);
size_t frame_zone_normal_free(void);
```

```
#endif
```

```

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 43) - [public] frame_free_count
    size_t frame_free_count(void);

    size_t frame_zone_dma32_total(void);
    size_t frame_zone_dma32_free(void);
    size_t frame_zone_normal_total(void);
    size_t frame_zone_normal_free(void);

    #endif

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 46) - [public]
frame_zone_dma32_total
    size_t frame_zone_dma32_total(void);
    size_t frame_zone_dma32_free(void);
    size_t frame_zone_normal_total(void);
    size_t frame_zone_normal_free(void);

    #endif

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 47) - [public]
frame_zone_dma32_free
    size_t frame_zone_dma32_free(void);
    size_t frame_zone_normal_total(void);
    size_t frame_zone_normal_free(void);

    #endif

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 48) - [public]
frame_zone_normal_total
    size_t frame_zone_normal_total(void);
    size_t frame_zone_normal_free(void);

    #endif

kernel_Workspace/kernel/src/mm/alloc/physical/frame.h (line 49) - [public]
frame_zone_normal_free
    size_t frame_zone_normal_free(void);

    #endif

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 10) - [public] pmm_init
    bool pmm_init(void);

    bool pmm_ready(void);

    bool pmm_alloc_frame(uint64_t *out_phys);

    bool pmm_alloc_zero_frame(uint64_t *out_phys);

    bool pmm_alloc_frames(size_t count, uint64_t *out_phys);

    bool pmm_alloc_frames_aligned(size_t count,
                                   size_t align_frames,
                                   // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 12) - [public] pmm_ready
    bool pmm_ready(void);

    bool pmm_alloc_frame(uint64_t *out_phys);

    bool pmm_alloc_zero_frame(uint64_t *out_phys);

    bool pmm_alloc_frames(size_t count, uint64_t *out_phys);

    bool pmm_alloc_frames_aligned(size_t count,
                                   size_t align_frames,
                                   uint64_t *out_phys);
    bool pmm_alloc_bytes(size_t bytes, uint64_t *out_phys);
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 14) - [public] pmm_alloc_frame

```
bool pmm_alloc_frame(uint64_t *out_phys);

bool pmm_alloc_zero_frame(uint64_t *out_phys);

bool pmm_alloc_frames(size_t count, uint64_t *out_phys);

bool pmm_alloc_frames_aligned(size_t count,
                              size_t align_frames,
                              uint64_t *out_phys);
bool pmm_alloc_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_zero_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_contiguous_bytes(size_t bytes,
                                // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 16) - [public] pmm_alloc_zero_frame

```
bool pmm_alloc_zero_frame(uint64_t *out_phys);

bool pmm_alloc_frames(size_t count, uint64_t *out_phys);

bool pmm_alloc_frames_aligned(size_t count,
                              size_t align_frames,
                              uint64_t *out_phys);
bool pmm_alloc_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_zero_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_contiguous_bytes(size_t bytes,
                                size_t align_bytes,
                                uint64_t *out_phys);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 18) - [public] pmm_alloc_frames

```
bool pmm_alloc_frames(size_t count, uint64_t *out_phys);

bool pmm_alloc_frames_aligned(size_t count,
                              size_t align_frames,
                              uint64_t *out_phys);
bool pmm_alloc_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_zero_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_contiguous_bytes(size_t bytes,
                                size_t align_bytes,
                                uint64_t *out_phys);

bool pmm_free_frame(uint64_t phys);
bool pmm_alloc_frames_below(size_t count,
                            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 23) - [public] pmm_alloc_bytes

```
bool pmm_alloc_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_zero_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_contiguous_bytes(size_t bytes,
                                size_t align_bytes,
                                uint64_t *out_phys);

bool pmm_free_frame(uint64_t phys);
bool pmm_alloc_frames_below(size_t count,
                            size_t align_frames,
                            uint64_t max_phys,
                            uint64_t *out_phys);
bool pmm_free_frames(uint64_t phys, size_t count);
bool pmm_free_bytes(uint64_t phys, size_t bytes);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 24) - [public] pmm_alloc_zero_bytes

```
bool pmm_alloc_zero_bytes(size_t bytes, uint64_t *out_phys);
bool pmm_alloc_contiguous_bytes(size_t bytes,
                                size_t align_bytes,
                                uint64_t *out_phys);
bool pmm_free_frame(uint64_t phys);
bool pmm_alloc_frames_below(size_t count,
                            size_t align_frames,
                            uint64_t max_phys,
                            uint64_t *out_phys);
bool pmm_free_frames(uint64_t phys, size_t count);
bool pmm_free_bytes(uint64_t phys, size_t bytes);
size_t pmm_stat_total_frames(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 28) - [public] pmm_free_frame

```
bool pmm_free_frame(uint64_t phys);
bool pmm_alloc_frames_below(size_t count,
                            size_t align_frames,
                            uint64_t max_phys,
                            uint64_t *out_phys);
bool pmm_free_frames(uint64_t phys, size_t count);
bool pmm_free_bytes(uint64_t phys, size_t bytes);
size_t pmm_stat_total_frames(void);
size_t pmm_stat_free_frames(void);
size_t pmm_stat_allocated_frames(void);

uint64_t pmm_stat_total_bytes(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 33) - [public] pmm_free_frames

```
bool pmm_free_frames(uint64_t phys, size_t count);
bool pmm_free_bytes(uint64_t phys, size_t bytes);
size_t pmm_stat_total_frames(void);
size_t pmm_stat_free_frames(void);
size_t pmm_stat_allocated_frames(void);

uint64_t pmm_stat_total_bytes(void);
uint64_t pmm_stat_free_bytes(void);

size_t pmm_stat_dma32_total_frames(void);
size_t pmm_stat_dma32_free_frames(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 34) - [public] pmm_free_bytes

```
bool pmm_free_bytes(uint64_t phys, size_t bytes);
size_t pmm_stat_total_frames(void);
size_t pmm_stat_free_frames(void);
size_t pmm_stat_allocated_frames(void);

uint64_t pmm_stat_total_bytes(void);
uint64_t pmm_stat_free_bytes(void);

size_t pmm_stat_dma32_total_frames(void);
size_t pmm_stat_dma32_free_frames(void);
size_t pmm_stat_normal_total_frames(void);
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 35) - [public] pmm_stat_total_frames
    size_t pmm_stat_total_frames(void);
    size_t pmm_stat_free_frames(void);
    size_t pmm_stat_allocated_frames(void);

    uint64_t pmm_stat_total_bytes(void);
    uint64_t pmm_stat_free_bytes(void);

    size_t pmm_stat_dma32_total_frames(void);
    size_t pmm_stat_dma32_free_frames(void);
    size_t pmm_stat_normal_total_frames(void);
    size_t pmm_stat_normal_free_frames(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 36) - [public] pmm_stat_free_frames
    size_t pmm_stat_free_frames(void);
    size_t pmm_stat_allocated_frames(void);

    uint64_t pmm_stat_total_bytes(void);
    uint64_t pmm_stat_free_bytes(void);

    size_t pmm_stat_dma32_total_frames(void);
    size_t pmm_stat_dma32_free_frames(void);
    size_t pmm_stat_normal_total_frames(void);
    size_t pmm_stat_normal_free_frames(void);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 37) - [public]
pmm_stat_allocated_frames
    size_t pmm_stat_allocated_frames(void);

    uint64_t pmm_stat_total_bytes(void);
    uint64_t pmm_stat_free_bytes(void);

    size_t pmm_stat_dma32_total_frames(void);
    size_t pmm_stat_dma32_free_frames(void);
    size_t pmm_stat_normal_total_frames(void);
    size_t pmm_stat_normal_free_frames(void);

    void pmm_dump(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 39) - [public] pmm_stat_total_bytes
    uint64_t pmm_stat_total_bytes(void);
    uint64_t pmm_stat_free_bytes(void);

    size_t pmm_stat_dma32_total_frames(void);
    size_t pmm_stat_dma32_free_frames(void);
    size_t pmm_stat_normal_total_frames(void);
    size_t pmm_stat_normal_free_frames(void);

    void pmm_dump(void);

#ifdef PMM_DEBUG
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 40) - [public] pmm_stat_free_bytes

```
uint64_t pmm_stat_free_bytes(void);
```

```
size_t pmm_stat_dma32_total_frames(void);  
size_t pmm_stat_dma32_free_frames(void);  
size_t pmm_stat_normal_total_frames(void);  
size_t pmm_stat_normal_free_frames(void);
```

```
void pmm_dump(void);
```

```
#ifdef PMM_DEBUG  
bool pmm_self_test(void);  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 43) - [public]

pmm_stat_dma32_total_frames

```
size_t pmm_stat_dma32_total_frames(void);  
size_t pmm_stat_dma32_free_frames(void);  
size_t pmm_stat_normal_total_frames(void);  
size_t pmm_stat_normal_free_frames(void);
```

```
void pmm_dump(void);
```

```
#ifdef PMM_DEBUG  
bool pmm_self_test(void);  
#endif
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 44) - [public]

pmm_stat_dma32_free_frames

```
size_t pmm_stat_dma32_free_frames(void);  
size_t pmm_stat_normal_total_frames(void);  
size_t pmm_stat_normal_free_frames(void);
```

```
void pmm_dump(void);
```

```
#ifdef PMM_DEBUG  
bool pmm_self_test(void);  
#endif
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 45) - [public]

pmm_stat_normal_total_frames

```
size_t pmm_stat_normal_total_frames(void);  
size_t pmm_stat_normal_free_frames(void);
```

```
void pmm_dump(void);
```

```
#ifdef PMM_DEBUG  
bool pmm_self_test(void);  
#endif
```

```
#endif
```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 46) - [public]

pmm_stat_normal_free_frames

```
size_t pmm_stat_normal_free_frames(void);
```

```
void pmm_dump(void);
```

```
#ifdef PMM_DEBUG  
bool pmm_self_test(void);  
#endif
```

```
#endif
```

```

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 48) - [public] pmm_dump
    void pmm_dump(void);

    #ifdef PMM_DEBUG
    bool pmm_self_test(void);
    #endif

    #endif

kernel_Workspace/kernel/src/mm/alloc/physical/pmm.h (line 51) - [public] pmm_self_test
    bool pmm_self_test(void);
    #endif

    #endif

kernel_Workspace/kernel/src/mm/alloc/special/contiguous.h (line 13) - [public] contig_free
    void contig_free(uint64_t phys, size_t bytes);

    #endif

kernel_Workspace/kernel/src/mm/alloc/special/dma.h (line 16) - [public] dma_free_coherent
    void dma_free_coherent(uint64_t phys, void *virt, size_t bytes);

    void dma_sync_for_device(void *virt, size_t len);
    void dma_sync_for_cpu(void *virt, size_t len);

    #endif

kernel_Workspace/kernel/src/mm/alloc/special/dma.h (line 18) - [public] dma_sync_for_device
    void dma_sync_for_device(void *virt, size_t len);
    void dma_sync_for_cpu(void *virt, size_t len);

    #endif

kernel_Workspace/kernel/src/mm/alloc/special/dma.h (line 19) - [public] dma_sync_for_cpu
    void dma_sync_for_cpu(void *virt, size_t len);

    #endif

kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.h (line 12) - [public] mapping_init
    void mapping_init(void);
    mapping_space_t mapping_kernel_space(void);

    bool mapping_map_range(mapping_space_t space,
                           uint64_t virt,
                           uint64_t phys,
                           size_t len,
                           uint64_t pte_flags);

    bool mapping_unmap_range(mapping_space_t space,
                             uint64_t virt,
                             size_t len);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.h (line 13) - [public]
mapping_kernel_space
    mapping_space_t mapping_kernel_space(void);

    bool mapping_map_range(mapping_space_t space,
                           uint64_t virt,
                           uint64_t phys,
                           size_t len,
                           uint64_t pte_flags);

    bool mapping_unmap_range(mapping_space_t space,
                             uint64_t virt,
                             size_t len);

    // ... (truncated, see source file for more)

```



```
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.h (line 30) - [public] mapping_translate
uint64_t mapping_translate(mapping_space_t space, uint64_t virt);
bool mapping_is_mapped(mapping_space_t space, uint64_t virt);

bool mapping_copy_range(mapping_space_t dst,
                        uint64_t dst_virt,
                        mapping_space_t src,
                        uint64_t src_virt,
                        size_t len);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/virtual/mapping.h (line 31) - [public] mapping_is_mapped
bool mapping_is_mapped(mapping_space_t space, uint64_t virt);

bool mapping_copy_range(mapping_space_t dst,
                        uint64_t dst_virt,
                        mapping_space_t src,
                        uint64_t src_virt,
                        size_t len);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 22) - [public] page_init
bool page_init(void);
bool page_ready(void);

page_t *page_alloc(uint32_t type);
page_t *page_alloc_zero(uint32_t type);

bool page_attach(uint64_t phys, uint32_t type);

bool page_get(page_t *page);
bool page_put(page_t *page);

uint64_t page_phys(const page_t *page);
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 23) - [public] page_ready
bool page_ready(void);

page_t *page_alloc(uint32_t type);
page_t *page_alloc_zero(uint32_t type);

bool page_attach(uint64_t phys, uint32_t type);

bool page_get(page_t *page);
bool page_put(page_t *page);

uint64_t page_phys(const page_t *page);
void *page_kva(const page_t *page);
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 28) - [public] page_attach
bool page_attach(uint64_t phys, uint32_t type);

bool page_get(page_t *page);
bool page_put(page_t *page);

uint64_t page_phys(const page_t *page);
void *page_kva(const page_t *page);
size_t page_pfn(const page_t *page);

page_t *page_from_phys(uint64_t phys);
page_t *page_from_pfn(size_t pfn);
page_t *page_from_kva(const void *va);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 30) - [public] page_get

```
bool page_get(page_t *page);
bool page_put(page_t *page);

uint64_t page_phys(const page_t *page);
void *page_kva(const page_t *page);
size_t page_pfn(const page_t *page);

page_t *page_from_phys(uint64_t phys);
page_t *page_from_pfn(size_t pfn);
page_t *page_from_kva(const void *va);

uint32_t page_refcount(const page_t *page);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 31) - [public] page_put

```
bool page_put(page_t *page);

uint64_t page_phys(const page_t *page);
void *page_kva(const page_t *page);
size_t page_pfn(const page_t *page);

page_t *page_from_phys(uint64_t phys);
page_t *page_from_pfn(size_t pfn);
page_t *page_from_kva(const void *va);

uint32_t page_refcount(const page_t *page);
uint32_t page_type(const page_t *page);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 33) - [public] page_phys

```
uint64_t page_phys(const page_t *page);
void *page_kva(const page_t *page);
size_t page_pfn(const page_t *page);

page_t *page_from_phys(uint64_t phys);
page_t *page_from_pfn(size_t pfn);
page_t *page_from_kva(const void *va);

uint32_t page_refcount(const page_t *page);
uint32_t page_type(const page_t *page);
void page_set_type(page_t *page, uint32_t type);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 35) - [public] page_pfn

```
size_t page_pfn(const page_t *page);

page_t *page_from_phys(uint64_t phys);
page_t *page_from_pfn(size_t pfn);
page_t *page_from_kva(const void *va);

uint32_t page_refcount(const page_t *page);
uint32_t page_type(const page_t *page);
void page_set_type(page_t *page, uint32_t type);

size_t page_stat_total(void);
size_t page_stat_referenced(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 41) - [public] page_refcount

```
uint32_t page_refcount(const page_t *page);
uint32_t page_type(const page_t *page);
void page_set_type(page_t *page, uint32_t type);

size_t page_stat_total(void);
size_t page_stat_referenced(void);

void page_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 42) - [public] page_type

```
uint32_t page_type(const page_t *page);
void page_set_type(page_t *page, uint32_t type);

size_t page_stat_total(void);
size_t page_stat_referenced(void);

void page_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 43) - [public] page_set_type

```
void page_set_type(page_t *page, uint32_t type);

size_t page_stat_total(void);
size_t page_stat_referenced(void);

void page_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 45) - [public] page_stat_total

```
size_t page_stat_total(void);
size_t page_stat_referenced(void);

void page_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 46) - [public] page_stat_referenced

```
size_t page_stat_referenced(void);

void page_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/page.h (line 48) - [public] page_dump

```
void page_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 18) - [public] vmm_init

```
bool vmm_init(void);
bool vmm_ready(void);

bool vmm_alloc(size_t bytes, uint32_t flags, uint64_t *out_virt);
bool vmm_alloc_aligned(size_t bytes, size_t align, uint32_t flags, uint64_t *out_virt);
bool vmm_map_device(uint64_t phys, size_t len, uint64_t *out_virt);

bool vmm_free(uint64_t virt, size_t bytes);
bool vmm_unmap_device(uint64_t virt, size_t len);

uint64_t vmm_translate(uint64_t virt);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 19) - [public] vmm_ready

```
bool vmm_ready(void);

bool vmm_alloc(size_t bytes, uint32_t flags, uint64_t *out_virt);
bool vmm_alloc_aligned(size_t bytes, size_t align, uint32_t flags, uint64_t *out_virt);
bool vmm_map_device(uint64_t phys, size_t len, uint64_t *out_virt);

bool vmm_free(uint64_t virt, size_t bytes);
bool vmm_unmap_device(uint64_t virt, size_t len);

uint64_t vmm_translate(uint64_t virt);

size_t vmm_stat_total_pages(void);
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 21) - [public] vmm_alloc
    bool vmm_alloc(size_t bytes, uint32_t flags, uint64_t *out_virt);
    bool vmm_alloc_aligned(size_t bytes, size_t align, uint32_t flags, uint64_t *out_virt);
    bool vmm_map_device(uint64_t phys, size_t len, uint64_t *out_virt);

    bool vmm_free(uint64_t virt, size_t bytes);
    bool vmm_unmap_device(uint64_t virt, size_t len);

    uint64_t vmm_translate(uint64_t virt);

    size_t vmm_stat_total_pages(void);
    size_t vmm_stat_free_pages(void);
    size_t vmm_stat_allocated_pages(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 22) - [public] vmm_alloc_aligned
    bool vmm_alloc_aligned(size_t bytes, size_t align, uint32_t flags, uint64_t *out_virt);
    bool vmm_map_device(uint64_t phys, size_t len, uint64_t *out_virt);

    bool vmm_free(uint64_t virt, size_t bytes);
    bool vmm_unmap_device(uint64_t virt, size_t len);

    uint64_t vmm_translate(uint64_t virt);

    size_t vmm_stat_total_pages(void);
    size_t vmm_stat_free_pages(void);
    size_t vmm_stat_allocated_pages(void);
    uint64_t vmm_stat_total_bytes(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 23) - [public] vmm_map_device
    bool vmm_map_device(uint64_t phys, size_t len, uint64_t *out_virt);

    bool vmm_free(uint64_t virt, size_t bytes);
    bool vmm_unmap_device(uint64_t virt, size_t len);

    uint64_t vmm_translate(uint64_t virt);

    size_t vmm_stat_total_pages(void);
    size_t vmm_stat_free_pages(void);
    size_t vmm_stat_allocated_pages(void);
    uint64_t vmm_stat_total_bytes(void);
    uint64_t vmm_stat_free_bytes(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 25) - [public] vmm_free
    bool vmm_free(uint64_t virt, size_t bytes);
    bool vmm_unmap_device(uint64_t virt, size_t len);

    uint64_t vmm_translate(uint64_t virt);

    size_t vmm_stat_total_pages(void);
    size_t vmm_stat_free_pages(void);
    size_t vmm_stat_allocated_pages(void);
    uint64_t vmm_stat_total_bytes(void);
    uint64_t vmm_stat_free_bytes(void);

    void vmm_dump(void);
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 26) - [public] vmm_unmap_device

```
bool vmm_unmap_device(uint64_t virt, size_t len);

uint64_t vmm_translate(uint64_t virt);

size_t vmm_stat_total_pages(void);
size_t vmm_stat_free_pages(void);
size_t vmm_stat_allocated_pages(void);
uint64_t vmm_stat_total_bytes(void);
uint64_t vmm_stat_free_bytes(void);

void vmm_dump(void);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 28) - [public] vmm_translate

```
uint64_t vmm_translate(uint64_t virt);

size_t vmm_stat_total_pages(void);
size_t vmm_stat_free_pages(void);
size_t vmm_stat_allocated_pages(void);
uint64_t vmm_stat_total_bytes(void);
uint64_t vmm_stat_free_bytes(void);

void vmm_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 30) - [public] vmm_stat_total_pages

```
size_t vmm_stat_total_pages(void);
size_t vmm_stat_free_pages(void);
size_t vmm_stat_allocated_pages(void);
uint64_t vmm_stat_total_bytes(void);
uint64_t vmm_stat_free_bytes(void);

void vmm_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 31) - [public] vmm_stat_free_pages

```
size_t vmm_stat_free_pages(void);
size_t vmm_stat_allocated_pages(void);
uint64_t vmm_stat_total_bytes(void);
uint64_t vmm_stat_free_bytes(void);

void vmm_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 32) - [public]

vmm_stat_allocated_pages

```
size_t vmm_stat_allocated_pages(void);
uint64_t vmm_stat_total_bytes(void);
uint64_t vmm_stat_free_bytes(void);

void vmm_dump(void);

#endif
```

kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 33) - [public] vmm_stat_total_bytes

```
uint64_t vmm_stat_total_bytes(void);
uint64_t vmm_stat_free_bytes(void);

void vmm_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 34) - [public] vmm_stat_free_bytes
    uint64_t vmm_stat_free_bytes(void);

    void vmm_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/alloc/virtual/vmm.h (line 36) - [public] vmm_dump
    void vmm_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 25) - [public] arch_memory_ready
    bool arch_memory_ready(void);

    bool arch_memory_boot_alloc(uint64_t len, uint64_t align, uint64_t *out);
    void arch_memory_reserve_range(uint64_t base, uint64_t len);

    bool arch_is_page_aligned(uint64_t a);
    uint64_t arch_page_align_up(uint64_t a);
    uint64_t arch_page_align_down(uint64_t a);

    void *arch_phys_to_virt(uint64_t phys);
    uint64_t arch_virt_to_phys(void *virt);

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 27) - [public]
arch_memory_boot_alloc
    bool arch_memory_boot_alloc(uint64_t len, uint64_t align, uint64_t *out);
    void arch_memory_reserve_range(uint64_t base, uint64_t len);

    bool arch_is_page_aligned(uint64_t a);
    uint64_t arch_page_align_up(uint64_t a);
    uint64_t arch_page_align_down(uint64_t a);

    void *arch_phys_to_virt(uint64_t phys);
    uint64_t arch_virt_to_phys(void *virt);

    uint64_t arch_memory_max_phys(void);
    void arch_memory_dump(void);
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 28) - [public]
arch_memory_reserve_range
    void arch_memory_reserve_range(uint64_t base, uint64_t len);

    bool arch_is_page_aligned(uint64_t a);
    uint64_t arch_page_align_up(uint64_t a);
    uint64_t arch_page_align_down(uint64_t a);

    void *arch_phys_to_virt(uint64_t phys);
    uint64_t arch_virt_to_phys(void *virt);

    uint64_t arch_memory_max_phys(void);
    void arch_memory_dump(void);

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 30) - [public] arch_is_page_aligned
    bool arch_is_page_aligned(uint64_t a);
    uint64_t arch_page_align_up(uint64_t a);
    uint64_t arch_page_align_down(uint64_t a);

    void *arch_phys_to_virt(uint64_t phys);
    uint64_t arch_virt_to_phys(void *virt);

    uint64_t arch_memory_max_phys(void);
    void arch_memory_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 31) - [public] arch_page_align_up
uint64_t arch_page_align_up(uint64_t a);
uint64_t arch_page_align_down(uint64_t a);

void *arch_phys_to_virt(uint64_t phys);
uint64_t arch_virt_to_phys(void *virt);

uint64_t arch_memory_max_phys(void);
void arch_memory_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 32) - [public] arch_page_align_down
uint64_t arch_page_align_down(uint64_t a);

void *arch_phys_to_virt(uint64_t phys);
uint64_t arch_virt_to_phys(void *virt);

uint64_t arch_memory_max_phys(void);
void arch_memory_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 35) - [public] arch_virt_to_phys
uint64_t arch_virt_to_phys(void *virt);

uint64_t arch_memory_max_phys(void);
void arch_memory_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 37) - [public] arch_memory_max_phys
uint64_t arch_memory_max_phys(void);
void arch_memory_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/memory.h (line 38) - [public] arch_memory_dump
void arch_memory_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 17) - [public] paging_init
void paging_init(uint64_t boot_phys_offset);

bool paging_enable_nx(void);

uint64_t paging_read_cr3(void);
void paging_write_cr3(uint64_t pml4_phys);

bool paging_map_page_in(uint64_t pml4, uint64_t virt, uint64_t phys, uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 19) - [public] paging_enable_nx
bool paging_enable_nx(void);

uint64_t paging_read_cr3(void);
void paging_write_cr3(uint64_t pml4_phys);

bool paging_map_page_in(uint64_t pml4, uint64_t virt, uint64_t phys, uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 21) - [public] paging_read_cr3

```
uint64_t paging_read_cr3(void);
void paging_write_cr3(uint64_t pml4_phys);

bool paging_map_page_in(uint64_t pml4, uint64_t virt, uint64_t phys, uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 22) - [public] paging_write_cr3

```
void paging_write_cr3(uint64_t pml4_phys);

bool paging_map_page_in(uint64_t pml4, uint64_t virt, uint64_t phys, uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 24) - [public] paging_map_page_in

```
bool paging_map_page_in(uint64_t pml4, uint64_t virt, uint64_t phys, uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 25) - [public] paging_unmap_page_in

```
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 26) - [public] paging_set_flags_in

```
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 27) - [public] paging_is_mapped_in

```
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/paging.h (line 28) - [public] paging_translate_in

```
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);

#endif
```


kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 8) - [public] tlb_init

```
bool tlb_init(void);
bool tlb_ready(void);

void tlb_flush_all(void);
void tlb_flush_all_including_global(void);

void tlb_flush_page(const void *addr);
void tlb_flush_page_addr(uint64_t addr);

void tlb_flush_range(const void *addr, size_t pages);
void tlb_flush_range_addr(uint64_t addr, size_t pages);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 9) - [public] tlb_ready

```
bool tlb_ready(void);

void tlb_flush_all(void);
void tlb_flush_all_including_global(void);

void tlb_flush_page(const void *addr);
void tlb_flush_page_addr(uint64_t addr);

void tlb_flush_range(const void *addr, size_t pages);
void tlb_flush_range_addr(uint64_t addr, size_t pages);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 11) - [public] tlb_flush_all

```
void tlb_flush_all(void);
void tlb_flush_all_including_global(void);

void tlb_flush_page(const void *addr);
void tlb_flush_page_addr(uint64_t addr);

void tlb_flush_range(const void *addr, size_t pages);
void tlb_flush_range_addr(uint64_t addr, size_t pages);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 12) - [public]

tlb_flush_all_including_global

```
void tlb_flush_all_including_global(void);

void tlb_flush_page(const void *addr);
void tlb_flush_page_addr(uint64_t addr);

void tlb_flush_range(const void *addr, size_t pages);
void tlb_flush_range_addr(uint64_t addr, size_t pages);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 14) - [public] tlb_flush_page

```
void tlb_flush_page(const void *addr);
void tlb_flush_page_addr(uint64_t addr);

void tlb_flush_range(const void *addr, size_t pages);
void tlb_flush_range_addr(uint64_t addr, size_t pages);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 15) - [public] tlb_flush_page_addr

```
void tlb_flush_page_addr(uint64_t addr);

void tlb_flush_range(const void *addr, size_t pages);
void tlb_flush_range_addr(uint64_t addr, size_t pages);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 17) - [public] tlb_flush_range

```
void tlb_flush_range(const void *addr, size_t pages);
void tlb_flush_range_addr(uint64_t addr, size_t pages);

#endif
```

kernel_Workspace/kernel/src/mm/arch/aarch64/tlb.h (line 18) - [public] tlb_flush_range_addr

```
void tlb_flush_range_addr(uint64_t addr, size_t pages);

#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 71) - [public] arch_memory_ready

```
bool arch_memory_ready(void);

bool arch_memory_boot_alloc(uint64_t len, uint64_t align, uint64_t *out_base);

const arch_mem_info_t *arch_memory_get(void);
size_t arch_memory_regions(const arch_mem_region_t **out);
uint64_t arch_memory_total_usable(void);
bool arch_memory_range_is_usable(uint64_t base, uint64_t len);
void arch_memory_reserve_range(uint64_t base, uint64_t len);
bool arch_memory_find_usable(uint64_t len,
                             uint64_t align,
                             uint64_t *out_base);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 73) - [public]

arch_memory_boot_alloc

```
bool arch_memory_boot_alloc(uint64_t len, uint64_t align, uint64_t *out_base);

const arch_mem_info_t *arch_memory_get(void);
size_t arch_memory_regions(const arch_mem_region_t **out);
uint64_t arch_memory_total_usable(void);
bool arch_memory_range_is_usable(uint64_t base, uint64_t len);
void arch_memory_reserve_range(uint64_t base, uint64_t len);
bool arch_memory_find_usable(uint64_t len,
                             uint64_t align,
                             uint64_t *out_base);

void arch_memory_dump(void);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 76) - [public] arch_memory_regions

```
size_t arch_memory_regions(const arch_mem_region_t **out);
uint64_t arch_memory_total_usable(void);
bool arch_memory_range_is_usable(uint64_t base, uint64_t len);
void arch_memory_reserve_range(uint64_t base, uint64_t len);
bool arch_memory_find_usable(uint64_t len,
                             uint64_t align,
                             uint64_t *out_base);

void arch_memory_dump(void);

static inline uint64_t arch_page_align_down(uint64_t v)
{
    return v & ~ARCH_PAGE_MASK;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 77) - [public]

arch_memory_total_usable

```
uint64_t arch_memory_total_usable(void);
bool arch_memory_range_is_usable(uint64_t base, uint64_t len);
void arch_memory_reserve_range(uint64_t base, uint64_t len);
bool arch_memory_find_usable(uint64_t len,
                             uint64_t align,
                             uint64_t *out_base);

void arch_memory_dump(void);

static inline uint64_t arch_page_align_down(uint64_t v)
{
    return v & ~ARCH_PAGE_MASK;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 78) - [public]

arch_memory_range_is_usable

```
bool arch_memory_range_is_usable(uint64_t base, uint64_t len);
void arch_memory_reserve_range(uint64_t base, uint64_t len);
bool arch_memory_find_usable(uint64_t len,
                             uint64_t align,
                             uint64_t *out_base);

void arch_memory_dump(void);

static inline uint64_t arch_page_align_down(uint64_t v)
{
    return v & ~ARCH_PAGE_MASK;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 79) - [public]

arch_memory_reserve_range

```
void arch_memory_reserve_range(uint64_t base, uint64_t len);
bool arch_memory_find_usable(uint64_t len,
                             uint64_t align,
                             uint64_t *out_base);

void arch_memory_dump(void);

static inline uint64_t arch_page_align_down(uint64_t v)
{
    return v & ~ARCH_PAGE_MASK;
}

static inline uint64_t arch_page_align_up(uint64_t v)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 83) - [public] arch_memory_dump

```
void arch_memory_dump(void);

static inline uint64_t arch_page_align_down(uint64_t v)
{
    return v & ~ARCH_PAGE_MASK;
}

static inline uint64_t arch_page_align_up(uint64_t v)
{
    if (v > UINT64_MAX - ARCH_PAGE_MASK) {
        return UINT64_MAX;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 92) - [public] if

```
    if (v > UINT64_MAX - ARCH_PAGE_MASK) {
        return UINT64_MAX;
    }

    return (v + ARCH_PAGE_MASK) & ~ARCH_PAGE_MASK;
}

static inline bool arch_is_page_aligned(uint64_t v)
{
    return (v & ARCH_PAGE_MASK) == 0;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 107) - [public] return

```
    return (void *)((uintptr_t)(phys + ARCH_DIRECT_MAP_BASE);
}

static inline uint64_t arch_virt_to_phys(const void *virt)
{
    return (uint64_t)((uintptr_t)virt - ARCH_DIRECT_MAP_BASE);
}

#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/memory.h (line 112) - [public] return

```
    return (uint64_t)((uintptr_t)virt - ARCH_DIRECT_MAP_BASE);
}

#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 35) - [public]

paging_set_boot_phys_offset

```
void paging_set_boot_phys_offset(uint64_t phys_offset);

bool paging_boot_phys_offset_valid(void);

void paging_init_direct_map(void);

void paging_init(uint64_t boot_phys_offset);

uint64_t paging_read_cr3(void);
void paging_flush_tlb_all(void);
void paging_flush_page(uint64_t addr);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 37) - [public]

paging_boot_phys_offset_valid

```
bool paging_boot_phys_offset_valid(void);

void paging_init_direct_map(void);

void paging_init(uint64_t boot_phys_offset);

uint64_t paging_read_cr3(void);
void paging_flush_tlb_all(void);
void paging_flush_page(uint64_t addr);

bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 39) - [public]

paging_init_direct_map

```
void paging_init_direct_map(void);

void paging_init(uint64_t boot_phys_offset);

uint64_t paging_read_cr3(void);
void paging_flush_tlb_all(void);
void paging_flush_page(uint64_t addr);

bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);

bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len, uint64_t flags);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 41) - [public] paging_init

```
void paging_init(uint64_t boot_phys_offset);

uint64_t paging_read_cr3(void);
void paging_flush_tlb_all(void);
void paging_flush_page(uint64_t addr);

bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);

bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len, uint64_t flags);

uint64_t paging_translate(uint64_t virt);
bool paging_is_mapped(uint64_t virt);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 43) - [public] paging_read_cr3

```
uint64_t paging_read_cr3(void);
void paging_flush_tlb_all(void);
void paging_flush_page(uint64_t addr);

bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);

bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len, uint64_t flags);

uint64_t paging_translate(uint64_t virt);
bool paging_is_mapped(uint64_t virt);
bool paging_unmap_page(uint64_t virt);
bool paging_set_flags(uint64_t virt, uint64_t flags);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 44) - [public] paging_flush_tlb_all

```
void paging_flush_tlb_all(void);
void paging_flush_page(uint64_t addr);

bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);

bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len, uint64_t flags);

uint64_t paging_translate(uint64_t virt);
bool paging_is_mapped(uint64_t virt);
bool paging_unmap_page(uint64_t virt);
bool paging_set_flags(uint64_t virt, uint64_t flags);
bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 45) - [public] paging_flush_page
    void paging_flush_page(uint64_t addr);

    bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);

    bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len, uint64_t flags);

    uint64_t paging_translate(uint64_t virt);
    bool paging_is_mapped(uint64_t virt);
    bool paging_unmap_page(uint64_t virt);
    bool paging_set_flags(uint64_t virt, uint64_t flags);
    bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
    void paging_enable_nx(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 47) - [public] paging_map_page
    bool paging_map_page(uint64_t virt, uint64_t phys, uint64_t flags);

    bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len, uint64_t flags);

    uint64_t paging_translate(uint64_t virt);
    bool paging_is_mapped(uint64_t virt);
    bool paging_unmap_page(uint64_t virt);
    bool paging_set_flags(uint64_t virt, uint64_t flags);
    bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
    void paging_enable_nx(void);
    bool paging_nx_enabled(void);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 49) - [public] paging_map_range
    bool paging_map_range(uint64_t virt, uint64_t phys, uint64_t len, uint64_t flags);

    uint64_t paging_translate(uint64_t virt);
    bool paging_is_mapped(uint64_t virt);
    bool paging_unmap_page(uint64_t virt);
    bool paging_set_flags(uint64_t virt, uint64_t flags);
    bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
    void paging_enable_nx(void);
    bool paging_nx_enabled(void);

    bool paging_map_page_in(uint64_t pml4,
                            uint64_t virt,
                            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 51) - [public] paging_translate
    uint64_t paging_translate(uint64_t virt);
    bool paging_is_mapped(uint64_t virt);
    bool paging_unmap_page(uint64_t virt);
    bool paging_set_flags(uint64_t virt, uint64_t flags);
    bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
    void paging_enable_nx(void);
    bool paging_nx_enabled(void);

    bool paging_map_page_in(uint64_t pml4,
                            uint64_t virt,
                            uint64_t phys,
                            uint64_t flags);
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 52) - [public] paging_is_mapped

```
bool paging_is_mapped(uint64_t virt);
bool paging_unmap_page(uint64_t virt);
bool paging_set_flags(uint64_t virt, uint64_t flags);
bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
void paging_enable_nx(void);
bool paging_nx_enabled(void);

bool paging_map_page_in(uint64_t pml4,
                        uint64_t virt,
                        uint64_t phys,
                        uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 53) - [public] paging_unmap_page

```
bool paging_unmap_page(uint64_t virt);
bool paging_set_flags(uint64_t virt, uint64_t flags);
bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
void paging_enable_nx(void);
bool paging_nx_enabled(void);

bool paging_map_page_in(uint64_t pml4,
                        uint64_t virt,
                        uint64_t phys,
                        uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 54) - [public] paging_set_flags

```
bool paging_set_flags(uint64_t virt, uint64_t flags);
bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
void paging_enable_nx(void);
bool paging_nx_enabled(void);

bool paging_map_page_in(uint64_t pml4,
                        uint64_t virt,
                        uint64_t phys,
                        uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 55) - [public] paging_get_flags

```
bool paging_get_flags(uint64_t virt, uint64_t *out_flags);
void paging_enable_nx(void);
bool paging_nx_enabled(void);

bool paging_map_page_in(uint64_t pml4,
                        uint64_t virt,
                        uint64_t phys,
                        uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 56) - [public] paging_enable_nx

```
void paging_enable_nx(void);
bool paging_nx_enabled(void);

bool paging_map_page_in(uint64_t pml4,
                        uint64_t virt,
                        uint64_t phys,
                        uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 57) - [public] paging_nx_enabled

```
bool paging_nx_enabled(void);

bool paging_map_page_in(uint64_t pml4,
                        uint64_t virt,
                        uint64_t phys,
                        uint64_t flags);
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 63) - [public] paging_unmap_page_in

```
bool paging_unmap_page_in(uint64_t pml4, uint64_t virt);
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);

uint64_t paging_create_pml4(void);
void paging_destroy_pml4(uint64_t pml4_phys);
void paging_switch_pml4(uint64_t pml4_phys);

bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 64) - [public] paging_translate_in

```
uint64_t paging_translate_in(uint64_t pml4, uint64_t virt);
bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);

uint64_t paging_create_pml4(void);
void paging_destroy_pml4(uint64_t pml4_phys);
void paging_switch_pml4(uint64_t pml4_phys);

bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

void paging_enable_write_protect(void);
// ... (truncated, see source file for more)
```



```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 65) - [public] paging_is_mapped_in
    bool paging_is_mapped_in(uint64_t pml4, uint64_t virt);
    bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
    bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);

    uint64_t paging_create_pml4(void);
    void paging_destroy_pml4(uint64_t pml4_phys);
    void paging_switch_pml4(uint64_t pml4_phys);

    bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

    void paging_enable_write_protect(void);
    void paging_disable_write_protect(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 66) - [public] paging_set_flags_in
    bool paging_set_flags_in(uint64_t pml4, uint64_t virt, uint64_t flags);
    bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);

    uint64_t paging_create_pml4(void);
    void paging_destroy_pml4(uint64_t pml4_phys);
    void paging_switch_pml4(uint64_t pml4_phys);

    bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

    void paging_enable_write_protect(void);
    void paging_disable_write_protect(void);
    bool paging_write_protect_enabled(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 67) - [public] paging_get_flags_in
    bool paging_get_flags_in(uint64_t pml4, uint64_t virt, uint64_t *out_flags);

    uint64_t paging_create_pml4(void);
    void paging_destroy_pml4(uint64_t pml4_phys);
    void paging_switch_pml4(uint64_t pml4_phys);

    bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

    void paging_enable_write_protect(void);
    void paging_disable_write_protect(void);
    bool paging_write_protect_enabled(void);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 69) - [public] paging_create_pml4
    uint64_t paging_create_pml4(void);
    void paging_destroy_pml4(uint64_t pml4_phys);
    void paging_switch_pml4(uint64_t pml4_phys);

    bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

    void paging_enable_write_protect(void);
    void paging_disable_write_protect(void);
    bool paging_write_protect_enabled(void);

    void paging_enable_smep(void);
    void paging_enable_smap(void);
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 70) - [public] paging_destroy_pml4

```
void paging_destroy_pml4(uint64_t pml4_phys);
void paging_switch_pml4(uint64_t pml4_phys);

bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

void paging_enable_write_protect(void);
void paging_disable_write_protect(void);
bool paging_write_protect_enabled(void);

void paging_enable_smep(void);
void paging_enable_smap(void);
bool paging_smep_enabled(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 71) - [public] paging_switch_pml4

```
void paging_switch_pml4(uint64_t pml4_phys);

bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

void paging_enable_write_protect(void);
void paging_disable_write_protect(void);
bool paging_write_protect_enabled(void);

void paging_enable_smep(void);
void paging_enable_smap(void);
bool paging_smep_enabled(void);
bool paging_smap_enabled(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 73) - [public] paging_map_mmio

```
bool paging_map_mmio(uint64_t virt, uint64_t phys, uint64_t len);

void paging_enable_write_protect(void);
void paging_disable_write_protect(void);
bool paging_write_protect_enabled(void);

void paging_enable_smep(void);
void paging_enable_smap(void);
bool paging_smep_enabled(void);
bool paging_smap_enabled(void);

bool paging_pcid_supported(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 75) - [public]

paging_enable_write_protect

```
void paging_enable_write_protect(void);
void paging_disable_write_protect(void);
bool paging_write_protect_enabled(void);

void paging_enable_smep(void);
void paging_enable_smap(void);
bool paging_smep_enabled(void);
bool paging_smap_enabled(void);

bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 76) - [public]

paging_disable_write_protect

```
void paging_disable_write_protect(void);
bool paging_write_protect_enabled(void);

void paging_enable_smep(void);
void paging_enable_smmap(void);
bool paging_smep_enabled(void);
bool paging_smmap_enabled(void);

bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpaid(uint64_t type, uint64_t pcid, uint64_t addr);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 77) - [public]

paging_write_protect_enabled

```
bool paging_write_protect_enabled(void);

void paging_enable_smep(void);
void paging_enable_smmap(void);
bool paging_smep_enabled(void);
bool paging_smmap_enabled(void);

bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpaid(uint64_t type, uint64_t pcid, uint64_t addr);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 79) - [public] paging_enable_smep

```
void paging_enable_smep(void);
void paging_enable_smmap(void);
bool paging_smep_enabled(void);
bool paging_smmap_enabled(void);

bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpaid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 80) - [public] paging_enable_smmap

```
void paging_enable_smmap(void);
bool paging_smep_enabled(void);
bool paging_smmap_enabled(void);

bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpaid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
void paging_assert_4level_paging(void);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 81) - [public] paging_smep_enabled

```
bool paging_smep_enabled(void);
bool paging_smap_enabled(void);

bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpcid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
void paging_assert_4level_paging(void);
void paging_write_cr3(uint64_t pml4_phys);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 82) - [public] paging_smap_enabled

```
bool paging_smap_enabled(void);

bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpcid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
void paging_assert_4level_paging(void);
void paging_write_cr3(uint64_t pml4_phys);
#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 84) - [public] paging_pcid_supported

```
bool paging_pcid_supported(void);
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpcid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
void paging_assert_4level_paging(void);
void paging_write_cr3(uint64_t pml4_phys);
#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 85) - [public] paging_enable_pcid

```
void paging_enable_pcid(void);
bool paging_pcid_enabled(void);
void paging_invpcid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
void paging_assert_4level_paging(void);
void paging_write_cr3(uint64_t pml4_phys);
#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 86) - [public] paging_pcid_enabled

```
bool paging_pcid_enabled(void);
void paging_invpcid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
void paging_assert_4level_paging(void);
void paging_write_cr3(uint64_t pml4_phys);
#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 87) - [public] paging_invpcid

```
void paging_invpcid(uint64_t type, uint64_t pcid, uint64_t addr);

bool paging_la57_supported(void);
void paging_assert_4level_paging(void);
void paging_write_cr3(uint64_t pml4_phys);
#endif
```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 90) - [public] paging_la57_supported
    bool paging_la57_supported(void);
    void paging_assert_4level_paging(void);
    void paging_write_cr3(uint64_t pml4_phys);
    #endif

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 91) - [public]
paging_assert_4level_paging
    void paging_assert_4level_paging(void);
    void paging_write_cr3(uint64_t pml4_phys);
    #endif

kernel_Workspace/kernel/src/mm/arch/x86_64/paging.h (line 92) - [public] paging_write_cr3
    void paging_write_cr3(uint64_t pml4_phys);
    #endif

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 14) - [public] tlb_init
    bool tlb_init(void);
    bool tlb_ready(void);
    bool tlb_has_pcid(void);
    bool tlb_has_invpcid(void);

    void tlb_flush_all(void);
    void tlb_flush_all_including_global(void);

    void tlb_flush_page(const void *addr);
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 15) - [public] tlb_ready
    bool tlb_ready(void);
    bool tlb_has_pcid(void);
    bool tlb_has_invpcid(void);

    void tlb_flush_all(void);
    void tlb_flush_all_including_global(void);

    void tlb_flush_page(const void *addr);
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 16) - [public] tlb_has_pcid
    bool tlb_has_pcid(void);
    bool tlb_has_invpcid(void);

    void tlb_flush_all(void);
    void tlb_flush_all_including_global(void);

    void tlb_flush_page(const void *addr);
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 17) - [public] tlb_has_invpcid
    bool tlb_has_invpcid(void);

    void tlb_flush_all(void);
    void tlb_flush_all_including_global(void);

    void tlb_flush_page(const void *addr);
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    void tlb_flush_pcid(uint16_t pcid);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 19) - [public] tlb_flush_all
    void tlb_flush_all(void);
    void tlb_flush_all_including_global(void);

    void tlb_flush_page(const void *addr);
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    void tlb_flush_pcid(uint16_t pcid);
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 20) - [public]
tlb_flush_all_including_global
    void tlb_flush_all_including_global(void);

    void tlb_flush_page(const void *addr);
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    void tlb_flush_pcid(uint16_t pcid);
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    void tlb_wbinvd(void);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 22) - [public] tlb_flush_page
    void tlb_flush_page(const void *addr);
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    void tlb_flush_pcid(uint16_t pcid);
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    void tlb_wbinvd(void);
    void tlb_clflush(uint64_t addr);

    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 23) - [public] tlb_flush_page_addr
    void tlb_flush_page_addr(uint64_t addr);

    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    void tlb_flush_pcid(uint16_t pcid);
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    void tlb_wbinvd(void);
    void tlb_clflush(uint64_t addr);

#ifdef

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 25) - [public] tlb_flush_range
    void tlb_flush_range(const void *addr, size_t pages);
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    void tlb_flush_pcid(uint16_t pcid);
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    void tlb_wbinvd(void);
    void tlb_clflush(uint64_t addr);

#ifdef

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 26) - [public] tlb_flush_range_addr
    void tlb_flush_range_addr(uint64_t addr, size_t pages);

    void tlb_flush_pcid(uint16_t pcid);
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    void tlb_wbinvd(void);
    void tlb_clflush(uint64_t addr);

#ifdef

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 28) - [public] tlb_flush_pcid
    void tlb_flush_pcid(uint16_t pcid);
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    void tlb_wbinvd(void);
    void tlb_clflush(uint64_t addr);

#ifdef

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 29) - [public] tlb_flush_pcid_addr
    void tlb_flush_pcid_addr(uint16_t pcid, uint64_t addr);

    void tlb_wbinvd(void);
    void tlb_clflush(uint64_t addr);

#ifdef

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 31) - [public] tlb_wbinvd
    void tlb_wbinvd(void);
    void tlb_clflush(uint64_t addr);

#ifdef

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb.h (line 32) - [public] tlb_clflush
    void tlb_clflush(uint64_t addr);

#ifdef

```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 6) - [public] tlb_asm_invlpg

```
void tlb_asm_invlpg(uint64_t addr);

uint64_t tlb_asm_read_cr3(void);
void tlb_asm_write_cr3(uint64_t value);

uint64_t tlb_asm_read_cr4(void);
void tlb_asm_write_cr4(uint64_t value);

void tlb_asm_invpaid(uint64_t type, const void *desc);

void tlb_asm_wbinvd(void);
void tlb_asm_clflush(uint64_t addr);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 8) - [public] tlb_asm_read_cr3

```
uint64_t tlb_asm_read_cr3(void);
void tlb_asm_write_cr3(uint64_t value);

uint64_t tlb_asm_read_cr4(void);
void tlb_asm_write_cr4(uint64_t value);

void tlb_asm_invpaid(uint64_t type, const void *desc);

void tlb_asm_wbinvd(void);
void tlb_asm_clflush(uint64_t addr);

#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 9) - [public] tlb_asm_write_cr3

```
void tlb_asm_write_cr3(uint64_t value);

uint64_t tlb_asm_read_cr4(void);
void tlb_asm_write_cr4(uint64_t value);

void tlb_asm_invpaid(uint64_t type, const void *desc);

void tlb_asm_wbinvd(void);
void tlb_asm_clflush(uint64_t addr);

#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 11) - [public] tlb_asm_read_cr4

```
uint64_t tlb_asm_read_cr4(void);
void tlb_asm_write_cr4(uint64_t value);

void tlb_asm_invpaid(uint64_t type, const void *desc);

void tlb_asm_wbinvd(void);
void tlb_asm_clflush(uint64_t addr);

#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 12) - [public] tlb_asm_write_cr4

```
void tlb_asm_write_cr4(uint64_t value);

void tlb_asm_invpaid(uint64_t type, const void *desc);

void tlb_asm_wbinvd(void);
void tlb_asm_clflush(uint64_t addr);

#endif
```

kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 14) - [public] tlb_asm_invpaid

```
void tlb_asm_invpaid(uint64_t type, const void *desc);

void tlb_asm_wbinvd(void);
void tlb_asm_clflush(uint64_t addr);

#endif
```



```
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 16) - [public] tlb_asm_wbinvd
    void tlb_asm_wbinvd(void);
    void tlb_asm_clflush(uint64_t addr);

#endif
```

```
kernel_Workspace/kernel/src/mm/arch/x86_64/tlb_asm.h (line 17) - [public] tlb_asm_clflush
    void tlb_asm_clflush(uint64_t addr);

#endif
```

```
kernel_Workspace/kernel/src/mm/cache/cache.h (line 7) - [public] cache_init
    bool cache_init(void);
    bool cache_ready(void);

    void cache_register(kcache_t *c);
    kcache_t *cache_find(const char *name);

    void cache_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/cache/cache.h (line 8) - [public] cache_ready
    bool cache_ready(void);

    void cache_register(kcache_t *c);
    kcache_t *cache_find(const char *name);

    void cache_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/cache/cache.h (line 10) - [public] cache_register
    void cache_register(kcache_t *c);
    kcache_t *cache_find(const char *name);

    void cache_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/cache/cache.h (line 13) - [public] cache_dump
    void cache_dump(void);

#endif
```

```
kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 12) - [public] void
typedef void (*kcache_ctor_t)(void *obj);

typedef struct kcache_chunk {
    struct kcache_chunk *next;
    uint64_t va;
} kcache_chunk_t;

typedef struct kcache_mag {
    void *objs[KCACHE_MAG_SIZE];
    size_t count;
} kcache_mag_t;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 47) - [public] kcache_destroy

```
void kcache_destroy(kcache_t *c);

void *kcache_alloc(kcache_t *c);
void *kcache_zalloc(kcache_t *c);
void kcache_free(kcache_t *c, void *ptr);

size_t kcache_object_size(kcache_t *c);
size_t kcache_free_count(kcache_t *c);
size_t kcache_total_count(kcache_t *c);

#endif
```

kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 51) - [public] kcache_free

```
void kcache_free(kcache_t *c, void *ptr);

size_t kcache_object_size(kcache_t *c);
size_t kcache_free_count(kcache_t *c);
size_t kcache_total_count(kcache_t *c);

#endif
```

kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 53) - [public] kcache_object_size

```
size_t kcache_object_size(kcache_t *c);
size_t kcache_free_count(kcache_t *c);
size_t kcache_total_count(kcache_t *c);

#endif
```

kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 54) - [public] kcache_free_count

```
size_t kcache_free_count(kcache_t *c);
size_t kcache_total_count(kcache_t *c);

#endif
```

kernel_Workspace/kernel/src/mm/cache/object_cashe.h (line 55) - [public] kcache_total_count

```
size_t kcache_total_count(kcache_t *c);

#endif
```

kernel_Workspace/kernel/src/mm/cache/per_cpu.h (line 12) - [public] per_cpu_init

```
bool per_cpu_init(void);
size_t per_cpu_count(void);
size_t per_cpu_id(void);

#endif
```

kernel_Workspace/kernel/src/mm/cache/per_cpu.h (line 13) - [public] per_cpu_count

```
size_t per_cpu_count(void);
size_t per_cpu_id(void);

#endif
```

kernel_Workspace/kernel/src/mm/cache/per_cpu.h (line 14) - [public] per_cpu_id

```
size_t per_cpu_id(void);

#endif
```

kernel_Workspace/kernel/src/mm/core/address.h (line 10) - [public] addr_is_canonical

```
bool addr_is_canonical(uint64_t a);
bool addr_is_user(uint64_t a);
bool addr_is_kernel(uint64_t a);
bool addr_is_direct_map(uint64_t a);

uint64_t addr_align_up(uint64_t a, uint64_t align);
uint64_t addr_align_down(uint64_t a, uint64_t align);

uint64_t addr_phys_to_direct(uint64_t phys);
uint64_t addr_direct_to_phys(uint64_t va);

const char *addr_describe(uint64_t a);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/address.h (line 11) - [public] addr_is_user

```
bool addr_is_user(uint64_t a);
bool addr_is_kernel(uint64_t a);
bool addr_is_direct_map(uint64_t a);

uint64_t addr_align_up(uint64_t a, uint64_t align);
uint64_t addr_align_down(uint64_t a, uint64_t align);

uint64_t addr_phys_to_direct(uint64_t phys);
uint64_t addr_direct_to_phys(uint64_t va);

const char *addr_describe(uint64_t a);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/address.h (line 12) - [public] addr_is_kernel

```
bool addr_is_kernel(uint64_t a);
bool addr_is_direct_map(uint64_t a);

uint64_t addr_align_up(uint64_t a, uint64_t align);
uint64_t addr_align_down(uint64_t a, uint64_t align);

uint64_t addr_phys_to_direct(uint64_t phys);
uint64_t addr_direct_to_phys(uint64_t va);

const char *addr_describe(uint64_t a);

#endif
```

kernel_Workspace/kernel/src/mm/core/address.h (line 13) - [public] addr_is_direct_map

```
bool addr_is_direct_map(uint64_t a);

uint64_t addr_align_up(uint64_t a, uint64_t align);
uint64_t addr_align_down(uint64_t a, uint64_t align);

uint64_t addr_phys_to_direct(uint64_t phys);
uint64_t addr_direct_to_phys(uint64_t va);

const char *addr_describe(uint64_t a);

#endif
```

kernel_Workspace/kernel/src/mm/core/address.h (line 15) - [public] addr_align_up

```
uint64_t addr_align_up(uint64_t a, uint64_t align);
uint64_t addr_align_down(uint64_t a, uint64_t align);

uint64_t addr_phys_to_direct(uint64_t phys);
uint64_t addr_direct_to_phys(uint64_t va);

const char *addr_describe(uint64_t a);

#endif
```

```

kernel_Workspace/kernel/src/mm/core/address.h (line 16) - [public] addr_align_down
    uint64_t addr_align_down(uint64_t a, uint64_t align);

    uint64_t addr_phys_to_direct(uint64_t phys);
    uint64_t addr_direct_to_phys(uint64_t va);

    const char *addr_describe(uint64_t a);

#endif

kernel_Workspace/kernel/src/mm/core/address.h (line 18) - [public] addr_phys_to_direct
    uint64_t addr_phys_to_direct(uint64_t phys);
    uint64_t addr_direct_to_phys(uint64_t va);

    const char *addr_describe(uint64_t a);

#endif

kernel_Workspace/kernel/src/mm/core/address.h (line 19) - [public] addr_direct_to_phys
    uint64_t addr_direct_to_phys(uint64_t va);

    const char *addr_describe(uint64_t a);

#endif

kernel_Workspace/kernel/src/mm/core/mm.h (line 26) - [public] mm_init
    bool mm_init(const mm_boot_params_t *params);
    bool mm_ready(void);

    uint64_t mm_total_ram(void);
    uint64_t mm_free_ram(void);

    void mm_dump(void);

#endif

kernel_Workspace/kernel/src/mm/core/mm.h (line 27) - [public] mm_ready
    bool mm_ready(void);

    uint64_t mm_total_ram(void);
    uint64_t mm_free_ram(void);

    void mm_dump(void);

#endif

kernel_Workspace/kernel/src/mm/core/mm.h (line 29) - [public] mm_total_ram
    uint64_t mm_total_ram(void);
    uint64_t mm_free_ram(void);

    void mm_dump(void);

#endif

kernel_Workspace/kernel/src/mm/core/mm.h (line 30) - [public] mm_free_ram
    uint64_t mm_free_ram(void);

    void mm_dump(void);

#endif

kernel_Workspace/kernel/src/mm/core/mm.h (line 32) - [public] mm_dump
    void mm_dump(void);

#endif

kernel_Workspace/kernel/src/mm/core/range.h (line 18) - [public] range_is_canonical
    bool range_is_canonical(uint64_t addr);

#endif

```

kernel_Workspace/kernel/src/mm/core/region.h (line 13) - [public] region_make

```
region_t region_make(uint64_t start, uint64_t len);
bool region_valid(region_t r);
uint64_t region_len(region_t r);

bool region_contains(region_t r, uint64_t addr);
bool region_overlaps(region_t a, region_t b);
region_t region_intersect(region_t a, region_t b);

size_t region_subtract(region_t r,
                        region_t cut,
                        region_t *out,
                        size_t max_out);
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/region.h (line 14) - [public] region_valid

```
bool region_valid(region_t r);
uint64_t region_len(region_t r);

bool region_contains(region_t r, uint64_t addr);
bool region_overlaps(region_t a, region_t b);
region_t region_intersect(region_t a, region_t b);

size_t region_subtract(region_t r,
                        region_t cut,
                        region_t *out,
                        size_t max_out);

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/mm/core/region.h (line 15) - [public] region_len

```
uint64_t region_len(region_t r);

bool region_contains(region_t r, uint64_t addr);
bool region_overlaps(region_t a, region_t b);
region_t region_intersect(region_t a, region_t b);

size_t region_subtract(region_t r,
                        region_t cut,
                        region_t *out,
                        size_t max_out);

#endif
```

kernel_Workspace/kernel/src/mm/core/region.h (line 17) - [public] region_contains

```
bool region_contains(region_t r, uint64_t addr);
bool region_overlaps(region_t a, region_t b);
region_t region_intersect(region_t a, region_t b);

size_t region_subtract(region_t r,
                        region_t cut,
                        region_t *out,
                        size_t max_out);

#endif
```

kernel_Workspace/kernel/src/mm/core/region.h (line 18) - [public] region_overlaps

```
bool region_overlaps(region_t a, region_t b);
region_t region_intersect(region_t a, region_t b);

size_t region_subtract(region_t r,
                        region_t cut,
                        region_t *out,
                        size_t max_out);

#endif
```

```

kernel_Workspace/kernel/src/mm/core/region.h (line 19) - [public] region_intersect
    region_t region_intersect(region_t a, region_t b);

    size_t region_subtract(region_t r,
                           region_t cut,
                           region_t *out,
                           size_t max_out);

#endif

kernel_Workspace/kernel/src/mm/core/sizeutil.h (line 8) - [public] size_is_pow2
    bool size_is_pow2(size_t v);

    bool size_round_up_pow2(size_t v, size_t *out);

    bool kfree_pages_validate(size_t pages,
                              size_t page_size,
                              size_t usable_bytes,
                              size_t *out_requested_bytes);

    bool size_bytes_to_pages_checked(size_t bytes,
                                     // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/core/sizeutil.h (line 11) - [public] size_round_up_pow2
    bool size_round_up_pow2(size_t v, size_t *out);

    bool kfree_pages_validate(size_t pages,
                              size_t page_size,
                              size_t usable_bytes,
                              size_t *out_requested_bytes);

    bool size_bytes_to_pages_checked(size_t bytes,
                                     size_t page_size,
                                     size_t *out_pages);

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/core/smp_lock.h (line 21) - [public] smp_lock_init
    void smp_lock_init(smp_ticket_lock_t *lock);

    void smp_lock_acquire(smp_ticket_lock_t *lock);

    bool smp_lock_release(smp_ticket_lock_t *lock);

    int64_t smp_current_cpu_id(void);

#endif

kernel_Workspace/kernel/src/mm/core/smp_lock.h (line 24) - [public] smp_lock_acquire
    void smp_lock_acquire(smp_ticket_lock_t *lock);

    bool smp_lock_release(smp_ticket_lock_t *lock);

    int64_t smp_current_cpu_id(void);

#endif

```

```

kernel_Workspace/kernel/src/mm/core/smp_lock.h (line 27) - [public] smp_lock_release
    bool smp_lock_release(smp_ticket_lock_t *lock);

    int64_t smp_current_cpu_id(void);

#endif

kernel_Workspace/kernel/src/mm/core/smp_lock.h (line 30) - [public] smp_current_cpu_id
    int64_t smp_current_cpu_id(void);

#endif

kernel_Workspace/kernel/src/mm/paging/paging.h (line 19) - [public] paging_subsystem_init
    bool paging_subsystem_init(void);

    address_space_t *paging_aspace_create(void);
    void paging_aspace_destroy(address_space_t *as);
    void paging_aspace_switch(address_space_t *as);
    uint64_t paging_aspace_cr3(const address_space_t *as);

    bool paging_aspace_map(address_space_t *as,
                           uint64_t virt,
                           uint64_t phys,
                           size_t len,
                           uint32_t prot);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/paging/paging.h (line 22) - [public] paging_aspace_destroy
    void paging_aspace_destroy(address_space_t *as);
    void paging_aspace_switch(address_space_t *as);
    uint64_t paging_aspace_cr3(const address_space_t *as);

    bool paging_aspace_map(address_space_t *as,
                           uint64_t virt,
                           uint64_t phys,
                           size_t len,
                           uint32_t prot);

    bool paging_aspace_unmap(address_space_t *as, uint64_t virt, size_t len);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/paging/paging.h (line 23) - [public] paging_aspace_switch
    void paging_aspace_switch(address_space_t *as);
    uint64_t paging_aspace_cr3(const address_space_t *as);

    bool paging_aspace_map(address_space_t *as,
                           uint64_t virt,
                           uint64_t phys,
                           size_t len,
                           uint32_t prot);

    bool paging_aspace_unmap(address_space_t *as, uint64_t virt, size_t len);

    bool paging_aspace_protect(address_space_t *as,
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/paging/paging.h (line 24) - [public] paging_aspace_cr3
uint64_t paging_aspace_cr3(const address_space_t *as);

bool paging_aspace_map(address_space_t *as,
                        uint64_t virt,
                        uint64_t phys,
                        size_t len,
                        uint32_t prot);

bool paging_aspace_unmap(address_space_t *as, uint64_t virt, size_t len);

bool paging_aspace_protect(address_space_t *as,
                            uint64_t virt,
                            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/paging/paging.h (line 32) - [public] paging_aspace_unmap
bool paging_aspace_unmap(address_space_t *as, uint64_t virt, size_t len);

bool paging_aspace_protect(address_space_t *as,
                            uint64_t virt,
                            size_t len,
                            uint32_t prot);

uint64_t paging_aspace_translate(address_space_t *as, uint64_t virt);

bool paging_kernel_map(uint64_t virt, uint64_t phys, size_t len, uint32_t prot);
bool paging_kernel_unmap(uint64_t virt, size_t len);

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/paging/paging.h (line 39) - [public] paging_aspace_translate
uint64_t paging_aspace_translate(address_space_t *as, uint64_t virt);

bool paging_kernel_map(uint64_t virt, uint64_t phys, size_t len, uint32_t prot);
bool paging_kernel_unmap(uint64_t virt, size_t len);

#endif

kernel_Workspace/kernel/src/mm/paging/paging.h (line 41) - [public] paging_kernel_map
bool paging_kernel_map(uint64_t virt, uint64_t phys, size_t len, uint32_t prot);
bool paging_kernel_unmap(uint64_t virt, size_t len);

#endif

kernel_Workspace/kernel/src/mm/paging/paging.h (line 42) - [public] paging_kernel_unmap
bool paging_kernel_unmap(uint64_t virt, size_t len);

#endif

kernel_Workspace/kernel/src/mm/paging/pml.h (line 14) - [public] pml_level_shift
uint64_t pml_level_shift(int level);
uint64_t pml_index(uint64_t virt, int level);

bool pml_entry_present(uint64_t entry);
bool pml_entry_large(uint64_t entry);
uint64_t pml_entry_addr(uint64_t entry);
uint64_t *pml_table_ptr(uint64_t table_phys);

#endif

kernel_Workspace/kernel/src/mm/paging/pml.h (line 15) - [public] pml_index
uint64_t pml_index(uint64_t virt, int level);

bool pml_entry_present(uint64_t entry);
bool pml_entry_large(uint64_t entry);
uint64_t pml_entry_addr(uint64_t entry);
uint64_t *pml_table_ptr(uint64_t table_phys);

#endif

```



```

kernel_Workspace/kernel/src/mm/paging/pml.h (line 17) - [public] pml_entry_present
    bool pml_entry_present(uint64_t entry);
    bool pml_entry_large(uint64_t entry);
    uint64_t pml_entry_addr(uint64_t entry);
    uint64_t *pml_table_ptr(uint64_t table_phys);

    #endif

kernel_Workspace/kernel/src/mm/paging/pml.h (line 18) - [public] pml_entry_large
    bool pml_entry_large(uint64_t entry);
    uint64_t pml_entry_addr(uint64_t entry);
    uint64_t *pml_table_ptr(uint64_t table_phys);

    #endif

kernel_Workspace/kernel/src/mm/paging/pml.h (line 19) - [public] pml_entry_addr
    uint64_t pml_entry_addr(uint64_t entry);
    uint64_t *pml_table_ptr(uint64_t table_phys);

    #endif

kernel_Workspace/kernel/src/mm/paging/tlb.h (line 16) - [public] tlb_batch_begin
    void tlb_batch_begin(tlb_batch_t *batch);
    void tlb_batch_add(tlb_batch_t *batch, uint64_t virt);
    void tlb_batch_full(tlb_batch_t *batch);
    void tlb_batch_commit(tlb_batch_t *batch);

    #endif

kernel_Workspace/kernel/src/mm/paging/tlb.h (line 17) - [public] tlb_batch_add
    void tlb_batch_add(tlb_batch_t *batch, uint64_t virt);
    void tlb_batch_full(tlb_batch_t *batch);
    void tlb_batch_commit(tlb_batch_t *batch);

    #endif

kernel_Workspace/kernel/src/mm/paging/tlb.h (line 18) - [public] tlb_batch_full
    void tlb_batch_full(tlb_batch_t *batch);
    void tlb_batch_commit(tlb_batch_t *batch);

    #endif

kernel_Workspace/kernel/src/mm/paging/tlb.h (line 19) - [public] tlb_batch_commit
    void tlb_batch_commit(tlb_batch_t *batch);

    #endif

kernel_Workspace/kernel/src/mm/process/address_space.h (line 30) - [public]
aspace_subsystem_init
    bool aspace_subsystem_init(void);

    proc_aspace_t *aspace_create(void);
    void aspace_destroy(proc_aspace_t *pa);

    address_space_t *aspace_paging_handle(proc_aspace_t *pa);

    uint64_t aspace_map_anon(proc_aspace_t *pa, uint64_t hint, size_t len, uint32_t prot);
    uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);
    uint64_t aspace_reserve_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t flags);
    bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len);
    bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);
    // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/mm/process/address_space.h (line 33) - [public] aspace_destroy
void aspace_destroy(proc_aspace_t *pa);

address_space_t *aspace_paging_handle(proc_aspace_t *pa);

uint64_t aspace_map_anon(proc_aspace_t *pa, uint64_t hint, size_t len, uint32_t prot);
uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);
uint64_t aspace_reserve_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t flags);
bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len);
bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
                           uint32_t checked_prot,
                           // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/process/address_space.h (line 37) - [public] aspace_map_anon
uint64_t aspace_map_anon(proc_aspace_t *pa, uint64_t hint, size_t len, uint32_t prot);
uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);
uint64_t aspace_reserve_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t flags);
bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len);
bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
                           uint32_t checked_prot,
                           uint32_t apply_prot,
                           bool (*allowed)(uint32_t old_prot,
                                             uint32_t checked_prot));

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/process/address_space.h (line 38) - [public] aspace_map_at
uint64_t aspace_map_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);
uint64_t aspace_reserve_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t flags);
bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len);
bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
                           uint32_t checked_prot,
                           uint32_t apply_prot,
                           bool (*allowed)(uint32_t old_prot,
                                             uint32_t checked_prot));

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/mm/process/address_space.h (line 39) - [public] aspace_reserve_at
uint64_t aspace_reserve_at(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t flags);
bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len);
bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
                           uint32_t checked_prot,
                           uint32_t apply_prot,
                           bool (*allowed)(uint32_t old_prot,
                                             uint32_t checked_prot));

vma_t *aspace_vma_find(proc_aspace_t *pa, uint64_t addr);
// ... (truncated, see source file for more)

```

```
kernel_Workspace/kernel/src/mm/process/address_space.h (line 40) - [public] aspace_unmap
    bool aspace_unmap(proc_aspace_t *pa, uint64_t addr, size_t len);
    bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

    bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
                                uint32_t checked_prot,
                                uint32_t apply_prot,
                                bool (*allowed)(uint32_t old_prot,
                                                  uint32_t checked_prot));

    vma_t *aspace_vma_find(proc_aspace_t *pa, uint64_t addr);
    uint64_t aspace_stack_base(void);
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/process/address_space.h (line 41) - [public] aspace_protect
    bool aspace_protect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

    bool aspace_protect_checked(proc_aspace_t *pa, uint64_t addr, size_t len,
                                uint32_t checked_prot,
                                uint32_t apply_prot,
                                bool (*allowed)(uint32_t old_prot,
                                                  uint32_t checked_prot));

    vma_t *aspace_vma_find(proc_aspace_t *pa, uint64_t addr);
    uint64_t aspace_stack_base(void);

    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/mm/process/address_space.h (line 51) - [public] aspace_stack_base
    uint64_t aspace_stack_base(void);

    uint64_t aspace_brk(proc_aspace_t *pa, uint64_t new_brk);

#endif
```

```
kernel_Workspace/kernel/src/mm/process/address_space.h (line 53) - [public] aspace_brk
    uint64_t aspace_brk(proc_aspace_t *pa, uint64_t new_brk);

#endif
```

```
kernel_Workspace/kernel/src/mm/process/mmap.h (line 20) - [public] munmap
    bool munmap(proc_aspace_t *pa, uint64_t addr, size_t len);
    bool mprotect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

#endif
```

```
kernel_Workspace/kernel/src/mm/process/mmap.h (line 21) - [public] mprotect
    bool mprotect(proc_aspace_t *pa, uint64_t addr, size_t len, uint32_t prot);

#endif
```

```
kernel_Workspace/kernel/src/mm/protection/guard.h (line 9) - [public] guard_install
    uint64_t guard_install(proc_aspace_t *pa, uint64_t addr, size_t len);
    bool guard_user_stack(proc_aspace_t *pa);

#endif
```

```
kernel_Workspace/kernel/src/mm/protection/guard.h (line 10) - [public] guard_user_stack
    bool guard_user_stack(proc_aspace_t *pa);

#endif
```

kernel_Workspace/kernel/src/mm/protection/isolation.h (line 8) - [public] isolation_init

```
bool isolation_init(void);

bool isolation_has_smep(void);
bool isolation_has_smap(void);
bool isolation_enable_smep(void);
bool isolation_enable_smap(void);

size_t isolation_audit_kernel(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/isolation.h (line 10) - [public] isolation_has_smep

```
bool isolation_has_smep(void);
bool isolation_has_smap(void);
bool isolation_enable_smep(void);
bool isolation_enable_smap(void);

size_t isolation_audit_kernel(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/isolation.h (line 11) - [public] isolation_has_smap

```
bool isolation_has_smap(void);
bool isolation_enable_smep(void);
bool isolation_enable_smap(void);

size_t isolation_audit_kernel(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/isolation.h (line 12) - [public]
isolation_enable_smep

```
bool isolation_enable_smep(void);
bool isolation_enable_smap(void);

size_t isolation_audit_kernel(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/isolation.h (line 13) - [public]
isolation_enable_smap

```
bool isolation_enable_smap(void);

size_t isolation_audit_kernel(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/isolation.h (line 15) - [public]
isolation_audit_kernel

```
size_t isolation_audit_kernel(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/permissions.h (line 8) - [public] perm_sanitiz

```
uint32_t perm_sanitiz(uint32_t prot);
bool perm_is_wx(uint32_t prot);
bool perm_mprotect_allowed(uint32_t old_prot, uint32_t new_prot);
void perm_set_strict_wx(bool on);

uint32_t perm_kernel_default(void);
uint32_t perm_user_default(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/permissions.h (line 9) - [public] perm_is_wx

```
bool perm_is_wx(uint32_t prot);
bool perm_mprotect_allowed(uint32_t old_prot, uint32_t new_prot);
void perm_set_strict_wx(bool on);

uint32_t perm_kernel_default(void);
uint32_t perm_user_default(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/permissions.h (line 10) - [public] perm_mprotect_allowed

```
bool perm_mprotect_allowed(uint32_t old_prot, uint32_t new_prot);
void perm_set_strict_wx(bool on);

uint32_t perm_kernel_default(void);
uint32_t perm_user_default(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/permissions.h (line 11) - [public] perm_set_strict_wx

```
void perm_set_strict_wx(bool on);

uint32_t perm_kernel_default(void);
uint32_t perm_user_default(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/permissions.h (line 13) - [public] perm_kernel_default

```
uint32_t perm_kernel_default(void);
uint32_t perm_user_default(void);

#endif
```

kernel_Workspace/kernel/src/mm/protection/permissions.h (line 14) - [public] perm_user_default

```
uint32_t perm_user_default(void);

#endif
```

kstd/include/kstd.h (line 12) - [public] tr_log

```
void tr_log(const char *s);

#endif
```

kstd/include/kstd_audio.h (line 7) - [public] tr_free

```
void tr_free(void *ptr, uint32_t bytes);
tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va);

#endif
```

kstd/include/kstd_audio.h (line 8) - [public] tr_map_mmio

```
tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va);

#endif
```

kstd/include/kstd_bt.h (line 6) - [public] tr_bt_info

```
tr_status_t tr_bt_info(uint8_t *hci_ver, uint8_t *addr6);
tr_status_t tr_bt_cmd(uint16_t opcode, const void *params, uint8_t len);
tr_status_t tr_bt_evt(void *buf, uint8_t *len);
tr_status_t tr_bt_acl_send(const void *data, uint16_t len);
tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len);

#endif
```

```

kstd/include/kstd_bt.h (line 7) - [public] tr_bt_cmd
    tr_status_t tr_bt_cmd(uint16_t opcode, const void *params, uint8_t len);
    tr_status_t tr_bt_evt(void *buf, uint8_t *len);
    tr_status_t tr_bt_acl_send(const void *data, uint16_t len);
    tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len);

    #endif

kstd/include/kstd_bt.h (line 8) - [public] tr_bt_evt
    tr_status_t tr_bt_evt(void *buf, uint8_t *len);
    tr_status_t tr_bt_acl_send(const void *data, uint16_t len);
    tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len);

    #endif

kstd/include/kstd_bt.h (line 9) - [public] tr_bt_acl_send
    tr_status_t tr_bt_acl_send(const void *data, uint16_t len);
    tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len);

    #endif

kstd/include/kstd_bt.h (line 10) - [public] tr_bt_acl_recv
    tr_status_t tr_bt_acl_recv(void *buf, uint16_t *len);

    #endif

kstd/include/kstd_fs.h (line 6) - [public] tr_fs_read
    tr_status_t tr_fs_read(const char *path, void *buf, uint32_t cap, uint32_t *out);
    tr_status_t tr_fs_exists(const char *path);

    #endif

kstd/include/kstd_fs.h (line 7) - [public] tr_fs_exists
    tr_status_t tr_fs_exists(const char *path);

    #endif

kstd/include/kstd_input.h (line 6) - [public] tr_input_key
    int32_t tr_input_key(void);

    #endif

kstd/include/kstd_mem.h (line 7) - [public] tr_free
    void tr_free(void *ptr, uint32_t bytes);
    tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va);

    #endif

kstd/include/kstd_mem.h (line 8) - [public] tr_map_mmio
    tr_status_t tr_map_mmio(uint64_t phys, uint32_t len, void **out_va);

    #endif

kstd/include/kstd_video.h (line 22) - [public] tr_video_init
    /* Inicjalizacja i stan */
    tr_status_t tr_video_init(void);
    bool tr_video_is_ready(void);
    tr_status_t tr_video_get_caps(tr_video_caps_t *caps);

    /* Zarz■dzenie trybami */
    uint32_t tr_video_mode_count(void);
    tr_status_t tr_video_mode_get(uint32_t index, tr_video_mode_t *mode);
    tr_status_t tr_video_mode_set(uint32_t mode_id);

    /* Operacje na framebufferze */
    tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
    tr_status_t tr_video_blit(const void *src, uint32_t src_stride,
        // ... (truncated, see source file for more)

```

kstd/include/kstd_video.h (line 23) - [public] tr_video_is_ready

```
bool tr_video_is_ready(void);
tr_status_t tr_video_get_caps(tr_video_caps_t *caps);

/* Zarządzenie trybami */
uint32_t tr_video_mode_count(void);
tr_status_t tr_video_mode_get(uint32_t index, tr_video_mode_t *mode);
tr_status_t tr_video_mode_set(uint32_t mode_id);

/* Operacje na framebufferze */
tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
tr_status_t tr_video_blit(const void *src, uint32_t src_stride,
                        uint32_t dst_x, uint32_t dst_y, uint32_t w, uint32_t h);
// ... (truncated, see source file for more)
```

kstd/include/kstd_video.h (line 24) - [public] tr_video_get_caps

```
tr_status_t tr_video_get_caps(tr_video_caps_t *caps);

/* Zarządzenie trybami */
uint32_t tr_video_mode_count(void);
tr_status_t tr_video_mode_get(uint32_t index, tr_video_mode_t *mode);
tr_status_t tr_video_mode_set(uint32_t mode_id);

/* Operacje na framebufferze */
tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
tr_status_t tr_video_blit(const void *src, uint32_t src_stride,
                        uint32_t dst_x, uint32_t dst_y, uint32_t w, uint32_t h);

// ... (truncated, see source file for more)
```

kstd/include/kstd_video.h (line 27) - [public] tr_video_mode_count

```
/* Zarządzenie trybami */

uint32_t tr_video_mode_count(void);
tr_status_t tr_video_mode_get(uint32_t index, tr_video_mode_t *mode);
tr_status_t tr_video_mode_set(uint32_t mode_id);

/* Operacje na framebufferze */
tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
tr_status_t tr_video_blit(const void *src, uint32_t src_stride,
                        uint32_t dst_x, uint32_t dst_y, uint32_t w, uint32_t h);

#endif
```

kstd/include/kstd_video.h (line 28) - [public] tr_video_mode_get

```
tr_status_t tr_video_mode_get(uint32_t index, tr_video_mode_t *mode);
tr_status_t tr_video_mode_set(uint32_t mode_id);

/* Operacje na framebufferze */
tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
tr_status_t tr_video_blit(const void *src, uint32_t src_stride,
                        uint32_t dst_x, uint32_t dst_y, uint32_t w, uint32_t h);

#endif
```

kstd/include/kstd_video.h (line 29) - [public] tr_video_mode_set

```
tr_status_t tr_video_mode_set(uint32_t mode_id);

/* Operacje na framebufferze */
tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
tr_status_t tr_video_blit(const void *src, uint32_t src_stride,
                        uint32_t dst_x, uint32_t dst_y, uint32_t w, uint32_t h);

#endif
```

kstd/include/kstd_video.h (line 32) - [public] tr_video_fill

/ Operacje na framebufferze */*

```
tr_status_t tr_video_fill(uint32_t color, uint32_t x, uint32_t y, uint32_t w, uint32_t h);
tr_status_t tr_video_blit(const void *src, uint32_t src_stride,
                        uint32_t dst_x, uint32_t dst_y, uint32_t w, uint32_t h);
```

#endif

libs/dsabi.h (line 93) - [public] volatile

```
__asm__ volatile("int $0x80" :: "a"(1));
}
```

```
typedef struct {
    uint64_t id;
    uint32_t cmd;
    uint32_t flags;
    uint64_t arg0;
    uint64_t arg1;
    uint64_t arg2;
    int32_t status;
    uint32_t pad;
    // ... (truncated, see source file for more)
```

libs/dsabi.h (line 124) - [public] ds_init

```
void ds_init(uint64_t params_va);
uint64_t ds_call(uint32_t cls, uint32_t op,
                uint64_t a0, uint64_t a1, uint64_t a2);
int ds_take(uint64_t id, ds_msg_t *out);
void ds_poll(void);
```

#endif

libs/dsabi.h (line 127) - [public] ds_take

```
int ds_take(uint64_t id, ds_msg_t *out);
void ds_poll(void);
```

#endif

libs/dsabi.h (line 128) - [public] ds_poll

```
void ds_poll(void);
```

#endif

libs/dsasync.h (line 21) - [public] if

```
if (r->done) {
    return 1;
}

ds_poll();

if (r->id != 0 && ds_take(r->id, &r->msg)) {
    r->done = 1;
}

return r->done;
}

// ... (truncated, see source file for more)
```



```

libs/dsasync.h (line 25) - [public] ds_poll
    ds_poll();

    if (r->id != 0 && ds_take(r->id, &r->msg)) {
        r->done = 1;
    }

    return r->done;
}

static inline int ds_req_ok(ds_req_t *r)
{
    return r->done && r->msg.status == 0;
    // ... (truncated, see source file for more)

libs/dsasync.h (line 27) - [public] if
    if (r->id != 0 && ds_take(r->id, &r->msg)) {
        r->done = 1;
    }

    return r->done;
}

static inline int ds_req_ok(ds_req_t *r)
{
    return r->done && r->msg.status == 0;
}

    // ... (truncated, see source file for more)

libs/dsaudio.h (line 7) - [public] ds_call
    return ds_call(SVC_AUDIO, AUD_PLAY, phys, len, 0);
}

static inline void ds_audio_stop(void)
{
    ds_call(SVC_AUDIO, AUD_STOP, 0, 0, 0);
}

static inline uint64_t ds_audio_jack_req(void)
{
    return ds_call(SVC_AUDIO, AUD_JACK, 0, 0, 0);
}

    // ... (truncated, see source file for more)

libs/dsaudio.h (line 12) - [public] ds_call
    ds_call(SVC_AUDIO, AUD_STOP, 0, 0, 0);
}

static inline uint64_t ds_audio_jack_req(void)
{
    return ds_call(SVC_AUDIO, AUD_JACK, 0, 0, 0);
}

static inline void ds_audio_amp(int on)
{
    ds_call(SVC_AUDIO, AUD_AMP, on ? 1 : 0, 0, 0);
}

    // ... (truncated, see source file for more)

libs/dsaudio.h (line 17) - [public] ds_call
    return ds_call(SVC_AUDIO, AUD_JACK, 0, 0, 0);
}

static inline void ds_audio_amp(int on)
{
    ds_call(SVC_AUDIO, AUD_AMP, on ? 1 : 0, 0, 0);
}

#endif

```

```

libs/dsaudio.h (line 22) - [public] ds_call
    ds_call(SVC_AUDIO, AUD_AMP, on ? 1 : 0, 0, 0);
}

#endif

libs/dsinput.h (line 7) - [public] ds_call
    return ds_call(SVC_INPUT, IN_KEY_POLL, 0, 0, 0);
}

static inline int ds_key_take(uint64_t id, uint8_t *out)
{
    ds_msg_t m;

    if (!ds_take(id, &m) || m.arg0 == 0) {
        return 0;
    }

    *out = (uint8_t)m.arg0;
    // ... (truncated, see source file for more)

libs/dsinput.h (line 14) - [public] if
    if (!ds_take(id, &m) || m.arg0 == 0) {
        return 0;
    }

    *out = (uint8_t)m.arg0;
    return 1;
}

#endif

libs/dslock.h (line 7) - [public] ds_call
    return ds_call(SVC_BLOCK, BLK_COUNT, disk, 0, 0);
}

static inline uint64_t ds_blk_read(uint64_t disk, uint64_t block)
{
    return ds_call(SVC_BLOCK, BLK_READ, disk, block, 0);
}

static inline uint64_t ds_blk_write(uint64_t disk, uint64_t block)
{
    return ds_call(SVC_BLOCK, BLK_WRITE, disk, block, 0);
}

// ... (truncated, see source file for more)

libs/dslock.h (line 12) - [public] ds_call
    return ds_call(SVC_BLOCK, BLK_READ, disk, block, 0);
}

static inline uint64_t ds_blk_write(uint64_t disk, uint64_t block)
{
    return ds_call(SVC_BLOCK, BLK_WRITE, disk, block, 0);
}

static inline volatile uint8_t *ds_blk_scratch(void)
{
    return (volatile uint8_t *)DS_SCRATCH_VA;
}

// ... (truncated, see source file for more)

```

```

libs/dslock.h (line 17) - [public] ds_call
    return ds_call(SVC_BLOCK, BLK_WRITE, disk, block, 0);
}

static inline volatile uint8_t *ds_blk_scratch(void)
{
    return (volatile uint8_t *)DS_SCRATCH_VA;
}

#endif

libs/dslog.h (line 10) - [public] while
    while (s[n] != '\0' && n < DS_SCRATCH_SIZE - 1) {
        scratch[n] = s[n];
        n++;
    }

    ds_call(SVC_SYS, OP_LOG, n, 0, 0);
}

#endif

libs/dslog.h (line 15) - [public] ds_call
    ds_call(SVC_SYS, OP_LOG, n, 0, 0);
}

#endif

libs/dsmem.h (line 7) - [public] ds_call
    return ds_call(SVC_SYS, OP_ALLOC, pages, 0, 0);
}

static inline uint64_t ds_free_pages(uint64_t va)
{
    return ds_call(SVC_SYS, OP_FREE, va, 0, 0);
}

static inline uint64_t ds_map_mmio(uint64_t phys, uint64_t len, uint64_t va)
{
    return ds_call(SVC_SYS, OP_MAPMMIO, phys, len, va);
}

// ... (truncated, see source file for more)

libs/dsmem.h (line 12) - [public] ds_call
    return ds_call(SVC_SYS, OP_FREE, va, 0, 0);
}

static inline uint64_t ds_map_mmio(uint64_t phys, uint64_t len, uint64_t va)
{
    return ds_call(SVC_SYS, OP_MAPMMIO, phys, len, va);
}

static inline uint64_t ds_page_phys(uint64_t va)
{
    return ds_call(SVC_SYS, OP_PAGEPHYS, va, 0, 0);
}

// ... (truncated, see source file for more)

libs/dsmem.h (line 17) - [public] ds_call
    return ds_call(SVC_SYS, OP_MAPMMIO, phys, len, va);
}

static inline uint64_t ds_page_phys(uint64_t va)
{
    return ds_call(SVC_SYS, OP_PAGEPHYS, va, 0, 0);
}

#endif

```

```

libs/dsmem.h (line 22) - [public] ds_call
    return ds_call(SVC_SYS, OP_PAGEPHYS, va, 0, 0);
}

#endif

libs/dsvgpu.h (line 6) - [public] ds_call
extern uint64_t ds_call(uint32_t, uint32_t, uint64_t, uint64_t, uint64_t);
extern void ds_poll(void);
extern int ds_take(uint64_t, ds_msg_t *);

static inline int dsvgpu_surface_create(uint32_t w, uint32_t h,
                                         uint64_t *phys, int32_t *sid)
{
    uint64_t id = ds_call(SVC_VGPU, VGPU_SURF_CREATE, w, h, 0);
    ds_poll();

    ds_msg_t r;

    // ... (truncated, see source file for more)
}

libs/dsvgpu.h (line 7) - [public] ds_poll
extern void ds_poll(void);
extern int ds_take(uint64_t, ds_msg_t *);

static inline int dsvgpu_surface_create(uint32_t w, uint32_t h,
                                         uint64_t *phys, int32_t *sid)
{
    uint64_t id = ds_call(SVC_VGPU, VGPU_SURF_CREATE, w, h, 0);
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        // ... (truncated, see source file for more)
    }
}

libs/dsvgpu.h (line 8) - [public] ds_take
extern int ds_take(uint64_t, ds_msg_t *);

static inline int dsvgpu_surface_create(uint32_t w, uint32_t h,
                                         uint64_t *phys, int32_t *sid)
{
    uint64_t id = ds_call(SVC_VGPU, VGPU_SURF_CREATE, w, h, 0);
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return 0;
        // ... (truncated, see source file for more)
    }
}

libs/dsvgpu.h (line 14) - [public] ds_poll
    ds_poll();

    ds_msg_t r;

    if (!ds_take(id, &r) || r.status != 0) {
        return 0;
    }

    *sid = (int32_t)r.arg0;
    *phys = r.arg1;
    return 1;
}

// ... (truncated, see source file for more)

```

```

libs/dsvgpu.h (line 18) - [public] if
    if (!ds_take(id, &r) || r.status != 0) {
        return 0;
    }

    *sid = (int32_t)r.arg0;
    *phys = r.arg1;
    return 1;
}

static inline int dsvgpu_present(int32_t sid, uint32_t x, uint32_t y)
{
    uint64_t id = ds_call(SVC_VGPU, VGPU_PRESENT,
        // ... (truncated, see source file for more)

libs/dsvgpu.h (line 31) - [public] ds_poll
    ds_poll();

    ds_msg_t r;
    return ds_take(id, &r) && r.status == 0;
}

#endif

libs/dsvideo.h (line 14) - [public] ds_call
    return ds_call(SVC_VIDEO, VID_FB_INFO, 0, 0, 0);
}

static inline int ds_fb_info_take(uint64_t id, ds_fb_t *fb)
{
    ds_msg_t m;

    if (!ds_take(id, &m) || m.status != 0) {
        return 0;
    }

    fb->w = (uint32_t)(m.arg0 >> 16);
    // ... (truncated, see source file for more)

libs/dsvideo.h (line 21) - [public] if
    if (!ds_take(id, &m) || m.status != 0) {
        return 0;
    }

    fb->w = (uint32_t)(m.arg0 >> 16);
    fb->h = (uint32_t)(m.arg0 & 0xFFFF);
    fb->stride = (uint32_t)m.arg1;
    fb->phys = m.arg2;

    return 1;
}

    // ... (truncated, see source file for more)

libs/dsvideo.h (line 35) - [public] ds_call
    ds_call(SVC_VIDEO, VID_FB_TAKEOVER, 0, 0, 0);
}

static inline void ds_fb_release(void)
{
    ds_call(SVC_VIDEO, VID_FB_RELEASE, 0, 0, 0);
}

static inline void ds_px(volatile uint32_t *fb, uint32_t stride,
    uint32_t x, uint32_t y, uint32_t c)
{
    fb[(uint64_t)y * stride + x] = c;
    // ... (truncated, see source file for more)

```

```

libs/dsvideo.h (line 40) - [public] ds_call
    ds_call(SVC_VIDEO, VID_FB_RELEASE, 0, 0, 0);
}

static inline void ds_px(volatile uint32_t *fb, uint32_t stride,
                        uint32_t x, uint32_t y, uint32_t c)
{
    fb[(uint64_t)y * stride + x] = c;
}

static inline uint32_t ds_rgb(uint8_t r, uint8_t g, uint8_t b)
{
    return 0xFF000000u | ((uint32_t)r << 16) | ((uint32_t)g << 8) | b;
    // ... (truncated, see source file for more)

```

Nim

kernel_Workspace/kernel/src/nimcore/banner.nim (line 13) - [internal] nim_banner

```

proc nim_banner(buf: ptr uint8, cap: uint32): uint32 {.exportc, cdecl.} =
    var o = 0u32

    for ch in ART:
        if o >= cap:
            break

        buf[o] = uint8(ch)
        o += 1

    return o

```

kernel_Workspace/kernel/src/nimcore/format.nim (line 1) - [internal] nim_u64_to_str

```

proc nim_u64_to_str(v: uint64, base: uint8, buf: ptr uint8, cap: uint32): uint32
    {.exportc, cdecl.} =
        var tmp: array[64, uint8]
        var n = 0u32
        var x = v

        let b = if base < 2: 10u64 else: uint64(base)

        if x == 0:
            tmp[0] = uint8('0')
            n = 1
        else:
            // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/nimcore/format.nim (line 29) - [internal] nim_hex_dump

```

proc nim_hex_dump(src: ptr uint8, len: uint32, buf: ptr uint8, cap: uint32): uint32
    {.exportc, cdecl.} =
        const hexd = "0123456789abcdef"
        var o = 0u32
        var i = 0u32

        while i < len and o + 2 < cap:
            let b = src[i]
            buf[o] = uint8(hexd[int(b shr 4)])
            buf[o + 1] = uint8(hexd[int(b and 15)])
            o += 2
            i += 1
            // ... (truncated, see source file for more)

```

```

kernel_Workspace/kernel/src/nimcore/parse.nim (line 1) - [internal] nim_parse_u64
proc nim_parse_u64(s: ptr uint8, len: uint32, base: uint8, out_v: ptr uint64): uint8
  {.exportc, cdecl.} =
    var v = 0u64
    var i = 0u32
    let b = uint64(base)

    if len == 0: return 0

    if base == 16 and len > 2 and s[0] == uint8('0') and (s[1] == uint8('x') or s[1] == uint8('X')):
      i = 2

    if i >= len: return 0
      // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/nimcore/ringbuf.nim (line 1) - [internal] Rb
type Rb = object
  data: array[1024, uint8]
  head: uint32
  tail: uint32

var key_rb: Rb

proc nim_rb_push(b: uint8): uint8 {.exportc, cdecl.} =
  let next = (key_rb.head + 1) mod 1024
  if next == key_rb.tail: return 0
  key_rb.data[key_rb.head] = b
  key_rb.head = next
  // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/nimcore/ringbuf.nim (line 8) - [internal] nim_rb_push
proc nim_rb_push(b: uint8): uint8 {.exportc, cdecl.} =
  let next = (key_rb.head + 1) mod 1024
  if next == key_rb.tail: return 0
  key_rb.data[key_rb.head] = b
  key_rb.head = next
  return 1

proc nim_rb_pop(): int32 {.exportc, cdecl.} =
  if key_rb.head == key_rb.tail: return -1
  let b = key_rb.data[key_rb.tail]
  key_rb.tail = (key_rb.tail + 1) mod 1024
  return int32(b)
  // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/nimcore/ringbuf.nim (line 15) - [internal] nim_rb_pop
proc nim_rb_pop(): int32 {.exportc, cdecl.} =
  if key_rb.head == key_rb.tail: return -1
  let b = key_rb.data[key_rb.tail]
  key_rb.tail = (key_rb.tail + 1) mod 1024
  return int32(b)

proc nim_rb_len(): uint32 {.exportc, cdecl.} =
  return (key_rb.head + 1024 - key_rb.tail) mod 1024

kernel_Workspace/kernel/src/nimcore/ringbuf.nim (line 21) - [internal] nim_rb_len
proc nim_rb_len(): uint32 {.exportc, cdecl.} =
  return (key_rb.head + 1024 - key_rb.tail) mod 1024

```

kernel_Workspace/kernel/src/nimcore/shell.nim (line 1) - [internal] Handler

```
type Handler = proc(arg: ptr uint8, len: uint32): int32 {.cdecl.}

type Cmd = object
  name: array[24, uint8]
  nlen: uint32
  used: bool
  h: Handler

var cmds: array[32, Cmd]

proc nim_shell_register(name: ptr uint8, nlen: uint32, h: Handler): uint8
  {.exportc, cdecl.} =
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/nimcore/shell.nim (line 3) - [internal] Cmd

```
type Cmd = object
  name: array[24, uint8]
  nlen: uint32
  used: bool
  h: Handler

var cmds: array[32, Cmd]

proc nim_shell_register(name: ptr uint8, nlen: uint32, h: Handler): uint8
  {.exportc, cdecl.} =
    if nlen == 0 or nlen >= 24: return 0

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/nimcore/shell.nim (line 11) - [internal] nim_shell_register

```
proc nim_shell_register(name: ptr uint8, nlen: uint32, h: Handler): uint8
  {.exportc, cdecl.} =
    if nlen == 0 or nlen >= 24: return 0

    for i in 0..<32:
      if not cmds[i].used:
        var k = 0u32
        while k < nlen:
          cmds[i].name[k] = name[k]
          k += 1
        cmds[i].nlen = nlen
        cmds[i].used = true
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/nimcore/shell.nim (line 28) - [internal] eq_name

```
proc eq_name(c: var Cmd, line: ptr uint8, nlen: uint32): bool =
  if c.nlen != nlen: return false
  var k = 0u32
  while k < nlen:
    if c.name[k] != line[k]: return false
    k += 1
  return true

proc nim_shell_run(line: ptr uint8, len: uint32): int32
  {.exportc, cdecl.} =
    var sp = 0u32
    while sp < len and line[sp] != uint8(' '):
      // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/nimcore/shell.nim (line 36) - [internal] nim_shell_run

```
proc nim_shell_run(line: ptr uint8, len: uint32): int32
  {.exportc, cdecl.} =
    var sp = 0u32
    while sp < len and line[sp] != uint8(' '):
      sp += 1

    if sp == 0: return -1

    var arg: ptr uint8 = nil
    var alen = 0u32

    if sp < len:
      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/nim/policynim.nim (line 10) - [internal] nim_policy_log

```
proc nim_policy_log(ring, cls, op, dec: uint8) {.exportc, cdecl.} =
  logbuf[head] = Entry(ring: ring, cls: cls, op: op, dec: dec)
  head = (head + 1) mod 256

  if dec == 1:
    denies += 1

proc nim_policy_denies(): uint64 {.exportc, cdecl.} =
  denies

proc nim_policy_get(idx: uint32,
  out_ring, out_cls, out_op, out_dec: ptr uint8): uint8
  // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/nim/policynim.nim (line 17) - [internal] nim_policy_denies

```
proc nim_policy_denies(): uint64 {.exportc, cdecl.} =
  denies

proc nim_policy_get(idx: uint32,
  out_ring, out_cls, out_op, out_dec: ptr uint8): uint8
  {.exportc, cdecl.} =
    if idx >= 256:
      return 0

    let e = logbuf[idx]

    if out_ring != nil: out_ring[] = e.ring
      // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/policy/nim/policynim.nim (line 20) - [internal] nim_policy_get

```
proc nim_policy_get(idx: uint32,
  out_ring, out_cls, out_op, out_dec: ptr uint8): uint8
  {.exportc, cdecl.} =
    if idx >= 256:
      return 0

    let e = logbuf[idx]

    if out_ring != nil: out_ring[] = e.ring
    if out_cls != nil: out_cls[] = e.cls
    if out_op != nil: out_op[] = e.op
    if out_dec != nil: out_dec[] = e.dec
      // ... (truncated, see source file for more)
```

Odin

drivers/audiodriver/src/odin/driver.odin (line 8) - [public] Bdl_Entry

```
Bdl_Entry :: struct {
  ■addr: u32,
  ■ctrl: u32,
}

BDL_N :: 8

// mixer (NAM) — 16-bit
nam16 :: proc(off: u32) -> ^u16 {
  ■return transmute(^u16)(nam_base + u64(off))
}

// ... (truncated, see source file for more)
```

drivers/audiodriver/src/odin/driver.odin (line 16) - [public] nam16

// mixer (NAM) — 16-bit

```
nam16 :: proc(off: u32) -> ^u16 {
  ■return transmute(^u16)(nam_base + u64(off))
}

// bus master — 8/16/32
bm8 :: proc(off: u32) -> ^u8 {
  ■return transmute(^u8)(bm_base + u64(off))
}

bm16 :: proc(off: u32) -> ^u16 {
  ■return transmute(^u16)(bm_base + u64(off))
}

// ... (truncated, see source file for more)
```

drivers/audiodriver/src/odin/driver.odin (line 21) - [public] bm8

// bus master — 8/16/32

```
bm8 :: proc(off: u32) -> ^u8 {
  ■return transmute(^u8)(bm_base + u64(off))
}

bm16 :: proc(off: u32) -> ^u16 {
  ■return transmute(^u16)(bm_base + u64(off))
}

bm32 :: proc(off: u32) -> ^u32 {
  ■return transmute(^u32)(bm_base + u64(off))
}

// ... (truncated, see source file for more)
```

drivers/audiodriver/src/odin/driver.odin (line 25) - [public] bm16

```
bm16 :: proc(off: u32) -> ^u16 {
  ■return transmute(^u16)(bm_base + u64(off))
}

bm32 :: proc(off: u32) -> ^u32 {
  ■return transmute(^u32)(bm_base + u64(off))
}

GLOB_CNT  :: 0x2C
GLOB_STA  :: 0x30

PI_BDBAR  :: 0x00
// ... (truncated, see source file for more)
```

drivers/audiodriver/src/odin/driver.odin (line 29) - [public] bm32

```
bm32 :: proc(off: u32) -> ^u32 {  
  ■return transmute(^u32)(bm_base + u64(off))  
}
```

```
GLOB_CNT  :: 0x2C  
GLOB_STA  :: 0x30
```

```
PI_BDBAR  :: 0x00  
PI_LVI    :: 0x05  
PI_SR     :: 0x06  
PI_CR     :: 0x0B
```

// ... (truncated, see source file for more)

drivers/audiodriver/src/odin/driver.odin (line 50) - [public] ad_init

```
ad_init :: proc "C" (nam_va: u64, bm_va: u64) -> i32 {  
  ■nam_base = nam_va  
  ■bm_base = bm_va  
  
  ■// cold reset  
  ■mem.volatile_store(bm32(GLOB_CNT), 0x04000000)  
  
  ■for _ in 0..<100000 {  
    ■■if mem.volatile_load(bm32(GLOB_STA)) & 0x04 != 0 {  
      ■■■break  
    }  
  }  
}
```

// ... (truncated, see source file for more)

drivers/audiodriver/src/odin/driver.odin (line 73) - [public] ad_stop

```
ad_stop :: proc "C" () -> i32 {  
  ■mem.volatile_store(bm8(P0_CR), 0)  
  ■mem.volatile_store(bm8(PI_CR), 0)  
  ■return 0  
}  
  
@(export)  
ad_position :: proc "C" () -> u32 {  
  ■return u32(mem.volatile_load(bm16(P0_PICB)))  
}
```

drivers/audiodriver/src/odin/driver.odin (line 80) - [public] ad_position

```
ad_position :: proc "C" () -> u32 {  
  ■return u32(mem.volatile_load(bm16(P0_PICB)))  
}
```

drivers/audiodriver/src/odin/jack.odin (line 8) - [public] ad_jack_present

```
ad_jack_present :: proc "C" () -> i32 {  
  ■pwr := mem.volatile_load(nam16(PWR_CTRL_STA))  
  
  ■// bity ready w powerdown status = tor aktywny  
  ■if pwr & 0x000F != 0 {  
    ■■return 1  
  }  
  
  ■// QEMU i tak nie zdejmuj jacka – fallback  
  ■return 1  
}
```

// ... (truncated, see source file for more)

drivers/audiodriver/src/odin/jack.odin (line 21) - [public] ad_set_amp

```
ad_set_amp :: proc "C" (on: i32) -> i32 {  
  ■if on == 0 {  
    ■■mem.volatile_store(nam16(0x02), 0x8000) // mute  
  } else {  
    ■■mem.volatile_store(nam16(0x02), 0x0000)  
  }  
  ■return 0  
}
```

```

drivers/audiodriver/src/odin/microphone.odin (line 6) - [public] ad_capture
    ad_capture :: proc "C" (data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32 {
        ■ bdl := transmute(^Bdl_Entry) bdl_va

        ■ n := 0
        ■ off: u64 = 0

        ■ for off < u64(len) && n < BDL_N {
            ■ ■ chunk: u32 = 4096
            ■ ■ rem := u32(u64(len) - off)

            ■ ■ if rem < chunk {
                ■ ■ ■ chunk = rem
                // ... (truncated, see source file for more)

drivers/audiodriver/src/odin/speaker.odin (line 6) - [public] ad_play
    ad_play :: proc "C" (data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32 {
        ■ bdl := transmute(^Bdl_Entry) bdl_va

        ■ n := 0
        ■ off: u64 = 0

        ■ for off < u64(len) && n < BDL_N {
            ■ ■ chunk: u32 = 4096
            ■ ■ rem := u32(u64(len) - off)

            ■ ■ if rem < chunk {
                ■ ■ ■ chunk = rem
                // ... (truncated, see source file for more)

```

Python

doc.py (line 131) - [public] Definition

```

class Definition:
    path: str          # path relative to the scanned root
    lineno: int
    kind: str          # "function" / "class" / "method" / "type" / "const" / "export"
    name: str
    signature: str      # signature line (short, used in section 1/2)
    is_public: bool
    comment: str = ""   # comment/docstring directly above the definition
    body_snippet: str = "" # first few lines of code (section 3)
    language: str = ""

```

doc.py (line 144) - [public] MarkdownFile

```

class MarkdownFile:
    path: str
    content: str

```

doc.py (line 153) - [public] iter_source_files

```

# ----- #
# Collecting files
# ----- #

def iter_source_files(root: str, exts: set[str] | None, exclude_dirs: set[str]):
    for dirpath, dirnames, filenames in os.walk(root):
        dirnames[:] = [d for d in dirnames if d not in exclude_dirs and not d.startswith(".")
                        or d in (".",)]
        for fn in filenames:
            ext = os.path.splitext(fn)[1].lower()
            full = os.path.join(dirpath, fn)
            rel = os.path.relpath(full, root)
            if ext in MD_EXTS:
                yield rel, full, ext
            elif exts is not None:
                if ext in exts:
                    # ... (truncated, see source file for more)

```

doc.py (line 170) - [public] read_text

```
def read_text(path: str) -> str:
    for enc in ("utf-8", "utf-8-sig", "latin-1"):
        try:
            with open(path, "r", encoding=enc) as f:
                return f.read()
        except (UnicodeDecodeError, LookupError):
            continue
    with open(path, "r", encoding="utf-8", errors="replace") as f:
        return f.read()
```

doc.py (line 185) - [public] extract_preceding_comment

```
# ----- #
# Extracting the comment directly above line `lineno` (1-indexed)
# ----- #

def extract_preceding_comment(lines: list[str], lineno_0based: int, ext: str) -> str:
    prefixes = COMMENT_PREFIXES.get(ext, ("#", "//"))
    collected = []
    i = lineno_0based - 1
    in_block_close = False
    while i >= 0:
        line = lines[i].rstrip("\n")
        stripped = line.strip()
        if stripped == "":
            i -= 1
            if collected:
                # a blank line ends the comment block (unless we haven't
                # ... (truncated, see source file for more)
```

doc.py (line 222) - [public] parse_python

```
# ----- #
# PYTHON - parsed via ast (no code execution)
# ----- #

def parse_python(rel_path: str, full_path: str, source: str) -> list[Definition]:
    defs: list[Definition] = []
    try:
        tree = ast.parse(source, filename=rel_path)
    except SyntaxError:
        return defs

    lines = source.splitlines()

    all_list = None
    for node in ast.walk(tree):
        if isinstance(node, ast.Assign):
            # ... (truncated, see source file for more)
```

doc.py (line 298) - [internal] _rules_js_ts

```
# Each rule: (kind, regex, group_index, decide_public)
# decide_public(match) -> bool, or None when visibility depends on context
# (tracked separately in parse_generic, e.g. Ruby's private/public keyword,
# Ada's "private" section, Odin's @(private) attribute)
```

```
def _rules_js_ts():
    return [
        ("function", re.compile(r"^\s*export\s+(default\s+)?function\s*\*\s*(\w+)"), 2, lambda m: True),
        ("function", re.compile(r"^\s*export\s+(default\s+)?function\s*\*\s*(\w+)"), 2, lambda m: True),
        ("class", re.compile(r"^\s*export\s+(default\s+)?class\s*(\w+)"), 2, lambda m: True),
        ("const", re.compile(r"^\s*export\s+(const|let|var)\s*(\w+)"), 2, lambda m: True),
        ("interface", re.compile(r"^\s*export\s+interface\s*(\w+)"), 1, lambda m: True),
        ("type", re.compile(r"^\s*export\s+type\s*(\w+)"), 1, lambda m: True),
        ("export", re.compile(r"^\s*module\.exports\.(.*)\s*="), 2, lambda m: True),
        ("function", re.compile(r"^\s*(async\s+)?function\s*\*\s*(\w+)"), 2, lambda m: False),
        ("const", re.compile(r"^\s*(const|let|var)\s*(\w+)\s*=\s*(\(|async\s*\(|function)"), 2, lambda m: True),
        ("method", re.compile(r"^\s*(private|)\s*(\w+)\s*\("), 2, lambda m: False),
        # ... (truncated, see source file for more)
```

doc.py (line 313) - [internal] _rules_java_cs

```
def _rules_java_cs():
    return [
        ("class", re.compile(r"^s*public\s+(static\s+)?(final\s+)?(abstract\s+)?(class|interface|enum|record)"
        ("method", re.compile(r"^s*public\s+(static\s+)?(final\s+)?[\w<>\[\],\s]+?\s(\w+)\s*(\s*\()", 2, lambda m: True),
        ("class", re.compile(r"^s*(private|protected|internal)\s+(static\s+)?(class|interface|enum|record)"
        ("method", re.compile(r"^s*(private|protected|internal)\s+(static\s+)?[\w<>\[\],\s]+?\s(\w+)\s*(\s*\("
```

doc.py (line 322) - [internal] _rules_go

```
def _rules_go():
    return [
        ("type", re.compile(r"^s*type\s+([A-Z]\w*)\s+(struct|interface)", 1, lambda m: True),
        ("type", re.compile(r"^s*type\s+([a-z_]\w*)\s+(struct|interface)", 1, lambda m: False),
        ("function", re.compile(r"^s*func\s+(\([^\)]*\)\s*)?([A-Z]\w*)\s*(\s*\()", 2, lambda m: True),
        ("function", re.compile(r"^s*func\s+(\([^\)]*\)\s*)?([a-z_]\w*)\s*(\s*\()", 2, lambda m: False),
    ]
```

doc.py (line 331) - [internal] _rules_rust

```
def _rules_rust():
    return [
        ("function", re.compile(r"^s*pub(\([^\)]*\))?\s+fn\s+(\w+)", 2, lambda m: True),
        ("struct", re.compile(r"^s*pub(\([^\)]*\))?\s+struct\s+(\w+)", 2, lambda m: True),
        ("enum", re.compile(r"^s*pub(\([^\)]*\))?\s+enum\s+(\w+)", 2, lambda m: True),
        ("trait", re.compile(r"^s*pub(\([^\)]*\))?\s+trait\s+(\w+)", 2, lambda m: True),
        ("function", re.compile(r"^s*fn\s+(\w+)", 1, lambda m: False),
        ("struct", re.compile(r"^s*struct\s+(\w+)", 1, lambda m: False),
    ]
```

doc.py (line 342) - [internal] _rules_c_cpp

```
def _rules_c_cpp():
    return [
        ("function", re.compile(r"^s*static\s+[\w:~*<>,\s]+?\s(\w+)\s*\([^\)]*\)\s*\{", 1, lambda m: True),
        ("function", re.compile(r"^s*[\w:~*<>,\s]+?\s(\w+)\s*\([^\)]*\)\s*\{", 1, lambda m: True),
        ("declaration", re.compile(r"^s*[\w:~*<>,\s]+?\s(\w+)\s*\([^\)]*\)\s*;", 1, lambda m: True),
    ]
```

doc.py (line 350) - [internal] _rules_php

```
def _rules_php():
    return [
        ("method", re.compile(r"^s*public\s+(static\s+)?function\s+(\w+)", 2, lambda m: True),
        ("method", re.compile(r"^s*(private|protected)\s+(static\s+)?function\s+(\w+)", 3, lambda m: False),
        ("function", re.compile(r"^s*function\s+(\w+)", 1, lambda m: True),
        ("class", re.compile(r"^s*class\s+(\w+)", 1, lambda m: True),
    ]
```

doc.py (line 359) - [internal] _rules_ruby

```
def _rules_ruby():
    return [
        ("method", re.compile(r"^s*def\s+(self\.)?(\w+[\?!]?)", 2, None), # visibility resolved context
        ("class", re.compile(r"^s*class\s+(\w+)", 1, lambda m: True),
        ("module", re.compile(r"^s*module\s+(\w+)", 1, lambda m: True),
    ]
```

doc.py (line 367) - [internal] _rules_kotlin_swift

```
def _rules_kotlin_swift():
    return [
        ("function", re.compile(r"^s*(public|open)\s+fun\s+(\w+)", 2, lambda m: True),
        ("function", re.compile(r"^s*(private|internal|fileprivate)\s+fun\s+(\w+)", 2, lambda m: False),
        ("function", re.compile(r"^s*fun\s+(\w+)", 1, lambda m: True),
        ("class", re.compile(r"^s*(public|open)\s+class\s+(\w+)", 2, lambda m: True),
        ("class", re.compile(r"^s*(private|internal|fileprivate)\s+class\s+(\w+)", 2, lambda m: False),
        ("class", re.compile(r"^s*class\s+(\w+)", 1, lambda m: True),
    ]
```

```

doc.py (line 378) - [internal] _rules_d
def _rules_d():
    return [
        ("class", re.compile(r"^s*(private\s+(class|struct|interface|enum|union)\s+(\w+))", 2, lambda m:
        ("class", re.compile(r"^s*(class|struct|interface|enum|union)\s+(\w+)", 2, lambda m: True),
        ("function", re.compile(r"^s*(private\s+[\w!\[\]\*\<>,\.\s]+?\s(\w+)\s*\([\^;{}]*\)\s*\{", 1, lambda m: True),
        ("function", re.compile(r"^s*[\w!\[\]\*\<>,\.\s]+?\s(\w+)\s*\([\^;{}]*\)\s*\{", 1, lambda m: True)
    ]

doc.py (line 387) - [internal] _rules_nim
def _rules_nim():
    return [
        ("proc", re.compile(r"^s*(proc|func|template|macro|method|iterator)\s+(\w+)\s*", 2, lambda m: True),
        ("proc", re.compile(r"^s*(proc|func|template|macro|method|iterator)\s+(\w+)\s*b", 2, lambda m: False),
        ("type", re.compile(r"^s*type\s+(\w+)\s*", 1, lambda m: True),
        ("type", re.compile(r"^s*type\s+(\w+)\s*b", 1, lambda m: False),
    ]

doc.py (line 399) - [internal] _rules_odin
def _rules_odin():
    # visibility resolved contextually (an @(private) attribute on the line
    # above) - see parse_generic
    return [
        ("proc", re.compile(r"^s*(\w+)\s*::\s*proc\b", 1, None),
        ("struct", re.compile(r"^s*(\w+)\s*::\s*struct\b", 1, None),
        ("enum", re.compile(r"^s*(\w+)\s*::\s*enum\b", 1, None),
        ("union", re.compile(r"^s*(\w+)\s*::\s*union\b", 1, None),
    ]

doc.py (line 410) - [internal] _rules_ada_spec
def _rules_ada_spec():
    # in a .ads file (package specification): visibility depends on whether
    # the declaration comes before or after the "private" section
    return [
        ("package", re.compile(r"^s*package\s+(\w+)", 1, lambda m: True),
        ("procedure", re.compile(r"^s*procedure\s+(\w+)", 1, None),
        ("function", re.compile(r"^s*function\s+(\w+)", 1, None),
    ]

doc.py (line 420) - [internal] _rules_ada_body
def _rules_ada_body():
    # in a .adb file (implementation) - treated as internal detail
    return [
        ("procedure", re.compile(r"^s*procedure\s+(\w+)", 1, lambda m: False),
        ("function", re.compile(r"^s*function\s+(\w+)", 1, lambda m: False),
    ]

doc.py (line 428) - [internal] _rules_asm
def _rules_asm():
    return [
        ("label", re.compile(r"^s*\.\?(global|globl)\s+(\w+)", 2, lambda m: True),
        ("label", re.compile(r"^[A-Za-z_]\w*\s*:", 1, lambda m: False),
    ]

doc.py (line 457) - [public] parse_generic
def parse_generic(rel_path: str, ext: str, source: str, max_body_lines: int) -> list[Definition]:
    rules = RULES_BY_EXT.get(ext)
    if not rules:
        return []
    lines = source.splitlines()
    lang = LANG_BY_EXT.get(ext, ext)
    defs: list[Definition] = []

    ruby_private_mode = False # .rb only
    ada_private_mode = False # .ads only

    for idx, raw_line in enumerate(lines):
        # ... (truncated, see source file for more)

```

doc.py (line 526) - [public] collect

```
# -----#
# Collecting everything
# -----#

def collect(root: str, exts: set[str] | None, exclude_dirs: set[str], max_body_lines: int):
    all_defs: list[Definition] = []
    md_files: list[MarkdownFile] = []

    for rel_path, full_path, ext in iter_source_files(root, exts, exclude_dirs):
        source = read_text(full_path)
        if ext in MD_EXTS:
            md_files.append(MarkdownFile(path=rel_path.replace(os.sep, "/"), content=source))
            continue
        rel_norm = rel_path.replace(os.sep, "/")
        if ext == ".py":
            all_defs.extend(parse_python(rel_norm, full_path, source))
    # ... (truncated, see source file for more)
```

doc.py (line 550) - [public] build_styles

```
# -----#
# Building the PDF
# -----#

def build_styles():
    styles = getSampleStyleSheet()
    styles.add(ParagraphStyle(
        name="SectionTitle", parent=styles["Heading1"],
        fontSize=18, spaceAfter=14, spaceBefore=6, textColor=colors.HexColor("#1a1a2e"),
    ))
    styles.add(ParagraphStyle(
        name="LangHeading", parent=styles["Heading2"],
        fontSize=13, spaceBefore=14, spaceAfter=6, textColor=colors.HexColor("#16213e"),
    ))
    styles.add(ParagraphStyle(
        name="ItemPath", parent=styles["Normal"],
    ))
    # ... (truncated, see source file for more)
```

doc.py (line 587) - [public] group_by_language

```
def group_by_language(defs: list[Definition]):
    groups: dict[str, list[Definition]] = {}
    for d in defs:
        groups.setdefault(d.language, []).append(d)
    return dict(sorted(groups.items()))
```

doc.py (line 594) - [public] path_line_label

```
def path_line_label(d: Definition) -> str:
    return f"{d.path} (line {d.lineno})"
```

doc.py (line 598) - [public] render_signature_section

```
def render_signature_section(story, styles, title: str, defs: list[Definition], max_items: int):
    story.append(Paragraph(escape(title), styles["SectionTitle"]))
    if not defs:
        story.append(Paragraph("No items in this section.", styles["Normal"]))
        return
    truncated = len(defs) > max_items
    defs = defs[:max_items]
    for lang, items in group_by_language(defs).items():
        story.append(Paragraph(escape(lang), styles["LangHeading"]))
        for d in items:
            block = [
                Paragraph(escape(path_line_label(d)), styles["ItemPath"]),
            ]
    # ... (truncated, see source file for more)
```



```

doc.py (line 622) - [public] render_fragments_section
def render_fragments_section(story, styles, defs: list[Definition], max_items: int):
    story.append(Paragraph("3. Code fragments with comments", styles["SectionTitle"]))
    if not defs:
        story.append(Paragraph("No items.", styles["Normal"]))
        return
    truncated = len(defs) > max_items
    defs = defs[:max_items]
    for lang, items in group_by_language(defs).items():
        story.append(Paragraph(escape(lang), styles["LangHeading"]))
        for d in items:
            block = [Paragraph(escape(path_line_label(d) + f" - [{ 'public' if d.is_public else 'internal' } ]"), styles["SectionTitle"])]
            if d.comment:
                # ... (truncated, see source file for more)

doc.py (line 647) - [public] render_markdown_section
def render_markdown_section(story, styles, md_files: list[MarkdownFile]):
    story.append(Paragraph("4. .md files (in order)", styles["SectionTitle"]))
    if not md_files:
        story.append(Paragraph("No .md files in the project.", styles["Normal"]))
        return
    for mf in md_files:
        story.append(Paragraph(escape(mf.path), styles["MdHeading"]))
        content = mf.content
        if len(content) > 20000:
            content = content[:20000] + "\n\n... (truncated, file is very long)"
        story.append(Preformatted(content, styles["CodeBlock"]))

doc.py (line 660) - [internal] _new_doc
def _new_doc(path: str):
    return SimpleDocTemplate(
        path, pagesize=A4,
        leftMargin=18 * mm, rightMargin=18 * mm, topMargin=16 * mm, bottomMargin=16 * mm,
    )

doc.py (line 667) - [internal] _summary_paragraph
def _summary_paragraph(styles, root, public_defs, internal_defs, md_files):
    block = [
        Paragraph("Project technical documentation", styles["Title"]),
        Paragraph(escape(f"Source directory: {os.path.abspath(root)}"), styles["Normal"]),
        Spacer(1, 4),
        Paragraph(
            f"Public API/ABI: {len(public_defs)} | Internal: {len(internal_defs)} | "
            f".md files: {len(md_files)}", styles["Normal"],
        ),
    ]
    return block

doc.py (line 680) - [public] build_pdf
def build_pdf(output_path: str, root: str, public_defs, internal_defs, all_defs_for_fragments,
              md_files, max_items: int, split: bool):
    styles = build_styles()

    if not split:
        doc = _new_doc(output_path)
        story = []
        story += _summary_paragraph(styles, root, public_defs, internal_defs, md_files)
        story.append(PageBreak())
        render_signature_section(story, styles, "1. Public API / ABI", public_defs, max_items)
        story.append(PageBreak())
        render_signature_section(story, styles, "2. Internal functions", internal_defs, max_items)
        # ... (truncated, see source file for more)

```

doc.py (line 734) - [public] main

```
# ----- #
# main
# ----- #

def main():
    parser = argparse.ArgumentParser(description="PDF documentation generator for a whole repo (no compilation)")
    parser.add_argument("root", help="Root directory of the project to scan")
    parser.add_argument("-o", "--output", default="documentation.pdf", help="Output PDF file")
    parser.add_argument("--max-body-lines", type=int, default=12,
                        help="How many lines of code to show in the fragments section (default: 12)")
    parser.add_argument("--max-items-per-section", type=int, default=4000,
                        help="Cap on entries per section (default: 4000)")
    parser.add_argument("--exclude", default="", help="Extra directories to skip, comma-separated")
    parser.add_argument("--ext", default="", help="Restrict to these extensions only, e.g.: .py,.js,.go")
    parser.add_argument("--split", action="store_true",
                        help="Instead of one PDF, generate a separate file per section ")
    # ... (truncated, see source file for more)
```

Rust

c.rs (line 57) - [internal] BlockHeader

```
/* Struktura nagłówka bloku pamięci */
typedef struct BlockHeader {
    size_t size; /* Rozmiar danych użytkownika */
    int is_free; /* Czy blok jest wolny */

    struct BlockHeader* next; /* Wskaźnik do następnego bloku */
    struct BlockHeader* prev; /* Wskaźnik do poprzedniego bloku */
} BlockHeader;

/* Globalna pula pamięci i lista bloków */
static uint8_t memory_pool[MEMORY_POOL_SIZE];
static BlockHeader* free_list = NULL;
static int allocator_initialized = 0;

/* Makro do wyrównywania rozmiaru */
#define ALIGN(size) (((size) + (ALIGNMENT - 1)) & ~(ALIGNMENT - 1))

// ... (truncated, see source file for more)
```

c.rs (line 58) - [internal] BlockHeader

```
/* Struktura nagłówka bloku pamięci */
typedef struct BlockHeader {
    size_t size; /* Rozmiar danych użytkownika */
    int is_free; /* Czy blok jest wolny */
    struct BlockHeader* next; /* Wskaźnik do następnego bloku */

    struct BlockHeader* prev; /* Wskaźnik do poprzedniego bloku */
} BlockHeader;

/* Globalna pula pamięci i lista bloków */
static uint8_t memory_pool[MEMORY_POOL_SIZE];
static BlockHeader* free_list = NULL;
static int allocator_initialized = 0;

/* Makro do wyrównywania rozmiaru */
#define ALIGN(size) (((size) + (ALIGNMENT - 1)) & ~(ALIGNMENT - 1))

/* Inicjalizacja alokatora */
// ... (truncated, see source file for more)
```

comgrub/build.rs (line 1) - [internal] main

```
fn main() {
    nasm_rs::compile_library("boot.asm", &["src/boot.asm"]).unwrap();

    // █cie█ka bezwzgl█dna: `cargo:rustc-link-arg` przekazuje argument do linkera
    // w katalogu docelowym, wi█c relatywna █cie█ka by si█ nie rozwi█za█a.
    let manifest = std::env::var("CARGO_MANIFEST_DIR").unwrap();

    println!("cargo:rustc-link-arg=-T{/linker.ld", manifest);
    println!("cargo:rustc-link-lib=static=boot");

    println!("cargo:rerun-if-changed=src/boot.asm");
    println!("cargo:rerun-if-changed=linker.ld");
    // ... (truncated, see source file for more)
```

comgrub/src/main.rs (line 11) - [internal] kernel_main

```
fn kernel_main(magic: u32, info: *const u8) -> !;
}

#[panic_handler]
fn panic(_info: &PanicInfo) -> ! {
    loop {
        unsafe { core::arch::asm!("hlt", options(nomem, nostack, preserves_flags)) }
    }
}

/// Punkt wej█cia Rust wywo█ywany z `boot.asm` (`call _rust_entry`).
#[unsafe(no_mangle)]
// ... (truncated, see source file for more)
```

comgrub/src/main.rs (line 15) - [internal] panic

```
fn panic(_info: &PanicInfo) -> ! {
    loop {
        unsafe { core::arch::asm!("hlt", options(nomem, nostack, preserves_flags)) }
    }
}

/// Punkt wej█cia Rust wywo█ywany z `boot.asm` (`call _rust_entry`).
#[unsafe(no_mangle)]
pub extern "C" fn _rust_entry(magic: u32, info: *const u8) -> ! {
    unsafe { kernel_main(magic, info) }
}
```

comlimine/build.rs (line 1) - [internal] main

```
fn main() {
    cc::Build::new()
        .file("src/main.c")
        .include("src")
        .no_default_flags(true)
        .flag("-ffreestanding")
        .flag("-nostdlib")
        .flag("-mno-red-zone")
        .flag("-mno-mmx")
        .flag("-mno-sse")
        .flag("-mno-sse2")
        .compile("limine_entry");
    // ... (truncated, see source file for more)
```

comlimine/src/main.rs (line 1) - [internal] main

```
fn main() {}
```

drivers/amdgpu_driver/src/main.rs (line 1) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

drivers/audiodriver/build.rs (line 12) - [internal] main

```
fn main() {
    let manifest = PathBuf::from(env::var("CARGO_MANIFEST_DIR").unwrap());
    let out = PathBuf::from(env::var("OUT_DIR").unwrap());

    let odin_dir = manifest.join("src/odin");
    let lib_name = "audiodriver_odin";
    let lib_path = out.join(format!("lib{}.a", lib_name));

    println!("cargo:rerun-if-changed=build.rs");
    println!("cargo:rerun-if-changed={}", odin_dir.display());

    let odin = env::var("ODIN").unwrap_or_else(|_| "odin".to_string());
    // ... (truncated, see source file for more)
```

drivers/audiodriver/src/jacklib.rs (line 4) - [public] JackMgr

```
pub struct JackMgr {
    present: bool,
    amp_on: bool,
}

impl JackMgr {
    pub const fn new() -> Self {
        Self { present: false, amp_on: false }
    }

    pub fn tick(&mut self) {
        let now = ad::jack_present();
        // ... (truncated, see source file for more)
```

drivers/audiodriver/src/jacklib.rs (line 14) - [public] tick

```
pub fn tick(&mut self) {
    let now = ad::jack_present();

    if now != self.present {
        self.present = now;

        if now {
            ds::log::ds_log("jack: present");
            self.set_amp(true);
        } else {
            ds::log::ds_log("jack: gone");
            self.set_amp(false);
        }
    }
    // ... (truncated, see source file for more)
```

drivers/audiodriver/src/jacklib.rs (line 30) - [public] set_amp

```
pub fn set_amp(&mut self, on: bool) {
    self.amp_on = on;
    ad::set_amp(on);
}

pub fn present(&self) -> bool {
    self.present
}

/// Czy wzmacniacz (amp) jest aktualnie włączony.
pub fn amp_enabled(&self) -> bool {
    self.amp_on
}
// ... (truncated, see source file for more)
```

drivers/audiodriver/src/jacklib.rs (line 35) - [public] present

```
pub fn present(&self) -> bool {
    self.present
}

/// Czy wzmacniacz (amp) jest aktualnie włączony.
pub fn amp_enabled(&self) -> bool {
    self.amp_on
}
}
```

drivers/audiodriver/src/jacklib.rs (line 40) - [public] amp_enabled

/// Czy wzmacniacz (amp) jest aktualnie włączony.

```
pub fn amp_enabled(&self) -> bool {
    self.amp_on
}
```

drivers/audiodriver/src/lib.rs (line 7) - [internal] ad_init

```
fn ad_init(nam_va: u64, bm_va: u64) -> i32;
fn ad_play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
fn ad_capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
fn ad_stop() -> i32;
fn ad_jack_present() -> i32;
fn ad_set_amp(on: i32) -> i32;
fn ad_position() -> u32;

pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

// ... (truncated, see source file for more)
```

drivers/audiodriver/src/lib.rs (line 8) - [internal] ad_play

```
fn ad_play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
fn ad_capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
fn ad_stop() -> i32;
fn ad_jack_present() -> i32;
fn ad_set_amp(on: i32) -> i32;
fn ad_position() -> u32;

pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

// ... (truncated, see source file for more)
```

drivers/audiodriver/src/lib.rs (line 9) - [internal] ad_capture

```
fn ad_capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> i32;
fn ad_stop() -> i32;
fn ad_jack_present() -> i32;
fn ad_set_amp(on: i32) -> i32;
fn ad_position() -> u32;

pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    // ... (truncated, see source file for more)
```

drivers/audiodriver/src/lib.rs (line 10) - [internal] ad_stop

```
fn ad_stop() -> i32;
fn ad_jack_present() -> i32;
fn ad_set_amp(on: i32) -> i32;
fn ad_position() -> u32;

pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    unsafe { ad_play(data_phys, len, bdl_va, bdl_phys) == 0 }
    // ... (truncated, see source file for more)
```

drivers/audiodriver/src/lib.rs (line 11) - [internal] ad_jack_present

```
fn ad_jack_present() -> i32;
fn ad_set_amp(on: i32) -> i32;
fn ad_position() -> u32;
}

pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    unsafe { ad_play(data_phys, len, bdl_va, bdl_phys) == 0 }
}

// ... (truncated, see source file for more)
```

drivers/audiodriver/src/lib.rs (line 12) - [internal] ad_set_amp

```
fn ad_set_amp(on: i32) -> i32;
fn ad_position() -> u32;
}

pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    unsafe { ad_play(data_phys, len, bdl_va, bdl_phys) == 0 }
}

// ... (truncated, see source file for more)
```

drivers/audiodriver/src/lib.rs (line 13) - [internal] ad_position

```
fn ad_position() -> u32;
}

pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    unsafe { ad_play(data_phys, len, bdl_va, bdl_phys) == 0 }
}

pub fn capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    // ... (truncated, see source file for more)
```

drivers/audiodriver/src/lib.rs (line 16) - [public] init

```
pub fn init(nam_va: u64, bm_va: u64) -> bool {
    unsafe { ad_init(nam_va, bm_va) == 0 }
}

pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    unsafe { ad_play(data_phys, len, bdl_va, bdl_phys) == 0 }
}

pub fn capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
    unsafe { ad_capture(data_phys, len, bdl_va, bdl_phys) == 0 }
}

// ... (truncated, see source file for more)
```

```
drivers/audiodriver/src/lib.rs (line 20) - [public] play
    pub fn play(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
        unsafe { ad_play(data_phys, len, bdl_va, bdl_phys) == 0 }
    }

    pub fn capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
        unsafe { ad_capture(data_phys, len, bdl_va, bdl_phys) == 0 }
    }

    pub fn stop() {
        unsafe { let _ = ad_stop(); }
    }

    // ... (truncated, see source file for more)
```

```
drivers/audiodriver/src/lib.rs (line 24) - [public] capture
    pub fn capture(data_phys: u64, len: u32, bdl_va: u64, bdl_phys: u64) -> bool {
        unsafe { ad_capture(data_phys, len, bdl_va, bdl_phys) == 0 }
    }

    pub fn stop() {
        unsafe { let _ = ad_stop(); }
    }

    pub fn jack_present() -> bool {
        unsafe { ad_jack_present() != 0 }
    }

    // ... (truncated, see source file for more)
```

```
drivers/audiodriver/src/lib.rs (line 28) - [public] stop
    pub fn stop() {
        unsafe { let _ = ad_stop(); }
    }

    pub fn jack_present() -> bool {
        unsafe { ad_jack_present() != 0 }
    }

    pub fn set_amp(on: bool) {
        unsafe { let _ = ad_set_amp(on as i32); }
    }

    // ... (truncated, see source file for more)
```

```
drivers/audiodriver/src/lib.rs (line 32) - [public] jack_present
    pub fn jack_present() -> bool {
        unsafe { ad_jack_present() != 0 }
    }

    pub fn set_amp(on: bool) {
        unsafe { let _ = ad_set_amp(on as i32); }
    }

    pub fn position() -> u32 {
        unsafe { ad_position() }
    }
```

```
drivers/audiodriver/src/lib.rs (line 36) - [public] set_amp
    pub fn set_amp(on: bool) {
        unsafe { let _ = ad_set_amp(on as i32); }
    }

    pub fn position() -> u32 {
        unsafe { ad_position() }
    }
```

```
drivers/audiodriver/src/lib.rs (line 40) - [public] position
    pub fn position() -> u32 {
        unsafe { ad_position() }
    }
```

drivers/audiodriver/src/main.rs (line 8) - [internal] main

```
/// Punkt wejścia binarki pomocniczej.
///
/// Właściwy sterownik działa w driver space przez `ds_entry` (patrz
/// `driverspace/src/main.rs`); ta binarka jest ciekim wrapperem CLI, który
/// pozwala sprawdzić stan gniazda jack z poziomu hosta.
```

```
fn main() {
    let mut mgr = ad::jacklib::JackMgr::new();
    mgr.tick();

    println!(
        "audiodriver: jack present = {}, amp on = {}",
        mgr.present(),
        mgr.amp_enabled()
    );
}
```

drivers/audiodriver/src/tone.rs (line 1) - [public] fill_square

```
pub fn fill_square(buf: &mut [u8], periods: u32) {
    let half = buf.len() as u32 / (periods * 2);

    for i in 0..buf.len() as u32 {
        let v = if (i / half.max(1)) % 2 == 0 { 0xC0 } else { 0x40 };
        buf[i as usize] = v;
    }
}
```

drivers/intelgpu_driver/src/main.rs (line 1) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

drivers/netcam_driver/src/main.rs (line 1) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

drivers/wacomgraphic_driver/src/main.rs (line 1) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

driverspace/src/drivers/storage.rs (line 9) - [public] StorageDrv

```
/// Storage driver stub. It only satisfies the driver registry so that
/// driver space can boot; a real disk driver can slot in later.
```

```
pub struct StorageDrv {
    registered: bool,
}

impl StorageDrv {
    pub const fn new() -> Self {
        Self { registered: false }
    }
}

impl Driver for StorageDrv {
    fn init(&mut self, _info: &DeviceInfo) -> Result<(), DsError> {
        // ... (truncated, see source file for more)
    }
}
```

driverspace/src/drivers/storage.rs (line 20) - [internal] init

```
fn init(&mut self, _info: &DeviceInfo) -> Result<(), DsError> {
    self.registered = true;
    Ok(())
}
```


driverspace/src/main.rs (line 32) - [internal] panic

```
fn panic(_info: &core::panic::PanicInfo) -> ! {  
    loop {}  
}
```

driverspace_workspace/crates/ds-manager/src/main.rs (line 1) - [internal] main

```
fn main() {  
    println!("Hello, world!");  
}
```

driverspace_workspace/drivers/amdgpu-driver/src/main.rs (line 1) - [internal] main

```
fn main() {  
    println!("Hello, world!");  
}
```

driverspace_workspace/drivers/audiodriver/src/main.rs (line 1) - [internal] main

```
fn main() {  
    println!("Hello, world!");  
}
```

driverspace_workspace/drivers/intelgpu-driver/src/main.rs (line 1) - [internal] main

```
fn main() {  
    println!("Hello, world!");  
}
```

driverspace_workspace/drivers/netcam_driver/src/main.rs (line 1) - [internal] main

```
fn main() {  
    println!("Hello, world!");  
}
```

driverspace_workspace/drivers/vgpu/src/main.rs (line 1) - [internal] main

```
fn main() {  
    println!("Hello, world!");  
}
```

driverspace_workspace/drivers/wacomgraphic_driver/src/main.rs (line 1) - [internal] main

```
fn main() {  
    println!("Hello, world!");  
}
```

driverspace_workspace/lib/ds-formal-ffi/build.rs (line 10) - [internal] main

```
fn main() {  
    // ■cie■ka do projektu Ada SPARK  
    let spark_project_dir = PathBuf::from("../..formal/ds-spark-core");  
  
    println!("cargo:rerun-if-changed=../..formal/ds-spark-core/src");  
  
    let gprbuild_available = Command::new("gprbuild")  
        .arg("--version")  
        .status()  
        .map(|s| s.success())  
        .unwrap_or(false);  
  
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 7) - [public] ds_spark_safe_add_32

// Odpowiednik: function Safe_Add_32 (A, B : UInt32) return UInt32

```
pub fn ds_spark_safe_add_32(a: u32, b: u32) -> u32;

// Odpowiednik: function Safe_Array_Index ...
pub fn ds_spark_safe_array_index(index: u32, max_size: u32) -> u32;
}

// Bezpieczne wrappery dla reszty kodu Rust w workspace
pub mod math {
    use super::*;

    /// Bezpieczne dodawanie. W trybie debug Rust i tak to sprawdzi,
    /// ale w release mode (gdzie overflow to undefined behavior w C/Ada),
    /// ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 10) - [public]

ds_spark_safe_array_index

// Odpowiednik: function Safe_Array_Index ...

```
pub fn ds_spark_safe_array_index(index: u32, max_size: u32) -> u32;
}

// Bezpieczne wrappery dla reszty kodu Rust w workspace
pub mod math {
    use super::*;

    /// Bezpieczne dodawanie. W trybie debug Rust i tak to sprawdzi,
    /// ale w release mode (gdzie overflow to undefined behavior w C/Ada),
    /// ta funkcja ma matematyczny dowód z SPARK, ■e nigdy nie overflownie.
    #[inline(always)]
    pub fn safe_add(a: u32, b: u32) -> u32 {
        // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 21) - [public] safe_add

```
pub fn safe_add(a: u32, b: u32) -> u32 {
    // W produkcji wo■amy udowodniony kod z Ady
    unsafe { ds_spark_safe_add_32(a, b) }
}

unsafe extern "C" {
    pub fn ds_spark_buddy_get_buddy(addr: u64, size_exp: u64) -> u64;
    pub fn ds_spark_safe_dma_end(base: u64, len: u64) -> u64;
}

pub mod pmm {
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 28) - [public]

ds_spark_buddy_get_buddy

```
pub fn ds_spark_buddy_get_buddy(addr: u64, size_exp: u64) -> u64;
pub fn ds_spark_safe_dma_end(base: u64, len: u64) -> u64;
}

pub mod pmm {
    use super::*;
    #[inline(always)]
    pub fn get_buddy(addr: u64, exp: u64) -> u64 {
        // Wywo■anie kodu, który ma MATEMATYCZNY DOWÓD poprawno■ci
        unsafe { ds_spark_buddy_get_buddy(addr, exp) }
    }
}
```

```

driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 29) - [public] ds_spark_safe_dma_end
    pub fn ds_spark_safe_dma_end(base: u64, len: u64) -> u64;
}

pub mod pmm {
    use super::*;
    #[inline(always)]
    pub fn get_buddy(addr: u64, exp: u64) -> u64 {
        // Wywołanie kodu, który ma MATEMATYCZNY DOWÓD poprawności
        unsafe { ds_spark_buddy_get_buddy(addr, exp) }
    }
}

driverspace_workspace/lib/ds-formal-ffi/src/lib.rs (line 35) - [public] get_buddy
    pub fn get_buddy(addr: u64, exp: u64) -> u64 {
        // Wywołanie kodu, który ma MATEMATYCZNY DOWÓD poprawności
        unsafe { ds_spark_buddy_get_buddy(addr, exp) }
    }
}

driverspace_workspace/lib/ds-fw-audio/src/lib.rs (line 1) - [public] add
    pub fn add(left: u64, right: u64) -> u64 {
        left + right
    }

    #[cfg(test)]
    mod tests {
        use super::*;

        #[test]
        fn it_works() {
            let result = add(2, 2);
            assert_eq!(result, 4);
            // ... (truncated, see source file for more)
        }
    }

driverspace_workspace/lib/ds-fw-audio/src/lib.rs (line 10) - [internal] it_works
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
    }
}

driverspace_workspace/lib/ds-fw-block/src/request.rs (line 7) - [public] RequestState
    pub enum RequestState {
        Pending = 0,
        InProgress = 1,
        Completed = 2,
        Failed = 3,
    }

    /// High-level I/O request with metadata.
    pub struct IoRequest {
        pub id: u64,
        pub lba: u64,
        pub block_count: u32,
        // ... (truncated, see source file for more)
    }

```

driverspace_workspace/lib/ds-fw-block/src/request.rs (line 15) - [public] IoRequest

/// High-level I/O request with metadata.

```
pub struct IoRequest {
    pub id: u64,
    pub lba: u64,
    pub block_count: u32,
    pub buffer_ptr: *mut u8,
    pub buffer_len: usize,
    pub is_write: bool,
    pub priority: u8,
    state: AtomicU32,
    pub callback: Option<fn(&IoRequest)>,
}
```

// ... (truncated, see source file for more)

driverspace_workspace/lib/ds-fw-block/src/request.rs (line 28) - [public] new

```
pub fn new(id: u64, lba: u64, block_count: u32, buffer: *mut u8, len: usize, is_write: bool) -> Self {
    Self {
        id,
        lba,
        block_count,
        buffer_ptr: buffer,
        buffer_len: len,
        is_write,
        priority: 0,
        state: AtomicU32::new(RequestState::Pending as u32),
        callback: None,
    }
}
```

// ... (truncated, see source file for more)

driverspace_workspace/lib/ds-fw-block/src/request.rs (line 42) - [public] state

```
pub fn state(&self) -> RequestState {
    match self.state.load(Ordering::Acquire) {
        0 => RequestState::Pending,
        1 => RequestState::InProgress,
        2 => RequestState::Completed,
        3 => RequestState::Failed,
        _ => RequestState::Failed,
    }
}
```

```
pub fn mark_in_progress(&self) {
    self.state.store(RequestState::InProgress as u32, Ordering::Release);
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-fw-block/src/request.rs (line 52) - [public] mark_in_progress

```
pub fn mark_in_progress(&self) {
    self.state.store(RequestState::InProgress as u32, Ordering::Release);
}
```

```
pub fn mark_completed(&self) {
    self.state.store(RequestState::Completed as u32, Ordering::Release);
    if let Some(cb) = self.callback {
        cb(self);
    }
}
```

```
pub fn mark_failed(&self) {
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-fw-block/src/request.rs (line 56) - [public] mark_completed

```
pub fn mark_completed(&self) {
    self.state.store(RequestState::Completed as u32, Ordering::Release);
    if let Some(cb) = self.callback {
        cb(self);
    }
}

pub fn mark_failed(&self) {
    self.state.store(RequestState::Failed as u32, Ordering::Release);
}
}
```

driverspace_workspace/lib/ds-fw-block/src/request.rs (line 63) - [public] mark_failed

```
pub fn mark_failed(&self) {
    self.state.store(RequestState::Failed as u32, Ordering::Release);
}
}
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 6) - [public] BlockRequest

/// Represents a single I/O request to a block device.

```
pub struct BlockRequest {
    pub lba: u64,
    pub block_count: u32,
    pub buffer_ptr: *mut u8,
    pub buffer_len: usize,
    pub is_write: bool,
}
```

/// Core trait for all block devices.

/// Drivers must implement this to integrate with the block subsystem.

```
pub trait BlockDevice {
    /// Read blocks from the device starting at LBA.
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 16) - [public] BlockDevice

/// Core trait for all block devices.

/// Drivers must implement this to integrate with the block subsystem.

```
pub trait BlockDevice {
    /// Read blocks from the device starting at LBA.
    fn read_blocks(&mut self, lba: u64, block_count: u32, buffer: &mut [u8]) -> Result<(), DsError>;

    /// Write blocks to the device starting at LBA.
    fn write_blocks(&mut self, lba: u64, block_count: u32, buffer: &[u8]) -> Result<(), DsError>;

    /// Flush internal caches to physical media.
    fn flush(&mut self) -> Result<(), DsError>;

    /// Get device geometry (block size, total blocks).
    fn geometry(&self) -> BlockGeometry;
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 18) - [internal] read_blocks

/// Read blocks from the device starting at LBA.

```
fn read_blocks(&mut self, lba: u64, block_count: u32, buffer: &mut [u8]) -> Result<(), DsError>;

/// Write blocks to the device starting at LBA.
fn write_blocks(&mut self, lba: u64, block_count: u32, buffer: &[u8]) -> Result<(), DsError>;

/// Flush internal caches to physical media.
fn flush(&mut self) -> Result<(), DsError>;

/// Get device geometry (block size, total blocks).
fn geometry(&self) -> BlockGeometry;

/// Check if device supports TRIM/discard.
// ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 21) - [internal] write_blocks

/// Write blocks to the device starting at LBA.

```
fn write_blocks(&mut self, lba: u64, block_count: u32, buffer: &[u8]) -> Result<(), DsError>;

/// Flush internal caches to physical media.
fn flush(&mut self) -> Result<(), DsError>;

/// Get device geometry (block size, total blocks).
fn geometry(&self) -> BlockGeometry;

/// Check if device supports TRIM/discard.
fn supports_trim(&self) -> bool { false }

/// Optional: Submit async request (for advanced schedulers).
// ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 24) - [internal] flush

/// Flush internal caches to physical media.

```
fn flush(&mut self) -> Result<(), DsError>;

/// Get device geometry (block size, total blocks).
fn geometry(&self) -> BlockGeometry;

/// Check if device supports TRIM/discard.
fn supports_trim(&self) -> bool { false }

/// Optional: Submit async request (for advanced schedulers).
fn submit_request(&mut self, _req: BlockRequest) -> Result<(), DsError> {
    Err(DsError::Unknown)
}
// ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 27) - [internal] geometry

/// Get device geometry (block size, total blocks).

```
fn geometry(&self) -> BlockGeometry;

/// Check if device supports TRIM/discard.
fn supports_trim(&self) -> bool { false }

/// Optional: Submit async request (for advanced schedulers).
fn submit_request(&mut self, _req: BlockRequest) -> Result<(), DsError> {
    Err(DsError::Unknown)
}

}

/// Physical geometry of a block device.
// ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 30) - [internal] supports_trim

/// Check if device supports TRIM/discard.

```
fn supports_trim(&self) -> bool { false }

/// Optional: Submit async request (for advanced schedulers).
fn submit_request(&mut self, _req: BlockRequest) -> Result<(), DsError> {
    Err(DsError::Unknown)
}

}

/// Physical geometry of a block device.
#[derive(Debug, Clone, Copy)]
pub struct BlockGeometry {
    pub block_size: u32,
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 33) - [internal] submit_request

/// Optional: Submit async request (for advanced schedulers).

```
    fn submit_request(&mut self, _req: BlockRequest) -> Result<(), DsError> {
        Err(DsError::Unknown)
    }
}

/// Physical geometry of a block device.
#[derive(Debug, Clone, Copy)]
pub struct BlockGeometry {
    pub block_size: u32,
    pub total_blocks: u64,
    pub max_transfer_blocks: u32,
}

// ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 40) - [public] BlockGeometry

```
pub struct BlockGeometry {
    pub block_size: u32,
    pub total_blocks: u64,
    pub max_transfer_blocks: u32,
}

impl BlockGeometry {
    pub fn total_bytes(&self) -> u64 {
        self.total_blocks * self.block_size as u64
    }
}
```

driverspace_workspace/lib/ds-fw-block/src/traits.rs (line 47) - [public] total_bytes

```
    pub fn total_bytes(&self) -> u64 {
        self.total_blocks * self.block_size as u64
    }
}
```

driverspace_workspace/lib/ds-fw-gpu/src/lib.rs (line 1) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/lib/ds-fw-gpu/src/lib.rs (line 10) - [internal] it_works

```
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
    }
}
```

driverspace_workspace/lib/ds-fw-input/src/lib.rs (line 1) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/lib/ds-fw-input/src/lib.rs (line 10) - [internal] it_works

```
fn it_works() {
    let result = add(2, 2);
    assert_eq!(result, 4);
}
}
```

driverspace_workspace/lib/ds-ipc/src/client.rs (line 7) - [public] ManagerClient

```
pub struct ManagerClient {
    manager_ep: Handle,
}

impl ManagerClient {
    pub const fn new(ep: Handle) -> Self {
        Self { manager_ep: ep }
    }

    pub fn request_mmio(&self, phys_base: u64, size: u64) -> Result<u64, DsError> {
        let mut payload = kapi_abi::payloads::mem::MmioMapPayload {
            phys_addr: kapi_abi::primitives::PhysAddr(phys_base),
            // ... (truncated, see source file for more)
        }
    }
}
```

driverspace_workspace/lib/ds-ipc/src/client.rs (line 16) - [public] request_mmio

```
pub fn request_mmio(&self, phys_base: u64, size: u64) -> Result<u64, DsError> {
    let mut payload = kapi_abi::payloads::mem::MmioMapPayload {
        phys_addr: kapi_abi::primitives::PhysAddr(phys_base),
        size,
        flags: 0,
        _pad: 0,
    };

    let mut reply_buf = [0u64; 2];

    let res = kapi_syscall::sys_ipc_call(
        self.manager_ep.0,
        // ... (truncated, see source file for more)
    )
}
```

driverspace_workspace/lib/ds-ipc/src/client.rs (line 41) - [public] bind_irq

```
pub fn bind_irq(&self, irq_num: u32) -> Result<(), DsError> {
    let payload = kapi_abi::payloads::irq::IrqBindPayload {
        irq_number: irq_num,
        flags: 0,
        _pad: 0,
    };

    kapi_syscall::sys_ipc_send(
        self.manager_ep.0,
        Opcode::IrqBind as u32,
        &payload as *const _ as *const u8,
        core::mem::size_of_val(&payload) as u32,
        // ... (truncated, see source file for more)
    )
}
```


driverspace_workspace/lib/ds-ipc/src/dispatcher.rs (line 10) - [public] MessageDispatcher

```
pub struct MessageDispatcher {
    handlers: [Option<HandlerFn>; 64],
}

impl MessageDispatcher {
    pub const fn new() -> Self {
        Self {
            handlers: [None; 64],
        }
    }

    pub fn register(&mut self, opcode: Opcode, handler: HandlerFn) -> Result<(), DsError> {
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/lib/ds-ipc/src/dispatcher.rs (line 21) - [public] register

```
pub fn register(&mut self, opcode: Opcode, handler: HandlerFn) -> Result<(), DsError> {
    let idx = (opcode as u32 & 0x3F) as usize;
    if self.handlers[idx].is_some() {
        return Err(DsError::DeviceBusy);
    }
    self.handlers[idx] = Some(handler);
    Ok(())
}

pub fn dispatch(&self, sender: Handle, opcode_raw: u32, payload_ptr: *const u8, payload_len: u32) ->
    let opcode = Opcode::from_u32(opcode_raw).ok_or(DsError::InvalidMessage?);
    let idx = (opcode_raw & 0x3F) as usize;
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-ipc/src/dispatcher.rs (line 30) - [public] dispatch

```
pub fn dispatch(&self, sender: Handle, opcode_raw: u32, payload_ptr: *const u8, payload_len: u32) ->
    let opcode = Opcode::from_u32(opcode_raw).ok_or(DsError::InvalidMessage?);
    let idx = (opcode_raw & 0x3F) as usize;

    if let Some(handler) = self.handlers[idx] {
        handler(sender, payload_ptr, payload_len)
    } else {
        ds_error!("Unhandled opcode: {:?}", opcode);
        Err(DsError::InvalidMessage)
    }
}
```

driverspace_workspace/lib/ds-log/src/formatter.rs (line 7) - [public] LogBuffer

```
pub struct LogBuffer {
    buf: [u8; MAX_LOG_LINE_LEN],
    pos: usize,
}

impl LogBuffer {
    pub const fn new() -> Self {
        Self {
            buf: [0; MAX_LOG_LINE_LEN],
            pos: 0,
        }
    }
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-log/src/formatter.rs (line 20) - [public] as_bytes

```
pub fn as_bytes(&self) -> &[u8] {
    &self.buf[..self.pos]
}

pub fn reset(&mut self) {
    self.pos = 0;
}

impl Write for LogBuffer {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        let bytes = s.as_bytes();
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/lib/ds-log/src/formatter.rs (line 24) - [public] reset

```
pub fn reset(&mut self) {
    self.pos = 0;
}

impl Write for LogBuffer {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        let bytes = s.as_bytes();
        let remaining = self.buf.len() - self.pos;
        let copy_len = if bytes.len() > remaining { remaining } else { bytes.len() };

        self.buf[self.pos..self.pos + copy_len].copy_from_slice(&bytes[..copy_len]);
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/lib/ds-log/src/formatter.rs (line 30) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> fmt::Result {
    let bytes = s.as_bytes();
    let remaining = self.buf.len() - self.pos;
    let copy_len = if bytes.len() > remaining { remaining } else { bytes.len() };

    self.buf[self.pos..self.pos + copy_len].copy_from_slice(&bytes[..copy_len]);
    self.pos += copy_len;
    Ok(())
}

}
```

driverspace_workspace/lib/ds-log/src/levels.rs (line 5) - [public] LogLevel

```
pub enum LogLevel {
    Trace = 0,
    Info  = 1,
    Warn  = 2,
    Error = 3,
    Fatal = 4,
}

impl LogLevel {
    pub const fn as_str(&self) -> &'static str {
        match self {
            Self::Trace => "TRACE",
            // ... (truncated, see source file for more)
        }
    }
}
```

driverspace_workspace/lib/ds-log/src/lib.rs (line 14) - [public] EndpointRef

```
pub struct EndpointRef;

impl EndpointRef {
    pub const fn new() -> Self { Self }

    pub fn is_valid(&self) -> bool {
        MANAGER_ENDPOINT_RAW.load(Ordering::Acquire) != 0
    }

    /// Handle endpointu Managera (0 = niezarejestrowany).
    pub fn handle(&self) -> u32 {
        MANAGER_ENDPOINT_RAW.load(Ordering::Acquire)
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/lib/ds-log/src/lib.rs (line 19) - [public] is_valid

```
pub fn is_valid(&self) -> bool {
    MANAGER_ENDPOINT_RAW.load(Ordering::Acquire) != 0
}

/// Handle endpointu Managera (0 = niezarejestrowany).
pub fn handle(&self) -> u32 {
    MANAGER_ENDPOINT_RAW.load(Ordering::Acquire)
}

pub fn set(&self, handle: Handle) {
    MANAGER_ENDPOINT_RAW.store(handle.0, Ordering::Release);
}
// ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-log/src/lib.rs (line 24) - [public] handle

/// Handle endpointu Managera (0 = niezarejestrowany).

```
pub fn handle(&self) -> u32 {
    MANAGER_ENDPOINT_RAW.load(Ordering::Acquire)
}

pub fn set(&self, handle: Handle) {
    MANAGER_ENDPOINT_RAW.store(handle.0, Ordering::Release);
}

pub static MANAGER_ENDPOINT: EndpointRef = EndpointRef::new();
```

driverspace_workspace/lib/ds-log/src/lib.rs (line 28) - [public] set

```
pub fn set(&self, handle: Handle) {
    MANAGER_ENDPOINT_RAW.store(handle.0, Ordering::Release);
}

pub static MANAGER_ENDPOINT: EndpointRef = EndpointRef::new();
```

driverspace_workspace/lib/ds-log/src/transport.rs (line 12) - [internal] uart_putchar

```
fn uart_putchar(c: u8) {
    #[cfg(target_arch = "x86_64")]
    unsafe {
        core::arch::asm!("out dx, al", in("dx") UART_PORT, in("al") c, options(nostack, preserves_flags))
    }
}

pub fn emit_log(level: LogLevel, module: &str, buf: &LogBuffer) {
    // 1. Fast fallback: UART (crucial for early boot before IPC is up)
    for &b in b"[" { uart_putchar(b); }
    for &b in level.as_str().as_bytes() { uart_putchar(b); }
    for &b in b"] " { uart_putchar(b); }
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-log/src/transport.rs (line 19) - [public] emit_log

```
pub fn emit_log(level: LogLevel, module: &str, buf: &LogBuffer) {
    // 1. Fast fallback: UART (crucial for early boot before IPC is up)
    for &b in b"[" { uart_putchar(b); }
    for &b in level.as_str().as_bytes() { uart_putchar(b); }
    for &b in b"] " { uart_putchar(b); }
    for &b in module.as_bytes() { uart_putchar(b); }
    for &b in b": " { uart_putchar(b); }
    for &b in buf.as_bytes() { uart_putchar(b); }
    uart_putchar(b'\n');

    // 2. IPC Transport (if Manager is ready)
    let manager_ep = &crate::MANAGER_ENDPOINT;
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-mem/src/alloc/buddy.rs (line 5) - [public] BuddyAllocator

```
pub struct BuddyAllocator {
    base_addr: usize,
    total_pages: usize,
    free_lists: [usize; MAX_ORDER + 1],
}

impl BuddyAllocator {
    pub const fn new(base_addr: usize, total_pages: usize) -> Self {
        Self {
            base_addr,
            total_pages,
            free_lists: [0; MAX_ORDER + 1],
            // ... (truncated, see source file for more)
        }
    }
}
```

driverspace_workspace/lib/ds-mem/src/alloc/buddy.rs (line 20) - [public] alloc_pages

```
pub fn alloc_pages(&mut self, order: usize) -> Option<usize> {
    if order > MAX_ORDER {
        return None;
    }
    // TODO: Implement actual buddy allocation (find free block, split if necessary)
    Some(self.base_addr)
}

pub fn free_pages(&mut self, addr: usize, order: usize) {
    // TODO: Implement buddy merge logic
}

}
```

driverspace_workspace/lib/ds-mem/src/alloc/buddy.rs (line 28) - [public] free_pages

```
pub fn free_pages(&mut self, addr: usize, order: usize) {
    // TODO: Implement buddy merge logic
}

}
```

driverspace_workspace/lib/ds-mem/src/alloc/heap.rs (line 6) - [public] DriverHeap

```
pub struct DriverHeap {
    buddy: BuddyAllocator,
    // TODO: Add array of SlabAllocator for sizes: 16, 32, 64, 128, 256, 512, 1024
}

impl DriverHeap {
    pub const fn new() -> Self {
        Self {
            buddy: BuddyAllocator::new(0, 0),
        }
    }

    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-mem/src/alloc/heap.rs (line 18) - [public] init

```
pub fn init(&mut self, base_addr: usize, total_pages: usize) {
    self.buddy = BuddyAllocator::new(base_addr, total_pages);
}

unsafe impl GlobalAlloc for DriverHeap {
    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        ds_error!("DriverHeap::alloc called before full initialization or unimplemented");
        null_mut()
    }

    unsafe fn dealloc(&self, ptr: *mut u8, layout: Layout) {
        // ... (truncated, see source file for more)
    }
}
```

driverspace_workspace/lib/ds-mem/src/alloc/slab.rs (line 1) - [public] SlabAllocator

```
pub struct SlabAllocator {
    object_size: usize,
    free_head: usize,
}

impl SlabAllocator {
    pub const fn new(object_size: usize) -> Self {
        Self {
            object_size,
            free_head: 0,
        }
    }
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-mem/src/alloc/slab.rs (line 14) - [public] alloc

```
pub fn alloc(&mut self) -> Option<usize> {
    // TODO: Pop address from free_head linked list
    Some(0)
}

pub fn free(&mut self, ptr: usize) {
    // TODO: Push address to free_head linked list
}
}
```

driverspace_workspace/lib/ds-mem/src/alloc/slab.rs (line 19) - [public] free

```
pub fn free(&mut self, ptr: usize) {
    // TODO: Push address to free_head linked list
}
}
```

driverspace_workspace/lib/ds-mem/src/dma/buffer.rs (line 10) - [public] DmaFlags

```
pub struct DmaFlags: u32 {
    const COHERENT = 1 << 0;
    const HIGH_MEM = 1 << 1;
    const CONTIGUOUS = 1 << 2;
}

pub struct DmaBuffer {
    pub phys_addr: PhysAddr,
    pub virt_addr: VirtAddr,
    pub size: u64,
    pub handle: Handle,
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-mem/src/dma/buffer.rs (line 17) - [public] DmaBuffer

```
pub struct DmaBuffer {
    pub phys_addr: PhysAddr,
    pub virt_addr: VirtAddr,
    pub size: u64,
    pub handle: Handle,
}

impl DmaBuffer {
    pub fn allocate(manager_ep: Handle, size: u64, flags: DmaFlags) -> Result<Self, DsError> {
        let mut payload = DmaAllocPayload {
            size,
            flags: flags.bits(),
        }
        // ... (truncated, see source file for more)
```

driverspace_workspace/lib/ds-mem/src/dma/buffer.rs (line 25) - [public] allocate

```
pub fn allocate(manager_ep: Handle, size: u64, flags: DmaFlags) -> Result<Self, DsError> {
    let mut payload = DmaAllocPayload {
        size,
        flags: flags.bits(),
        _pad: 0,
    };

    let mut reply_buf = [0u64; 3];

    let res = unsafe {
        sys_ipc_call(
            manager_ep.0,
            // ... (truncated, see source file for more)
        )
    }
```

driverspace_workspace/lib/ds-mem/src/dma/buffer.rs (line 61) - [internal] drop

```
fn drop(&mut self) {
    ds_log::ds_trace!("Dropping DMA buffer at {:?} ", self.phys_addr);
    // TODO: IPC call to ds-manager to free the DMA buffer
}
}
```

driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 8) - [public] MmioRegion

```
pub struct MmioRegion {
    pub phys_base: PhysAddr,
    pub virt_base: VirtAddr,
    pub size: u64,
    pub handle: Handle,
}

impl MmioRegion {
    pub fn map(manager_ep: Handle, phys_base: u64, size: u64) -> Result<Self, DsError> {
        let mut payload = MmioMapPayload {
            phys_addr: PhysAddr(phys_base),
            size,
            // ... (truncated, see source file for more)
        }
    }
}
```

driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 16) - [public] map

```
pub fn map(manager_ep: Handle, phys_base: u64, size: u64) -> Result<Self, DsError> {
    let mut payload = MmioMapPayload {
        phys_addr: PhysAddr(phys_base),
        size,
        flags: 0,
        _pad: 0,
    };

    let mut reply_buf = [0u64; 2];

    let res = unsafe {
        sys_ipc_call(
            // ... (truncated, see source file for more)
        )
    }
```

driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 67) - [public] Volatile

```
pub struct Volatile<T>(&mut T);

impl<T: Copy> Volatile<T> {
    #[inline(always)]
    pub fn read(&self) -> T {
        core::ptr::read_volatile(&self.0)
    }

    #[inline(always)]
    pub fn write(&mut self, value: T) {
        core::ptr::write_volatile(&mut self.0, value);
    }
    // ... (truncated, see source file for more)
}
```

driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 71) - [public] read

```
pub fn read(&self) -> T {
    core::ptr::read_volatile(&self.0)
}

#[inline(always)]
pub fn write(&mut self, value: T) {
    core::ptr::write_volatile(&mut self.0, value);
}
}
```

driverspace_workspace/lib/ds-mem/src/dma/mapping.rs (line 76) - [public] write

```
pub fn write(&mut self, value: T) {
    core::ptr::write_volatile(&mut self.0, value);
}
}
```

driverspace_workspace/lib/ds-mem/src/vmm.rs (line 6) - [public] map_memory

```
pub fn map_memory(
    manager_ep: Handle,
    phys_addr: u64,
    virt_addr: u64,
    size: u64,
    flags: u64,
) -> Result<(), DsError> {
    let mut payload = [phys_addr, virt_addr, size, flags];
    let mut reply_buf = [0u64; 1];

    let res = unsafe {
        sys_ipc_call(
            // ... (truncated, see source file for more)
        )
    }
```

driverspace_workspace/lib/ds-mem/src/vmm.rs (line 33) - [public] unmap_memory

```
pub fn unmap_memory(manager_ep: Handle, virt_addr: u64, size: u64) -> Result<(), DsError> {
    let mut payload = [virt_addr, size];
    let mut reply_buf = [0u64; 1];

    let res = unsafe {
        sys_ipc_call(
            manager_ep.0,
            Opcode::MemMapMmio as u32,
            payload.as_mut_ptr() as *mut u8,
            core::mem::size_of_val(&payload) as u32,
            reply_buf.as_mut_ptr() as *mut u8,
            core::mem::size_of_val(&reply_buf) as u32,
            // ... (truncated, see source file for more)
        )
    }
```

driverspace_workspace/lib/kapi-abi/build.rs (line 9) - [internal] main

```
/// Build script dla `kapi-abi`.
///
/// Crate jest w 100% `no_std` i nie generuje kodu — skrypt istnieje tylko po to,
/// aby Cargo miało poprawny entry point (pusty `build.rs` powoduje błąd
/// "main` function not found in crate `build_script_build`").
///
/// Gdy pojawi się generowanie nagłówek C/Odin z typów ABI, kod trafi tutaj.

fn main() {
    println!("cargo:rerun-if-changed=src");
}
```

driverspace_workspace/lib/kapi-abi/src/errors.rs (line 7) - [public] DsError

```
pub enum DsError {  
    /// ■■■■  
    Success = 0,  
  
    /// ■■/■■■■  
    Unknown = 1,  
  
    /// ■■■■■■■■■■  
    InvalidHandle = 2,  
  
    /// ■■■■■■■■■■  
    OutOfMemory = 3,  
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/errors.rs (line 44) - [public] from_u32

```
/// ■ u32 ■■■■ DsError■■■■■■■■■■ Unknown■  
  
pub fn from_u32(val: u32) -> Self {  
    match val {  
        0 => Self::Success,  
        1 => Self::Unknown,  
        2 => Self::InvalidHandle,  
        3 => Self::OutOfMemory,  
        4 => Self::PermissionDenied,  
        5 => Self::DeviceNotFound,  
        6 => Self::DeviceBusy,  
        7 => Self::InvalidMessage,  
        8 => Self::BufferTooSmall,  
        9 => Self::DeviceFault,  
        // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/opcodes.rs (line 6) - [public] Opcode

```
pub enum Opcode {  
    // === ■■■■ (0x0000 - 0x00FF) ===  
    /// ■■■ Manager ■■■■  
    DriverRegister = 0x0001,  
    /// ■■■■■/■  
    DriverUnregister = 0x0002,  
    /// ■■/■■■■  
    Heartbeat = 0x0003,  
  
    // === ■■■■ (0x0100 - 0x01FF) ===  
    /// ■■■■ Manager■■■■■■■  
    DevAttach = 0x0100,  
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/opcodes.rs (line 47) - [public] from_u32

```
pub fn from_u32(val: u32) -> Option<Self> {  
    // ■■■■■■■■■■■■■■■■■■■■■ match ■■■■  
    match val {  
        0x0001 => Some(Self::DriverRegister),  
        0x0002 => Some(Self::DriverUnregister),  
        0x0003 => Some(Self::Heartbeat),  
        0x0100 => Some(Self::DevAttach),  
        0x0101 => Some(Self::DevDetach),  
        0x0102 => Some(Self::DevBind),  
        0x0103 => Some(Self::DevStatusChange),  
        0x0200 => Some(Self::MemMapMmio),  
        0x0201 => Some(Self::MemAllocDma),  
        // ... (truncated, see source file for more)
```


driverspace_workspace/lib/kapi-abi/src/payloads/mem.rs (line 25) - [public] DmaAllocPayload

```
pub struct DmaAllocPayload {  
    /// XXXXXXXX  
    pub size: u64,  
  
    /// XXXXXXXX  
    pub align: u64,  
  
    /// XXXXXXXXXXXXXXXX  
    pub flags: u32,  
  
    /// XXXXXXXX  
    pub _pad: u32,  
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/payloads/sys.rs (line 9) - [public] LogPayload

```
pub struct LogPayload {  
    /// XXXXXXXX `ds-log` ■ `LogLevel`■  
    pub level: u8,  
  
    /// XXXXXXXX  
    pub module_len: u16,  
  
    /// XXXXXXXX  
    pub msg_len: u16,  
  
    /// XXXXXXXX  
    pub _pad: u32,  
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 8) - [public] Handle

```
pub struct Handle(pub u32);  
  
impl Handle {  
    pub const NULL: Self = Self(0);  
  
    #[inline(always)]  
    pub fn is_valid(&self) -> bool {  
        self.0 != 0  
    }  
}  
  
/// XXXXXXCapability ID■  
// ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 14) - [public] is_valid

```
pub fn is_valid(&self) -> bool {  
    self.0 != 0  
}  
  
/// XXXXXXCapability ID■  
/// XXXXXXXXXXXXXXXXXXXXXXXX  
#[derive(Debug, Clone, Copy, PartialEq, Eq, Hash)]  
#[repr(transparent)]  
pub struct CapId(pub u64);  
  
/// XXXXXXXXXXXXXXXXXXXX  
// ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 23) - [public] CapId

```
pub struct CapId(pub u64);

/// ████████████████████████████████████████
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
#[repr(transparent)]
pub struct PhysAddr(pub u64);

/// ████████
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
#[repr(transparent)]
pub struct VirtAddr(pub u64);
```

driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 28) - [public] PhysAddr

```
pub struct PhysAddr(pub u64);

/// ████████
#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord)]
#[repr(transparent)]
pub struct VirtAddr(pub u64);
```

driverspace_workspace/lib/kapi-abi/src/primitives.rs (line 33) - [public] VirtAddr

```
pub struct VirtAddr(pub u64);
```

driverspace_workspace/lib/kapi-syscall/src/lib.rs (line 16) - [internal] kapi_ipc_send

```
/// Wysyła jednokierunkowo wiadomość IPC do `target`.
/// Zwraca 0 przy sukcesie albo kod `DsError`.

fn kapi_ipc_send(target: u32, opcode: u32, payload: *const u8, payload_len: u32) -> i32;

/// Wysyła dane IPC i czeka na odpowiedź do bufora `reply`.
/// Zwraca 0 przy sukcesie albo kod `DsError`.
fn kapi_ipc_call(
    target: u32,
    opcode: u32,
    payload: *const u8,
    payload_len: u32,
    reply: *mut u8,
    reply_len: u32,
) -> i32;
// ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-syscall/src/lib.rs (line 20) - [internal] kapi_ipc_call

```
/// Wysyła dane IPC i czeka na odpowiedź do bufora `reply`.
/// Zwraca 0 przy sukcesie albo kod `DsError`.

fn kapi_ipc_call(
    target: u32,
    opcode: u32,
    payload: *const u8,
    payload_len: u32,
    reply: *mut u8,
    reply_len: u32,
) -> i32;
}

#[inline]
fn status(rc: i32) -> Result<(), DsError> {
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-syscall/src/lib.rs (line 31) - [internal] status

```
fn status(rc: i32) -> Result<(), DsError> {
    if rc == 0 {
        Ok(())
    } else {
        Err(DsError::from_u32(rc as u32))
    }
}

/// Wysyła wiadomość IPC (bez oczekiwania na odpowiedź).
pub fn sys_ipc_send(
    target: u32,
    opcode: u32,
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-syscall/src/lib.rs (line 40) - [public] sys_ipc_send

/// Wysyła wiadomość IPC (bez oczekiwania na odpowiedź).

```
pub fn sys_ipc_send(
    target: u32,
    opcode: u32,
    payload: *const u8,
    payload_len: u32,
) -> Result<(), DsError> {
    status(unsafe { kapi_ipc_send(target, opcode, payload, payload_len) })
}

/// Wysyła żądanie IPC i odbiera odpowiedź.
pub fn sys_ipc_call(
    target: u32,
    // ... (truncated, see source file for more)
```

driverspace_workspace/lib/kapi-syscall/src/lib.rs (line 50) - [public] sys_ipc_call

/// Wysyła żądanie IPC i odbiera odpowiedź.

```
pub fn sys_ipc_call(
    target: u32,
    opcode: u32,
    payload: *const u8,
    payload_len: u32,
    reply: *mut u8,
    reply_len: u32,
) -> Result<(), DsError> {
    status(unsafe {
        kapi_ipc_call(target, opcode, payload, payload_len, reply, reply_len)
    })
}
```

driverspace_lib/src/abi.rs (line 53) - [public] DsCmd

```
pub enum DsCmd {
    None = 0,
    Init = 1,
    Caps = 2,
    Ready = 3,
    RegisterDriver = 4,
    AttachDevice = 5,
    DetachDevice = 6,
    MapDeviceMemory = 7,
    BindIrq = 8,
    Shutdown = 9,
    Log = 10,
    // ... (truncated, see source file for more)
```

driverspacelib/src/abi.rs (line 83) - [public] DsRing

```
pub struct DsRing {
    pub head: u64,
    pub tail: u64,
    pub cap: u64,
}

pub const DS_RING_HDR_SIZE: usize = core::mem::size_of::<DsRing>();

/// A single message exchanged between kernel and driver space.
#[repr(C)]
#[derive(Clone, Copy, Default)]
pub struct DsMsg {
    // ... (truncated, see source file for more)
```

driverspacelib/src/abi.rs (line 94) - [public] DsMsg

```
pub struct DsMsg {
    pub id: u64,
    pub cmd: u32,
    pub flags: u32,
    pub arg0: u64,
    pub arg1: u64,
    pub arg2: u64,
    pub status: i32,
    pub pad: u32,
}

pub const DS_MSG_SIZE: usize = core::mem::size_of::<DsMsg>();
// ... (truncated, see source file for more)
```

driverspacelib/src/abi.rs (line 110) - [public] DsInitParams

```
pub struct DsInitParams {
    pub magic: u64,
    pub version: u32,
    pub pad: u32,
    pub k2d_va: u64,
    pub d2k_va: u64,
    pub ring_cap: u64,
    pub ds_va_base: u64,
    pub ds_va_size: u64,
}

/// Library-side error type for failed driver-space calls.
// ... (truncated, see source file for more)
```

driverspacelib/src/abi.rs (line 123) - [public] DsError

```
pub enum DsError {
    Timeout,
    QueueFull,
    BadStatus(i32),
    NoAspace,
    NoMemory,
    NotPrepared,
}

impl From<i32> for DsError {
    fn from(v: i32) -> Self {
        DsError::BadStatus(v)
        // ... (truncated, see source file for more)
```

driverspacelib/src/abi.rs (line 133) - [internal] from

```
fn from(v: i32) -> Self {
    DsError::BadStatus(v)
}
}
```

driverspacelib/src/audio.rs (line 4) - [public] AudioBars

```
pub struct AudioBars {
    pub nam_phys: u64,
    pub bm_phys: u64,
}

pub fn info_req() -> u64 {
    request(DsCmd::AudioInfo, 0, 0, 0)
}

pub fn info_take(id: u64) -> Option<AudioBars> {
    let r = take_resp(id)?;

    // ... (truncated, see source file for more)
```

driverspacelib/src/audio.rs (line 9) - [public] info_req

```
pub fn info_req() -> u64 {
    request(DsCmd::AudioInfo, 0, 0, 0)
}

pub fn info_take(id: u64) -> Option<AudioBars> {
    let r = take_resp(id)?;

    if r.status != 0 {
        return None;
    }

    Some(AudioBars { nam_phys: r.arg0, bm_phys: r.arg1 })
    // ... (truncated, see source file for more)
```

driverspacelib/src/audio.rs (line 13) - [public] info_take

```
pub fn info_take(id: u64) -> Option<AudioBars> {
    let r = take_resp(id)?;

    if r.status != 0 {
        return None;
    }

    Some(AudioBars { nam_phys: r.arg0, bm_phys: r.arg1 })
}

pub fn page_phys_req(va: u64) -> u64 {
    request(DsCmd::PagePhys, va, 0, 0)
    // ... (truncated, see source file for more)
```

driverspacelib/src/audio.rs (line 23) - [public] page_phys_req

```
pub fn page_phys_req(va: u64) -> u64 {
    request(DsCmd::PagePhys, va, 0, 0)
}

pub fn page_phys_take(id: u64) -> Option<u64> {
    let r = take_resp(id)?;

    if r.status != 0 {
        return None;
    }

    Some(r.arg0)
    // ... (truncated, see source file for more)
```

driverspacelib/src/audio.rs (line 27) - [public] page_phys_take

```
pub fn page_phys_take(id: u64) -> Option<u64> {
    let r = take_resp(id)?;

    if r.status != 0 {
        return None;
    }

    Some(r.arg0)
}
```

driverspacelib/src/driver.rs (line 6) - [public] DeviceInfo

```
pub struct DeviceInfo {
    pub device_id: u64,
    pub vendor_id: u64,
    pub class_code: u64,
}

/// Any driver that lives in driver space and is registered with the kernel
/// driver-space manager.
pub trait Driver {
    fn init(&mut self, info: &DeviceInfo) -> Result<(), DsError>;
}
```

driverspacelib/src/driver.rs (line 14) - [public] Driver

```
/// Any driver that lives in driver space and is registered with the kernel
/// driver-space manager.

pub trait Driver {
    fn init(&mut self, info: &DeviceInfo) -> Result<(), DsError>;
}
```

driverspacelib/src/driver.rs (line 15) - [internal] init

```
fn init(&mut self, info: &DeviceInfo) -> Result<(), DsError>;
}
```

driverspacelib/src/input.rs (line 4) - [public] key_req

```
pub fn key_req() -> u64 {
    svc::call(SVC_INPUT, IN_KEY_POLL, 0, 0, 0)
}

pub fn key_take(id: u64) -> Option<u8> {
    let r = svc::take(id)?;

    if r.arg0 == 0 { None } else { Some(r.arg0 as u8) }
}
```

driverspacelib/src/input.rs (line 8) - [public] key_take

```
pub fn key_take(id: u64) -> Option<u8> {
    let r = svc::take(id)?;

    if r.arg0 == 0 { None } else { Some(r.arg0 as u8) }
}
```

driverspacelib/src/jack.rs (line 4) - [public] JackInfo

```
pub struct JackInfo {
    pub present: bool,
    pub amp_on: bool,
}

pub fn query_req() -> u64 {
    request(DsCmd::JackQuery, 0, 0, 0)
}

pub fn query_take(id: u64) -> Option<JackInfo> {
    let r = take_resp(id)?;

    // ... (truncated, see source file for more)
}
```

driverspacelib/src/jack.rs (line 9) - [public] query_req

```
pub fn query_req() -> u64 {
    request(DsCmd::JackQuery, 0, 0, 0)
}

pub fn query_take(id: u64) -> Option<JackInfo> {
    let r = take_resp(id)?;

    Some(JackInfo {
        present: r.arg0 & 1 != 0,
        amp_on: r.arg0 & 2 != 0,
    })
}

// ... (truncated, see source file for more)
```

driverspacelib/src/jack.rs (line 13) - [public] query_take

```
pub fn query_take(id: u64) -> Option<JackInfo> {
    let r = take_resp(id)?;

    Some(JackInfo {
        present: r.arg0 & 1 != 0,
        amp_on: r.arg0 & 2 != 0,
    })
}

pub fn set_amp(on: bool) {
    request(DsCmd::JackSetAmp, on as u64, 0, 0);
}

// ... (truncated, see source file for more)
```

driverspacelib/src/jack.rs (line 22) - [public] set_amp

```
pub fn set_amp(on: bool) {
    request(DsCmd::JackSetAmp, on as u64, 0, 0);
}

pub fn play(va: u64, len: u32) -> u64 {
    request(DsCmd::AudioPlay, va, len as u64, 0)
}

pub fn stop() {
    request(DsCmd::AudioStop, 0, 0, 0);
}
```

driverspacelib/src/jack.rs (line 26) - [public] play

```
pub fn play(va: u64, len: u32) -> u64 {
    request(DsCmd::AudioPlay, va, len as u64, 0)
}

pub fn stop() {
    request(DsCmd::AudioStop, 0, 0, 0);
}
```

driverspacelib/src/jack.rs (line 30) - [public] stop

```
pub fn stop() {
    request(DsCmd::AudioStop, 0, 0, 0);
}
```

driverspacelib/src/log.rs (line 10) - [public] ds_log

/// Wysyła tekst do logu i dra (komenda `DsCmd::Log`).

```
pub fn ds_log(msg: &str) {
    let _ = request(DsCmd::Log, msg.as_ptr() as u64, msg.len() as u64, 0);
}

/// To samo, ale dla surowego wskaźnika i długości (np. bufora bajtów).
pub fn ds_log_raw(ptr: *const u8, len: usize) {
    let _ = request(DsCmd::Log, ptr as u64, len as u64, 0);
}
```


driverspacelib/src/log.rs (line 15) - [public] ds_log_raw

/// To samo, ale dla surowego wskaźnika i długości (np. bufora bajtów).

```
pub fn ds_log_raw(ptr: *const u8, len: usize) {
    let _ = request(DsCmd::Log, ptr as u64, len as u64, 0);
}
```

driverspacelib/src/runtime.rs (line 38) - [public] init_once

/// Reads the boot parameters block and points our ring views at the

/// kernel-provided buffers.

```
pub fn init_once(params_va: u64) {
    unsafe {
        let p = &*(params_va as *const DsInitParams);

        K2D = p.k2d_va as *mut DsRing;
        D2K = p.d2k_va as *mut DsRing;
        K2D_MSGS = (p.k2d_va + DS_RING_HDR_SIZE as u64) as *mut DsMsg;
        D2K_MSGS = (p.d2k_va + DS_RING_HDR_SIZE as u64) as *mut DsMsg;
    }

    /// Registers a driver (by calling its `Driver::init` hook) and notifies
    // ... (truncated, see source file for more)
```

driverspacelib/src/runtime.rs (line 51) - [public] register

/// Registers a driver (by calling its `Driver::init` hook) and notifies

/// the kernel that this driver space is now managed.

```
pub fn register<D: Driver>(drv: &mut D) {
    let _ = drv.init(&DeviceInfo::default());
    let _ = request(DsCmd::RegisterDriver, 0, 0, 0);
}

unsafe fn push_raw(cmd: u32, a0: u64, a1: u64, a2: u64) -> u64 {
    let ring = D2K;
    if ring.is_null() {
        return 0;
    }

    let id = NEXT_ID;
    // ... (truncated, see source file for more)
```

driverspacelib/src/runtime.rs (line 94) - [public] request

/// Sends a driver-space command to the kernel. Returns the id used later

/// with [take_resp].

```
pub fn request(cmd: DsCmd, a0: u64, a1: u64, a2: u64) -> u64 {
    unsafe { push_raw(cmd as u32, a0, a1, a2) }
}

/// Sends a raw (service-class packed) command to the kernel.
pub fn request_raw(cmd: u32, a0: u64, a1: u64, a2: u64) -> u64 {
    unsafe { push_raw(cmd, a0, a1, a2) }
}

/// Drains pending kernel->driver responses into the local cache.
pub fn tick() {
    unsafe {
        // ... (truncated, see source file for more)
```

driverspacelib/src/runtime.rs (line 99) - [public] request_raw

/// Sends a raw (service-class packed) command to the kernel.

```
pub fn request_raw(cmd: u32, a0: u64, a1: u64, a2: u64) -> u64 {
    unsafe { push_raw(cmd, a0, a1, a2) }
}
```

/// Drains pending kernel->driver responses into the local cache.

```
pub fn tick() {
    unsafe {
        let ring = K2D;
        if ring.is_null() {
            return;
        }
    }
}
```

// ... (truncated, see source file for more)

driverspacelib/src/runtime.rs (line 104) - [public] tick

/// Drains pending kernel->driver responses into the local cache.

```
pub fn tick() {
    unsafe {
        let ring = K2D;
        if ring.is_null() {
            return;
        }

        while (*ring).tail != (*ring).head {
            let slot = K2D_MSGS.add((*ring).tail as usize);
            let copy = *slot;
            (*ring).tail = ((*ring).tail + 1) % (*ring).cap;
        }
    }
}
```

// ... (truncated, see source file for more)

driverspacelib/src/runtime.rs (line 128) - [public] take_resp

/// Returns (and consumes) the response with the given request id.

```
pub fn take_resp(id: u64) -> Option<DsMsg> {
    unsafe {
        for i in 0..RESP_CACHE {
            if CACHE_USED[i] && CACHE_MSG[i].id == id {
                let m = CACHE_MSG[i];
                CACHE_USED[i] = false;
                return Some(m);
            }
        }
    }
    None
}
```

// ... (truncated, see source file for more)

driverspacelib/src/svc.rs (line 4) - [public] call

```
pub fn call(class: u32, op: u32, a0: u64, a1: u64, a2: u64) -> u64 {
    request_raw(svc_cmd(class, op), a0, a1, a2)
}
```

```
pub fn take(id: u64) -> Option<DsMsg> {
    take_resp(id)
}
```

driverspacelib/src/svc.rs (line 8) - [public] take

```
pub fn take(id: u64) -> Option<DsMsg> {
    take_resp(id)
}
```

driverspacelib/src/video.rs (line 4) - [public] FbInfo

```
pub struct FbInfo {
    pub width: u32,
    pub height: u32,
    pub stride: u32,
    pub phys: u64,
}

pub fn fb_info_req() -> u64 {
    svc::call(SVC_VIDEO, VID_FB_INFO, 0, 0, 0)
}

pub fn fb_info_take(id: u64) -> Option<FbInfo> {
    // ... (truncated, see source file for more)
```

driverspacelib/src/video.rs (line 11) - [public] fb_info_req

```
pub fn fb_info_req() -> u64 {
    svc::call(SVC_VIDEO, VID_FB_INFO, 0, 0, 0)
}

pub fn fb_info_take(id: u64) -> Option<FbInfo> {
    let r = svc::take(id)?;

    Some(FbInfo {
        width: (r.arg0 >> 16) as u32,
        height: (r.arg0 & 0xFFFF) as u32,
        stride: r.arg1 as u32,
        phys: r.arg2,
    })
    // ... (truncated, see source file for more)
```

driverspacelib/src/video.rs (line 15) - [public] fb_info_take

```
pub fn fb_info_take(id: u64) -> Option<FbInfo> {
    let r = svc::take(id)?;

    Some(FbInfo {
        width: (r.arg0 >> 16) as u32,
        height: (r.arg0 & 0xFFFF) as u32,
        stride: r.arg1 as u32,
        phys: r.arg2,
    })
}

pub fn takeover() -> u64 {
    // ... (truncated, see source file for more)
```

driverspacelib/src/video.rs (line 26) - [public] takeover

```
pub fn takeover() -> u64 {
    svc::call(SVC_VIDEO, VID_FB_TAKEOVER, 0, 0, 0)
}

pub fn release() -> u64 {
    svc::call(SVC_VIDEO, VID_FB_RELEASE, 0, 0, 0)
}
```

driverspacelib/src/video.rs (line 30) - [public] release

```
pub fn release() -> u64 {
    svc::call(SVC_VIDEO, VID_FB_RELEASE, 0, 0, 0)
}
```

kernel_workspace/C_base/kc-abi-menager/src/main.rs (line 2) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

kernel_Workspace/C_base/kc-abi/src/lib.rs (line 2) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/C_base/kc-abi/src/lib.rs (line 11) - [internal] it_works

```
fn it_works() {
    let result = add(2, 2);
    assert_eq!(result, 4);
}
```

kernel_Workspace/Odin_base/odin-abi-bridge/src/lib.rs (line 2) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/Odin_base/odin-abi-bridge/src/lib.rs (line 11) - [internal] it_works

```
fn it_works() {
    let result = add(2, 2);
    assert_eq!(result, 4);
}
```

kernel_Workspace/Odin_base/odin-abi/src/lib.rs (line 2) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/Odin_base/odin-abi/src/lib.rs (line 11) - [internal] it_works

```
fn it_works() {
    let result = add(2, 2);
    assert_eq!(result, 4);
}
```

kernel_Workspace/Odin_base/odin-bin-loader/src/main.rs (line 2) - [internal] main

```
fn main() {
    println!("Hello, world!");
}
```

kernel_workspace/base/kw-C-abi/src/lib.rs (line 2) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-C-abi/src/lib.rs (line 11) - [internal] it_works

```
fn it_works() {
    let result = add(2, 2);
    assert_eq!(result, 4);
}
}
```

kernel_workspace/base/kw-base/src/api/dispatch.rs (line 21) - [public] dispatch

// Main syscall dispatcher. Called from the trampoline/assembly entry point.

// Returns i64: positive for success/data, negative for KernelError codes.

```
pub fn dispatch(num: u64, a0: u64, a1: u64, a2: u64, a3: u64) -> i64 {
    let res: Result<i64, KernelError> = match num {
        SYS_READ => fs_ops::read(a0 as u32, a1 as *mut u8, a2 as usize).map(|v| v as i64),
        SYS_WRITE => fs_ops::write(a0 as u32, a1 as *const u8, a2 as usize).map(|v| v as i64),
        SYS_OPEN => fs_ops::open(a0 as *const u8, a1 as u32).map(|v| v as i64),
        SYS_CLOSE => fs_ops::close(a0 as u32).map(|_| 0),

        SYS_MMAP => memory::mmap(a0, a1 as usize, a2 as u32, a3 as u32).map(|v| v as i64),
        SYS_MUNMAP => memory::munmap(a0, a1 as usize).map(|_| 0),

        SYS_SPAWN => process::spawn(a0 as u32, a1, a2).map(|v| v as i64),
        SYS_EXIT => { process::exit(a0 as i32); },
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/api/fs_ops.rs (line 4) - [public] open

```
pub fn open(path_ptr: *const u8, flags: u32) -> KResult<u32> {
    if path_ptr.is_null() { return Err(KernelError::BadAddress); }
    let path = unsafe { core::ffi::CStr::from_ptr(path_ptr as *const i8) }
        .to_str().map_err(|_| KernelError::InvalidArg)?;
    let handle = Vfs::open(path, flags)?;
    Ok(handle.0)
}

pub fn read(fd: u32, buf: *mut u8, len: usize) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let slice = unsafe { core::slice::from_raw_parts_mut(buf, len) };
    Vfs::read(Handle(fd as i32), slice)
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/api/fs_ops.rs (line 12) - [public] read

```
pub fn read(fd: u32, buf: *mut u8, len: usize) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let slice = unsafe { core::slice::from_raw_parts_mut(buf, len) };
    Vfs::read(Handle(fd as i32), slice)
}

pub fn write(fd: u32, buf: *const u8, len: usize) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let slice = unsafe { core::slice::from_raw_parts(buf, len) };
    Vfs::write(Handle(fd as i32), slice)
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/api/fs_ops.rs (line 18) - [public] write

```
pub fn write(fd: u32, buf: *const u8, len: usize) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let slice = unsafe { core::slice::from_raw_parts(buf, len) };
    Vfs::write(Handle(fd as i32), slice)
}

pub fn close(fd: u32) -> KResult<()> {
    extern "C" { fn k_fs_close(fd: i32) -> i32; }
    let rc = unsafe { k_fs_close(fd as i32) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(()) }
}
```

kernel_workspace/base/kw-base/src/api/fs_ops.rs (line 24) - [public] close

```
pub fn close(fd: u32) -> KResult<()> {
    extern "C" { fn k_fs_close(fd: i32) -> i32; }
    let rc = unsafe { k_fs_close(fd as i32) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(()) }
}
```

kernel_workspace/base/kw-base/src/api/memory.rs (line 8) - [public] mmap

```
pub fn mmap(addr: u64, len: usize, prot: u32, _flags: u32) -> KResult<u64> {
    if len == 0 { return Err(KernelError::InvalidArg); }

    let mut vmm_flags = 0;
    if prot & PROT_READ != 0 { vmm_flags |= vmm::VMM_FLAG_READ; }
    if prot & PROT_WRITE != 0 { vmm_flags |= vmm::VMM_FLAG_WRITE; }
    if prot & PROT_EXEC != 0 { vmm_flags |= vmm::VMM_FLAG_EXEC; }

    let virt = vmm::alloc(len, vmm_flags).map_err(KernelError::from)?;
    Ok(virt.0)
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/api/memory.rs (line 20) - [public] munmap

```
pub fn munmap(addr: u64, len: usize) -> KResult<()> {
    if len == 0 { return Err(KernelError::InvalidArg); }
    vmm::free(kstd_base::VirtAddr(addr), len).map_err(KernelError::from)
}
```

kernel_workspace/base/kw-base/src/api/net_ops.rs (line 4) - [internal] k_net_socket

```
fn k_net_socket(domain: u32, sock_type: u32, proto: u32) -> i32;
fn k_net_bind(fd: i32, addr: *const u8, len: u32) -> i32;
fn k_net_send(fd: i32, buf: *const u8, len: u32, flags: u32) -> i32;
fn k_net_recv(fd: i32, buf: *mut u8, len: u32, flags: u32) -> i32;

pub fn socket(domain: u32, sock_type: u32, proto: u32) -> KResult<u32> {
    let rc = unsafe { k_net_socket(domain, sock_type, proto) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as u32) }
}

pub fn send(fd: u32, buf: *const u8, len: u32, flags: u32) -> KResult<usize> {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/api/net_ops.rs (line 5) - [internal] k_net_bind

```
fn k_net_bind(fd: i32, addr: *const u8, len: u32) -> i32;
fn k_net_send(fd: i32, buf: *const u8, len: u32, flags: u32) -> i32;
fn k_net_recv(fd: i32, buf: *mut u8, len: u32, flags: u32) -> i32;

pub fn socket(domain: u32, sock_type: u32, proto: u32) -> KResult<u32> {
    let rc = unsafe { k_net_socket(domain, sock_type, proto) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as u32) }
}

pub fn send(fd: u32, buf: *const u8, len: u32, flags: u32) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/api/net_ops.rs (line 6) - [internal] k_net_send

```
fn k_net_send(fd: i32, buf: *const u8, len: u32, flags: u32) -> i32;
fn k_net_recv(fd: i32, buf: *mut u8, len: u32, flags: u32) -> i32;
}

pub fn socket(domain: u32, sock_type: u32, proto: u32) -> KResult<u32> {
    let rc = unsafe { k_net_socket(domain, sock_type, proto) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as u32) }
}

pub fn send(fd: u32, buf: *const u8, len: u32, flags: u32) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let rc = unsafe { k_net_send(fd as i32, buf, len, flags) };
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/api/net_ops.rs (line 7) - [internal] k_net_recv

```
fn k_net_recv(fd: i32, buf: *mut u8, len: u32, flags: u32) -> i32;
}

pub fn socket(domain: u32, sock_type: u32, proto: u32) -> KResult<u32> {
    let rc = unsafe { k_net_socket(domain, sock_type, proto) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as u32) }
}

pub fn send(fd: u32, buf: *const u8, len: u32, flags: u32) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let rc = unsafe { k_net_send(fd as i32, buf, len, flags) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/api/net_ops.rs (line 10) - [public] socket

```
pub fn socket(domain: u32, sock_type: u32, proto: u32) -> KResult<u32> {
    let rc = unsafe { k_net_socket(domain, sock_type, proto) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as u32) }
}

pub fn send(fd: u32, buf: *const u8, len: u32, flags: u32) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let rc = unsafe { k_net_send(fd as i32, buf, len, flags) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}

pub fn recv(fd: u32, buf: *mut u8, len: u32, flags: u32) -> KResult<usize> {
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/api/net_ops.rs (line 15) - [public] send

```
pub fn send(fd: u32, buf: *const u8, len: u32, flags: u32) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let rc = unsafe { k_net_send(fd as i32, buf, len, flags) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}

pub fn recv(fd: u32, buf: *mut u8, len: u32, flags: u32) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let rc = unsafe { k_net_recv(fd as i32, buf, len, flags) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}
```

kernel_Workspace/base/kw-base/src/api/net_ops.rs (line 21) - [public] recv

```
pub fn recv(fd: u32, buf: *mut u8, len: u32, flags: u32) -> KResult<usize> {
    if buf.is_null() { return Err(KernelError::BadAddress); }
    let rc = unsafe { k_net_recv(fd as i32, buf, len, flags) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}
```

```

kernel_Workspace/base/kw-base/src/api/process.rs (line 4) - [internal] k_proc_spawn
    fn k_proc_spawn(world_id: u32, entry: u64, stack: u64) -> i64;
    fn k_proc_exit(code: i32);
    fn k_proc_yield();
    fn k_proc_get_pid() -> u32;
}

pub fn spawn(world_id: u32, entry: u64, stack: u64) -> KResult<u64> {
    let rc = unsafe { k_proc_spawn(world_id, entry, stack) };
    if rc < 0 { Err(KernelError::from_code(rc as i32)) } else { Ok(rc as u64) }
}

pub fn exit(code: i32) -> ! {
    // ... (truncated, see source file for more)
}

kernel_Workspace/base/kw-base/src/api/process.rs (line 5) - [internal] k_proc_exit
    fn k_proc_exit(code: i32);
    fn k_proc_yield();
    fn k_proc_get_pid() -> u32;
}

pub fn spawn(world_id: u32, entry: u64, stack: u64) -> KResult<u64> {
    let rc = unsafe { k_proc_spawn(world_id, entry, stack) };
    if rc < 0 { Err(KernelError::from_code(rc as i32)) } else { Ok(rc as u64) }
}

pub fn exit(code: i32) -> ! {
    unsafe { k_proc_exit(code); }
    // ... (truncated, see source file for more)
}

kernel_Workspace/base/kw-base/src/api/process.rs (line 6) - [internal] k_proc_yield
    fn k_proc_yield();
    fn k_proc_get_pid() -> u32;
}

pub fn spawn(world_id: u32, entry: u64, stack: u64) -> KResult<u64> {
    let rc = unsafe { k_proc_spawn(world_id, entry, stack) };
    if rc < 0 { Err(KernelError::from_code(rc as i32)) } else { Ok(rc as u64) }
}

pub fn exit(code: i32) -> ! {
    unsafe { k_proc_exit(code); }
    loop { core::hint::spin_loop(); }
    // ... (truncated, see source file for more)
}

kernel_Workspace/base/kw-base/src/api/process.rs (line 7) - [internal] k_proc_get_pid
    fn k_proc_get_pid() -> u32;
}

pub fn spawn(world_id: u32, entry: u64, stack: u64) -> KResult<u64> {
    let rc = unsafe { k_proc_spawn(world_id, entry, stack) };
    if rc < 0 { Err(KernelError::from_code(rc as i32)) } else { Ok(rc as u64) }
}

pub fn exit(code: i32) -> ! {
    unsafe { k_proc_exit(code); }
    loop { core::hint::spin_loop(); }
}
// ... (truncated, see source file for more)

```


kernel_Workspace/base/kw-base/src/api/process.rs (line 10) - [public] spawn

```
pub fn spawn(world_id: u32, entry: u64, stack: u64) -> KResult<u64> {
    let rc = unsafe { k_proc_spawn(world_id, entry, stack) };
    if rc < 0 { Err(KernelError::from_code(rc as i32)) } else { Ok(rc as u64) }
}

pub fn exit(code: i32) -> ! {
    unsafe { k_proc_exit(code); }
    loop { core::hint::spin_loop(); }
}

pub fn yield_now() {
    unsafe { k_proc_yield(); }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/api/process.rs (line 15) - [public] exit

```
pub fn exit(code: i32) -> ! {
    unsafe { k_proc_exit(code); }
    loop { core::hint::spin_loop(); }
}

pub fn yield_now() {
    unsafe { k_proc_yield(); }
}

pub fn get_pid() -> u32 {
    unsafe { k_proc_get_pid() }
}
```

kernel_Workspace/base/kw-base/src/api/process.rs (line 20) - [public] yield_now

```
pub fn yield_now() {
    unsafe { k_proc_yield(); }
}

pub fn get_pid() -> u32 {
    unsafe { k_proc_get_pid() }
}
```

kernel_Workspace/base/kw-base/src/api/process.rs (line 24) - [public] get_pid

```
pub fn get_pid() -> u32 {
    unsafe { k_proc_get_pid() }
}
```

kernel_Workspace/base/kw-base/src/arch/interrupts.rs (line 4) - [public] disable

```
pub fn disable() {
    #[cfg(target_arch = "x86_64")]
    unsafe { core::arch::asm!("cli", options(nomem, nostack)); }
}

#[inline(always)]
pub fn enable() {
    #[cfg(target_arch = "x86_64")]
    unsafe { core::arch::asm!("sti", options(nomem, nostack)); }
}

#[inline(always)]
// ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/arch/interrupts.rs (line 10) - [public] enable

```
pub fn enable() {
    #[cfg(target_arch = "x86_64")]
    unsafe { core::arch::asm!("sti", options(nomem, nostack)); }
}

#[inline(always)]
pub fn halt() {
    unsafe { core::arch::asm!("hlt", options(nomem, nostack)); }
}

#[inline(always)]
pub fn are_enabled() -> bool {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/arch/interrupts.rs (line 16) - [public] halt

```
pub fn halt() {
    unsafe { core::arch::asm!("hlt", options(nomem, nostack)); }
}

#[inline(always)]
pub fn are_enabled() -> bool {
    #[cfg(target_arch = "x86_64")]
    {
        let flags: u64;
        unsafe { core::arch::asm!("pushfq; pop {}", out(reg) flags, options(nomem)); }
        (flags & (1 << 9)) != 0
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/arch/interrupts.rs (line 21) - [public] are_enabled

```
pub fn are_enabled() -> bool {
    #[cfg(target_arch = "x86_64")]
    {
        let flags: u64;
        unsafe { core::arch::asm!("pushfq; pop {}", out(reg) flags, options(nomem)); }
        (flags & (1 << 9)) != 0
    }
    #[cfg(not(target_arch = "x86_64"))]
    { false }
}

pub struct InterruptGuard {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/arch/interrupts.rs (line 32) - [public] InterruptGuard

```
pub struct InterruptGuard {
    was_enabled: bool,
}

impl InterruptGuard {
    pub fn disable() -> Self {
        let was_enabled = are_enabled();
        disable();
        Self { was_enabled }
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/arch/interrupts.rs (line 37) - [public] disable

```
pub fn disable() -> Self {
    let was_enabled = are_enabled();
    disable();
    Self { was_enabled }
}

impl Drop for InterruptGuard {
    fn drop(&mut self) {
        if self.was_enabled { enable(); }
    }
}
```

kernel_workspace/base/kw-base/src/arch/interrupts.rs (line 45) - [internal] drop

```
fn drop(&mut self) {
    if self.was_enabled { enable(); }
}
```

kernel_workspace/base/kw-base/src/arch/registers.rs (line 3) - [public] CpuContext

```
pub struct CpuContext {
    pub r15: u64, pub r14: u64, pub r13: u64, pub r12: u64,
    pub rbp: u64, pub rbx: u64,
    pub r11: u64, pub r10: u64, pub r9: u64, pub r8: u64,
    pub rax: u64, pub rcx: u64, pub rdx: u64, pub rsi: u64,
    pub rdi: u64,
    pub rip: u64,
    pub cs: u64,
    pub rflags: u64,
    pub rsp: u64,
    pub ss: u64,
}
```

kernel_workspace/base/kw-base/src/arch/x86_64.rs (line 30) - [public] read_cr3

```
pub fn read_cr3() -> u64 {
    let val: u64;
    unsafe { asm!("mov {}, cr3", out(reg) val, options(nomem, nostack, preserves_flags)); }
    val
}

#[inline(always)]
pub fn write_cr3(val: u64) {
    unsafe { asm!("mov cr3, {}", in(reg) val, options(nomem, nostack, preserves_flags)); }
}
```

kernel_workspace/base/kw-base/src/arch/x86_64.rs (line 37) - [public] write_cr3

```
pub fn write_cr3(val: u64) {
    unsafe { asm!("mov cr3, {}", in(reg) val, options(nomem, nostack, preserves_flags)); }
}
```

kernel_workspace/base/kw-base/src/boot/early_print.rs (line 5) - [public] init_serial

```
pub fn init_serial() {
    unsafe {
        outb(COM1 + 1, 0x00); // Disable interrupts
        outb(COM1 + 3, 0x80); // Enable DLAB
        outb(COM1 + 0, 0x03); // 38400 baud
        outb(COM1 + 1, 0x00);
        outb(COM1 + 3, 0x03); // 8 bits, no parity, one stop bit
        outb(COM1 + 2, 0xC7); // Enable FIFO
        outb(COM1 + 4, 0x0B); // IRQs enabled, RTS/DSR set
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/boot/early_print.rs (line 17) - [internal]

is_transmitter_empty

```
fn is_transmitter_empty() -> bool {
    unsafe { (inb(COM1 + 5) & 0x20) != 0 }
}

pub fn print_char(c: u8) {
    while !is_transmitter_empty() { core::hint::spin_loop(); }
    unsafe { outb(COM1, c); }
}

pub fn print_str(s: &str) {
    for b in s.bytes() {
        if b == b'\n' { print_char(b'\r'); }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/boot/early_print.rs (line 21) - [public] print_char

```
pub fn print_char(c: u8) {
    while !is_transmitter_empty() { core::hint::spin_loop(); }
    unsafe { outb(COM1, c); }
}

pub fn print_str(s: &str) {
    for b in s.bytes() {
        if b == b'\n' { print_char(b'\r'); }
        print_char(b);
    }
}
```

kernel_workspace/base/kw-base/src/boot/early_print.rs (line 26) - [public] print_str

```
pub fn print_str(s: &str) {
    for b in s.bytes() {
        if b == b'\n' { print_char(b'\r'); }
        print_char(b);
    }
}
```

kernel_workspace/base/kw-base/src/boot/memory_detect.rs (line 5) - [public] MemoryRegion

```
pub struct MemoryRegion {
    pub base: PhysAddr,
    pub size: Size,
    pub is_usable: bool,
}

pub fn extract_memory_map(info: &BootInfo) -> (usize, [MemoryRegion; 32]) {
    let mut regions = [MemoryRegion { base: PhysAddr(0), size: Size(0), is_usable: false }; 32];
    let mut count = 0;

    let tag_ptr = info.find_tag(TAG_TYPE_MMAP);
    if tag_ptr.is_null() { return (0, regions); }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/boot/memory_detect.rs (line 11) - [public]

extract_memory_map

```
pub fn extract_memory_map(info: &BootInfo) -> (usize, [MemoryRegion; 32]) {
    let mut regions = [MemoryRegion { base: PhysAddr(0), size: Size(0), is_usable: false }; 32];
    let mut count = 0;

    let tag_ptr = info.find_tag(TAG_TYPE_MMAP);
    if tag_ptr.is_null() { return (0, regions); }

    let mmap = unsafe { &*(tag_ptr as *const MmapTag) };
    let mut entry_ptr = unsafe { (tag_ptr as *const u8).add(core::mem::size_of::<MmapTag>()) };
    let end_ptr = unsafe { (tag_ptr as *const u8).add(mmap.tag.size as usize) };

    while entry_ptr < end_ptr && count < 32 {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/boot/multiboot.rs (line 6) - [public] Multiboot2Info

```
pub struct Multiboot2Info {
    pub total_size: u32,
    pub reserved: u32,
}

#[repr(C)]
pub struct Multiboot2Tag {
    pub tag_type: u32,
    pub size: u32,
}

pub const TAG_TYPE_END: u32 = 0;
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/boot/multiboot.rs (line 12) - [public] Multiboot2Tag

```
pub struct Multiboot2Tag {
    pub tag_type: u32,
    pub size: u32,
}

pub const TAG_TYPE_END: u32 = 0;
pub const TAG_TYPE_CMDLINE: u32 = 1;
pub const TAG_TYPE_BOOTLOADER: u32 = 2;
pub const TAG_TYPE_MODULE: u32 = 3;
pub const TAG_TYPE_BASIC_MEMINFO: u32 = 4;
pub const TAG_TYPE_BOOTDEV: u32 = 5;
pub const TAG_TYPE_MMAP: u32 = 6;
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/boot/multiboot.rs (line 27) - [public] MmapTag

```
pub struct MmapTag {
    pub tag: Multiboot2Tag,
    pub entry_size: u32,
    pub entry_version: u32,
}

#[repr(C)]
pub struct MmapEntry {
    pub base_addr: u64,
    pub length: u64,
    pub entry_type: u32,
    pub reserved: u32,
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/boot/multiboot.rs (line 34) - [public] MmapEntry

```
pub struct MmapEntry {
    pub base_addr: u64,
    pub length: u64,
    pub entry_type: u32,
    pub reserved: u32,
}

pub struct BootInfo {
    pub ptr: *const Multiboot2Info,
}

unsafe impl Send for BootInfo {}
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/boot/multiboot.rs (line 41) - [public] BootInfo

```
pub struct BootInfo {
    pub ptr: *const Multiboot2Info,
}

unsafe impl Send for BootInfo {}
unsafe impl Sync for BootInfo {}

impl BootInfo {
    pub unsafe fn from_ptr(ptr: *const u8) -> Self {
        Self { ptr: ptr as *const Multiboot2Info }
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/boot/multiboot.rs (line 53) - [public] first_tag

```
pub fn first_tag(&self) -> *const Multiboot2Tag {
    unsafe { (self.ptr as *const u8).add(8) as *const Multiboot2Tag }
}

pub fn next_tag(tag: *const Multiboot2Tag) -> *const Multiboot2Tag {
    unsafe {
        let addr = tag as usize;
        let next = (addr + (*tag).size as usize + 7) & !7;
        next as *const Multiboot2Tag
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/boot/multiboot.rs (line 57) - [public] next_tag

```
pub fn next_tag(tag: *const Multiboot2Tag) -> *const Multiboot2Tag {
    unsafe {
        let addr = tag as usize;
        let next = (addr + (*tag).size as usize + 7) & !7;
        next as *const Multiboot2Tag
    }
}

pub fn find_tag(&self, tag_type: u32) -> *const Multiboot2Tag {
    let mut tag = self.first_tag();
    unsafe {
        while (*tag).tag_type != TAG_TYPE_END {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/base/kw-base/src/boot/multiboot.rs (line 65) - [public] find_tag

```
pub fn find_tag(&self, tag_type: u32) -> *const Multiboot2Tag {
    let mut tag = self.first_tag();
    unsafe {
        while (*tag).tag_type != TAG_TYPE_END {
            if (*tag).tag_type == tag_type { return tag; }
            tag = Self::next_tag(tag);
        }
    }
    core::ptr::null()
}
```

kernel_Workspace/base/kw-base/src/boot/stack.rs (line 2) - [public] BootStack

```
pub struct BootStack {
    pub data: [u8; 65536],
}

impl BootStack {
    pub const fn new() -> Self {
        Self { data: [0; 65536] }
    }

    pub fn top(&self) -> u64 {
        (self.data.as_ptr() as u64) + (self.data.len() as u64)
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/boot/stack.rs (line 11) - [public] top

```
pub fn top(&self) -> u64 {
    (self.data.as_ptr() as u64) + (self.data.len() as u64)
}
}
```

kernel_Workspace/base/kw-base/src/core/error.rs (line 5) - [public] KernelError

```
pub enum KernelError {
    Ok = 0,
    NoMemory = -1,
    InvalidArg = -2,
    NotFound = -3,
    AlreadyExists = -4,
    PermissionDenied = -5,
    Busy = -6,
    Timeout = -7,
    NotSupported = -8,
    IoError = -9,
    BadAddress = -10,
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/error.rs (line 65) - [internal] from

```
fn from(s: kstd_base::Status) -> Self {
    match s {
        kstd_base::Status::Success => Self::Ok,
        kstd_base::Status::OutOfMemory => Self::NoMemory,
        kstd_base::Status::InvalidArgument => Self::InvalidArg,
        kstd_base::Status::NotFound => Self::NotFound,
        kstd_base::Status::AlreadyExists => Self::AlreadyExists,
        kstd_base::Status::PermissionDenied => Self::PermissionDenied,
        kstd_base::Status::Busy => Self::Busy,
        kstd_base::Status::TimedOut => Self::Timeout,
        kstd_base::Status::NotSupported => Self::NotSupported,
        kstd_base::Status::IoError => Self::IoError,
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/error.rs (line 84) - [internal] fmt

```
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    write!(f, "{:?}", self)
}
}
```

kernel_Workspace/base/kw-base/src/core/result.rs (line 5) - [public] IntoKResult

```
pub trait IntoKResult<T> {
    fn into_kresult(self) -> KResult<T>;
}

impl<T> IntoKResult<T> for Option<T> {
    fn into_kresult(self) -> KResult<T> {
        self.ok_or(KernelError::NotFound)
    }
}

impl<T> IntoKResult<T> for Result<T, kstd_base::Status> {
    fn into_kresult(self) -> KResult<T> {
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/base/kw-base/src/core/result.rs (line 6) - [internal] into_kresult
    fn into_kresult(self) -> KResult<T>;
}

impl<T> IntoKResult<T> for Option<T> {
    fn into_kresult(self) -> KResult<T> {
        self.ok_or(KernelError::NotFound)
    }
}

impl<T> IntoKResult<T> for Result<T, kstd_base::Status> {
    fn into_kresult(self) -> KResult<T> {
        self.map_err(KernelError::from)
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/base/kw-base/src/core/result.rs (line 10) - [internal] into_kresult
    fn into_kresult(self) -> KResult<T> {
        self.ok_or(KernelError::NotFound)
    }
}

impl<T> IntoKResult<T> for Result<T, kstd_base::Status> {
    fn into_kresult(self) -> KResult<T> {
        self.map_err(KernelError::from)
    }
}

kernel_Workspace/base/kw-base/src/core/result.rs (line 16) - [internal] into_kresult
    fn into_kresult(self) -> KResult<T> {
        self.map_err(KernelError::from)
    }
}

kernel_Workspace/base/kw-base/src/core/traits.rs (line 4) - [public] BlockDevice
pub trait BlockDevice {
    fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(), KernelError>;
    fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), KernelError>;
    fn block_size(&self) -> usize;
    fn block_count(&self) -> u64;
}

pub trait CharDevice {
    fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

// ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/core/traits.rs (line 5) - [internal] read_block
    fn read_block(&self, block: u64, buf: &mut [u8]) -> Result<(), KernelError>;
    fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), KernelError>;
    fn block_size(&self) -> usize;
    fn block_count(&self) -> u64;
}

pub trait CharDevice {
    fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

pub trait NetworkDevice {
    // ... (truncated, see source file for more)
}

```


kernel_workspace/base/kw-base/src/core/traits.rs (line 6) - [internal] write_block

```
    fn write_block(&self, block: u64, buf: &[u8]) -> Result<(), KernelError>;
    fn block_size(&self) -> usize;
    fn block_count(&self) -> u64;
}

pub trait CharDevice {
    fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

pub trait NetworkDevice {
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 7) - [internal] block_size

```
    fn block_size(&self) -> usize;
    fn block_count(&self) -> u64;
}

pub trait CharDevice {
    fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

pub trait NetworkDevice {
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 8) - [internal] block_count

```
    fn block_count(&self) -> u64;
}

pub trait CharDevice {
    fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

pub trait NetworkDevice {
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn mac_address(&self) -> [u8; 6];
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 11) - [public] CharDevice

```
pub trait CharDevice {
    fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

pub trait NetworkDevice {
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn mac_address(&self) -> [u8; 6];
}

pub trait FileSystem {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 12) - [internal] read

```
fn read(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

pub trait NetworkDevice {
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn mac_address(&self) -> [u8; 6];
}

pub trait FileSystem {
    fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 13) - [internal] write

```
fn write(&self, buf: &[u8]) -> Result<usize, KernelError>;
}

pub trait NetworkDevice {
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn mac_address(&self) -> [u8; 6];
}

pub trait FileSystem {
    fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
    fn close(&self, handle: Handle) -> Result<(), KernelError>;
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 16) - [public] NetworkDevice

```
pub trait NetworkDevice {
    fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
    fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
    fn mac_address(&self) -> [u8; 6];
}

pub trait FileSystem {
    fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
    fn close(&self, handle: Handle) -> Result<(), KernelError>;
    fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
    fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
    fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 17) - [internal] send

```
fn send(&self, frame: &[u8]) -> Result<(), KernelError>;
fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
fn mac_address(&self) -> [u8; 6];
}

pub trait FileSystem {
    fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
    fn close(&self, handle: Handle) -> Result<(), KernelError>;
    fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
    fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
    fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 18) - [internal] recv

```
fn recv(&self, buf: &mut [u8]) -> Result<usize, KernelError>;
fn mac_address(&self) -> [u8; 6];
}

pub trait FileSystem {
    fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
    fn close(&self, handle: Handle) -> Result<(), KernelError>;
    fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
    fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
    fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 19) - [internal] mac_address

```
fn mac_address(&self) -> [u8; 6];
}

pub trait FileSystem {
    fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
    fn close(&self, handle: Handle) -> Result<(), KernelError>;
    fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
    fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
    fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

#[derive(Debug, Clone, Copy)]
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 22) - [public] FileSystem

```
pub trait FileSystem {
    fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
    fn close(&self, handle: Handle) -> Result<(), KernelError>;
    fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
    fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
    fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

#[derive(Debug, Clone, Copy)]
pub struct FileStat {
    pub size: u64,
    pub mode: u32,
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 23) - [internal] open

```
fn open(&self, path: &str, flags: u32) -> Result<Handle, KernelError>;
fn close(&self, handle: Handle) -> Result<(), KernelError>;
fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

#[derive(Debug, Clone, Copy)]
pub struct FileStat {
    pub size: u64,
    pub mode: u32,
    pub uid: u32,
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 24) - [internal] close

```
fn close(&self, handle: Handle) -> Result<(), KernelError>;
fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

#[derive(Debug, Clone, Copy)]
pub struct FileStat {
    pub size: u64,
    pub mode: u32,
    pub uid: u32,
    pub gid: u32,
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 25) - [internal] read

```
fn read(&self, handle: Handle, buf: &mut [u8], offset: u64) -> Result<usize, KernelError>;
fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

#[derive(Debug, Clone, Copy)]
pub struct FileStat {
    pub size: u64,
    pub mode: u32,
    pub uid: u32,
    pub gid: u32,
    pub atime: u64,
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 26) - [internal] write

```
fn write(&self, handle: Handle, buf: &[u8], offset: u64) -> Result<usize, KernelError>;
fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

#[derive(Debug, Clone, Copy)]
pub struct FileStat {
    pub size: u64,
    pub mode: u32,
    pub uid: u32,
    pub gid: u32,
    pub atime: u64,
    pub mtime: u64,
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 27) - [internal] stat

```
fn stat(&self, path: &str) -> Result<FileStat, KernelError>;
}

#[derive(Debug, Clone, Copy)]
pub struct FileStat {
    pub size: u64,
    pub mode: u32,
    pub uid: u32,
    pub gid: u32,
    pub atime: u64,
    pub mtime: u64,
    pub ctime: u64,
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 31) - [public] FileStat

```
pub struct FileStat {
    pub size: u64,
    pub mode: u32,
    pub uid: u32,
    pub gid: u32,
    pub atime: u64,
    pub mtime: u64,
    pub ctime: u64,
    pub is_dir: bool,
}

pub trait Scheduler {
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 42) - [public] Scheduler

```
pub trait Scheduler {
    fn schedule(&mut self) -> Option<TaskId>;
    fn enqueue(&mut self, task: TaskId);
    fn dequeue(&mut self, task: TaskId);
    fn yield_current(&mut self);
}

pub trait InterruptController {
    fn enable_irq(&self, irq: u32);
    fn disable_irq(&self, irq: u32);
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 43) - [internal] schedule

```
    fn schedule(&mut self) -> Option<TaskId>;
    fn enqueue(&mut self, task: TaskId);
    fn dequeue(&mut self, task: TaskId);
    fn yield_current(&mut self);
}

pub trait InterruptController {
    fn enable_irq(&self, irq: u32);
    fn disable_irq(&self, irq: u32);
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
}
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 44) - [internal] enqueue

```
    fn enqueue(&mut self, task: TaskId);
    fn dequeue(&mut self, task: TaskId);
    fn yield_current(&mut self);
}

pub trait InterruptController {
    fn enable_irq(&self, irq: u32);
    fn disable_irq(&self, irq: u32);
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
}
```

kernel_Workspace/base/kw-base/src/core/traits.rs (line 45) - [internal] dequeue

```
    fn dequeue(&mut self, task: TaskId);
    fn yield_current(&mut self);
}

pub trait InterruptController {
    fn enable_irq(&self, irq: u32);
    fn disable_irq(&self, irq: u32);
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 46) - [internal] yield_current

```
    fn yield_current(&mut self);
}

pub trait InterruptController {
    fn enable_irq(&self, irq: u32);
    fn disable_irq(&self, irq: u32);
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 49) - [public] InterruptController

```
pub trait InterruptController {
    fn enable_irq(&self, irq: u32);
    fn disable_irq(&self, irq: u32);
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 50) - [internal] enable_irq

```
    fn enable_irq(&self, irq: u32);
    fn disable_irq(&self, irq: u32);
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 51) - [internal] disable_irq

```
    fn disable_irq(&self, irq: u32);
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 52) - [internal] ack_irq

```
    fn ack_irq(&self, irq: u32);
    fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/traits.rs (line 53) - [internal] end_of_interrupt

```
    fn end_of_interrupt(&self);
}
```

kernel_workspace/base/kw-base/src/core/types.rs (line 5) - [public] Pid

```
pub struct Pid(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct Tid(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct Fd(pub i32);

impl Fd {
    pub const INVALID: Self = Self(-1);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/core/types.rs (line 9) - [public] Tid

```
pub struct Tid(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct Fd(pub i32);

impl Fd {
    pub const INVALID: Self = Self(-1);
    pub const STDIN: Self = Self(0);
    pub const STDOUT: Self = Self(1);
    pub const STDERR: Self = Self(2);

    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/core/types.rs (line 13) - [public] Fd

```
pub struct Fd(pub i32);

impl Fd {
    pub const INVALID: Self = Self(-1);
    pub const STDIN: Self = Self(0);
    pub const STDOUT: Self = Self(1);
    pub const STDERR: Self = Self(2);

    #[inline]
    pub const fn is_valid(self) -> bool {
        self.0 >= 0
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/types.rs (line 29) - [public] PrivilegeLevel

```
pub enum PrivilegeLevel {
    Kernel = 0,
    Driver = 1,
    System = 2,
    User = 3,
}

#[derive(Debug, Clone, Copy, PartialEq, Eq)]
#[repr(u8)]
pub enum Arch {
    X86_64 = 0,
    AArch64 = 1,
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/core/types.rs (line 38) - [public] Arch

```
pub enum Arch {
    X86_64 = 0,
    AArch64 = 1,
    Riscv64 = 2,
}
```

kernel_workspace/base/kw-base/src/crypto/ciphers.rs (line 4) - [internal] k_aes256_enc

```
fn k_aes256_enc(key: *const u8, iv: *const u8, data: *mut u8, len: usize) -> bool;
fn k_aes256_dec(key: *const u8, iv: *const u8, data: *mut u8, len: usize) -> bool;
}

pub trait BlockCipher {
    const KEY_LEN: usize;
    const IV_LEN: usize;
    const BLOCK_LEN: usize;

    fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
    fn decrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
}
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/crypto/ciphers.rs (line 5) - [internal] k_aes256_dec

```
fn k_aes256_dec(key: *const u8, iv: *const u8, data: *mut u8, len: usize) -> bool;
}

pub trait BlockCipher {
    const KEY_LEN: usize;
    const IV_LEN: usize;
    const BLOCK_LEN: usize;

    fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
    fn decrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 8) - [public] BlockCipher

```
pub trait BlockCipher {
    const KEY_LEN: usize;
    const IV_LEN: usize;
    const BLOCK_LEN: usize;

    fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
    fn decrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
}

pub struct Aes256Cbc;
impl BlockCipher for Aes256Cbc {
    const KEY_LEN: usize = 32;
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 13) - [internal] encrypt

```
    fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
    fn decrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
}

pub struct Aes256Cbc;
impl BlockCipher for Aes256Cbc {
    const KEY_LEN: usize = 32;
    const IV_LEN: usize = 16;
    const BLOCK_LEN: usize = 16;

    fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<> {
        if key.len() != Self::KEY_LEN || iv.len() != Self::IV_LEN {
            // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 14) - [internal] decrypt

```
    fn decrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<>;
}

pub struct Aes256Cbc;
impl BlockCipher for Aes256Cbc {
    const KEY_LEN: usize = 32;
    const IV_LEN: usize = 16;
    const BLOCK_LEN: usize = 16;

    fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<> {
        if key.len() != Self::KEY_LEN || iv.len() != Self::IV_LEN {
            return Err(KernelError::InvalidArg);
        }
        // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 17) - [public] Aes256Cbc

```
pub struct Aes256Cbc;
impl BlockCipher for Aes256Cbc {
    const KEY_LEN: usize = 32;
    const IV_LEN: usize = 16;
    const BLOCK_LEN: usize = 16;

    fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<> {
        if key.len() != Self::KEY_LEN || iv.len() != Self::IV_LEN {
            return Err(KernelError::InvalidArg);
        }
        if data.len() % Self::BLOCK_LEN != 0 {
            return Err(KernelError::InvalidArg);
        }
        // ... (truncated, see source file for more)
```


kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 23) - [internal] encrypt

```
fn encrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<()> {
    if key.len() != Self::KEY_LEN || iv.len() != Self::IV_LEN {
        return Err(KernelError::InvalidArg);
    }
    if data.len() % Self::BLOCK_LEN != 0 {
        return Err(KernelError::InvalidArg);
    }
    if unsafe { k_aes256_enc(key.as_ptr(), iv.as_ptr(), data.as_mut_ptr(), data.len()) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/ciphers.rs (line 37) - [internal] decrypt

```
fn decrypt(key: &[u8], iv: &[u8], data: &mut [u8]) -> KResult<()> {
    if key.len() != Self::KEY_LEN || iv.len() != Self::IV_LEN {
        return Err(KernelError::InvalidArg);
    }
    if data.len() % Self::BLOCK_LEN != 0 {
        return Err(KernelError::InvalidArg);
    }
    if unsafe { k_aes256_dec(key.as_ptr(), iv.as_ptr(), data.as_mut_ptr(), data.len()) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/hashes.rs (line 4) - [internal] k_sha256

```
fn k_sha256(data: *const u8, len: usize, out: *mut u8) -> bool;
fn k_sha512(data: *const u8, len: usize, out: *mut u8) -> bool;
}

pub trait Hasher {
    const OUTPUT_LEN: usize;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<()>;
}

pub struct Sha256;
impl Hasher for Sha256 {
    const OUTPUT_LEN: usize = 32;
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/crypto/hashes.rs (line 5) - [internal] k_sha512

```
fn k_sha512(data: *const u8, len: usize, out: *mut u8) -> bool;
}

pub trait Hasher {
    const OUTPUT_LEN: usize;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<()>;
}

pub struct Sha256;
impl Hasher for Sha256 {
    const OUTPUT_LEN: usize = 32;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<()> {
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/crypto/hashes.rs (line 8) - [public] Hasher

```
pub trait Hasher {
    const OUTPUT_LEN: usize;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<>;
}

pub struct Sha256;
impl Hasher for Sha256 {
    const OUTPUT_LEN: usize = 32;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<> {
        if out.len() < Self::OUTPUT_LEN { return Err(KernelError::InvalidArg); }
        if unsafe { k_sha256(data.as_ptr(), data.len(), out.as_mut_ptr()) } {
            Ok(())
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/crypto/hashes.rs (line 10) - [internal] hash

```
fn hash(data: &[u8], out: &mut [u8]) -> KResult<>;
}

pub struct Sha256;
impl Hasher for Sha256 {
    const OUTPUT_LEN: usize = 32;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<> {
        if out.len() < Self::OUTPUT_LEN { return Err(KernelError::InvalidArg); }
        if unsafe { k_sha256(data.as_ptr(), data.len(), out.as_mut_ptr()) } {
            Ok(())
        } else {
            Err(KernelError::IoError)
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/crypto/hashes.rs (line 13) - [public] Sha256

```
pub struct Sha256;
impl Hasher for Sha256 {
    const OUTPUT_LEN: usize = 32;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<> {
        if out.len() < Self::OUTPUT_LEN { return Err(KernelError::InvalidArg); }
        if unsafe { k_sha256(data.as_ptr(), data.len(), out.as_mut_ptr()) } {
            Ok(())
        } else {
            Err(KernelError::IoError)
        }
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/crypto/hashes.rs (line 16) - [internal] hash

```
fn hash(data: &[u8], out: &mut [u8]) -> KResult<> {
    if out.len() < Self::OUTPUT_LEN { return Err(KernelError::InvalidArg); }
    if unsafe { k_sha256(data.as_ptr(), data.len(), out.as_mut_ptr()) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}

pub struct Sha512;
impl Hasher for Sha512 {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/crypto/hashes.rs (line 26) - [public] Sha512

```
pub struct Sha512;
impl Hasher for Sha512 {
    const OUTPUT_LEN: usize = 64;
    fn hash(data: &[u8], out: &mut [u8]) -> KResult<> {
        if out.len() < Self::OUTPUT_LEN { return Err(KernelError::InvalidArg); }
        if unsafe { k_sha512(data.as_ptr(), data.len(), out.as_mut_ptr()) } {
            Ok(())
        } else {
            Err(KernelError::IoError)
        }
    }
}
```

kernel_workspace/base/kw-base/src/crypto/hashes.rs (line 29) - [internal] hash

```
fn hash(data: &[u8], out: &mut [u8]) -> KResult<> {
    if out.len() < Self::OUTPUT_LEN { return Err(KernelError::InvalidArg); }
    if unsafe { k_sha512(data.as_ptr(), data.len(), out.as_mut_ptr()) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}
```

kernel_workspace/base/kw-base/src/crypto/keys.rs (line 4) - [public] SymmetricKey

```
pub struct SymmetricKey {
    inner: SecureBuffer,
}

impl SymmetricKey {
    pub fn generate(len: usize) -> KResult<Self> {
        let mut buf = SecureBuffer::new(len).ok_or(KernelError::NoMemory)?;
        super::random::fill_random(buf.as_mut_slice());
        Ok(Self { inner: buf })
    }

    pub fn from_bytes(bytes: &[u8]) -> KResult<Self> {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/crypto/keys.rs (line 9) - [public] generate

```
pub fn generate(len: usize) -> KResult<Self> {
    let mut buf = SecureBuffer::new(len).ok_or(KernelError::NoMemory)?;
    super::random::fill_random(buf.as_mut_slice());
    Ok(Self { inner: buf })
}

pub fn from_bytes(bytes: &[u8]) -> KResult<Self> {
    let mut buf = SecureBuffer::new(bytes.len()).ok_or(KernelError::NoMemory)?;
    buf.as_mut_slice().copy_from_slice(bytes);
    Ok(Self { inner: buf })
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/crypto/keys.rs (line 15) - [public] from_bytes

```
pub fn from_bytes(bytes: &[u8]) -> KResult<Self> {
    let mut buf = SecureBuffer::new(bytes.len()).ok_or(KernelError::NoMemory)?;
    buf.as_mut_slice().copy_from_slice(bytes);
    Ok(Self { inner: buf })
}

pub fn as_slice(&self) -> &[u8] {
    self.inner.as_slice()
}

}
```

kernel_workspace/base/kw-base/src/crypto/keys.rs (line 21) - [public] as_slice

```
pub fn as_slice(&self) -> &[u8] {  
    self.inner.as_slice()  
}
```

kernel_workspace/base/kw-base/src/crypto/random.rs (line 4) - [internal] k_get_random

```
fn k_get_random(buf: *mut u8, len: usize) -> bool;  
fn k_get_hw_random(buf: *mut u8, len: usize) -> bool;  
  
pub fn fill_random(buf: &mut [u8]) -> KResult<()> {  
    if unsafe { k_get_random(buf.as_mut_ptr(), buf.len()) } {  
        Ok(())  
    } else {  
        Err(KernelError::IoError)  
    }  
}  
  
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/crypto/random.rs (line 5) - [internal] k_get_hw_random

```
fn k_get_hw_random(buf: *mut u8, len: usize) -> bool;  
  
pub fn fill_random(buf: &mut [u8]) -> KResult<()> {  
    if unsafe { k_get_random(buf.as_mut_ptr(), buf.len()) } {  
        Ok(())  
    } else {  
        Err(KernelError::IoError)  
    }  
}  
  
pub fn fill_hw_random(buf: &mut [u8]) -> KResult<()> {  
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/crypto/random.rs (line 8) - [public] fill_random

```
pub fn fill_random(buf: &mut [u8]) -> KResult<()> {  
    if unsafe { k_get_random(buf.as_mut_ptr(), buf.len()) } {  
        Ok(())  
    } else {  
        Err(KernelError::IoError)  
    }  
}  
  
pub fn fill_hw_random(buf: &mut [u8]) -> KResult<()> {  
    if unsafe { k_get_hw_random(buf.as_mut_ptr(), buf.len()) } {  
        Ok(())  
    } else {  
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/crypto/random.rs (line 16) - [public] fill_hw_random

```
pub fn fill_hw_random(buf: &mut [u8]) -> KResult<()> {  
    if unsafe { k_get_hw_random(buf.as_mut_ptr(), buf.len()) } {  
        Ok(())  
    } else {  
        Err(KernelError::NotSupported)  
    }  
}  
  
pub fn next_u32() -> KResult<u32> {  
    let mut val = 0u32;  
    fill_random(unsafe { core::slice::from_raw_parts_mut(&mut val as *mut u32 as *mut u8, 4) })?;  
    Ok(val)  
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/crypto/random.rs (line 24) - [public] next_u32

```
pub fn next_u32() -> KResult<u32> {
    let mut val = 0u32;
    fill_random(unsafe { core::slice::from_raw_parts_mut(&mut val as *mut u32 as *mut u8, 4) })?;
    Ok(val)
}
```

kernel_workspace/base/kw-base/src/crypto/secure_mem.rs (line 5) - [public] secure_zero

*// Securely zero out memory. Uses volatile writes to prevent the compiler
// from optimizing the memset away (crucial for wiping keys).*

```
pub fn secure_zero(ptr: *mut u8, len: usize) {
    unsafe {
        for i in 0..len {
            ptr::write_volatile(ptr.add(i), 0);
        }
        core::sync::atomic::compiler_fence(core::sync::atomic::Ordering::SeqCst);
    }
}

pub struct SecureBuffer {
    ptr: *mut u8,
    len: usize,
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/crypto/secure_mem.rs (line 14) - [public] SecureBuffer

```
pub struct SecureBuffer {
    ptr: *mut u8,
    len: usize,
}

impl SecureBuffer {
    pub fn new(len: usize) -> Option<Self> {
        let layout = core::alloc::Layout::from_size_align(len, 8).ok()?;
        let ptr = unsafe { kstd_alloc::KernelHeap.alloc(layout) }.ok()?;
        secure_zero(ptr, len);
        Some(Self { ptr, len })
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/crypto/secure_mem.rs (line 20) - [public] new

```
pub fn new(len: usize) -> Option<Self> {
    let layout = core::alloc::Layout::from_size_align(len, 8).ok()?;
    let ptr = unsafe { kstd_alloc::KernelHeap.alloc(layout) }.ok()?;
    secure_zero(ptr, len);
    Some(Self { ptr, len })
}

pub fn as_slice(&self) -> &[u8] {
    unsafe { core::slice::from_raw_parts(self.ptr, self.len) }
}

pub fn as_mut_slice(&mut self) -> &mut [u8] {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/crypto/secure_mem.rs (line 27) - [public] as_slice

```
pub fn as_slice(&self) -> &[u8] {
    unsafe { core::slice::from_raw_parts(self.ptr, self.len) }
}

pub fn as_mut_slice(&mut self) -> &mut [u8] {
    unsafe { core::slice::from_raw_parts_mut(self.ptr, self.len) }
}

impl Drop for SecureBuffer {
    fn drop(&mut self) {
        secure_zero(self.ptr, self.len);
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/crypto/secure_mem.rs (line 31) - [public] as_mut_slice

```
pub fn as_mut_slice(&mut self) -> &mut [u8] {
    unsafe { core::slice::from_raw_parts_mut(self.ptr, self.len) }
}

impl Drop for SecureBuffer {
    fn drop(&mut self) {
        secure_zero(self.ptr, self.len);
        let layout = core::alloc::Layout::from_size_align(self.len, 8).unwrap();
        unsafe { kstd_alloc::KernelHeap.dealloc(self.ptr, layout); }
    }
}
```

kernel_workspace/base/kw-base/src/crypto/secure_mem.rs (line 37) - [internal] drop

```
fn drop(&mut self) {
    secure_zero(self.ptr, self.len);
    let layout = core::alloc::Layout::from_size_align(self.len, 8).unwrap();
    unsafe { kstd_alloc::KernelHeap.dealloc(self.ptr, layout); }
}
```

kernel_workspace/base/kw-base/src/drivers/ps2.rs (line 7) - [public] Ps2Controller

```
pub struct Ps2Controller;

impl Ps2Controller {
    pub fn init() {
        // Disable devices
        unsafe {
            outb(PS2_CMD, 0xAD);
            outb(PS2_CMD, 0xA7);

            // Flush output buffer
            while (inb(PS2_STATUS) & 0x01) != 0 { inb(PS2_DATA); }

            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/base/kw-base/src/drivers/ps2.rs (line 10) - [public] init

```
pub fn init() {
    // Disable devices
    unsafe {
        outb(PS2_CMD, 0xAD);
        outb(PS2_CMD, 0xA7);

        // Flush output buffer
        while (inb(PS2_STATUS) & 0x01) != 0 { inb(PS2_DATA); }

        // Enable devices
        outb(PS2_CMD, 0xAE);
        outb(PS2_CMD, 0xA8);
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/drivers/ps2.rs (line 25) - [public] read_scancode

```
pub fn read_scancode() -> Option<u8> {
    unsafe {
        if (inb(PS2_STATUS) & 0x01) != 0 {
            Some(inb(PS2_DATA))
        } else {
            None
        }
    }
}
```

kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 4) - [public] Uart16550

```
pub struct Uart16550 {
    port: u16,
}

impl Uart16550 {
    pub const COM1: u16 = 0x3F8;

    pub fn new(port: u16) -> Self {
        let mut uart = Self { port };
        uart.init();
        uart
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 11) - [public] new

```
pub fn new(port: u16) -> Self {
    let mut uart = Self { port };
    uart.init();
    uart
}

fn init(&mut self) {
    unsafe {
        outb(self.port + 1, 0x00); // Disable interrupts
        outb(self.port + 3, 0x80); // Enable DLAB
        outb(self.port + 0, 0x03); // 38400 baud
        outb(self.port + 1, 0x00);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 17) - [internal] init

```
fn init(&mut self) {
    unsafe {
        outb(self.port + 1, 0x00); // Disable interrupts
        outb(self.port + 3, 0x80); // Enable DLAB
        outb(self.port + 0, 0x03); // 38400 baud
        outb(self.port + 1, 0x00);
        outb(self.port + 3, 0x03); // 8 bits, no parity, one stop bit
        outb(self.port + 2, 0xC7); // Enable FIFO
        outb(self.port + 4, 0x0B); // IRQs enabled, RTS/DSR set
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 29) - [internal] is_transmitter_empty

```
fn is_transmitter_empty(&self) -> bool {
    unsafe { (inb(self.port + 5) & 0x20) != 0 }
}

pub fn write_byte(&mut self, b: u8) {
    while !self.is_transmitter_empty() { core::hint::spin_loop(); }
    unsafe { outb(self.port, b); }
}

impl Write for Uart16550 {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 33) - [public] write_byte

```
pub fn write_byte(&mut self, b: u8) {
    while !self.is_transmitter_empty() { core::hint::spin_loop(); }
    unsafe { outb(self.port, b); }
}

impl Write for Uart16550 {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        for b in s.bytes() {
            if b == b'\n' { self.write_byte(b'\r'); }
            self.write_byte(b);
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/base/kw-base/src/drivers/uart.rs (line 40) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> Result<(), ()> {
    for b in s.bytes() {
        if b == b'\n' { self.write_byte(b'\r'); }
        self.write_byte(b);
    }
    Ok(())
}
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 5) - [public] FileType

```
pub enum FileType { Regular, Directory, CharDevice, BlockDevice, Pipe, Socket }

pub trait Inode {
    fn stat(&self) -> KResult<FileStat>;
    fn read(&self, offset: u64, buf: &mut [u8]) -> KResult<usize>;
    fn write(&self, offset: u64, buf: &[u8]) -> KResult<usize>;
    fn lookup(&self, name: &str) -> KResult<Handle>;
}

pub trait File {
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 7) - [public] Inode

```
pub trait Inode {
    fn stat(&self) -> KResult<FileStat>;
    fn read(&self, offset: u64, buf: &mut [u8]) -> KResult<usize>;
    fn write(&self, offset: u64, buf: &[u8]) -> KResult<usize>;
    fn lookup(&self, name: &str) -> KResult<Handle>;
}

pub trait File {
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 8) - [internal] stat

```
fn stat(&self) -> KResult<FileStat>;
fn read(&self, offset: u64, buf: &mut [u8]) -> KResult<usize>;
fn write(&self, offset: u64, buf: &[u8]) -> KResult<usize>;
fn lookup(&self, name: &str) -> KResult<Handle>;
}

pub trait File {
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
}

pub trait FileSystem {
    // ... (truncated, see source file for more)
}
```


kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 9) - [internal] read

```
    fn read(&self, offset: u64, buf: &mut [u8]) -> KResult<usize>;
    fn write(&self, offset: u64, buf: &[u8]) -> KResult<usize>;
    fn lookup(&self, name: &str) -> KResult<Handle>;
}

pub trait File {
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
}

pub trait FileSystem {
    fn name(&self) -> &str;
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 10) - [internal] write

```
    fn write(&self, offset: u64, buf: &[u8]) -> KResult<usize>;
    fn lookup(&self, name: &str) -> KResult<Handle>;
}

pub trait File {
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
}

pub trait FileSystem {
    fn name(&self) -> &str;
    fn root_inode(&self) -> Handle;
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 11) - [internal] lookup

```
    fn lookup(&self, name: &str) -> KResult<Handle>;
}

pub trait File {
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
}

pub trait FileSystem {
    fn name(&self) -> &str;
    fn root_inode(&self) -> Handle;
    fn sync(&self) -> KResult<()>;
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 14) - [public] File

```
pub trait File {
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
}

pub trait FileSystem {
    fn name(&self) -> &str;
    fn root_inode(&self) -> Handle;
    fn sync(&self) -> KResult<()>;
}

pub struct Vfs;
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 15) - [internal] inode

```
    fn inode(&self) -> &dyn Inode;
    fn flags(&self) -> u32;
}

pub trait FileSystem {
    fn name(&self) -> &str;
    fn root_inode(&self) -> Handle;
    fn sync(&self) -> KResult<>;
}

pub struct Vfs;

    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 16) - [internal] flags

```
    fn flags(&self) -> u32;
}

pub trait FileSystem {
    fn name(&self) -> &str;
    fn root_inode(&self) -> Handle;
    fn sync(&self) -> KResult<>;
}

pub struct Vfs;

impl Vfs {
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 19) - [public] FileSystem

```
pub trait FileSystem {
    fn name(&self) -> &str;
    fn root_inode(&self) -> Handle;
    fn sync(&self) -> KResult<>;
}

pub struct Vfs;

impl Vfs {
    pub fn open(path: &str, flags: u32) -> KResult<Handle> {
        // Route to legacy kernel VFS via FFI if needed, or internal registry
        extern "C" { fn k_fs_open(path: *const u8, flags: u32) -> i32; }
        // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 20) - [internal] name

```
    fn name(&self) -> &str;
    fn root_inode(&self) -> Handle;
    fn sync(&self) -> KResult<>;
}

pub struct Vfs;

impl Vfs {
    pub fn open(path: &str, flags: u32) -> KResult<Handle> {
        // Route to legacy kernel VFS via FFI if needed, or internal registry
        extern "C" { fn k_fs_open(path: *const u8, flags: u32) -> i32; }
        let rc = unsafe { k_fs_open(path.as_ptr(), flags) };
        // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 21) - [internal] root_inode

```
fn root_inode(&self) -> Handle;
fn sync(&self) -> KResult<()>;
}

pub struct Vfs;

impl Vfs {
    pub fn open(path: &str, flags: u32) -> KResult<Handle> {
        // Route to legacy kernel VFS via FFI if needed, or internal registry
        extern "C" { fn k_fs_open(path: *const u8, flags: u32) -> i32; }
        let rc = unsafe { k_fs_open(path.as_ptr(), flags) };
        if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(Handle(rc as u32)) }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 22) - [internal] sync

```
fn sync(&self) -> KResult<()>;
}

pub struct Vfs;

impl Vfs {
    pub fn open(path: &str, flags: u32) -> KResult<Handle> {
        // Route to legacy kernel VFS via FFI if needed, or internal registry
        extern "C" { fn k_fs_open(path: *const u8, flags: u32) -> i32; }
        let rc = unsafe { k_fs_open(path.as_ptr(), flags) };
        if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(Handle(rc as u32)) }
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 25) - [public] Vfs

```
pub struct Vfs;

impl Vfs {
    pub fn open(path: &str, flags: u32) -> KResult<Handle> {
        // Route to legacy kernel VFS via FFI if needed, or internal registry
        extern "C" { fn k_fs_open(path: *const u8, flags: u32) -> i32; }
        let rc = unsafe { k_fs_open(path.as_ptr(), flags) };
        if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(Handle(rc as u32)) }
    }

    pub fn read(fd: Handle, buf: &mut [u8]) -> KResult<usize> {
        extern "C" { fn k_fs_read(fd: i32, buf: *mut u8, cap: u32) -> i32; }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/base/kw-base/src/fs/vfs.rs (line 28) - [public] open

```
pub fn open(path: &str, flags: u32) -> KResult<Handle> {
    // Route to legacy kernel VFS via FFI if needed, or internal registry
    extern "C" { fn k_fs_open(path: *const u8, flags: u32) -> i32; }
    let rc = unsafe { k_fs_open(path.as_ptr(), flags) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(Handle(rc as u32)) }
}

pub fn read(fd: Handle, buf: &mut [u8]) -> KResult<usize> {
    extern "C" { fn k_fs_read(fd: i32, buf: *mut u8, cap: u32) -> i32; }
    let rc = unsafe { k_fs_read(fd.0 as i32, buf.as_mut_ptr(), buf.len() as u32) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/fs/vfs.rs (line 35) - [public] read

```
pub fn read(fd: Handle, buf: &mut [u8]) -> KResult<usize> {
    extern "C" { fn k_fs_read(fd: i32, buf: *mut u8, cap: u32) -> i32; }
    let rc = unsafe { k_fs_read(fd.0 as i32, buf.as_mut_ptr(), buf.len() as u32) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}

pub fn write(fd: Handle, buf: &[u8]) -> KResult<usize> {
    extern "C" { fn k_fs_write(fd: i32, buf: *const u8, len: u32) -> i32; }
    let rc = unsafe { k_fs_write(fd.0 as i32, buf.as_ptr(), buf.len() as u32) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}
}
```

kernel_workspace/base/kw-base/src/fs/vfs.rs (line 41) - [public] write

```
pub fn write(fd: Handle, buf: &[u8]) -> KResult<usize> {
    extern "C" { fn k_fs_write(fd: i32, buf: *const u8, len: u32) -> i32; }
    let rc = unsafe { k_fs_write(fd.0 as i32, buf.as_ptr(), buf.len() as u32) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as usize) }
}
}
```

kernel_workspace/base/kw-base/src/hal/acpi.rs (line 4) - [internal] k_acpi_get_rsdp

```
fn k_acpi_get_rsdp() -> *const u8;
fn k_acpi_find_table(sig: *const u8, idx: u32) -> *const u8;
}

pub fn get_rsdp() -> Option<*const u8> {
    let ptr = unsafe { k_acpi_get_rsdp() };
    if ptr.is_null() { None } else { Some(ptr) }
}

pub fn find_table(sig: &[u8; 4], index: u32) -> Option<*const u8> {
    let ptr = unsafe { k_acpi_find_table(sig.as_ptr(), index) };
    if ptr.is_null() { None } else { Some(ptr) }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/hal/acpi.rs (line 5) - [internal] k_acpi_find_table

```
fn k_acpi_find_table(sig: *const u8, idx: u32) -> *const u8;
}

pub fn get_rsdp() -> Option<*const u8> {
    let ptr = unsafe { k_acpi_get_rsdp() };
    if ptr.is_null() { None } else { Some(ptr) }
}

pub fn find_table(sig: &[u8; 4], index: u32) -> Option<*const u8> {
    let ptr = unsafe { k_acpi_find_table(sig.as_ptr(), index) };
    if ptr.is_null() { None } else { Some(ptr) }
}
}
```

kernel_workspace/base/kw-base/src/hal/acpi.rs (line 8) - [public] get_rsdp

```
pub fn get_rsdp() -> Option<*const u8> {
    let ptr = unsafe { k_acpi_get_rsdp() };
    if ptr.is_null() { None } else { Some(ptr) }
}

pub fn find_table(sig: &[u8; 4], index: u32) -> Option<*const u8> {
    let ptr = unsafe { k_acpi_find_table(sig.as_ptr(), index) };
    if ptr.is_null() { None } else { Some(ptr) }
}
}
```

kernel_workspace/base/kw-base/src/hal/acpi.rs (line 13) - [public] find_table

```
pub fn find_table(sig: &[u8; 4], index: u32) -> Option<*const u8> {
    let ptr = unsafe { k_acpi_find_table(sig.as_ptr(), index) };
    if ptr.is_null() { None } else { Some(ptr) }
}
}
```

kernel_workspace/base/kw-base/src/hal/dma.rs (line 5) - [public] DmaBuffer

```
pub struct DmaBuffer {
    frames: ContiguousFrames,
    virt: VirtAddr,
}

impl DmaBuffer {
    pub fn allocate(pages: usize) -> KResult<Self> {
        let frames = ContiguousFrames::alloc(pages)?;
        let virt = gluecore::bridge::mm::vmm::alloc(
            frames.byte_size(),
            gluecore::bridge::mm::vmm::VMM_FLAG_READ | gluecore::bridge::mm::vmm::VMM_FLAG_WRITE | gluecore::bridge::mm::vmm::VMM_FLAG_EXEC,
        ).map_err(KernelError::from)?;
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/hal/dma.rs (line 11) - [public] allocate

```
pub fn allocate(pages: usize) -> KResult<Self> {
    let frames = ContiguousFrames::alloc(pages)?;
    let virt = gluecore::bridge::mm::vmm::alloc(
        frames.byte_size(),
        gluecore::bridge::mm::vmm::VMM_FLAG_READ | gluecore::bridge::mm::vmm::VMM_FLAG_WRITE | gluecore::bridge::mm::vmm::VMM_FLAG_EXEC,
    ).map_err(KernelError::from)?;

    Ok(Self { frames, virt })
}

pub fn phys_addr(&self) -> PhysAddr { self.frames.phys_addr() }
pub fn virt_addr(&self) -> VirtAddr { self.virt }
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/hal/dma.rs (line 21) - [public] phys_addr

```
pub fn phys_addr(&self) -> PhysAddr { self.frames.phys_addr() }
pub fn virt_addr(&self) -> VirtAddr { self.virt }
pub fn size(&self) -> usize { self.frames.byte_size() }

pub fn as_slice(&self) -> &[u8] {
    unsafe { core::slice::from_raw_parts(self.virt.0 as *const u8, self.size()) }
}

pub fn as_mut_slice(&mut self) -> &mut [u8] {
    unsafe { core::slice::from_raw_parts_mut(self.virt.0 as *mut u8, self.size()) }
}
}
```

kernel_workspace/base/kw-base/src/hal/dma.rs (line 22) - [public] virt_addr

```
pub fn virt_addr(&self) -> VirtAddr { self.virt }
pub fn size(&self) -> usize { self.frames.byte_size() }

pub fn as_slice(&self) -> &[u8] {
    unsafe { core::slice::from_raw_parts(self.virt.0 as *const u8, self.size()) }
}

pub fn as_mut_slice(&mut self) -> &mut [u8] {
    unsafe { core::slice::from_raw_parts_mut(self.virt.0 as *mut u8, self.size()) }
}
}
```

kernel_workspace/base/kw-base/src/hal/dma.rs (line 23) - [public] size

```
pub fn size(&self) -> usize { self.frames.byte_size() }

pub fn as_slice(&self) -> &[u8] {
    unsafe { core::slice::from_raw_parts(self.virt.0 as *const u8, self.size()) }
}

pub fn as_mut_slice(&mut self) -> &mut [u8] {
    unsafe { core::slice::from_raw_parts_mut(self.virt.0 as *mut u8, self.size()) }
}
}
```

kernel_workspace/base/kw-base/src/hal/dma.rs (line 25) - [public] as_slice

```
pub fn as_slice(&self) -> &[u8] {
    unsafe { core::slice::from_raw_parts(self.virt.0 as *const u8, self.size()) }
}

pub fn as_mut_slice(&mut self) -> &mut [u8] {
    unsafe { core::slice::from_raw_parts_mut(self.virt.0 as *mut u8, self.size()) }
}
}
```

kernel_workspace/base/kw-base/src/hal/dma.rs (line 29) - [public] as_mut_slice

```
pub fn as_mut_slice(&mut self) -> &mut [u8] {
    unsafe { core::slice::from_raw_parts_mut(self.virt.0 as *mut u8, self.size()) }
}
}
```

kernel_workspace/base/kw-base/src/hal/pci.rs (line 7) - [public] PciAddress

```
pub struct PciAddress {
    pub bus: u8,
    pub device: u8,
    pub function: u8,
}

impl PciAddress {
    pub const fn new(bus: u8, device: u8, function: u8) -> Self {
        Self { bus, device, function }
    }

    fn config_addr(self, offset: u8) -> u32 {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/hal/pci.rs (line 18) - [internal] config_addr

```
fn config_addr(self, offset: u8) -> u32 {
    0x8000_0000
    | ((self.bus as u32) << 16)
    | ((self.device as u32 & 0x1F) << 11)
    | ((self.function as u32 & 0x07) << 8)
    | (offset as u32 & 0xFC)
}
}
```

```
#[derive(Debug, Clone, Copy)]
pub struct PciDevice {
    pub addr: PciAddress,
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/hal/pci.rs (line 28) - [public] PciDevice

```
pub struct PciDevice {
    pub addr: PciAddress,
    pub vendor_id: u16,
    pub device_id: u16,
    pub class_code: u8,
    pub subclass: u8,
}

pub fn read_config32(addr: PciAddress, offset: u8) -> u32 {
    unsafe {
        outl(PCI_CONFIG_ADDR, addr.config_addr(offset));
        inl(PCI_CONFIG_DATA)
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/hal/pci.rs (line 36) - [public] read_config32

```
pub fn read_config32(addr: PciAddress, offset: u8) -> u32 {
    unsafe {
        outl(PCI_CONFIG_ADDR, addr.config_addr(offset));
        inl(PCI_CONFIG_DATA)
    }
}

pub fn write_config32(addr: PciAddress, offset: u8, value: u32) {
    unsafe {
        outl(PCI_CONFIG_ADDR, addr.config_addr(offset));
        outl(PCI_CONFIG_DATA, value);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/hal/pci.rs (line 43) - [public] write_config32

```
pub fn write_config32(addr: PciAddress, offset: u8, value: u32) {
    unsafe {
        outl(PCI_CONFIG_ADDR, addr.config_addr(offset));
        outl(PCI_CONFIG_DATA, value);
    }
}

pub fn scan_bus() -> kstd_data::Vec<PciDevice> {
    let mut devices = kstd_data::Vec::new();
    for bus in 0..=255 {
        for dev in 0..32 {
            let addr = PciAddress::new(bus, dev, 0);
            // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/hal/pci.rs (line 50) - [public] scan_bus

```
pub fn scan_bus() -> kstd_data::Vec<PciDevice> {
    let mut devices = kstd_data::Vec::new();
    for bus in 0..=255 {
        for dev in 0..32 {
            let addr = PciAddress::new(bus, dev, 0);
            let id = read_config32(addr, 0);
            let vendor = id as u16;
            if vendor == 0xFFFF { continue; }

            devices.push(PciDevice {
                addr,
                vendor_id: vendor,
            });
            // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/hal/timers.rs (line 4) - [internal] k_timer_get_ticks

```
fn k_timer_get_ticks() -> u64;
fn k_timer_get_frequency() -> u64;
fn k_timer_sleep_ms(ms: u32);
fn k_timer_set_oneshot(ns: u64) -> bool;
}

pub fn get_ticks() -> u64 {
    unsafe { k_timer_get_ticks() }
}

pub fn get_frequency() -> u64 {
    unsafe { k_timer_get_frequency() }
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/base/kw-base/src/hal/timers.rs (line 5) - [internal] k_timer_get_frequency
    fn k_timer_get_frequency() -> u64;
    fn k_timer_sleep_ms(ms: u32);
    fn k_timer_set_oneshot(ns: u64) -> bool;
}

pub fn get_ticks() -> u64 {
    unsafe { k_timer_get_ticks() }
}

pub fn get_frequency() -> u64 {
    unsafe { k_timer_get_frequency() }
}

// ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/hal/timers.rs (line 6) - [internal] k_timer_sleep_ms
    fn k_timer_sleep_ms(ms: u32);
    fn k_timer_set_oneshot(ns: u64) -> bool;
}

pub fn get_ticks() -> u64 {
    unsafe { k_timer_get_ticks() }
}

pub fn get_frequency() -> u64 {
    unsafe { k_timer_get_frequency() }
}

// ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/hal/timers.rs (line 7) - [internal] k_timer_set_oneshot
    fn k_timer_set_oneshot(ns: u64) -> bool;
}

pub fn get_ticks() -> u64 {
    unsafe { k_timer_get_ticks() }
}

pub fn get_frequency() -> u64 {
    unsafe { k_timer_get_frequency() }
}

pub fn sleep_ms(ms: u32) {
    // ... (truncated, see source file for more)
}

kernel_Workspace/base/kw-base/src/hal/timers.rs (line 10) - [public] get_ticks
pub fn get_ticks() -> u64 {
    unsafe { k_timer_get_ticks() }
}

pub fn get_frequency() -> u64 {
    unsafe { k_timer_get_frequency() }
}

pub fn sleep_ms(ms: u32) {
    unsafe { k_timer_sleep_ms(ms); }
}

// ... (truncated, see source file for more)

```


kernel_Workspace/base/kw-base/src/hal/timers.rs (line 14) - [public] get_frequency

```
pub fn get_frequency() -> u64 {
    unsafe { k_timer_get_frequency() }
}

pub fn sleep_ms(ms: u32) {
    unsafe { k_timer_sleep_ms(ms); }
}

pub fn set_oneshot(ns: u64) -> KResult<()> {
    if unsafe { k_timer_set_oneshot(ns) } {
        Ok(())
    } else {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/base/kw-base/src/hal/timers.rs (line 18) - [public] sleep_ms

```
pub fn sleep_ms(ms: u32) {
    unsafe { k_timer_sleep_ms(ms); }
}

pub fn set_oneshot(ns: u64) -> KResult<()> {
    if unsafe { k_timer_set_oneshot(ns) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}
```

kernel_Workspace/base/kw-base/src/hal/timers.rs (line 22) - [public] set_oneshot

```
pub fn set_oneshot(ns: u64) -> KResult<()> {
    if unsafe { k_timer_set_oneshot(ns) } {
        Ok(())
    } else {
        Err(KernelError::IoError)
    }
}
```

kernel_Workspace/base/kw-base/src/hal/usb.rs (line 4) - [internal] k_usb_init

```
fn k_usb_init() -> bool;
fn k_usb_enum_devices() -> u32;
fn k_usb_get_device_count() -> u32;

pub fn init() -> KResult<()> {
    if unsafe { k_usb_init() } {
        Ok(())
    } else {
        Err(KernelError::HardwareFault)
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/hal/usb.rs (line 5) - [internal] k_usb_enum_devices

```
fn k_usb_enum_devices() -> u32;
fn k_usb_get_device_count() -> u32;

pub fn init() -> KResult<()> {
    if unsafe { k_usb_init() } {
        Ok(())
    } else {
        Err(KernelError::HardwareFault)
    }
}

// ... (truncated, see source file for more)
```

```

kernel_Workspace/base/kw-base/src/hal/usb.rs (line 6) - [internal] k_usb_get_device_count
    fn k_usb_get_device_count() -> u32;
}

pub fn init() -> KResult<()> {
    if unsafe { k_usb_init() } {
        Ok(())
    } else {
        Err(KernelError::HardwareFault)
    }
}

pub fn enumerate_devices() -> KResult<u32> {
    // ... (truncated, see source file for more)
}

kernel_Workspace/base/kw-base/src/hal/usb.rs (line 9) - [public] init
pub fn init() -> KResult<()> {
    if unsafe { k_usb_init() } {
        Ok(())
    } else {
        Err(KernelError::HardwareFault)
    }
}

pub fn enumerate_devices() -> KResult<u32> {
    let count = unsafe { k_usb_enum_devices() };
    if count == 0 && unsafe { k_usb_get_device_count() } == 0 {
        Err(KernelError::NotFound)
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/base/kw-base/src/hal/usb.rs (line 17) - [public] enumerate_devices
pub fn enumerate_devices() -> KResult<u32> {
    let count = unsafe { k_usb_enum_devices() };
    if count == 0 && unsafe { k_usb_get_device_count() } == 0 {
        Err(KernelError::NotFound)
    } else {
        Ok(count)
    }
}

kernel_Workspace/base/kw-base/src/ipc/message_queue.rs (line 4) - [internal] k_ipc_send
    fn k_ipc_send(dst: u32, a0: u64, a1: u64) -> i32;
    fn k_ipc_recv(a0: *mut u64, a1: *mut u64) -> i32;
}

#[derive(Debug, Clone, Copy, Default)]
pub struct IpcMessage {
    pub from: u32,
    pub arg0: u64,
    pub arg1: u64,
}

pub fn send(target_pid: u32, arg0: u64, arg1: u64) -> KResult<()> {
    // ... (truncated, see source file for more)
}

kernel_Workspace/base/kw-base/src/ipc/message_queue.rs (line 5) - [internal] k_ipc_recv
    fn k_ipc_recv(a0: *mut u64, a1: *mut u64) -> i32;
}

#[derive(Debug, Clone, Copy, Default)]
pub struct IpcMessage {
    pub from: u32,
    pub arg0: u64,
    pub arg1: u64,
}

pub fn send(target_pid: u32, arg0: u64, arg1: u64) -> KResult<()> {
    let rc = unsafe { k_ipc_send(target_pid, arg0, arg1) };
    // ... (truncated, see source file for more)
}

```

kernel_workspace/base/kw-base/src/ipc/message_queue.rs (line 9) - [public] IpcMessage

```
pub struct IpcMessage {
    pub from: u32,
    pub arg0: u64,
    pub arg1: u64,
}

pub fn send(target_pid: u32, arg0: u64, arg1: u64) -> KResult<()> {
    let rc = unsafe { k_ipc_send(target_pid, arg0, arg1) };
    if rc == 0 { Ok(()) } else { Err(KernelError::from_code(rc)) }
}

pub fn recv() -> KResult<IpcMessage> {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/ipc/message_queue.rs (line 15) - [public] send

```
pub fn send(target_pid: u32, arg0: u64, arg1: u64) -> KResult<()> {
    let rc = unsafe { k_ipc_send(target_pid, arg0, arg1) };
    if rc == 0 { Ok(()) } else { Err(KernelError::from_code(rc)) }
}

pub fn recv() -> KResult<IpcMessage> {
    let mut a0: u64 = 0;
    let mut a1: u64 = 0;
    let from = unsafe { k_ipc_recv(&mut a0, &mut a1) };
    if from < 0 {
        Err(KernelError::from_code(from))
    } else {
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/ipc/message_queue.rs (line 20) - [public] recv

```
pub fn recv() -> KResult<IpcMessage> {
    let mut a0: u64 = 0;
    let mut a1: u64 = 0;
    let from = unsafe { k_ipc_recv(&mut a0, &mut a1) };
    if from < 0 {
        Err(KernelError::from_code(from))
    } else {
        Ok(IpcMessage { from: from as u32, arg0: a0, arg1: a1 })
    }
}
```

kernel_workspace/base/kw-base/src/ipc/shared_mem.rs (line 5) - [public] SharedMemory

```
pub struct SharedMemory {
    frames: ContiguousFrames,
    virt: VirtAddr,
}

impl SharedMemory {
    pub fn allocate(pages: usize) -> KResult<Self> {
        let frames = ContiguousFrames::alloc(pages)?;
        let virt = gluecore::bridge::mm::vmm::alloc(
            frames.byte_size(),
            gluecore::bridge::mm::vmm::VMM_FLAG_READ | gluecore::bridge::mm::vmm::VMM_FLAG_WRITE
        ).map_err(KernelError::from)?;
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/ipc/shared_mem.rs (line 11) - [public] allocate

```
pub fn allocate(pages: usize) -> KResult<Self> {
    let frames = ContiguousFrames::alloc(pages)?;
    let virt = gluecore::bridge::mm::vmm::alloc(
        frames.byte_size(),
        gluecore::bridge::mm::vmm::VMM_FLAG_READ | gluecore::bridge::mm::vmm::VMM_FLAG_WRITE
    ).map_err(KernelError::from)?;

    for i in 0..pages {
        let phys = PhysAddr(frames.phys_addr().0 + (i as u64 * 4096));
        let v = VirtAddr(virt.0 + (i as u64 * 4096));
        paging::map_page(v, phys, paging::PAGE_PRESENT | paging::PAGE_WRITABLE)?;
    }
    // ... (truncated, see source file for more)
```

```

kernel_workspace/base/kw-base/src/ipc/shared_mem.rs (line 27) - [public] virt_addr
    pub fn virt_addr(&self) -> VirtAddr { self.virt }
    pub fn size(&self) -> usize { self.frames.byte_size() }
}

impl Drop for SharedMemory {
    fn drop(&mut self) {
        let _ = gluecore::bridge::mm::vmm::free(self.virt, self.frames.byte_size());
    }
}

kernel_workspace/base/kw-base/src/ipc/shared_mem.rs (line 28) - [public] size
    pub fn size(&self) -> usize { self.frames.byte_size() }
}

impl Drop for SharedMemory {
    fn drop(&mut self) {
        let _ = gluecore::bridge::mm::vmm::free(self.virt, self.frames.byte_size());
    }
}

kernel_workspace/base/kw-base/src/ipc/shared_mem.rs (line 32) - [internal] drop
    fn drop(&mut self) {
        let _ = gluecore::bridge::mm::vmm::free(self.virt, self.frames.byte_size());
    }
}

kernel_workspace/base/kw-base/src/mem/buddy.rs (line 4) - [public] ContiguousFrames
pub struct ContiguousFrames {
    addr: PhysAddr,
    count: usize,
}

impl ContiguousFrames {
    pub fn alloc(count: usize) -> KResult<Self> {
        let addr = pmm::alloc_frames(count).map_err(KernelError::from)?;
        Ok(Self { addr, count })
    }

    pub fn alloc_aligned(count: usize, align_frames: usize) -> KResult<Self> {
        // ... (truncated, see source file for more)
    }
}

kernel_workspace/base/kw-base/src/mem/buddy.rs (line 10) - [public] alloc
    pub fn alloc(count: usize) -> KResult<Self> {
        let addr = pmm::alloc_frames(count).map_err(KernelError::from)?;
        Ok(Self { addr, count })
    }

    pub fn alloc_aligned(count: usize, align_frames: usize) -> KResult<Self> {
        let addr = pmm::alloc_frames_aligned(count, align_frames).map_err(KernelError::from)?;
        Ok(Self { addr, count })
    }

    pub fn phys_addr(&self) -> PhysAddr { self.addr }
    pub fn frame_count(&self) -> usize { self.count }
    // ... (truncated, see source file for more)
}

kernel_workspace/base/kw-base/src/mem/buddy.rs (line 15) - [public] alloc_aligned
    pub fn alloc_aligned(count: usize, align_frames: usize) -> KResult<Self> {
        let addr = pmm::alloc_frames_aligned(count, align_frames).map_err(KernelError::from)?;
        Ok(Self { addr, count })
    }

    pub fn phys_addr(&self) -> PhysAddr { self.addr }
    pub fn frame_count(&self) -> usize { self.count }
    pub fn byte_size(&self) -> usize { self.count * 4096 }

    pub fn into_raw(self) -> (PhysAddr, usize) {
        let addr = self.addr;
        let count = self.count;
        // ... (truncated, see source file for more)
    }
}

```

kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 20) - [public] phys_addr

```
pub fn phys_addr(&self) -> PhysAddr { self.addr }
pub fn frame_count(&self) -> usize { self.count }
pub fn byte_size(&self) -> usize { self.count * 4096 }

pub fn into_raw(self) -> (PhysAddr, usize) {
    let addr = self.addr;
    let count = self.count;
    core::mem::forget(self);
    (addr, count)
}

pub unsafe fn from_raw(addr: PhysAddr, count: usize) -> Self {
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 21) - [public] frame_count

```
pub fn frame_count(&self) -> usize { self.count }
pub fn byte_size(&self) -> usize { self.count * 4096 }

pub fn into_raw(self) -> (PhysAddr, usize) {
    let addr = self.addr;
    let count = self.count;
    core::mem::forget(self);
    (addr, count)
}

pub unsafe fn from_raw(addr: PhysAddr, count: usize) -> Self {
    Self { addr, count }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 22) - [public] byte_size

```
pub fn byte_size(&self) -> usize { self.count * 4096 }

pub fn into_raw(self) -> (PhysAddr, usize) {
    let addr = self.addr;
    let count = self.count;
    core::mem::forget(self);
    (addr, count)
}

pub unsafe fn from_raw(addr: PhysAddr, count: usize) -> Self {
    Self { addr, count }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 24) - [public] into_raw

```
pub fn into_raw(self) -> (PhysAddr, usize) {
    let addr = self.addr;
    let count = self.count;
    core::mem::forget(self);
    (addr, count)
}

pub unsafe fn from_raw(addr: PhysAddr, count: usize) -> Self {
    Self { addr, count }
}
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/buddy.rs (line 37) - [internal] drop

```
fn drop(&mut self) {
    let _ = pmm::free_frames(self.addr, self.count);
}
}
```

kernel_Workspace/base/kw-base/src/mem/frame.rs (line 4) - [public] Frame

```
pub struct Frame {
    addr: PhysAddr,
}

impl Frame {
    pub fn alloc() -> KResult<Self> {
        let addr = pmm::alloc_frame().map_err(KernelError::from)?;
        Ok(Self { addr })
    }

    pub fn alloc_zeroed() -> KResult<Self> {
        let addr = pmm::alloc_zero_frame().map_err(KernelError::from)?;
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/base/kw-base/src/mem/frame.rs (line 9) - [public] alloc

```
pub fn alloc() -> KResult<Self> {
    let addr = pmm::alloc_frame().map_err(KernelError::from)?;
    Ok(Self { addr })
}

pub fn alloc_zeroed() -> KResult<Self> {
    let addr = pmm::alloc_zero_frame().map_err(KernelError::from)?;
    Ok(Self { addr })
}

pub fn phys_addr(&self) -> PhysAddr {
    self.addr
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/mem/frame.rs (line 14) - [public] alloc_zeroed

```
pub fn alloc_zeroed() -> KResult<Self> {
    let addr = pmm::alloc_zero_frame().map_err(KernelError::from)?;
    Ok(Self { addr })
}

pub fn phys_addr(&self) -> PhysAddr {
    self.addr
}

pub fn into_raw(self) -> PhysAddr {
    let addr = self.addr;
    core::mem::forget(self);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/mem/frame.rs (line 19) - [public] phys_addr

```
pub fn phys_addr(&self) -> PhysAddr {
    self.addr
}

pub fn into_raw(self) -> PhysAddr {
    let addr = self.addr;
    core::mem::forget(self);
    addr
}

pub unsafe fn from_raw(addr: PhysAddr) -> Self {
    Self { addr }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/mem/frame.rs (line 23) - [public] into_raw

```
pub fn into_raw(self) -> PhysAddr {
    let addr = self.addr;
    core::mem::forget(self);
    addr
}

pub unsafe fn from_raw(addr: PhysAddr) -> Self {
    Self { addr }
}

impl Drop for Frame {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/mem/frame.rs (line 35) - [internal] drop

```
fn drop(&mut self) {
    let _ = pmm::free_frame(self.addr);
}
```

kernel_workspace/base/kw-base/src/mem/heap.rs (line 4) - [public] alloc

```
pub fn alloc<T>() -> KResult<*mut T> {
    let layout = Layout::new(core::mem::size_of::<T>(), core::mem::align_of::<T>());
    let ptr = KernelHeap.alloc(layout).map_err(KernelError::from)?;
    Ok(ptr as *mut T)
}

pub fn alloc_zeroed<T>() -> KResult<*mut T> {
    let ptr = alloc::<T>()?;
    unsafe { core::ptr::write_bytes(ptr, 0, 1); }
    Ok(ptr)
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/mem/heap.rs (line 10) - [public] alloc_zeroed

```
pub fn alloc_zeroed<T>() -> KResult<*mut T> {
    let ptr = alloc::<T>()?;
    unsafe { core::ptr::write_bytes(ptr, 0, 1); }
    Ok(ptr)
}

pub unsafe fn free<T>(ptr: *mut T) {
    let layout = Layout::new(core::mem::size_of::<T>(), core::mem::align_of::<T>());
    KernelHeap.dealloc(ptr as *mut u8, layout);
}

pub fn alloc_slice<T>(len: usize) -> KResult<*mut T> {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/mem/heap.rs (line 21) - [public] alloc_slice

```
pub fn alloc_slice<T>(len: usize) -> KResult<*mut T> {
    let layout = Layout::new(core::mem::size_of::<T>() * len, core::mem::align_of::<T>());
    let ptr = KernelHeap.alloc(layout).map_err(KernelError::from)?;
    Ok(ptr as *mut T)
}

pub unsafe fn free_slice<T>(ptr: *mut T, len: usize) {
    let layout = Layout::new(core::mem::size_of::<T>() * len, core::mem::align_of::<T>());
    KernelHeap.dealloc(ptr as *mut u8, layout);
}
```

kernel_Workspace/base/kw-base/src/mem/paging.rs (line 14) - [internal] paging_map_page

```
fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
fn paging_unmap_page(virt: u64) -> bool;
}

pub fn map_page(virt: VirtAddr, phys: PhysAddr, flags: u64) -> KResult<()> {
    if unsafe { paging_map_page(virt.0, phys.0, flags) } {
        Ok(())
    } else {
        Err(KernelError::NoMemory)
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/paging.rs (line 15) - [internal] paging_unmap_page

```
fn paging_unmap_page(virt: u64) -> bool;
}

pub fn map_page(virt: VirtAddr, phys: PhysAddr, flags: u64) -> KResult<()> {
    if unsafe { paging_map_page(virt.0, phys.0, flags) } {
        Ok(())
    } else {
        Err(KernelError::NoMemory)
    }
}

pub fn unmap_page(virt: VirtAddr) -> KResult<()> {
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/paging.rs (line 18) - [public] map_page

```
pub fn map_page(virt: VirtAddr, phys: PhysAddr, flags: u64) -> KResult<()> {
    if unsafe { paging_map_page(virt.0, phys.0, flags) } {
        Ok(())
    } else {
        Err(KernelError::NoMemory)
    }
}

pub fn unmap_page(virt: VirtAddr) -> KResult<()> {
    if unsafe { paging_unmap_page(virt.0) } {
        Ok(())
    } else {
        // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/paging.rs (line 26) - [public] unmap_page

```
pub fn unmap_page(virt: VirtAddr) -> KResult<()> {
    if unsafe { paging_unmap_page(virt.0) } {
        Ok(())
    } else {
        Err(KernelError::InvalidArg)
    }
}

pub fn flush_tlb_page(virt: VirtAddr) {
    context::flush_page(virt);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/mem/paging.rs (line 34) - [public] flush_tlb_page

```
pub fn flush_tlb_page(virt: VirtAddr) {
    context::flush_page(virt);
}

pub fn flush_tlb_all() {
    context::flush_tlb_all();
}
```



```

kernel_Workspace/base/kw-base/src/mem/paging.rs (line 38) - [public] flush_tlb_all
    pub fn flush_tlb_all() {
        context::flush_tlb_all();
    }

kernel_Workspace/base/kw-base/src/mem/slab.rs (line 4) - [internal] slab_cache_create
    fn slab_cache_create(name: *const u8, obj_size: usize, align: usize) -> *mut u8;
    fn slab_cache_destroy(cache: *mut u8);
    fn slab_alloc(cache: *mut u8) -> *mut u8;
    fn slab_free(cache: *mut u8, ptr: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}
// ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/mem/slab.rs (line 5) - [internal] slab_cache_destroy
    fn slab_cache_destroy(cache: *mut u8);
    fn slab_alloc(cache: *mut u8) -> *mut u8;
    fn slab_free(cache: *mut u8, ptr: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

// ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/mem/slab.rs (line 6) - [internal] slab_alloc
    fn slab_alloc(cache: *mut u8) -> *mut u8;
    fn slab_free(cache: *mut u8, ptr: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

impl SlabCache {
    // ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/mem/slab.rs (line 7) - [internal] slab_free
    fn slab_free(cache: *mut u8, ptr: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

impl SlabCache {
    pub fn new(name: &str, obj_size: usize, align: usize) -> KResult<Self> {
        // ... (truncated, see source file for more)
    }
}

```

kernel_workspace/base/kw-base/src/mem/slab.rs (line 10) - [public] SlabCache

```
pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

impl SlabCache {
    pub fn new(name: &str, obj_size: usize, align: usize) -> KResult<Self> {
        let c_name = name.as_ptr();
        let cache = unsafe { slab_cache_create(c_name, obj_size, align) };
        if cache.is_null() {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/base/kw-base/src/mem/slab.rs (line 18) - [public] new

```
pub fn new(name: &str, obj_size: usize, align: usize) -> KResult<Self> {
    let c_name = name.as_ptr();
    let cache = unsafe { slab_cache_create(c_name, obj_size, align) };
    if cache.is_null() {
        Err(KernelError::NoMemory)
    } else {
        Ok(Self { inner: cache })
    }
}

pub fn alloc<T>(&self) -> KResult<*mut T> {
    let ptr = unsafe { slab_alloc(self.inner) };
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/mem/slab.rs (line 28) - [public] alloc

```
pub fn alloc<T>(&self) -> KResult<*mut T> {
    let ptr = unsafe { slab_alloc(self.inner) };
    if ptr.is_null() {
        Err(KernelError::NoMemory)
    } else {
        Ok(ptr as *mut T)
    }
}

pub fn free<T>(&self, ptr: *mut T) {
    if !ptr.is_null() {
        unsafe { slab_free(self.inner, ptr as *mut u8) };
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/mem/slab.rs (line 37) - [public] free

```
pub fn free<T>(&self, ptr: *mut T) {
    if !ptr.is_null() {
        unsafe { slab_free(self.inner, ptr as *mut u8) };
    }
}

impl Drop for SlabCache {
    fn drop(&mut self) {
        if !self.inner.is_null() {
            unsafe { slab_cache_destroy(self.inner) };
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/mem/slab.rs (line 45) - [internal] drop

```
fn drop(&mut self) {
    if !self.inner.is_null() {
        unsafe { slab_cache_destroy(self.inner) };
    }
}
```

kernel_Workspace/base/kw-base/src/sync/atomic.rs (line 5) - [public] compiler_fence

```
pub fn compiler_fence() {
    core::sync::atomic::compiler_fence(Ordering::SeqCst);
}

#[inline(always)]
pub fn memory_barrier() {
    #[cfg(target_arch = "x86_64")]
    unsafe { core::arch::asm!("mfence", options(nostack, preserves_flags)); }
}
```

kernel_Workspace/base/kw-base/src/sync/atomic.rs (line 10) - [public] memory_barrier

```
pub fn memory_barrier() {
    #[cfg(target_arch = "x86_64")]
    unsafe { core::arch::asm!("mfence", options(nostack, preserves_flags)); }
}
```

kernel_Workspace/base/kw-base/src/sync/futex.rs (line 4) - [internal] k_futex_wait

```
fn k_futex_wait(uaddr: *const u32, expected_val: u32, timeout_ns: u64) -> i32;
fn k_futex_wake(uaddr: *const u32, wake_count: u32) -> i32;

pub const FUTEX_WAIT: u32 = 0;
pub const FUTEX_WAKE: u32 = 1;

pub fn wait(uaddr: *const u32, expected: u32, timeout_ns: u64) -> KResult<()> {
    let rc = unsafe { k_futex_wait(uaddr, expected, timeout_ns) };
    if rc == 0 { Ok(()) } else { Err(KernelError::from_code(rc)) }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/sync/futex.rs (line 5) - [internal] k_futex_wake

```
fn k_futex_wake(uaddr: *const u32, wake_count: u32) -> i32;

pub const FUTEX_WAIT: u32 = 0;
pub const FUTEX_WAKE: u32 = 1;

pub fn wait(uaddr: *const u32, expected: u32, timeout_ns: u64) -> KResult<()> {
    let rc = unsafe { k_futex_wait(uaddr, expected, timeout_ns) };
    if rc == 0 { Ok(()) } else { Err(KernelError::from_code(rc)) }
}

pub fn wake(uaddr: *const u32, count: u32) -> KResult<u32> {
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/sync/futex.rs (line 11) - [public] wait

```
pub fn wait(uaddr: *const u32, expected: u32, timeout_ns: u64) -> KResult<()> {
    let rc = unsafe { k_futex_wait(uaddr, expected, timeout_ns) };
    if rc == 0 { Ok(()) } else { Err(KernelError::from_code(rc)) }
}

pub fn wake(uaddr: *const u32, count: u32) -> KResult<u32> {
    let rc = unsafe { k_futex_wake(uaddr, count) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as u32) }
}
```

kernel_Workspace/base/kw-base/src/sync/futex.rs (line 16) - [public] wake

```
pub fn wake(uaddr: *const u32, count: u32) -> KResult<u32> {
    let rc = unsafe { k_futex_wake(uaddr, count) };
    if rc < 0 { Err(KernelError::from_code(rc)) } else { Ok(rc as u32) }
}
```

kernel_workspace/base/kw-base/src/sync/mutex.rs (line 11) - [public] Mutex

// A sleeping mutex. Uses atomics for the fast path.

// In a full implementation, the slow path would enqueue the thread

// into a WaitQueue and call the scheduler. Here we use yield as a placeholder.

```
pub struct Mutex<T> {
    state: AtomicU8,
    data: UnsafeCell<T>,
}

unsafe impl<T: Send> Sync for Mutex<T> {}
unsafe impl<T: Send> Send for Mutex<T> {}

impl<T> Mutex<T> {
    pub const fn new(data: T) -> Self {
        Self {
            state: AtomicU8::new(UNLOCKED),
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/base/kw-base/src/sync/mutex.rs (line 27) - [public] lock

```
pub fn lock(&self) -> MutexGuard<'_, T> {
    loop {
        if self.state.compare_exchange_weak(
            UNLOCKED, LOCKED, Ordering::Acquire, Ordering::Relaxed
        ).is_ok() {
            return MutexGuard { lock: self };
        }
        // Slow path: yield CPU instead of busy-spinning
        crate::task::scheduler::yield_now();
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/mutex.rs (line 39) - [public] try_lock

```
pub fn try_lock(&self) -> Option<MutexGuard<'_, T>> {
    if self.state.compare_exchange(
        UNLOCKED, LOCKED, Ordering::Acquire, Ordering::Relaxed
    ).is_ok() {
        Some(MutexGuard { lock: self })
    } else {
        None
    }
}

fn unlock(&self) {
    self.state.store(UNLOCKED, Ordering::Release);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/sync/mutex.rs (line 49) - [internal] unlock

```
fn unlock(&self) {
    self.state.store(UNLOCKED, Ordering::Release);
    // TODO: Wake up one thread from the waitqueue if any
}

pub struct MutexGuard<'a, T> {
    lock: &'a Mutex<T>,
}

impl<T> Deref for MutexGuard<'_, T> {
    type Target = T;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/sync/mutex.rs (line 55) - [public] MutexGuard

```
pub struct MutexGuard<'a, T> {
    lock: &'a Mutex<T>,
}

impl<T> Deref for MutexGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}

impl<T> DerefMut for MutexGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() } }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/mutex.rs (line 61) - [internal] deref

```
fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}

impl<T> DerefMut for MutexGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() } }
}

impl<T> Drop for MutexGuard<'_, T> {
    fn drop(&mut self) { self.lock.unlock(); }
}
```

kernel_workspace/base/kw-base/src/sync/mutex.rs (line 65) - [internal] deref_mut

```
fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() } }
}

impl<T> Drop for MutexGuard<'_, T> {
    fn drop(&mut self) { self.lock.unlock(); }
}
```

kernel_workspace/base/kw-base/src/sync/mutex.rs (line 69) - [internal] drop

```
fn drop(&mut self) { self.lock.unlock(); }
}
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 5) - [public] RwLock

```
pub struct RwLock<T> {
    // -1 = write locked, >0 = number of readers, 0 = unlocked
    state: AtomicIsz,
    data: UnsafeCell<T>,
}

unsafe impl<T: Send + Sync> Sync for RwLock<T> {}
unsafe impl<T: Send> Send for RwLock<T> {}

impl<T> RwLock<T> {
    pub const fn new(data: T) -> Self {
        Self { state: AtomicIsz::new(0), data: UnsafeCell::new(data) }
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 19) - [public] read

```
pub fn read(&self) -> RwLockReadGuard<'_, T> {
    loop {
        let val = self.state.load(Ordering::Relaxed);
        if val >= 0 {
            if self.state.compare_exchange_weak(
                val, val + 1, Ordering::Acquire, Ordering::Relaxed
            ).is_ok() {
                return RwLockReadGuard { lock: self };
            }
        }
        crate::task::scheduler::yield_now();
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 33) - [public] write

```
pub fn write(&self) -> RwLockWriteGuard<'_, T> {
    loop {
        if self.state.compare_exchange_weak(
            0, -1, Ordering::Acquire, Ordering::Relaxed
        ).is_ok() {
            return RwLockWriteGuard { lock: self };
        }
        crate::task::scheduler::yield_now();
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 45) - [public] RwLockReadGuard

```
pub struct RwLockReadGuard<'a, T> { lock: &'a RwLock<T> }
impl<T> Deref for RwLockReadGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}
impl<T> Drop for RwLockReadGuard<'_, T> {
    fn drop(&mut self) { self.lock.state.fetch_sub(1, Ordering::Release); }
}

pub struct RwLockWriteGuard<'a, T> { lock: &'a RwLock<T> }
impl<T> Deref for RwLockWriteGuard<'_, T> {
    type Target = T;
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 48) - [internal] deref

```
fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}
impl<T> Drop for RwLockReadGuard<'_, T> {
    fn drop(&mut self) { self.lock.state.fetch_sub(1, Ordering::Release); }
}

pub struct RwLockWriteGuard<'a, T> { lock: &'a RwLock<T> }
impl<T> Deref for RwLockWriteGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}
impl<T> DerefMut for RwLockWriteGuard<'_, T> {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 51) - [internal] drop

```
fn drop(&mut self) { self.lock.state.fetch_sub(1, Ordering::Release); }
}

pub struct RwLockWriteGuard<'a, T> { lock: &'a RwLock<T> }
impl<T> Deref for RwLockWriteGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}
impl<T> DerefMut for RwLockWriteGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() } }
}
impl<T> Drop for RwLockWriteGuard<'_, T> {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 54) - [public] RwLockWriteGuard

```
pub struct RwLockWriteGuard<'a, T> { lock: &'a RwLock<T> }
impl<T> Deref for RwLockWriteGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}
impl<T> DerefMut for RwLockWriteGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() } }
}
impl<T> Drop for RwLockWriteGuard<'_, T> {
    fn drop(&mut self) { self.lock.state.store(0, Ordering::Release); }
}
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 57) - [internal] deref

```
fn deref(&self) -> &T { unsafe { &*self.lock.data.get() } }
}
impl<T> DerefMut for RwLockWriteGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() } }
}
impl<T> Drop for RwLockWriteGuard<'_, T> {
    fn drop(&mut self) { self.lock.state.store(0, Ordering::Release); }
}
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 60) - [internal] deref_mut

```
fn deref_mut(&mut self) -> &mut T { unsafe { &mut *self.lock.data.get() } }
}
impl<T> Drop for RwLockWriteGuard<'_, T> {
    fn drop(&mut self) { self.lock.state.store(0, Ordering::Release); }
}
```

kernel_workspace/base/kw-base/src/sync/rwlock.rs (line 63) - [internal] drop

```
fn drop(&mut self) { self.lock.state.store(0, Ordering::Release); }
}
```

kernel_workspace/base/kw-base/src/sync/semaphore.rs (line 3) - [public] Semaphore

```
pub struct Semaphore {
    count: AtomicI32,
}

impl Semaphore {
    pub const fn new(initial: i32) -> Self {
        Self { count: AtomicI32::new(initial) }
    }

    pub fn acquire(&self) {
        loop {
            let val = self.count.load(Ordering::Relaxed);
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/base/kw-base/src/sync/semaphore.rs (line 12) - [public] acquire

```
pub fn acquire(&self) {
    loop {
        let val = self.count.load(Ordering::Relaxed);
        if val > 0 {
            if self.count.compare_exchange_weak(
                val, val - 1, Ordering::Acquire, Ordering::Relaxed
            ).is_ok() {
                return;
            }
        }
        crate::task::scheduler::yield_now();
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/sync/semaphore.rs (line 26) - [public] try_acquire

```
pub fn try_acquire(&self) -> bool {
    let val = self.count.load(Ordering::Relaxed);
    if val > 0 {
        self.count.compare_exchange(
            val, val - 1, Ordering::Acquire, Ordering::Relaxed
        ).is_ok()
    } else {
        false
    }
}

pub fn release(&self) {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/sync/semaphore.rs (line 37) - [public] release

```
pub fn release(&self) {
    self.count.fetch_add(1, Ordering::Release);
    // TODO: Wake up one thread from the waitqueue
}
}
```

kernel_workspace/base/kw-base/src/task/context.rs (line 6) - [public] context_switch

```
pub fn context_switch(old_ctx: *mut TaskContext, new_ctx: *mut TaskContext);
}

pub unsafe fn switch(old: &mut TaskContext, new: &TaskContext) {
    context_switch(old, new as *const _ as *mut _);
}
}
```

kernel_workspace/base/kw-base/src/task/process.rs (line 6) - [public] ProcessState

```
pub enum ProcessState {
    Created,
    Running,
    Sleeping,
    Zombie,
    Dead,
}

pub struct Process {
    pub pid: Pid,
    pub world_id: WorldId,
    pub state: ProcessState,
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/task/process.rs (line 14) - [public] Process

```
pub struct Process {
    pub pid: Pid,
    pub world_id: WorldId,
    pub state: ProcessState,
    pub cr3_phys: u64,
}

impl Process {
    pub fn new(pid: Pid, world_id: WorldId) -> KResult<Self> {
        let cr3 = pmm::alloc_zero_frame().map_err(KernelError::from)?;
        Ok(Self {
            pid,
            // ... (truncated, see source file for more)
```


kernel_workspace/base/kw-base/src/task/process.rs (line 22) - [public] new

```
pub fn new(pid: Pid, world_id: WorldId) -> KResult<Self> {
    let cr3 = pmm::alloc_zero_frame().map_err(KernelError::from)?;
    Ok(Self {
        pid,
        world_id,
        state: ProcessState::Created,
        cr3_phys: cr3.0,
    })
}

pub fn transition(&mut self, new_state: ProcessState) -> KResult<()> {
    match (self.state, new_state) {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/task/process.rs (line 32) - [public] transition

```
pub fn transition(&mut self, new_state: ProcessState) -> KResult<()> {
    match (self.state, new_state) {
        (ProcessState::Created, ProcessState::Running) |
        (ProcessState::Running, ProcessState::Sleeping) |
        (ProcessState::Sleeping, ProcessState::Running) |
        (ProcessState::Running, ProcessState::Zombie) |
        (ProcessState::Zombie, ProcessState::Dead) => {
            self.state = new_state;
            Ok(())
        }
        _ => Err(KernelError::InvalidArg),
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/task/process.rs (line 48) - [internal] drop

```
fn drop(&mut self) {
    if self.cr3_phys != 0 {
        let _ = pmm::free_frame(kstd_base::PhysAddr(self.cr3_phys));
    }
}
}
```

kernel_workspace/base/kw-base/src/task/queue.rs (line 5) - [public] WaitQueue

```
pub struct WaitQueue {
    lock: SpinLock<()>,
    head: ListHead,
}

impl WaitQueue {
    pub const fn new() -> Self {
        Self {
            lock: SpinLock::new(),
            head: ListHead::new(),
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/task/queue.rs (line 18) - [public] init

```
pub fn init(&mut self) {
    self.head.init();
}

pub fn sleep(&self, thread: &mut Thread) {
    let _guard = self.lock.lock();
    thread.state = super::thread::ThreadState::Blocked;
    // In a real implementation, we would link thread's list_node into self.head
    // and then call context::switch() to the scheduler.
}

pub fn wake_one(&self) {
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/task/queue.rs (line 22) - [public] sleep

```
pub fn sleep(&self, thread: &mut Thread) {
    let _guard = self.lock.lock();
    thread.state = super::thread::ThreadState::Blocked;
    // In a real implementation, we would link thread's list_node into self.head
    // and then call context::switch() to the scheduler.
}

pub fn wake_one(&self) {
    let _guard = self.lock.lock();
    if self.head.is_empty() { return; }
    // In a real implementation, we would pop the first thread from self.head,
    // set its state to Ready, and add it to the scheduler runqueue.
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/task/queue.rs (line 29) - [public] wake_one

```
pub fn wake_one(&self) {
    let _guard = self.lock.lock();
    if self.head.is_empty() { return; }
    // In a real implementation, we would pop the first thread from self.head,
    // set its state to Ready, and add it to the scheduler runqueue.
}

pub fn wake_all(&self) {
    let _guard = self.lock.lock();
    // Wake all threads in the list
}
}
```

kernel_Workspace/base/kw-base/src/task/queue.rs (line 36) - [public] wake_all

```
pub fn wake_all(&self) {
    let _guard = self.lock.lock();
    // Wake all threads in the list
}
}
```

kernel_Workspace/base/kw-base/src/task/scheduler.rs (line 4) - [public] yield_now

```
pub fn yield_now() {
    unsafe {
        core::arch::asm!("int 0x30", options(nomem, nostack)); // Trigger scheduler IPI/interrupt
    }
}

pub fn halt_cpu() {
    loop {
        crate::arch::interrupts::enable();
        crate::arch::interrupts::halt();
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/task/scheduler.rs (line 10) - [public] halt_cpu

```
pub fn halt_cpu() {
    loop {
        crate::arch::interrupts::enable();
        crate::arch::interrupts::halt();
    }
}

pub fn set_current_priority(prio: i32) -> KResult<> {
    if prio < -20 || prio > 19 {
        return Err(KernelError::InvalidArg);
    }
    // Hook into legacy kernel scheduler via gluecore if needed
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/task/scheduler.rs (line 17) - [public] set_current_priority

```
pub fn set_current_priority(prio: i32) -> KResult<()> {
    if prio < -20 || prio > 19 {
        return Err(KernelError::InvalidArg);
    }
    // Hook into legacy kernel scheduler via gluecore if needed
    Ok(())
}
```

kernel_workspace/base/kw-base/src/task/thread.rs (line 5) - [public] ThreadState

```
pub enum ThreadState {
    Ready,
    Running,
    Blocked,
    Terminated,
}

pub struct Thread {
    pub tid: Tid,
    pub state: ThreadState,
    pub context: TaskContext,
    pub kernel_stack_top: u64,
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/task/thread.rs (line 12) - [public] Thread

```
pub struct Thread {
    pub tid: Tid,
    pub state: ThreadState,
    pub context: TaskContext,
    pub kernel_stack_top: u64,
}

impl Thread {
    pub fn new(tid: Tid, entry_point: u64, stack_top: u64) -> Self {
        let mut ctx = TaskContext::default();
        ctx.rip = entry_point;
        ctx.rsp = stack_top;
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/task/thread.rs (line 20) - [public] new

```
pub fn new(tid: Tid, entry_point: u64, stack_top: u64) -> Self {
    let mut ctx = TaskContext::default();
    ctx.rip = entry_point;
    ctx.rsp = stack_top;
    ctx.rflags = 0x202; // Interrupts enabled

    Self {
        tid,
        state: ThreadState::Ready,
        context: ctx,
        kernel_stack_top: stack_top,
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/utils/bitmap.rs (line 3) - [public] Bitmap

```
pub struct Bitmap {
    data: *mut u64,
    bits: usize,
    owned: bool,
}

impl Bitmap {
    pub fn new_external(ptr: *mut u64, bits: usize) -> Self {
        Self { data: ptr, bits, owned: false }
    }

    pub fn words_needed(bits: usize) -> usize {
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 10) - [public] new_external
    pub fn new_external(ptr: *mut u64, bits: usize) -> Self {
        Self { data: ptr, bits, owned: false }
    }

    pub fn words_needed(bits: usize) -> usize {
        (bits + 63) / 64
    }

    #[inline]
    fn word_idx(&self, bit: usize) -> usize {
        bit / 64
    }
    // ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 14) - [public] words_needed
    pub fn words_needed(bits: usize) -> usize {
        (bits + 63) / 64
    }

    #[inline]
    fn word_idx(&self, bit: usize) -> usize {
        bit / 64
    }

    #[inline]
    fn bit_mask(&self, bit: usize) -> u64 {
        1u64 << (bit % 64)
    }
    // ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 19) - [internal] word_idx
    fn word_idx(&self, bit: usize) -> usize {
        bit / 64
    }

    #[inline]
    fn bit_mask(&self, bit: usize) -> u64 {
        1u64 << (bit % 64)
    }

    pub fn set(&mut self, bit: usize) {
        if bit < self.bits {
            unsafe { *self.data.add(self.word_idx(bit)) |= self.bit_mask(bit); }
        }
    }
    // ... (truncated, see source file for more)

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 24) - [internal] bit_mask
    fn bit_mask(&self, bit: usize) -> u64 {
        1u64 << (bit % 64)
    }

    pub fn set(&mut self, bit: usize) {
        if bit < self.bits {
            unsafe { *self.data.add(self.word_idx(bit)) |= self.bit_mask(bit); }
        }
    }

    pub fn clear(&mut self, bit: usize) {
        if bit < self.bits {
            // ... (truncated, see source file for more)
        }
    }

```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 28) - [public] set

```
pub fn set(&mut self, bit: usize) {
    if bit < self.bits {
        unsafe { *self.data.add(self.word_idx(bit)) |= self.bit_mask(bit); }
    }
}

pub fn clear(&mut self, bit: usize) {
    if bit < self.bits {
        unsafe { *self.data.add(self.word_idx(bit)) &= !self.bit_mask(bit); }
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 34) - [public] clear

```
pub fn clear(&mut self, bit: usize) {
    if bit < self.bits {
        unsafe { *self.data.add(self.word_idx(bit)) &= !self.bit_mask(bit); }
    }
}

pub fn test(&self, bit: usize) -> bool {
    if bit >= self.bits { return false; }
    unsafe { (*self.data.add(self.word_idx(bit)) & self.bit_mask(bit)) != 0 }
}

pub fn find_first_zero(&self) -> Option<usize> {
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 40) - [public] test

```
pub fn test(&self, bit: usize) -> bool {
    if bit >= self.bits { return false; }
    unsafe { (*self.data.add(self.word_idx(bit)) & self.bit_mask(bit)) != 0 }
}

pub fn find_first_zero(&self) -> Option<usize> {
    for i in 0..Self::words_needed(self.bits) {
        let word = unsafe { *self.data.add(i) };
        if word != u64::MAX {
            let bit = (!word).trailing_zeros() as usize;
            let idx = i * 64 + bit;
            if idx < self.bits { return Some(idx); }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 45) - [public] find_first_zero

```
pub fn find_first_zero(&self) -> Option<usize> {
    for i in 0..Self::words_needed(self.bits) {
        let word = unsafe { *self.data.add(i) };
        if word != u64::MAX {
            let bit = (!word).trailing_zeros() as usize;
            let idx = i * 64 + bit;
            if idx < self.bits { return Some(idx); }
        }
    }
    None
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 57) - [public] find_first_set

```
pub fn find_first_set(&self) -> Option<usize> {
    for i in 0..Self::words_needed(self.bits) {
        let word = unsafe { *self.data.add(i) };
        if word != 0 {
            let bit = word.trailing_zeros() as usize;
            let idx = i * 64 + bit;
            if idx < self.bits { return Some(idx); }
        }
    }
    None
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 69) - [public] alloc_first_zero

```
pub fn alloc_first_zero(&mut self) -> Option<usize> {
    let idx = self.find_first_zero()?;
    self.set(idx);
    Some(idx)
}

pub fn count_set(&self) -> usize {
    let mut count = 0;
    for i in 0..Self::words_needed(self.bits) {
        count += unsafe { (*self.data.add(i)).count_ones() as usize };
    }
    count
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 75) - [public] count_set

```
pub fn count_set(&self) -> usize {
    let mut count = 0;
    for i in 0..Self::words_needed(self.bits) {
        count += unsafe { (*self.data.add(i)).count_ones() as usize };
    }
    count
}

pub fn count_free(&self) -> usize {
    self.bits - self.count_set()
}

}
```

kernel_Workspace/base/kw-base/src/utils/bitmap.rs (line 83) - [public] count_free

```
pub fn count_free(&self) -> usize {
    self.bits - self.count_set()
}

}
```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 9) - [internal] Node

```
struct Node<K, V> {
    keys: [mem::MaybeUninit<K>; MAX_KEYS],
    vals: [mem::MaybeUninit<V>; MAX_KEYS],
    children: [*mut Node<K, V>; MAX_KEYS + 1],
    len: usize,
    leaf: bool,
}

impl<K: Ord, V> Node<K, V> {
    fn new_leaf() -> *mut Self {
        let layout = Layout::new(mem::size_of::<Self>(), mem::align_of::<Self>());
        let ptr = KernelHeap.alloc(layout).unwrap() as *mut Self;
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/utils/btree.rs (line 18) - [internal] new_leaf

```
fn new_leaf() -> *mut Self {
    let layout = Layout::new(mem::size_of::<Self>(), mem::align_of::<Self>());
    let ptr = KernelHeap.alloc(layout).unwrap() as *mut Self;
    unsafe {
        (*ptr).len = 0;
        (*ptr).leaf = true;
        (*ptr).children = [core::ptr::null_mut(); MAX_KEYS + 1];
    }
    ptr
}

fn new_internal() -> *mut Self {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/utils/btree.rs (line 29) - [internal] new_internal

```
fn new_internal() -> *mut Self {
    let ptr = Self::new_leaf();
    unsafe { (*ptr).leaf = false; }
    ptr
}

pub struct BTreeMap<K, V> {
    root: *mut Node<K, V>,
    len: usize,
}
```

// ... (truncated, see source file for more)

kernel_workspace/base/kw-base/src/utils/btree.rs (line 36) - [public] BTreeMap

```
pub struct BTreeMap<K, V> {
    root: *mut Node<K, V>,
    len: usize,
}

unsafe impl<K: Send, V: Send> Send for BTreeMap<K, V> {}
unsafe impl<K: Sync, V: Sync> Sync for BTreeMap<K, V> {}

impl<K: Ord, V> BTreeMap<K, V> {
    pub fn new() -> Self {
        Self { root: core::ptr::null_mut(), len: 0 }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/utils/btree.rs (line 45) - [public] new

```
pub fn new() -> Self {
    Self { root: core::ptr::null_mut(), len: 0 }
}

pub fn insert(&mut self, key: K, val: V) {
    if self.root.is_null() {
        self.root = Node::new_leaf();
    }

    let r = self.root;
    if unsafe { (*r).len == MAX_KEYS } {
        let s = Node::new_internal();
        // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 49) - [public] insert

```
pub fn insert(&mut self, key: K, val: V) {
    if self.root.is_null() {
        self.root = Node::new_leaf();
    }

    let r = self.root;
    if unsafe { (*r).len == MAX_KEYS } {
        let s = Node::new_internal();
        unsafe {
            (*s).children[0] = r;
            self.split_child(s, 0);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 69) - [internal] split_child

```
fn split_child(&mut self, parent: *mut Node<K, V>, i: usize) {
    let child = unsafe { (*parent).children[i] };
    let new_node = if unsafe { (*child).leaf } { Node::new_leaf() } else { Node::new_internal() };
    let mid = MIN_KEYS;

    unsafe {
        for j in 0..MIN_KEYS {
            (*new_node).keys[j] = mem::MaybeUninit::new((*child).keys[mid + 1 + j].assume_init_read());
            (*new_node).vals[j] = mem::MaybeUninit::new((*child).vals[mid + 1 + j].assume_init_read());
        }
        if !(*child).leaf {
            for j in 0..B {
                // ... (truncated, see source file for more)
            }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 102) - [internal] insert_non_full

```
fn insert_non_full(&mut self, node: *mut Node<K, V>, key: K, val: V) {
    let mut i = unsafe { (*node).len };
    unsafe {
        if (*node).leaf {
            while i > 0 && key < (*node).keys[i - 1].assume_init_ref() {
                (*node).keys[i] = mem::MaybeUninit::new((*node).keys[i - 1].assume_init_read());
                (*node).vals[i] = mem::MaybeUninit::new((*node).vals[i - 1].assume_init_read());
                i -= 1;
            }
            (*node).keys[i] = mem::MaybeUninit::new(key);
            (*node).vals[i] = mem::MaybeUninit::new(val);
            (*node).len += 1;
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 129) - [public] get

```
pub fn get(&self, key: &K) -> Option<V> {
    if self.root.is_null() { return None; }
    let mut node = self.root;
    loop {
        let n = unsafe { &*node };
        let mut i = 0;
        while i < n.len && *key > *unsafe { n.keys[i].assume_init_ref() } {
            i += 1;
        }
        if i < n.len && *key == *unsafe { n.keys[i].assume_init_ref() } {
            return Some(unsafe { n.vals[i].assume_init_ref() });
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 146) - [public] len

```
pub fn len(&self) -> usize { self.len }
pub fn is_empty(&self) -> bool { self.len == 0 }
}
```

kernel_Workspace/base/kw-base/src/utils/btree.rs (line 147) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.len == 0 }
}
```


kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 6) - [internal] FnvHasher

```
struct FnvHasher(u64);
impl Hasher for FnvHasher {
    fn finish(&self) -> u64 { self.0 }
    fn write(&mut self, bytes: &[u8]) {
        for &b in bytes {
            self.0 ^= b as u64;
            self.0 = self.0.wrapping_mul(0x1000000001b3);
        }
    }
}

fn hash_key<K: Hash>(key: &K) -> usize {
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 8) - [internal] finish

```
fn finish(&self) -> u64 { self.0 }
fn write(&mut self, bytes: &[u8]) {
    for &b in bytes {
        self.0 ^= b as u64;
        self.0 = self.0.wrapping_mul(0x1000000001b3);
    }
}

fn hash_key<K: Hash>(key: &K) -> usize {
    let mut h = FnvHasher(0xcbf29ce484222325);
    key.hash(&mut h);
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 9) - [internal] write

```
fn write(&mut self, bytes: &[u8]) {
    for &b in bytes {
        self.0 ^= b as u64;
        self.0 = self.0.wrapping_mul(0x1000000001b3);
    }
}

fn hash_key<K: Hash>(key: &K) -> usize {
    let mut h = FnvHasher(0xcbf29ce484222325);
    key.hash(&mut h);
    h.finish() as usize
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 17) - [internal] hash_key

```
fn hash_key<K: Hash>(key: &K) -> usize {
    let mut h = FnvHasher(0xcbf29ce484222325);
    key.hash(&mut h);
    h.finish() as usize
}

enum Slot<K, V> { Empty, Occupied(K, V), Tombstone }

pub struct HashMap<K, V> {
    slots: *mut Slot<K, V>,
    cap: usize,
    len: usize,
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/utils/hashmap.rs (line 25) - [public] HashMap

```
pub struct HashMap<K, V> {
    slots: *mut Slot<K, V>,
    cap: usize,
    len: usize,
}

unsafe impl<K: Send, V: Send> Send for HashMap<K, V> {}
unsafe impl<K: Sync, V: Sync> Sync for HashMap<K, V> {}

impl<K: Hash + Eq, V> HashMap<K, V> {
    pub fn new() -> Self {
        Self { slots: core::ptr::null_mut(), cap: 0, len: 0 }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/utils/hashmap.rs (line 35) - [public] new

```
pub fn new() -> Self {
    Self { slots: core::ptr::null_mut(), cap: 0, len: 0 }
}

fn grow(&mut self, new_cap: usize) {
    let layout = Layout::new(new_cap * mem::size_of::<Slot<K, V>>(), mem::align_of::<Slot<K, V>>());
    let new_slots = KernelHeap.alloc(layout).unwrap() as *mut Slot<K, V>;
    for i in 0..new_cap {
        unsafe { new_slots.add(i).write(Slot::Empty); }
    }

    let old_slots = self.slots;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/utils/hashmap.rs (line 39) - [internal] grow

```
fn grow(&mut self, new_cap: usize) {
    let layout = Layout::new(new_cap * mem::size_of::<Slot<K, V>>(), mem::align_of::<Slot<K, V>>());
    let new_slots = KernelHeap.alloc(layout).unwrap() as *mut Slot<K, V>;
    for i in 0..new_cap {
        unsafe { new_slots.add(i).write(Slot::Empty); }
    }

    let old_slots = self.slots;
    let old_cap = self.cap;
    self.slots = new_slots;
    self.cap = new_cap;
    self.len = 0;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-base/src/utils/hashmap.rs (line 64) - [public] insert

```
pub fn insert(&mut self, key: K, val: V) {
    if self.cap == 0 || self.len * 4 >= self.cap * 3 {
        self.grow(if self.cap == 0 { 8 } else { self.cap * 2 });
    }
    let h = hash_key(&key);
    let mut idx = h % self.cap;
    loop {
        let slot = unsafe { &mut *self.slots.add(idx) };
        match slot {
            Slot::Empty | Slot::Tombstone => {
                *slot = Slot::Occupied(key, val);
                self.len += 1;
            }
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 87) - [public] get

```
pub fn get(&self, key: &K) -> Option<&V> {
    if self.cap == 0 { return None; }
    let h = hash_key(key);
    let mut idx = h % self.cap;
    loop {
        let slot = unsafe { &*self.slots.add(idx) };
        match slot {
            Slot::Empty => return None,
            Slot::Occupied(k, v) if *k == *key => return Some(v),
            _ => { idx = (idx + 1) % self.cap; }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 101) - [public] get_mut

```
pub fn get_mut(&mut self, key: &K) -> Option<&mut V> {
    if self.cap == 0 { return None; }
    let h = hash_key(key);
    let mut idx = h % self.cap;
    loop {
        let slot = unsafe { &mut *self.slots.add(idx) };
        match slot {
            Slot::Empty => return None,
            Slot::Occupied(k, v) if *k == *key => return Some(v),
            _ => { idx = (idx + 1) % self.cap; }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 115) - [public] remove

```
pub fn remove(&mut self, key: &K) -> Option<V> {
    if self.cap == 0 { return None; }
    let h = hash_key(key);
    let mut idx = h % self.cap;
    loop {
        let slot = unsafe { &mut *self.slots.add(idx) };
        match slot {
            Slot::Empty => return None,
            Slot::Occupied(k, _) if *k == *key => {
                let old = mem::replace(slot, Slot::Tombstone);
                self.len -= 1;
                if let Slot::Occupied(_, v) = old { return Some(v); }
            }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-base/src/utils/hashmap.rs (line 133) - [public] len

```
pub fn len(&self) -> usize { self.len }
pub fn is_empty(&self) -> bool { self.len == 0 }
}

impl<K, V> Drop for HashMap<K, V> {
    fn drop(&mut self) {
        if !self.slots.is_null() {
            for i in 0..self.cap {
                unsafe { self.slots.add(i).drop_in_place(); }
            }
            let layout = Layout::new(self.cap * mem::size_of::<Slot<K, V>>(), mem::align_of::<Slot<K, V>>());
            KernelHeap.dealloc(self.slots as *mut u8, layout);
        }
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/utils/hashmap.rs (line 134) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.len == 0 }
}

impl<K, V> Drop for HashMap<K, V> {
    fn drop(&mut self) {
        if !self.slots.is_null() {
            for i in 0..self.cap {
                unsafe { self.slots.add(i).drop_in_place(); }
            }
            let layout = Layout::new(self.cap * mem::size_of::<Slot<K, V>>(), mem::align_of::<Slot<K, V>>(),
                KernelHeap.dealloc(self.slots as *mut u8, layout);
            }
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-base/src/utils/hashmap.rs (line 138) - [internal] drop

```
fn drop(&mut self) {
    if !self.slots.is_null() {
        for i in 0..self.cap {
            unsafe { self.slots.add(i).drop_in_place(); }
        }
        let layout = Layout::new(self.cap * mem::size_of::<Slot<K, V>>(), mem::align_of::<Slot<K, V>>(),
            KernelHeap.dealloc(self.slots as *mut u8, layout);
        }
    }
}
}
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 1) - [public] RingBuffer

```
pub struct RingBuffer<T, const N: usize> {
    data: [core::mem::MaybeUninit<T>; N],
    head: usize,
    tail: usize,
    count: usize,
}

impl<T, const N: usize> RingBuffer<T, N> {
    pub const fn new() -> Self {
        Self {
            data: unsafe { core::mem::MaybeUninit::uninit().assume_init() },
            head: 0,
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 18) - [public] push

```
pub fn push(&mut self, val: T) -> bool {
    if self.count == N { return false; }
    self.data[self.head] = core::mem::MaybeUninit::new(val);
    self.head = (self.head + 1) % N;
    self.count += 1;
    true
}

pub fn pop(&mut self) -> Option<T> {
    if self.count == 0 { return None; }
    let val = unsafe { self.data[self.tail].as_ptr().read() };
    self.tail = (self.tail + 1) % N;
    // ... (truncated, see source file for more)
}
}
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 26) - [public] pop

```
pub fn pop(&mut self) -> Option<T> {
    if self.count == 0 { return None; }
    let val = unsafe { self.data[self.tail].as_ptr().read() };
    self.tail = (self.tail + 1) % N;
    self.count -= 1;
    Some(val)
}

pub fn peek(&self) -> Option<&T> {
    if self.count == 0 { return None; }
    Some(unsafe { self.data[self.tail].assume_init_ref() })
}
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 34) - [public] peek

```
pub fn peek(&self) -> Option<T> {
    if self.count == 0 { return None; }
    Some(unsafe { self.data[self.tail].assume_init_ref() })
}

pub fn len(&self) -> usize { self.count }
pub fn is_empty(&self) -> bool { self.count == 0 }
pub fn is_full(&self) -> bool { self.count == N }
pub fn capacity(&self) -> usize { N }
}

impl<T, const N: usize> Drop for RingBuffer<T, N> {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 39) - [public] len

```
pub fn len(&self) -> usize { self.count }
pub fn is_empty(&self) -> bool { self.count == 0 }
pub fn is_full(&self) -> bool { self.count == N }
pub fn capacity(&self) -> usize { N }
}

impl<T, const N: usize> Drop for RingBuffer<T, N> {
    fn drop(&mut self) {
        while self.pop().is_some() {}
    }
}
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 40) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.count == 0 }
pub fn is_full(&self) -> bool { self.count == N }
pub fn capacity(&self) -> usize { N }
}

impl<T, const N: usize> Drop for RingBuffer<T, N> {
    fn drop(&mut self) {
        while self.pop().is_some() {}
    }
}
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 41) - [public] is_full

```
pub fn is_full(&self) -> bool { self.count == N }
pub fn capacity(&self) -> usize { N }
}

impl<T, const N: usize> Drop for RingBuffer<T, N> {
    fn drop(&mut self) {
        while self.pop().is_some() {}
    }
}
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 42) - [public] capacity

```
pub fn capacity(&self) -> usize { N }
}

impl<T, const N: usize> Drop for RingBuffer<T, N> {
    fn drop(&mut self) {
        while self.pop().is_some() {}
    }
}
```

kernel_workspace/base/kw-base/src/utils/ring_buffer.rs (line 46) - [internal] drop

```
fn drop(&mut self) {
    while self.pop().is_some() {}
}
}
```

kernel_workspace/base/kw-lib/src/kernel/syscall.rs (line 7) - [public] syscall0

```
pub fn syscall0(num: u64) -> i64 {
    let ret: i64;
    unsafe {
        core::arch::asm!(
            "int 0x80",
            in("rax") num,
            out("rax") ret,
            options(nostack, preserves_flags)
        );
    }
    ret
}
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/kernel/syscall.rs (line 22) - [public] syscall3

```
pub fn syscall3(num: u64, a1: u64, a2: u64, a3: u64) -> i64 {
    let ret: i64;
    unsafe {
        core::arch::asm!(
            "int 0x80",
            in("rax") num,
            in("rdi") a1,
            in("rsi") a2,
            in("rdx") a3,
            out("rax") ret,
            options(nostack, preserves_flags)
        );
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-lib/src/kernel/time.rs (line 3) - [public] ticks

// High-level time API for libraries and drivers.

```
pub fn ticks() -> u64 {
    kw_base::hal::timers::get_ticks()
}

pub fn frequency() -> u64 {
    kw_base::hal::timers::get_frequency()
}

pub fn sleep_ms(ms: u32) {
    kw_base::hal::timers::sleep_ms(ms);
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/kernel/time.rs (line 7) - [public] frequency

```
pub fn frequency() -> u64 {
    kw_base::hal::timers::get_frequency()
}

pub fn sleep_ms(ms: u32) {
    kw_base::hal::timers::sleep_ms(ms);
}

pub fn sleep_ticks(ticks: u64) {
    let start = ticks();
    while ticks() - start < ticks {
        core::hint::spin_loop();
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-lib/src/kernel/time.rs (line 11) - [public] sleep_ms

```
pub fn sleep_ms(ms: u32) {
    kw_base::hal::timers::sleep_ms(ms);
}

pub fn sleep_ticks(ticks: u64) {
    let start = ticks();
    while ticks() - start < ticks {
        core::hint::spin_loop();
    }
}
```

kernel_workspace/base/kw-lib/src/kernel/time.rs (line 15) - [public] sleep_ticks

```
pub fn sleep_ticks(ticks: u64) {
    let start = ticks();
    while ticks() - start < ticks {
        core::hint::spin_loop();
    }
}
```

kernel_workspace/base/kw-lib/src/mem/allocators/slab.rs (line 8) - [internal] SlabPage

```
struct SlabPage {
    next: *mut SlabPage,
    free_head: *mut u8,
    in_use: u32,
}

pub struct SlabCache {
    obj_size: usize,
    align: usize,
    objs_per_page: usize,
    partial_pages: *mut SlabPage,
    full_pages: *mut SlabPage,
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/allocators/slab.rs (line 14) - [public] SlabCache

```
pub struct SlabCache {
    obj_size: usize,
    align: usize,
    objs_per_page: usize,
    partial_pages: *mut SlabPage,
    full_pages: *mut SlabPage,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

impl SlabCache {
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/allocators/slab.rs (line 26) - [public] new

```
pub fn new(obj_size: usize, align: usize) -> Self {
    let actual_size = if obj_size < core::mem::size_of::<mut u8>() {
        core::mem::size_of::<mut u8>()
    } else {
        obj_size
    };

    let aligned_size = (actual_size + align - 1) & !(align - 1);
    let objs_per_page = (4096 - core::mem::size_of::<SlabPage>()) / aligned_size;

    Self {
        obj_size: aligned_size,
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/allocators/slab.rs (line 45) - [internal] alloc_page

```
fn alloc_page(&self) -> KResult<*mut SlabPage> {
    let layout = core::alloc::Layout::from_size_align(4096, 4096).unwrap();
    let ptr = unsafe { kstd_alloc::KernelHeap.alloc(layout) }.map_err(|_| KernelError::NoMemory)?;

    let page = ptr as *mut SlabPage;
    unsafe {
        (*page).next = core::ptr::null_mut();
        (*page).in_use = 0;

        // Build free list inside the page
        let data_start = ptr.add(core::mem::size_of::<SlabPage>());
        let mut current = data_start;
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/allocators/slab.rs (line 69) - [public] alloc

```
pub fn alloc(&mut self) -> KResult<*mut u8> {
    if self.partial_pages.is_null() {
        let page = self.alloc_page()?;
        unsafe {
            (*page).next = self.partial_pages;
            self.partial_pages = page;
        }
    }

    let page = self.partial_pages;
    let obj = unsafe { (*page).free_head };

    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/allocators/slab.rs (line 99) - [public] free

```
pub fn free(&mut self, obj: *mut u8) {
    // Find the page this object belongs to
    let page_addr = (obj as usize) & !(4096 - 1);
    let page = page_addr as *mut SlabPage;

    unsafe {
        *(obj as *mut *mut u8) = (*page).free_head;
        (*page).free_head = obj;
        (*page).in_use -= 1;

        if (*page).in_use == 0 {
            // TODO: Remove from partial/full list and free page
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 9) - [internal] FreeList

```
struct FreeList {
    head: u64, // Physical address of the first free block, 0 if empty
}

impl FreeList {
    const fn new() -> Self { Self { head: 0 } }
}

pub struct BuddyAllocator {
    base_phys: PhysAddr,
    total_frames: usize,
    free_lists: [FreeList; MAX_ORDER],
    // ... (truncated, see source file for more)
```


kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 17) - [public] BuddyAllocator

```
pub struct BuddyAllocator {
    base_phys: PhysAddr,
    total_frames: usize,
    free_lists: [FreeList; MAX_ORDER],
    // Bitmap to track if a block is split or free
    // For simplicity, we use a basic tracking mechanism here.
    // In a full implementation, this would be a per-order bitmap.
    split_bitmap: *mut u64,
}

unsafe impl Send for BuddyAllocator {}
unsafe impl Sync for BuddyAllocator {}
// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 40) - [public] init

```
pub fn init(&mut self, base: PhysAddr, frames: usize, bitmap_ptr: *mut u64) {
    self.base_phys = base;
    self.total_frames = frames;
    self.split_bitmap = bitmap_ptr;

    // Clear bitmap
    let words = kw_base::utils::Bitmap::words_needed(frames);
    unsafe {
        core::ptr::write_bytes(bitmap_ptr, 0, words * 8);
    }

    // Add all frames to the highest possible order blocks
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 78) - [internal] frame_to_phys

```
fn frame_to_phys(&self, frame: usize) -> PhysAddr {
    PhysAddr(self.base_phys.0 + (frame as u64 * 4096))
}

fn phys_to_frame(&self, phys: PhysAddr) -> usize {
    ((phys.0 - self.base_phys.0) / 4096) as usize
}

fn free_block(&mut self, frame: usize, order: usize) {
    // Mark as not split in bitmap (simplified logic)
    let mut bitmap = Bitmap::new_external(self.split_bitmap, self.total_frames);
    bitmap.clear(frame);
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 82) - [internal] phys_to_frame

```
fn phys_to_frame(&self, phys: PhysAddr) -> usize {
    ((phys.0 - self.base_phys.0) / 4096) as usize
}

fn free_block(&mut self, frame: usize, order: usize) {
    // Mark as not split in bitmap (simplified logic)
    let mut bitmap = Bitmap::new_external(self.split_bitmap, self.total_frames);
    bitmap.clear(frame);

    // Push to free list
    let phys = self.frame_to_phys(frame);
    unsafe {
        // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 86) - [internal] free_block

```
fn free_block(&mut self, frame: usize, order: usize) {
    // Mark as not split in bitmap (simplified logic)
    let mut bitmap = Bitmap::new_external(self.split_bitmap, self.total_frames);
    bitmap.clear(frame);

    // Push to free list
    let phys = self.frame_to_phys(frame);
    unsafe {
        let ptr = phys.0 as *mut u64;
        *ptr = self.free_lists[order].head;
    }
    self.free_lists[order].head = phys.0;
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 100) - [public] alloc

```
pub fn alloc(&mut self, order: usize) -> KResult<PhysAddr> {
    if order >= MAX_ORDER { return Err(KernelError::InvalidArg); }

    // Find the smallest available order >= requested
    let mut current_order = order;
    while current_order < MAX_ORDER && self.free_lists[current_order].head == 0 {
        current_order += 1;
    }

    if current_order == MAX_ORDER {
        return Err(KernelError::NoMemory);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/physical/buddy.rs (line 133) - [public] free

```
pub fn free(&mut self, phys: PhysAddr, order: usize) {
    let mut frame = self.phys_to_frame(phys);
    let mut current_order = order;

    let mut bitmap = Bitmap::new_external(self.split_bitmap, self.total_frames);

    // Merge with buddy if possible
    while current_order < MAX_ORDER - 1 {
        let buddy_frame = frame ^ (1 << current_order);
        if buddy_frame >= self.total_frames { break; }

        // Check if buddy is free and of the same order
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 7) - [public] AddressSpace

```
pub struct AddressSpace {
    pub cr3_phys: u64,
    vma_head: *mut Vma,
    vma_count: usize,
}

unsafe impl Send for AddressSpace {}
unsafe impl Sync for AddressSpace {}

impl AddressSpace {
    pub fn new(cr3: u64) -> Self {
        Self {
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 17) - [public] new

```
pub fn new(cr3: u64) -> Self {
    Self {
        cr3_phys: cr3,
        vma_head: core::ptr::null_mut(),
        vma_count: 0,
    }
}

pub fn map_region(&mut self, start: VirtAddr, size: usize, flags: u32, vma_type: VmaType) -> KResult<Vma> {
    let end = VirtAddr(start.0 + size as u64);

    // Check for overlaps
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 25) - [public] map_region

```
pub fn map_region(&mut self, start: VirtAddr, size: usize, flags: u32, vma_type: VmaType) -> KResult<Vma> {
    let end = VirtAddr(start.0 + size as u64);

    // Check for overlaps
    let mut current = self.vma_head;
    while !current.is_null() {
        let vma = unsafe { &*current };
        if vma.overlaps(start, end) {
            return Err(KernelError::AlreadyExists);
        }
        current = vma.next;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 53) - [public] find_vma

```
pub fn find_vma(&self, addr: VirtAddr) -> Option<&Vma> {
    let mut current = self.vma_head;
    while !current.is_null() {
        let vma = unsafe { &*current };
        if vma.contains(addr) {
            return Some(vma);
        }
        current = vma.next;
    }
    None
}

// ... (truncated, see source file for more)
```

kernel_Workspace/base/kw-lib/src/mem/virtual/aspace.rs (line 67) - [internal] drop

```
fn drop(&mut self) {
    let mut current = self.vma_head;
    while !current.is_null() {
        let next = unsafe { (*current).next };
        let layout = core::alloc::Layout::new::(<Vma>());
        unsafe { kstd_alloc::KernelHeap.dealloc(current as *mut u8, layout); }
        current = next;
    }
}

}
```

kernel_Workspace/base/kw-lib/src/mem/virtual/vma.rs (line 6) - [public] VmaType

```
pub enum VmaType {
    Anonymous,
    FileBacked,
    DeviceMapped,
    Stack,
    Heap,
}

pub struct Vma {
    pub start: VirtAddr,
    pub end: VirtAddr,
    pub flags: u32, // PROT_READ | PROT_WRITE | PROT_EXEC
    // ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/mem/virtual/vma.rs (line 14) - [public] Vma

```
pub struct Vma {
    pub start: VirtAddr,
    pub end: VirtAddr,
    pub flags: u32, // PROT_READ | PROT_WRITE | PROT_EXEC
    pub vma_type: VmaType,
    pub file_offset: u64, // If FileBacked
    pub next: *mut Vma,    // For linked list in AddressSpace
}

impl Vma {
    pub fn new(start: VirtAddr, end: VirtAddr, flags: u32, vma_type: VmaType) -> Self {
        Self {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/base/kw-lib/src/mem/virtual/vma.rs (line 24) - [public] new

```
pub fn new(start: VirtAddr, end: VirtAddr, flags: u32, vma_type: VmaType) -> Self {
    Self {
        start,
        end,
        flags,
        vma_type,
        file_offset: 0,
        next: core::ptr::null_mut(),
    }
}

pub fn size(&self) -> usize {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/base/kw-lib/src/mem/virtual/vma.rs (line 35) - [public] size

```
pub fn size(&self) -> usize {
    (self.end.0 - self.start.0) as usize
}

pub fn contains(&self, addr: VirtAddr) -> bool {
    addr.0 >= self.start.0 && addr.0 < self.end.0
}

pub fn overlaps(&self, start: VirtAddr, end: VirtAddr) -> bool {
    self.start.0 < end.0 && start.0 < self.end.0
}
}
```

kernel_workspace/base/kw-lib/src/mem/virtual/vma.rs (line 39) - [public] contains

```
pub fn contains(&self, addr: VirtAddr) -> bool {
    addr.0 >= self.start.0 && addr.0 < self.end.0
}

pub fn overlaps(&self, start: VirtAddr, end: VirtAddr) -> bool {
    self.start.0 < end.0 && start.0 < self.end.0
}
}
```

kernel_workspace/base/kw-lib/src/mem/virtual/vma.rs (line 43) - [public] overlaps

```
pub fn overlaps(&self, start: VirtAddr, end: VirtAddr) -> bool {
    self.start.0 < end.0 && start.0 < self.end.0
}
}
```

kernel_workspace/base/kw-lib/src/Utils/string.rs (line 20) - [public] StrBuffer

// Simple fixed-capacity string buffer for formatting paths/logs without heap allocation.

```
pub struct StrBuffer<const N: usize> {
    buf: [u8; N],
    len: usize,
}

impl<const N: usize> StrBuffer<N> {
    pub const fn new() -> Self {
        Self { buf: [0; N], len: 0 }
    }

    pub fn push_str(&mut self, s: &str) -> bool {
        let bytes = s.as_bytes();
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-lib/src/Utils/string.rs (line 30) - [public] push_str

```
pub fn push_str(&mut self, s: &str) -> bool {
    let bytes = s.as_bytes();
    if self.len + bytes.len() > N { return false; }
    self.buf[self.len..self.len + bytes.len()].copy_from_slice(bytes);
    self.len += bytes.len();
    true
}

pub fn as_str(&self) -> &str {
    unsafe { core::str::from_utf8_unchecked(&self.buf[..self.len]) }
}

// ... (truncated, see source file for more)
```

kernel_workspace/base/kw-lib/src/Utils/string.rs (line 38) - [public] as_str

```
pub fn as_str(&self) -> &str {
    unsafe { core::str::from_utf8_unchecked(&self.buf[..self.len]) }
}

pub fn as_cstr_ptr(&mut self) -> *const u8 {
    if self.len < N {
        self.buf[self.len] = 0;
    }
    self.buf.as_ptr()
}

}
```

kernel_workspace/base/kw-lib/src/Utils/string.rs (line 42) - [public] as_cstr_ptr

```
pub fn as_cstr_ptr(&mut self) -> *const u8 {
    if self.len < N {
        self.buf[self.len] = 0;
    }
    self.buf.as_ptr()
}

}
```

kernel_workspace/base/kw-odin-abi/src/lib.rs (line 2) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/base/kw-odin-abi/src/lib.rs (line 11) - [internal] it_works

```
fn it_works() {  
    let result = add(2, 2);  
    assert_eq!(result, 4);  
}
```

kernel_workspace/core/gluecore/build.rs (line 1) - [internal] main

```
fn main() {}
```

kernel_workspace/core/gluecore/src/bindings.rs (line 6) - [public] CPhysRange

```
pub struct CPhysRange {  
    pub start: u64,  
    pub end: u64,  
}  
  
impl From<CPhysRange> for (PhysAddr, usize) {  
    fn from(val: CPhysRange) -> Self {  
        (PhysAddr(val.start), (val.end - val.start) as usize)  
    }  
}
```

kernel_workspace/core/gluecore/src/bindings.rs (line 12) - [internal] from

```
fn from(val: CPhysRange) -> Self {  
    (PhysAddr(val.start), (val.end - val.start) as usize)  
}
```

kernel_workspace/core/gluecore/src/boot.rs (line 4) - [internal] boot_get_memory_map

```
fn boot_get_memory_map(out_ptr: *mut u64, out_size: *mut usize) -> bool;  
fn boot_get_cmdline(out_ptr: *mut u64, out_len: *mut usize) -> bool;  
fn boot_get_framebuffer(out_phys: *mut u64, out_w: *mut u32, out_h: *mut u32, out_pitch: *mut u32) ->  
}  
  
pub struct BootMemoryMap {  
    pub ptr: u64,  
    pub size: usize,  
}  
  
pub fn get_memory_map() -> Option<BootMemoryMap> {  
    let mut ptr: u64 = 0;  
    // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/boot.rs (line 5) - [internal] boot_get_cmdline

```
fn boot_get_cmdline(out_ptr: *mut u64, out_len: *mut usize) -> bool;  
fn boot_get_framebuffer(out_phys: *mut u64, out_w: *mut u32, out_h: *mut u32, out_pitch: *mut u32) ->  
}  
  
pub struct BootMemoryMap {  
    pub ptr: u64,  
    pub size: usize,  
}  
  
pub fn get_memory_map() -> Option<BootMemoryMap> {  
    let mut ptr: u64 = 0;  
    let mut size: usize = 0;  
    // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/boot.rs (line 6) - [internal] boot_get_framebuffer

```
fn boot_get_framebuffer(out_phys: *mut u64, out_w: *mut u32, out_h: *mut u32, out_pitch: *mut u32) ->
}

pub struct BootMemoryMap {
    pub ptr: u64,
    pub size: usize,
}

pub fn get_memory_map() -> Option<BootMemoryMap> {
    let mut ptr: u64 = 0;
    let mut size: usize = 0;
    if unsafe { boot_get_memory_map(&mut ptr, &mut size) } {
        // ... (truncated, see source file for more)
    }
```

kernel_workspace/core/gluecore/src/boot.rs (line 9) - [public] BootMemoryMap

```
pub struct BootMemoryMap {
    pub ptr: u64,
    pub size: usize,
}

pub fn get_memory_map() -> Option<BootMemoryMap> {
    let mut ptr: u64 = 0;
    let mut size: usize = 0;
    if unsafe { boot_get_memory_map(&mut ptr, &mut size) } {
        Some(BootMemoryMap { ptr, size })
    } else {
        None
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/gluecore/src/boot.rs (line 14) - [public] get_memory_map

```
pub fn get_memory_map() -> Option<BootMemoryMap> {
    let mut ptr: u64 = 0;
    let mut size: usize = 0;
    if unsafe { boot_get_memory_map(&mut ptr, &mut size) } {
        Some(BootMemoryMap { ptr, size })
    } else {
        None
    }
}

pub struct BootFramebuffer {
    pub phys: PhysAddr,
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/gluecore/src/boot.rs (line 24) - [public] BootFramebuffer

```
pub struct BootFramebuffer {
    pub phys: PhysAddr,
    pub width: u32,
    pub height: u32,
    pub pitch: u32,
}

pub fn get_framebuffer() -> Option<BootFramebuffer> {
    let mut phys: u64 = 0;
    let mut w: u32 = 0;
    let mut h: u32 = 0;
    let mut pitch: u32 = 0;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/gluecore/src/boot.rs (line 31) - [public] get_framebuffer

```
pub fn get_framebuffer() -> Option<BootFramebuffer> {
    let mut phys: u64 = 0;
    let mut w: u32 = 0;
    let mut h: u32 = 0;
    let mut pitch: u32 = 0;
    if unsafe { boot_get_framebuffer(&mut phys, &mut w, &mut h, &mut pitch) } {
        Some(BootFramebuffer { phys: PhysAddr(phys), width: w, height: h, pitch })
    } else {
        None
    }
}
```

kernel_workspace/core/gluecore/src/bridge/caps/check.rs (line 6) - [public] has_cap

```
pub fn has_cap(world_id: u32, cap: CapId) -> bool {
    unsafe { extern_block::caps_has(world_id, cap.0 as u8) == 1 }
}

pub fn request_cap(target_world: u32, cap: CapId) -> Result<(), Status> {
    let rc = unsafe { extern_block::caps_request(target_world, cap.0 as u8) };
    if rc == 0 {
        Ok(())
    } else {
        Err(Status::PermissionDenied)
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/caps/check.rs (line 10) - [public] request_cap

```
pub fn request_cap(target_world: u32, cap: CapId) -> Result<(), Status> {
    let rc = unsafe { extern_block::caps_request(target_world, cap.0 as u8) };
    if rc == 0 {
        Ok(())
    } else {
        Err(Status::PermissionDenied)
    }
}

pub fn release_cap(world_id: u32, cap: CapId) -> Result<(), Status> {
    let rc = unsafe { extern_block::caps_release(world_id, cap.0 as u8) };
    if rc == 0 {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/gluecore/src/bridge/caps/check.rs (line 19) - [public] release_cap

```
pub fn release_cap(world_id: u32, cap: CapId) -> Result<(), Status> {
    let rc = unsafe { extern_block::caps_release(world_id, cap.0 as u8) };
    if rc == 0 {
        Ok(())
    } else {
        Err(Status::InvalidArgument)
    }
}
```

kernel_workspace/core/gluecore/src/bridge/mm/pmm.rs (line 7) - [public] alloc_frame

/// Allocates a single physical frame (4KB).

```
pub fn alloc_frame() -> Result<PhysAddr, Status> {
    let mut phys: u64 = 0;
    if unsafe { extern_block::pmm_alloc_frame(&mut phys) } {
        Ok(PhysAddr(phys))
    } else {
        Err(Status::OutOfMemory)
    }
}
```

/// Allocates a single zeroed physical frame.

```
pub fn alloc_zero_frame() -> Result<PhysAddr, Status> {
    let mut phys: u64 = 0;
    // ... (truncated, see source file for more)
}
```


kernel_workspace/core/gluecore/src/bridge/mm/pmm.rs (line 17) - [public] alloc_zero_frame

/// Allocates a single zeroed physical frame.

```
pub fn alloc_zero_frame() -> Result<PhysAddr, Status> {
    let mut phys: u64 = 0;
    if unsafe { extern_block::pmm_alloc_zero_frame(&mut phys) } {
        Ok(PhysAddr(phys))
    } else {
        Err(Status::OutOfMemory)
    }
}

/// Allocates contiguous physical frames.
pub fn alloc_frames(count: usize) -> Result<PhysAddr, Status> {
    let mut phys: u64 = 0;
    // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/mm/pmm.rs (line 27) - [public] alloc_frames

/// Allocates contiguous physical frames.

```
pub fn alloc_frames(count: usize) -> Result<PhysAddr, Status> {
    let mut phys: u64 = 0;
    if unsafe { extern_block::pmm_alloc_frames(count, &mut phys) } {
        Ok(PhysAddr(phys))
    } else {
        Err(Status::OutOfMemory)
    }
}

/// Allocates contiguous physical frames with a specific alignment (in frames).
pub fn alloc_frames_aligned(count: usize, align_frames: usize) -> Result<PhysAddr, Status> {
    let mut phys: u64 = 0;
    // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/mm/pmm.rs (line 37) - [public] alloc_frames_aligned

/// Allocates contiguous physical frames with a specific alignment (in frames).

```
pub fn alloc_frames_aligned(count: usize, align_frames: usize) -> Result<PhysAddr, Status> {
    let mut phys: u64 = 0;
    if unsafe { extern_block::pmm_alloc_frames_aligned(count, align_frames, &mut phys) } {
        Ok(PhysAddr(phys))
    } else {
        Err(Status::OutOfMemory)
    }
}

/// Frees a single physical frame.
pub fn free_frame(addr: PhysAddr) -> Result<(), Status> {
    if unsafe { extern_block::pmm_free_frame(addr.0) } {
        // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/mm/pmm.rs (line 47) - [public] free_frame

/// Frees a single physical frame.

```
pub fn free_frame(addr: PhysAddr) -> Result<(), Status> {
    if unsafe { extern_block::pmm_free_frame(addr.0) } {
        Ok(())
    } else {
        Err(Status::InvalidArgument)
    }
}

/// Frees contiguous physical frames.
pub fn free_frames(addr: PhysAddr, count: usize) -> Result<(), Status> {
    if unsafe { extern_block::pmm_free_frames(addr.0, count) } {
        Ok(())
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/mm/pmm.rs (line 56) - [public] free_frames

/// Frees contiguous physical frames.

```
pub fn free_frames(addr: PhysAddr, count: usize) -> Result<(), Status> {
    if unsafe { extern_block::pmm_free_frames(addr.0, count) } {
        Ok(())
    } else {
        Err(Status::InvalidArgument)
    }
}
```

kernel_workspace/core/gluecore/src/bridge/mm/vmm.rs (line 14) - [public] alloc

/// Allocates virtual memory and maps it to physical frames.

```
pub fn alloc(bytes: usize, flags: u32) -> Result<VirtAddr, Status> {
    let mut virt: u64 = 0;
    if unsafe { extern_block::vmm_alloc(bytes, flags, &mut virt) } {
        Ok(VirtAddr(virt))
    } else {
        Err(Status::OutOfMemory)
    }
}
```

/// Allocates virtual memory with a specific alignment.

```
pub fn alloc_aligned(bytes: usize, align: usize, flags: u32) -> Result<VirtAddr, Status> {
    let mut virt: u64 = 0;
    // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/mm/vmm.rs (line 24) - [public] alloc_aligned

/// Allocates virtual memory with a specific alignment.

```
pub fn alloc_aligned(bytes: usize, align: usize, flags: u32) -> Result<VirtAddr, Status> {
    let mut virt: u64 = 0;
    if unsafe { extern_block::vmm_alloc_aligned(bytes, align, flags, &mut virt) } {
        Ok(VirtAddr(virt))
    } else {
        Err(Status::OutOfMemory)
    }
}
```

/// Frees previously allocated virtual memory.

```
pub fn free(addr: VirtAddr, bytes: usize) -> Result<(), Status> {
    if unsafe { extern_block::vmm_free(addr.0, bytes) } {
        // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/mm/vmm.rs (line 34) - [public] free

/// Frees previously allocated virtual memory.

```
pub fn free(addr: VirtAddr, bytes: usize) -> Result<(), Status> {
    if unsafe { extern_block::vmm_free(addr.0, bytes) } {
        Ok(())
    } else {
        Err(Status::InvalidArgument)
    }
}
```

/// Maps a physical device region (MMIO) into the kernel virtual space.

```
pub fn map_device(phys: PhysAddr, len: usize) -> Result<VirtAddr, Status> {
    let mut virt: u64 = 0;
    if unsafe { extern_block::vmm_map_device(phys.0, len, &mut virt) } {
        // ... (truncated, see source file for more)
```

kernel_workspace/core/gluecore/src/bridge/mm/vmm.rs (line 43) - [public] map_device

/// Maps a physical device region (MMIO) into the kernel virtual space.

```
pub fn map_device(phys: PhysAddr, len: usize) -> Result<VirtAddr, Status> {
    let mut virt: u64 = 0;
    if unsafe { extern_block::vmm_map_device(phys.0, len, &mut virt) } {
        Ok(VirtAddr(virt))
    } else {
        Err(Status::BadAddress)
    }
}

/// Unmaps a device region.
pub fn unmap_device(addr: VirtAddr, len: usize) -> Result<(), Status> {
    if unsafe { extern_block::vmm_unmap_device(addr.0, len) } {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/gluecore/src/bridge/mm/vmm.rs (line 53) - [public] unmap_device

/// Unmaps a device region.

```
pub fn unmap_device(addr: VirtAddr, len: usize) -> Result<(), Status> {
    if unsafe { extern_block::vmm_unmap_device(addr.0, len) } {
        Ok(())
    } else {
        Err(Status::InvalidArgument)
    }
}

/// Translates a kernel virtual address to a physical address.
pub fn virt_to_phys(vaddr: VirtAddr) -> PhysAddr {
    let ptr = vaddr.0 as *const u8;
    PhysAddr(unsafe { extern_block::kvirt_to_phys(ptr) })
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/gluecore/src/bridge/mm/vmm.rs (line 62) - [public] virt_to_phys

/// Translates a kernel virtual address to a physical address.

```
pub fn virt_to_phys(vaddr: VirtAddr) -> PhysAddr {
    let ptr = vaddr.0 as *const u8;
    PhysAddr(unsafe { extern_block::kvirt_to_phys(ptr) })
}
```

kernel_workspace/core/gluecore/src/context.rs (line 4) - [internal] paging_read_cr3

```
fn paging_read_cr3() -> u64;
fn paging_write_cr3(pml4_phys: u64);
fn paging_flush_tlb_all();
fn paging_flush_page(addr: u64);
fn context_switch_save_restore(old_ctx: *mut u8, new_ctx: *mut u8);

pub fn read_cr3() -> PhysAddr {
    PhysAddr(unsafe { paging_read_cr3() })
}

pub fn write_cr3(pml4: PhysAddr) {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/gluecore/src/context.rs (line 5) - [internal] paging_write_cr3

```
fn paging_write_cr3(pml4_phys: u64);
fn paging_flush_tlb_all();
fn paging_flush_page(addr: u64);
fn context_switch_save_restore(old_ctx: *mut u8, new_ctx: *mut u8);

pub fn read_cr3() -> PhysAddr {
    PhysAddr(unsafe { paging_read_cr3() })
}

pub fn write_cr3(pml4: PhysAddr) {
    unsafe { paging_write_cr3(pml4.0); }
    // ... (truncated, see source file for more)
}
```

```

kernel_Workspace/core/gluecore/src/context.rs (line 6) - [internal] paging_flush_tlb_all
    fn paging_flush_tlb_all();
    fn paging_flush_page(addr: u64);
    fn context_switch_save_restore(old_ctx: *mut u8, new_ctx: *mut u8);
}

pub fn read_cr3() -> PhysAddr {
    PhysAddr(unsafe { paging_read_cr3() })
}

pub fn write_cr3(pml4: PhysAddr) {
    unsafe { paging_write_cr3(pml4.0); }
}

// ... (truncated, see source file for more)

kernel_Workspace/core/gluecore/src/context.rs (line 7) - [internal] paging_flush_page
    fn paging_flush_page(addr: u64);
    fn context_switch_save_restore(old_ctx: *mut u8, new_ctx: *mut u8);
}

pub fn read_cr3() -> PhysAddr {
    PhysAddr(unsafe { paging_read_cr3() })
}

pub fn write_cr3(pml4: PhysAddr) {
    unsafe { paging_write_cr3(pml4.0); }
}

// ... (truncated, see source file for more)

kernel_Workspace/core/gluecore/src/context.rs (line 8) - [internal]
context_switch_save_restore
    fn context_switch_save_restore(old_ctx: *mut u8, new_ctx: *mut u8);
}

pub fn read_cr3() -> PhysAddr {
    PhysAddr(unsafe { paging_read_cr3() })
}

pub fn write_cr3(pml4: PhysAddr) {
    unsafe { paging_write_cr3(pml4.0); }
}

pub fn flush_tlb_all() {
    // ... (truncated, see source file for more)

kernel_Workspace/core/gluecore/src/context.rs (line 11) - [public] read_cr3
pub fn read_cr3() -> PhysAddr {
    PhysAddr(unsafe { paging_read_cr3() })
}

pub fn write_cr3(pml4: PhysAddr) {
    unsafe { paging_write_cr3(pml4.0); }
}

pub fn flush_tlb_all() {
    unsafe { paging_flush_tlb_all(); }
}

// ... (truncated, see source file for more)

```

kernel_Workspace/core/gluecore/src/context.rs (line 15) - [public] write_cr3

```
pub fn write_cr3(pml4: PhysAddr) {
    unsafe { paging_write_cr3(pml4.0); }
}

pub fn flush_tlb_all() {
    unsafe { paging_flush_tlb_all(); }
}

pub fn flush_page(addr: VirtAddr) {
    unsafe { paging_flush_page(addr.0); }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/context.rs (line 19) - [public] flush_tlb_all

```
pub fn flush_tlb_all() {
    unsafe { paging_flush_tlb_all(); }
}

pub fn flush_page(addr: VirtAddr) {
    unsafe { paging_flush_page(addr.0); }
}

pub unsafe fn switch_context(old_ctx: *mut u8, new_ctx: *mut u8) {
    context_switch_save_restore(old_ctx, new_ctx);
}
```

kernel_Workspace/core/gluecore/src/context.rs (line 23) - [public] flush_page

```
pub fn flush_page(addr: VirtAddr) {
    unsafe { paging_flush_page(addr.0); }
}

pub unsafe fn switch_context(old_ctx: *mut u8, new_ctx: *mut u8) {
    context_switch_save_restore(old_ctx, new_ctx);
}
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 8) - [public] pmm_alloc_frame

// --- Physical Memory Manager (PMM) ---

```
pub fn pmm_alloc_frame(out_phys: *mut u64) -> bool;
pub fn pmm_alloc_zero_frame(out_phys: *mut u64) -> bool;
pub fn pmm_alloc_frames(count: usize, out_phys: *mut u64) -> bool;
pub fn pmm_alloc_frames_aligned(count: usize, align_frames: usize, out_phys: *mut u64) -> bool;
pub fn pmm_free_frame(phys: u64) -> bool;
pub fn pmm_free_frames(phys: u64, count: usize) -> bool;

// --- Virtual Memory Manager (VMM) ---
pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_free(virt: u64, bytes: usize) -> bool;
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 9) - [public]

pmm_alloc_zero_frame

```
pub fn pmm_alloc_zero_frame(out_phys: *mut u64) -> bool;
pub fn pmm_alloc_frames(count: usize, out_phys: *mut u64) -> bool;
pub fn pmm_alloc_frames_aligned(count: usize, align_frames: usize, out_phys: *mut u64) -> bool;
pub fn pmm_free_frame(phys: u64) -> bool;
pub fn pmm_free_frames(phys: u64, count: usize) -> bool;

// --- Virtual Memory Manager (VMM) ---
pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_free(virt: u64, bytes: usize) -> bool;
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;
// ... (truncated, see source file for more)
```

```

kernel_workspace/core/gluecore/src/ffi/c/extern_block.rs (line 10) - [public] pmm_alloc_frames
pub fn pmm_alloc_frames(count: usize, out_phys: *mut u64) -> bool;
pub fn pmm_alloc_frames_aligned(count: usize, align_frames: usize, out_phys: *mut u64) -> bool;
pub fn pmm_free_frame(phys: u64) -> bool;
pub fn pmm_free_frames(phys: u64, count: usize) -> bool;

// --- Virtual Memory Manager (VMM) ---
pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_free(virt: u64, bytes: usize) -> bool;
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;

// ... (truncated, see source file for more)

kernel_workspace/core/gluecore/src/ffi/c/extern_block.rs (line 11) - [public]
pmm_alloc_frames_aligned
pub fn pmm_alloc_frames_aligned(count: usize, align_frames: usize, out_phys: *mut u64) -> bool;
pub fn pmm_free_frame(phys: u64) -> bool;
pub fn pmm_free_frames(phys: u64, count: usize) -> bool;

// --- Virtual Memory Manager (VMM) ---
pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_free(virt: u64, bytes: usize) -> bool;
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;

// --- Kernel Heap ---
// ... (truncated, see source file for more)

kernel_workspace/core/gluecore/src/ffi/c/extern_block.rs (line 12) - [public] pmm_free_frame
pub fn pmm_free_frame(phys: u64) -> bool;
pub fn pmm_free_frames(phys: u64, count: usize) -> bool;

// --- Virtual Memory Manager (VMM) ---
pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_free(virt: u64, bytes: usize) -> bool;
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;

// --- Kernel Heap ---
pub fn kmalloc(size: usize) -> *mut u8;
// ... (truncated, see source file for more)

kernel_workspace/core/gluecore/src/ffi/c/extern_block.rs (line 13) - [public] pmm_free_frames
pub fn pmm_free_frames(phys: u64, count: usize) -> bool;

// --- Virtual Memory Manager (VMM) ---
pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_free(virt: u64, bytes: usize) -> bool;
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;

// --- Kernel Heap ---
pub fn kmalloc(size: usize) -> *mut u8;
pub fn kzalloc(size: usize) -> *mut u8;
// ... (truncated, see source file for more)

```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 16) - [public] vmm_alloc

// --- Virtual Memory Manager (VMM) ---

```
pub fn vmm_alloc(bytes: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_free(virt: u64, bytes: usize) -> bool;
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;
```

// --- Kernel Heap ---

```
pub fn kmalloc(size: usize) -> *mut u8;
pub fn kzalloc(size: usize) -> *mut u8;
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 17) - [public]

vmm_alloc_aligned

```
pub fn vmm_alloc_aligned(bytes: usize, align: usize, flags: u32, out_virt: *mut u64) -> bool;
pub fn vmm_free(virt: u64, bytes: usize) -> bool;
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;
```

// --- Kernel Heap ---

```
pub fn kmalloc(size: usize) -> *mut u8;
pub fn kzalloc(size: usize) -> *mut u8;
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 18) - [public] vmm_free

```
pub fn vmm_free(virt: u64, bytes: usize) -> bool;
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;
```

// --- Kernel Heap ---

```
pub fn kmalloc(size: usize) -> *mut u8;
pub fn kzalloc(size: usize) -> *mut u8;
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 19) - [public] vmm_map_device

```
pub fn vmm_map_device(phys: u64, len: usize, out_virt: *mut u64) -> bool;
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;
```

// --- Kernel Heap ---

```
pub fn kmalloc(size: usize) -> *mut u8;
pub fn kzalloc(size: usize) -> *mut u8;
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;
```

// ... (truncated, see source file for more)

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 20) - [public] vmm_unmap_device

```
pub fn vmm_unmap_device(virt: u64, len: usize) -> bool;

// --- Kernel Heap ---
pub fn kmalloc(size: usize) -> *mut u8;
pub fn kzalloc(size: usize) -> *mut u8;
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 23) - [public] kmalloc

// --- Kernel Heap ---

```
pub fn kmalloc(size: usize) -> *mut u8;
pub fn kzalloc(size: usize) -> *mut u8;
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 24) - [public] kzalloc

```
pub fn kzalloc(size: usize) -> *mut u8;
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
pub fn paging_unmap_page(virt: u64) -> bool;
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 25) - [public] krealloc

```
pub fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
pub fn paging_unmap_page(virt: u64) -> bool;
pub fn paging_flush_tlb_all();
// ... (truncated, see source file for more)
```


kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 26) - [public] kfree

```
pub fn kfree(ptr: *mut u8);
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
pub fn paging_unmap_page(virt: u64) -> bool;
pub fn paging_flush_tlb_all();
pub fn paging_flush_page(addr: u64);
// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 27) - [public] kalloc_pages

```
pub fn kalloc_pages(pages: usize) -> *mut u8;
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
pub fn paging_unmap_page(virt: u64) -> bool;
pub fn paging_flush_tlb_all();
pub fn paging_flush_page(addr: u64);
}
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 28) - [public] kfree_pages

```
pub fn kfree_pages(ptr: *mut u8, pages: usize);
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
pub fn paging_unmap_page(virt: u64) -> bool;
pub fn paging_flush_tlb_all();
pub fn paging_flush_page(addr: u64);
}
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 29) - [public] kvirt_to_phys

```
pub fn kvirt_to_phys(ptr: *const u8) -> u64;

// --- Paging / Address Spaces ---
pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
pub fn paging_unmap_page(virt: u64) -> bool;
pub fn paging_flush_tlb_all();
pub fn paging_flush_page(addr: u64);
}
```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 32) - [public] paging_read_cr3

```
// --- Paging / Address Spaces ---

pub fn paging_read_cr3() -> u64;
pub fn paging_write_cr3(pml4_phys: u64);
pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
pub fn paging_unmap_page(virt: u64) -> bool;
pub fn paging_flush_tlb_all();
pub fn paging_flush_page(addr: u64);
}
```

```

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 33) - [public] paging_write_cr3
    pub fn paging_write_cr3(pml4_phys: u64);
    pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
    pub fn paging_unmap_page(virt: u64) -> bool;
    pub fn paging_flush_tlb_all();
    pub fn paging_flush_page(addr: u64);
}

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 34) - [public] paging_map_page
    pub fn paging_map_page(virt: u64, phys: u64, flags: u64) -> bool;
    pub fn paging_unmap_page(virt: u64) -> bool;
    pub fn paging_flush_tlb_all();
    pub fn paging_flush_page(addr: u64);
}

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 35) - [public]
paging_unmap_page
    pub fn paging_unmap_page(virt: u64) -> bool;
    pub fn paging_flush_tlb_all();
    pub fn paging_flush_page(addr: u64);
}

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 36) - [public]
paging_flush_tlb_all
    pub fn paging_flush_tlb_all();
    pub fn paging_flush_page(addr: u64);
}

kernel_Workspace/core/gluecore/src/ffi/c/extern_block.rs (line 37) - [public]
paging_flush_page
    pub fn paging_flush_page(addr: u64);
}

kernel_Workspace/core/gluecore/src/interrupts.rs (line 4) - [public] disable
    pub fn disable() {
        unsafe { asm!("cli", options(nomem, nostack)); }
    }

    #[inline(always)]
    pub fn enable() {
        unsafe { asm!("sti", options(nomem, nostack)); }
    }

    #[inline(always)]
    pub fn halt() {
        unsafe { asm!("hlt", options(nomem, nostack)); }
        // ... (truncated, see source file for more)
    }

kernel_Workspace/core/gluecore/src/interrupts.rs (line 9) - [public] enable
    pub fn enable() {
        unsafe { asm!("sti", options(nomem, nostack)); }
    }

    #[inline(always)]
    pub fn halt() {
        unsafe { asm!("hlt", options(nomem, nostack)); }
    }

    #[inline(always)]
    pub fn are_enabled() -> bool {
        let flags: u64;
        // ... (truncated, see source file for more)
    }

```

kernel_Workspace/core/gluecore/src/interrupts.rs (line 14) - [public] halt

```
pub fn halt() {
    unsafe { asm!("hlt", options(nomem, nostack)); }
}

#[inline(always)]
pub fn are_enabled() -> bool {
    let flags: u64;
    unsafe { asm!("pushfq; pop {}", out(reg) flags, options(nomem)); }
    (flags & (1 << 9)) != 0
}

pub struct InterruptGuard {
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/interrupts.rs (line 19) - [public] are_enabled

```
pub fn are_enabled() -> bool {
    let flags: u64;
    unsafe { asm!("pushfq; pop {}", out(reg) flags, options(nomem)); }
    (flags & (1 << 9)) != 0
}

pub struct InterruptGuard {
    was_enabled: bool,
}

impl InterruptGuard {
    pub fn disable() -> Self {
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/interrupts.rs (line 25) - [public] InterruptGuard

```
pub struct InterruptGuard {
    was_enabled: bool,
}

impl InterruptGuard {
    pub fn disable() -> Self {
        let was_enabled = are_enabled();
        disable();
        Self { was_enabled }
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/interrupts.rs (line 30) - [public] disable

```
pub fn disable() -> Self {
    let was_enabled = are_enabled();
    disable();
    Self { was_enabled }
}

impl Drop for InterruptGuard {
    fn drop(&mut self) {
        if self.was_enabled {
            enable();
        }
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/gluecore/src/interrupts.rs (line 38) - [internal] drop

```
fn drop(&mut self) {
    if self.was_enabled {
        enable();
    }
}
```

```
kernel_Workspace/core/gluecore/src/syscalls.rs (line 4) - [internal] syscall_dispatch
    fn syscall_dispatch(num: u64, arg1: u64, arg2: u64, arg3: u64, arg4: u64) -> i64;
}

pub fn invoke(num: u64, args: [u64; 4]) -> Result<u64, Status> {
    let ret = unsafe {
        syscall_dispatch(num, args[0], args[1], args[2], args[3])
    };

    if ret < 0 {
        Err(Status::from((-ret) as u32))
    } else {
        Ok(ret as u64)
    }
    // ... (truncated, see source file for more)
}
```

```
kernel_Workspace/core/gluecore/src/syscalls.rs (line 7) - [public] invoke
pub fn invoke(num: u64, args: [u64; 4]) -> Result<u64, Status> {
    let ret = unsafe {
        syscall_dispatch(num, args[0], args[1], args[2], args[3])
    };

    if ret < 0 {
        Err(Status::from((-ret) as u32))
    } else {
        Ok(ret as u64)
    }
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 5) - [internal] kmalloc
    fn kmalloc(size: usize) -> *mut u8;
    fn kcalloc(size: usize) -> *mut u8;
    fn kcalloc(n: usize, size: usize) -> *mut u8;
    fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
    fn kfree(ptr: *mut u8);
    fn kmalloc_aligned(size: usize, align: usize) -> *mut u8;
}

pub struct KernelHeap;

impl Allocator for KernelHeap {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
        // ... (truncated, see source file for more)
    }
}
```

```
kernel_Workspace/core/kstd_alloc/src/heap.rs (line 6) - [internal] kcalloc
    fn kcalloc(size: usize) -> *mut u8;
    fn kcalloc(n: usize, size: usize) -> *mut u8;
    fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
    fn kfree(ptr: *mut u8);
    fn kmalloc_aligned(size: usize, align: usize) -> *mut u8;
}

pub struct KernelHeap;

impl Allocator for KernelHeap {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
        let ptr = if layout.align() > 8 {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/core/kstd_alloc/src/heap.rs (line 7) - [internal] kcalloc

```
fn kcalloc(n: usize, size: usize) -> *mut u8;
fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
fn kfree(ptr: *mut u8);
fn kmalloc_aligned(size: usize, align: usize) -> *mut u8;
}

pub struct KernelHeap;

impl Allocator for KernelHeap {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
        let ptr = if layout.align() > 8 {
            unsafe { kmalloc_aligned(layout.size(), layout.align()) }
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_alloc/src/heap.rs (line 8) - [internal] krealloc

```
fn krealloc(ptr: *mut u8, size: usize) -> *mut u8;
fn kfree(ptr: *mut u8);
fn kmalloc_aligned(size: usize, align: usize) -> *mut u8;
}

pub struct KernelHeap;

impl Allocator for KernelHeap {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
        let ptr = if layout.align() > 8 {
            unsafe { kmalloc_aligned(layout.size(), layout.align()) }
        } else {
            unsafe { kmalloc(layout.size()) }
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_alloc/src/heap.rs (line 9) - [internal] kfree

```
fn kfree(ptr: *mut u8);
fn kmalloc_aligned(size: usize, align: usize) -> *mut u8;
}

pub struct KernelHeap;

impl Allocator for KernelHeap {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
        let ptr = if layout.align() > 8 {
            unsafe { kmalloc_aligned(layout.size(), layout.align()) }
        } else {
            unsafe { kmalloc(layout.size()) }
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_alloc/src/heap.rs (line 10) - [internal] kmalloc_aligned

```
fn kmalloc_aligned(size: usize, align: usize) -> *mut u8;
}

pub struct KernelHeap;

impl Allocator for KernelHeap {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
        let ptr = if layout.align() > 8 {
            unsafe { kmalloc_aligned(layout.size(), layout.align()) }
        } else {
            unsafe { kmalloc(layout.size()) }
        };
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_alloc/src/heap.rs (line 13) - [public] KernelHeap

```
pub struct KernelHeap;

impl Allocator for KernelHeap {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
        let ptr = if layout.align() > 8 {
            unsafe { kmalloc_aligned(layout.size(), layout.align()) }
        } else {
            unsafe { kmalloc(layout.size()) }
        };
        if ptr.is_null() {
            Err(Status::OutOfMemory)
        } else {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/core/kstd_alloc/src/heap.rs (line 16) - [internal] alloc

```
fn alloc(&self, layout: Layout) -> Result<*mut u8, Status> {
    let ptr = if layout.align() > 8 {
        unsafe { kmalloc_aligned(layout.size(), layout.align()) }
    } else {
        unsafe { kmalloc(layout.size()) }
    };
    if ptr.is_null() {
        Err(Status::OutOfMemory)
    } else {
        Ok(ptr)
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_alloc/src/heap.rs (line 29) - [internal] dealloc

```
fn dealloc(&self, ptr: *mut u8, _layout: Layout) {
    if !ptr.is_null() {
        unsafe { kfree(ptr) };
    }
}

fn realloc(&self, ptr: *mut u8, _old: Layout, new_size: usize) -> Result<*mut u8, Status> {
    let new_ptr = unsafe { krealloc(ptr, new_size) };
    if new_ptr.is_null() {
        Err(Status::OutOfMemory)
    } else {
        Ok(new_ptr)
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_alloc/src/heap.rs (line 35) - [internal] realloc

```
fn realloc(&self, ptr: *mut u8, _old: Layout, new_size: usize) -> Result<*mut u8, Status> {
    let new_ptr = unsafe { krealloc(ptr, new_size) };
    if new_ptr.is_null() {
        Err(Status::OutOfMemory)
    } else {
        Ok(new_ptr)
    }
}

pub fn alloc_zeroed(size: usize) -> Result<*mut u8, Status> {
    let ptr = unsafe { kzalloc(size) };
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/kstd_alloc/src/heap.rs (line 45) - [public] alloc_zeroed

```
pub fn alloc_zeroed(size: usize) -> Result<*mut u8, Status> {
    let ptr = unsafe { kzalloc(size) };
    if ptr.is_null() { Err(Status::OutOfMemory) } else { Ok(ptr) }
}

pub fn alloc_array(n: usize, elem_size: usize) -> Result<*mut u8, Status> {
    let ptr = unsafe { kcalloc(n, elem_size) };
    if ptr.is_null() { Err(Status::OutOfMemory) } else { Ok(ptr) }
}
```

```
kernel_workspace/core/kstd_alloc/src/heap.rs (line 50) - [public] alloc_array
pub fn alloc_array(n: usize, elem_size: usize) -> Result<*mut u8, Status> {
    let ptr = unsafe { kcalloc(n, elem_size) };
    if ptr.is_null() { Err(Status::OutOfMemory) } else { Ok(ptr) }
}
```

```
kernel_workspace/core/kstd_alloc/src/lib.rs (line 15) - [internal] oom
fn oom() -> ! {
    loop {
        core::hint::spin_loop();
    }
}
```

```
pub struct KernelAllocator;

unsafe impl GlobalAlloc for KernelAllocator {
    unsafe fn alloc(&self, layout: CoreLayout) -> *mut u8 {
        let l = Layout::new(layout.size(), layout.align());
        match KernelHeap.alloc(l) {
            // ... (truncated, see source file for more)
        }
    }
}
```

```
kernel_workspace/core/kstd_alloc/src/lib.rs (line 21) - [public] KernelAllocator
pub struct KernelAllocator;

unsafe impl GlobalAlloc for KernelAllocator {
    unsafe fn alloc(&self, layout: CoreLayout) -> *mut u8 {
        let l = Layout::new(layout.size(), layout.align());
        match KernelHeap.alloc(l) {
            Ok(ptr) => ptr,
            Err(_) => oom(),
        }
    }

    unsafe fn dealloc(&self, ptr: *mut u8, layout: CoreLayout) {
        // ... (truncated, see source file for more)
    }
}
```

```
kernel_workspace/core/kstd_alloc/src/omm.rs (line 5) - [public] OomHandler
pub trait OomHandler: Fn() -> ! + Send + Sync {}
impl<T: Fn() -> ! + Send + Sync> OomHandler for T {}

static mut HANDLER: Option<&'static dyn OomHandler> = None;

pub fn set_handler(handler: &'static dyn OomHandler) {
    unsafe { HANDLER = Some(handler) };
}

pub fn invoke() -> ! {
    if OOM_HANDLER_ACTIVE.swap(true, Ordering::SeqCst) {
        loop { core::hint::spin_loop(); }
    }
    // ... (truncated, see source file for more)
}
```

```
kernel_workspace/core/kstd_alloc/src/omm.rs (line 10) - [public] set_handler
pub fn set_handler(handler: &'static dyn OomHandler) {
    unsafe { HANDLER = Some(handler) };
}

pub fn invoke() -> ! {
    if OOM_HANDLER_ACTIVE.swap(true, Ordering::SeqCst) {
        loop { core::hint::spin_loop(); }
    }
    unsafe {
        if let Some(h) = HANDLER {
            h();
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/kstd_alloc/src/omm.rs (line 14) - [public] invoke

```
pub fn invoke() -> ! {
    if OOM_HANDLER_ACTIVE.swap(true, Ordering::SeqCst) {
        loop { core::hint::spin_loop(); }
    }
    unsafe {
        if let Some(h) = HANDLER {
            h();
        }
    }
    panic!("OOM");
}
```

kernel_workspace/core/kstd_alloc/src/page.rs (line 5) - [internal] kalloc_pages

```
fn kalloc_pages(order: usize) -> *mut u8;
fn kfree_pages(ptr: *mut u8, pages: usize);

pub fn alloc_pages(order: usize) -> Result<VirtAddr, Status> {
    let ptr = unsafe { kalloc_pages(order) };
    if ptr.is_null() {
        Err(Status::OutOfMemory)
    } else {
        Ok(VirtAddr(ptr as u64))
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_alloc/src/page.rs (line 6) - [internal] kfree_pages

```
fn kfree_pages(ptr: *mut u8, pages: usize);

pub fn alloc_pages(order: usize) -> Result<VirtAddr, Status> {
    let ptr = unsafe { kalloc_pages(order) };
    if ptr.is_null() {
        Err(Status::OutOfMemory)
    } else {
        Ok(VirtAddr(ptr as u64))
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_alloc/src/page.rs (line 9) - [public] alloc_pages

```
pub fn alloc_pages(order: usize) -> Result<VirtAddr, Status> {
    let ptr = unsafe { kalloc_pages(order) };
    if ptr.is_null() {
        Err(Status::OutOfMemory)
    } else {
        Ok(VirtAddr(ptr as u64))
    }
}

pub fn free_pages(addr: VirtAddr, order: usize) {
    let pages = 1 << order;
    unsafe { kfree_pages(addr.0 as *mut u8, pages) };
    // ... (truncated, see source file for more)
}
```


kernel_workspace/core/kstd_alloc/src/page.rs (line 18) - [public] free_pages

```
pub fn free_pages(addr: VirtAddr, order: usize) {
    let pages = 1 << order;
    unsafe { kfree_pages(addr.0 as *mut u8, pages) };
}

pub fn alloc_single_page() -> Result<VirtAddr, Status> {
    alloc_pages(0)
}

pub fn free_single_page(addr: VirtAddr) {
    free_pages(addr, 0)
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_alloc/src/page.rs (line 23) - [public] alloc_single_page

```
pub fn alloc_single_page() -> Result<VirtAddr, Status> {
    alloc_pages(0)
}

pub fn free_single_page(addr: VirtAddr) {
    free_pages(addr, 0)
}

pub fn pages_for_size(size: usize) -> usize {
    (size + PAGE_SIZE - 1) / PAGE_SIZE
}
```

kernel_workspace/core/kstd_alloc/src/page.rs (line 27) - [public] free_single_page

```
pub fn free_single_page(addr: VirtAddr) {
    free_pages(addr, 0)
}

pub fn pages_for_size(size: usize) -> usize {
    (size + PAGE_SIZE - 1) / PAGE_SIZE
}
```

kernel_workspace/core/kstd_alloc/src/page.rs (line 31) - [public] pages_for_size

```
pub fn pages_for_size(size: usize) -> usize {
    (size + PAGE_SIZE - 1) / PAGE_SIZE
}
```

kernel_workspace/core/kstd_alloc/src/slab.rs (line 5) - [internal] slab_alloc

```
fn slab_alloc(cache: *mut u8) -> *mut u8;
fn slab_free(cache: *mut u8, ptr: *mut u8);
fn slab_cache_create(name: *const u8, obj_size: usize, align: usize) -> *mut u8;
fn slab_cache_destroy(cache: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}
// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_alloc/src/slab.rs (line 6) - [internal] slab_free

```
fn slab_free(cache: *mut u8, ptr: *mut u8);
fn slab_cache_create(name: *const u8, obj_size: usize, align: usize) -> *mut u8;
fn slab_cache_destroy(cache: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

// ... (truncated, see source file for more)
```

```

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 7) - [internal] slab_cache_create
    fn slab_cache_create(name: *const u8, obj_size: usize, align: usize) -> *mut u8;
    fn slab_cache_destroy(cache: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

impl SlabCache {
    // ... (truncated, see source file for more)
}

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 8) - [internal] slab_cache_destroy
    fn slab_cache_destroy(cache: *mut u8);
}

pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

impl SlabCache {
    pub fn new(name: &str, obj_size: usize, align: usize) -> Result<Self, Status> {
        // ... (truncated, see source file for more)
    }
}

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 11) - [public] SlabCache
pub struct SlabCache {
    inner: *mut u8,
}

unsafe impl Send for SlabCache {}
unsafe impl Sync for SlabCache {}

impl SlabCache {
    pub fn new(name: &str, obj_size: usize, align: usize) -> Result<Self, Status> {
        let c_name = name.as_ptr();
        let cache = unsafe { slab_cache_create(c_name, obj_size, align) };
        if cache.is_null() {
            // ... (truncated, see source file for more)
        }
    }
}

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 19) - [public] new
    pub fn new(name: &str, obj_size: usize, align: usize) -> Result<Self, Status> {
        let c_name = name.as_ptr();
        let cache = unsafe { slab_cache_create(c_name, obj_size, align) };
        if cache.is_null() {
            Err(Status::OutOfMemory)
        } else {
            Ok(Self { inner: cache })
        }
    }
}

impl Allocator for SlabCache {
    // ... (truncated, see source file for more)
}

```

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 31) - [internal] alloc

```
fn alloc(&self, _layout: Layout) -> Result<*mut u8, Status> {
    let ptr = unsafe { slab_alloc(self.inner) };
    if ptr.is_null() { Err(Status::OutOfMemory) } else { Ok(ptr) }
}

fn dealloc(&self, ptr: *mut u8, _layout: Layout) {
    if !ptr.is_null() {
        unsafe { slab_free(self.inner, ptr) };
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 36) - [internal] dealloc

```
fn dealloc(&self, ptr: *mut u8, _layout: Layout) {
    if !ptr.is_null() {
        unsafe { slab_free(self.inner, ptr) };
    }
}

impl Drop for SlabCache {
    fn drop(&mut self) {
        if !self.inner.is_null() {
            unsafe { slab_cache_destroy(self.inner) };
        }
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_alloc/src/slab.rs (line 44) - [internal] drop

```
fn drop(&mut self) {
    if !self.inner.is_null() {
        unsafe { slab_cache_destroy(self.inner) };
    }
}

}
```

kernel_Workspace/core/kstd_alloc/src/traits.rs (line 3) - [public] Allocator

```
pub trait Allocator {
    fn alloc(&self, layout: Layout) -> Result<*mut u8, Status>;
    fn dealloc(&self, ptr: *mut u8, layout: Layout);
    fn realloc(&self, ptr: *mut u8, old: Layout, new_size: usize) -> Result<*mut u8, Status> {
        let new_ptr = self.alloc(Layout::new(new_size, old.align()))?;
        if !ptr.is_null() {
            unsafe {
                kstd_base::intrinsics::memcpy(new_ptr, ptr, old.size().min(new_size));
            }
            self.dealloc(ptr, old);
        }
        Ok(new_ptr)
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_alloc/src/traits.rs (line 4) - [internal] alloc

```
fn alloc(&self, layout: Layout) -> Result<*mut u8, Status>;
fn dealloc(&self, ptr: *mut u8, layout: Layout);
fn realloc(&self, ptr: *mut u8, old: Layout, new_size: usize) -> Result<*mut u8, Status> {
    let new_ptr = self.alloc(Layout::new(new_size, old.align()))?;
    if !ptr.is_null() {
        unsafe {
            kstd_base::intrinsics::memcpy(new_ptr, ptr, old.size().min(new_size));
        }
        self.dealloc(ptr, old);
    }
    Ok(new_ptr)
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_alloc/src/traits.rs (line 5) - [internal] dealloc

```
fn dealloc(&self, ptr: *mut u8, layout: Layout);
fn realloc(&self, ptr: *mut u8, old: Layout, new_size: usize) -> Result<*mut u8, Status> {
    let new_ptr = self.alloc(Layout::new(new_size, old.align()))?;
    if !ptr.is_null() {
        unsafe {
            kstd_base::intrinsics::memcpy(new_ptr, ptr, old.size().min(new_size));
        }
        self.dealloc(ptr, old);
    }
    Ok(new_ptr)
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_alloc/src/traits.rs (line 6) - [internal] realloc

```
fn realloc(&self, ptr: *mut u8, old: Layout, new_size: usize) -> Result<*mut u8, Status> {
    let new_ptr = self.alloc(Layout::new(new_size, old.align()))?;
    if !ptr.is_null() {
        unsafe {
            kstd_base::intrinsics::memcpy(new_ptr, ptr, old.size().min(new_size));
        }
        self.dealloc(ptr, old);
    }
    Ok(new_ptr)
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_alloc/src/traits.rs (line 19) - [public] Layout

```
pub struct Layout {
    size: usize,
    align: usize,
}

impl Layout {
    pub const fn new(size: usize, align: usize) -> Self {
        Self { size, align }
    }

    pub const fn from_size(size: usize) -> Self {
        Self { size, align: 8 }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_alloc/src/traits.rs (line 41) - [public] padded

```
pub fn padded(&self) -> usize {
    (self.size + self.align - 1) & !(self.align - 1)
}
}
```

kernel_Workspace/core/kstd_base/src/errors.rs (line 3) - [public] Status

```
pub enum Status {
    Success = 0,
    OutOfMemory = 1,
    InvalidArgument = 2,
    PermissionDenied = 3,
    NotFound = 4,
    AlreadyExists = 5,
    Busy = 6,
    TimedOut = 7,
    NotSupported = 8,
    IOError = 9,
    BadAddress = 10,
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/errors.rs (line 20) - [internal] from

```
fn from(val: u32) -> Self {
    match val {
        0 => Self::Success,
        1 => Self::OutOfMemory,
        2 => Self::InvalidArgument,
        3 => Self::PermissionDenied,
        4 => Self::NotFound,
        5 => Self::AlreadyExists,
        6 => Self::Busy,
        7 => Self::TimedOut,
        8 => Self::NotSupported,
        9 => Self::IOError,
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/errors.rs (line 40) - [internal] from

```
fn from(val: Status) -> Self {
    val as u32
}

impl Status {
    #[inline]
    pub const fn is_ok(self) -> bool {
        matches!(self, Self::Success)
    }

    #[inline]
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/intrinsics.rs (line 12) - [public] memory_barrier

```
pub fn memory_barrier() {
    core::sync::atomic::compiler_fence(core::sync::atomic::Ordering::SeqCst);
}

#[inline(always)]
pub unsafe fn memcpy(dst: *mut u8, src: *const u8, n: usize) {
    core::ptr::copy_nonoverlapping(src, dst, n);
}

#[inline(always)]
pub unsafe fn memset(dst: *mut u8, val: u8, n: usize) {
    core::ptr::write_bytes(dst, val, n);
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/markers.rs (line 3) - [public] SendSync

```
pub struct SendSync<T>(PhantomData<T>);
unsafe impl<T> Send for SendSync<T> {}
unsafe impl<T> Sync for SendSync<T> {}

pub struct Immutable<T>(PhantomData<T>);
impl<T> Immutable<T> {
    pub const fn new() -> Self {
        Self(PhantomData)
    }
}

pub struct Unpin<T>(PhantomData<T>);
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/markers.rs (line 7) - [public] Immutable

```
pub struct Immutable<T>(PhantomData<T>);
impl<T> Immutable<T> {
    pub const fn new() -> Self {
        Self(PhantomData)
    }
}

pub struct Unpin<T>(PhantomData<T>);
impl<T> core::marker::Unpin for Unpin<T> {}
```

kernel_workspace/core/kstd_base/src/markers.rs (line 14) - [public] Unpin

```
pub struct Unpin<T>(PhantomData<T>);  
impl<T> core::marker::Unpin for Unpin<T> {}
```

kernel_workspace/core/kstd_base/src/primitives.rs (line 7) - [public] Ring

```
pub enum Ring {  
    Kernel = 0,  
    Driver = 1,  
    User    = 3,  
}  
  
#[derive(Debug, Clone, Copy, PartialEq, Eq)]  
#[repr(transparent)]  
pub struct ProtFlags(pub u32);  
  
impl ProtFlags {  
    pub const NONE: Self = Self(0);  
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_base/src/primitives.rs (line 15) - [public] ProtFlags

```
pub struct ProtFlags(pub u32);  
  
impl ProtFlags {  
    pub const NONE: Self = Self(0);  
    pub const READ: Self = Self(1 << 0);  
    pub const WRITE: Self = Self(1 << 1);  
    pub const EXEC: Self = Self(1 << 2);  
    pub const USER: Self = Self(1 << 3);  
    pub const DEVICE: Self = Self(1 << 4);  
  
    #[inline]  
    pub const fn contains(self, other: Self) -> bool {  
        // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_base/src/primitives.rs (line 39) - [internal] bitor

```
    fn bitor(self, rhs: Self) -> Self {  
        self.union(rhs)  
    }  
}  
  
#[derive(Debug, Clone, Copy, PartialEq, Eq)]  
#[repr(transparent)]  
pub struct MapFlags(pub u32);  
  
impl MapFlags {  
    pub const NONE: Self = Self(0);  
    pub const FIXED: Self = Self(1 << 0);  
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_base/src/primitives.rs (line 46) - [public] MapFlags

```
pub struct MapFlags(pub u32);  
  
impl MapFlags {  
    pub const NONE: Self = Self(0);  
    pub const FIXED: Self = Self(1 << 0);  
    pub const ANONYMOUS: Self = Self(1 << 1);  
    pub const SHARED: Self = Self(1 << 2);  
    pub const PRIVATE: Self = Self(1 << 3);  
}  
  
#[derive(Debug, Clone, Copy, PartialEq, Eq)]  
#[repr(transparent)]  
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_base/src/primitives.rs (line 58) - [public] IrqFlags

```
pub struct IrqFlags(pub u32);

impl IrqFlags {
    pub const NONE: Self = Self(0);
    pub const LEVEL_TRIGGERED: Self = Self(1 << 0);
    pub const ACTIVE_LOW: Self = Self(1 << 1);
    pub const SHARED: Self = Self(1 << 2);
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 3) - [public] PageAlign

```
pub trait PageAlign {
    fn align_up(self) -> Self;
    fn align_down(self) -> Self;
    fn is_aligned(self) -> bool;
}

impl PageAlign for u64 {
    #[inline]
    fn align_up(self) -> Self {
        (self + PAGE_SIZE as u64 - 1) & !(PAGE_SIZE as u64 - 1)
    }

    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 4) - [internal] align_up

```
fn align_up(self) -> Self;
fn align_down(self) -> Self;
fn is_aligned(self) -> bool;
}

impl PageAlign for u64 {
    #[inline]
    fn align_up(self) -> Self {
        (self + PAGE_SIZE as u64 - 1) & !(PAGE_SIZE as u64 - 1)
    }

    #[inline]
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 5) - [internal] align_down

```
fn align_down(self) -> Self;
fn is_aligned(self) -> bool;
}

impl PageAlign for u64 {
    #[inline]
    fn align_up(self) -> Self {
        (self + PAGE_SIZE as u64 - 1) & !(PAGE_SIZE as u64 - 1)
    }

    #[inline]
    fn align_down(self) -> Self {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 6) - [internal] is_aligned

```
fn is_aligned(self) -> bool;
}

impl PageAlign for u64 {
    #[inline]
    fn align_up(self) -> Self {
        (self + PAGE_SIZE as u64 - 1) & !(PAGE_SIZE as u64 - 1)
    }

    #[inline]
    fn align_down(self) -> Self {
        self & !(PAGE_SIZE as u64 - 1)
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 11) - [internal] align_up

```
fn align_up(self) -> Self {
    (self + PAGE_SIZE as u64 - 1) & !(PAGE_SIZE as u64 - 1)
}

#[inline]
fn align_down(self) -> Self {
    self & !(PAGE_SIZE as u64 - 1)
}

#[inline]
fn is_aligned(self) -> bool {
    self & (PAGE_SIZE as u64 - 1) == 0
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 16) - [internal] align_down

```
fn align_down(self) -> Self {
    self & !(PAGE_SIZE as u64 - 1)
}

#[inline]
fn is_aligned(self) -> bool {
    self & (PAGE_SIZE as u64 - 1) == 0
}
}

impl PageAlign for usize {
    #[inline]
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 21) - [internal] is_aligned

```
fn is_aligned(self) -> bool {
    self & (PAGE_SIZE as u64 - 1) == 0
}
}

impl PageAlign for usize {
    #[inline]
    fn align_up(self) -> Self {
        (self + PAGE_SIZE - 1) & !(PAGE_SIZE - 1)
    }

    #[inline]
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 28) - [internal] align_up

```
fn align_up(self) -> Self {
    (self + PAGE_SIZE - 1) & !(PAGE_SIZE - 1)
}

#[inline]
fn align_down(self) -> Self {
    self & !(PAGE_SIZE - 1)
}

#[inline]
fn is_aligned(self) -> bool {
    self & (PAGE_SIZE - 1) == 0
}
// ... (truncated, see source file for more)
```


kernel_Workspace/core/kstd_base/src/trails.rs (line 33) - [internal] align_down

```
fn align_down(self) -> Self {
    self & !(PAGE_SIZE - 1)
}

#[inline]
fn is_aligned(self) -> bool {
    self & (PAGE_SIZE - 1) == 0
}

pub trait IoPort {
    fn read_u8(&self, port: u16) -> u8;
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 38) - [internal] is_aligned

```
fn is_aligned(self) -> bool {
    self & (PAGE_SIZE - 1) == 0
}

pub trait IoPort {
    fn read_u8(&self, port: u16) -> u8;
    fn write_u8(&self, port: u16, val: u8);
    fn read_u16(&self, port: u16) -> u16;
    fn write_u16(&self, port: u16, val: u16);
    fn read_u32(&self, port: u16) -> u32;
    fn write_u32(&self, port: u16, val: u32);
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 43) - [public] IoPort

```
pub trait IoPort {
    fn read_u8(&self, port: u16) -> u8;
    fn write_u8(&self, port: u16, val: u8);
    fn read_u16(&self, port: u16) -> u16;
    fn write_u16(&self, port: u16, val: u16);
    fn read_u32(&self, port: u16) -> u32;
    fn write_u32(&self, port: u16, val: u32);
}

pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 44) - [internal] read_u8

```
fn read_u8(&self, port: u16) -> u8;
fn write_u8(&self, port: u16, val: u8);
fn read_u16(&self, port: u16) -> u16;
fn write_u16(&self, port: u16, val: u16);
fn read_u32(&self, port: u16) -> u32;
fn write_u32(&self, port: u16, val: u32);
}

pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    fn read_volatile_u64(&self, offset: usize) -> u64;
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 45) - [internal] write_u8

```
fn write_u8(&self, port: u16, val: u8);
fn read_u16(&self, port: u16) -> u16;
fn write_u16(&self, port: u16, val: u16);
fn read_u32(&self, port: u16) -> u32;
fn write_u32(&self, port: u16, val: u32);
}

pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    fn read_volatile_u64(&self, offset: usize) -> u64;
    fn write_volatile_u64(&self, offset: usize, val: u64);
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 46) - [internal] read_u16

```
fn read_u16(&self, port: u16) -> u16;
fn write_u16(&self, port: u16, val: u16);
fn read_u32(&self, port: u16) -> u32;
fn write_u32(&self, port: u16, val: u32);
}

pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    fn read_volatile_u64(&self, offset: usize) -> u64;
    fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 47) - [internal] write_u16

```
fn write_u16(&self, port: u16, val: u16);
fn read_u32(&self, port: u16) -> u32;
fn write_u32(&self, port: u16, val: u32);
}

pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    fn read_volatile_u64(&self, offset: usize) -> u64;
    fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 48) - [internal] read_u32

```
fn read_u32(&self, port: u16) -> u32;
fn write_u32(&self, port: u16, val: u32);
}

pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    fn read_volatile_u64(&self, offset: usize) -> u64;
    fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_Workspace/core/kstd_base/src/trails.rs (line 49) - [internal] write_u32

```
fn write_u32(&self, port: u16, val: u32);
}

pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    fn read_volatile_u64(&self, offset: usize) -> u64;
    fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 52) - [public] MmioRegion

```
pub trait MmioRegion {
    fn read_volatile_u32(&self, offset: usize) -> u32;
    fn write_volatile_u32(&self, offset: usize, val: u32);
    fn read_volatile_u64(&self, offset: usize) -> u64;
    fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 53) - [internal] read_volatile_u32

```
fn read_volatile_u32(&self, offset: usize) -> u32;
fn write_volatile_u32(&self, offset: usize, val: u32);
fn read_volatile_u64(&self, offset: usize) -> u64;
fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 54) - [internal] write_volatile_u32

```
fn write_volatile_u32(&self, offset: usize, val: u32);
fn read_volatile_u64(&self, offset: usize) -> u64;
fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 55) - [internal] read_volatile_u64

```
fn read_volatile_u64(&self, offset: usize) -> u64;
fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_workspace/core/kstd_base/src/trails.rs (line 56) - [internal] write_volatile_u64

```
fn write_volatile_u64(&self, offset: usize, val: u64);
}
```

kernel_workspace/core/kstd_base/src/types.rs (line 3) - [public] PhysAddr

```
pub struct PhysAddr(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct VirtAddr(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
#[repr(transparent)]
pub struct Handle(pub u32);

impl Handle {
    pub const INVALID: Self = Self(u32::MAX);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/kstd_base/src/types.rs (line 7) - [public] VirtAddr

```
pub struct VirtAddr(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash)]
#[repr(transparent)]
pub struct Handle(pub u32);

impl Handle {
    pub const INVALID: Self = Self(u32::MAX);

    #[inline]
    pub const fn is_valid(self) -> bool {
        self.0 != Self::INVALID.0
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_base/src/types.rs (line 11) - [public] Handle

```
pub struct Handle(pub u32);

impl Handle {
    pub const INVALID: Self = Self(u32::MAX);

    #[inline]
    pub const fn is_valid(self) -> bool {
        self.0 != Self::INVALID.0
    }
}

impl Default for Handle {
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/types.rs (line 23) - [internal] default

```
fn default() -> Self {
    Self::INVALID
}

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct CapId(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct WorldId(pub u32);
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/types.rs (line 30) - [public] CapId

```
pub struct CapId(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct WorldId(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct TaskId(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_base/src/types.rs (line 34) - [public] WorldId

```
pub struct WorldId(pub u32);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct TaskId(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct Size(pub usize);

impl PhysAddr {
    #[inline]
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_base/src/types.rs (line 38) - [public] TaskId

```
pub struct TaskId(pub u64);

#[derive(Debug, Clone, Copy, PartialEq, Eq, PartialOrd, Ord, Hash, Default)]
#[repr(transparent)]
pub struct Size(pub usize);

impl PhysAddr {
    #[inline]
    pub const fn is_aligned_to(self, align: u64) -> bool {
        self.0 & (align - 1) == 0
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_base/src/types.rs (line 42) - [public] Size

```
pub struct Size(pub usize);

impl PhysAddr {
    #[inline]
    pub const fn is_aligned_to(self, align: u64) -> bool {
        self.0 & (align - 1) == 0
    }

    #[inline]
    pub const fn page_offset(self) -> u64 {
        self.0 & 0xFFF
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_core/src/panic.rs (line 5) - [internal] write_all

```
fn write_all(serial: &mut kstd_io::Serial, vga: &mut kstd_io::VgaConsole, s: &str) {
    let _ = serial.write_str(s);
    let _ = vga.write_str(s);
}

#[panic_handler]
fn panic(info: &PanicInfo) -> ! {
    let mut serial = kstd_io::Serial;
    let mut vga = kstd_io::VgaConsole;

    write_all(&mut serial, &mut vga, "\n!!! KERNEL PANIC !!!\n");

    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_core/src/panic.rs (line 11) - [internal] panic

```
fn panic(info: &PanicInfo) -> ! {
    let mut serial = kstd_io::Serial;
    let mut vga = kstd_io::VgaConsole;

    write_all(&mut serial, &mut vga, "\n!!! KERNEL PANIC !!!\n");

    if let Some(location) = info.location() {
        let _ = core::write!(FmtWriter(&mut serial), "At {}:{}:{}\n", location.file(), location.line(), 1);
        let _ = core::write!(FmtWriter(&mut vga), "At {}:{}:{}\n", location.file(), location.line(), loca
    }

    {
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/panic.rs (line 34) - [internal] FmtWriter

```
struct FmtWriter<'a, W: Write>(&'a mut W);

impl<'a, W: Write> core::fmt::Write for FmtWriter<'a, W> {
    fn write_str(&mut self, s: &str) -> core::fmt::Result {
        let _ = self.0.write_str(s);
        Ok(())
    }
}

// Render the panic message into a fixed buffer without using alloc.
// Note: kstd_io::Serial already truncates to 4096 bytes internally, we pass &str directly.
fn heapless_msg(message: core::panic::PanicMessage) -> kstd_data::string::String {
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/panic.rs (line 37) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> core::fmt::Result {
    let _ = self.0.write_str(s);
    Ok(())
}

// Render the panic message into a fixed buffer without using alloc.
// Note: kstd_io::Serial already truncates to 4096 bytes internally, we pass &str directly.
fn heapless_msg(message: core::panic::PanicMessage) -> kstd_data::string::String {
    use core::fmt::Write as _;
    let mut s = kstd_data::string::String::new();
    let _ = core::write!(FmtMsg(&mut s), "{}", message);
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/panic.rs (line 45) - [internal] heapless_msg

// Render the panic message into a fixed buffer without using alloc.

// Note: kstd_io::Serial already truncates to 4096 bytes internally, we pass &str directly.

```
fn heapless_msg(message: core::panic::PanicMessage) -> kstd_data::string::String {
    use core::fmt::Write as _;
    let mut s = kstd_data::string::String::new();
    let _ = core::write!(FmtMsg(&mut s), "{}", message);
    s
}

struct FmtMsg<'a>(&'a mut kstd_data::string::String);

impl<'a> core::fmt::Write for FmtMsg<'a> {
    fn write_str(&mut self, s: &str) -> core::fmt::Result {
        self.0.push_str(s);
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/panic.rs (line 52) - [internal] FmtMsg

```
struct FmtMsg<'a>(&'a mut kstd_data::string::String);

impl<'a> core::fmt::Write for FmtMsg<'a> {
    fn write_str(&mut self, s: &str) -> core::fmt::Result {
        self.0.push_str(s);
        Ok(())
    }
}
```

kernel_Workspace/core/kstd_core/src/panic.rs (line 55) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> core::fmt::Result {
    self.0.push_str(s);
    Ok(())
}
```

kernel_workspace/core/kstd_core/src/sync.rs (line 5) - [public] SpinLock

```
pub struct SpinLock<T> {
    locked: AtomicBool,
    data: UnsafeCell<T>,
}

unsafe impl<T: Send> Sync for SpinLock<T> {}
unsafe impl<T: Send> Send for SpinLock<T> {}

impl<T> SpinLock<T> {
    pub const fn new(data: T) -> Self {
        Self {
            locked: AtomicBool::new(false),
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/core/kstd_core/src/sync.rs (line 21) - [public] lock

```
pub fn lock(&self) -> SpinLockGuard<'_, T> {
    while self.locked.compare_exchange_weak(false, true, Ordering::Acquire, Ordering::Relaxed).is_err() {
        while self.locked.load(Ordering::Relaxed) {
            core::hint::spin_loop();
        }
    }
    SpinLockGuard { lock: self }
}

pub fn try_lock(&self) -> Option<SpinLockGuard<'_, T>> {
    if self.locked.compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed).is_ok() {
        Some(SpinLockGuard { lock: self })
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/kstd_core/src/sync.rs (line 30) - [public] try_lock

```
pub fn try_lock(&self) -> Option<SpinLockGuard<'_, T>> {
    if self.locked.compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed).is_ok() {
        Some(SpinLockGuard { lock: self })
    } else {
        None
    }
}

pub struct SpinLockGuard<'a, T> {
    lock: &'a SpinLock<T>,
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_core/src/sync.rs (line 39) - [public] SpinLockGuard

```
pub struct SpinLockGuard<'a, T> {
    lock: &'a SpinLock<T>,
}

impl<T> Deref for SpinLockGuard<'_, T> {
    type Target = T;
    fn deref(&self) -> &T {
        unsafe { &*self.lock.data.get() }
    }
}

impl<T> DerefMut for SpinLockGuard<'_, T> {
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_core/src/sync.rs (line 45) - [internal] deref

```
    fn deref(&self) -> &T {
        unsafe { &*self.lock.data.get() }
    }
}

impl<T> DerefMut for SpinLockGuard<'_, T> {
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.lock.data.get() }
    }
}

impl<T> Drop for SpinLockGuard<'_, T> {
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/sync.rs (line 51) - [internal] deref_mut

```
    fn deref_mut(&mut self) -> &mut T {
        unsafe { &mut *self.lock.data.get() }
    }
}

impl<T> Drop for SpinLockGuard<'_, T> {
    fn drop(&mut self) {
        self.lock.locked.store(false, Ordering::Release);
    }
}
```

kernel_Workspace/core/kstd_core/src/sync.rs (line 57) - [internal] drop

```
    fn drop(&mut self) {
        self.lock.locked.store(false, Ordering::Release);
    }
}
```

kernel_Workspace/core/kstd_core/src/task.rs (line 4) - [internal] sched_create_task

```
    fn sched_create_task(world_id: u32, flags: u32, out_task_id: *mut u64) -> bool;
    fn sched_destroy_task(task_id: u64) -> bool;
    fn sched_yield() -> bool;
    fn sched_get_current_task() -> u64;
}

pub struct Task {
    pub id: TaskId,
    pub world: WorldId,
}

impl Task {
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/task.rs (line 5) - [internal] sched_destroy_task

```
    fn sched_destroy_task(task_id: u64) -> bool;
    fn sched_yield() -> bool;
    fn sched_get_current_task() -> u64;
}

pub struct Task {
    pub id: TaskId,
    pub world: WorldId,
}

impl Task {
    pub fn create(world: WorldId, flags: u32) -> Result<Self, Status> {
        // ... (truncated, see source file for more)
```


kernel_Workspace/core/kstd_core/src/task.rs (line 6) - [internal] sched_yield

```
fn sched_yield() -> bool;
fn sched_get_current_task() -> u64;
}

pub struct Task {
    pub id: TaskId,
    pub world: WorldId,
}

impl Task {
    pub fn create(world: WorldId, flags: u32) -> Result<Self, Status> {
        let mut id: u64 = 0;
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_core/src/task.rs (line 7) - [internal] sched_get_current_task

```
fn sched_get_current_task() -> u64;
}

pub struct Task {
    pub id: TaskId,
    pub world: WorldId,
}

impl Task {
    pub fn create(world: WorldId, flags: u32) -> Result<Self, Status> {
        let mut id: u64 = 0;
        if unsafe { sched_create_task(world.0, flags, &mut id) } {
            Ok(Self { id: TaskId(id), world })
        } else {
            Err(Status::OutOfMemory)
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_core/src/task.rs (line 10) - [public] Task

```
pub struct Task {
    pub id: TaskId,
    pub world: WorldId,
}

impl Task {
    pub fn create(world: WorldId, flags: u32) -> Result<Self, Status> {
        let mut id: u64 = 0;
        if unsafe { sched_create_task(world.0, flags, &mut id) } {
            Ok(Self { id: TaskId(id), world })
        } else {
            Err(Status::OutOfMemory)
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_core/src/task.rs (line 16) - [public] create

```
pub fn create(world: WorldId, flags: u32) -> Result<Self, Status> {
    let mut id: u64 = 0;
    if unsafe { sched_create_task(world.0, flags, &mut id) } {
        Ok(Self { id: TaskId(id), world })
    } else {
        Err(Status::OutOfMemory)
    }
}

pub fn destroy(&self) -> Result<(), Status> {
    if unsafe { sched_destroy_task(self.id.0) } {
        Ok(())
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/kstd_core/src/task.rs (line 25) - [public] destroy

```
pub fn destroy(&self) -> Result<(), Status> {
    if unsafe { sched_destroy_task(self.id.0) } {
        Ok(())
    } else {
        Err(Status::InvalidArgument)
    }
}

pub fn current() -> TaskId {
    TaskId(unsafe { sched_get_current_task() })
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_core/src/task.rs (line 33) - [public] current

```
pub fn current() -> TaskId {
    TaskId(unsafe { sched_get_current_task() })
}

pub fn yield_now() {
    unsafe { sched_yield(); }
}
}
```

kernel_workspace/core/kstd_core/src/task.rs (line 37) - [public] yield_now

```
pub fn yield_now() {
    unsafe { sched_yield(); }
}
}
```

kernel_workspace/core/kstd_core/src/thread.rs (line 4) - [internal] thread_create

```
fn thread_create(task_id: u64, entry: u64, stack_ptr: u64, arg: u64, out_tid: *mut u64) -> bool;
fn thread_exit() -> !;
fn thread_get_current_id() -> u64;
}

pub struct Thread {
    pub id: u64,
    pub task: TaskId,
}

impl Thread {
    pub fn spawn(task: TaskId, entry: u64, stack_ptr: u64, arg: u64) -> Result<Self, Status> {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_core/src/thread.rs (line 5) - [internal] thread_exit

```
fn thread_exit() -> !;
fn thread_get_current_id() -> u64;
}

pub struct Thread {
    pub id: u64,
    pub task: TaskId,
}

impl Thread {
    pub fn spawn(task: TaskId, entry: u64, stack_ptr: u64, arg: u64) -> Result<Self, Status> {
        let mut tid: u64 = 0;
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_core/src/thread.rs (line 6) - [internal] thread_get_current_id

```
fn thread_get_current_id() -> u64;
}

pub struct Thread {
    pub id: u64,
    pub task: TaskId,
}

impl Thread {
    pub fn spawn(task: TaskId, entry: u64, stack_ptr: u64, arg: u64) -> Result<Self, Status> {
        let mut tid: u64 = 0;
        if unsafe { thread_create(task.0, entry, stack_ptr, arg, &mut tid) } {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/core/kstd_core/src/thread.rs (line 9) - [public] Thread

```
pub struct Thread {
    pub id: u64,
    pub task: TaskId,
}

impl Thread {
    pub fn spawn(task: TaskId, entry: u64, stack_ptr: u64, arg: u64) -> Result<Self, Status> {
        let mut tid: u64 = 0;
        if unsafe { thread_create(task.0, entry, stack_ptr, arg, &mut tid) } {
            Ok(Self { id: tid, task })
        } else {
            Err(Status::OutOfMemory)
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_core/src/thread.rs (line 15) - [public] spawn

```
pub fn spawn(task: TaskId, entry: u64, stack_ptr: u64, arg: u64) -> Result<Self, Status> {
    let mut tid: u64 = 0;
    if unsafe { thread_create(task.0, entry, stack_ptr, arg, &mut tid) } {
        Ok(Self { id: tid, task })
    } else {
        Err(Status::OutOfMemory)
    }
}

pub fn exit() -> ! {
    unsafe { thread_exit(); }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_core/src/thread.rs (line 24) - [public] exit

```
pub fn exit() -> ! {
    unsafe { thread_exit(); }
}

pub fn current_id() -> u64 {
    unsafe { thread_get_current_id() }
}
}
```

kernel_Workspace/core/kstd_core/src/thread.rs (line 28) - [public] current_id

```
pub fn current_id() -> u64 {
    unsafe { thread_get_current_id() }
}
}
```

kernel_Workspace/core/kstd_core/src/time.rs (line 2) - [internal] k_tick

```
fn k_tick() -> u64;
}

#[inline(always)]
pub fn kernel_tick() -> u64 {
    unsafe { k_tick() }
}

#[inline(always)]
pub fn rdtsc() -> u64 {
    #[cfg(target_arch = "x86_64")]
    unsafe {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_core/src/time.rs (line 6) - [public] kernel_tick

```
pub fn kernel_tick() -> u64 {
    unsafe { k_tick() }
}

#[inline(always)]
pub fn rdtsc() -> u64 {
    #[cfg(target_arch = "x86_64")]
    unsafe {
        core::arch::x86_64::_rdtsc()
    }
    #[cfg(not(target_arch = "x86_64"))]
    {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_core/src/time.rs (line 11) - [public] rdtsc

```
pub fn rdtsc() -> u64 {
    #[cfg(target_arch = "x86_64")]
    unsafe {
        core::arch::x86_64::_rdtsc()
    }
    #[cfg(not(target_arch = "x86_64"))]
    {
        kernel_tick()
    }
}
```

kernel_Workspace/core/kstd_data/src/list.rs (line 4) - [public] ListHead

```
pub struct ListHead {
    next: *mut ListHead,
    prev: *mut ListHead,
}

impl ListHead {
    pub const fn new() -> Self {
        Self { next: ptr::null_mut(), prev: ptr::null_mut() }
    }

    pub fn init(&mut self) {
        self.next = self as *mut _;
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/list.rs (line 14) - [public] init

```
pub fn init(&mut self) {
    self.next = self as *mut _;
    self.prev = self as *mut _;
}

pub fn add(&mut self, new_node: *mut ListHead) {
    unsafe {
        let next = self.next;
        (*new_node).next = next;
        (*new_node).prev = self as *mut _;
        (*next).prev = new_node;
        self.next = new_node;
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/list.rs (line 19) - [public] add

```
pub fn add(&mut self, new_node: *mut ListHead) {
    unsafe {
        let next = self.next;
        (*new_node).next = next;
        (*new_node).prev = self as *mut _;
        (*next).prev = new_node;
        self.next = new_node;
    }
}

pub fn add_tail(&mut self, new_node: *mut ListHead) {
    unsafe {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/list.rs (line 29) - [public] add_tail

```
pub fn add_tail(&mut self, new_node: *mut ListHead) {
    unsafe {
        let prev = self.prev;
        (*new_node).next = self as *mut _;
        (*new_node).prev = prev;
        (*prev).next = new_node;
        self.prev = new_node;
    }
}

pub fn del(&mut self) {
    unsafe {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/list.rs (line 39) - [public] del

```
pub fn del(&mut self) {
    unsafe {
        let prev = self.prev;
        let next = self.next;
        (*prev).next = next;
        (*next).prev = prev;
        self.init();
    }
}

pub fn is_empty(&self) -> bool {
    self.next as *const _ == self as *const _
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_data/src/list.rs (line 49) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.next as *const _ == self as *const _
}

#[macro_export]
macro_rules! list_entry {
    ($ptr:expr, $ty:ty, $field:ident) => {
        ($ptr as usize - $crate::offset_of!($ty, $field)) as *mut $ty
    };
}
```

kernel_Workspace/core/kstd_data/src/map.rs (line 4) - [internal] FnvHasher

```
struct FnvHasher(u64);
impl Hasher for FnvHasher {
    fn finish(&self) -> u64 { self.0 }
    fn write(&mut self, bytes: &[u8]) {
        for &b in bytes {
            self.0 ^= b as u64;
            self.0 = self.0.wrapping_mul(0x1000000001b3);
        }
    }
}

enum Bucket<K, V> { Empty, Occupied(K, V), Tombstone }
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 6) - [internal] finish

```
fn finish(&self) -> u64 { self.0 }
fn write(&mut self, bytes: &[u8]) {
    for &b in bytes {
        self.0 ^= b as u64;
        self.0 = self.0.wrapping_mul(0x1000000001b3);
    }
}

enum Bucket<K, V> { Empty, Occupied(K, V), Tombstone }

pub struct HashMap<K, V> {
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 7) - [internal] write

```
fn write(&mut self, bytes: &[u8]) {
    for &b in bytes {
        self.0 ^= b as u64;
        self.0 = self.0.wrapping_mul(0x1000000001b3);
    }
}

enum Bucket<K, V> { Empty, Occupied(K, V), Tombstone }

pub struct HashMap<K, V> {
    buckets: Vec<Bucket<K, V>>,
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 17) - [public] HashMap

```
pub struct HashMap<K, V> {
    buckets: Vec<Bucket<K, V>>,
    len: usize,
}

impl<K: Hash + Eq, V> HashMap<K, V> {
    pub fn new() -> Self {
        Self { buckets: Vec::new(), len: 0 }
    }

    fn hash_key(key: &K) -> usize {
        let mut hasher = FnvHasher(0xcbf29ce484222325);
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 23) - [public] new

```
pub fn new() -> Self {
    Self { buckets: Vec::new(), len: 0 }
}

fn hash_key(key: &K) -> usize {
    let mut hasher = FnvHasher(0xcbf29ce484222325);
    key.hash(&mut hasher);
    hasher.finish() as usize
}

fn idx(&self, hash: usize) -> usize { hash % self.buckets.len() }

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 27) - [internal] hash_key

```
fn hash_key(key: &K) -> usize {
    let mut hasher = FnvHasher(0xcbf29ce484222325);
    key.hash(&mut hasher);
    hasher.finish() as usize
}

fn idx(&self, hash: usize) -> usize { hash % self.buckets.len() }

pub fn insert(&mut self, key: K, val: V) {
    if self.buckets.is_empty() || (self.len * 4) >= (self.buckets.len() * 3) {
        self.resize(if self.buckets.is_empty() { 8 } else { self.buckets.len() * 2 });
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 33) - [internal] idx

```
fn idx(&self, hash: usize) -> usize { hash % self.buckets.len() }

pub fn insert(&mut self, key: K, val: V) {
    if self.buckets.is_empty() || (self.len * 4) >= (self.buckets.len() * 3) {
        self.resize(if self.buckets.is_empty() { 8 } else { self.buckets.len() * 2 });
    }

    let hash = Self::hash_key(&key);
    let mut idx = self.idx(hash);

    loop {
        match &self.buckets[idx] {
// ... (truncated, see source file for more)

```

kernel_Workspace/core/kstd_data/src/map.rs (line 35) - [public] insert

```
pub fn insert(&mut self, key: K, val: V) {
    if self.buckets.is_empty() || (self.len * 4) >= (self.buckets.len() * 3) {
        self.resize(if self.buckets.is_empty() { 8 } else { self.buckets.len() * 2 });
    }

    let hash = Self::hash_key(&key);
    let mut idx = self.idx(hash);

    loop {
        match &self.buckets[idx] {
            Bucket::Empty | Bucket::Tombstone => {
                self.buckets[idx] = Bucket::Occupied(key, val);
            }
// ... (truncated, see source file for more)

```

kernel_Workspace/core/kstd_data/src/map.rs (line 59) - [public] get

```
pub fn get(&self, key: &K) -> Option<&V> {
    if self.buckets.is_empty() { return None; }
    let hash = Self::hash_key(key);
    let mut idx = self.idx(hash);

    loop {
        match &self.buckets[idx] {
            Bucket::Empty => return None,
            Bucket::Occupied(k, v) if *k == *key => return Some(v),
            _ => { idx = (idx + 1) % self.buckets.len(); }
        }
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 73) - [public] remove

```
pub fn remove(&mut self, key: &K) -> Option<V> {
    if self.buckets.is_empty() { return None; }
    let hash = Self::hash_key(key);
    let mut idx = self.idx(hash);

    loop {
        match &self.buckets[idx] {
            Bucket::Empty => return None,
            Bucket::Occupied(k, _) if *k == *key => {
                let old = core::mem::replace(&mut self.buckets[idx], Bucket::Tombstone);
                self.len -= 1;
                if let Bucket::Occupied(_, v) = old { return Some(v); }
            }
        }
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 91) - [internal] resize

```
fn resize(&mut self, new_cap: usize) {
    let old_buckets = core::mem::replace(&mut self.buckets, Vec::with_capacity(new_cap));
    for _ in 0..new_cap { self.buckets.push(Bucket::Empty); }

    self.len = 0;
    for b in old_buckets.as_slice() {
        if let Bucket::Occupied(k, v) = b {
            let k = unsafe { core::ptr::read(k) };
            let v = unsafe { core::ptr::read(v) };
            self.insert(k, v);
        }
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/map.rs (line 107) - [public] len

```
pub fn len(&self) -> usize { self.len }
pub fn is_empty(&self) -> bool { self.len == 0 }
}
```

kernel_Workspace/core/kstd_data/src/map.rs (line 108) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.len == 0 }
}
```

kernel_Workspace/core/kstd_data/src/string.rs (line 4) - [public] String

```
pub struct String { vec: Vec<u8> }

impl String {
    pub const fn new() -> Self { Self { vec: Vec::new() } }

    pub fn from_str(s: &str) -> Self {
        let mut v = Vec::with_capacity(s.len());
        v.extend_from_slice(s.as_bytes());
        Self { vec: v }
    }

    pub fn push_str(&mut self, s: &str) {
        // ... (truncated, see source file for more)
    }
}
```


kernel_Workspace/core/kstd_data/src/string.rs (line 9) - [public] from_str

```
pub fn from_str(s: &str) -> Self {
    let mut v = Vec::with_capacity(s.len());
    v.extend_from_slice(s.as_bytes());
    Self { vec: v }
}

pub fn push_str(&mut self, s: &str) {
    self.vec.extend_from_slice(s.as_bytes());
}

pub fn as_str(&self) -> &str {
    unsafe { core::str::from_utf8_unchecked(self.vec.as_slice()) }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/string.rs (line 15) - [public] push_str

```
pub fn push_str(&mut self, s: &str) {
    self.vec.extend_from_slice(s.as_bytes());
}

pub fn as_str(&self) -> &str {
    unsafe { core::str::from_utf8_unchecked(self.vec.as_slice()) }
}

pub fn len(&self) -> usize { self.vec.len() }
pub fn is_empty(&self) -> bool { self.vec.is_empty() }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/string.rs (line 19) - [public] as_str

```
pub fn as_str(&self) -> &str {
    unsafe { core::str::from_utf8_unchecked(self.vec.as_slice()) }
}

pub fn len(&self) -> usize { self.vec.len() }
pub fn is_empty(&self) -> bool { self.vec.is_empty() }
}

impl core::ops::Deref for String {
    type Target = str;
    fn deref(&self) -> &str { self.as_str() }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/string.rs (line 23) - [public] len

```
pub fn len(&self) -> usize { self.vec.len() }
pub fn is_empty(&self) -> bool { self.vec.is_empty() }
}

impl core::ops::Deref for String {
    type Target = str;
    fn deref(&self) -> &str { self.as_str() }
}

impl fmt::Write for String {
    fn write_str(&mut self, s: &str) -> fmt::Result {
        self.push_str(s);
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_data/src/string.rs (line 24) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.vec.is_empty() }  
}  
  
impl core::ops::Deref for String {  
    type Target = str;  
    fn deref(&self) -> &str { self.as_str() }  
}  
  
impl fmt::Write for String {  
    fn write_str(&mut self, s: &str) -> fmt::Result {  
        self.push_str(s);  
        Ok(())  
    }  
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/string.rs (line 29) - [internal] deref

```
fn deref(&self) -> &str { self.as_str() }  
}  
  
impl fmt::Write for String {  
    fn write_str(&mut self, s: &str) -> fmt::Result {  
        self.push_str(s);  
        Ok(())  
    }  
}
```

kernel_Workspace/core/kstd_data/src/string.rs (line 33) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> fmt::Result {  
    self.push_str(s);  
    Ok(())  
}  
}
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 3) - [internal] Node

```
struct Node<K, V> {  
    key: K, val: V,  
    left: usize, right: usize,  
    height: i32,  
}  
  
pub struct TreeMap<K, V> {  
    nodes: Vec<Node<K, V>>,  
    free_list: Vec<usize>,  
    root: usize,  
}  
  
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 9) - [public] TreeMap

```
pub struct TreeMap<K, V> {  
    nodes: Vec<Node<K, V>>,  
    free_list: Vec<usize>,  
    root: usize,  
}  
  
impl<K: Ord, V> TreeMap<K, V> {  
    pub fn new() -> Self {  
        Self { nodes: Vec::new(), free_list: Vec::new(), root: 0 }  
    }  
  
    fn alloc_node(&mut self, key: K, val: V) -> usize {  
        // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 16) - [public] new

```
pub fn new() -> Self {
    Self { nodes: Vec::new(), free_list: Vec::new(), root: 0 }
}

fn alloc_node(&mut self, key: K, val: V) -> usize {
    if let Some(idx) = self.free_list.pop() {
        self.nodes[idx - 1] = Node { key, val, left: 0, right: 0, height: 1 };
        idx
    } else {
        self.nodes.push(Node { key, val, left: 0, right: 0, height: 1 });
        self.nodes.len()
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 20) - [internal] alloc_node

```
fn alloc_node(&mut self, key: K, val: V) -> usize {
    if let Some(idx) = self.free_list.pop() {
        self.nodes[idx - 1] = Node { key, val, left: 0, right: 0, height: 1 };
        idx
    } else {
        self.nodes.push(Node { key, val, left: 0, right: 0, height: 1 });
        self.nodes.len()
    }
}

fn free_node(&mut self, idx: usize) {
    self.free_list.push(idx);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 30) - [internal] free_node

```
fn free_node(&mut self, idx: usize) {
    self.free_list.push(idx);
}

fn height(&self, idx: usize) -> i32 {
    if idx == 0 { 0 } else { self.nodes[idx - 1].height }
}

fn update_height(&mut self, idx: usize) {
    let l = self.height(self.nodes[idx - 1].left);
    let r = self.height(self.nodes[idx - 1].right);
    self.nodes[idx - 1].height = 1 + if l > r { l } else { r };
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 34) - [internal] height

```
fn height(&self, idx: usize) -> i32 {
    if idx == 0 { 0 } else { self.nodes[idx - 1].height }
}

fn update_height(&mut self, idx: usize) {
    let l = self.height(self.nodes[idx - 1].left);
    let r = self.height(self.nodes[idx - 1].right);
    self.nodes[idx - 1].height = 1 + if l > r { l } else { r };
}

fn balance_factor(&self, idx: usize) -> i32 {
    if idx == 0 { 0 }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 38) - [internal] update_height

```
fn update_height(&mut self, idx: usize) {
    let l = self.height(self.nodes[idx - 1].left);
    let r = self.height(self.nodes[idx - 1].right);
    self.nodes[idx - 1].height = 1 + if l > r { l } else { r };
}

fn balance_factor(&self, idx: usize) -> i32 {
    if idx == 0 { 0 }
    else { self.height(self.nodes[idx - 1].left) - self.height(self.nodes[idx - 1].right) }
}

fn rotate_right(&mut self, y: usize) -> usize {
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 44) - [internal] balance_factor

```
fn balance_factor(&self, idx: usize) -> i32 {
    if idx == 0 { 0 }
    else { self.height(self.nodes[idx - 1].left) - self.height(self.nodes[idx - 1].right) }
}

fn rotate_right(&mut self, y: usize) -> usize {
    let x = self.nodes[y - 1].left;
    let t2 = self.nodes[x - 1].right;
    self.nodes[x - 1].right = y;
    self.nodes[y - 1].left = t2;
    self.update_height(y);
    self.update_height(x);
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 49) - [internal] rotate_right

```
fn rotate_right(&mut self, y: usize) -> usize {
    let x = self.nodes[y - 1].left;
    let t2 = self.nodes[x - 1].right;
    self.nodes[x - 1].right = y;
    self.nodes[y - 1].left = t2;
    self.update_height(y);
    self.update_height(x);
    x
}

fn rotate_left(&mut self, x: usize) -> usize {
    let y = self.nodes[x - 1].right;
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 59) - [internal] rotate_left

```
fn rotate_left(&mut self, x: usize) -> usize {
    let y = self.nodes[x - 1].right;
    let t2 = self.nodes[y - 1].left;
    self.nodes[y - 1].left = x;
    self.nodes[x - 1].right = t2;
    self.update_height(x);
    self.update_height(y);
    y
}

fn balance(&mut self, idx: usize) -> usize {
    self.update_height(idx);
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 69) - [internal] balance

```
fn balance(&mut self, idx: usize) -> usize {
    self.update_height(idx);
    let bf = self.balance_factor(idx);
    if bf > 1 {
        if self.balance_factor(self.nodes[idx - 1].left) < 0 {
            let l = self.nodes[idx - 1].left;
            self.nodes[idx - 1].left = self.rotate_left(l);
        }
        return self.rotate_right(idx);
    }
    if bf < -1 {
        if self.balance_factor(self.nodes[idx - 1].right) > 0 {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 89) - [internal] insert_rec

```
fn insert_rec(&mut self, idx: usize, key: K, val: V) -> usize {
    if idx == 0 { return self.alloc_node(key, val); }

    let i = idx - 1;
    if key < self.nodes[i].key {
        self.nodes[i].left = self.insert_rec(self.nodes[i].left, key, val);
    } else if key > self.nodes[i].key {
        self.nodes[i].right = self.insert_rec(self.nodes[i].right, key, val);
    } else {
        self.nodes[i].val = val;
        return idx;
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 104) - [public] insert

```
pub fn insert(&mut self, key: K, val: V) {
    self.root = self.insert_rec(self.root, key, val);
}

pub fn get(&self, key: &K) -> Option<&V> {
    let mut idx = self.root;
    while idx != 0 {
        let i = idx - 1;
        if *key < self.nodes[i].key { idx = self.nodes[i].left; }
        else if *key > self.nodes[i].key { idx = self.nodes[i].right; }
        else { return Some(&self.nodes[i].val); }
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 108) - [public] get

```
pub fn get(&self, key: &K) -> Option<&V> {
    let mut idx = self.root;
    while idx != 0 {
        let i = idx - 1;
        if *key < self.nodes[i].key { idx = self.nodes[i].left; }
        else if *key > self.nodes[i].key { idx = self.nodes[i].right; }
        else { return Some(&self.nodes[i].val); }
    }
    None
}

pub fn len(&self) -> usize { self.nodes.len() - self.free_list.len() }
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 119) - [public] len

```
pub fn len(&self) -> usize { self.nodes.len() - self.free_list.len() }
pub fn is_empty(&self) -> bool { self.root == 0 }
}
```

kernel_Workspace/core/kstd_data/src/tree.rs (line 120) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.root == 0 }
}
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 4) - [public] Vec

```
pub struct Vec<T> {
    ptr: NonNull<T>,
    len: usize,
    cap: usize,
}

unsafe impl<T: Send> Send for Vec<T> {}
unsafe impl<T: Sync> Sync for Vec<T> {}

impl<T> Vec<T> {
    pub const fn new() -> Self {
        Self { ptr: NonNull::dangling(), len: 0, cap: 0 }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 18) - [public] with_capacity

```
pub fn with_capacity(cap: usize) -> Self {
    let mut v = Self::new();
    if cap > 0 { v.grow_to(cap); }
    v
}

fn grow_to(&mut self, new_cap: usize) {
    let elem_size = mem::size_of::<T>();
    let align = mem::align_of::<T>();
    let new_ptr = if self.cap == 0 {
        KernelHeap.alloc(Layout::new(new_cap * elem_size, align)).unwrap() as *mut T
    } else {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 24) - [internal] grow_to

```
fn grow_to(&mut self, new_cap: usize) {
    let elem_size = mem::size_of::<T>();
    let align = mem::align_of::<T>();
    let new_ptr = if self.cap == 0 {
        KernelHeap.alloc(Layout::new(new_cap * elem_size, align)).unwrap() as *mut T
    } else {
        KernelHeap.realloc(
            self.ptr.as_ptr() as *mut u8,
            Layout::new(self.cap * elem_size, align),
            new_cap * elem_size
        ).unwrap() as *mut T
    };
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 40) - [public] push

```
pub fn push(&mut self, val: T) {
    if self.len == self.cap {
        let new_cap = if self.cap == 0 { 4 } else { self.cap * 2 };
        self.grow_to(new_cap);
    }
    unsafe { ptr::write(self.ptr.as_ptr().add(self.len), val); }
    self.len += 1;
}

pub fn pop(&mut self) -> Option<T> {
    if self.len == 0 { None }
    else {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 49) - [public] pop

```
pub fn pop(&mut self) -> Option<T> {
    if self.len == 0 { None }
    else {
        self.len -= 1;
        Some(unsafe { ptr::read(self.ptr.as_ptr().add(self.len)) })
    }
}

pub fn len(&self) -> usize { self.len }
pub fn is_empty(&self) -> bool { self.len == 0 }
pub fn capacity(&self) -> usize { self.cap }

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 57) - [public] len

```
pub fn len(&self) -> usize { self.len }
pub fn is_empty(&self) -> bool { self.len == 0 }
pub fn capacity(&self) -> usize { self.cap }

pub fn as_slice(&self) -> &[T] {
    unsafe { core::slice::from_raw_parts(self.ptr.as_ptr(), self.len) }
}

pub fn as_mut_slice(&mut self) -> &mut [T] {
    unsafe { core::slice::from_raw_parts_mut(self.ptr.as_ptr(), self.len) }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 58) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.len == 0 }
pub fn capacity(&self) -> usize { self.cap }

pub fn as_slice(&self) -> &[T] {
    unsafe { core::slice::from_raw_parts(self.ptr.as_ptr(), self.len) }
}

pub fn as_mut_slice(&mut self) -> &mut [T] {
    unsafe { core::slice::from_raw_parts_mut(self.ptr.as_ptr(), self.len) }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 59) - [public] capacity

```
pub fn capacity(&self) -> usize { self.cap }

pub fn as_slice(&self) -> &[T] {
    unsafe { core::slice::from_raw_parts(self.ptr.as_ptr(), self.len) }
}

pub fn as_mut_slice(&mut self) -> &mut [T] {
    unsafe { core::slice::from_raw_parts_mut(self.ptr.as_ptr(), self.len) }
}

impl<T: Copy> Vec<T> {
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_data/src/vec.rs (line 61) - [public] as_slice

```
pub fn as_slice(&self) -> &[T] {
    unsafe { core::slice::from_raw_parts(self.ptr.as_ptr(), self.len) }
}

pub fn as_mut_slice(&mut self) -> &mut [T] {
    unsafe { core::slice::from_raw_parts_mut(self.ptr.as_ptr(), self.len) }
}

impl<T: Copy> Vec<T> {
    pub fn extend_from_slice(&mut self, slice: &[T]) {
        let new_len = self.len + slice.len();
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_data/src/vec.rs (line 65) - [public] as_mut_slice

```
pub fn as_mut_slice(&mut self) -> &mut [T] {
    unsafe { core::slice::from_raw_parts_mut(self.ptr.as_ptr(), self.len) }
}

impl<T: Copy> Vec<T> {
    pub fn extend_from_slice(&mut self, slice: &[T]) {
        let new_len = self.len + slice.len();
        if new_len > self.cap {
            let mut new_cap = if self.cap == 0 { 4 } else { self.cap };
            while new_cap < new_len { new_cap *= 2; }
            self.grow_to(new_cap);
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_data/src/vec.rs (line 71) - [public] extend_from_slice

```
pub fn extend_from_slice(&mut self, slice: &[T]) {
    let new_len = self.len + slice.len();
    if new_len > self.cap {
        let mut new_cap = if self.cap == 0 { 4 } else { self.cap };
        while new_cap < new_len { new_cap *= 2; }
        self.grow_to(new_cap);
    }
    unsafe {
        ptr::copy_nonoverlapping(slice.as_ptr(), self.ptr.as_ptr().add(self.len), slice.len());
    }
    self.len = new_len;
}
// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_data/src/vec.rs (line 87) - [internal] deref

```
fn deref(&self) -> &[T] { self.as_slice() }

impl<T> DerefMut for Vec<T> {
    fn deref_mut(&mut self) -> &mut [T] { self.as_mut_slice() }
}

impl<T> Drop for Vec<T> {
    fn drop(&mut self) {
        if self.cap > 0 {
            unsafe {
                for i in 0..self.len { ptr::drop_in_place(self.ptr.as_ptr().add(i)); }
            }
        }
        // ... (truncated, see source file for more)
    }
}
```


kernel_Workspace/core/kstd_data/src/vec.rs (line 91) - [internal] deref_mut

```
fn deref_mut(&mut self) -> &mut [T] { self.as_mut_slice() }
}

impl<T> Drop for Vec<T> {
    fn drop(&mut self) {
        if self.cap > 0 {
            unsafe {
                for i in 0..self.len { ptr::drop_in_place(self.ptr.as_ptr().add(i)); }
                let layout = Layout::new(self.cap * mem::size_of::<T>(), mem::align_of::<T>());
                KernelHeap.dealloc(self.ptr.as_ptr() as *mut u8, layout);
            }
        }
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_data/src/vec.rs (line 95) - [internal] drop

```
fn drop(&mut self) {
    if self.cap > 0 {
        unsafe {
            for i in 0..self.len { ptr::drop_in_place(self.ptr.as_ptr().add(i)); }
            let layout = Layout::new(self.cap * mem::size_of::<T>(), mem::align_of::<T>());
            KernelHeap.dealloc(self.ptr.as_ptr() as *mut u8, layout);
        }
    }
}
}
```

kernel_Workspace/core/kstd_io/src/buffer.rs (line 1) - [public] RingBuffer

```
pub struct RingBuffer<const N: usize> {
    data: [u8; N],
    head: usize,
    tail: usize,
}

impl<const N: usize> RingBuffer<N> {
    pub const fn new() -> Self {
        Self { data: [0; N], head: 0, tail: 0 }
    }

    pub fn push(&mut self, b: u8) -> bool {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_io/src/buffer.rs (line 12) - [public] push

```
pub fn push(&mut self, b: u8) -> bool {
    let next_head = (self.head + 1) % N;
    if next_head == self.tail { return false; }
    self.data[self.head] = b;
    self.head = next_head;
    true
}

pub fn pop(&mut self) -> Option<u8> {
    if self.head == self.tail { return None; }
    let b = self.data[self.tail];
    self.tail = (self.tail + 1) % N;
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/core/kstd_io/src/buffer.rs (line 20) - [public] pop

```
pub fn pop(&mut self) -> Option<u8> {
    if self.head == self.tail { return None; }
    let b = self.data[self.tail];
    self.tail = (self.tail + 1) % N;
    Some(b)
}

pub fn is_empty(&self) -> bool { self.head == self.tail }
pub fn is_full(&self) -> bool { (self.head + 1) % N == self.tail }
pub fn clear(&mut self) { self.head = 0; self.tail = 0; }
}

// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_io/src/buffer.rs (line 27) - [public] is_empty

```
pub fn is_empty(&self) -> bool { self.head == self.tail }
pub fn is_full(&self) -> bool { (self.head + 1) % N == self.tail }
pub fn clear(&mut self) { self.head = 0; self.tail = 0; }
}

impl<const N: usize> crate::traits::Write for RingBuffer<N> {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        for b in s.as_bytes() {
            if !self.push(*b) { return Err(()); }
        }
        Ok(())
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_io/src/buffer.rs (line 28) - [public] is_full

```
pub fn is_full(&self) -> bool { (self.head + 1) % N == self.tail }
pub fn clear(&mut self) { self.head = 0; self.tail = 0; }
}

impl<const N: usize> crate::traits::Write for RingBuffer<N> {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        for b in s.as_bytes() {
            if !self.push(*b) { return Err(()); }
        }
        Ok(())
    }
}
}
```

kernel_workspace/core/kstd_io/src/buffer.rs (line 29) - [public] clear

```
pub fn clear(&mut self) { self.head = 0; self.tail = 0; }
}

impl<const N: usize> crate::traits::Write for RingBuffer<N> {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        for b in s.as_bytes() {
            if !self.push(*b) { return Err(()); }
        }
        Ok(())
    }
}
}
```

kernel_workspace/core/kstd_io/src/buffer.rs (line 33) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> Result<(), ()> {
    for b in s.as_bytes() {
        if !self.push(*b) { return Err(()); }
    }
    Ok(())
}
}
```

kernel_workspace/core/kstd_io/src/console.rs (line 4) - [internal] kprintf

```
fn kprintf(fmt: *const u8, ...);
fn serial_write_str(s: *const u8);
}

pub struct Serial;
impl Write for Serial {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        let bytes = s.as_bytes();
        if bytes.is_empty() { return Ok(()); }
        unsafe {
            let mut null_terminated = [0u8; 4096];
            let len = bytes.len().min(4095);
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/core/kstd_io/src/console.rs (line 5) - [internal] serial_write_str

```
fn serial_write_str(s: *const u8);
}

pub struct Serial;
impl Write for Serial {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        let bytes = s.as_bytes();
        if bytes.is_empty() { return Ok(()); }
        unsafe {
            let mut null_terminated = [0u8; 4096];
            let len = bytes.len().min(4095);
            core::ptr::copy_nonoverlapping(bytes.as_ptr(), null_terminated.as_mut_ptr(), len);
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/core/kstd_io/src/console.rs (line 8) - [public] Serial

```
pub struct Serial;
impl Write for Serial {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        let bytes = s.as_bytes();
        if bytes.is_empty() { return Ok(()); }
        unsafe {
            let mut null_terminated = [0u8; 4096];
            let len = bytes.len().min(4095);
            core::ptr::copy_nonoverlapping(bytes.as_ptr(), null_terminated.as_mut_ptr(), len);
            null_terminated[len] = 0;
            serial_write_str(null_terminated.as_ptr());
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/kstd_io/src/console.rs (line 10) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> Result<(), ()> {
    let bytes = s.as_bytes();
    if bytes.is_empty() { return Ok(()); }
    unsafe {
        let mut null_terminated = [0u8; 4096];
        let len = bytes.len().min(4095);
        core::ptr::copy_nonoverlapping(bytes.as_ptr(), null_terminated.as_mut_ptr(), len);
        null_terminated[len] = 0;
        serial_write_str(null_terminated.as_ptr());
    }
    Ok(())
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_io/src/console.rs (line 24) - [public] VgaConsole

```
pub struct VgaConsole;
impl Write for VgaConsole {
    fn write_str(&mut self, s: &str) -> Result<(), ()> {
        let fmt = b"%s\0".as_ptr();
        let mut null_terminated = [0u8; 4096];
        let len = s.as_bytes().len().min(4095);
        unsafe {
            core::ptr::copy_nonoverlapping(s.as_ptr(), null_terminated.as_mut_ptr(), len);
            null_terminated[len] = 0;
            kprintf(fmt, null_terminated.as_ptr());
        }
        Ok(())
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_io/src/console.rs (line 26) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> Result<(), ()> {
    let fmt = b"%s\0".as_ptr();
    let mut null_terminated = [0u8; 4096];
    let len = s.as_bytes().len().min(4095);
    unsafe {
        core::ptr::copy_nonoverlapping(s.as_ptr(), null_terminated.as_mut_ptr(), len);
        null_terminated[len] = 0;
        kprintf(fmt, null_terminated.as_ptr());
    }
    Ok(())
}
// ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_io/src/input.rs (line 2) - [internal] k_input_key

```
fn k_input_key() -> i32;
fn k_input_keycode() -> u32;
fn kbuf_pop() -> i32;

pub fn raw_scancode() -> Option<u32> {
    let code = unsafe { k_input_keycode() };
    if code == 0 { None } else { Some(code) }
}

pub fn cooked_char() -> Option<u8> {
    let key = unsafe { k_input_key() };
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_io/src/input.rs (line 3) - [internal] k_input_keycode

```
fn k_input_keycode() -> u32;
fn kbuf_pop() -> i32;

pub fn raw_scancode() -> Option<u32> {
    let code = unsafe { k_input_keycode() };
    if code == 0 { None } else { Some(code) }
}

pub fn cooked_char() -> Option<u8> {
    let key = unsafe { k_input_key() };
    if key < 0 { None } else { Some(key as u8) }
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_io/src/input.rs (line 4) - [internal] kbuf_pop

```
fn kbuf_pop() -> i32;

pub fn raw_scancode() -> Option<u32> {
    let code = unsafe { k_input_keycode() };
    if code == 0 { None } else { Some(code) }
}

pub fn cooked_char() -> Option<u8> {
    let key = unsafe { k_input_key() };
    if key < 0 { None } else { Some(key as u8) }
}
// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_io/src/input.rs (line 7) - [public] raw_scancode

```
pub fn raw_scancode() -> Option<u32> {  
    let code = unsafe { k_input_keycode() };  
    if code == 0 { None } else { Some(code) }  
}  
  
pub fn cooked_char() -> Option<u8> {  
    let key = unsafe { k_input_key() };  
    if key < 0 { None } else { Some(key as u8) }  
}  
  
pub fn buffered_byte() -> Option<u8> {  
    let b = unsafe { kbuf_pop() };  
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_io/src/input.rs (line 12) - [public] cooked_char

```
pub fn cooked_char() -> Option<u8> {  
    let key = unsafe { k_input_key() };  
    if key < 0 { None } else { Some(key as u8) }  
}  
  
pub fn buffered_byte() -> Option<u8> {  
    let b = unsafe { kbuf_pop() };  
    if b < 0 { None } else { Some(b as u8) }  
}  
  
pub struct RawInput;  
impl crate::traits::Read for RawInput {  
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_io/src/input.rs (line 17) - [public] buffered_byte

```
pub fn buffered_byte() -> Option<u8> {  
    let b = unsafe { kbuf_pop() };  
    if b < 0 { None } else { Some(b as u8) }  
}  
  
pub struct RawInput;  
impl crate::traits::Read for RawInput {  
    fn read_byte(&mut self) -> Option<u8> {  
        raw_scancode().map(|c| c as u8)  
    }  
}  
  
// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_io/src/input.rs (line 22) - [public] RawInput

```
pub struct RawInput;  
impl crate::traits::Read for RawInput {  
    fn read_byte(&mut self) -> Option<u8> {  
        raw_scancode().map(|c| c as u8)  
    }  
}  
  
pub struct CookedInput;  
impl crate::traits::Read for CookedInput {  
    fn read_byte(&mut self) -> Option<u8> {  
        cooked_char()  
    }  
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_io/src/input.rs (line 24) - [internal] read_byte

```
fn read_byte(&mut self) -> Option<u8> {
    raw_scancode().map(|c| c as u8)
}

pub struct CookedInput;
impl crate::traits::Read for CookedInput {
    fn read_byte(&mut self) -> Option<u8> {
        cooked_char()
    }
}
```

kernel_workspace/core/kstd_io/src/input.rs (line 29) - [public] CookedInput

```
pub struct CookedInput;
impl crate::traits::Read for CookedInput {
    fn read_byte(&mut self) -> Option<u8> {
        cooked_char()
    }
}
```

kernel_workspace/core/kstd_io/src/input.rs (line 31) - [internal] read_byte

```
fn read_byte(&mut self) -> Option<u8> {
    cooked_char()
}
}
```

kernel_workspace/core/kstd_io/src/traits.rs (line 1) - [public] Write

```
pub trait Write {
    fn write_str(&mut self, s: &str) -> Result<(), ()>;

    fn write_u64(&mut self, mut val: u64, base: u8) -> Result<(), ()> {
        let mut buf = [0u8; 64];
        let mut i = 64;
        if val == 0 {
            i -= 1;
            buf[i] = b'0';
        } else {
            while val > 0 {
                i -= 1;
            }
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_io/src/traits.rs (line 2) - [internal] write_str

```
fn write_str(&mut self, s: &str) -> Result<(), ()>;

fn write_u64(&mut self, mut val: u64, base: u8) -> Result<(), ()> {
    let mut buf = [0u8; 64];
    let mut i = 64;
    if val == 0 {
        i -= 1;
        buf[i] = b'0';
    } else {
        while val > 0 {
            i -= 1;
            let rem = (val % base as u64) as u8;
            buf[i] = if rem < 10 { b'0' + rem } else { b'a' + rem - 10 };
            val /= base as u64;
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_io/src/traits.rs (line 4) - [internal] write_u64

```
fn write_u64(&mut self, mut val: u64, base: u8) -> Result<(), ()> {
    let mut buf = [0u8; 64];
    let mut i = 64;
    if val == 0 {
        i -= 1;
        buf[i] = b'0';
    } else {
        while val > 0 {
            i -= 1;
            let rem = (val % base as u64) as u8;
            buf[i] = if rem < 10 { b'0' + rem } else { b'a' + rem - 10 };
            val /= base as u64;
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/core/kstd_io/src/traits.rs (line 22) - [public] Read

```
pub trait Read {  
    fn read_byte(&mut self) -> Option<u8>;  
}
```

kernel_workspace/core/kstd_io/src/traits.rs (line 23) - [internal] read_byte

```
fn read_byte(&mut self) -> Option<u8>;  
}
```

kernel_workspace/core/kstd_os_workspace/src/lib.rs (line 14) - [public] PartitionKind

```
pub enum PartitionKind {  
    KernelCore    = 0,  
    DriverSpace   = 1,  
    UserSpace     = 3,  
    LinuxCompat   = 4,  
    WindowsCompat = 5,  
}  
  
#[derive(Debug, Clone, Copy, PartialEq, Eq)]  
#[repr(u8)]  
pub enum AbiFlavor {  
    NativeTrangorge,  
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_os_workspace/src/lib.rs (line 24) - [public] AbiFlavor

```
pub enum AbiFlavor {  
    NativeTrangorge,  
    PosixLinux,  
    Win32Nt,  
    DriverSpaceRpc,  
}  
  
#[derive(Debug, Clone, Copy)]  
pub struct SandboxRules {  
    pub virt_start: VirtAddr,  
    pub virt_end: VirtAddr,  
    pub phys_base: PhysAddr,  
    // ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_os_workspace/src/lib.rs (line 32) - [public] SandboxRules

```
pub struct SandboxRules {  
    pub virt_start: VirtAddr,  
    pub virt_end: VirtAddr,  
    pub phys_base: PhysAddr,  
    pub mem_size: Size,  
    pub max_ipc_ports: u32,  
    pub max_open_handles: u32,  
    pub allowed_irq_mask: u64,  
    pub can_access_hw_direct: bool,  
    pub can_map_arbitrary_phys: bool,  
}  
  
// ... (truncated, see source file for more)
```

kernel_workspace/core/kstd_os_workspace/src/lib.rs (line 53) - [public] OsWorkspace

// Represents an isolated execution environment.

// Binds a World/Process to its memory sandbox, capabilities, and ABI translation hooks.

```
pub struct OsWorkspace {  
    pub id: WorldId,  
    pub kind: PartitionKind,  
    pub abi: AbiFlavor,  
    pub asid: u16,  
    pub cr3_phys: u64,  
    pub rules: SandboxRules,  
    pub syscall_entry: u64,  
    pub abi_translate_hook: u64,  
}  
  
impl OsWorkspace {  
    // ... (truncated, see source file for more)
```

kernel_Workspace/core/kstd_os_workspace/src/lib.rs (line 81) - [public] check_memory_access

```
pub fn check_memory_access(&self, addr: VirtAddr, size: usize) -> bool {
    let end = addr.0.saturating_add(size as u64);
    end <= self.rules.virt_end.0 && addr.0 >= self.rules.virt_start.0
}
```

kernel_Workspace/core/linux_abi/src/caps.rs (line 15) - [public] requires_native_cap

```
pub fn requires_native_cap(linux_cap: u32) -> Option<kstd_base::CapId> {
    match linux_cap {
        CAP_SYS_ADMIN | CAP_SYS_MODULE => Some(kstd_base::CapId(0x01)), // Native Root/Kernel
        CAP_NET_RAW | CAP_NET_BIND_SERVICE => Some(kstd_base::CapId(0x10)), // Native Net
        CAP_SYS_RAWIO => Some(kstd_base::CapId(0x20)), // Native HW/IO
        _ => None, // Mapped via standard user sandbox rules
    }
}
```

kernel_Workspace/core/linux_abi/src/errcodes.rs (line 25) - [public] status_to_errno

```
pub fn status_to_errno(s: Status) -> u32 {
    match s {
        Status::Success => 0,
        Status::OutOfMemory => ENOMEM,
        Status::InvalidArgument => EINVAL,
        Status::PermissionDenied => EACCES,
        Status::NotFound => ENOENT,
        Status::AlreadyExists => EEXIST,
        Status::Busy => EBUSY,
        Status::TimedOut => EAGAIN,
        Status::NotSupported => ENOSYS,
        Status::IOError => EIO,
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/linux_abi/src/errcodes.rs (line 44) - [public] errno_to_status

```
pub fn errno_to_status(e: u32) -> Status {
    match e {
        0 => Status::Success,
        ENOMEM => Status::OutOfMemory,
        EINVAL => Status::InvalidArgument,
        EACCES | EPERM => Status::PermissionDenied,
        ENOENT => Status::NotFound,
        EEXIST => Status::AlreadyExists,
        EBUSY => Status::Busy,
        EAGAIN => Status::TimedOut,
        ENOSYS => Status::NotSupported,
        EIO => Status::IOError,
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/core/linux_abi/src/ipc.rs (line 4) - [public] IpcPerm

```
pub struct IpcPerm {
    pub key: u32,
    pub uid: u32,
    pub gid: u32,
    pub cuid: u32,
    pub cgid: u32,
    pub mode: u16,
    pub __seq: u16,
}
```

```
#[repr(C)]
#[derive(Debug, Clone, Copy)]
// ... (truncated, see source file for more)
```


kernel_workspace/core/linux_abi/src/ipc.rs (line 16) - [public] ShmidDs

```
pub struct ShmidDs {
    pub shm_perm: IpcPerm,
    pub shm_segsz: usize,
    pub shm_atime: u64,
    pub shm_dtime: u64,
    pub shm_ctime: u64,
    pub shm_cpid: u32,
    pub shm_lpid: u32,
    pub shm_nattch: u64,
}
```

kernel_workspace/core/linux_abi/src/sysnums.rs (line 25) - [public] map_syscall_to_native_op

```
pub fn map_syscall_to_native_op(sys_num: u64) -> Option<u32> {
    match sys_num {
        SYS_READ | SYS_WRITE => Some(0x1001), // Native FS/IO opcode
        SYS_MMAP | SYS_MUNMAP | SYS_BRK => Some(0x2001), // Native MM opcode
        SYS_GETPID => Some(0x3001), // Native Proc opcode
        SYS_EXIT | SYS_EXIT_GROUP => Some(0x3002),
        SYS_SOCKET | SYS_CONNECT => Some(0x4001), // Native Net opcode
        _ => None, // Unhandled, requires full POSIX shim
    }
}
```

kernel_workspace/core/linux_abi/src/uctx.rs (line 4) - [public] UserContext

```
pub struct UserContext {
    pub r15: u64, pub r14: u64, pub r13: u64, pub r12: u64,
    pub rbp: u64, pub rbx: u64,
    pub r11: u64, pub r10: u64, pub r9: u64, pub r8: u64,
    pub rax: u64, pub rcx: u64, pub rdx: u64, pub rsi: u64,
    pub rdi: u64,
    pub orig_rax: u64,
    pub rip: u64,
    pub cs: u64,
    pub eflags: u64,
    pub rsp: u64,
    pub ss: u64,
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/linux_abi/src/uctx.rs (line 21) - [public] syscall_args

```
pub fn syscall_args(&self) -> (u64, u64, u64, u64, u64, u64) {
    (self.rdi, self.rsi, self.rdx, self.r10, self.r8, self.r9)
}

// Set syscall return value
#[inline]
pub fn set_return(&mut self, val: u64) {
    self.rax = val;
}
}
```

kernel_workspace/core/linux_abi/src/uctx.rs (line 27) - [public] set_return

```
pub fn set_return(&mut self, val: u64) {
    self.rax = val;
}
}
```

kernel_workspace/core/linux_abi_com/src/msgpass.rs (line 5) - [public] LinuxMsgBuf

```
pub struct LinuxMsgBuf {
    pub mtype: u64,
    // Followed by variable length payload in actual usage
}

// Native equivalent of msgsnd/msgrcv via TrangoOS Ports
pub struct PosixMqShim {
    pub native_port: Handle,
}

impl PosixMqShim {
    // Conceptual mapping: Linux mq -> Native Port Send
    // ... (truncated, see source file for more)
}
```

kernel_workspace/core/linux_abi_com/src/msgpass.rs (line 11) - [public] PosixMqShim

// Native equivalent of msgsnd/msgrcv via TrangoOS Ports

```
pub struct PosixMqShim {
    pub native_port: Handle,
}

impl PosixMqShim {
    // Conceptual mapping: Linux mq -> Native Port Send
    pub fn send_msg(&self, _mtype: u64, _payload: &[u8]) -> Result<(), Status> {
        // TODO: Route to native IPC port using kstd_os_workspace rules
        Err(Status::NotSupported)
    }

    // Conceptual mapping: Linux mq -> Native Port Recv
    // ... (truncated, see source file for more)
```

kernel_workspace/core/linux_abi_com/src/msgpass.rs (line 17) - [public] send_msg

// Conceptual mapping: Linux mq -> Native Port Send

```
pub fn send_msg(&self, _mtype: u64, _payload: &[u8]) -> Result<(), Status> {
    // TODO: Route to native IPC port using kstd_os_workspace rules
    Err(Status::NotSupported)
}

// Conceptual mapping: Linux mq -> Native Port Recv
pub fn recv_msg(&self, _out: &mut [u8]) -> Result<u64, Status> {
    // TODO: Block/Wait on native IPC port
    Err(Status::NotSupported)
}
}
```

kernel_workspace/core/linux_abi_com/src/msgpass.rs (line 23) - [public] recv_msg

// Conceptual mapping: Linux mq -> Native Port Recv

```
pub fn recv_msg(&self, _out: &mut [u8]) -> Result<u64, Status> {
    // TODO: Block/Wait on native IPC port
    Err(Status::NotSupported)
}
}
```

kernel_workspace/core/linux_abi_com/src/sockets.rs (line 15) - [public] SockAddrIn

```
pub struct SockAddrIn {
    pub sin_family: u16,
    pub sin_port: u16, // Network byte order
    pub sin_addr: Ipv4Addr,
    pub sin_zero: [u8; 8],
}

#[repr(C)]
#[derive(Debug, Clone, Copy)]
pub struct SockAddrIn6 {
    pub sin6_family: u16,
    pub sin6_port: u16,
    // ... (truncated, see source file for more)
```

kernel_workspace/core/linux_abi_com/src/sockets.rs (line 24) - [public] SockAddrIn6

```
pub struct SockAddrIn6 {
    pub sin6_family: u16,
    pub sin6_port: u16,
    pub sin6_flowinfo: u32,
    pub sin6_addr: Ipv6Addr,
    pub sin6_scope_id: u32,
}

// Translate Linux socket type to native TrangoOS IPC/Net channel kind
#[inline]
pub fn map_socket_type(family: u16, sock_type: u16) -> Option<u8> {
    match (family, sock_type) {
        // ... (truncated, see source file for more)
```

kernel_workspace/core/linux_abi_com/src/sockets.rs (line 34) - [public] map_socket_type

```
pub fn map_socket_type(family: u16, sock_type: u16) -> Option<u8> {
    match (family, sock_type) {
        (AF_INET, SOCK_STREAM) => Some(0x01), // Native TCP Channel
        (AF_INET, SOCK_DGRAM)  => Some(0x02), // Native UDP Port
        (AF_UNIX, SOCK_STREAM) => Some(0x10), // Native Local Pipe/Channel
        _                      => None,
    }
}
```

kernel_workspace/core/linux_abi_driverspace/src/devmodel.rs (line 4) - [public] LinuxDevice

// Linux Device Model abstraction (maps to native DriverSpace registry)

```
pub struct LinuxDevice {
    pub name: [u8; 32],
    pub bus_id: [u8; 32],
    pub driver_data: u64,
}

pub trait LinuxDriver {
    fn probe(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
    fn remove(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
}

// Map Linux driver registration to native DriverSpace IPC handshake
// ... (truncated, see source file for more)
```

kernel_workspace/core/linux_abi_driverspace/src/devmodel.rs (line 10) - [public] LinuxDriver

```
pub trait LinuxDriver {
    fn probe(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
    fn remove(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
}

// Map Linux driver registration to native DriverSpace IPC handshake
#[inline]
pub fn register_linux_driver(_drv: &dyn LinuxDriver) -> Result<(), Status> {
    // TODO: Send DevAttachPayload via ds-ipc to ds-manager
    Err(Status::NotSupported)
}
```

kernel_workspace/core/linux_abi_driverspace/src/devmodel.rs (line 11) - [internal] probe

```
fn probe(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
fn remove(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
}

// Map Linux driver registration to native DriverSpace IPC handshake
#[inline]
pub fn register_linux_driver(_drv: &dyn LinuxDriver) -> Result<(), Status> {
    // TODO: Send DevAttachPayload via ds-ipc to ds-manager
    Err(Status::NotSupported)
}
```

kernel_workspace/core/linux_abi_driverspace/src/devmodel.rs (line 12) - [internal] remove

```
fn remove(&mut self, dev: &LinuxDevice) -> Result<(), Status>;
}

// Map Linux driver registration to native DriverSpace IPC handshake
#[inline]
pub fn register_linux_driver(_drv: &dyn LinuxDriver) -> Result<(), Status> {
    // TODO: Send DevAttachPayload via ds-ipc to ds-manager
    Err(Status::NotSupported)
}
```

kernel_workspace/core/linux_abi_driverspace/src/devmodel.rs (line 17) - [public] register_linux_driver

```
pub fn register_linux_driver(_drv: &dyn LinuxDriver) -> Result<(), Status> {
    // TODO: Send DevAttachPayload via ds-ipc to ds-manager
    Err(Status::NotSupported)
}
```

kernel_workspace/core/linux_abi_driverspace/src/dma.rs (line 4) - [public] DmaDirection

```
pub enum DmaDirection {
    ToDevice = 1,
    FromDevice = 2,
    Bidirectional = 0,
}

// Linux DMA API shim.
// Under the hood, this must request physical grants from the kernel via DriverSpace IPC.
pub struct DmaMapping {
    pub virt: VirtAddr,
    pub phys: PhysAddr,
    pub size: Size,
    // ... (truncated, see source file for more)
```

kernel_workspace/core/linux_abi_driverspace/src/dma.rs (line 12) - [public] DmaMapping

// Linux DMA API shim.

// Under the hood, this must request physical grants from the kernel via DriverSpace IPC.

```
pub struct DmaMapping {
    pub virt: VirtAddr,
    pub phys: PhysAddr,
    pub size: Size,
}

impl DmaMapping {
    // Equivalent to dma_map_single
    pub fn map_single(_cpu_addr: VirtAddr, _size: Size, _dir: DmaDirection) -> Result<PhysAddr, Status> {
        // TODO: Call native DMA grant IPC
        Err(Status::NotSupported)
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/core/linux_abi_driverspace/src/dma.rs (line 20) - [public] map_single

// Equivalent to dma_map_single

```
pub fn map_single(_cpu_addr: VirtAddr, _size: Size, _dir: DmaDirection) -> Result<PhysAddr, Status> {
    // TODO: Call native DMA grant IPC
    Err(Status::NotSupported)
}

// Equivalent to dma_unmap_single
pub fn unmap_single(_phys: PhysAddr, _size: Size, _dir: DmaDirection) {
    // TODO: Revoke DMA grant IPC
}
}
```

kernel_workspace/core/linux_abi_driverspace/src/dma.rs (line 26) - [public] unmap_single

// Equivalent to dma_unmap_single

```
pub fn unmap_single(_phys: PhysAddr, _size: Size, _dir: DmaDirection) {
    // TODO: Revoke DMA grant IPC
}
}
```

kernel_workspace/core/windows-com/src/lib.rs (line 2) - [public] add

```
pub fn add(left: u64, right: u64) -> u64 {
    left + right
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn it_works() {
        let result = add(2, 2);
        assert_eq!(result, 4);
        // ... (truncated, see source file for more)
```

kernel_workspace/core/windows-com/src/lib.rs (line 11) - [internal] it_works

```
fn it_works() {
    let result = add(2, 2);
    assert_eq!(result, 4);
}
```

kernel_workspace/kernel/build.rs (line 6) - [internal] collect_c_files

```
fn collect_c_files(dir: &Path, out: &mut Vec<PathBuf>) {
    if let Ok(entries) = fs::read_dir(dir) {
        for entry in entries.flatten() {
            let path = entry.path();
            if path.is_dir() {
                let name = path.file_name().and_then(|n| n.to_str());
                if matches!(name, Some("linuxcom") | Some("wincom") | Some("aarch64") | Some("arm64") | S
                    continue;
            }
            collect_c_files(&path, out);
        } else if matches!(path.extension().and_then(|e| e.to_str()), Some("c") | Some("s")) {
            out.push(path);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/build.rs (line 23) - [internal] clang_target

```
fn clang_target(arch: &str) -> &'static str {
    match arch {
        "x86_64" => "x86_64-unknown-none-elf",
        "riscv64" => "riscv64gc-unknown-none-elf",
        other => panic!(
            "cannot build the C bridge for unsupported target architecture: {other}"
        ),
    }
}

fn main() {
    let manifest_dir = env::var("CARGO_MANIFEST_DIR").expect("CARGO_MANIFEST_DIR");
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/build.rs (line 33) - [internal] main

```
fn main() {
    let manifest_dir = env::var("CARGO_MANIFEST_DIR").expect("CARGO_MANIFEST_DIR");
    let out_dir = env::var("OUT_DIR").expect("OUT_DIR");
    let src_dir = Path::new(&manifest_dir).join("src");

    println!("cargo:rerun-if-changed=src");

    let arch = env::var("CARGO_CFG_TARGET_ARCH").unwrap_or_else(|_| "x86_64".to_string());

    match arch.as_str() {
        "x86_64" => {
            let llvm_target = clang_target("x86_64");
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/mod.rs (line 13) - [public] init

```
pub fn init() {
    imp::init();
}

pub fn hlt_loop() -> ! {
    imp::hlt_loop()
}

pub fn now() -> u64 {
    imp::now()
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/mod.rs (line 18) - [public] hlt_loop

```
pub fn hlt_loop() -> ! {
    imp::hlt_loop()
}

pub fn now() -> u64 {
    imp::now()
}

pub fn current_cpu() -> usize {
    imp::current_cpu()
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/mod.rs (line 23) - [public] now

```
pub fn now() -> u64 {
    imp::now()
}

pub fn current_cpu() -> usize {
    imp::current_cpu()
}
```

kernel_workspace/kernel/src/arch/mod.rs (line 28) - [public] current_cpu

```
pub fn current_cpu() -> usize {
    imp::current_cpu()
}
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 9) - [internal] HeapSpace

```
struct HeapSpace([u8; HEAP_SIZE]);

static HEAP_MEM: HeapSpace = HeapSpace([0; HEAP_SIZE]);

struct Bump {
    next: usize,
    end: usize,
}

static BUMP: Mutex<Bump> = Mutex::new(Bump { next: 0, end: 0 });

pub fn heap_init() {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 13) - [internal] Bump

```
struct Bump {
    next: usize,
    end: usize,
}

static BUMP: Mutex<Bump> = Mutex::new(Bump { next: 0, end: 0 });

pub fn heap_init() {
    let base = &HEAP_MEM as *const HeapSpace as usize;
    let mut b = BUMP.lock();
    b.next = base;
    b.end = base + HEAP_SIZE;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 20) - [public] heap_init

```
pub fn heap_init() {
    let base = &HEAP_MEM as *const HeapSpace as usize;
    let mut b = BUMP.lock();
    b.next = base;
    b.end = base + HEAP_SIZE;
}

pub struct KernelHeap;

unsafe impl GlobalAlloc for KernelHeap {
    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        let mut b = BUMP.lock();
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 27) - [public] KernelHeap

```
pub struct KernelHeap;

unsafe impl GlobalAlloc for KernelHeap {
    unsafe fn alloc(&self, layout: Layout) -> *mut u8 {
        let mut b = BUMP.lock();
        let align = layout.align().max(core::mem::size_of::<usize>());
        let cur = (b.next + align - 1) & !(align - 1);
        match cur.checked_add(layout.size()) {
            Some(new_next) if new_next <= b.end => {
                b.next = new_next;
                cur as *mut u8
            }
            _ => {
                // ... (truncated, see source file for more)
            }
        }
    }
}
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 51) - [public] now

```
pub fn now() -> u64 {
    let t: u64;
    unsafe {
        core::arch::asm!("rdtime {0}", out(reg) t, options(nomem, nostack));
    }
    t
}

#[panic_handler]
fn riscv_panic(info: &PanicInfo) -> ! {
    crate::serial::write_str("\n[PANIC] ");
    crate::serial::print_args(format_args!("{}", info));
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 60) - [internal] riscv_panic

```
fn riscv_panic(info: &PanicInfo) -> ! {
    crate::serial::write_str("\n[PANIC] ");
    crate::serial::print_args(format_args!("{}", info));
    crate::hlt_loop()
}

pub fn init() {
    early_uart("[C]");
    heap_init();
    early_uart("[D]");
    crate::serial::init();
    early_uart("[E]");
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 66) - [public] init

```
pub fn init() {
    early_uart("[C]");
    heap_init();
    early_uart("[D]");
    crate::serial::init();
    early_uart("[E]");
}

pub fn current_cpu() -> usize {
    0
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 74) - [public] current_cpu

```
pub fn current_cpu() -> usize {
    0
}

pub fn hlt_loop() -> ! {
    loop {
        unsafe {
            core::arch::asm!("wfi", options(nomem, nostack, preserves_flags));
        }
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 78) - [public] hlt_loop

```
pub fn hlt_loop() -> ! {
    loop {
        unsafe {
            core::arch::asm!("wfi", options(nomem, nostack, preserves_flags));
        }
    }
}

pub fn early_uart(s: &str) {
    const THR: *mut u8 = 0x1000_0000 as *mut u8;
    const LSR: *const u8 = 0x1000_0005 as *const u8;
    for b in s.bytes() {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/arch/riscv64.rs (line 86) - [public] early_uart

```
pub fn early_uart(s: &str) {
    const THR: *mut u8 = 0x1000_0000 as *mut u8;
    const LSR: *const u8 = 0x1000_0005 as *const u8;
    for b in s.bytes() {
        unsafe {
            while LSR.read_volatile() & 0x20 == 0 {}
            THR.write_volatile(b);
        }
    }
}

// ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/arch/x86_64.rs (line 6) - [internal] x86_panic

```
fn x86_panic(info: &PanicInfo) -> ! {
    crate::gfx::panic_screen::show(info)
}

pub fn init() {
    crate::serial::init();
    crate::gdt::init();
    crate::interrupts::init_idt();
    unsafe { crate::interrupts::PICS.lock().initialize() };
    x86_64::instructions::interrupts::enable();
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/x86_64.rs (line 10) - [public] init

```
pub fn init() {
    crate::serial::init();
    crate::gdt::init();
    crate::interrupts::init_idt();
    unsafe { crate::interrupts::PICS.lock().initialize() };
    x86_64::instructions::interrupts::enable();
}

pub fn hlt_loop() -> ! {
    loop {
        x86_64::instructions::hlt();
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/x86_64.rs (line 18) - [public] hlt_loop

```
pub fn hlt_loop() -> ! {
    loop {
        x86_64::instructions::hlt();
    }
}

pub fn current_cpu() -> usize {
    #[cfg(target_arch = "x86_64")]
    unsafe {
        use core::arch::x86_64::__cpuid;
        (__cpuid(1).ebx >> 24) as usize
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/x86_64.rs (line 24) - [public] current_cpu

```
pub fn current_cpu() -> usize {
    #[cfg(target_arch = "x86_64")]
    unsafe {
        use core::arch::x86_64::__cpuid;
        (__cpuid(1).ebx >> 24) as usize
    }
    #[cfg(not(target_arch = "x86_64"))]
    {
        0
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/arch/x86_64.rs (line 36) - [public] now

```
pub fn now() -> u64 {
    unsafe { core::arch::x86_64::_rdtsc() }
}

#[cfg(all(target_arch = "x86_64", not(test)))]
entry_point!(x86_boot);

#[cfg(all(target_arch = "x86_64", not(test)))]
fn x86_boot(boot_info: &'static BootInfo) -> ! {
    crate::kernel_main(boot_info)
}
```

kernel_workspace/kernel/src/arch/x86_64.rs (line 44) - [internal] x86_boot

```
fn x86_boot(boot_info: &'static BootInfo) -> ! {
    crate::kernel_main(boot_info)
}
```

kernel_workspace/kernel/src/audio/jack.rs (line 6) - [public] init

```
pub fn init(base: u32) -> bool {
    hal::init(base)
}

pub fn poll_jack() -> bool {
    let now = hal::jack_present();

    unsafe {
        if now != PRESENT {
            PRESENT = now;
            return true;
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/audio/jack.rs (line 10) - [public] poll_jack

```
pub fn poll_jack() -> bool {
    let now = hal::jack_present();

    unsafe {
        if now != PRESENT {
            PRESENT = now;
            return true;
        }
    }

    false
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/audio/jack.rs (line 23) - [public] query

```
pub fn query() -> u32 {
    unsafe {
        (PRESENT as u32) | ((AMP_ON as u32) << 1)
    }
}

pub fn set_amp(on: bool) {
    unsafe { AMP_ON = on; }
    hal::set_amp(on);
}

pub fn play_phys(phys: u64, len: u32) -> bool {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/audio/jack.rs (line 29) - [public] set_amp

```
pub fn set_amp(on: bool) {
    unsafe { AMP_ON = on; }
    hal::set_amp(on);
}

pub fn play_phys(phys: u64, len: u32) -> bool {
    hal::play_phys(phys, len)
}

pub fn stop() {
    hal::stop();
}
```

kernel_workspace/kernel/src/audio/jack.rs (line 34) - [public] play_phys

```
pub fn play_phys(phys: u64, len: u32) -> bool {
    hal::play_phys(phys, len)
}

pub fn stop() {
    hal::stop();
}
```

kernel_workspace/kernel/src/audio/jack.rs (line 38) - [public] stop

```
pub fn stop() {
    hal::stop();
}
```

kernel_workspace/kernel/src/battery/aut.rs (line 4) - [public] authorize

```
pub fn authorize(ring: u8, op: u8) -> bool {
    if op == BATT_THRESH as u8 && ring >= 3 {
        return false;
    }

    true
}
```

kernel_workspace/kernel/src/battery/bridge.rs (line 5) - [internal] battery_status_packed

```
fn battery_status_packed(a0: *mut u64, a1: *mut u64, a2: *mut u64) -> bool;
fn battery_set_threshold(pct: u32) -> bool;
}

pub fn batt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op as u8) {
        return -1;
    }

    match op {
        x if x == aut::BATT_STATUS => {
            let mut a0 = 0u64;
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/battery/bridge.rs (line 6) - [internal] battery_set_threshold

```
fn battery_set_threshold(pct: u32) -> bool;
}

pub fn batt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op as u8) {
        return -1;
    }

    match op {
        x if x == aut::BATT_STATUS => {
            let mut a0 = 0u64;
            let mut a1 = 0u64;
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/battery/bridge.rs (line 9) - [public] batt_call

```
pub fn batt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op as u8) {
        return -1;
    }

    match op {
        x if x == aut::BATT_STATUS => {
            let mut a0 = 0u64;
            let mut a1 = 0u64;
            let mut a2 = 0u64;

            let ok = unsafe { battery_status_packed(&mut a0, &mut a1, &mut a2) };
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/battery/init.rs (line 2) - [internal] battery_init

```
fn battery_init() -> bool;
}

pub fn init() -> bool {
    unsafe { battery_init() }
}
```

kernel_workspace/kernel/src/battery/init.rs (line 5) - [public] init

```
pub fn init() -> bool {
    unsafe { battery_init() }
}
```

kernel_workspace/kernel/src/bluetooth/aut.rs (line 8) - [public] authorize

```
pub fn authorize(ring: u8, op: u32) -> bool {
    if op == BT_INIT {
        return ring == 0;
    }

    if ring == 0 {
        return true;
    }

    if op == BT_CMD || op == BT_ACL_OUT {
        return ring < 3;
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/bluetooth/bridge.rs (line 7) - [internal] bt_init

```
fn bt_init() -> bool;
fn bt_ready() -> bool;
fn bt_info(ver: *mut u8, bdaddr: *mut u8);
fn bt_hci_cmd(opcode: u16, params: *const u8, len: u8) -> bool;
fn bt_event_poll(buf: *mut u8, cap: u8, len: *mut u8) -> bool;
fn bt_acl_send(data: *const u8, len: u16) -> bool;
fn bt_acl_recv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
}

const HCI_EVENT_MAX: usize = 64;
const ACL_MAX: usize = 256;
const HCI_PARAM_MAX: usize = 64;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 8) - [internal] bt_ready

```
fn bt_ready() -> bool;
fn bt_info(ver: *mut u8, bdaddr: *mut u8);
fn bt_hci_cmd(opcode: u16, params: *const u8, len: u8) -> bool;
fn bt_event_poll(buf: *mut u8, cap: u8, len: *mut u8) -> bool;
fn bt_acl_send(data: *const u8, len: u16) -> bool;
fn bt_acl_recv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
}

const HCI_EVENT_MAX: usize = 64;
const ACL_MAX: usize = 256;
const HCI_PARAM_MAX: usize = 64;

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 9) - [internal] bt_info

```
fn bt_info(ver: *mut u8, bdaddr: *mut u8);
fn bt_hci_cmd(opcode: u16, params: *const u8, len: u8) -> bool;
fn bt_event_poll(buf: *mut u8, cap: u8, len: *mut u8) -> bool;
fn bt_acl_send(data: *const u8, len: u16) -> bool;
fn bt_acl_recv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
}

const HCI_EVENT_MAX: usize = 64;
const ACL_MAX: usize = 256;
const HCI_PARAM_MAX: usize = 64;
```

```
pub fn bt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 10) - [internal] bt_hci_cmd

```
fn bt_hci_cmd(opcode: u16, params: *const u8, len: u8) -> bool;
fn bt_event_poll(buf: *mut u8, cap: u8, len: *mut u8) -> bool;
fn bt_acl_send(data: *const u8, len: u16) -> bool;
fn bt_acl_recv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
}

const HCI_EVENT_MAX: usize = 64;
const ACL_MAX: usize = 256;
const HCI_PARAM_MAX: usize = 64;

pub fn bt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 11) - [internal] bt_event_poll

```
fn bt_event_poll(buf: *mut u8, cap: u8, len: *mut u8) -> bool;
fn bt_acl_send(data: *const u8, len: u16) -> bool;
fn bt_acl_recv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
}

const HCI_EVENT_MAX: usize = 64;
const ACL_MAX: usize = 256;
const HCI_PARAM_MAX: usize = 64;

pub fn bt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op) {
        return -1;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 12) - [internal] bt_acl_send

```
fn bt_acl_send(data: *const u8, len: u16) -> bool;
fn bt_acl_rcv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
}

const HCI_EVENT_MAX: usize = 64;
const ACL_MAX: usize = 256;
const HCI_PARAM_MAX: usize = 64;

pub fn bt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op) {
        return -1;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 13) - [internal] bt_acl_rcv

```
fn bt_acl_rcv(data: *mut u8, cap: u16, len: *mut u16) -> bool;
}

const HCI_EVENT_MAX: usize = 64;
const ACL_MAX: usize = 256;
const HCI_PARAM_MAX: usize = 64;

pub fn bt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op) {
        return -1;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/bridge.rs (line 20) - [public] bt_call

```
pub fn bt_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op) {
        return -1;
    }

    if op == aut::BT_INIT {
        return if unsafe { bt_init() } { 0 } else { -2 };
    }

    if !unsafe { bt_ready() } {
        return -2;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/bluetooth/init.rs (line 2) - [internal] bt_init

```
fn bt_init() -> bool;
fn bt_ready() -> bool;
}

pub fn init() -> bool {
    unsafe { bt_init() }
}

pub fn ready() -> bool {
    unsafe { bt_ready() }
}
```

kernel_Workspace/kernel/src/bluetooth/init.rs (line 3) - [internal] bt_ready

```
fn bt_ready() -> bool;
}

pub fn init() -> bool {
    unsafe { bt_init() }
}

pub fn ready() -> bool {
    unsafe { bt_ready() }
}
```

kernel_Workspace/kernel/src/bluetooth/init.rs (line 6) - [public] init

```
pub fn init() -> bool {
    unsafe { bt_init() }
}

pub fn ready() -> bool {
    unsafe { bt_ready() }
}
```

kernel_Workspace/kernel/src/bluetooth/init.rs (line 10) - [public] ready

```
pub fn ready() -> bool {
    unsafe { bt_ready() }
}
```

kernel_Workspace/kernel/src/camera/aut.rs (line 6) - [public] authorize

```
pub fn authorize(ring: u8, op: u8) -> bool {
    if ring == 0 {
        return true;
    }

    if op == CAM_START as u8 || op == CAM_STOP as u8 || op == CAM_FRAME as u8 {
        if ring >= 3 {
            return false;
        }
    }

    true
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/camera/bridge.rs (line 5) - [internal] camera_caps_get_w

```
fn camera_caps_get_w(w: *mut u32, h: *mut u32, fmt: *mut u32, fps: *mut u32) -> bool;
fn camera_start() -> bool;
fn camera_stop() -> bool;
fn camera_frame_to_phys(phys: u64, cap: u32, fid: *mut u64) -> bool;
}

fn grant_phys(va: u64) -> Option<u64> {
    crate::driverspaceinit::init::service::grant_phys(va).map(|(p, _)| p)
}

pub fn cam_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op as u8) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/camera/bridge.rs (line 6) - [internal] camera_start

```
fn camera_start() -> bool;
fn camera_stop() -> bool;
fn camera_frame_to_phys(phys: u64, cap: u32, fid: *mut u64) -> bool;
}

fn grant_phys(va: u64) -> Option<u64> {
    crate::driverspaceinit::init::service::grant_phys(va).map(|(p, _)| p)
}

pub fn cam_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op as u8) {
        return -1;
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/camera/bridge.rs (line 7) - [internal] camera_stop

```
fn camera_stop() -> bool;
fn camera_frame_to_phys(phys: u64, cap: u32, fid: *mut u64) -> bool;
}

fn grant_phys(va: u64) -> Option<u64> {
    crate::driverspaceinit::init::service::grant_phys(va).map(|(p, _)| p)
}

pub fn cam_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op as u8) {
        return -1;
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/camera/bridge.rs (line 8) - [internal] camera_frame_to_phys

```
fn camera_frame_to_phys(phys: u64, cap: u32, fid: *mut u64) -> bool;
}

fn grant_phys(va: u64) -> Option<u64> {
    crate::driverspaceinit::init::service::grant_phys(va).map(|(p, _)| p)
}

pub fn cam_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op as u8) {
        return -1;
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/camera/bridge.rs (line 11) - [internal] grant_phys

```
fn grant_phys(va: u64) -> Option<u64> {
    crate::driverspaceinit::init::service::grant_phys(va).map(|(p, _)| p)
}

pub fn cam_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op as u8) {
        return -1;
    }

    match op {
        x if x == aut::CAM_CAPS => {
            let mut w = 0u32;
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/camera/bridge.rs (line 15) - [public] cam_call

```
pub fn cam_call(op: u32, m: &DsMsg, r: &mut DsMsg, ring: u8) -> i32 {
    if !aut::authorize(ring, op as u8) {
        return -1;
    }

    match op {
        x if x == aut::CAM_CAPS => {
            let mut w = 0u32;
            let mut h = 0u32;
            let mut fmt = 0u32;
            let mut fps = 0u32;

            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/camera/init.rs (line 2) - [internal] camera_init

```
fn camera_init() -> bool;
}

pub fn init() -> bool {
    unsafe { camera_init() }
}
```


kernel_workspace/kernel/src/camera/init.rs (line 5) - [public] init

```
pub fn init() -> bool {
    unsafe { camera_init() }
}
```

kernel_workspace/kernel/src/caps/audit.rs (line 6) - [public] EventKind

```
pub enum EventKind {
    CheckOk,
    CheckDeny,
    Grant,
    Revoke,
    Register,
    Unregister,
}
```

```
#[derive(Debug, Clone, Copy)]
pub struct AuditEvent {
    pub seq: u64,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 16) - [public] AuditEvent

```
pub struct AuditEvent {
    pub seq: u64,
    pub tick: u64,
    pub world_id: u32,
    pub target_world: u32,
    pub cap: Capability,
    pub kind: EventKind,
}
```

```
const AUDIT_CAP: usize = 2048;
```

```
struct AuditInner {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 27) - [internal] AuditInner

```
struct AuditInner {
    buf: [Option<AuditEvent>; AUDIT_CAP],
    head: usize,
    seq: u64,
    total: u64,
}
```

```
static AUDIT: Mutex<AuditInner> = Mutex::new(AuditInner {
    buf: [None; AUDIT_CAP],
    head: 0,
    seq: 0,
    total: 0,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 41) - [public] init_audit_log

```
pub fn init_audit_log() -> Result<(), &'static str> {
    let mut a = AUDIT.lock();
    a.head = 0;
    a.seq = 0;
    a.total = 0;
    for i in 0..AUDIT_CAP {
        a.buf[i] = None;
    }
    Ok(())
}
```

```
fn now_tick() -> u64 {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 52) - [internal] now_tick

```
fn now_tick() -> u64 {
    crate::arch::now()
}

fn push(kind: EventKind, world: u32, target: u32, cap: Capability) {
    let mut a = AUDIT.lock();
    let idx = a.head as usize % AUDIT_CAP;
    let ev = AuditEvent {
        seq: a.seq,
        tick: now_tick(),
        world_id: world,
        target_world: target,
        // ... (truncated, see source file for more)
    }
```

kernel_workspace/kernel/src/caps/audit.rs (line 56) - [internal] push

```
fn push(kind: EventKind, world: u32, target: u32, cap: Capability) {
    let mut a = AUDIT.lock();
    let idx = a.head as usize % AUDIT_CAP;
    let ev = AuditEvent {
        seq: a.seq,
        tick: now_tick(),
        world_id: world,
        target_world: target,
        cap,
        kind,
    };
    a.buf[idx] = Some(ev);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/audit.rs (line 73) - [public] log_check

```
pub fn log_check(world: u32, cap: Capability, ok: bool) {
    push(if ok { EventKind::CheckOk } else { EventKind::CheckDeny },
        world, world, cap);
}

pub fn log_grant(granter: u32, target: u32, cap: Capability, ok: bool) {
    if ok {
        push(EventKind::Grant, granter, target, cap);
    }
}

pub fn log_revoke(world: u32, cap: Capability, ok: bool) {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/audit.rs (line 78) - [public] log_grant

```
pub fn log_grant(granter: u32, target: u32, cap: Capability, ok: bool) {
    if ok {
        push(EventKind::Grant, granter, target, cap);
    }
}

pub fn log_revoke(world: u32, cap: Capability, ok: bool) {
    if ok {
        push(EventKind::Revoke, world, world, cap);
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 84) - [public] log_revoke

```
pub fn log_revoke(world: u32, cap: Capability, ok: bool) {
    if ok {
        push(EventKind::Revoke, world, world, cap);
    }
}

pub fn log_register(world: u32) {
    push(EventKind::Register, world, world, Capability::User);
}

pub fn log_unregister(world: u32) {
    push(EventKind::Unregister, world, world, Capability::User);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 90) - [public] log_register

```
pub fn log_register(world: u32) {
    push(EventKind::Register, world, world, Capability::User);
}

pub fn log_unregister(world: u32) {
    push(EventKind::Unregister, world, world, Capability::User);
}

pub fn count() -> usize {
    AUDIT.lock().total as usize
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 94) - [public] log_unregister

```
pub fn log_unregister(world: u32) {
    push(EventKind::Unregister, world, world, Capability::User);
}

pub fn count() -> usize {
    AUDIT.lock().total as usize
}

pub fn recent(n: usize) -> Vec<AuditEvent> {
    let a = AUDIT.lock();
    let mut out = Vec::new();

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/audit.rs (line 98) - [public] count

```
pub fn count() -> usize {
    AUDIT.lock().total as usize
}

pub fn recent(n: usize) -> Vec<AuditEvent> {
    let a = AUDIT.lock();
    let mut out = Vec::new();

    let mut idx = if a.head == 0 { AUDIT_CAP - 1 } else { a.head - 1 };

    for _ in 0..n {
        if let Some(ev) = a.buf[idx] {
            // ... (truncated, see source file for more)
```

```

kernel_workspace/kernel/src/caps/audit.rs (line 102) - [public] recent
pub fn recent(n: usize) -> Vec<AuditEvent> {
    let a = AUDIT.lock();
    let mut out = Vec::new();

    let mut idx = if a.head == 0 { AUDIT_CAP - 1 } else { a.head - 1 };

    for _ in 0..n {
        if let Some(ev) = a.buf[idx] {
            out.push(ev);
        }
        if idx == 0 { idx = AUDIT_CAP - 1; } else { idx -= 1; }
        if out.len() >= a.total as usize { break; }
    }
    // ... (truncated, see source file for more)
}

kernel_workspace/kernel/src/caps/audit.rs (line 119) - [public] by_world
pub fn by_world(world: u32, limit: usize) -> Vec<AuditEvent> {
    recent(AUDIT_CAP).into_iter()
        .filter(|e| e.world_id == world || e.target_world == world)
        .take(limit)
        .collect()
}

pub fn by_kind(kind: EventKind, limit: usize) -> Vec<AuditEvent> {
    recent(AUDIT_CAP).into_iter()
        .filter(|e| e.kind == kind)
        .take(limit)
        .collect()
    // ... (truncated, see source file for more)
}

kernel_workspace/kernel/src/caps/audit.rs (line 126) - [public] by_kind
pub fn by_kind(kind: EventKind, limit: usize) -> Vec<AuditEvent> {
    recent(AUDIT_CAP).into_iter()
        .filter(|e| e.kind == kind)
        .take(limit)
        .collect()
}

pub fn deny_count() -> usize {
    by_kind(EventKind::CheckDeny, AUDIT_CAP).len()
}

#[cfg(test)]
// ... (truncated, see source file for more)

kernel_workspace/kernel/src/caps/audit.rs (line 133) - [public] deny_count
pub fn deny_count() -> usize {
    by_kind(EventKind::CheckDeny, AUDIT_CAP).len()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_audit() {
        init_audit_log().unwrap();

        // ... (truncated, see source file for more)
    }
}

```

kernel_workspace/kernel/src/caps/audit.rs (line 142) - [internal] test_audit

```
fn test_audit() {
    init_audit_log().unwrap();

    log_check(1, Capability::User, true);
    log_check(1, Capability::Driver, false);
    log_grant(0, 1, Capability::User, true);

    assert_eq!(count(), 3);
    assert_eq!(deny_count(), 1);

    let r = recent(3);
    assert_eq!(r.len(), 3);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 10) - [public] kernel_world_id

```
pub fn kernel_world_id() -> u32 {
    KERNEL_WORLD.load(Ordering::Relaxed)
}

pub fn set_kernel_world_id(id: u32) {
    KERNEL_WORLD.store(id, Ordering::Relaxed);
}

pub fn current_world_id_pub() -> u32 {
    CURRENT_WORLD.load(Ordering::Relaxed)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 14) - [public] set_kernel_world_id

```
pub fn set_kernel_world_id(id: u32) {
    KERNEL_WORLD.store(id, Ordering::Relaxed);
}

pub fn current_world_id_pub() -> u32 {
    CURRENT_WORLD.load(Ordering::Relaxed)
}

pub fn set_current_world(world_id: u32) {
    CURRENT_WORLD.store(world_id, Ordering::Relaxed);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 18) - [public] current_world_id_pub

```
pub fn current_world_id_pub() -> u32 {
    CURRENT_WORLD.load(Ordering::Relaxed)
}

pub fn set_current_world(world_id: u32) {
    CURRENT_WORLD.store(world_id, Ordering::Relaxed);
}

fn current_world_id() -> u32 {
    current_world_id_pub()
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 22) - [public] set_current_world

```
pub fn set_current_world(world_id: u32) {
    CURRENT_WORLD.store(world_id, Ordering::Relaxed);
}

fn current_world_id() -> u32 {
    current_world_id_pub()
}

pub fn has_cap(cap: Capability) -> bool {
    let wid = current_world_id();
    store::world_has_cap(wid, cap)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 26) - [internal] current_world_id

```
fn current_world_id() -> u32 {
    current_world_id_pub()
}

pub fn has_cap(cap: Capability) -> bool {
    let wid = current_world_id();
    store::world_has_cap(wid, cap)
}

pub fn world_has_cap(world_id: u32, cap: Capability) -> bool {
    store::world_has_cap(world_id, cap)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 30) - [public] has_cap

```
pub fn has_cap(cap: Capability) -> bool {
    let wid = current_world_id();
    store::world_has_cap(wid, cap)
}

pub fn world_has_cap(world_id: u32, cap: Capability) -> bool {
    store::world_has_cap(world_id, cap)
}

pub fn require_cap(cap: Capability) -> CapResult<()> {
    let wid = current_world_id();

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 35) - [public] world_has_cap

```
pub fn world_has_cap(world_id: u32, cap: Capability) -> bool {
    store::world_has_cap(world_id, cap)
}

pub fn require_cap(cap: Capability) -> CapResult<()> {
    let wid = current_world_id();

    if store::world_has_cap(wid, cap) {
        audit::log_check(wid, cap, true);
        Ok(())
    } else {
        audit::log_check(wid, cap, false);
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 39) - [public] require_cap

```
pub fn require_cap(cap: Capability) -> CapResult<()> {
    let wid = current_world_id();

    if store::world_has_cap(wid, cap) {
        audit::log_check(wid, cap, true);
        Ok(())
    } else {
        audit::log_check(wid, cap, false);
        Err(CapabilityError {
            required: cap,
            world_id: Some(wid),
        })
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 54) - [public] require_world_cap

```
pub fn require_world_cap(world_id: u32, cap: Capability) -> CapResult<()> {
    if store::world_has_cap(world_id, cap) {
        audit::log_check(world_id, cap, true);
        Ok(())
    } else {
        audit::log_check(world_id, cap, false);
        Err(CapabilityError {
            required: cap,
            world_id: Some(world_id),
        })
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 67) - [public] require_caps

```
pub fn require_caps(caps: &[Capability]) -> CapResult<()> {
    for &cap in caps {
        require_cap(cap)?;
    }
    Ok(())
}

pub fn with_cap<T, F>(cap: Capability, f: F) -> CapResult<T>
where
    F: FnOnce() -> T,
{
    require_cap(cap)?;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 74) - [public] with_cap

```
pub fn with_cap<T, F>(cap: Capability, f: F) -> CapResult<T>
where
    F: FnOnce() -> T,
{
    require_cap(cap)?;
    Ok(f())
}

pub fn with_caps<T, F>(caps: &[Capability], f: F) -> CapResult<T>
where
    F: FnOnce() -> T,
{
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 82) - [public] with_caps

```
pub fn with_caps<T, F>(caps: &[Capability], f: F) -> CapResult<T>
where
    F: FnOnce() -> T,
{
    require_caps(caps)?;
    Ok(f())
}

#[macro_export]
macro_rules! require_cap {
    ($cap:expr) => {
        $crate::caps::check::require_cap($cap)?
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/caps/check.rs (line 104) - [public] assert_cap

```
pub fn assert_cap(cap: Capability) {
    if !has_cap(cap) {
        panic!("CAPABILITY VIOLATION: {} required", cap.name());
    }
}

#[inline(always)]
pub fn fast_check(cap: Capability) -> bool {
    store::world_has_cap(current_world_id(), cap)
}

pub fn if_cap<T, F>(cap: Capability, f: F, default: T) -> T
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 111) - [public] fast_check

```
pub fn fast_check(cap: Capability) -> bool {
    store::world_has_cap(current_world_id(), cap)
}

pub fn if_cap<T, F>(cap: Capability, f: F, default: T) -> T
where
    F: FnOnce() -> T,
{
    if has_cap(cap) {
        f()
    } else {
        default
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/check.rs (line 115) - [public] if_cap

```
pub fn if_cap<T, F>(cap: Capability, f: F, default: T) -> T
where
    F: FnOnce() -> T,
{
    if has_cap(cap) {
        f()
    } else {
        default
    }
}

pub struct TemporaryCaps {
    // ... (truncated, see source file for more)
}
```


kernel_workspace/kernel/src/caps/check.rs (line 126) - [public] TemporaryCaps

```
pub struct TemporaryCaps {
    world_id: u32,
    original: CapabilitySet,
}

impl TemporaryCaps {
    pub fn enter(extra: CapabilitySet) -> CapResult<Self> {
        let wid = current_world_id();
        let original = store::get_world_caps(wid)
            .map_err(|_| CapabilityError { required: Capability::User, world_id: Some(wid) })?;

        require_cap(Capability::Admin)?;
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/caps/check.rs (line 132) - [public] enter

```
pub fn enter(extra: CapabilitySet) -> CapResult<Self> {
    let wid = current_world_id();
    let original = store::get_world_caps(wid)
        .map_err(|_| CapabilityError { required: Capability::User, world_id: Some(wid) })?;

    require_cap(Capability::Admin)?;

    store::set_world_caps(wid, original.union(extra))
        .map_err(|_| CapabilityError { required: Capability::Admin, world_id: Some(wid) })?;

    Ok(Self { world_id: wid, original })
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/check.rs (line 147) - [internal] drop

```
fn drop(&mut self) {
    let _ = store::set_world_caps(self.world_id, self.original);
}

#[cfg(test)]
mod tests {
    use super::*;
    use crate::caps::sets;

    #[test]
    fn test_require() {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/caps/check.rs (line 158) - [internal] test_require

```
fn test_require() {
    crate::caps::init().unwrap();

    let wid = store::register_world(None, sets::presets::standard_user()).unwrap();

    assert!(store::world_has_cap(wid, Capability::User));
    assert!(!store::world_has_cap(wid, Capability::Driver));
}
}
```

kernel_workspace/kernel/src/caps/defaults.rs (line 6) - [public] default_for_ring

```
pub fn default_for_ring(ring: u8) -> CapabilitySet {
    match ring {
        0 => sets::presets::kernel(),
        1 => sets::presets::driver(),
        2 => sets::presets::privileged_user(),
        _ => sets::presets::standard_user(),
    }
}

pub fn install_defaults() -> Result<(), &'static str> {
    let id = store::register_world(None, sets::presets::kernel())?;
    crate::caps::check::set_kernel_world_id(id);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/defaults.rs (line 15) - [public] install_defaults

```
pub fn install_defaults() -> Result<(), &'static str> {
    let id = store::register_world(None, sets::presets::kernel());?;
    crate::caps::check::set_kernel_world_id(id);
    audit::log_register(id);
    Ok(())
}

pub fn register_for_ring(ring: u8, parent: Option<u32>) -> Result<u32, &'static str> {
    let set = default_for_ring(ring);
    let wid = store::register_world(parent, set)?;
    audit::log_register(wid);
    Ok(wid)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/defaults.rs (line 22) - [public] register_for_ring

```
pub fn register_for_ring(ring: u8, parent: Option<u32>) -> Result<u32, &'static str> {
    let set = default_for_ring(ring);
    let wid = store::register_world(parent, set)?;
    audit::log_register(wid);
    Ok(wid)
}
```

kernel_workspace/kernel/src/caps/export.rs (line 7) - [internal] cap_from_id

```
fn cap_from_id(id: u8) -> Option<Capability> {
    Capability::iter_all().find(|c| c.id() == id)
}

#[no_mangle]
pub extern "C" fn caps_self_bits() -> u32 {
    let wid = check::current_world_id_pub();
    store::get_world_caps(wid).map(|c| c.bits()).unwrap_or(0)
}

#[no_mangle]
pub extern "C" fn caps_world_bits(world_id: u32) -> u32 {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/grant.rs (line 18) - [public] grant_cap

```
pub fn grant_cap(granter: u32, target: u32, cap: Capability) -> CapResult<()> {
    if !store::world_has_cap(granter, cap) {
        audit::log_grant(granter, target, cap, false);
        return Err(CapabilityError { required: cap, world_id: Some(granter) });
    }

    store::add_world_cap(target, cap)
        .map_err(|_| CapabilityError { required: cap, world_id: Some(target) })?;

    audit::log_grant(granter, target, cap, true);
    Ok(())
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/grant.rs (line 31) - [public] delegate_caps

```
pub fn delegate_caps(granter: u32, target: u32, caps: CapabilitySet) -> CapResult<()> {
    for cap in caps.iter() {
        grant_cap(granter, target, cap)?;
    }
    Ok(())
}

pub fn inherit_set(parent_world: u32) -> CapabilitySet {
    let parent = match store::get_world_caps(parent_world) {
        Ok(c) => c,
        Err(_) => CapabilitySet::empty(),
    };
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/grant.rs (line 38) - [public] inherit_set

```
pub fn inherit_set(parent_world: u32) -> CapabilitySet {
    let parent = match store::get_world_caps(parent_world) {
        Ok(c) => c,
        Err(_) => CapabilitySet::empty(),
    };

    let mut child = parent;
    for &cap in NON_INHERITABLE {
        child = child.remove(cap);
    }
    child
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/grant.rs (line 51) - [public] spawn_child

```
pub fn spawn_child(parent_world: u32) -> Result<u32, &'static str> {
    let child_set = inherit_set(parent_world);
    store::register_world(Some(parent_world), child_set)
}
```

```
#[derive(Clone, Copy)]
struct TempGrant {
    world_id: u32,
    cap: Capability,
    expires_tick: u64,
    active: bool,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/grant.rs (line 58) - [internal] TempGrant

```
struct TempGrant {
    world_id: u32,
    cap: Capability,
    expires_tick: u64,
    active: bool,
}

const MAX_TEMP: usize = 128;

static TEMP: Mutex<[TempGrant; MAX_TEMP]> = Mutex::new([TempGrant {
    world_id: 0,
    cap: Capability::User,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/grant.rs (line 74) - [internal] now_tick

```
fn now_tick() -> u64 {
    crate::arch::now()
}

pub fn grant_temporary(granter: u32, target: u32, cap: Capability,
    ttl_ticks: u64) -> CapResult<()>
{
    grant_cap(granter, target, cap)?;

    let mut t = TEMP.lock();
    let slot = t.iter_mut().find(|g| !g.active)
        .ok_or(CapabilityError { required: cap, world_id: Some(target) })?;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/grant.rs (line 78) - [public] grant_temporary

```
pub fn grant_temporary(granter: u32, target: u32, cap: Capability,
                       ttl_ticks: u64) -> CapResult<()>
{
    grant_cap(granter, target, cap)?;

    let mut t = TEMP.lock();
    let slot = t.iter_mut().find(|g| !g.active)
        .ok_or(CapabilityError { required: cap, world_id: Some(target) })?;

    *slot = TempGrant {
        world_id: target,
        cap,
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/caps/grant.rs (line 97) - [public] prune_expired

```
pub fn prune_expired() {
    let t_now = now_tick();
    let mut t = TEMP.lock();

    for g in t.iter_mut() {
        if g.active && t_now >= g.expires_tick {
            let _ = store::remove_world_cap(g.world_id, g.cap);
            audit::log_revoke(g.world_id, g.cap, true);
            g.active = false;
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/caps/grant.rs (line 110) - [public] is_temporary

```
pub fn is_temporary(world_id: u32, cap: Capability) -> bool {
    let t = TEMP.lock();
    t.iter().any(|g| g.active && g.world_id == world_id && g.cap == cap)
}

#[cfg(test)]
mod tests {
    use super::*;
    use crate::caps::sets;

    #[test]
    fn test_inherit() {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/caps/grant.rs (line 121) - [internal] test_inherit

```
fn test_inherit() {
    crate::caps::init().unwrap();

    let parent = store::register_world(None, sets::presets::privileged_user()).unwrap();
    let child_set = inherit_set(parent);

    assert!(child_set.has(Capability::User));
    assert!(child_set.has(Capability::Spawn));
    assert!(!child_set.has(Capability::Admin));
    assert!(!child_set.has(Capability::SyscallAll));
}
}
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 5) - [public] parent

```
pub fn parent(cap: Capability) -> Option<Capability> {
    Some(match cap {
        Capability::Root => return None,

        Capability::Ring0 => Capability::Root,
        Capability::Driver => Capability::Ring0,
        Capability::User => Capability::Root,
        Capability::Admin => Capability::Ring0,

        Capability::PhysAlloc => Capability::Driver,
        Capability::VirtMap => Capability::Driver,
        Capability::Dma => Capability::Driver,
    })
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 47) - [public] implies

```
pub fn implies(held: Capability, required: Capability) -> bool {
    if held == required {
        return true;
    }

    let mut cur = required;
    while let Some(p) = parent(cur) {
        if p == held {
            return true;
        }
        cur = p;
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 62) - [public] set_implies

```
pub fn set_implies(held: CapabilitySet, required: Capability) -> bool {
    for cap in held.iter() {
        if implies(cap, required) {
            return true;
        }
    }
    false
}

pub fn expand_hierarchy(set: CapabilitySet) -> CapabilitySet {
    let mut result = set;

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 71) - [public] expand_hierarchy

```
pub fn expand_hierarchy(set: CapabilitySet) -> CapabilitySet {
    let mut result = set;

    for cap in set.iter() {
        let mut cur = cap;
        while let Some(p) = parent(cur) {
            result = result.add(p);
            cur = p;
        }
    }

    result
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 85) - [public] path_to_root

```
pub fn path_to_root(cap: Capability) -> Vec<Capability> {
    let mut path = vec![cap];
    let mut cur = cap;
    while let Some(p) = parent(cur) {
        path.push(p);
        cur = p;
    }
    path.reverse();
    path
}

pub fn depth(cap: Capability) -> usize {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 96) - [public] depth

```
pub fn depth(cap: Capability) -> usize {
    let mut d = 0;
    let mut cur = cap;
    while let Some(p) = parent(cur) {
        d += 1;
        cur = p;
    }
    d
}

pub fn subtree(root: Capability) -> Vec<Capability> {
    let mut result = vec![root];
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 106) - [public] subtree

```
pub fn subtree(root: Capability) -> Vec<Capability> {
    let mut result = vec![root];
    let mut i = 0;

    while i < result.len() {
        let cur = result[i];
        for cap in Capability::iter_all() {
            if parent(cap) == Some(cur) && !result.contains(&cap) {
                result.push(cap);
            }
        }
        i += 1;
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 123) - [public] can_delegate

```
pub fn can_delegate(holder: CapabilitySet, candidate: Capability) -> bool {
    set_implies(holder, candidate)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_hierarchy() {
        assert!(implies(Capability::Root, Capability::Driver));
        assert!(implies(Capability::Driver, Capability::DevPci));
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 132) - [internal] test_hierarchy

```
fn test_hierarchy() {
    assert!(implies(Capability::Root, Capability::Driver));
    assert!(implies(Capability::Driver, Capability::DevPci));
    assert!(!implies(Capability::DevPci, Capability::Driver));

    let driver_set = CapabilitySet::single(Capability::Driver);
    assert!(set_implies(driver_set, Capability::DevPci));
    assert!(!set_implies(driver_set, Capability::Admin));

    let expanded = expand_hierarchy(CapabilitySet::single(Capability::DevPci));
    assert!(expanded.has(Capability::DevPci));
    assert!(expanded.has(Capability::Driver));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/hierarchy.rs (line 149) - [internal] test_depth

```
fn test_depth() {
    assert_eq!(depth(Capability::Root), 0);
    assert_eq!(depth(Capability::Driver), 2);
    assert_eq!(depth(Capability::DevPci), 3);
}
}
```

kernel_workspace/kernel/src/caps/mod.rs (line 31) - [public] init

```
pub fn init() -> Result<(), &'static str> {
    store::init_store()?;
    audit::init_audit_log()?;
    defaults::install_defaults()?;
    Ok(())
}

pub fn snapshot() -> SystemSnapshot {
    SystemSnapshot {
        global_caps: store::global_caps(),
        revoked: revoke::revoked_list(),
        audit_count: audit::count(),
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/mod.rs (line 38) - [public] snapshot

```
pub fn snapshot() -> SystemSnapshot {
    SystemSnapshot {
        global_caps: store::global_caps(),
        revoked: revoke::revoked_list(),
        audit_count: audit::count(),
        world_count: store::world_count(),
    }
}

#[derive(Debug, Clone)]
pub struct SystemSnapshot {
    pub global_caps: CapabilitySet,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/mod.rs (line 48) - [public] SystemSnapshot

```
pub struct SystemSnapshot {
    pub global_caps: CapabilitySet,
    pub revoked: Vec<Capability>,
    pub audit_count: usize,
    pub world_count: usize,
}

pub fn self_test() -> crate::testing::TestResult {
    let mut ok = true;

    let mut chk = |step: u32, cond: bool| -> bool {
        if !cond {
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/mod.rs (line 55) - [public] self_test

```
pub fn self_test() -> crate::testing::TestResult {
    let mut ok = true;

    let mut chk = |step: u32, cond: bool| -> bool {
        if !cond {
            println!("[caps] self_test step {} FAILED", step);
        }
        cond
    };

    let a = CapabilitySet::empty().add(Capability::User);
    let b = CapabilitySet::single(Capability::Driver);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/policy.rs (line 9) - [public] set_hook

```
pub fn set_hook(h: PolicyHook) {
    let raw: *mut () = unsafe { core::mem::transmute(h as PolicyHook) };
    HOOK.store(raw, Ordering::Relaxed);
}

fn policy_allows(world: u32, cap: Capability) -> bool {
    let raw = HOOK.load(Ordering::Relaxed);
    if raw.is_null() {
        return true;
    }
    let hook: PolicyHook = unsafe { core::mem::transmute(raw) };
    hook(world, cap)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/policy.rs (line 14) - [internal] policy_allows

```
fn policy_allows(world: u32, cap: Capability) -> bool {
    let raw = HOOK.load(Ordering::Relaxed);
    if raw.is_null() {
        return true;
    }
    let hook: PolicyHook = unsafe { core::mem::transmute(raw) };
    hook(world, cap)
}

pub fn enforce(world: u32, cap: Capability) -> CapResult<()> {
    if !store::world_has_cap(world, cap) {
        return Err(CapabilityError { required: cap, world_id: Some(world) });
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/policy.rs (line 23) - [public] enforce

```
pub fn enforce(world: u32, cap: Capability) -> CapResult<()> {
    if !store::world_has_cap(world, cap) {
        return Err(CapabilityError { required: cap, world_id: Some(world) });
    }

    if !policy_allows(world, cap) {
        return Err(CapabilityError { required: cap, world_id: Some(world) });
    }

    Ok(())
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/policy.rs (line 35) - [public] enforce_self

```
pub fn enforce_self(cap: Capability) -> CapResult<()> {
    let wid = super::check::current_world_id_pub();
    enforce(wid, cap)
}

pub fn allowed(world: u32, cap: Capability) -> bool {
    enforce(world, cap).is_ok()
}
```


kernel_workspace/kernel/src/caps/policy.rs (line 40) - [public] allowed

```
pub fn allowed(world: u32, cap: Capability) -> bool {
    enforce(world, cap).is_ok()
}
```

kernel_workspace/kernel/src/caps/revoke.rs (line 7) - [public] revoke_from_world

```
pub fn revoke_from_world(world_id: u32, cap: Capability) -> CapResult<()> {
    store::remove_world_cap(world_id, cap)
        .map_err(|_| CapabilityError { required: cap, world_id: Some(world_id) })?;

    audit::log_revoke(world_id, cap, true);
    Ok(())
}

pub fn revoke_subtree(world_id: u32, cap: Capability) -> CapResult<()> {
    for c in hierarchy::subtree(cap) {
        let _ = store::remove_world_cap(world_id, c);
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/revoke.rs (line 15) - [public] revoke_subtree

```
pub fn revoke_subtree(world_id: u32, cap: Capability) -> CapResult<()> {
    for c in hierarchy::subtree(cap) {
        let _ = store::remove_world_cap(world_id, c);
    }
    audit::log_revoke(world_id, cap, true);
    Ok(())
}

pub fn revoke_global(cap: Capability) {
    store::add_global_revoked(cap);
    audit::log_revoke(0, cap, true);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/revoke.rs (line 23) - [public] revoke_global

```
pub fn revoke_global(cap: Capability) {
    store::add_global_revoked(cap);
    audit::log_revoke(0, cap, true);
}

pub fn restore_global(cap: Capability) {
    store::remove_global_revoked(cap);
}

pub fn revoked_list() -> Vec<Capability> {
    let mut out = Vec::new();
    let mut bits = CapabilitySet::empty();
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/revoke.rs (line 28) - [public] restore_global

```
pub fn restore_global(cap: Capability) {
    store::remove_global_revoked(cap);
}

pub fn revoked_list() -> Vec<Capability> {
    let mut out = Vec::new();
    let mut bits = CapabilitySet::empty();

    let revoked = CapabilitySet::all().diff(store::global_caps());
    for cap in revoked.iter() {
        out.push(cap);
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/revoke.rs (line 32) - [public] revoked_list

```
pub fn revoked_list() -> Vec<Capability> {
    let mut out = Vec::new();
    let mut bits = CapabilitySet::empty();

    let revoked = CapabilitySet::all().diff(store::global_caps());
    for cap in revoked.iter() {
        out.push(cap);
    }
    out
}

pub fn is_globally_revoked(cap: Capability) -> bool {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/revoke.rs (line 43) - [public] is_globally_revoked

```
pub fn is_globally_revoked(cap: Capability) -> bool {
    !store::global_caps().has(cap)
}
```

```
pub fn lockdown(world_id: u32) {
    let keep = CapabilitySet::empty()
        .add(Capability::User)
        .add(Capability::FsRead)
        .add(Capability::IpcRecv);

    for cap in Capability::iter_all() {
        if !keep.has(cap) {
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/revoke.rs (line 47) - [public] lockdown

```
pub fn lockdown(world_id: u32) {
    let keep = CapabilitySet::empty()
        .add(Capability::User)
        .add(Capability::FsRead)
        .add(Capability::IpcRecv);

    for cap in Capability::iter_all() {
        if !keep.has(cap) {
            let _ = store::remove_world_cap(world_id, cap);
        }
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/revoke.rs (line 68) - [internal] test_revoke

```
fn test_revoke() {
    crate::caps::init().unwrap();

    let wid = store::register_world(None, sets::presets::driver()).unwrap();
    assert!(store::world_has_cap(wid, Capability::DevPci));

    revoke_from_world(wid, Capability::Driver).unwrap();

    assert!(!store::world_has_cap(wid, Capability::Driver));
}
}
```

kernel_Workspace/kernel/src/caps/sets.rs (line 4) - [public] CapSetBuilder

```
pub struct CapSetBuilder {
    set: CapabilitySet,
}

impl CapSetBuilder {
    pub fn new() -> Self {
        Self { set: CapabilitySet::empty() }
    }

    pub fn empty() -> Self {
        Self::new()
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/sets.rs (line 9) - [public] new

```
pub fn new() -> Self {
    Self { set: CapabilitySet::empty() }
}

pub fn empty() -> Self {
    Self::new()
}

pub fn add(mut self, cap: Capability) -> Self {
    self.set = self.set.add(cap);
    self
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/sets.rs (line 13) - [public] empty

```
pub fn empty() -> Self {
    Self::new()
}

pub fn add(mut self, cap: Capability) -> Self {
    self.set = self.set.add(cap);
    self
}

pub fn add_many(mut self, caps: &[Capability]) -> Self {
    for &cap in caps {
        self.set = self.set.add(cap);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/sets.rs (line 17) - [public] add

```
pub fn add(mut self, cap: Capability) -> Self {
    self.set = self.set.add(cap);
    self
}

pub fn add_many(mut self, caps: &[Capability]) -> Self {
    for &cap in caps {
        self.set = self.set.add(cap);
    }
    self
}
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/caps/sets.rs (line 22) - [public] add_many
    pub fn add_many(mut self, caps: &[Capability]) -> Self {
        for &cap in caps {
            self.set = self.set.add(cap);
        }
        self
    }

    pub fn add_category(mut self, cat: super::types::CapCategory) -> Self {
        for cap in Capability::iter_all() {
            if cap.category() == cat {
                self.set = self.set.add(cap);
            }
        }
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/caps/sets.rs (line 29) - [public] add_category
    pub fn add_category(mut self, cat: super::types::CapCategory) -> Self {
        for cap in Capability::iter_all() {
            if cap.category() == cat {
                self.set = self.set.add(cap);
            }
        }
        self
    }

    pub fn remove(mut self, cap: Capability) -> Self {
        self.set = self.set.remove(cap);
        self
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/caps/sets.rs (line 38) - [public] remove
    pub fn remove(mut self, cap: Capability) -> Self {
        self.set = self.set.remove(cap);
        self
    }

    pub fn with_hierarchy(self) -> Self {
        Self { set: hierarchy::expand_hierarchy(self.set) }
    }

    pub fn build(self) -> CapabilitySet {
        self.set
    }
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/caps/sets.rs (line 43) - [public] with_hierarchy
    pub fn with_hierarchy(self) -> Self {
        Self { set: hierarchy::expand_hierarchy(self.set) }
    }

    pub fn build(self) -> CapabilitySet {
        self.set
    }
}

pub mod presets {
    use super::*;

    // ... (truncated, see source file for more)
}

```

kernel_workspace/kernel/src/caps/sets.rs (line 47) - [public] build

```
pub fn build(self) -> CapabilitySet {
    self.set
}

pub mod presets {
    use super::*;

    pub fn minimal_user() -> CapabilitySet {
        CapSetBuilder::new()
            .add(Capability::User)
            .add(Capability::Mmap)
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/caps/sets.rs (line 55) - [public] minimal_user

```
pub fn minimal_user() -> CapabilitySet {
    CapSetBuilder::new()
        .add(Capability::User)
        .add(Capability::Mmap)
        .add(Capability::IpcSend)
        .add(Capability::IpcRecv)
        .add(Capability::FsRead)
        .add(Capability::Spawn)
        .add(Capability::SyscallRestricted)
        .build()
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 67) - [public] standard_user

```
pub fn standard_user() -> CapabilitySet {
    CapSetBuilder::new()
        .add(Capability::User)
        .add(Capability::Mmap)
        .add(Capability::Protect)
        .add(Capability::Spawn)
        .add(Capability::Kill)
        .add(Capability::IpcSend)
        .add(Capability::IpcRecv)
        .add(Capability::IpcBroadcast)
        .add(Capability::FsRead)
        .add(Capability::FsWrite)
        // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/sets.rs (line 84) - [public] privileged_user

```
pub fn privileged_user() -> CapabilitySet {
    CapSetBuilder::new()
        .add(Capability::User)
        .add_many(&[
            Capability::Mmap, Capability::Protect, Capability::Spawn,
            Capability::Kill, Capability::IpcSend, Capability::IpcRecv,
            Capability::IpcBroadcast, Capability::IpcPrivileged,
            Capability::FsRead, Capability::FsWrite,
            Capability::FsCreate, Capability::FsMount,
            Capability::Sched, Capability::SyscallAll,
        ])
        .build()
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/sets.rs (line 98) - [public] driver

```
pub fn driver() -> CapabilitySet {
    CapSetBuilder::new()
        .add(Capability::Driver)
        .add_many(&[
            Capability::PhysAlloc, Capability::VirtMap, Capability::Dma,
            Capability::DevPci, Capability::DevPort,
            Capability::DevMmio, Capability::DevIrq,
            Capability::IpcSend, Capability::IpcRecv,
            Capability::IpcPrivileged, Capability::SyscallAll,
        ])
        .build()
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 111) - [public] kernel

```
pub fn kernel() -> CapabilitySet {
    CapabilitySet::all()
}

pub fn sandbox() -> CapabilitySet {
    CapSetBuilder::new()
        .add(Capability::User)
        .add(Capability::FsRead)
        .add(Capability::IpcRecv)
        .add(Capability::SyscallRestricted)
        .build()
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 115) - [public] sandbox

```
pub fn sandbox() -> CapabilitySet {
    CapSetBuilder::new()
        .add(Capability::User)
        .add(Capability::FsRead)
        .add(Capability::IpcRecv)
        .add(Capability::SyscallRestricted)
        .build()
}

pub fn validate_hierarchy(set: CapabilitySet) -> Result<(), &'static str> {
    for cap in set.iter() {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/caps/sets.rs (line 125) - [public] validate_hierarchy

```
pub fn validate_hierarchy(set: CapabilitySet) -> Result<(), &'static str> {
    for cap in set.iter() {
        let mut cur = cap;
        while let Some(p) = hierarchy::parent(cur) {
            if p != Capability::Root && !set.has(p) {
                return Err("hierarchy inconsistency: child without parent");
            }
            cur = p;
            if cur == Capability::Root {
                break;
            }
        }
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 141) - [public] effective

```
pub fn effective(set: CapabilitySet) -> CapabilitySet {
    hierarchy::expand_hierarchy(set)
}

pub fn effective_diff(a: CapabilitySet, b: CapabilitySet) -> CapabilitySet {
    let a_eff = effective(a);
    let b_eff = effective(b);
    a_eff.diff(b_eff)
}

pub fn is_subset_with_hierarchy(subset: CapabilitySet, superset: CapabilitySet) -> bool {
    let sub_eff = effective(subset);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 145) - [public] effective_diff

```
pub fn effective_diff(a: CapabilitySet, b: CapabilitySet) -> CapabilitySet {
    let a_eff = effective(a);
    let b_eff = effective(b);
    a_eff.diff(b_eff)
}

pub fn is_subset_with_hierarchy(subset: CapabilitySet, superset: CapabilitySet) -> bool {
    let sub_eff = effective(subset);
    let sup_eff = effective(superset);
    sup_eff.contains(sub_eff)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 151) - [public] is_subset_with_hierarchy

```
pub fn is_subset_with_hierarchy(subset: CapabilitySet, superset: CapabilitySet) -> bool {
    let sub_eff = effective(subset);
    let sup_eff = effective(superset);
    sup_eff.contains(sub_eff)
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_presets() {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 162) - [internal] test_presets

```
fn test_presets() {
    let user = presets::minimal_user();
    assert!(user.has(Capability::User));
    assert!(!user.has(Capability::Driver));
    assert!(!user.has(Capability::DevPci));

    let drv = presets::driver();
    assert!(drv.has(Capability::Driver));
    assert!(drv.has(Capability::DevPci));
}

#[test]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/sets.rs (line 174) - [internal] test_effective

```
fn test_effective() {
    let set = CapabilitySet::single(Capability::DevPci);
    let eff = effective(set);

    assert!(eff.has(Capability::DevPci));
    assert!(eff.has(Capability::Driver));
    assert!(eff.has(Capability::Ring0));
    assert!(eff.has(Capability::Root));
}
}
```

kernel_workspace/kernel/src/caps/store.rs (line 9) - [public] WorldCaps

```
pub struct WorldCaps {
    pub world_id: u32,
    pub caps: CapabilitySet,
    pub inherited_from: Option<u32>,
    pub revokable: bool,
}

static STORE: Mutex<StoreInner> = Mutex::new(StoreInner {
    worlds: [None; MAX_WORLDS],
    global_revoked: CapabilitySet::empty(),
    next_id: 1,
});
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/store.rs (line 22) - [internal] StoreInner

```
struct StoreInner {
    worlds: [Option<WorldCaps>; MAX_WORLDS],
    global_revoked: CapabilitySet,
    next_id: u32,
}

pub fn init_store() -> Result<(), &'static str> {
    let mut s = STORE.lock();
    for i in 0..MAX_WORLDS {
        s.worlds[i] = None;
    }
    Ok(())
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/store.rs (line 28) - [public] init_store

```
pub fn init_store() -> Result<(), &'static str> {
    let mut s = STORE.lock();
    for i in 0..MAX_WORLDS {
        s.worlds[i] = None;
    }
    Ok(())
}

pub fn global_caps() -> CapabilitySet {
    let s = STORE.lock();
    CapabilitySet::all().diff(s.global_revoked)
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/store.rs (line 36) - [public] global_caps

```
pub fn global_caps() -> CapabilitySet {
    let s = STORE.lock();
    CapabilitySet::all().diff(s.global_revoked)
}

pub fn register_world(parent_world: Option<u32>, initial: CapabilitySet) -> Result<u32, &'static str> {
    let mut s = STORE.lock();

    if let Some(pwid) = parent_world {
        let parent_caps = find_world_inner(&s, pwid)?;
        let parent_eff = parent_caps.caps.diff(s.global_revoked);

        // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/caps/store.rs (line 41) - [public] register_world

```
pub fn register_world(parent_world: Option<u32>, initial: CapabilitySet) -> Result<u32, &'static str> {
    let mut s = STORE.lock();

    if let Some(pwid) = parent_world {
        let parent_caps = find_world_inner(&s, pwid)?;
        let parent_eff = parent_caps.caps.diff(s.global_revoked);

        for cap in initial.iter() {
            if !hierarchy::set_implies(parent_eff, cap) {
                return Err("child capability exceeds parent");
            }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/store.rs (line 71) - [internal] find_world_inner

```
fn find_world_inner(s: &StoreInner, world_id: u32) -> Result<WorldCaps, &'static str> {
    for w in &s.worlds {
        if let Some(wc) = w {
            if wc.world_id == world_id {
                return Ok(*wc);
            }
        }
    }
    Err("world not found")
}
```

```
pub fn get_world_caps(world_id: u32) -> Result<CapabilitySet, &'static str> {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/store.rs (line 82) - [public] get_world_caps

```
pub fn get_world_caps(world_id: u32) -> Result<CapabilitySet, &'static str> {
    let s = STORE.lock();
    let wc = find_world_inner(&s, world_id)?;
    Ok(wc.caps.diff(s.global_revoked))
}
```

```
pub fn set_world_caps(world_id: u32, new_caps: CapabilitySet) -> Result<(), &'static str> {
    let mut s = STORE.lock();

    let wc = find_world_inner(&s, world_id)?;
    if let Some(parent_id) = wc.inherited_from {
        let parent_caps = find_world_inner(&s, parent_id)?;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/caps/store.rs (line 88) - [public] set_world_caps

```
pub fn set_world_caps(world_id: u32, new_caps: CapabilitySet) -> Result<(), &'static str> {
    let mut s = STORE.lock();

    let wc = find_world_inner(&s, world_id)?;
    if let Some(parent_id) = wc.inherited_from {
        let parent_caps = find_world_inner(&s, parent_id)?;
        let parent_eff = parent_caps.caps.diff(s.global_revoked);

        for cap in new_caps.iter() {
            if !hierarchy::set_implies(parent_eff, cap) {
                return Err("cannot grant capability exceeding parent");
            }
        }
    }
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/caps/store.rs (line 115) - [public] add_world_cap
pub fn add_world_cap(world_id: u32, cap: Capability) -> Result<(), &'static str> {
    let current = get_world_caps(world_id)?;
    set_world_caps(world_id, current.add(cap))
}

pub fn remove_world_cap(world_id: u32, cap: Capability) -> Result<(), &'static str> {
    let mut s = STORE.lock();

    for w in &mut s.worlds {
        if let Some(wc) = w {
            if wc.world_id == world_id {
                if !wc.revokable {
                    // ... (truncated, see source file for more)
                }
            }
        }
    }
}

kernel_Workspace/kernel/src/caps/store.rs (line 120) - [public] remove_world_cap
pub fn remove_world_cap(world_id: u32, cap: Capability) -> Result<(), &'static str> {
    let mut s = STORE.lock();

    for w in &mut s.worlds {
        if let Some(wc) = w {
            if wc.world_id == world_id {
                if !wc.revokable {
                    return Err("world capabilities are not revokable");
                }
                wc.caps = wc.caps.remove(cap);
                return Ok(());
            }
        }
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/caps/store.rs (line 138) - [public] unregister_world
pub fn unregister_world(world_id: u32) -> Result<(), &'static str> {
    let mut s = STORE.lock();

    for w in &mut s.worlds {
        if let Some(wc) = w {
            if wc.world_id == world_id {
                *w = None;
                return Ok(());
            }
        }
    }
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/caps/store.rs (line 153) - [public] world_has_cap
pub fn world_has_cap(world_id: u32, cap: Capability) -> bool {
    let s = STORE.lock();
    if let Ok(wc) = find_world_inner(&s, world_id) {
        let eff = wc.caps.diff(s.global_revoked);
        hierarchy::set_implies(eff, cap)
    } else {
        false
    }
}

pub fn world_count() -> usize {
    let s = STORE.lock();
    // ... (truncated, see source file for more)
}

```

kernel_workspace/kernel/src/caps/store.rs (line 163) - [public] world_count

```
pub fn world_count() -> usize {
    let s = STORE.lock();
    s.worlds.iter().filter(|w| w.is_some()).count()
}

pub fn iter_worlds<F: FnMut(u32, CapabilitySet)>(mut f: F) {
    let s = STORE.lock();
    for w in &s.worlds {
        if let Some(wc) = w {
            f(wc.world_id, wc.caps);
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/store.rs (line 168) - [public] iter_worlds

```
pub fn iter_worlds<F: FnMut(u32, CapabilitySet)>(mut f: F) {
    let s = STORE.lock();
    for w in &s.worlds {
        if let Some(wc) = w {
            f(wc.world_id, wc.caps);
        }
    }
}

pub fn add_global_revoked(cap: Capability) {
    let mut s = STORE.lock();
    s.global_revoked.add(cap);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/store.rs (line 177) - [public] add_global_revoked

```
pub fn add_global_revoked(cap: Capability) {
    let mut s = STORE.lock();
    s.global_revoked.add(cap);
}

pub fn remove_global_revoked(cap: Capability) {
    let mut s = STORE.lock();
    s.global_revoked.remove(cap);
}

#[cfg(test)]
mod tests {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/store.rs (line 182) - [public] remove_global_revoked

```
pub fn remove_global_revoked(cap: Capability) {
    let mut s = STORE.lock();
    s.global_revoked.remove(cap);
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_world_registration() {
        init_store().unwrap();
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/caps/store.rs (line 192) - [internal] test_world_registration

```
fn test_world_registration() {
    init_store().unwrap();

    let wid = register_world(None, sets::presets::standard_user()).unwrap();
    assert!(world_has_cap(wid, Capability::User));
    assert!(!world_has_cap(wid, Capability::Driver));

    let child_result = register_world(Some(wid),
        CapabilitySet::single(Capability::DevPci));
    assert!(child_result.is_err());
}
}
```

kernel_workspace/kernel/src/caps/syscalls.rs (line 13) - [internal] cap_from_id

```
fn cap_from_id(id: u8) -> Option<Capability> {
    Capability::iter_all().find(|c| c.id() == id)
}

pub fn cap_syscall(num: u64, a0: u64, a1: u64, a2: u64) -> u64 {
    let wid = check::current_world_id_pub();

    match num {
        SYS_CAP_QUERY => {
            match cap_from_id(a0 as u8) {
                Some(cap) => store::world_has_cap(wid, cap) as u64,
                None => 0,
            }
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/caps/syscalls.rs (line 17) - [public] cap_syscall

```
pub fn cap_syscall(num: u64, a0: u64, a1: u64, a2: u64) -> u64 {
    let wid = check::current_world_id_pub();

    match num {
        SYS_CAP_QUERY => {
            match cap_from_id(a0 as u8) {
                Some(cap) => store::world_has_cap(wid, cap) as u64,
                None => 0,
            }
        }
        SYS_CAP_REQUEST => {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/caps/tests.rs (line 8) - [internal] test_full_flow

```
fn test_full_flow() {
    init().unwrap();

    let kernel = crate::caps::check::kernel_world_id();

    let drv = store::register_world(Some(kernel), sets::presets::driver()).unwrap();
    assert!(store::world_has_cap(drv, Capability::DevPci));
    assert!(!store::world_has_cap(drv, Capability::Admin));

    let usr = store::register_world(Some(kernel), sets::presets::standard_user()).unwrap();
    assert!(store::world_has_cap(usr, Capability::FsRead));
    assert!(!store::world_has_cap(usr, Capability::DevPci));
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/caps/tests.rs (line 29) - [internal] test_hierarchy_enforcement

```
fn test_hierarchy_enforcement() {
    init().unwrap();

    let parent = store::register_world(None, sets::presets::standard_user()).unwrap();

    let bad = store::register_world(Some(parent),
        CapabilitySet::single(Capability::DevPci));
    assert!(bad.is_err());
}

#[test]
fn test_global_revocation() {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/tests.rs (line 40) - [internal] test_global_revocation

```
fn test_global_revocation() {
    init().unwrap();

    let wid = store::register_world(None, sets::presets::driver()).unwrap();
    assert!(store::world_has_cap(wid, Capability::Dma));

    revoke::revoke_global(Capability::Dma);
    assert!(!store::world_has_cap(wid, Capability::Dma));

    revoke::restore_global(Capability::Dma);
    assert!(store::world_has_cap(wid, Capability::Dma));
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/tests.rs (line 54) - [internal] test_audit_trail

```
fn test_audit_trail() {
    init().unwrap();

    let wid = store::register_world(None, sets::presets::standard_user()).unwrap();

    store::world_has_cap(wid, Capability::User);
    assert!(audit::count() > 0);
}
```

kernel_workspace/kernel/src/caps/types.rs (line 10) - [public] Capability

```
pub enum Capability {
    Root = 0,
    Ring0 = 1,
    Driver = 2,
    User = 3,
    Admin = 4,

    PhysAlloc = 5,
    VirtMap = 6,
    Dma = 7,
    Mmap = 8,
    Protect = 9,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 99) - [public] iter_all

```
pub fn iter_all() -> impl Iterator<Item = Capability> {
    [
        Self::Root, Self::Ring0, Self::Driver, Self::User, Self::Admin,
        Self::PhysAlloc, Self::VirtMap, Self::Dma, Self::Mmap,
        Self::Protect, Self::HugePage,
        Self::Spawn, Self::Kill, Self::Debug, Self::Ptrace, Self::Sched,
        Self::IpcSend, Self::IpcRecv, Self::IpcBroadcast, Self::IpcPrivileged,
        Self::FsRead, Self::FsWrite, Self::FsCreate, Self::FsMount,
        Self::DevPci, Self::DevPort, Self::DevMmio, Self::DevIrq,
        Self::SyscallAll, Self::SyscallRestricted,
    ].into_iter()
}
```

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/caps/types.rs (line 112) - [public] category

```
pub fn category(self) -> CapCategory {
    match self as u8 {
        0..=4 => CapCategory::Ring,
        5..=10 => CapCategory::Memory,
        11..=15 => CapCategory::Process,
        16..=19 => CapCategory::Ipc,
        20..=23 => CapCategory::Fs,
        24..=27 => CapCategory::Device,
        28..=29 => CapCategory::Syscall,
        _ => CapCategory::Unknown,
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 127) - [public] CapCategory

```
pub enum CapCategory {
    Ring,
    Memory,
    Process,
    Ipc,
    Fs,
    Device,
    Syscall,
    Unknown,
}

impl CapCategory {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 154) - [public] CapabilitySet

```
pub struct CapabilitySet {
    bits: u32,
}

impl CapabilitySet {
    pub const fn empty() -> Self {
        Self { bits: 0 }
    }

    pub const fn all() -> Self {
        Self { bits: 0x3FFF_FFFF }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 215) - [public] iter

```
pub fn iter(self) -> impl Iterator<Item = Capability> {
    Capability::iter_all().filter(move |&c| self.has(c))
}

impl fmt::Debug for CapabilitySet {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "CapSet[")?;
        let mut first = true;
        for cap in self.iter() {
            if !first { write!(f, ",")?; }
            write!(f, "{}", cap.name())?;
        }
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 221) - [internal] fmt

```
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    write!(f, "CapSet[")?;
    let mut first = true;
    for cap in self.iter() {
        if !first { write!(f, ",")?; }
        write!(f, "{}", cap.name())?;
        first = false;
    }
    write!(f, "]")
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 234) - [public] CapabilityError

```
pub struct CapabilityError {
    pub required: Capability,
    pub world_id: Option<u32>,
}

impl fmt::Display for CapabilityError {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        match self.world_id {
            Some(wid) => write!(f, "capability {} required (world {})",
                                self.required.name(), wid),
            None => write!(f, "capability {} required", self.required.name()),
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 240) - [internal] fmt

```
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    match self.world_id {
        Some(wid) => write!(f, "capability {} required (world {})",
                            self.required.name(), wid),
        None => write!(f, "capability {} required", self.required.name()),
    }
}

pub type CapResult<T> = Result<T, CapabilityError>;

#[cfg(test)]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/caps/types.rs (line 256) - [internal] test_set_operations

```
fn test_set_operations() {
    let s1 = CapabilitySet::empty().add(Capability::Root);
    let s2 = CapabilitySet::empty().add(Capability::Driver);

    assert!(s1.has(Capability::Root));
    assert!(!s1.has(Capability::Driver));
    assert_eq!(s1.union(s2).count(), 2);
    assert_eq!(s1.intersect(s2).count(), 0);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/acpi.rs (line 4) - [public] CpuEntry

```
pub struct CpuEntry {
    pub apic_id: u32,
    pub enabled: bool,
}

pub struct IoApic {
    pub id: u8,
    pub address: u32,
    pub gsi_base: u32,
}

pub struct MadtInfo {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/acpi.rs (line 9) - [public] IoApic

```
pub struct IoApic {
    pub id: u8,
    pub address: u32,
    pub gsi_base: u32,
}

pub struct MadtInfo {
    pub lapic_base: u64,
    pub cpus: Vec<CpuEntry>,
    pub io_apics: Vec<IoApic>,
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/acpi.rs (line 15) - [public] MadtInfo

```
pub struct MadtInfo {
    pub lapic_base: u64,
    pub cpus: Vec<CpuEntry>,
    pub io_apics: Vec<IoApic>,
}

pub struct FadtInfo {
    pub pm1a_cnt_blk: u32,
}

pub struct Rsdp {
    pub revision: u8,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/acpi.rs (line 21) - [public] FadtInfo

```
pub struct FadtInfo {
    pub pm1a_cnt_blk: u32,
}

pub struct Rsdp {
    pub revision: u8,
    pub rsdt_addr: u32,
    pub xsdt_addr: u64,
}

const RSDP_SIGNATURE: &[u8; 8] = b"RSD PTR ";
const MADT_SIGNATURE: &[u8; 4] = b"APIC";
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/acpi.rs (line 25) - [public] Rsdp

```
pub struct Rsdp {
    pub revision: u8,
    pub rsdt_addr: u32,
    pub xsdt_addr: u64,
}

const RSDP_SIGNATURE: &[u8; 8] = b"RSD PTR ";
const MADT_SIGNATURE: &[u8; 4] = b"APIC";
const FADT_SIGNATURE: &[u8; 4] = b"FACP";

unsafe fn phys_read<T: Copy>(phys_offset: u64, phys: u64) -> T {
    ptr::read_unaligned((phys_offset + phys) as *const T)
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/acpi.rs (line 43) - [internal] checksum

```
fn checksum(bytes: &[u8]) -> bool {
    bytes.iter().fold(0u8, |acc, &b| acc.wrapping_add(b)) == 0
}

unsafe fn check_rsdp(phys_offset: u64, addr: u64) -> Option<u64> {
    if phys_slice(phys_offset, addr, 8) != RSDP_SIGNATURE {
        return None;
    }
    if !checksum(phys_slice(phys_offset, addr, 20)) {
        return None;
    }
    Some(addr)
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/acpi.rs (line 57) - [public] find_rsdp

```
pub fn find_rsdp(phys_offset: u64) -> Option<u64> {
    unsafe {
        let ebda_seg = u16::from_le_bytes([
            phys_read::<u8>(phys_offset, 0x40E),
            phys_read::<u8>(phys_offset, 0x40F),
        ]);
        if ebda_seg != 0 {
            let ebda = (ebda_seg as u64) << 4;
            let mut addr = ebda;
            while addr < ebda + 0x400 {
                if let Some(rsdp) = check_rsdp(phys_offset, addr) {
                    return Some(rsdp);
                }
            }
        }
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 20) - [internal] x2apic_msr

```
fn x2apic_msr(reg: u32) -> u32 {
    0x800 + (reg >> 4)
}

pub fn init(base_phys: u64) -> bool {
    unsafe {
        let msr = Msr::new(APIC_BASE_MSR).read();
        if msr & (1 << 10) != 0 {
            X2APIC = true;
            return true;
        }
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 24) - [public] init

```
pub fn init(base_phys: u64) -> bool {
    unsafe {
        let msr = Msr::new(APIC_BASE_MSR).read();
        if msr & (1 << 10) != 0 {
            X2APIC = true;
            return true;
        }
    }

    let base = if base_phys >= 0xFFFFF80000000000 {
        base_phys
    } else {
        match crate::mm::virt::map_device(base_phys, 0x1000) {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/lapic.rs (line 46) - [public] is_x2apic

```
pub fn is_x2apic() -> bool {
    unsafe { X2APIC }
}

pub fn read(reg: u32) -> u32 {
    unsafe {
        if X2APIC {
            Msr::new(x2apic_msr(reg)).read() as u32
        } else {
            ptr::read_volatile((LAPIC_BASE as *const u8).add(reg as usize) as *const u32)
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/lapic.rs (line 50) - [public] read

```
pub fn read(reg: u32) -> u32 {
    unsafe {
        if X2APIC {
            Msr::new(x2apic_msr(reg)).read() as u32
        } else {
            ptr::read_volatile((LAPIC_BASE as *const u8).add(reg as usize) as *const u32)
        }
    }
}

pub fn write(reg: u32, val: u32) {
    unsafe {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/lapic.rs (line 60) - [public] write

```
pub fn write(reg: u32, val: u32) {
    unsafe {
        if X2APIC {
            Msr::new(x2apic_msr(reg)).write(val as u64);
        } else {
            ptr::write_volatile((LAPIC_BASE as *mut u8).add(reg as usize) as *mut u32, val);
        }
    }
}

pub fn id() -> u32 {
    if unsafe { X2APIC } {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/lapic.rs (line 70) - [public] id

```
pub fn id() -> u32 {
    if unsafe { X2APIC } {
        read(REG_ID)
    } else {
        read(REG_ID) >> 24
    }
}

pub fn version() -> u32 {
    read(REG_VERSION)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 78) - [public] version

```
pub fn version() -> u32 {
    read(REG_VERSION)
}

pub fn enable_bsp() {
    write(REG_SVR, SPURIOUS_ENABLE | 0xFF);
    write(REG_LVT_LINT0, 0x700);
}

pub fn enable_ap() {
    write(REG_SVR, SPURIOUS_ENABLE | 0xFF);
    write(REG_LVT_LINT0, 1 << 16);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 82) - [public] enable_bsp

```
pub fn enable_bsp() {
    write(REG_SVR, SPURIOUS_ENABLE | 0xFF);
    write(REG_LVT_LINT0, 0x700);
}

pub fn enable_ap() {
    write(REG_SVR, SPURIOUS_ENABLE | 0xFF);
    write(REG_LVT_LINT0, 1 << 16);
    write(REG_LVT_LINT1, 1 << 16);
}
```

```
pub fn eoi() {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 87) - [public] enable_ap

```
pub fn enable_ap() {
    write(REG_SVR, SPURIOUS_ENABLE | 0xFF);
    write(REG_LVT_LINT0, 1 << 16);
    write(REG_LVT_LINT1, 1 << 16);
}

pub fn eoi() {
    write(REG_EOI, 0);
}

pub fn send_ipi(icr: u32, dest_apic_id: u32) {
    unsafe {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 93) - [public] eoi

```
pub fn eoi() {
    write(REG_EOI, 0);
}

pub fn send_ipi(icr: u32, dest_apic_id: u32) {
    unsafe {
        if X2APIC {
            let val = ((dest_apic_id as u64) << 32) | icr as u64;
            Msr::new(x2apic_msr(REG_ICR0)).write(val);
        } else {
            write(REG_ICR1, dest_apic_id << 24);
            write(REG_ICR0, icr);
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 97) - [public] send_ipi

```
pub fn send_ipi(icr: u32, dest_apic_id: u32) {
    unsafe {
        if X2APIC {
            let val = ((dest_apic_id as u64) << 32) | icr as u64;
            Msr::new(x2apic_msr(REG_ICR0)).write(val);
        } else {
            write(REG_ICR1, dest_apic_id << 24);
            write(REG_ICR0, icr);
        }
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 109) - [public] send_init_ipi

```
pub fn send_init_ipi() {
    send_ipi((5 << 8) | (1 << 14) | (1 << 15) | (3 << 18), 0);

    send_ipi((5 << 8) | (3 << 18), 0);
}

pub fn send_startup_ipi(apic_id: u32, vector: u8) {
    send_ipi((6 << 8) | (vector as u32), apic_id);
}
```

kernel_workspace/kernel/src/cpu/lapic.rs (line 116) - [public] send_startup_ipi

```
pub fn send_startup_ipi(apic_id: u32, vector: u8) {
    send_ipi((6 << 8) | (vector as u32), apic_id);
}
```

kernel_workspace/kernel/src/cpu/mod.rs (line 18) - [public] init_riscv

```
pub fn init_riscv() {
    riscv::init();
}

#[cfg(target_arch = "x86_64")]
pub fn total_cpus() -> u32 {
    smp::total_cpus()
}

#[cfg(target_arch = "riscv64")]
pub fn total_cpus() -> u32 {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/mod.rs (line 24) - [public] total_cpus

```
pub fn total_cpus() -> u32 {
    smp::total_cpus()
}

#[cfg(target_arch = "riscv64")]
pub fn total_cpus() -> u32 {
    1
}

//.
//.
```

kernel_workspace/kernel/src/cpu/mod.rs (line 29) - [public] total_cpus

```
pub fn total_cpus() -> u32 {
    1
}

//.
//.
```

kernel_workspace/kernel/src/cpu/riscv.rs (line 7) - [public] init

```
pub fn init() {  
  
}  
  
pub fn current_hart() -> u64 {  
    0  
}  
  
pub fn poweroff() -> ! {  
    system_reset(SRST_TYPE_SHUTDOWN)  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/riscv.rs (line 12) - [public] current_hart

```
pub fn current_hart() -> u64 {  
    0  
}  
  
pub fn poweroff() -> ! {  
    system_reset(SRST_TYPE_SHUTDOWN)  
}
```

```
pub fn reboot() -> ! {  
    system_reset(SRST_TYPE_COLD_REBOOT)  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/riscv.rs (line 17) - [public] poweroff

```
pub fn poweroff() -> ! {  
    system_reset(SRST_TYPE_SHUTDOWN)  
}  
  
pub fn reboot() -> ! {  
    system_reset(SRST_TYPE_COLD_REBOOT)  
}  
  
fn system_reset(reset_type: u32) -> ! {  
    let err: i64;  
    let _val: i64;  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/riscv.rs (line 22) - [public] reboot

```
pub fn reboot() -> ! {  
    system_reset(SRST_TYPE_COLD_REBOOT)  
}  
  
fn system_reset(reset_type: u32) -> ! {  
    let err: i64;  
    let _val: i64;  
    unsafe {  
        core::arch::asm!(  
            "ecall",  
            inlateout("a0") reset_type as usize => err,  
            inlateout("a1") 0usize => _val,  
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/riscv.rs (line 26) - [internal] system_reset

```
fn system_reset(reset_type: u32) -> ! {
    let err: i64;
    let _val: i64;
    unsafe {
        core::arch::asm!(
            "ecall",
            inlateout("a0") reset_type as usize => err,
            inlateout("a1") 0usize => _val,
            in("a2") 0usize,
            in("a6") SBI_FID_SYSTEM_RESET,
            in("a7") SBI_EID_SRST,
            options(nomem, nostack)
        );
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 639) - [public]
get_switch_count

```
pub fn get_switch_count() -> u64 {
    SWITCH_COUNT.load(Ordering::Relaxed)
}

pub fn get_fpu_switch_count() -> u64 {
    FPU_CONTEXT_SWITCHES.load(Ordering::Relaxed)
}

pub fn enable_lazy_fpu() {
    LAZY_FPU_ENABLED.store(true, Ordering::Relaxed);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 643) - [public]
get_fpu_switch_count

```
pub fn get_fpu_switch_count() -> u64 {
    FPU_CONTEXT_SWITCHES.load(Ordering::Relaxed)
}

pub fn enable_lazy_fpu() {
    LAZY_FPU_ENABLED.store(true, Ordering::Relaxed);
}

pub fn disable_lazy_fpu() {
    LAZY_FPU_ENABLED.store(false, Ordering::Relaxed);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 647) - [public]
enable_lazy_fpu

```
pub fn enable_lazy_fpu() {
    LAZY_FPU_ENABLED.store(true, Ordering::Relaxed);
}

pub fn disable_lazy_fpu() {
    LAZY_FPU_ENABLED.store(false, Ordering::Relaxed);
}

pub fn is_lazy_fpu_enabled() -> bool {
    LAZY_FPU_ENABLED.load(Ordering::Relaxed)
}

// ... (truncated, see source file for more)
```

```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 651) - [public]
disable_lazy_fpu
    pub fn disable_lazy_fpu() {
        LAZY_FPU_ENABLED.store(false, Ordering::Relaxed);
    }

    pub fn is_lazy_fpu_enabled() -> bool {
        LAZY_FPU_ENABLED.load(Ordering::Relaxed)
    }

    pub unsafe fn kernel_fpu_begin() {
        if !FPU_AVAILABLE {
            return;
        }
        // ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 655) - [public]
is_lazy_fpu_enabled
    pub fn is_lazy_fpu_enabled() -> bool {
        LAZY_FPU_ENABLED.load(Ordering::Relaxed)
    }

    pub unsafe fn kernel_fpu_begin() {
        if !FPU_AVAILABLE {
            return;
        }

        enable_fpu();

        if !LAST_FPU_TASK.is_null() {
            // ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 692) - [public]
satp_mode_name
    pub fn satp_mode_name(satp: u64) -> &'static str {
        let mode = (satp >> SATP_MODE_SHIFT) & 0xF;
        match mode {
            SATP_MODE_BARE => "Bare",
            SATP_MODE_SV39 => "Sv39",
            SATP_MODE_SV48 => "Sv48",
            SATP_MODE_SV57 => "Sv57",
            _ => "Unknown",
        }
    }

    pub fn satp_asid(satp: u64) -> u64 {
        // ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 703) - [public]
satp_asid
    pub fn satp_asid(satp: u64) -> u64 {
        (satp >> 44) & ((1 << SATP_ASID_BITS) - 1)
    }

    pub fn satp_ppn(satp: u64) -> u64 {
        satp & SATP_PPN_MASK
    }

#[cfg(test)]
mod tests {
    use super::*;

    // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 707) - [public]

satp_ppn

```
pub fn satp_ppn(satp: u64) -> u64 {
    satp & SATP_PPN_MASK
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn test_fpu_save_area_size() {
        assert_eq!(FPU_SAVE_SIZE, 256);
        assert!(FPU_SAVE_SIZE + 8 <= 512);
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 716) - [internal]

test_fpu_save_area_size

```
fn test_fpu_save_area_size() {
    assert_eq!(FPU_SAVE_SIZE, 256);
    assert!(FPU_SAVE_SIZE + 8 <= 512);
}

#[test]
fn test_satp_constants() {
    assert_eq!(SATP_MODE_SHIFT, 60);
    assert_eq!(SATP_ASID_BITS, 16);
    assert_eq!(SATP_MODE_SV39, 8);
    assert_eq!(SATP_MODE_SV48, 9);
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 722) - [internal]

test_satp_constants

```
fn test_satp_constants() {
    assert_eq!(SATP_MODE_SHIFT, 60);
    assert_eq!(SATP_ASID_BITS, 16);
    assert_eq!(SATP_MODE_SV39, 8);
    assert_eq!(SATP_MODE_SV48, 9);
}

#[test]
fn test_sstatus_fs_mask() {
    assert_eq!(SSTATUS_FS_MASK, 0x3 << 13);
    assert_eq!(SSTATUS_FS_OFF, 0);
    assert_eq!(SSTATUS_FS_INITIAL, 1);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 730) - [internal]

test_sstatus_fs_mask

```
fn test_sstatus_fs_mask() {
    assert_eq!(SSTATUS_FS_MASK, 0x3 << 13);
    assert_eq!(SSTATUS_FS_OFF, 0);
    assert_eq!(SSTATUS_FS_INITIAL, 1);
    assert_eq!(SSTATUS_FS_CLEAN, 2);
    assert_eq!(SSTATUS_FS_DIRTY, 3);
}

#[test]
fn test_satp_mode_name() {
    assert_eq!(satp_mode_name(SATP_MODE_BARE << SATP_MODE_SHIFT), "Bare");
    assert_eq!(satp_mode_name(SATP_MODE_SV39 << SATP_MODE_SHIFT), "Sv39");
    // ... (truncated, see source file for more)
}
```



```

kernel_workspace/kernel/src/cpu/scheduler/arch/risc_v/context_swich.rs (line 739) - [internal]
test_satp_mode_name
    fn test_satp_mode_name() {
        assert_eq!(satp_mode_name(SATP_MODE_BARE << SATP_MODE_SHIFT), "Bare");
        assert_eq!(satp_mode_name(SATP_MODE_SV39 << SATP_MODE_SHIFT), "Sv39");
        assert_eq!(satp_mode_name(SATP_MODE_SV48 << SATP_MODE_SHIFT), "Sv48");
        assert_eq!(satp_mode_name(SATP_MODE_SV57 << SATP_MODE_SHIFT), "Sv57");
    }
}

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 45) - [internal]
XSaveArea
    struct XSaveArea {
        data: [u8; XSAVE_AREA_SIZE],
    }

    impl XSaveArea {
        const fn new() -> Self {
            Self { data: [0; XSAVE_AREA_SIZE] }
        }
    }

    #[repr(C)]
    struct ExtendedContext {
        // ... (truncated, see source file for more)
    }

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 56) - [internal]
ExtendedContext
    struct ExtendedContext {
        xsave_area: XSaveArea,
        xcomp_bv: u64,
        header_bv: u64,
    }

    #[inline(always)]
    unsafe fn read_cr3() -> u64 {
        let cr3: u64;
        asm!("mov {0}, cr3", out(reg) cr3, options(nomem, nostack, preserves_flags));
        cr3
    }

    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 458) - [internal]
TssStruct
    struct TssStruct {
        reserved0: u32,
        rsp0: u64,
        rsp1: u64,
        rsp2: u64,
        reserved1: u64,
        ist1: u64,
        ist2: u64,
        ist3: u64,
        ist4: u64,
        ist5: u64,
        ist6: u64,
        // ... (truncated, see source file for more)
    }

```

```

kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 731) - [public]
get_switch_count
    pub fn get_switch_count() -> u64 {
        SWITCH_COUNT.load(Ordering::Relaxed)
    }

    pub fn get_fpu_switch_count() -> u64 {
        FPU_CONTEXT_SWITCHES.load(Ordering::Relaxed)
    }

    pub fn enable_lazy_fpu() {
        LAZY_FPU_ENABLED.store(true, Ordering::Relaxed);
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 735) - [public]
get_fpu_switch_count
    pub fn get_fpu_switch_count() -> u64 {
        FPU_CONTEXT_SWITCHES.load(Ordering::Relaxed)
    }

    pub fn enable_lazy_fpu() {
        LAZY_FPU_ENABLED.store(true, Ordering::Relaxed);
    }

    pub fn disable_lazy_fpu() {
        LAZY_FPU_ENABLED.store(false, Ordering::Relaxed);
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 739) - [public]
enable_lazy_fpu
    pub fn enable_lazy_fpu() {
        LAZY_FPU_ENABLED.store(true, Ordering::Relaxed);
    }

    pub fn disable_lazy_fpu() {
        LAZY_FPU_ENABLED.store(false, Ordering::Relaxed);
    }

    pub fn is_lazy_fpu_enabled() -> bool {
        LAZY_FPU_ENABLED.load(Ordering::Relaxed)
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 743) - [public]
disable_lazy_fpu
    pub fn disable_lazy_fpu() {
        LAZY_FPU_ENABLED.store(false, Ordering::Relaxed);
    }

    pub fn is_lazy_fpu_enabled() -> bool {
        LAZY_FPU_ENABLED.load(Ordering::Relaxed)
    }

    pub unsafe fn kernel_fpu_begin() {
        clts();

        if !LAST_FPU_TASK.is_null() {
            // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 747) - [public]
is_lazy_fpu_enabled

```
pub fn is_lazy_fpu_enabled() -> bool {
    LAZY_FPU_ENABLED.load(Ordering::Relaxed)
}

pub unsafe fn kernel_fpu_begin() {
    clts();

    if !LAST_FPU_TASK.is_null() {
        save_fpu_state(LAST_FPU_TASK);
        LAST_FPU_TASK = core::ptr::null_mut();
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 769) - [internal]
test_xsave_area_alignment

```
fn test_xsave_area_alignment() {
    assert_eq!(core::mem::align_of::<XSaveArea>(), 64);
}

#[test]
fn test_msr_constants() {
    assert_eq!(MSR_FS_BASE, 0xC0000100);
    assert_eq!(MSR_GS_BASE, 0xC0000101);
    assert_eq!(MSR_KERNEL_GS_BASE, 0xC0000102);
}

#[test]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 774) - [internal]
test_msr_constants

```
fn test_msr_constants() {
    assert_eq!(MSR_FS_BASE, 0xC0000100);
    assert_eq!(MSR_GS_BASE, 0xC0000101);
    assert_eq!(MSR_KERNEL_GS_BASE, 0xC0000102);
}

#[test]
fn test_cr3_masks() {
    assert_eq!(CR3_PCID_MASK, 0xFFF);
    assert_eq!(CR3_NO_FLUSH, 1 << 63);
}

}
```

kernel_workspace/kernel/src/cpu/scheduler/arch/x86_64/context_swich.rs (line 781) - [internal]
test_cr3_masks

```
fn test_cr3_masks() {
    assert_eq!(CR3_PCID_MASK, 0xFFF);
    assert_eq!(CR3_NO_FLUSH, 1 << 63);
}

}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 49) - [public] register_runqueue

```
pub fn register_runqueue(cpu: u32, rq: *mut RunQueue) {
    if (cpu as usize) < MAX_CPUS {
        RQ_REGISTRY[cpu as usize].store(rq, Ordering::Release);
    }
}

pub fn register_apic_id(cpu: u32, apic_id: u32) {
    if (cpu as usize) < MAX_CPUS {
        APIC_ID_TABLE[cpu as usize].store(apic_id, Ordering::Release);
    }
}

// ... (truncated, see source file for more)
```

```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 55) - [public] register_apic_id
pub fn register_apic_id(cpu: u32, apic_id: u32) {
    if (cpu as usize) < MAX_CPUS {
        APIC_ID_TABLE[cpu as usize].store(apic_id, Ordering::Release);
    }
}

pub fn snapshot_registry() -> [*mut RunQueue; MAX_CPUS] {
    let mut out = [ptr::null_mut(); MAX_CPUS];
    for i in 0..MAX_CPUS {
        out[i] = RQ_REGISTRY[i].load(Ordering::Acquire);
    }
    out
    // ... (truncated, see source file for more)
}

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 61) - [public] snapshot_registry
pub fn snapshot_registry() -> [*mut RunQueue; MAX_CPUS] {
    let mut out = [ptr::null_mut(); MAX_CPUS];
    for i in 0..MAX_CPUS {
        out[i] = RQ_REGISTRY[i].load(Ordering::Acquire);
    }
    out
}

fn cpu_to_apic_id(cpu: u32) -> Option<u32> {
    if (cpu as usize) >= MAX_CPUS {
        return None;
    }
    // ... (truncated, see source file for more)
}

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 69) - [internal] cpu_to_apic_id
fn cpu_to_apic_id(cpu: u32) -> Option<u32> {
    if (cpu as usize) >= MAX_CPUS {
        return None;
    }
    let id = APIC_ID_TABLE[cpu as usize].load(Ordering::Acquire);
    if id == u32::MAX {
        None
    } else {
        Some(id)
    }
}

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 81) - [public] set_tsc_frequency
pub fn set_tsc_frequency(hz: u64) {
    TSC_HZ.store(hz, Ordering::Release);
}

pub fn tsc_frequency() -> u64 {
    TSC_HZ.load(Ordering::Acquire)
}

pub fn booted_cpu_count() -> u32 {
    BOOTED_CPUS.load(Ordering::Acquire)
}

// ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 85) - [public] tsc_frequency

```
pub fn tsc_frequency() -> u64 {
    TSC_HZ.load(Ordering::Acquire)
}

pub fn booted_cpu_count() -> u32 {
    BOOTED_CPUS.load(Ordering::Acquire)
}

// -----
// Port I/O – uprzywilejowane, więc wyliczenie prawdziwe na jędrzym x86_64
// -----

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 89) - [public] booted_cpu_count

```
pub fn booted_cpu_count() -> u32 {
    BOOTED_CPUS.load(Ordering::Acquire)
}

// -----
// Port I/O – uprzywilejowane, więc wyliczenie prawdziwe na jędrzym x86_64
// -----

#[cfg(all(target_arch = "x86_64", not(test)))]
unsafe fn out_u8(port: u16, val: u8) {
    core::arch::asm!("out dx, al", in("dx") port, in("al") val, options(nomem, nostack, preserves_flags))
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 175) - [internal] ns_from_tsc

```
fn ns_from_tsc(tsc_delta: u64, hz: u64) -> u64 {
    if hz == 0 {
        0
    } else {
        ((tsc_delta as u128) * 1_000_000_000u128 / hz as u128) as u64
    }
}

pub fn now_ns() -> u64 {
    ns_from_tsc(unsafe { rdtsc() }, tsc_frequency())
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 183) - [public] now_ns

```
pub fn now_ns() -> u64 {
    ns_from_tsc(unsafe { rdtsc() }, tsc_frequency())
}

/// Kalibruje TSC względem kanału 2 PIT (jak w prawdziwym jądrze – PIT
/// jest wolniejszy, ale zawsze obecny). Wymaga wyliczonych przerwań.
pub unsafe fn calibrate_tsc_via_pit(calib_ms: u32) -> u64 {
    const PIT_GATE_PORT: u16 = 0x61;
    const PIT_CMD_PORT: u16 = 0x43;
    const PIT_CH2_DATA_PORT: u16 = 0x42;

    let count = ((PIT_FREQUENCY_HZ as u64 * calib_ms as u64) / 1000) as u16;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 217) - [internal]

timer_count_for_hz

```
fn timer_count_for_hz(hz: u64) -> u32 {
    if hz == 0 {
        1_000_000
    } else {
        cmp_max_1((hz / TICK_HZ) as u32)
    }
}

fn cmp_max_1(v: u32) -> u32 {
    if v == 0 {
        1
    } else {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 225) - [internal] cmp_max_1

```
fn cmp_max_1(v: u32) -> u32 {
    if v == 0 {
        1
    } else {
        v
    }
}

fn default_timer_count() -> u32 {
    timer_count_for_hz(tsc_frequency())
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 233) - [internal]

default_timer_count

```
fn default_timer_count() -> u32 {
    timer_count_for_hz(tsc_frequency())
}

// -----
// Local APIC MMIO – nieodzwzorowane w przestrzeni testowej, więc stub
// dla `test`/architektur innych niż x86_64
// -----

#[cfg(all(target_arch = "x86_64", not(test)))]
unsafe fn apic_write(reg: usize, value: u32) {
    ptr::write_volatile((LOCAL_APIC_BASE + reg) as *mut u32, value);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 270) - [public] current_cpu_id

```
pub fn current_cpu_id() -> u32 {
    unsafe {
        let apic_id = apic_read(APIC_REG_ID) >> 24;
        for cpu in 0..MAX_CPUS as u32 {
            if cpu_to_apic_id(cpu) == Some(apic_id) {
                return cpu;
            }
        }
        CPU_NONE
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 338) - [public] IdtEntry

```
pub struct IdtEntry {
    offset_low: u16,
    selector: u16,
    ist: u8,
    type_attr: u8,
    offset_mid: u16,
    offset_high: u32,
    zero: u32,
}

impl IdtEntry {
    pub const fn missing() -> Self {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 353) - [public] set

```
pub fn set(&mut self, handler: usize, selector: u16, ist: u8, type_attr: u8) {
    self.offset_low = (handler & 0xFFFF) as u16;
    self.offset_mid = ((handler >> 16) & 0xFFFF) as u16;
    self.offset_high = ((handler >> 32) & 0xFFFF_FFFF) as u32;
    self.selector = selector;
    self.ist = ist & 0x7;
    self.type_attr = type_attr;
    self.zero = 0;
}

pub fn is_present(&self) -> bool {
    self.type_attr & 0x80 != 0
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 363) - [public] is_present

```
pub fn is_present(&self) -> bool {
    self.type_attr & 0x80 != 0
}

#[repr(C, packed)]
struct IdtDescriptor {
    limit: u16,
    base: u64,
}
```

```
static mut IDT: [IdtEntry; IDT_SIZE] = [IdtEntry::missing(); IDT_SIZE];
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 369) - [internal] IdtDescriptor

```
struct IdtDescriptor {
    limit: u16,
    base: u64,
}

static mut IDT: [IdtEntry; IDT_SIZE] = [IdtEntry::missing(); IDT_SIZE];

pub unsafe fn install_gate(vector: u8, handler: usize, selector: u16, ist: u8, type_attr: u8) {
    let entry_ptr = ptr::addr_of_mut!(IDT[vector as usize]);
    (*entry_ptr).set(handler, selector, ist, type_attr);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 437) - [internal]

arch_context_switch

```
fn arch_context_switch(prev_ctx: *mut CpuContext, next_ctx: *const CpuContext);
}

pub unsafe fn context_switch(rq: &mut RunQueue) {
    let flags = rq.lock.lock_irqsave();
    let (prev, next) = rq.schedule();
    rq.lock.unlock_irqrestore(flags);

    if core::ptr::eq(prev, next) {
        return;
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 462) - [internal] maybe_balance

```
fn maybe_balance(cpu: u32, registry: &[*mut RunQueue; MAX_CPUS]) {
    let rq_ptr = registry[cpu as usize];
    if rq_ptr.is_null() {
        return;
    }
    unsafe {
        let rq = &*rq_ptr;
        let now = rq.clock.load(Ordering::Relaxed);
        let next = rq.next_balance.load(Ordering::Relaxed);
        if now >= next {
            rq.next_balance.store(now + BALANCE_INTERVAL_NS, Ordering::Relaxed);
            smp::load_balance(registry);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 587) - [internal]

registry_defaults_to_null_for_untouched_cpu

```
fn registry_defaults_to_null_for_untouched_cpu() {
    let snap = snapshot_registry();
    assert!(snap[MAX_CPUS - 1].is_null());
}

#[test]
fn apic_id_table_defaults_to_none_then_roundtrips() {
    assert_eq!(cpu_to_apic_id(2), None);
    register_apic_id(2, 9);
    assert_eq!(cpu_to_apic_id(2), Some(9));
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 593) - [internal]

apic_id_table_defaults_to_none_then_roundtrips

```
fn apic_id_table_defaults_to_none_then_roundtrips() {
    assert_eq!(cpu_to_apic_id(2), None);
    register_apic_id(2, 9);
    assert_eq!(cpu_to_apic_id(2), Some(9));
}

#[test]
fn register_runqueue_ignores_out_of_range_cpu() {
    register_runqueue(MAX_CPUS as u32 + 10, ptr::null_mut());
}

#[test]
// ... (truncated, see source file for more)
```



```

kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 600) - [internal]
register_runqueue_ignores_out_of_range_cpu

    fn register_runqueue_ignores_out_of_range_cpu() {
        register_runqueue(MAX_CPUS as u32 + 10, ptr::null_mut());
    }

#[test]
fn ns_from_tsc_is_zero_without_calibration() {
    assert_eq!(ns_from_tsc(1_000_000, 0), 0);
}

#[test]
fn ns_from_tsc_scales_correctly() {
    // 1 GHz zegar: 1_000_000 cykli = 1_000_000 ns.
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 605) - [internal]
ns_from_tsc_is_zero_without_calibration

    fn ns_from_tsc_is_zero_without_calibration() {
        assert_eq!(ns_from_tsc(1_000_000, 0), 0);
    }

#[test]
fn ns_from_tsc_scales_correctly() {
    // 1 GHz zegar: 1_000_000 cykli = 1_000_000 ns.
    assert_eq!(ns_from_tsc(1_000_000, 1_000_000_000), 1_000_000);
    // 2 GHz zegar: te same cykle to po nowa czasu.
    assert_eq!(ns_from_tsc(1_000_000, 2_000_000_000), 500_000);
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 610) - [internal]
ns_from_tsc_scales_correctly

    fn ns_from_tsc_scales_correctly() {
        // 1 GHz zegar: 1_000_000 cykli = 1_000_000 ns.
        assert_eq!(ns_from_tsc(1_000_000, 1_000_000_000), 1_000_000);
        // 2 GHz zegar: te same cykle to po nowa czasu.
        assert_eq!(ns_from_tsc(1_000_000, 2_000_000_000), 500_000);
    }

#[test]
fn timer_count_for_hz_has_sane_fallback() {
    assert_eq!(timer_count_for_hz(0), 1_000_000);
    assert_eq!(timer_count_for_hz(1_000_000_000), 1_000_000);
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 618) - [internal]
timer_count_for_hz_has_sane_fallback

    fn timer_count_for_hz_has_sane_fallback() {
        assert_eq!(timer_count_for_hz(0), 1_000_000);
        assert_eq!(timer_count_for_hz(1_000_000_000), 1_000_000);
    }

#[test]
fn timer_count_for_hz_never_returns_zero() {
    assert_eq!(timer_count_for_hz(TICK_HZ / 2), 1);
}

#[test]
fn tlb_pending_flag_sets_and_clears() {
    // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 624) - [internal]

timer_count_for_hz_never_returns_zero

```
fn timer_count_for_hz_never_returns_zero() {
    assert_eq!(timer_count_for_hz(TICK_HZ / 2), 1);
}

#[test]
fn tlb_pending_flag_sets_and_clears() {
    let idx = MAX_CPUS - 2;
    TLB_SHOOTDOWN_PENDING[idx].store(true, Ordering::Release);
    assert!(TLB_SHOOTDOWN_PENDING[idx].swap(false, Ordering::AcqRel));
    assert!(TLB_SHOOTDOWN_PENDING[idx].load(Ordering::Acquire));
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 629) - [internal]

tlb_pending_flag_sets_and_clears

```
fn tlb_pending_flag_sets_and_clears() {
    let idx = MAX_CPUS - 2;
    TLB_SHOOTDOWN_PENDING[idx].store(true, Ordering::Release);
    assert!(TLB_SHOOTDOWN_PENDING[idx].swap(false, Ordering::AcqRel));
    assert!(TLB_SHOOTDOWN_PENDING[idx].load(Ordering::Acquire));
}

#[test]
fn idt_entry_encodes_and_reports_present() {
    let mut e = IdtEntry::missing();
    assert!(e.is_present());
    e.set(0xDEAD_BEEF_1234, 0x08, 1, GATE_INTERRUPT);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 637) - [internal]

idt_entry_encodes_and_reports_present

```
fn idt_entry_encodes_and_reports_present() {
    let mut e = IdtEntry::missing();
    assert!(e.is_present());
    e.set(0xDEAD_BEEF_1234, 0x08, 1, GATE_INTERRUPT);
    assert!(e.is_present());
}

#[test]
fn idt_entry_missing_is_not_present() {
    assert!(IdtEntry::missing().is_present());
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 645) - [internal]

idt_entry_missing_is_not_present

```
fn idt_entry_missing_is_not_present() {
    assert!(IdtEntry::missing().is_present());
}

#[cfg(target_arch = "x86_64")]
#[test]
fn cpuid_leaf_zero_reports_a_nonzero_max_leaf() {
    unsafe {
        let (eax, _, _, _) = cpuid(0);
        assert!(eax >= 1);
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 651) - [internal]

cpuid_leaf_zero_reports_a_nonzero_max_leaf

```
fn cpuid_leaf_zero_reports_a_nonzero_max_leaf() {
    unsafe {
        let (eax, _, _, _) = cpuid(0);
        assert!(eax >= 1);
    }
}

#[cfg(target_arch = "x86_64")]
#[test]
fn rdtsc_does_not_go_backwards_across_two_reads() {
    unsafe {
        let a = rdtsc();
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 660) - [internal]

rdtsc_does_not_go_backwards_across_two_reads

```
fn rdtsc_does_not_go_backwards_across_two_reads() {
    unsafe {
        let a = rdtsc();
        let b = rdtsc();
        assert!(b >= a);
    }
}

#[cfg(not(target_arch = "x86_64"))]
#[test]
fn cpuid_and_rdtsc_are_zero_off_x86_64() {
    unsafe {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 670) - [internal]

cpuid_and_rdtsc_are_zero_off_x86_64

```
fn cpuid_and_rdtsc_are_zero_off_x86_64() {
    unsafe {
        assert_eq!(cpuid(0), (0, 0, 0, 0));
        assert_eq!(rdtsc(), 0);
    }
}

#[test]
fn set_and_read_tsc_frequency_is_stable_within_this_test() {
    set_tsc_frequency(3_000_000_000);
    assert_eq!(tsc_frequency(), 3_000_000_000);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 678) - [internal]

set_and_read_tsc_frequency_is_stable_within_this_test

```
fn set_and_read_tsc_frequency_is_stable_within_this_test() {
    set_tsc_frequency(3_000_000_000);
    assert_eq!(tsc_frequency(), 3_000_000_000);
}

#[test]
fn booted_cpu_count_starts_at_or_above_zero() {
    assert!(booted_cpu_count() < u32::MAX);
}
}
```

kernel_workspace/kernel/src/cpu/scheduler/arch_hooks.rs (line 684) - [internal]

booted_cpu_count_starts_at_or_above_zero

```
fn booted_cpu_count_starts_at_or_above_zero() {
    assert!(booted_cpu_count() < u32::MAX);
}
}
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 12) - [public] SchedClassOps

```
pub struct SchedClassOps {
    pub name: &'static str,
    pub class: SchedClass,
    pub next: Option<&'static SchedClassOps>,

    pub enqueue_task: unsafe fn(&mut RunQueue, *mut TaskStruct, EnqueueFlags),
    pub dequeue_task: unsafe fn(&mut RunQueue, *mut TaskStruct, DequeueFlags),
    pub yield_task: unsafe fn(&mut RunQueue, *mut TaskStruct),
    pub pick_next_task: unsafe fn(&mut RunQueue) -> *mut TaskStruct,
    pub check_preempt_curr: unsafe fn(&RunQueue, *mut TaskStruct) -> bool,
    pub task_tick: unsafe fn(&mut RunQueue, *mut TaskStruct),
    pub charge_exec: unsafe fn(&mut RunQueue, *mut TaskStruct, u64, u64),
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 336) - [public] class_of

```
// -----
// Dispatch
// -----

pub fn class_of(class: SchedClass) -> &'static SchedClassOps {
    match class {
        SchedClass::Stop => &STOP_SCHED_CLASS,
        SchedClass::Deadline => &DL_SCHED_CLASS,
        SchedClass::RealTime => &RT_SCHED_CLASS,
        SchedClass::Fair => &FAIR_SCHED_CLASS,
        SchedClass::Idle => &IDLE_SCHED_CLASS,
    }
}

pub unsafe fn pick_next(rq: &mut RunQueue) -> *mut TaskStruct {
    let mut cursor: &'static SchedClassOps = &STOP_SCHED_CLASS;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 514) - [internal] make_idle

```
fn make_idle(pid: TaskId) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, SchedPolicy::Idle, 0);
    t
}

fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, policy, nice);
    t
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 520) - [internal] make_task

```
fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, policy, nice);
    t
}

fn ptr_of(t: &mut TaskStruct) -> *mut TaskStruct {
    t as *mut TaskStruct
}

#[test]
fn chain_is_ordered_stop_dl_rt_fair_idle() {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 526) - [internal] ptr_of

```
fn ptr_of(t: &mut TaskStruct) -> *mut TaskStruct {
    t as *mut TaskStruct
}

#[test]
fn chain_is_ordered_stop_dl_rt_fair_idle() {
    assert_eq!(STOP_SCHED_CLASS.class, SchedClass::Stop);
    assert!(core::ptr::eq(STOP_SCHED_CLASS.next.unwrap(), &DL_SCHED_CLASS));
    assert_eq!(DL_SCHED_CLASS.class, SchedClass::Deadline);
    assert!(core::ptr::eq(DL_SCHED_CLASS.next.unwrap(), &RT_SCHED_CLASS));
    assert_eq!(RT_SCHED_CLASS.class, SchedClass::RealTime);
    assert!(core::ptr::eq(RT_SCHED_CLASS.next.unwrap(), &FAIR_SCHED_CLASS));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 531) - [internal]

chain_is_ordered_stop_dl_rt_fair_idle

```
fn chain_is_ordered_stop_dl_rt_fair_idle() {
    assert_eq!(STOP_SCHED_CLASS.class, SchedClass::Stop);
    assert!(core::ptr::eq(STOP_SCHED_CLASS.next.unwrap(), &DL_SCHED_CLASS));
    assert_eq!(DL_SCHED_CLASS.class, SchedClass::Deadline);
    assert!(core::ptr::eq(DL_SCHED_CLASS.next.unwrap(), &RT_SCHED_CLASS));
    assert_eq!(RT_SCHED_CLASS.class, SchedClass::RealTime);
    assert!(core::ptr::eq(RT_SCHED_CLASS.next.unwrap(), &FAIR_SCHED_CLASS));
    assert_eq!(FAIR_SCHED_CLASS.class, SchedClass::Fair);
    assert!(core::ptr::eq(FAIR_SCHED_CLASS.next.unwrap(), &IDLE_SCHED_CLASS));
    assert_eq!(IDLE_SCHED_CLASS.class, SchedClass::Idle);
    assert!(IDLE_SCHED_CLASS.next.is_none());
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 545) - [internal]

class_of_maps_every_variant_correctly

```
fn class_of_maps_every_variant_correctly() {
    assert!(core::ptr::eq(class_of(SchedClass::Stop), &STOP_SCHED_CLASS));
    assert!(core::ptr::eq(class_of(SchedClass::Deadline), &DL_SCHED_CLASS));
    assert!(core::ptr::eq(class_of(SchedClass::RealTime), &RT_SCHED_CLASS));
    assert!(core::ptr::eq(class_of(SchedClass::Fair), &FAIR_SCHED_CLASS));
    assert!(core::ptr::eq(class_of(SchedClass::Idle), &IDLE_SCHED_CLASS));
}

#[test]
fn pick_next_walks_the_chain_in_priority_order() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 554) - [internal]

pick_next_walks_the_chain_in_priority_order

```
fn pick_next_walks_the_chain_in_priority_order() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut fair_task = make_task(1, SchedPolicy::Normal, 0);
    let mut rt_task = make_task(2, SchedPolicy::Fifo, 0);
    rt_task.prio = 10;
    let mut dl_task = make_task(3, SchedPolicy::Deadline, 0);
    dl_task.dl.dl_runtime = 1000;
    dl_task.dl.dl_deadline = 100_000;
    dl_task.dl.dl_period = 100_000;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 596) - [internal]

rt_get_rr_interval_is_nonzero_only_for_round_robin

```
fn rt_get_rr_interval_is_nonzero_only_for_round_robin() {
    let fifo = make_task(1, SchedPolicy::Fifo, 0);
    let rr = make_task(2, SchedPolicy::RoundRobin, 0);
    unsafe {
        assert_eq!(get_rr_interval(&fifo as *const _), 0);
        assert!(get_rr_interval(&rr as *const _) > 0);
    }
}

#[test]
fn fair_and_deadline_and_idle_have_zero_rr_interval() {
    let fair = make_task(1, SchedPolicy::Normal, 0);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 606) - [internal]

fair_and_deadline_and_idle_have_zero_rr_interval

```
fn fair_and_deadline_and_idle_have_zero_rr_interval() {
    let fair = make_task(1, SchedPolicy::Normal, 0);
    let dl = make_task(2, SchedPolicy::Deadline, 0);
    let idle = make_idle(3);
    unsafe {
        assert_eq!(get_rr_interval(&fair as *const _), 0);
        assert_eq!(get_rr_interval(&dl as *const _), 0);
        assert_eq!(get_rr_interval(&idle as *const _), 0);
    }
}

#[test]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 618) - [internal]

check_preempt_cross_class_ignores_intra_class_ops

```
fn check_preempt_cross_class_ignores_intra_class_ops() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut fair_task = make_task(1, SchedPolicy::Normal, 0);
    let mut rt_task = make_task(2, SchedPolicy::Fifo, 0);
    rt_task.prio = 5;

    unsafe {
        let flags = rq.lock.lock_irqsave();
        enqueue(&mut rq, ptr_of(&mut fair_task), EnqueueFlags::ENQUEUE_NEW);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 639) - [internal]

switched_to_reschedules_when_new_class_outranks_current

```
fn switched_to_reschedules_when_new_class_outranks_current() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut fair_task = make_task(1, SchedPolicy::Normal, 0);
    let mut promoted = make_task(2, SchedPolicy::Normal, 0);

    unsafe {
        let flags = rq.lock.lock_irqsave();
        enqueue(&mut rq, ptr_of(&mut fair_task), EnqueueFlags::ENQUEUE_NEW);
        rq.set_curr_task(ptr_of(&mut fair_task));
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 664) - [internal]
change_task_class_moves_task_between_underlying_queues

```
fn change_task_class_moves_task_between_underlying_queues() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut t = make_task(1, SchedPolicy::Normal, 0);

    unsafe {
        let flags = rq.lock.lock_irqsave();
        enqueue(&mut rq, ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);
        assert_eq!(rq.fair.nr_running, 1);
        assert_eq!(rq.rt.nr_running, 0);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 688) - [internal]
change_task_class_on_non_queued_task_does_not_touch_queues

```
fn change_task_class_on_non_queued_task_does_not_touch_queues() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();
    let mut t = make_task(1, SchedPolicy::Normal, 0);

    unsafe {
        let flags = rq.lock.lock_irqsave();
        change_task_class(&mut rq, ptr_of(&mut t), SchedPolicy::Fifo);
        assert_eq!(rq.fair.nr_running, 0);
        assert_eq!(rq.rt.nr_running, 0);
        assert!(!(*ptr_of(&mut t)).se.on_rq);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 706) - [internal]
rt_task_tick_only_requeues_round_robin_on_slice_expiry

```
fn rt_task_tick_only_requeues_round_robin_on_slice_expiry() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut fifo_task = make_task(1, SchedPolicy::Fifo, 0);
    fifo_task.prio = 10;

    unsafe {
        let flags = rq.lock.lock_irqsave();
        enqueue(&mut rq, ptr_of(&mut fifo_task), EnqueueFlags::ENQUEUE_NEW);
        rq.set_curr_task(ptr_of(&mut fifo_task));
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/class.rs (line 727) - [internal]
rt_task_tick_marks_resched_when_round_robin_slice_expires

```
fn rt_task_tick_marks_resched_when_round_robin_slice_expires() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut rr_task = make_task(1, SchedPolicy::RoundRobin, 0);
    rr_task.prio = 10;
    rr_task.rt.time_slice = 0;

    unsafe {
        let flags = rq.lock.lock_irqsave();
        enqueue(&mut rq, ptr_of(&mut rr_task), EnqueueFlags::ENQUEUE_NEW);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 750) - [internal]

dl_task_tick_dequeues_when_throttled

```
fn dl_task_tick_dequeues_when_throttled() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut t = make_task(1, SchedPolicy::Deadline, 0);
    t.dl.dl_runtime = 1000;
    t.dl.dl_deadline = 100_000;
    t.dl.dl_period = 100_000;

    unsafe {
        let flags = rq.lock.lock_irqsave();
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/class.rs (line 776) - [internal]

fair_charge_dispatches_to_underlying_rqfair

```
fn fair_charge_dispatches_to_underlying_rqfair() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();
    let mut t = make_task(1, SchedPolicy::Normal, 0);

    unsafe {
        let flags = rq.lock.lock_irqsave();
        enqueue(&mut rq, ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);
        let before = (*ptr_of(&mut t)).se.vruntime;
        charge(&mut rq, ptr_of(&mut t), 1_000_000, 1_000_000);
        assert!((*ptr_of(&mut t)).se.vruntime > before);
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 27) - [public] Bitmap

```
pub struct Bitmap<const WORDS: usize> {
    words: [u64; WORDS],
}

impl<const WORDS: usize> Bitmap<WORDS> {
    pub const CAPACITY: usize = WORDS * BITS_PER_WORD;

    pub const fn new() -> Self {
        Self { words: [0; WORDS] }
    }

    pub const fn filled() -> Self {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 43) - [public] set

```
pub fn set(&mut self, bit: usize) {
    if bit < Self::CAPACITY {
        self.words[word_of(bit)] |= mask_of(bit);
    }
}

#[inline]
pub fn clear(&mut self, bit: usize) {
    if bit < Self::CAPACITY {
        self.words[word_of(bit)] &= !mask_of(bit);
    }
}

// ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 50) - [public] clear

```
pub fn clear(&mut self, bit: usize) {
    if bit < Self::CAPACITY {
        self.words[word_of(bit)] &= !mask_of(bit);
    }
}

#[inline]
pub fn toggle(&mut self, bit: usize) {
    if bit < Self::CAPACITY {
        self.words[word_of(bit)] ^= mask_of(bit);
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 57) - [public] toggle

```
pub fn toggle(&mut self, bit: usize) {
    if bit < Self::CAPACITY {
        self.words[word_of(bit)] ^= mask_of(bit);
    }
}

#[inline]
pub fn test(&self, bit: usize) -> bool {
    bit < Self::CAPACITY && self.words[word_of(bit)] & mask_of(bit) != 0
}

pub fn set_range(&mut self, start: usize, end: usize) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 64) - [public] test

```
pub fn test(&self, bit: usize) -> bool {
    bit < Self::CAPACITY && self.words[word_of(bit)] & mask_of(bit) != 0
}

pub fn set_range(&mut self, start: usize, end: usize) {
    let end = end.min(Self::CAPACITY);
    let mut bit = start;
    while bit < end {
        self.set(bit);
        bit += 1;
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 68) - [public] set_range

```
pub fn set_range(&mut self, start: usize, end: usize) {
    let end = end.min(Self::CAPACITY);
    let mut bit = start;
    while bit < end {
        self.set(bit);
        bit += 1;
    }
}

pub fn is_empty(&self) -> bool {
    self.words.iter().all(|w| *w == 0)
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 77) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.words.iter().all(|w| *w == 0)
}

pub fn is_full(&self) -> bool {
    self.words.iter().all(|w| *w == u64::MAX)
}

pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.count_ones()).sum()
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 81) - [public] is_full

```
pub fn is_full(&self) -> bool {
    self.words.iter().all(|w| *w == u64::MAX)
}

pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.count_ones()).sum()
}

pub fn find_first_set(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != 0 {
            return Some(idx * BITS_PER_WORD + word.trailing_zeros() as usize);
        }
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 85) - [public] weight

```
pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.count_ones()).sum()
}

pub fn find_first_set(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != 0 {
            return Some(idx * BITS_PER_WORD + word.trailing_zeros() as usize);
        }
    }
    None
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 89) - [public] find_first_set

```
pub fn find_first_set(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != 0 {
            return Some(idx * BITS_PER_WORD + word.trailing_zeros() as usize);
        }
    }
    None
}

pub fn find_first_zero(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != u64::MAX {
            return Some(idx * BITS_PER_WORD + word.trailing_zeros() as usize);
        }
    }
    None
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 98) - [public]

find_first_zero

```
pub fn find_first_zero(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != u64::MAX {
            let bit = idx * BITS_PER_WORD + (!word).trailing_zeros() as usize;
            if bit < Self::CAPACITY {
                return Some(bit);
            }
        }
        return None;
    }
}
None
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 111) - [public]

find_next_set

```
pub fn find_next_set(&self, after: usize) -> Option<usize> {
    let mut bit = after + 1;
    while bit < Self::CAPACITY {
        let idx = word_of(bit);
        let shift = bit % BITS_PER_WORD;
        let masked = self.words[idx] & (u64::MAX << shift);
        if masked != 0 {
            let found = idx * BITS_PER_WORD + masked.trailing_zeros() as usize;
            return if found < Self::CAPACITY { Some(found) } else { None };
        }
        bit = (idx + 1) * BITS_PER_WORD;
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 126) - [public] and

```
pub fn and(&self, other: &Self) -> Self {
    let mut out = Self::new();
    for i in 0..WORDS {
        out.words[i] = self.words[i] & other.words[i];
    }
    out
}

pub fn or(&self, other: &Self) -> Self {
    let mut out = Self::new();
    for i in 0..WORDS {
        out.words[i] = self.words[i] | other.words[i];
    }
    out
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 134) - [public] or

```
pub fn or(&self, other: &Self) -> Self {
    let mut out = Self::new();
    for i in 0..WORDS {
        out.words[i] = self.words[i] | other.words[i];
    }
    out
}

pub fn xor(&self, other: &Self) -> Self {
    let mut out = Self::new();
    for i in 0..WORDS {
        out.words[i] = self.words[i] ^ other.words[i];
    }
    out
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 142) - [public] xor

```
pub fn xor(&self, other: &Self) -> Self {
    let mut out = Self::new();
    for i in 0..WORDS {
        out.words[i] = self.words[i] ^ other.words[i];
    }
    out
}

pub fn andnot(&self, other: &Self) -> Self {
    let mut out = Self::new();
    for i in 0..WORDS {
        out.words[i] = self.words[i] & !other.words[i];
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 150) - [public] andnot

```
pub fn andnot(&self, other: &Self) -> Self {
    let mut out = Self::new();
    for i in 0..WORDS {
        out.words[i] = self.words[i] & !other.words[i];
    }
    out
}

pub fn intersects(&self, other: &Self) -> bool {
    self.words.iter().zip(other.words.iter()).any(|(a, b)| a & b != 0)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 158) - [public] intersects

```
pub fn intersects(&self, other: &Self) -> bool {
    self.words.iter().zip(other.words.iter()).any(|(a, b)| a & b != 0)
}

pub fn iter(&self) -> BitmapIter<'_> {
    BitmapIter { words: &self.words, next_bit: 0, capacity: Self::CAPACITY }
}

impl<const WORDS: usize> Default for Bitmap<WORDS> {
    fn default() -> Self {
        Self::new()
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 162) - [public] iter

```
pub fn iter(&self) -> BitmapIter<'_> {
    BitmapIter { words: &self.words, next_bit: 0, capacity: Self::CAPACITY }
}

impl<const WORDS: usize> Default for Bitmap<WORDS> {
    fn default() -> Self {
        Self::new()
    }
}

pub struct BitmapIter<'a> {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 168) - [internal]

default

```
    fn default() -> Self {
        Self::new()
    }
}

pub struct BitmapIter<'a> {
    words: &'a [u64],
    next_bit: usize,
    capacity: usize,
}

impl<'a> Iterator for BitmapIter<'a> {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 173) - [public]

BitmapIter

```
pub struct BitmapIter<'a> {
    words: &'a [u64],
    next_bit: usize,
    capacity: usize,
}

impl<'a> Iterator for BitmapIter<'a> {
    type Item = usize;

    fn next(&mut self) -> Option<usize> {
        while self.next_bit < self.capacity {
            let bit = self.next_bit;
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 182) - [internal] next

```
    fn next(&mut self) -> Option<usize> {
        while self.next_bit < self.capacity {
            let bit = self.next_bit;
            self.next_bit += 1;
            if self.words[word_of(bit)] & mask_of(bit) != 0 {
                return Some(bit);
            }
        }
        None
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 200) - [public]

BitmapSlice

```
// -----
// BitmapSlice — widok nad `&mut [u64]` o rozmiarze nieznanym w
// czasie kompilacji (np. maski o rozmiarze zależnym od liczby CPU
// wykrytej w ACPI/MADT).
// -----

pub struct BitmapSlice<'a> {
    words: &'a mut [u64],
    nbits: usize,
}

impl<'a> BitmapSlice<'a> {
    pub fn new(words: &'a mut [u64], nbits: usize) -> Self {
        debug_assert!(words.len() >= words_for_bits(nbits));
        Self { words, nbits }
    }

    pub fn capacity(&self) -> usize {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 206) - [public] new

```
pub fn new(words: &'a mut [u64], nbits: usize) -> Self {
    debug_assert!(words.len() >= words_for_bits(nbits));
    Self { words, nbits }
}

pub fn capacity(&self) -> usize {
    self.nbits
}

pub fn set(&mut self, bit: usize) {
    if bit < self.nbits {
        self.words[word_of(bit)] |= mask_of(bit);
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 211) - [public] capacity

```
pub fn capacity(&self) -> usize {
    self.nbits
}

pub fn set(&mut self, bit: usize) {
    if bit < self.nbits {
        self.words[word_of(bit)] |= mask_of(bit);
    }
}

pub fn clear(&mut self, bit: usize) {
    if bit < self.nbits {
        self.words[word_of(bit)] &= !mask_of(bit);
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 215) - [public] set

```
pub fn set(&mut self, bit: usize) {
    if bit < self.nbits {
        self.words[word_of(bit)] |= mask_of(bit);
    }
}

pub fn clear(&mut self, bit: usize) {
    if bit < self.nbits {
        self.words[word_of(bit)] &= !mask_of(bit);
    }
}

// ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 221) - [public] clear

```
pub fn clear(&mut self, bit: usize) {
    if bit < self.nbits {
        self.words[word_of(bit)] &= !mask_of(bit);
    }
}

pub fn test(&self, bit: usize) -> bool {
    bit < self.nbits && self.words[word_of(bit)] & mask_of(bit) != 0
}

pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.count_ones()).sum()
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 227) - [public] test

```
pub fn test(&self, bit: usize) -> bool {
    bit < self.nbits && self.words[word_of(bit)] & mask_of(bit) != 0
}

pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.count_ones()).sum()
}

pub fn find_first_zero(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != u64::MAX {
            let bit = idx * BITS_PER_WORD + (!word).trailing_zeros() as usize;
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 231) - [public] weight

```
pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.count_ones()).sum()
}

pub fn find_first_zero(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != u64::MAX {
            let bit = idx * BITS_PER_WORD + (!word).trailing_zeros() as usize;
            if bit < self.nbits {
                return Some(bit);
            }
        }
        return None;
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 235) - [public]

find_first_zero

```
pub fn find_first_zero(&self) -> Option<usize> {
    for (idx, word) in self.words.iter().enumerate() {
        if *word != u64::MAX {
            let bit = idx * BITS_PER_WORD + (!word).trailing_zeros() as usize;
            if bit < self.nbits {
                return Some(bit);
            }
        }
        return None;
    }
}
None
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 248) - [public]

clear_all

```
pub fn clear_all(&mut self) {
    for w in self.words.iter_mut() {
        *w = 0;
    }
}

// -----
// AtomicBitmap<WORDS> – bezpieczna współbieżnie mapa bitowa, w tym
// bez-blokadowy przydział pierwszego wolnego bitu (test-and-set przez
// CAS) – używana m.in. przez alokator PID w `core.rs`.
// -----
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 263) - [public]

AtomicBitmap

```
pub struct AtomicBitmap<const WORDS: usize> {
    words: [AtomicU64; WORDS],
}

impl<const WORDS: usize> AtomicBitmap<WORDS> {
    pub const CAPACITY: usize = WORDS * BITS_PER_WORD;

    pub const fn new() -> Self {
        Self { words: [ZERO_WORD; WORDS] }
    }

    #[inline]
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 275) - [public] test

```
pub fn test(&self, bit: usize) -> bool {
    bit < Self::CAPACITY && self.words[word_of(bit)].load(Ordering::Acquire) & mask_of(bit) != 0
}

/// Ustawia bit atomowo, zwraca POPRZEDNI stan (true = już by ustawiony).
pub fn test_and_set(&self, bit: usize) -> bool {
    if bit >= Self::CAPACITY {
        return true;
    }
    let prev = self.words[word_of(bit)].fetch_or(mask_of(bit), Ordering::AcqRel);
    prev & mask_of(bit) != 0
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 280) - [public]

test_and_set

/// Ustawia bit atomowo, zwraca POPRZEDNI stan (true = już by ustawiony).

```
pub fn test_and_set(&self, bit: usize) -> bool {
    if bit >= Self::CAPACITY {
        return true;
    }
    let prev = self.words[word_of(bit)].fetch_or(mask_of(bit), Ordering::AcqRel);
    prev & mask_of(bit) != 0
}

/// Czy ci bit atomowo, zwraca POPRZEDNI stan (true = by ustawiony).
pub fn test_and_clear(&self, bit: usize) -> bool {
    if bit >= Self::CAPACITY {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 289) - [public]

test_and_clear

/// Czy ci bit atomowo, zwraca POPRZEDNI stan (true = by ustawiony).

```
pub fn test_and_clear(&self, bit: usize) -> bool {
    if bit >= Self::CAPACITY {
        return false;
    }
    let prev = self.words[word_of(bit)].fetch_and(!mask_of(bit), Ordering::AcqRel);
    prev & mask_of(bit) != 0
}

pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.load(Ordering::Relaxed).count_ones()).sum()
}

// ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 297) - [public] weight

```
pub fn weight(&self) -> u32 {
    self.words.iter().map(|w| w.load(Ordering::Relaxed).count_ones()).sum()
}

pub fn is_full(&self) -> bool {
    self.words.iter().all(|w| w.load(Ordering::Relaxed) == u64::MAX)
}

/// Bez blokad: skanuje słowa w poszukiwaniu pierwszego wyzerowanego
/// bitu i próbuje go zająć przez `compare_exchange_weak`. Przegrany
/// wyścig oznacza, że ktoś inny właśnie zajął ten sam bit (albo inny
/// w tym samym słowie) – ponawiamy próbę na TYM SAMYM słowie zamiast
/// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 301) - [public] is_full

```
pub fn is_full(&self) -> bool {
    self.words.iter().all(|w| w.load(Ordering::Relaxed) == u64::MAX)
}

/// Bez blokad: skanuje słowa w poszukiwaniu pierwszego wyzerowanego
/// bitu i próbuje go zająć przez `compare_exchange_weak`. Przegrany
/// wyścig oznacza, że ktoś inny właśnie zajął ten sam bit (albo inny
/// w tym samym słowie) – ponawiamy próbę na TYM SAMYM słowie zamiast
/// przechodzi dalej, żeby nie pominąć bitów zwolnionych w
/// międzyczasie.
pub fn find_first_zero_and_set(&self) -> Option<usize> {
    for idx in 0..WORDS {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 311) - [public]

find_first_zero_and_set

```
/// Bez blokad: skanuje słowa w poszukiwaniu pierwszego wyzerowanego
/// bitu i próbuje go zająć przez `compare_exchange_weak`. Przegrany
/// wyścig oznacza, że ktoś inny właśnie zajął ten sam bit (albo inny
/// w tym samym słowie) – ponawiamy próbę na TYM SAMYM słowie zamiast
/// przechodzi dalej, żeby nie pominąć bitów zwolnionych w
/// międzyczasie.
```

```
pub fn find_first_zero_and_set(&self) -> Option<usize> {
    for idx in 0..WORDS {
        loop {
            let cur = self.words[idx].load(Ordering::Acquire);
            if cur == u64::MAX {
                break;
            }
            let bit_in_word = (!cur).trailing_zeros() as usize;
            let global_bit = idx * BITS_PER_WORD + bit_in_word;
            if global_bit >= Self::CAPACITY {
                return None;
            }
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 340) - [internal]

default

```
fn default() -> Self {
    Self::new()
}

#[cfg(test)]
mod tests {
    use super::*;

    #[test]
    fn words_for_bits_rounds_up() {
        assert_eq!(words_for_bits(1), 1);
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 350) - [internal]

words_for_bits_rounds_up

```
fn words_for_bits_rounds_up() {
    assert_eq!(words_for_bits(1), 1);
    assert_eq!(words_for_bits(64), 1);
    assert_eq!(words_for_bits(65), 2);
    assert_eq!(words_for_bits(128), 2);
}

#[test]
fn set_clear_test_roundtrip() {
    let mut b: Bitmap<2> = Bitmap::new();
    assert!(b.test(70));
    b.set(70);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 358) - [internal]

set_clear_test_roundtrip

```
fn set_clear_test_roundtrip() {
    let mut b: Bitmap<2> = Bitmap::new();
    assert!(b.test(70));
    b.set(70);
    assert!(b.test(70));
    assert!(b.test(69));
    b.clear(70);
    assert!(b.test(70));
}

#[test]
fn out_of_range_access_is_a_safe_noop() {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 369) - [internal]

out_of_range_access_is_a_safe_noop

```
fn out_of_range_access_is_a_safe_noop() {
    let mut b: Bitmap<1> = Bitmap::new();
    b.set(1000);
    assert!(b.test(1000));
    assert!(b.is_empty());
}

#[test]
fn weight_counts_set_bits_across_words() {
    let mut b: Bitmap<2> = Bitmap::new();
    b.set(0);
    b.set(63);
    b.set(64);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 377) - [internal]

weight_counts_set_bits_across_words

```
fn weight_counts_set_bits_across_words() {
    let mut b: Bitmap<2> = Bitmap::new();
    b.set(0);
    b.set(63);
    b.set(64);
    b.set(127);
    assert_eq!(b.weight(), 4);
}

#[test]
fn find_first_set_crosses_word_boundary() {
    let mut b: Bitmap<3> = Bitmap::new();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 387) - [internal]

find_first_set_crosses_word_boundary

```
fn find_first_set_crosses_word_boundary() {
    let mut b: Bitmap<3> = Bitmap::new();
    b.set(130);
    assert_eq!(b.find_first_set(), Some(130));
}

#[test]
fn find_first_zero_skips_full_words() {
    let mut b: Bitmap<2> = Bitmap::filled();
    b.clear(70);
    assert_eq!(b.find_first_zero(), Some(70));
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 394) - [internal]

find_first_zero_skips_full_words

```
fn find_first_zero_skips_full_words() {
    let mut b: Bitmap<2> = Bitmap::filled();
    b.clear(70);
    assert_eq!(b.find_first_zero(), Some(70));
}

#[test]
fn find_first_zero_none_when_full() {
    let b: Bitmap<2> = Bitmap::filled();
    assert_eq!(b.find_first_zero(), None);
    assert!(b.is_full());
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 401) - [internal]

find_first_zero_none_when_full

```
fn find_first_zero_none_when_full() {
    let b: Bitmap<2> = Bitmap::filled();
    assert_eq!(b.find_first_zero(), None);
    assert!(b.is_full());
}

#[test]
fn find_next_set_after_given_bit() {
    let mut b: Bitmap<2> = Bitmap::new();
    b.set(5);
    b.set(80);
    assert_eq!(b.find_next_set(5), Some(80));
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 408) - [internal]

find_next_set_after_given_bit

```
fn find_next_set_after_given_bit() {
    let mut b: Bitmap<2> = Bitmap::new();
    b.set(5);
    b.set(80);
    assert_eq!(b.find_next_set(5), Some(80));
    assert_eq!(b.find_next_set(80), None);
}

#[test]
fn set_range_sets_contiguous_span() {
    let mut b: Bitmap<2> = Bitmap::new();
    b.set_range(60, 70);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 417) - [internal]

set_range_sets_contiguous_span

```
fn set_range_sets_contiguous_span() {
    let mut b: Bitmap<2> = Bitmap::new();
    b.set_range(60, 70);
    for i in 60..70 {
        assert!(b.test(i), "bit {i} powinien by■ ustawiony");
    }
    assert!(!b.test(59));
    assert!(!b.test(70));
    assert_eq!(b.weight(), 10);
}

#[test]
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 429) - [internal]

boolean_ops_behave_as_expected

```
fn boolean_ops_behave_as_expected() {
    let mut a: Bitmap<1> = Bitmap::new();
    let mut b: Bitmap<1> = Bitmap::new();
    a.set(1);
    a.set(2);
    b.set(2);
    b.set(3);
    assert_eq!(a.and(&b).weight(), 1);
    assert!(a.and(&b).test(2));
    assert_eq!(a.or(&b).weight(), 3);
    assert_eq!(a.xor(&b).weight(), 2);
    assert_eq!(a.andnot(&b).weight(), 1);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 446) - [internal]

iter_yields_set_bits_in_order

```
fn iter_yields_set_bits_in_order() {
    let mut b: Bitmap<2> = Bitmap::new();
    b.set(3);
    b.set(64);
    b.set(100);
    let collected: [usize; 3] = {
        let mut it = b.iter();
        [it.next().unwrap(), it.next().unwrap(), it.next().unwrap()]
    };
    assert_eq!(collected, [3, 64, 100]);
    assert!(b.iter().nth(3).is_none());
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 460) - [internal]

slice_bitmap_respects_logical_bit_count_not_word_count

```
fn slice_bitmap_respects_logical_bit_count_not_word_count() {
    let mut storage = [0u64; 2];
    let mut view = BitmapSlice::new(&mut storage, 70);
    assert_eq!(view.capacity(), 70);
    view.set(69);
    assert!(view.test(69));
    view.set(100); // poza `nbits`, mimo ■e mie■ci si■ w drugim s■owie
    assert!(!view.test(100));
    assert_eq!(view.find_first_zero(), Some(0));
}

#[test]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 472) - [internal]

slice_bitmap_clear_all_resets_backing_storage

```
fn slice_bitmap_clear_all_resets_backing_storage() {
    let mut storage = [u64::MAX; 1];
    let mut view = BitmapSlice::new(&mut storage, 64);
    assert_eq!(view.weight(), 64);
    view.clear_all();
    assert_eq!(view.weight(), 0);
}

#[test]
fn atomic_bitmap_test_and_set_reports_previous_state() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    assert_eq!(bm.test_and_set(5));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 481) - [internal]

atomic_bitmap_test_and_set_reports_previous_state

```
fn atomic_bitmap_test_and_set_reports_previous_state() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    assert_eq!(bm.test_and_set(5));
    assert!(bm.test(5));
    assert!(bm.test_and_set(5));
}

#[test]
fn atomic_bitmap_test_and_clear_reports_previous_state() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    bm.test_and_set(9);
    assert!(bm.test_and_clear(9));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 489) - [internal]

atomic_bitmap_test_and_clear_reports_previous_state

```
fn atomic_bitmap_test_and_clear_reports_previous_state() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    bm.test_and_set(9);
    assert!(bm.test_and_clear(9));
    assert!(bm.test(9));
    assert!(bm.test_and_clear(9));
}

#[test]
fn atomic_bitmap_find_first_zero_and_set_claims_sequentially() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    assert_eq!(bm.find_first_zero_and_set(), Some(0));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 498) - [internal]

atomic_bitmap_find_first_zero_and_set_claims_sequentially

```
fn atomic_bitmap_find_first_zero_and_set_claims_sequentially() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    assert_eq!(bm.find_first_zero_and_set(), Some(0));
    assert_eq!(bm.find_first_zero_and_set(), Some(1));
    bm.test_and_clear(0);
    assert_eq!(bm.find_first_zero_and_set(), Some(0));
}

#[test]
fn atomic_bitmap_find_first_zero_and_set_returns_none_when_full() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    for _ in 0..64 {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 507) - [internal]

atomic_bitmap_find_first_zero_and_set_returns_none_when_full

```
fn atomic_bitmap_find_first_zero_and_set_returns_none_when_full() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    for _ in 0..64 {
        assert!(bm.find_first_zero_and_set().is_some());
    }
    assert!(bm.is_full());
    assert_eq!(bm.find_first_zero_and_set(), None);
}

#[test]
fn atomic_bitmap_weight_matches_manual_count() {
    let bm: AtomicBitmap<2> = AtomicBitmap::new();
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 517) - [internal]

atomic_bitmap_weight_matches_manual_count

```
fn atomic_bitmap_weight_matches_manual_count() {
    let bm: AtomicBitmap<2> = AtomicBitmap::new();
    bm.test_and_set(0);
    bm.test_and_set(64);
    bm.test_and_set(127);
    assert_eq!(bm.weight(), 3);
}

#[test]
fn atomic_bitmap_out_of_range_set_is_harmless() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    assert!(bm.test_and_set(1000));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/bitmap.rs (line 526) - [internal]

atomic_bitmap_out_of_range_set_is_harmless

```
fn atomic_bitmap_out_of_range_set_is_harmless() {
    let bm: AtomicBitmap<1> = AtomicBitmap::new();
    assert!(bm.test_and_set(1000));
    assert!(!bm.test_and_clear(1000));
}

}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 17) - [public]

AtomicCpuMask

```
pub struct AtomicCpuMask {
    bits: [AtomicU64; WORDS],
}

impl AtomicCpuMask {
    #[inline]
    pub const fn new() -> Self {
        const INIT: AtomicU64 = AtomicU64::new(0);
        Self { bits: [INIT; WORDS] }
    }

    #[inline]
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 29) - [public] set

```
pub fn set(&self, cpu: u32) {
    let idx = cpu as usize;
    if idx < MAX_CPUS {
        self.bits[idx / 64].fetch_or(1u64 << (idx % 64), Ordering::Relaxed);
    }
}

#[inline]
pub fn clear(&self, cpu: u32) {
    let idx = cpu as usize;
    if idx < MAX_CPUS {
        self.bits[idx / 64].fetch_and(!(1u64 << (idx % 64)), Ordering::Relaxed);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 37) - [public] clear

```
pub fn clear(&self, cpu: u32) {
    let idx = cpu as usize;
    if idx < MAX_CPUS {
        self.bits[idx / 64].fetch_and(!(1u64 << (idx % 64)), Ordering::Relaxed);
    }
}

#[inline]
pub fn test(&self, cpu: u32) -> bool {
    let idx = cpu as usize;
    idx < MAX_CPUS && (self.bits[idx / 64].load(Ordering::Acquire) & (1u64 << (idx % 64))) != 0
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 45) - [public] test

```
pub fn test(&self, cpu: u32) -> bool {
    let idx = cpu as usize;
    idx < MAX_CPUS && (self.bits[idx / 64].load(Ordering::Acquire) & (1u64 << (idx % 64))) != 0
}

#[inline]
pub fn snapshot(&self) -> CpuMask {
    let mut m = CpuMask::empty();
    for cpu in 0..MAX_CPUS as u32 {
        if self.test(cpu) {
            m.set(cpu);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 51) - [public] snapshot

```
pub fn snapshot(&self) -> CpuMask {
    let mut m = CpuMask::empty();
    for cpu in 0..MAX_CPUS as u32 {
        if self.test(cpu) {
            m.set(cpu);
        }
    }
    m
}

static POSSIBLE: AtomicCpuMask = AtomicCpuMask::new();
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 74) - [public]

mark_possible

```
pub fn mark_possible(cpu: u32) {
    POSSIBLE.set(cpu);
}

#[inline]
pub fn mark_present(cpu: u32) {
    PRESENT.set(cpu);
}

#[inline]
pub fn mark_online(cpu: u32) {
    ONLINE.set(cpu);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 79) - [public]

mark_present

```
pub fn mark_present(cpu: u32) {
    PRESENT.set(cpu);
}

#[inline]
pub fn mark_online(cpu: u32) {
    ONLINE.set(cpu);
    ACTIVE.set(cpu);
}

#[inline]
pub fn mark_offline(cpu: u32) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 84) - [public]

mark_online

```
pub fn mark_online(cpu: u32) {
    ONLINE.set(cpu);
    ACTIVE.set(cpu);
}

#[inline]
pub fn mark_offline(cpu: u32) {
    ACTIVE.clear(cpu);
    ONLINE.clear(cpu);
}

#[inline]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 90) - [public]

mark_offline

```
pub fn mark_offline(cpu: u32) {
    ACTIVE.clear(cpu);
    ONLINE.clear(cpu);
}

#[inline]
pub fn is_online(cpu: u32) -> bool {
    ONLINE.test(cpu)
}

#[inline]
pub fn online_mask() -> CpuMask {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 96) - [public]

is_online

```
pub fn is_online(cpu: u32) -> bool {
    ONLINE.test(cpu)
}

#[inline]
pub fn online_mask() -> CpuMask {
    ONLINE.snapshot()
}

#[inline]
pub fn active_mask() -> CpuMask {
    ACTIVE.snapshot()
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 101) - [public]

online_mask

```
pub fn online_mask() -> CpuMask {
    ONLINE.snapshot()
}

#[inline]
pub fn active_mask() -> CpuMask {
    ACTIVE.snapshot()
}

#[inline]
pub fn set_topology(cpu: u32, pkg: u32, core: u32) {
    if (cpu as usize) < MAX_CPUS {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 106) - [public]

active_mask

```
pub fn active_mask() -> CpuMask {
    ACTIVE.snapshot()
}

#[inline]
pub fn set_topology(cpu: u32, pkg: u32, core: u32) {
    if (cpu as usize) < MAX_CPUS {
        PACKAGE_ID[cpu as usize].store(pkg, Ordering::Release);
        CORE_ID[cpu as usize].store(core, Ordering::Release);
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 111) - [public]

set_topology

```
pub fn set_topology(cpu: u32, pkg: u32, core: u32) {
    if (cpu as usize) < MAX_CPUS {
        PACKAGE_ID[cpu as usize].store(pkg, Ordering::Release);
        CORE_ID[cpu as usize].store(core, Ordering::Release);
    }
}

#[inline]
pub fn sibling_mask(cpu: u32) -> CpuMask {
    let mut out = CpuMask::empty();
    let pkg = PACKAGE_ID[cpu as usize].load(Ordering::Acquire);
    let core = CORE_ID[cpu as usize].load(Ordering::Acquire);
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 119) - [public]

sibling_mask

```
pub fn sibling_mask(cpu: u32) -> CpuMask {
    let mut out = CpuMask::empty();
    let pkg = PACKAGE_ID[cpu as usize].load(Ordering::Acquire);
    let core = CORE_ID[cpu as usize].load(Ordering::Acquire);
    if pkg == NO_TOPO || core == NO_TOPO {
        return out;
    }

    for c in online_mask().iter() {
        let c_idx = c as usize;
        if PACKAGE_ID[c_idx].load(Ordering::Acquire) == pkg && CORE_ID[c_idx].load(Ordering::Acquire) ==
            out.set(c);
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/cpumask.rs (line 137) - [public]
package_mask

```
pub fn package_mask(cpu: u32) -> CpuMask {
    let mut out = CpuMask::empty();
    let pkg = PACKAGE_ID[cpu as usize].load(Ordering::Acquire);
    if pkg == NO_TOPO {
        return out;
    }

    for c in online_mask().iter() {
        let c_idx = c as usize;
        if PACKAGE_ID[c_idx].load(Ordering::Acquire) == pkg {
            out.set(c);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 20) - [internal]
plist_prio

```
fn plist_prio(node: *mut TaskStruct) -> i32 {
    unsafe { *plist_prio_slot(node) }
}

/// Odtwarza `TaskStruct` z wzmiana `plist_node`.
///
/// Przechodzi przez `TaskStruct::container_of`, więc w trybie debug
/// wynik jest zweryfikowany round-tripem – błądny `PLIST_NODE_OFFSET`
/// (np. po zmianie layoutu `TaskStruct` bez aktualizacji stałej)
/// wywala się jako panika w testach, a nie jako ciche uszkodzenie
/// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 44) - [public] PList

```
pub struct PList {
    pub head: ListHead,
}

impl PList {
    pub const fn new() -> Self {
        Self { head: ListHead::new() }
    }

    pub fn is_empty(&self) -> bool {
        self.head.is_empty()
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 53) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.head.is_empty()
}

/// Wstawia `task` z priorytetem `priority`, zachowując porządek
/// rosnący po poziomach oraz FIFO w obrębie tego samego priorytetu.
///
/// # Bezpieczeństwo
/// `task` musi być przypięty (nie przenoszony w pamięci) przez
/// cały czas obecności na liście, i nie może już być na niej
/// obecny (podwójny insert bez remove uszkodzi listę – patrz
/// `debug_assert` poniżej, który to wyłapuje w testach).
/// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 152) - [public] first

```
/// Zadanie o najwyższym priorytecie (head listy poziomów),
/// pierwsze w kolejności FIFO na tym poziomie. O(1).

pub fn first(&self) -> *mut TaskStruct {
    if self.head.is_empty() { return ptr::null_mut(); }
    unsafe { task_from_node(self.head.next) }
}

/// Zadanie o najniższym priorytecie (ostatni poziom), ostatnie
/// w kolejności FIFO na tym poziomie.
///
/// O(1) — poprzednio ta funkcja mogła degenerować się do O(N)
/// przy wielu zadaniach na tym samym (najniższym) priorytecie,
/// bo szła liniowo przez `same_prio.prev` licząc na to, że
/// wskaźnik "prev" head'a wskazuje bezpośrednio na ogon (co nie
/// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/plist.rs (line 167) - [public] last

```
/// Zadanie o najniższym priorytecie (ostatni poziom), ostatnie
/// w kolejności FIFO na tym poziomie.
///
/// O(1) — poprzednio ta funkcja mogła degenerować się do O(N)
/// przy wielu zadaniach na tym samym (najniższym) priorytecie,
/// bo szła liniowo przez `same_prio.prev` licząc na to, że
/// wskaźnik "prev" head'a wskazuje bezpośrednio na ogon (co nie
/// było gwarantowane przez ówczesny `insert`). Teraz `insert`
/// utrzymuje ten inwariant jawnie: `same_prio.prev` head'a
/// zawsze wskazuje ogon FIFO, więc odczyt jest bezpośredni.

pub fn last(&self) -> *mut TaskStruct {
    if self.head.is_empty() { return ptr::null_mut(); }
    unsafe {
        let last_level = task_from_node(self.head.prev);
        let same_prio = plist_same_prio(last_level);
        if (*same_prio).is_empty() {
            last_level
        } else {
            task_from_same_prio((*same_prio).prev)
        }
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 23) - [internal] rb_parent

```
fn rb_parent(node: *mut TaskStruct) -> *mut TaskStruct {
    if node.is_null() { return ptr::null_mut(); }
    unsafe { (*rb_parent_color_slot(node) & !1) as *mut TaskStruct }
}

#[inline(always)]
fn rb_color(node: *mut TaskStruct) -> usize {
    if node.is_null() { return RB_BLACK; }
    unsafe { *rb_parent_color_slot(node) & 1 }
}

#[inline(always)]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 29) - [internal]
rb_color

```
fn rb_color(node: *mut TaskStruct) -> usize {
    if node.is_null() { return RB_BLACK; }
    unsafe { *rb_parent_color_slot(node) & 1 }
}

#[inline(always)]
fn rb_is_red(node: *mut TaskStruct) -> bool {
    !node.is_null() && rb_color(node) == RB_RED
}

#[inline(always)]
fn rb_set_parent_color(node: *mut TaskStruct, parent: *mut TaskStruct, color: usize) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 35) - [internal]
rb_is_red

```
fn rb_is_red(node: *mut TaskStruct) -> bool {
    !node.is_null() && rb_color(node) == RB_RED
}

#[inline(always)]
fn rb_set_parent_color(node: *mut TaskStruct, parent: *mut TaskStruct, color: usize) {
    if node.is_null() { return; }
    unsafe {
        *rb_parent_color_slot(node) = (parent as usize & !1) | color;
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 40) - [internal]
rb_set_parent_color

```
fn rb_set_parent_color(node: *mut TaskStruct, parent: *mut TaskStruct, color: usize) {
    if node.is_null() { return; }
    unsafe {
        *rb_parent_color_slot(node) = (parent as usize & !1) | color;
    }
}

#[inline(always)]
fn rb_set_parent(node: *mut TaskStruct, parent: *mut TaskStruct) {
    let color = rb_color(node);
    rb_set_parent_color(node, parent, color);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 48) - [internal]
rb_set_parent

```
fn rb_set_parent(node: *mut TaskStruct, parent: *mut TaskStruct) {
    let color = rb_color(node);
    rb_set_parent_color(node, parent, color);
}

#[inline(always)]
fn rb_set_color(node: *mut TaskStruct, color: usize) {
    if node.is_null() { return; }
    unsafe {
        let slot = rb_parent_color_slot(node);
        *slot = (*slot & !1) | color;
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 54) - [internal]

rb_set_color

```
fn rb_set_color(node: *mut TaskStruct, color: usize) {
    if node.is_null() { return; }
    unsafe {
        let slot = rb_parent_color_slot(node);
        *slot = (*slot & !1) | color;
    }
}

pub struct RbTree {
    pub root: *mut TaskStruct,
    pub leftmost: *mut TaskStruct,
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 62) - [public] RbTree

```
pub struct RbTree {
    pub root: *mut TaskStruct,
    pub leftmost: *mut TaskStruct,
}

impl RbTree {
    pub const fn new() -> Self {
        Self { root: ptr::null_mut(), leftmost: ptr::null_mut() }
    }

    pub unsafe fn insert<F>(&mut self, node: *mut TaskStruct, mut less: F)
    where
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rbtree.rs (line 116) - [internal]

rb_insert_fixup

```
fn rb_insert_fixup(&mut self, mut node: *mut TaskStruct) {
    unsafe {
        while rb_is_red(rb_parent(node)) {
            let parent = rb_parent(node);
            let grandparent = rb_parent(parent);
            debug_assert(!grandparent.is_null(), "czerwony rodzic implikuje istnienie czarnego dziadka");

            if parent == *rb_left(grandparent) {
                let uncle = *rb_right(grandparent);
                if rb_is_red(uncle) {
                    rb_set_color(parent, RB_BLACK);
                    rb_set_color(uncle, RB_BLACK);
                }
            }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 4) - [public] RtArray

```
pub struct RtArray {
    pub queue: [ListHead; MAX_RT_PRIO as usize],
    pub bitmap: [u64; 2],
    pub nr_running: usize,
}

impl RtArray {
    pub const fn new() -> Self {
        Self {
            queue: [ListHead::new(); MAX_RT_PRIO as usize],
            bitmap: [0, 0],
            nr_running: 0,
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 20) - [public] set_bit

```
pub fn set_bit(&mut self, prio: usize) {
    let word = prio / 64;
    let bit = prio % 64;
    self.bitmap[word] |= 1u64 << bit;
}

#[inline(always)]
pub fn clear_bit(&mut self, prio: usize) {
    let word = prio / 64;
    let bit = prio % 64;
    self.bitmap[word] &= !(1u64 << bit);
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 27) - [public] clear_bit

```
pub fn clear_bit(&mut self, prio: usize) {
    let word = prio / 64;
    let bit = prio % 64;
    self.bitmap[word] &= !(1u64 << bit);
}

#[inline(always)]
fn bit_is_set(&self, prio: usize) -> bool {
    let word = prio / 64;
    let bit = prio % 64;
    (self.bitmap[word] & (1u64 << bit)) != 0
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 34) - [internal] bit_is_set

```
fn bit_is_set(&self, prio: usize) -> bool {
    let word = prio / 64;
    let bit = prio % 64;
    (self.bitmap[word] & (1u64 << bit)) != 0
}

#[inline(always)]
pub fn highest_prio(&self) -> Option<usize> {
    if self.bitmap[0] != 0 {
        Some(self.bitmap[0].trailing_zeros() as usize)
    } else if self.bitmap[1] != 0 {
        Some(64 + self.bitmap[1].trailing_zeros() as usize)
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 41) - [public] highest_prio

```
pub fn highest_prio(&self) -> Option<usize> {
    if self.bitmap[0] != 0 {
        Some(self.bitmap[0].trailing_zeros() as usize)
    } else if self.bitmap[1] != 0 {
        Some(64 + self.bitmap[1].trailing_zeros() as usize)
    } else {
        None
    }
}

/// Dodaje `task` do kolejki jego priorytetu i ustawia bit
/// w bitmapie. 0(1).
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/rt_array.rs (line 149) - [public]

active_levels

```
/// Liczba unikalnych aktywnych poziomów priorytetu (nie mylić
/// z `nr_running`, które liczy wszystkie zakolejkowane zadania).
///
/// Dopisane, bo bez tego nie da się odróżnić "jeden proces o
/// wysokim priorytecie" od "wiele procesów rozłożonych na wiele
/// priorytetów" bez przechodzenia całej bitmapy ręcznie za każdym
/// razem, gdy taka informacja jest potrzebna (np. w heurystykach
/// load-balancingu między CPU).

pub fn active_levels(&self) -> u32 {
    self.bitmap[0].count_ones() + self.bitmap[1].count_ones()
}

/// Odtwarza `TaskStruct` z wzmiana `rt.run_list`.
///
/// Przechodzi przez `TaskStruct::container_of`, więc w trybie debug
/// wynik jest zweryfikowany round-tripem (patrz dokumentacja
/// `container_of` w `task.rs`) – spójnie z tym, jak `plist.rs`
/// odtwarza `TaskStruct` ze swoich wzmianek.
#[inline(always)]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 15) - [public]

PidAllocator

```
pub struct PidAllocator {
    bitmap: [AtomicU64; PID_BITMAP_WORDS],
    next_hint: AtomicU32,
    allocated: AtomicU32,
}

impl PidAllocator {
    pub const fn new() -> Self {
        const ZERO: AtomicU64 = AtomicU64::new(0);
        Self {
            bitmap: [ZERO; PID_BITMAP_WORDS],
            next_hint: AtomicU32::new(RESERVED_PIDS),
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 31) - [public] alloc

```
pub fn alloc(&self) -> Option<TaskId> {
    let hint = self.next_hint.load(Ordering::Relaxed) as usize;
    let start_idx = hint / 64;

    for offset in 0..PID_BITMAP_WORDS {
        let idx = (start_idx + offset) % PID_BITMAP_WORDS;
        let word = self.bitmap[idx].load(Ordering::Relaxed);
        if word == u64::MAX { continue; }

        let bit = (!word).trailing_zeros();
        let pid = (idx * 64 + bit as usize) as u32;
        if pid >= MAX_PID { continue; }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 54) - [public] free

```
pub fn free(&self, pid: TaskId) {
    let pid = pid as u32;
    if pid >= MAX_PID { return; }
    let idx = pid as usize / 64;
    let bit = 1u64 << (pid as usize % 64);
    if self.bitmap[idx].fetch_and(!bit, Ordering::AcqRel) & bit != 0 {
        self.allocated.fetch_sub(1, Ordering::Relaxed);
    }
}

pub fn reserve(&self, pid: TaskId) -> bool {
    let pid = pid as u32;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 64) - [public] reserve

```
pub fn reserve(&self, pid: TaskId) -> bool {
    let pid = pid as u32;
    if pid >= MAX_PID { return false; }
    let idx = pid as usize / 64;
    let bit = 1u64 << (pid as usize % 64);
    if self.bitmap[idx].fetch_or(bit, Ordering::AcqRel) & bit == 0 {
        self.allocated.fetch_add(1, Ordering::Relaxed);
        true
    } else {
        false
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 77) - [public] is_used

```
pub fn is_used(&self, pid: TaskId) -> bool {
    let pid = pid as u32;
    if pid >= MAX_PID { return true; }
    let idx = pid as usize / 64;
    let bit = 1u64 << (pid as usize % 64);
    (self.bitmap[idx].load(Ordering::Acquire) & bit) != 0
}

pub fn count_allocated(&self) -> u32 {
    self.allocated.load(Ordering::Relaxed)
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 85) - [public] count_allocated

```
pub fn count_allocated(&self) -> u32 {
    self.allocated.load(Ordering::Relaxed)
}

<<<<<<< HEAD
pub const MAX_TASKS: usize = 65536;
const TGID_SLOTS: usize = MAX_TASKS / 4;

pub struct TaskTable {
    pid_slots: [AtomicPtr<TaskStruct>; MAX_TASKS],
    tgid_slots: [AtomicPtr<TaskStruct>; TGID_SLOTS],
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 94) - [public] TaskTable

```
pub struct TaskTable {
    pid_slots: [AtomicPtr<TaskStruct>; MAX_TASKS],
    tgid_slots: [AtomicPtr<TaskStruct>; TGID_SLOTS],
    count: AtomicU32,
    =====
    #[repr(C)]
    pub struct RtFields {
        pub rt_priority: u32,
        pub run_list: ListHead,
    }
    >>>>>>> da9f7b9 (wip: opis zmian)
}

// ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 100) - [public] RtFields

```
pub struct RtFields {
    pub rt_priority: u32,
    pub run_list: ListHead,
    >>>>>> da9f7b9 (wip: opis zmian)
}
```

```
impl TaskTable {
    pub const fn new() -> Self {
        const NULL_PTR: AtomicPtr<TaskStruct> = AtomicPtr::new(ptr::null_mut());
        Self {
            pid_slots: [NULL_PTR; MAX_TASKS],
            tgid_slots: [NULL_PTR; TGID_SLOTS],
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 116) - [internal] hash_pid

```
fn hash_pid(pid: TaskId) -> usize { (pid as usize) % MAX_TASKS }
fn hash_tgid(tgid: TaskId) -> usize { (tgid as usize) % TGID_SLOTS }

pub fn insert(&self, task: *mut TaskStruct) -> bool {
    if task.is_null() { return false; }
    let pid = unsafe { (*task).pid };
    let tgid = unsafe { (*task).tgid };

    let start = Self::hash_pid(pid);
    for offset in 0..MAX_TASKS {
        let idx = (start + offset) % MAX_TASKS;
        let expected = ptr::null_mut();
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 117) - [internal]
hash_tgid

```
fn hash_tgid(tgid: TaskId) -> usize { (tgid as usize) % TGID_SLOTS }

pub fn insert(&self, task: *mut TaskStruct) -> bool {
    if task.is_null() { return false; }
    let pid = unsafe { (*task).pid };
    let tgid = unsafe { (*task).tgid };

    let start = Self::hash_pid(pid);
    for offset in 0..MAX_TASKS {
        let idx = (start + offset) % MAX_TASKS;
        let expected = ptr::null_mut();
        if self.pid_slots[idx].compare_exchange(
            expected, task, Ordering::AcqRel, Ordering::Acquire
        ).is_ok() {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 119) - [public] insert

```
pub fn insert(&self, task: *mut TaskStruct) -> bool {
    if task.is_null() { return false; }
    let pid = unsafe { (*task).pid };
    let tgid = unsafe { (*task).tgid };

    let start = Self::hash_pid(pid);
    for offset in 0..MAX_TASKS {
        let idx = (start + offset) % MAX_TASKS;
        let expected = ptr::null_mut();
        if self.pid_slots[idx].compare_exchange(
            expected, task, Ordering::AcqRel, Ordering::Acquire
        ).is_ok() {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 154) - [public] remove

```
pub fn remove(&self, pid: TaskId) -> *mut TaskStruct {
    let start = Self::hash_pid(pid);
    for offset in 0..MAX_TASKS {
        let idx = (start + offset) % MAX_TASKS;
        let slot = self.pid_slots[idx].load(Ordering::Acquire);
        if slot.is_null() { return ptr::null_mut(); }

        if unsafe { (*slot).pid == pid } {
            if self.pid_slots[idx].compare_exchange(
                slot, ptr::null_mut(), Ordering::AcqRel, Ordering::Acquire
            ).is_ok() {
                self.count.fetch_sub(1, Ordering::Relaxed);
            }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 185) - [public] lookup_pid

```
pub fn lookup_pid(&self, pid: TaskId) -> *mut TaskStruct {
    let start = Self::hash_pid(pid);
    for offset in 0..MAX_TASKS {
        let idx = (start + offset) % MAX_TASKS;
        let slot = self.pid_slots[idx].load(Ordering::Acquire);
        if slot.is_null() { return ptr::null_mut(); }
        if unsafe { (*slot).pid == pid } { return slot; }
    }
    ptr::null_mut()
}

pub fn lookup_tgid(&self, tgid: TaskId) -> *mut TaskStruct {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 196) - [public]
lookup_tgid

```
pub fn lookup_tgid(&self, tgid: TaskId) -> *mut TaskStruct {
    let start = Self::hash_tgid(tgid);
    for offset in 0..TGID_SLOTS {
        let idx = (start + offset) % TGID_SLOTS;
        let slot = self.tgid_slots[idx].load(Ordering::Acquire);
        if slot.is_null() { return ptr::null_mut(); }
        if unsafe { (*slot).tgid == tgid && (*slot).pid == tgid } { return slot; }
    }
    ptr::null_mut()
}

pub fn count(&self) -> u32 {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 207) - [public] count

```
pub fn count(&self) -> u32 {
    self.count.load(Ordering::Relaxed)
}

unsafe fn task_from_list_node(node: *mut ListHead) -> *mut TaskStruct {
    let dummy = MaybeUninit:::<TaskStruct>::uninit();
    let base = dummy.as_ptr() as *mut u8;
    let field = core::ptr::addr_of!((*dummy.as_ptr()).thread_group) as *mut u8;
    let offset = field.offset_from(base) as usize;
    (node as *mut u8).sub(offset) as *mut TaskStruct
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 221) - [public]

TaskIterator

```
pub struct TaskIterator<'a> {
    table: &'a TaskTable,
    index: usize,
    =====
impl TaskStruct {
    pub const RB_LEFT_OFFSET: usize = core::mem::offset_of!(TaskStruct, rb_left);
    pub const RB_RIGHT_OFFSET: usize = core::mem::offset_of!(TaskStruct, rb_right);
    pub const RB_PARENT_COLOR_OFFSET: usize = core::mem::offset_of!(TaskStruct, rb_parent_color);

    pub const PLIST_PRIO_OFFSET: usize = core::mem::offset_of!(TaskStruct, plist_prio);
    pub const PLIST_SAME_PRIO_OFFSET: usize = core::mem::offset_of!(TaskStruct, plist_same_prio);
    pub const PLIST_NODE_OFFSET: usize = core::mem::offset_of!(TaskStruct, plist_node);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 279) - [public] new

```
pub fn new(table: &'a TaskTable) -> Self {
    Self { table, index: 0 }
}

impl<'a> Iterator for TaskIterator<'a> {
    type Item = *mut TaskStruct;

    fn next(&mut self) -> Option<Self::Item> {
        while self.index < MAX_TASKS {
            let slot = self.table.pid_slots[self.index].load(Ordering::Acquire);
            self.index += 1;
        }
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 287) - [internal] next

```
fn next(&mut self) -> Option<Self::Item> {
    while self.index < MAX_TASKS {
        let slot = self.table.pid_slots[self.index].load(Ordering::Acquire);
        self.index += 1;
        if !slot.is_null() {
            return Some(slot);
        }
    }
    None
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 299) - [public]

ThreadGroupIterator

```
pub struct ThreadGroupIterator {
    current: *mut TaskStruct,
    head: *mut ListHead,
}

impl ThreadGroupIterator {
    pub unsafe fn new(leader: *mut TaskStruct) -> Self {
        if leader.is_null() {
            return Self { current: ptr::null_mut(), head: ptr::null_mut() };
        }
        let head = &(*leader).thread_group as *const ListHead as *mut ListHead;
        let first = (*head).next;
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 319) - [internal] next

```
fn next(&mut self) -> Option<Self::Item> {
    if self.current.is_null() || self.head.is_null() { return None; }
    let result = self.current;
    let list_node = &(*self.current).thread_group as *const ListHead as *mut ListHead;
    let next_node = (*list_node).next;

    if next_node == self.head || next_node.is_null() {
        self.current = ptr::null_mut();
    } else {
        self.current = task_from_list_node(next_node);
    }
    Some(result)
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 334) - [public]

ChildIterator

```
pub struct ChildIterator {
    current: *mut TaskStruct,
}

impl ChildIterator {
    pub unsafe fn new(parent: *mut TaskStruct) -> Self {
        if parent.is_null() {
            return Self { current: ptr::null_mut() };
        }
        Self { current: (*parent).children }
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 350) - [internal] next

```
fn next(&mut self) -> Option<Self::Item> {
    if self.current.is_null() { return None; }
    let result = self.current;
    self.current = (*self.current).sibling;
    Some(result)
}

pub struct DescendantIterator {
    stack: Vec<*mut TaskStruct>,
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 358) - [public]

DescendantIterator

```
pub struct DescendantIterator {
    stack: Vec<*mut TaskStruct>,
}

impl DescendantIterator {
    pub unsafe fn new(root: *mut TaskStruct) -> Self {
        let mut stack = Vec::new();
        if !root.is_null() {
            stack.push(root);
        }
        Self { stack }
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 375) - [internal] next

```
fn next(&mut self) -> Option<Self::Item> {
    let current = self.stack.pop()?;
    let mut child = (*current).children;
    while !child.is_null() {
        self.stack.push(child);
        child = (*child).sibling;
    }
    Some(current)
}
```

```
pub struct ProcessGroup {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 386) - [public]
ProcessGroup

```
pub struct ProcessGroup {
    pub pgid: TaskId,
    pub leader: AtomicPtr<TaskStruct>,
    pub session: AtomicPtr<Session>,
    pub tasks: ListHead,
    pub lock: SpinLock,
    pub refcount: AtomicU32,
    pub is_orphaned: AtomicBool,
}
```

```
impl ProcessGroup {
    pub fn new(pgid: TaskId, leader: *mut TaskStruct) -> Self {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 397) - [public] new

```
pub fn new(pgid: TaskId, leader: *mut TaskStruct) -> Self {
    let mut pg = Self {
        pgid,
        leader: AtomicPtr::new(leader),
        session: AtomicPtr::new(ptr::null_mut()),
        tasks: ListHead::new(),
        lock: SpinLock::new(),
        refcount: AtomicU32::new(1),
        is_orphaned: AtomicBool::new(false),
    };
    pg.tasks.init();
    pg
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 427) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.refcount.load(Ordering::Relaxed) <= 1
}

pub unsafe fn send_signal(&self, sig: u8) {
    let mut node = self.tasks.next;
    while node != &self.tasks as *const ListHead as *mut ListHead && !node.is_null() {
        let task = task_from_list_node(node);
        if !task.is_null() {
            (*task).sig.raise(sig);
        }
        node = (*node).next;
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 443) - [public] Session

```
pub struct Session {
    pub sid: TaskId,
    pub leader: AtomicPtr<TaskStruct>,
    pub groups: ListHead,
    pub lock: SpinLock,
    pub refcount: AtomicU32,
    pub foreground_pgid: AtomicU32,
}

impl Session {
    pub fn new(sid: TaskId, leader: *mut TaskStruct) -> Self {
        let mut s = Self {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 453) - [public] new

```
pub fn new(sid: TaskId, leader: *mut TaskStruct) -> Self {
    let mut s = Self {
        sid,
        leader: AtomicPtr::new(leader),
        groups: ListHead::new(),
        lock: SpinLock::new(),
        refcount: AtomicU32::new(1),
        foreground_pgid: AtomicU32::new(0),
    };
    s.groups.init();
    s
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 473) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.refcount.load(Ordering::Relaxed) <= 1
}

pub fn set_foreground(&self, pgid: TaskId) {
    self.foreground_pgid.store(pgid as u32, Ordering::Release);
}

pub fn get_foreground(&self) -> TaskId {
    self.foreground_pgid.load(Ordering::Acquire) as TaskId
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 477) - [public] set_foreground

```
pub fn set_foreground(&self, pgid: TaskId) {
    self.foreground_pgid.store(pgid as u32, Ordering::Release);
}

pub fn get_foreground(&self) -> TaskId {
    self.foreground_pgid.load(Ordering::Acquire) as TaskId
}

pub const MAX_UID: usize = 65536;

pub struct UidTaskCount {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 481) - [public]

get_foreground

```
pub fn get_foreground(&self) -> TaskId {
    self.foreground_pgid.load(Ordering::Acquire) as TaskId
}

pub const MAX_UID: usize = 65536;

pub struct UidTaskCount {
    counts: [AtomicU32; MAX_UID],
}

impl UidTaskCount {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 488) - [public]

UidTaskCount

```
pub struct UidTaskCount {
    counts: [AtomicU32; MAX_UID],
}

impl UidTaskCount {
    pub const fn new() -> Self {
        const ZERO: AtomicU32 = AtomicU32::new(0);
        Self { counts: [ZERO; MAX_UID] }
    }

    pub fn inc(&self, uid: u32) -> u32 {
        let idx = (uid as usize) % MAX_UID;
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 498) - [public] inc

```
pub fn inc(&self, uid: u32) -> u32 {
    let idx = (uid as usize) % MAX_UID;
    self.counts[idx].fetch_add(1, Ordering::Relaxed) + 1
}

pub fn dec(&self, uid: u32) {
    let idx = (uid as usize) % MAX_UID;
    self.counts[idx].fetch_update(Ordering::Relaxed, Ordering::Relaxed, |v| {
        if v > 0 { Some(v - 1) } else { Some(0) }
    }).ok();
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 503) - [public] dec

```
pub fn dec(&self, uid: u32) {
    let idx = (uid as usize) % MAX_UID;
    self.counts[idx].fetch_update(Ordering::Relaxed, Ordering::Relaxed, |v| {
        if v > 0 { Some(v - 1) } else { Some(0) }
    }).ok();
}

pub fn get(&self, uid: u32) -> u32 {
    let idx = (uid as usize) % MAX_UID;
    self.counts[idx].load(Ordering::Relaxed)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 510) - [public] get

```
pub fn get(&self, uid: u32) -> u32 {
    let idx = (uid as usize) % MAX_UID;
    self.counts[idx].load(Ordering::Relaxed)
}
}
```

```
pub enum TaskRegistryError {
    PidExhausted,
    PidAlreadyUsed,
    TableFull,
    InvalidTask,
    NprocExceeded,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 516) - [public]

TaskRegistryError

```
pub enum TaskRegistryError {
    PidExhausted,
    PidAlreadyUsed,
    TableFull,
    InvalidTask,
    NprocExceeded,
}
```

```
pub struct TaskRegistry {
    pub pids: PidAllocator,
    pub table: TaskTable,
    pub init_task: AtomicPtr<TaskStruct>,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 524) - [public]

TaskRegistry

```
pub struct TaskRegistry {
    pub pids: PidAllocator,
    pub table: TaskTable,
    pub init_task: AtomicPtr<TaskStruct>,
    pub total_forks: AtomicU64,
    pub total_exits: AtomicU64,
    pub global_lock: SpinLock,
    pub uid_counts: UidTaskCount,
}
```

```
impl TaskRegistry {
    pub const fn new() -> Self {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 585) - [public]

find_by_pid

```
pub fn find_by_pid(&self, pid: TaskId) -> *mut TaskStruct {
    self.table.lookup_pid(pid)
}

pub fn find_by_tgid(&self, tgid: TaskId) -> *mut TaskStruct {
    self.table.lookup_tgid(tgid)
}

pub unsafe fn link_hierarchy(&self, parent: *mut TaskStruct, child: *mut TaskStruct) {
    if parent.is_null() || child.is_null() { return; }
    let _guard = self.global_lock.lock_guard();

    // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 589) - [public]

find_by_tgid

```
pub fn find_by_tgid(&self, tgid: TaskId) -> *mut TaskStruct {
    self.table.lookup_tgid(tgid)
}

pub unsafe fn link_hierarchy(&self, parent: *mut TaskStruct, child: *mut TaskStruct) {
    if parent.is_null() || child.is_null() { return; }
    let _guard = self.global_lock.lock_guard();

    (*child).parent = parent;
    (*child).ppid = (*parent).pid;
    (*child).sibling = (*parent).children;
    (*parent).children = child;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 685) - [public] iter

```
pub fn iter(&self) -> TaskIterator {
    TaskIterator::new(&self.table)
}

pub unsafe fn send_signal_to_group(&self, pgid: TaskId, sig: u8) {
    let leader = self.find_by_tgid(pgid);
    if leader.is_null() { return; }

    for task_ptr in ThreadGroupIterator::new(leader) {
        if !task_ptr.is_null() {
            (*task_ptr).sig.raise(sig);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 700) - [public]

set_init_task

```
pub fn set_init_task(&self, task: *mut TaskStruct) {
    self.init_task.store(task, Ordering::Release);
}

pub fn stats(&self) -> (u32, u64, u64) {
    (
        self.table.count(),
        self.total_forks.load(Ordering::Relaxed),
        self.total_exits.load(Ordering::Relaxed),
    )
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/collections/task.rs (line 704) - [public] stats

```
pub fn stats(&self) -> (u32, u64, u64) {
    (
        self.table.count(),
        self.total_forks.load(Ordering::Relaxed),
        self.total_exits.load(Ordering::Relaxed),
    )
}

pub struct PidNamespace {
    pub id: u32,
    pub level: u32,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/collections/task.rs (line 713) - [public]

PidNamespace

```
pub struct PidNamespace {
    pub id: u32,
    pub level: u32,
    pub parent: *mut PidNamespace,
    pub registry: TaskRegistry,
    pub lock: SpinLock,
}

impl PidNamespace {
    pub const fn new(id: u32, level: u32, parent: *mut PidNamespace) -> Self {
        Self {
            id,
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 22) - [public] alloc_pid

```
pub fn alloc_pid() -> Option<TaskId> {
    PID_BITMAP.find_first_zero_and_set().map(|b| b as TaskId)
}

pub fn free_pid(pid: TaskId) {
    if (pid as usize) < PID_MAX {
        PID_BITMAP.test_and_clear(pid as usize);
    }
}
```

```
/// Rezerwuje konkretny PID (np. 0 dla `idle`/`swapper`, albo odtworzenie
/// procesu z checkpointa). Zwraca `false`, je■li PID by■ ju■ zaj■ty.
/// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 26) - [public] free_pid

```
pub fn free_pid(pid: TaskId) {
    if (pid as usize) < PID_MAX {
        PID_BITMAP.test_and_clear(pid as usize);
    }
}
```

```
/// Rezerwuje konkretny PID (np. 0 dla `idle`/`swapper`, albo odtworzenie
/// procesu z checkpointa). Zwraca `false`, je■li PID by■ ju■ zaj■ty.
```

```
pub fn reserve_pid(pid: TaskId) -> bool {
    if (pid as usize) >= PID_MAX {
        return false;
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 34) - [public] reserve_pid

```
/// Rezerwuje konkretny PID (np. 0 dla `idle`/`swapper`, albo odtworzenie
/// procesu z checkpointa). Zwraca `false`, je■li PID by■ ju■ zaj■ty.
```

```
pub fn reserve_pid(pid: TaskId) -> bool {
    if (pid as usize) >= PID_MAX {
        return false;
    }
    !PID_BITMAP.test_and_set(pid as usize)
}

pub fn pid_in_use(pid: TaskId) -> bool {
    (pid as usize) < PID_MAX && PID_BITMAP.test(pid as usize)
}
```

```
// -----
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 41) - [public] pid_in_use

```
pub fn pid_in_use(pid: TaskId) -> bool {
    (pid as usize) < PID_MAX && PID_BITMAP.test(pid as usize)
}

// -----
// Rozruch CPU
// -----

pub unsafe fn boot_bsp(cpu: u32, apic_id: u32, idle: *mut TaskStruct, rq: &mut RunQueue) {
    reserve_pid((*idle).pid);
    rq.bind_idle_task();
    arch_hooks::register_apic_id(cpu, apic_id);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 219) - [public]

sys_sched_get_priority_max

```
pub fn sys_sched_get_priority_max(policy: SchedPolicy) -> i32 {
    if policy.is_realtime() {
        MAX_RT_PRIO - 1
    } else {
        0
    }
}

pub fn sys_sched_get_priority_min(policy: SchedPolicy) -> i32 {
    if policy.is_realtime() {
        0
    } else {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 227) - [public]

sys_sched_get_priority_min

```
pub fn sys_sched_get_priority_min(policy: SchedPolicy) -> i32 {
    if policy.is_realtime() {
        0
    } else {
        0
    }
}

#[cfg(test)]
mod tests {
    use super::*;
    use crate::cpu::scheduler::entities::task::TaskStructT;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 240) - [internal] make_idle

```
fn make_idle(pid: TaskId) -> TaskStructT {
    let mut t = TaskStructT::blank();
    t.init_test_stub(pid, SchedPolicy::Idle, 0);
    t
}

fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStructT {
    let mut t = TaskStructT::blank();
    t.init_test_stub(pid, policy, nice);
    t
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 246) - [internal] make_task

```
fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStructT {
    let mut t = TaskStructT::blank();
    t.init_test_stub(pid, policy, nice);
    t
}

fn ptr_of(t: &mut TaskStructT) -> *mut TaskStructT {
    t as *mut TaskStructT
}

#[test]
fn two_consecutive_allocations_are_distinct() {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 252) - [internal] ptr_of

```
fn ptr_of(t: &mut TaskStructT) -> *mut TaskStructT {
    t as *mut TaskStructT
}

#[test]
fn two_consecutive_allocations_are_distinct() {
    let a = alloc_pid().unwrap();
    let b = alloc_pid().unwrap();
    assert_ne!(a, b);
    free_pid(a);
    free_pid(b);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 257) - [internal]
two_consecutive_allocations_are_distinct

```
fn two_consecutive_allocations_are_distinct() {
    let a = alloc_pid().unwrap();
    let b = alloc_pid().unwrap();
    assert_ne!(a, b);
    free_pid(a);
    free_pid(b);
}

#[test]
fn reserve_and_free_pid_roundtrip_on_a_dedicated_high_pid() {
    let pid = 25_000;
    assert!(reserve_pid(pid));
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 266) - [internal]
reserve_and_free_pid_roundtrip_on_a_dedicated_high_pid

```
fn reserve_and_free_pid_roundtrip_on_a_dedicated_high_pid() {
    let pid = 25_000;
    assert!(reserve_pid(pid));
    assert!(!reserve_pid(pid));
    assert!(pid_in_use(pid));
    free_pid(pid);
    assert!(!pid_in_use(pid));
    assert!(reserve_pid(pid));
    free_pid(pid);
}

#[test]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 278) - [internal]

reserve_pid_rejects_out_of_range

```
fn reserve_pid_rejects_out_of_range() {
    assert!(!reserve_pid(PID_MAX as TaskId + 1));
}

#[test]
fn boot_bsp_registers_apic_and_runqueue() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(200, ptr_of(&mut idle));
    unsafe {
        boot_bsp(200, 42, ptr_of(&mut idle), &mut rq);
    }
    assert!(core::ptr::eq(arch_hooks::snapshot_registry()[200], &rq as *const RunQueue as *mut RunQueue));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 283) - [internal]

boot_bsp_registers_apic_and_runqueue

```
fn boot_bsp_registers_apic_and_runqueue() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(200, ptr_of(&mut idle));
    unsafe {
        boot_bsp(200, 42, ptr_of(&mut idle), &mut rq);
    }
    assert!(core::ptr::eq(arch_hooks::snapshot_registry()[200], &rq as *const RunQueue as *mut RunQueue));
    arch_hooks::register_runqueue(200, core::ptr::null_mut());
}

#[test]
fn wake_up_new_task_enqueues_and_marks_runnable() {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 294) - [internal]

wake_up_new_task_enqueues_and_marks_runnable

```
fn wake_up_new_task_enqueues_and_marks_runnable() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut child = make_task(90_001, SchedPolicy::Normal, 0);
    child.set_state(TaskState::Interruptible).unwrap();

    let registry: [*mut RunQueue; 1] = [&mut rq as *mut RunQueue];
    unsafe {
        wake_up_new_task(ptr_of(&mut child), &registry).unwrap();
        assert_eq!((*ptr_of(&mut child)).state(), TaskState::Runnable);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/core.rs (line 312) - [internal]

exit_bookkeeping_dequeues_and_marks_zombie

```
fn exit_bookkeeping_dequeues_and_marks_zombie() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();
    let mut t = make_task(1, SchedPolicy::Normal, 0);

    unsafe {
        let flags = rq.lock.lock_irqsave();
        class::enqueue(&mut rq, ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);
        rq.lock.unlock_irqrestore(flags);

        let returned_rq = exit_bookkeeping(ptr_of(&mut t));
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 332) - [internal]

exit_bookkeeping_on_unqueued_task_returns_null_rq

```
fn exit_bookkeeping_on_unqueued_task_returns_null_rq() {
    let mut t = make_task(1, SchedPolicy::Normal, 0);
    unsafe {
        let rq_ptr = exit_bookkeeping(ptr_of(&mut t));
        assert!(rq_ptr.is_null());
        assert_eq!((*ptr_of(&mut t)).state(), TaskState::Zombie);
    }
}

#[test]
fn reap_zombie_destroys_and_frees_pid() {
    let mut t = make_task(1, SchedPolicy::Normal, 0);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 342) - [internal]

reap_zombie_destroys_and_frees_pid

```
fn reap_zombie_destroys_and_frees_pid() {
    let mut t = make_task(1, SchedPolicy::Normal, 0);
    let pid = 26_000;
    t.pid = pid;
    assert!(reserve_pid(pid));

    unsafe {
        let _ = t.set_state(TaskState::Zombie);
        reap_zombie(ptr_of(&mut t));
    }

    assert_eq!(t.state(), TaskState::Dead);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 358) - [internal]

maybe_reschedule_is_noop_when_not_needed

```
fn maybe_reschedule_is_noop_when_not_needed() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();
    let registry: [*mut RunQueue; 1] = [&mut rq as *mut RunQueue];
    unsafe {
        maybe_reschedule(0, &registry);
    }
}

#[test]
fn sys_nice_clamps_to_valid_range() {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 369) - [internal]

sys_nice_clamps_to_valid_range

```
fn sys_nice_clamps_to_valid_range() {
    let mut t = make_task(1, SchedPolicy::Normal, 0);
    unsafe {
        sys_nice(ptr_of(&mut t), 100).unwrap();
        assert_eq!(weight_to_nice((*ptr_of(&mut t)).se.weight), NICE_MAX);
        sys_nice(ptr_of(&mut t), -100).unwrap();
        assert_eq!(weight_to_nice((*ptr_of(&mut t)).se.weight), NICE_MIN);
    }
}

#[test]
fn sys_nice_rejects_non_fair_policy() {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 380) - [internal]

sys_nice_rejects_non_fair_policy

```
fn sys_nice_rejects_non_fair_policy() {
    let mut t = make_task(1, SchedPolicy::Fifo, 0);
    unsafe {
        assert!(sys_nice(ptr_of(&mut t), 1).is_err());
    }
}
```

#[test]

```
fn sys_sched_setscheduler_moves_task_to_new_class() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 388) - [internal]

sys_sched_setscheduler_moves_task_to_new_class

```
fn sys_sched_setscheduler_moves_task_to_new_class() {
    let mut idle = make_idle(0);
    let mut rq = RunQueue::new(0, ptr_of(&mut idle));
    rq.bind_idle_task();
    let mut t = make_task(1, SchedPolicy::Normal, 0);

    unsafe {
        let flags = rq.lock.lock_irqsave();
        class::enqueue(&mut rq, ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);
        rq.lock.unlock_irqrestore(flags);

        sys_sched_setscheduler(&mut rq, ptr_of(&mut t), SchedPolicy::Fifo);
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 407) - [internal]

sys_sched_rr_get_interval_matches_class_dispatch

```
fn sys_sched_rr_get_interval_matches_class_dispatch() {
    let fifo = make_task(1, SchedPolicy::Fifo, 0);
    let rr = make_task(2, SchedPolicy::RoundRobin, 0);
    unsafe {
        assert_eq!(sys_sched_rr_get_interval(&fifo as *const _), 0);
        assert!(sys_sched_rr_get_interval(&rr as *const _) > 0);
    }
}
```

#[test]

```
fn sys_sched_affinity_roundtrips() {
    let mut t = make_task(1, SchedPolicy::Normal, 0);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 417) - [internal]

sys_sched_affinity_roundtrips

```
fn sys_sched_affinity_roundtrips() {
    let mut t = make_task(1, SchedPolicy::Normal, 0);
    unsafe {
        let mask = CpuMask::single(3);
        sys_sched_setaffinity(ptr_of(&mut t), mask).unwrap();
        assert_eq!(sys_sched_getaffinity(ptr_of(&mut t)), mask);
    }
}
```

#[test]

```
fn priority_range_helpers_match_posix_expectations() {
    assert_eq!(sys_sched_get_priority_min(SchedPolicy::Fifo), 0);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/core.rs (line 427) - [internal]

priority_range_helpers_match_posix_expectations

```
fn priority_range_helpers_match_posix_expectations() {  
    assert_eq!(sys_sched_get_priority_min(SchedPolicy::Fifo), 0);  
    assert_eq!(sys_sched_get_priority_max(SchedPolicy::Fifo), MAX_RT_PRIO - 1);  
    assert_eq!(sys_sched_get_priority_max(SchedPolicy::Normal), 0);  
    assert_eq!(sys_sched_get_priority_min(SchedPolicy::Normal), 0);  
}  
}
```

kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 1) - [public] Tracepoint

```
pub struct Tracepoint {}  
  
pub struct TracePointId {}  
  
pub struct Tracepointkind{}  
  
pub struct tracepoints {  
    // za chwile  
}  
  
pub struct TraceEvent{  
  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 3) - [public]

TracePointId

```
pub struct TracePointId {}  
  
pub struct Tracepointkind{}  
  
pub struct tracepoints {  
    // za chwile  
}  
  
pub struct TraceEvent{  
  
}  
  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 5) - [public]

Tracepointkind

```
pub struct Tracepointkind{}  
  
pub struct tracepoints {  
    // za chwile  
}  
  
pub struct TraceEvent{  
  
}  
  
pub enum TraceData{  
  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 7) - [public] tracepoints

```
pub struct tracepoints {  
    // za chwile  
}  
  
pub struct TraceEvent{  
  
}  
  
pub enum TraceData{  
  
}  
  
// ... (truncated, see source file for more)
```



```
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 11) - [public] TraceEvent
pub struct TraceEvent{
}

pub enum TraceData{
}

pub struct ContextSwichTraceData{
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 15) - [public] TraceData
pub enum TraceData{
}

pub struct ContextSwichTraceData{
}

pub struct TraceContext{
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 19) - [public]
ContextSwichTraceData
pub struct ContextSwichTraceData{
}

pub struct TraceContext{
}

pub struct TracepointRegistry{
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 23) - [public]
TraceContext
pub struct TraceContext{
}

pub struct TracepointRegistry{
}

pub struct RegisteredTracepoint{}

pub struct TraceBuffer{}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 27) - [public]

TracepointRegistry

```
pub struct TracepointRegistry{  
  
}  
  
pub struct RegisteredTracepoint{  
  
pub struct TraceBuffer{  
  
pub struct PerCpuTracebuffer{  
  
// TODO uzupełnij potencjalnie bo na ten moment nie ma czym
```

kernel_workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 31) - [public]

RegisteredTracepoint

```
pub struct RegisteredTracepoint{  
  
pub struct TraceBuffer{  
  
pub struct PerCpuTracebuffer{  
  
// TODO uzupełnij potencjalnie bo na ten moment nie ma czym
```

kernel_workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 33) - [public]

TraceBuffer

```
pub struct TraceBuffer{  
  
pub struct PerCpuTracebuffer{  
  
// TODO uzupełnij potencjalnie bo na ten moment nie ma czym
```

kernel_workspace/kernel/src/cpu/scheduler/debug/tracepoints.rs (line 35) - [public]

PerCpuTracebuffer

```
pub struct PerCpuTracebuffer{  
  
// TODO uzupełnij potencjalnie bo na ten moment nie ma czym
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 19) - [public]

EnqueueFlags

```
pub struct EnqueueFlags: u32 {  
    const ENQUEUE_RESTORE    = 1 << 0;  
    const ENQUEUE_WAKEUP     = 1 << 1;  
    const ENQUEUE_NEW        = 1 << 2;  
    const ENQUEUE_MIGRATED   = 1 << 3;  
    const ENQUEUE_HEAD       = 1 << 4;  
}  
  
bitflags::bitflags! {  
    #[derive(Debug, Clone, Copy, PartialEq, Eq)]  
    pub struct DequeueFlags: u32 {  
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 30) - [public]

DequeueFlags

```
pub struct DequeueFlags: u32 {  
    const DEQUEUE_SLEEP      = 1 << 0;  
    const DEQUEUE_SAVE       = 1 << 1;  
    const DEQUEUE_MIGRATING  = 1 << 2;  
}  
  
pub const MIN_GRANULARITY_NS: u64 = 750_000; // 0.75 ms  
pub const TARGET_LATENCY_NS: u64 = 6_000_000; // 6 ms  
pub const BALANCE_INTERVAL_NS: u64 = 4_000_000;  
pub const IMBALANCE_THRESHOLD: u32 = 2;  
  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 189) - [internal]

insert_fixup

```
fn insert_fixup(root: &mut *mut TaskStruct, mut z: *mut TaskStruct) {
    unsafe {
        while color_of(parent_of(z)) == RB_RED {
            let zp = parent_of(z);
            let zpp = parent_of(zp);
            debug_assert(!zpp.is_null(), "rodzic czerwonego w zla nie moze by korzeniem");
            if zp == left_of(zpp) {
                let y = right_of(zpp);
                if color_of(y) == RB_RED {
                    set_color(zp, RB_BLACK);
                    set_color(y, RB_BLACK);
                    set_color(zpp, RB_RED);
                }
            }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 240) - [public] insert

```
pub fn insert(root: &mut *mut TaskStruct, leftmost: &mut *mut TaskStruct, node: *mut TaskStruct, key: i32) {
    unsafe {
        debug_assert(!node.is_null());
        (*node).se.rb_left = ptr::null_mut();
        (*node).se.rb_right = ptr::null_mut();
        (*node).se.rb_parent_color = rb_make_parent_color(ptr::null_mut(), RB_RED);

        let node_key = key_of(node);
        let mut parent: *mut TaskStruct = ptr::null_mut();
        let mut cur = *root;
        let mut is_leftmost = true;

        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 293) - [internal]

delete_fixup

```
fn delete_fixup(root: &mut *mut TaskStruct, mut x: *mut TaskStruct, mut x_parent: *mut TaskStruct) {
    unsafe {
        while x != *root && color_of(x) == RB_BLACK {
            if x_parent.is_null() {
                break;
            }
            if x == left_of(x_parent) {
                let mut w = right_of(x_parent);
                if color_of(w) == RB_RED {
                    set_color(w, RB_BLACK);
                    set_color(x_parent, RB_RED);
                    rotate_left(root, x_parent);
                }
            }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 357) - [public] delete

```
pub fn delete(root: &mut *mut TaskStruct, leftmost: &mut *mut TaskStruct, z: *mut TaskStruct) {
    unsafe {
        debug_assert(!z.is_null());
        let mut y = z;
        let mut y_original_color = color_of(y);
        let x: *mut TaskStruct;
        let x_parent: *mut TaskStruct;

        if left_of(z).is_null() {
            x = right_of(z);
            x_parent = parent_of(z);
            transplant(root, z, right_of(z));
        } else {
            x = left_of(z);
            x_parent = parent_of(x);
            if x == left_of(x_parent) {
                right_rotate(x_parent, x, z);
            } else {
                left_rotate(x_parent, x, z);
            }
        }
    }
    // ... (truncated, see source file for more)
```

```
kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 423) - [internal] check
fn check(node: *mut TaskStruct, key_of: KeyOf) -> Result<usize, &'static str> {
    unsafe {
        if node.is_null() {
            return Ok(1);
        }
        if color_of(node) == RB_RED {
            if color_of(left_of(node)) == RB_RED || color_of(right_of(node)) == RB_RED {
                return Err("czerwony w████ze████ ma czerwone dziecko");
            }
        }
        if !left_of(node).is_null() && key_of(left_of(node)) > key_of(node) {
            return Err("naruszony porz████dek BST (lewe dziecko)");
        }
    }
    // ... (truncated, see source file for more)
}
```

```
kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 452) - [internal]
fair_key_of
```

```
fn fair_key_of(node: *const TaskStruct) -> u64 {
    unsafe { (*node).se.vruntime }
}

#[repr(C)]
pub struct RqFair {
    root: *mut TaskStruct,
    leftmost: *mut TaskStruct,
    pub nr_running: u32,
    pub load_weight: u64,
    pub min_vruntime: u64,
}

// ... (truncated, see source file for more)
```

```
kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 457) - [public] RqFair
```

```
pub struct RqFair {
    root: *mut TaskStruct,
    leftmost: *mut TaskStruct,
    pub nr_running: u32,
    pub load_weight: u64,
    pub min_vruntime: u64,
}

impl Default for RqFair {
    fn default() -> Self {
        Self {
            root: ptr::null_mut(),
        }
        // ... (truncated, see source file for more)
    }
}
```

```
kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 466) - [internal] default
```

```
fn default() -> Self {
    Self {
        root: ptr::null_mut(),
        leftmost: ptr::null_mut(),
        nr_running: 0,
        load_weight: 0,
        min_vruntime: 0,
    }
}

impl RqFair {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 478) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.root.is_null()
}

pub fn leftmost(&self) -> *mut TaskStruct {
    self.leftmost
}

fn place_entity(&self, task: *mut TaskStruct, flags: EnqueueFlags) {
    unsafe {
        let se = &mut (*task).se;
        if flags.intersects(EnqueueFlags::ENQUEUE_NEW | EnqueueFlags::ENQUEUE_WAKEUP)
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 482) - [public] leftmost

```
pub fn leftmost(&self) -> *mut TaskStruct {
    self.leftmost
}

fn place_entity(&self, task: *mut TaskStruct, flags: EnqueueFlags) {
    unsafe {
        let se = &mut (*task).se;
        if flags.intersects(EnqueueFlags::ENQUEUE_NEW | EnqueueFlags::ENQUEUE_WAKEUP)
            && !flags.contains(EnqueueFlags::ENQUEUE_MIGRATED)
        {
            se.vruntime = cmp::max(se.vruntime, self.min_vruntime);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 486) - [internal]

place_entity

```
fn place_entity(&self, task: *mut TaskStruct, flags: EnqueueFlags) {
    unsafe {
        let se = &mut (*task).se;
        if flags.intersects(EnqueueFlags::ENQUEUE_NEW | EnqueueFlags::ENQUEUE_WAKEUP)
            && !flags.contains(EnqueueFlags::ENQUEUE_MIGRATED)
        {
            se.vruntime = cmp::max(se.vruntime, self.min_vruntime);
        }
    }
}

/// Naprawa wady #2: prawdziwe wstawienie do drzewa RB (nie
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 499) - [public] enqueue

/// Naprawa wady #2: prawdziwe wstawienie do drzewa RB (nie

/// pojedynczy wskaźnik udajemy drzewo).

```
pub fn enqueue(&mut self, task: *mut TaskStruct, flags: EnqueueFlags) {
    debug_assert(!task.is_null());
    self.place_entity(task, flags);
    rbtree::insert(&mut self.root, &mut self.leftmost, task, fair_key_of);
    unsafe {
        (*task).se.on_rq = true;
        self.load_weight = self.load_weight.saturating_add((*task).se.weight);
    }
    self.nr_running += 1;
}

pub fn dequeue(&mut self, task: *mut TaskStruct) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 510) - [public] dequeue

```
pub fn dequeue(&mut self, task: *mut TaskStruct) {
    debug_assert(!task.is_null());
    rbtrees::delete(&mut self.root, &mut self.leftmost, task);
    unsafe {
        (*task).se.on_rq = false;
        self.load_weight = self.load_weight.saturating_sub((*task).se.weight);
    }
    self.nr_running = self.nr_running.saturating_sub(1);
}

pub fn pick_first(&self) -> *mut TaskStruct {
    self.leftmost
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 520) - [public] pick_first

```
pub fn pick_first(&self) -> *mut TaskStruct {
    self.leftmost
}

pub fn charge_exec(&mut self, task: *mut TaskStruct, delta_exec: u64, now: u64) -> u64 {
    unsafe {
        let se = &mut (*task).se;
        let delta_vruntime = calc_delta_fair(delta_exec, se.weight, se.inv_weight);
        se.vruntime = se.vruntime.saturating_add(delta_vruntime);
        se.load.accumulate(now, delta_exec, se.weight, true);

        self.min_vruntime = cmp::max(self.min_vruntime, se.vruntime);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 524) - [public] charge_exec

```
pub fn charge_exec(&mut self, task: *mut TaskStruct, delta_exec: u64, now: u64) -> u64 {
    unsafe {
        let se = &mut (*task).se;
        let delta_vruntime = calc_delta_fair(delta_exec, se.weight, se.inv_weight);
        se.vruntime = se.vruntime.saturating_add(delta_vruntime);
        se.load.accumulate(now, delta_exec, se.weight, true);

        self.min_vruntime = cmp::max(self.min_vruntime, se.vruntime);
        if !self.leftmost.is_null() {
            let leftmost_key = fair_key_of(self.leftmost);
            self.min_vruntime = cmp::min(self.min_vruntime, cmp::max(leftmost_key, self.min_vruntime));
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 540) - [public] should_preempt

```
pub fn should_preempt(&self, current: *const TaskStruct, candidate: *const TaskStruct) -> bool {
    unsafe {
        if !fair_has_priority(&*candidate, &*current) {
            return false;
        }
        let vdiff = (*current).se.vruntime.saturating_sub((*candidate).se.vruntime);
        let ran_for = (*current).se.sum_exec_runtime.saturating_sub((*current).se.prev_sum_exec_runtime);
        vdiff > 0 && ran_for >= MIN_GRANULARITY_NS || vdiff > TARGET_LATENCY_NS
    }
}

#[cfg(test)]
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 552) - [public]

rb_invariants_ok

```
pub fn rb_invariants_ok(&self) -> bool {
    unsafe { rbtree::check_invariants(self.root, fair_key_of).is_ok() }
}

#[cfg(test)]
pub fn rb_count(&self) -> usize {
    unsafe { rbtree::count(self.root) }
}

fn dl_key_of(node: *const TaskStruct) -> u64 {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 557) - [public] rb_count

```
pub fn rb_count(&self) -> usize {
    unsafe { rbtree::count(self.root) }
}

fn dl_key_of(node: *const TaskStruct) -> u64 {
    unsafe { (*node).dl.deadline }
}

#[repr(C)]
pub struct RqDl {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 563) - [internal]

dl_key_of

```
fn dl_key_of(node: *const TaskStruct) -> u64 {
    unsafe { (*node).dl.deadline }
}

#[repr(C)]
pub struct RqDl {
    root: *mut TaskStruct,
    leftmost: *mut TaskStruct,
    pub dl_nr_running: u32,
    pub running_bw: u64,
    pub max_bw: u64,
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 568) - [public] RqDl

```
pub struct RqDl {
    root: *mut TaskStruct,
    leftmost: *mut TaskStruct,
    pub dl_nr_running: u32,
    pub running_bw: u64,
    pub max_bw: u64,
}

pub const DL_BW_SCALE: u64 = 1 << 20;

impl Default for RqDl {
    fn default() -> Self {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 579) - [internal] default

```
fn default() -> Self {
    Self {
        root: ptr::null_mut(),
        leftmost: ptr::null_mut(),
        dl_nr_running: 0,
        running_bw: 0,
        max_bw: (DL_BW_SCALE * 95) / 100,
    }
}

impl RqDl {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 591) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.root.is_null()
}

pub fn leftmost(&self) -> *mut TaskStruct {
    self.leftmost
}

pub fn admission_control(&self, dl_runtime: u64, dl_period: u64) -> bool {
    if dl_period == 0 {
        return false;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 595) - [public] leftmost

```
pub fn leftmost(&self) -> *mut TaskStruct {
    self.leftmost
}

pub fn admission_control(&self, dl_runtime: u64, dl_period: u64) -> bool {
    if dl_period == 0 {
        return false;
    }
    let bw = (dl_runtime.saturating_mul(DL_BW_SCALE)) / dl_period;
    self.running_bw.saturating_add(bw) <= self.max_bw
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 599) - [public] admission_control

```
pub fn admission_control(&self, dl_runtime: u64, dl_period: u64) -> bool {
    if dl_period == 0 {
        return false;
    }
    let bw = (dl_runtime.saturating_mul(DL_BW_SCALE)) / dl_period;
    self.running_bw.saturating_add(bw) <= self.max_bw
}

fn task_bw(task: *const TaskStruct) -> u64 {
    unsafe {
        let dl = &(*task).dl;
        if dl.dl_period == 0 {
            // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 607) - [internal] task_bw

```
fn task_bw(task: *const TaskStruct) -> u64 {
    unsafe {
        let dl = &(*task).dl;
        if dl.dl_period == 0 {
            0
        } else {
            (dl.dl_runtime.saturating_mul(DL_BW_SCALE)) / dl.dl_period
        }
    }
}

pub fn enqueue(&mut self, task: *mut TaskStruct, now: u64, flags: EnqueueFlags) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 618) - [public] enqueue

```
pub fn enqueue(&mut self, task: *mut TaskStruct, now: u64, flags: EnqueueFlags) {
    debug_assert(!task.is_null());
    unsafe {
        let dl = &mut (*task).dl;
        if flags.contains(EnqueueFlags::ENQUEUE_NEW) {
            // Pierwsze wejście do systemu: zacznij nowy okres CBS
            // teraz, z pełnym budżetem.
            dl.runtime = dl.dl_runtime as i64;
            dl.deadline = now.saturating_add(dl.dl_deadline);
            dl.replenish_at = now.saturating_add(dl.dl_period);
            self.running_bw = self.running_bw.saturating_add(Self::task_bw(task));
        } else if dl.throttled {
            // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 639) - [public] dequeue

```
pub fn dequeue(&mut self, task: *mut TaskStruct, removing_permanently: bool) {
    debug_assert(!task.is_null());
    rbtree::delete(&mut self.root, &mut self.leftmost, task);
    unsafe { (*task).se.on_rq = false };
    self.dl_nr_running = self.dl_nr_running.saturating_sub(1);
    if removing_permanently {
        self.running_bw = self.running_bw.saturating_sub(Self::task_bw(task));
    }
}

pub fn pick_first(&self) -> *mut TaskStruct {
    self.leftmost
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 649) - [public]

pick_first

```
pub fn pick_first(&self) -> *mut TaskStruct {
    self.leftmost
}

pub fn update_curr(&mut self, task: *mut TaskStruct, delta_exec: u64, now: u64) {
    unsafe {
        let dl = &mut (*task).dl;
        dl.runtime -= delta_exec as i64;

        if now >= dl.deadline {
            dl.deadline = now.saturating_add(dl.dl_deadline);
            dl.runtime = dl.dl_runtime as i64;
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 653) - [public]
update_curr

```
pub fn update_curr(&mut self, task: *mut TaskStruct, delta_exec: u64, now: u64) {
    unsafe {
        let dl = &mut (*task).dl;
        dl.runtime -= delta_exec as i64;

        if now >= dl.deadline {
            dl.deadline = now.saturating_add(dl.dl_deadline);
            dl.runtime = dl.dl_runtime as i64;
            dl.throttled = false;
            (*task).flags.fetch_remove(TaskFlags::PF_DL_THROTTLED);
        } else if dl.runtime <= 0 {
            dl.throttled = true;
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 670) - [public]
should_preempt

```
pub fn should_preempt(&self, current: *const TaskStruct, candidate: *const TaskStruct) -> bool {
    unsafe { deadline_has_priority(&*candidate, &*current) }
}

#[cfg(test)]
pub fn rb_invariants_ok(&self) -> bool {
    unsafe { rbtree::check_invariants(self.root, dl_key_of).is_ok() }
}
}
```

```
const RT_BITMAP_WORDS: usize = (MAX_RT_PRIO as usize + 63) / 64;
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 675) - [public]
rb_invariants_ok

```
pub fn rb_invariants_ok(&self) -> bool {
    unsafe { rbtree::check_invariants(self.root, dl_key_of).is_ok() }
}
}
```

```
const RT_BITMAP_WORDS: usize = (MAX_RT_PRIO as usize + 63) / 64;
```

```
#[repr(C)]
pub struct RqRt {
    active: [ListHead; MAX_RT_PRIO as usize],
    bitmap: [u64; RT_BITMAP_WORDS],
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 684) - [public] RqRt

```
pub struct RqRt {
    active: [ListHead; MAX_RT_PRIO as usize],
    bitmap: [u64; RT_BITMAP_WORDS],
    pub nr_running: u32,
    pub rt_nr_migratory: u32,
    pub overloaded: bool,
    highest_prio: i32,
}

impl RqRt {
    pub fn new() -> Self {
        const INIT: ListHead = ListHead { prev: ptr::null_mut(), next: ptr::null_mut() };
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 694) - [public] new

```
pub fn new() -> Self {
    const INIT: ListHead = ListHead { prev: ptr::null_mut(), next: ptr::null_mut() };
    let mut rq = Self {
        active: [INIT; MAX_RT_PRIO as usize],
        bitmap: [0; RT_BITMAP_WORDS],
        nr_running: 0,
        rt_nr_migratory: 0,
        overloaded: false,
        highest_prio: MAX_RT_PRIO,
    };
    for head in rq.active.iter_mut() {
        head.init();
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 710) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.nr_running == 0
}

fn set_bit(&mut self, prio: usize) {
    self.bitmap[prio / 64] |= 1u64 << (prio % 64);
}

fn clear_bit(&mut self, prio: usize) {
    self.bitmap[prio / 64] &= !(1u64 << (prio % 64));
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 714) - [internal] set_bit

```
fn set_bit(&mut self, prio: usize) {
    self.bitmap[prio / 64] |= 1u64 << (prio % 64);
}

fn clear_bit(&mut self, prio: usize) {
    self.bitmap[prio / 64] &= !(1u64 << (prio % 64));
}

fn sched_find_first_bit(&self) -> Option<usize> {
    for (word_idx, word) in self.bitmap.iter().enumerate() {
        if *word != 0 {
            let bit = word.trailing_zeros() as usize;
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 718) - [internal]

clear_bit

```
fn clear_bit(&mut self, prio: usize) {
    self.bitmap[prio / 64] &= !(1u64 << (prio % 64));
}

fn sched_find_first_bit(&self) -> Option<usize> {
    for (word_idx, word) in self.bitmap.iter().enumerate() {
        if *word != 0 {
            let bit = word.trailing_zeros() as usize;
            let prio = word_idx * 64 + bit;
            if prio < MAX_RT_PRIO as usize {
                return Some(prio);
            }
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 722) - [internal]
sched_find_first_bit

```
fn sched_find_first_bit(&self) -> Option<usize> {  
    for (word_idx, word) in self.bitmap.iter().enumerate() {  
        if *word != 0 {  
            let bit = word.trailing_zeros() as usize;  
            let prio = word_idx * 64 + bit;  
            if prio < MAX_RT_PRIO as usize {  
                return Some(prio);  
            }  
        }  
    }  
    None  
}  
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 735) - [public] enqueue

```
pub fn enqueue(&mut self, task: *mut TaskStruct, flags: EnqueueFlags) {  
    debug_assert(!task.is_null());  
    unsafe {  
        let prio = (*task).prio;  
        debug_assert!(prio >= 0 && prio < MAX_RT_PRIO, "prio RT poza zakresem 0..MAX_RT_PRIO");  
        let prio = prio as usize;  
  
        debug_assert!(  
            core::ptr::eq((*task).rt.owner, task),  
            "RtSchedEntity.owner musi wskazywać na właściwą cielę – zapomniano TaskStruct::init?"  
        );  
  
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 769) - [public] dequeue

```
pub fn dequeue(&mut self, task: *mut TaskStruct) {  
    debug_assert(!task.is_null());  
    unsafe {  
        let prio = (*task).rt.queued_prio as usize;  
        debug_assert!(prio < MAX_RT_PRIO as usize, "dequeue zadania, które nie było zakolejkowane");  
  
        (*task).rt.rt_list.remove();  
        (*task).se.on_rq = false;  
        (*task).rt.queued_prio = MAX_RT_PRIO as u32;  
  
        if self.active[prio].is_empty() {  
            self.clear_bit(prio);  
        }  
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 793) - [public]
pick_first

```
pub fn pick_first(&self) -> *mut TaskStruct {  
    match self.sched_find_first_bit() {  
        Some(prio) => unsafe {  
            let head = &self.active[prio] as *const ListHead as *mut ListHead;  
            let front = (*head).next;  
            if front.is_null() || core::ptr::eq(front, head) {  
                ptr::null_mut()  
            } else {  
                let rt_entity = front as *mut super::task::RtSchedEntity;  
                (*rt_entity).owner  
            }  
        },  
        None => {  
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 809) - [public] requeue

```
pub fn requeue(&mut self, task: *mut TaskStruct) {
    debug_assert!(!task.is_null());
    unsafe {
        let prio = (*task).rt.queued_prio as usize;
        debug_assert!(prio < MAX_RT_PRIO as usize);
        (*task).rt.rt_list.remove();
        let node_ptr = &mut (*task).rt.rt_list as *mut ListHead;
        self.active[prio].insert_before(node_ptr);
    }
}

pub fn highest_priority(&self) -> i32 {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 820) - [public] highest_priority

```
pub fn highest_priority(&self) -> i32 {
    self.highest_prio
}

#[repr(C)]
#[derive(Default)]
pub struct RqStop {
    task: *mut TaskStruct,
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 828) - [public] RqStop

```
pub struct RqStop {
    task: *mut TaskStruct,
}

impl RqStop {
    pub fn is_empty(&self) -> bool {
        self.task.is_null()
    }

    pub fn enqueue(&mut self, task: *mut TaskStruct) {
        debug_assert!(self.task.is_null(), "co najwyzej jedno zadanie Stop naraz na CPU");
        unsafe { (*task).se.on_rq = true };
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 833) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.task.is_null()
}

pub fn enqueue(&mut self, task: *mut TaskStruct) {
    debug_assert!(self.task.is_null(), "co najwyzej jedno zadanie Stop naraz na CPU");
    unsafe { (*task).se.on_rq = true };
    self.task = task;
}

pub fn dequeue(&mut self, task: *mut TaskStruct) {
    if self.task == task {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 837) - [public] enqueue

```
pub fn enqueue(&mut self, task: *mut TaskStruct) {
    debug_assert!(self.task.is_null(), "co najwyzej jedno zadanie Stop naraz na CPU");
    unsafe { (*task).se.on_rq = true };
    self.task = task;
}

pub fn dequeue(&mut self, task: *mut TaskStruct) {
    if self.task == task {
        unsafe { (*task).se.on_rq = false };
        self.task = ptr::null_mut();
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 843) - [public] dequeue

```
pub fn dequeue(&mut self, task: *mut TaskStruct) {
    if self.task == task {
        unsafe { (*task).se.on_rq = false };
        self.task = ptr::null_mut();
    }
}

pub fn pick_first(&self) -> *mut TaskStruct {
    self.task
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 850) - [public]

pick_first

```
pub fn pick_first(&self) -> *mut TaskStruct {
    self.task
}
}
```

#[repr(C)]

```
pub struct RunQueue {
    pub lock: SpinLock,
    pub cpu: u32,
    pub online: AtomicBool,
    pub clock: AtomicU64,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 857) - [public] RunQueue

```
pub struct RunQueue {
    pub lock: SpinLock,
    pub cpu: u32,
    pub online: AtomicBool,
    pub clock: AtomicU64,
    pub clock_task: AtomicU64,

    pub fair: RqFair,
    pub rt: RqRt,
    pub dl: RqDl,
    pub stop: RqStop,

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 882) - [public] new

```
pub fn new(cpu: u32, idle_task: *mut TaskStruct) -> Self {
    assert!(!idle_task.is_null(), "RunQueue::new wymaga poprawnego zadania idle");
    unsafe {
        debug_assert!(
            (*idle_task).sched_class == SchedClass::Idle,
            "zadanie przekazane jako idle musi mieć sched_class == Idle"
        );
    }
    let rq = Self {
        lock: SpinLock::new(),
        cpu,
        online: AtomicBool::new(true),
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 914) - [public]

bind_idle_task

```
pub fn bind_idle_task(&self) {
    unsafe {
        (*self.idle).set_rq_ptr(self as *const RunQueue as *mut RunQueue as *mut core::ffi::c_void);
    }
}

fn now(&self) -> u64 {
    self.clock.load(Ordering::Relaxed)
}

pub unsafe fn advance_clock(&self, delta_ns: u64) {
    debug_assert!(self.lock.is_locked(), "advance_clock wymaga self.lock");
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 920) - [internal] now

```
fn now(&self) -> u64 {
    self.clock.load(Ordering::Relaxed)
}

pub unsafe fn advance_clock(&self, delta_ns: u64) {
    debug_assert!(self.lock.is_locked(), "advance_clock wymaga self.lock");
    self.clock.fetch_add(delta_ns, Ordering::Relaxed);
    self.clock_task.fetch_add(delta_ns, Ordering::Relaxed);
}

pub fn current(&self) -> *mut TaskStruct {
    self.curr.load(Ordering::Acquire)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 930) - [public] current

```
pub fn current(&self) -> *mut TaskStruct {
    self.curr.load(Ordering::Acquire)
}

fn set_current(&self, task: *mut TaskStruct) {
    self.curr.store(task, Ordering::Release);
}

pub fn nr_running(&self) -> u32 {
    self.nr_running.load(Ordering::Relaxed)
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 934) - [internal]

set_current

```
fn set_current(&self, task: *mut TaskStruct) {
    self.curr.store(task, Ordering::Release);
}

pub fn nr_running(&self) -> u32 {
    self.nr_running.load(Ordering::Relaxed)
}

pub fn is_idle(&self) -> bool {
    core::ptr::eq(self.current(), self.idle)
}

pub unsafe fn enqueue_task(&mut self, task: *mut TaskStruct, flags: EnqueueFlags) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 938) - [public]

nr_running

```
pub fn nr_running(&self) -> u32 {
    self.nr_running.load(Ordering::Relaxed)
}

pub fn is_idle(&self) -> bool {
    core::ptr::eq(self.current(), self.idle)
}

pub unsafe fn enqueue_task(&mut self, task: *mut TaskStruct, flags: EnqueueFlags) {
    debug_assert!(task.is_null(), "enqueue_task: task == null");
    debug_assert!(self.lock.is_locked(), "enqueue_task wymaga self.lock");
    debug_assert!(
        !(*task).se.on_rq,
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 942) - [public] is_idle

```
pub fn is_idle(&self) -> bool {
    core::ptr::eq(self.current(), self.idle)
}

pub unsafe fn enqueue_task(&mut self, task: *mut TaskStruct, flags: EnqueueFlags) {
    debug_assert!(task.is_null(), "enqueue_task: task == null");
    debug_assert!(self.lock.is_locked(), "enqueue_task wymaga self.lock");
    debug_assert!(
        !(*task).se.on_rq,
        "enqueue_task: zadanie jest już zakolejkowane (podwójny enqueue)"
    );

    let now = self.now();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1001) - [internal]

nr_uninterruptible_dec_if

```
fn nr_uninterruptible_dec_if(&self, task: *mut TaskStruct) {
    unsafe {
        if (*task).state() == TaskState::Runnable {
            let prev = self.nr_uninterruptible.load(Ordering::Relaxed);
            if prev > 0 {
                self.nr_uninterruptible.fetch_sub(1, Ordering::Relaxed);
            }
        }
    }
}

pub unsafe fn activate_task(&mut self, task: *mut TaskStruct, flags: EnqueueFlags) {
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1202) - [internal]

fair_max_vruntime

```
fn fair_max_vruntime(&self) -> Option<u64> {
    unsafe {
        let max_node = rbtree::subtree_max(self.fair_root_for_test());
        if max_node.is_null() {
            None
        } else {
            Some((*max_node).se.vruntime)
        }
    }
}

#[cfg(test)]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1214) - [public]

fair_root_for_test

```
pub fn fair_root_for_test(&self) -> *mut TaskStruct {
    self.fair_root_internal()
}

#[cfg(not(test))]
fn fair_root_for_test(&self) -> *mut TaskStruct {
    self.fair_root_internal()
}

fn fair_root_internal(&self) -> *mut TaskStruct {
    self.fair.root
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1219) - [internal]

fair_root_for_test

```
fn fair_root_for_test(&self) -> *mut TaskStruct {
    self.fair_root_internal()
}

fn fair_root_internal(&self) -> *mut TaskStruct {
    self.fair.root
}

pub unsafe fn schedule_tail(&mut self, prev: *mut TaskStruct) {
    if prev.is_null() {
        return;
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1223) - [internal]

fair_root_internal

```
fn fair_root_internal(&self) -> *mut TaskStruct {
    self.fair.root
}

pub unsafe fn schedule_tail(&mut self, prev: *mut TaskStruct) {
    if prev.is_null() {
        return;
    }
    if (*prev).is_zombie() {
        (*prev).on_cpu.store(false, Ordering::Release);
    } else if (*prev).se.on_rq {
        (*prev).record_involuntary_switch();
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1246) - [internal] fmt

```
fn fmt(&self, f: &mut core::fmt::Formatter<'>) -> core::fmt::Result {
    f.debug_struct("RunQueue")
        .field("cpu", &self.cpu)
        .field("nr_running", &self.nr_running.load(Ordering::Relaxed))
        .field("fair_nr", &self.fair.nr_running)
        .field("rt_nr", &self.rt.nr_running)
        .field("dl_nr", &self.dl.dl_nr_running)
        .field("clock", &self.clock.load(Ordering::Relaxed))
        .finish()
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1294) - [internal]
can_migrate_task

```
fn can_migrate_task(task: *const TaskStruct, dst_cpu: u32) -> bool {
    unsafe {
        if (*task).flags.contains(TaskFlags::PF_NO_MIGRATE) {
            return false;
        }
        if (*task).on_cpu.load(Ordering::Relaxed) {
            // Fizycznie wykonywane teraz na jakimś CPU – przez ten cznik
            // kontekstu jest w toku, nie ruszamy.
            return false;
        }
        (*task).se.cpus_allowed.is_set(dst_cpu)
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1489) - [internal]
make_idle

```
fn make_idle(pid: TaskId, cpu: u32) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, SchedPolicy::Idle, 0);
    let _ = cpu;
    t
}

fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, policy, nice);
    t
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1496) - [internal]
make_task

```
fn make_task(pid: TaskId, policy: SchedPolicy, nice: i8) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, policy, nice);
    t
}

fn ptr_of(t: &mut TaskStruct) -> *mut TaskStruct {
    t as *mut TaskStruct
}

#[test]
fn fair_tree_maintains_rb_invariants_under_pseudorandom_inserts() {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1502) - [internal] ptr_of

```
fn ptr_of(t: &mut TaskStruct) -> *mut TaskStruct {
    t as *mut TaskStruct
}

#[test]
fn fair_tree_maintains_rb_invariants_under_pseudorandom_inserts() {
    const N: usize = 40;
    let mut tasks: [TaskStruct; N] = core::array::from_fn(|_| TaskStruct::blank());
    let mut seed: u64 = 88172645463325252;
    let mut vruntimes = [0u64; N];
    for i in 0..N {
        seed ^= seed << 13;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1507) - [internal]

fair_tree_maintains_rb_invariants_under_pseudorandom_inserts

```
fn fair_tree_maintains_rb_invariants_under_pseudorandom_inserts() {
    const N: usize = 40;
    let mut tasks: [TaskStruct; N] = core::array::from_fn(|_| TaskStruct::blank());
    let mut seed: u64 = 88172645463325252;
    let mut vruntimes = [0u64; N];
    for i in 0..N {
        seed ^= seed << 13;
        seed ^= seed >> 7;
        seed ^= seed << 17;
        vruntimes[i] = seed % 1_000_000;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1537) - [internal]

fair_tree_inorder_traversal_is_sorted_by_vruntime

```
fn fair_tree_inorder_traversal_is_sorted_by_vruntime() {
    const N: usize = 25;
    let mut tasks: [TaskStruct; N] = core::array::from_fn(|_| TaskStruct::blank());
    let mut fair = RqFair::default();
    for i in 0..N {
        tasks[i].init_test_stub(i as u64, SchedPolicy::Normal, 0);
        tasks[i].se.vruntime = ((N - i) * 1000) as u64;
        let p = ptr_of(&mut tasks[i]);
        fair.enqueue(p, EnqueueFlags::ENQUEUE_RESTORE);
    }
    let mut collected = [0u64; N];
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1566) - [internal]

fair_tree_survives_random_removals_keeping_invariants

```
fn fair_tree_survives_random_removals_keeping_invariants() {
    const N: usize = 30;
    let mut tasks: [TaskStruct; N] = core::array::from_fn(|_| TaskStruct::blank());
    let mut fair = RqFair::default();
    let mut ptrs: [*mut TaskStruct; N] = [ptr::null_mut(); N];
    for i in 0..N {
        tasks[i].init_test_stub(i as u64, SchedPolicy::Normal, 0);
        tasks[i].se.vruntime = (i as u64) * 137 % 5000;
        ptrs[i] = ptr_of(&mut tasks[i]);
        fair.enqueue(ptrs[i], EnqueueFlags::ENQUEUE_RESTORE);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1588) - [internal]
deadline_tree_orders_by_earliest_deadline

```
fn deadline_tree_orders_by_earliest_deadline() {
    let mut a = make_task(1, SchedPolicy::Deadline, 0);
    let mut b = make_task(2, SchedPolicy::Deadline, 0);
    let mut c = make_task(3, SchedPolicy::Deadline, 0);
    a.dl.deadline = 5000;
    b.dl.deadline = 1000;
    c.dl.deadline = 3000;

    let mut dl = RqDl::default();
    dl.enqueue(ptr_of(&mut a), 0, EnqueueFlags::ENQUEUE_RESTORE);
    dl.enqueue(ptr_of(&mut b), 0, EnqueueFlags::ENQUEUE_RESTORE);
    dl.enqueue(ptr_of(&mut c), 0, EnqueueFlags::ENQUEUE_RESTORE);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1609) - [internal]
rt_queue_fifo_order_at_same_priority_without_cycles

```
fn rt_queue_fifo_order_at_same_priority_without_cycles() {
    let mut a = make_task(1, SchedPolicy::Fifo, 0);
    let mut b = make_task(2, SchedPolicy::Fifo, 0);
    let mut c = make_task(3, SchedPolicy::Fifo, 0);
    // Ta sama priorytetowa "szufladka".
    a.prio = 10;
    b.prio = 10;
    c.prio = 10;
    a.rt.rt_priority = 10;
    b.rt.rt_priority = 10;
    c.rt.rt_priority = 10;

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1648) - [internal]
rt_queue_picks_highest_priority_across_levels

```
fn rt_queue_picks_highest_priority_across_levels() {
    let mut low = make_task(1, SchedPolicy::Fifo, 0);
    let mut high = make_task(2, SchedPolicy::Fifo, 0);
    low.prio = 50;
    low.rt.rt_priority = 50;
    high.prio = 5;
    high.rt.rt_priority = 5;

    let mut rt = RqRt::new();
    rt.enqueue(ptr_of(&mut low), EnqueueFlags::ENQUEUE_RESTORE);
    rt.enqueue(ptr_of(&mut high), EnqueueFlags::ENQUEUE_RESTORE);

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1667) - [internal]
rt_queue_requeue_moves_task_to_back_of_its_level

```
fn rt_queue_requeue_moves_task_to_back_of_its_level() {
    let mut a = make_task(1, SchedPolicy::RoundRobin, 0);
    let mut b = make_task(2, SchedPolicy::RoundRobin, 0);
    a.prio = 20;
    a.rt.rt_priority = 20;
    b.prio = 20;
    b.rt.rt_priority = 20;

    let mut rt = RqRt::new();
    rt.enqueue(ptr_of(&mut a), EnqueueFlags::ENQUEUE_RESTORE);
    rt.enqueue(ptr_of(&mut b), EnqueueFlags::ENQUEUE_RESTORE);

    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1689) - [internal]
pick_next_task_respects_stop_over_deadline_over_rt_over_fair_over_idle
    fn pick_next_task_respects_stop_over_deadline_over_rt_over_fair_over_idle() {
        let mut idle = make_idle(0, 0);
        let mut rq = RunQueue::new(0, ptr_of(&mut idle));
        rq.bind_idle_task();

        let mut fair_task = make_task(1, SchedPolicy::Normal, 0);
        let mut rt_task = make_task(2, SchedPolicy::Fifo, 0);
        rt_task.prio = 10;
        rt_task.rt.rt_priority = 10;
        let mut dl_task = make_task(3, SchedPolicy::Deadline, 0);
        dl_task.dl.dl_runtime = 1000;
        dl_task.dl.dl_deadline = 100_000;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1739) - [internal]
pick_next_task_never_returns_null_even_when_fully_empty
    fn pick_next_task_never_returns_null_even_when_fully_empty() {
        let mut idle = make_idle(0, 1);
        let mut rq = RunQueue::new(1, ptr_of(&mut idle));
        rq.bind_idle_task();
        unsafe {
            let flags = rq.lock.lock_irqsave();
            let picked = rq.pick_next_task();
            assert(!picked.is_null());
            assert!(core::ptr::eq(picked, ptr_of(&mut idle)));
            rq.lock.unlock_irqrestore(flags);
        }
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1754) - [internal]
deadline_task_is_picked_even_before_its_deadline_has_passed
    fn deadline_task_is_picked_even_before_its_deadline_has_passed() {
        let mut idle = make_idle(0, 2);
        let mut rq = RunQueue::new(2, ptr_of(&mut idle));
        rq.bind_idle_task();

        let mut rt_task = make_task(1, SchedPolicy::Fifo, 0);
        rt_task.prio = 0; // najwyższy możliwy priorytet RT
        rt_task.rt.rt_priority = 99;
        let mut dl_task = make_task(2, SchedPolicy::Deadline, 0);
        dl_task.dl.dl_runtime = 1_000;
        dl_task.dl.dl_deadline = 1_000_000_000; // bardzo odległy termin
        dl_task.dl.dl_period = 1_000_000_000;
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1783) - [internal]
min_vruntime_never_decreases_across_updates
    fn min_vruntime_never_decreases_across_updates() {
        let mut t = make_task(1, SchedPolicy::Normal, 0);
        let mut fair = RqFair::default();
        fair.enqueue(ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);

        let m1 = fair.charge_exec(ptr_of(&mut t), 1_000_000, 1_000_000);
        let m2 = fair.charge_exec(ptr_of(&mut t), 2_000_000, 3_000_000);
        assert!(m2 >= m1);

        let m3 = fair.charge_exec(ptr_of(&mut t), 0, 3_000_000);
        assert!(m3 >= m2);
    }
    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1797) - [internal]
newly_woken_task_does_not_start_before_min_vruntime

```
fn newly_woken_task_does_not_start_before_min_vruntime() {
    let mut veteran = make_task(1, SchedPolicy::Normal, 0);
    let mut newcomer = make_task(2, SchedPolicy::Normal, 0);

    let mut fair = RqFair::default();
    fair.enqueue(ptr_of(&mut veteran), EnqueueFlags::ENQUEUE_NEW);
    fair.charge_exec(ptr_of(&mut veteran), 50_000_000, 50_000_000);

    newcomer.se.vruntime = 0;
    fair.enqueue(ptr_of(&mut newcomer), EnqueueFlags::ENQUEUE_WAKEUP);

    unsafe {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1817) - [internal]
higher_class_always_preempts_lower_class

```
fn higher_class_always_preempts_lower_class() {
    let mut idle = make_idle(0, 3);
    let mut rq = RunQueue::new(3, ptr_of(&mut idle));
    rq.bind_idle_task();

    let mut fair_task = make_task(1, SchedPolicy::Normal, 0);
    unsafe {
        let flags = rq.lock.lock_irqsave();
        rq.enqueue_task(ptr_of(&mut fair_task), EnqueueFlags::ENQUEUE_NEW);
        rq.set_curr_task(ptr_of(&mut fair_task));
        assert!(ptr_of(&mut fair_task).needs_resched());
    }

    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1842) - [internal]
fair_preemption_requires_minimum_granularity

```
fn fair_preemption_requires_minimum_granularity() {
    let idle_owner_vrt = 0u64;
    let _ = idle_owner_vrt;
    let mut current = make_task(1, SchedPolicy::Normal, 0);
    let mut candidate = make_task(2, SchedPolicy::Normal, 0);
    current.se.vruntime = 1000;
    current.se.sum_exec_runtime = 100; // dopiero co zacz██, poni██ej MIN_GRANULARITY_NS
    current.se.prev_sum_exec_runtime = 0;
    candidate.se.vruntime = 0;

    let fair = RqFair::default();
    assert!(!fair.should_preempt(&current as *const _, &candidate as *const _));
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1860) - [internal]
deadline_preempts_via_earliest_deadline_first

```
fn deadline_preempts_via_earliest_deadline_first() {
    let mut current = make_task(1, SchedPolicy::Deadline, 0);
    let mut candidate = make_task(2, SchedPolicy::Deadline, 0);
    current.dl.deadline = 5000;
    candidate.dl.deadline = 1000;

    let dl = RqDl::default();
    assert!(dl.should_preempt(&current as *const _, &candidate as *const _));
    candidate.dl.deadline = 9000;
    assert!(!dl.should_preempt(&current as *const _, &candidate as *const _));
}

// ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1874) - [internal]
deadline_task_gets_throttled_when_budget_is_exhausted

```
fn deadline_task_gets_throttled_when_budget_is_exhausted() {
    let mut t = make_task(1, SchedPolicy::Deadline, 0);
    t.dl.dl_runtime = 10_000;
    t.dl.dl_deadline = 100_000;
    t.dl.dl_period = 100_000;

    let mut dl = RqDl::default();
    dl.enqueue(ptr_of(&mut t), 0, EnqueueFlags::ENQUEUE_NEW);
    assert!(!t.dl.throttled);

    dl.update_curr(ptr_of(&mut t), 15_000, 20_000);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1892) - [internal]
deadline_task_is_replenished_after_period_rollover

```
fn deadline_task_is_replenished_after_period_rollover() {
    let mut t = make_task(1, SchedPolicy::Deadline, 0);
    t.dl.dl_runtime = 10_000;
    t.dl.dl_deadline = 100_000;
    t.dl.dl_period = 100_000;

    let mut dl = RqDl::default();
    dl.enqueue(ptr_of(&mut t), 0, EnqueueFlags::ENQUEUE_NEW);
    assert_eq!(t.dl.deadline, 100_000);
    dl.update_curr(ptr_of(&mut t), 10_000, 100_000);

    assert!(!t.dl.throttled, "po zakończeniu okresu budżetu musi zostać odnowiony");
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1909) - [internal]
deadline_admission_control_rejects_overcommitted_bandwidth

```
fn deadline_admission_control_rejects_overcommitted_bandwidth() {
    let mut dl = RqDl::default();
    assert!(dl.admission_control(60_000, 100_000));

    let mut a = make_task(1, SchedPolicy::Deadline, 0);
    a.dl.dl_runtime = 60_000;
    a.dl.dl_period = 100_000;
    dl.enqueue(ptr_of(&mut a), 0, EnqueueFlags::ENQUEUE_NEW);
    assert!(!dl.admission_control(60_000, 100_000));
    // Ale 30% wciśnięcie się mieści (60% + 30% = 90% <= 95%).
    assert!(dl.admission_control(30_000, 100_000));
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1924) - [internal]
activate_and_deactivate_round_trip_updates_bookkeeping

```
fn activate_and_deactivate_round_trip_updates_bookkeeping() {
    let mut idle = make_idle(0, 4);
    let mut rq = RunQueue::new(4, ptr_of(&mut idle));
    rq.bind_idle_task();
    let mut t = make_task(1, SchedPolicy::Normal, 0);

    unsafe {
        let flags = rq.lock.lock_irqsave();
        assert_eq!(rq.nr_running(), 0);

        rq.activate_task(ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);
        assert_eq!(rq.nr_running(), 1);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1951) - [internal]
double_enqueue_is_rejected_in_debug_builds

```
fn double_enqueue_is_rejected_in_debug_builds() {  
    let mut idle = make_idle(0, 5);  
    let mut rq = RunQueue::new(5, ptr_of(&mut idle));  
    rq.bind_idle_task();  
    let mut t = make_task(1, SchedPolicy::Normal, 0);  
    unsafe {  
        let flags = rq.lock.lock_irqsave();  
        rq.enqueue_task(ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);  
        assert!((*ptr_of(&mut t)).se.on_rq);  
        rq.lock.unlock_irqrestore(flags);  
    }  
}  
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1966) - [internal]
select_task_rq_prefers_idle_cpu_with_cache_affinity

```
fn select_task_rq_prefers_idle_cpu_with_cache_affinity() {  
    let mut idle0 = make_idle(100, 0);  
    let mut idle1 = make_idle(101, 1);  
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));  
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));  
    rq0.bind_idle_task();  
    rq1.bind_idle_task();  
  
    let mut t = make_task(1, SchedPolicy::Normal, 0);  
    t.se.last_cpu = 1;  
  
    let registry: [*mut RunQueue; 2] = [&mut rq0 as *mut RunQueue, &mut rq1 as *mut RunQueue];  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 1985) - [internal]
select_task_rq_picks_least_loaded_when_no_affinity_hit

```
fn select_task_rq_picks_least_loaded_when_no_affinity_hit() {  
    let mut idle0 = make_idle(100, 0);  
    let mut idle1 = make_idle(101, 1);  
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));  
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));  
    rq0.bind_idle_task();  
    rq1.bind_idle_task();  
  
    let mut filler = make_task(2, SchedPolicy::Normal, 0);  
    unsafe {  
        let f0 = rq0.lock.lock_irqsave();  
        rq0.enqueue_task(ptr_of(&mut filler), EnqueueFlags::ENQUEUE_NEW);  
    }  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2011) - [internal]
wake_up_process_transitions_state_and_enqueues_remotely

```
fn wake_up_process_transitions_state_and_enqueues_remotely() {  
    let mut idle0 = make_idle(100, 0);  
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));  
    rq0.bind_idle_task();  
  
    let mut t = make_task(1, SchedPolicy::Normal, 0);  
    t.set_state(TaskState::Uninterruptible).unwrap();  
    t.se.last_cpu = CPU_NONE;  
  
    let registry: [*mut RunQueue; 1] = [&mut rq0 as *mut RunQueue];  
    unsafe {  
        smp::wake_up_process(ptr_of(&mut t), &registry).unwrap();  
    }  
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2030) - [internal]
wake_up_process_is_idempotent_when_already_queued

```
fn wake_up_process_is_idempotent_when_already_queued() {
    let mut idle0 = make_idle(100, 0);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    rq0.bind_idle_task();
    let mut t = make_task(1, SchedPolicy::Normal, 0);

    let registry: [*mut RunQueue; 1] = [&mut rq0 as *mut RunQueue];
    unsafe {
        let flags = rq0.lock.lock_irqsave();
        rq0.activate_task(ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);
        rq0.lock.unlock_irqrestore(flags);
    }

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2049) - [internal]
idle_balance_pulls_one_task_from_busy_cpu

```
fn idle_balance_pulls_one_task_from_busy_cpu() {
    let mut idle0 = make_idle(200, 0);
    let mut idle1 = make_idle(201, 1);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));
    rq0.bind_idle_task();
    rq1.bind_idle_task();

    let mut t0 = make_task(1, SchedPolicy::Normal, 0);
    let mut t1 = make_task(2, SchedPolicy::Normal, 0);
    let mut t2 = make_task(3, SchedPolicy::Normal, 0);

    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2081) - [internal]
idle_balance_does_nothing_when_balanced

```
fn idle_balance_does_nothing_when_balanced() {
    let mut idle0 = make_idle(200, 0);
    let mut idle1 = make_idle(201, 1);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));
    rq0.bind_idle_task();
    rq1.bind_idle_task();

    let mut t0 = make_task(1, SchedPolicy::Normal, 0);
    unsafe {
        let f0 = rq0.lock.lock_irqsave();
        rq0.activate_task(ptr_of(&mut t0), EnqueueFlags::ENQUEUE_NEW);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2105) - [internal]
migrated_task_gets_updated_cpu_and_rq_pointer

```
fn migrated_task_gets_updated_cpu_and_rq_pointer() {
    let mut idle0 = make_idle(200, 0);
    let mut idle1 = make_idle(201, 1);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));
    rq0.bind_idle_task();
    rq1.bind_idle_task();

    let mut t0 = make_task(1, SchedPolicy::Normal, 0);
    let mut t1 = make_task(2, SchedPolicy::Normal, 0);
    unsafe {
        let f0 = rq0.lock.lock_irqsave();
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2140) - [internal]
no_migration_respects_cpus_allowed_mask

```
fn no_migration_respects_cpus_allowed_mask() {  
    let mut idle0 = make_idle(200, 0);  
    let mut idle1 = make_idle(201, 1);  
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));  
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));  
    rq0.bind_idle_task();  
    rq1.bind_idle_task();  
  
    let mut t0 = make_task(1, SchedPolicy::Normal, 0);  
    let mut t1 = make_task(2, SchedPolicy::Normal, 0);  
    t0.se.cpus_allowed = CpuMask::single(0);  
    t1.se.cpus_allowed = CpuMask::single(0);  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2169) - [internal]
yield_task_pushes_vruntime_to_back_of_fair_queue

```
fn yield_task_pushes_vruntime_to_back_of_fair_queue() {  
    let mut idle = make_idle(0, 6);  
    let mut rq = RunQueue::new(6, ptr_of(&mut idle));  
    rq.bind_idle_task();  
  
    let mut a = make_task(1, SchedPolicy::Normal, 0);  
    let mut b = make_task(2, SchedPolicy::Normal, 0);  
    a.se.vruntime = 100;  
    b.se.vruntime = 5000;  
  
    unsafe {  
        let flags = rq.lock.lock_irqsave();  
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2192) - [internal]
schedule_tail_records_voluntary_vs_involuntary_switch

```
fn schedule_tail_records_voluntary_vs_involuntary_switch() {  
    let mut idle = make_idle(0, 7);  
    let mut rq = RunQueue::new(7, ptr_of(&mut idle));  
    rq.bind_idle_task();  
  
    let mut sleeper = make_task(1, SchedPolicy::Normal, 0);  
    let mut preempted = make_task(2, SchedPolicy::Normal, 0);  
  
    unsafe {  
        let flags = rq.lock.lock_irqsave();  
        rq.enqueue_task(ptr_of(&mut sleeper), EnqueueFlags::ENQUEUE_NEW);  
        rq.enqueue_task(ptr_of(&mut preempted), EnqueueFlags::ENQUEUE_NEW);  
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2218) - [internal]
clock_advances_and_update_curr_charges_real_delta

```
fn clock_advances_and_update_curr_charges_real_delta() {  
    let mut idle = make_idle(0, 8);  
    let mut rq = RunQueue::new(8, ptr_of(&mut idle));  
    rq.bind_idle_task();  
    let mut t = make_task(1, SchedPolicy::Normal, 0);  
  
    unsafe {  
        let flags = rq.lock.lock_irqsave();  
        rq.enqueue_task(ptr_of(&mut t), EnqueueFlags::ENQUEUE_NEW);  
        rq.set_curr_task(ptr_of(&mut t));  
  
        assert_eq!(rq.clock.load(Ordering::Relaxed), 0);  
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2243) - [internal]
update_curr_is_a_noop_on_idle_task

```
fn update_curr_is_a_noop_on_idle_task() {
    let mut idle = make_idle(0, 9);
    let mut rq = RunQueue::new(9, ptr_of(&mut idle));
    rq.bind_idle_task();
    unsafe {
        let flags = rq.lock.lock_irqsave();
        rq.advance_clock(1_000_000);
        rq.update_curr(); // curr == idle, nie powinno panikowa■ ani nic zmieni■
        rq.lock.unlock_irqrestore(flags);
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2257) - [internal]
lower_nice_task_accumulates_vruntime_slower

```
fn lower_nice_task_accumulates_vruntime_slower() {
    let mut nice_low = make_task(1, SchedPolicy::Normal, -10); // wysoki priorytet
    let mut nice_high = make_task(2, SchedPolicy::Normal, 10); // niski priorytet
    assert!(nice_to_weight(-10) > nice_to_weight(10));

    let mut fair = RqFair::default();
    fair.enqueue(ptr_of(&mut nice_low), EnqueueFlags::ENQUEUE_NEW);
    fair.enqueue(ptr_of(&mut nice_high), EnqueueFlags::ENQUEUE_NEW);

    fair.charge_exec(ptr_of(&mut nice_low), 10_000_000, 10_000_000);
    fair.charge_exec(ptr_of(&mut nice_high), 10_000_000, 10_000_000);

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2281) - [internal]
load_balance_moves_task_between_two_uneven_queues

```
fn load_balance_moves_task_between_two_uneven_queues() {
    let mut idle0 = make_idle(0, 0);
    let mut idle1 = make_idle(0, 1);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));
    rq0.bind_idle_task();
    rq1.bind_idle_task();

    let mut tasks: [TaskStruct; 4] = core::array::from_fn(|_| TaskStruct::blank());
    unsafe {
        let f0 = rq0.lock.lock_irqsave();
        for (i, t) in tasks.iter_mut().enumerate() {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/runqueue.rs (line 2310) - [internal]
load_balance_is_noop_below_imbalance_threshold

```
fn load_balance_is_noop_below_imbalance_threshold() {
    let mut idle0 = make_idle(0, 0);
    let mut idle1 = make_idle(0, 1);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));
    rq0.bind_idle_task();
    rq1.bind_idle_task();

    let mut t0 = make_task(1, SchedPolicy::Normal, 0);
    unsafe {
        let f0 = rq0.lock.lock_irqsave();
        rq0.activate_task(ptr_of(&mut t0), EnqueueFlags::ENQUEUE_NEW);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 21) - [public]

account_switch

// Wygodne funkcje globalne (dla kompatybilności i szybkiego użycia)

```
pub fn account_switch() {
    TOTAL_SWITCHES.fetch_add(1, Ordering::Relaxed);
}

pub fn account_voluntary_switch(task: &mut TaskStruct) {
    TOTAL_VOLUNTARY_SWITCHES.fetch_add(1, Ordering::Relaxed);
    task.stats.nr_voluntary_switches = task.stats.nr_voluntary_switches.saturating_add(1);
}

pub fn account_involuntary_switch(task: &mut TaskStruct) {
    TOTAL_INVOLUNTARY_SWITCHES.fetch_add(1, Ordering::Relaxed);
    task.stats.nr_involuntary_switches = task.stats.nr_involuntary_switches.saturating_add(1);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 25) - [public]

account_voluntary_switch

```
pub fn account_voluntary_switch(task: &mut TaskStruct) {
    TOTAL_VOLUNTARY_SWITCHES.fetch_add(1, Ordering::Relaxed);
    task.stats.nr_voluntary_switches = task.stats.nr_voluntary_switches.saturating_add(1);
}

pub fn account_involuntary_switch(task: &mut TaskStruct) {
    TOTAL_INVOLUNTARY_SWITCHES.fetch_add(1, Ordering::Relaxed);
    task.stats.nr_involuntary_switches = task.stats.nr_involuntary_switches.saturating_add(1);
}

pub fn account_migration(task: &mut TaskStruct) {
    TOTAL_MIGRATIONS.fetch_add(1, Ordering::Relaxed);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 30) - [public]

account_involuntary_switch

```
pub fn account_involuntary_switch(task: &mut TaskStruct) {
    TOTAL_INVOLUNTARY_SWITCHES.fetch_add(1, Ordering::Relaxed);
    task.stats.nr_involuntary_switches = task.stats.nr_involuntary_switches.saturating_add(1);
}

pub fn account_migration(task: &mut TaskStruct) {
    TOTAL_MIGRATIONS.fetch_add(1, Ordering::Relaxed);
    task.stats.nr_migrations = task.stats.nr_migrations.saturating_add(1);
    task.se.nr_migrations = task.se.nr_migrations.saturating_add(1);
}

pub fn account_user_time(task: &mut TaskStruct, delta_ns: u64) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 35) - [public]

account_migration

```
pub fn account_migration(task: &mut TaskStruct) {
    TOTAL_MIGRATIONS.fetch_add(1, Ordering::Relaxed);
    task.stats.nr_migrations = task.stats.nr_migrations.saturating_add(1);
    task.se.nr_migrations = task.se.nr_migrations.saturating_add(1);
}

pub fn account_user_time(task: &mut TaskStruct, delta_ns: u64) {
    TOTAL_USER_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    TOTAL_RUNNING_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    task.stats.utime = task.stats.utime.saturating_add(delta_ns);
    task.se.sum_exec_runtime = task.se.sum_exec_runtime.saturating_add(delta_ns);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 41) - [public]

account_user_time

```
pub fn account_user_time(task: &mut TaskStruct, delta_ns: u64) {
    TOTAL_USER_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    TOTAL_RUNNING_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    task.stats.uptime = task.stats.uptime.saturating_add(delta_ns);
    task.se.sum_exec_runtime = task.se.sum_exec_runtime.saturating_add(delta_ns);
}

pub fn account_system_time(task: &mut TaskStruct, delta_ns: u64) {
    TOTAL_SYSTEM_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    TOTAL_RUNNING_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    task.stats.stime = task.stats.stime.saturating_add(delta_ns);
    task.se.sum_exec_runtime = task.se.sum_exec_runtime.saturating_add(delta_ns);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 48) - [public]

account_system_time

```
pub fn account_system_time(task: &mut TaskStruct, delta_ns: u64) {
    TOTAL_SYSTEM_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    TOTAL_RUNNING_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    task.stats.stime = task.stats.stime.saturating_add(delta_ns);
    task.se.sum_exec_runtime = task.se.sum_exec_runtime.saturating_add(delta_ns);
}

pub fn account_idle_time(delta_ns: u64) {
    TOTAL_IDLE_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

pub fn account_iowait_time(delta_ns: u64) {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 55) - [public]

account_idle_time

```
pub fn account_idle_time(delta_ns: u64) {
    TOTAL_IDLE_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

pub fn account_iowait_time(delta_ns: u64) {
    TOTAL_IOWAIT_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

// Gettery globalne
pub fn total_switches() -> u64 { TOTAL_SWITCHES.load(Ordering::Relaxed) }
pub fn total_voluntary_switches() -> u64 { TOTAL_VOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_involuntary_switches() -> u64 { TOTAL_INVOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 59) - [public]

account_iowait_time

```
pub fn account_iowait_time(delta_ns: u64) {
    TOTAL_IOWAIT_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

// Gettery globalne
pub fn total_switches() -> u64 { TOTAL_SWITCHES.load(Ordering::Relaxed) }
pub fn total_voluntary_switches() -> u64 { TOTAL_VOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_involuntary_switches() -> u64 { TOTAL_INVOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_migrations() -> u64 { TOTAL_MIGRATIONS.load(Ordering::Relaxed) }
pub fn total_user_time_ns() -> u64 { TOTAL_USER_TIME_NS.load(Ordering::Relaxed) }
pub fn total_system_time_ns() -> u64 { TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 64) - [public]

total_switches

// Gettery globalne

```
pub fn total_switches() -> u64 { TOTAL_SWITCHES.load(Ordering::Relaxed) }
pub fn total_voluntary_switches() -> u64 { TOTAL_VOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_involuntary_switches() -> u64 { TOTAL_INVOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_migrations() -> u64 { TOTAL_MIGRATIONS.load(Ordering::Relaxed) }
pub fn total_user_time_ns() -> u64 { TOTAL_USER_TIME_NS.load(Ordering::Relaxed) }
pub fn total_system_time_ns() -> u64 { TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

#[repr(C)]

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 65) - [public]

total_voluntary_switches

```
pub fn total_voluntary_switches() -> u64 { TOTAL_VOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_involuntary_switches() -> u64 { TOTAL_INVOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_migrations() -> u64 { TOTAL_MIGRATIONS.load(Ordering::Relaxed) }
pub fn total_user_time_ns() -> u64 { TOTAL_USER_TIME_NS.load(Ordering::Relaxed) }
pub fn total_system_time_ns() -> u64 { TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

#[repr(C)]

#[derive(Debug, Default, Clone, Copy)]

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 66) - [public]

total_involuntary_switches

```
pub fn total_involuntary_switches() -> u64 { TOTAL_INVOLUNTARY_SWITCHES.load(Ordering::Relaxed) }
pub fn total_migrations() -> u64 { TOTAL_MIGRATIONS.load(Ordering::Relaxed) }
pub fn total_user_time_ns() -> u64 { TOTAL_USER_TIME_NS.load(Ordering::Relaxed) }
pub fn total_system_time_ns() -> u64 { TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

#[repr(C)]

#[derive(Debug, Default, Clone, Copy)]

pub struct RqStats {

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 67) - [public]

total_migrations

```
pub fn total_migrations() -> u64 { TOTAL_MIGRATIONS.load(Ordering::Relaxed) }
pub fn total_user_time_ns() -> u64 { TOTAL_USER_TIME_NS.load(Ordering::Relaxed) }
pub fn total_system_time_ns() -> u64 { TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

#[repr(C)]

#[derive(Debug, Default, Clone, Copy)]

pub struct RqStats {

/// Liczba przebiegów kontekstu na tym CPU.

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 68) - [public]

total_user_time_ns

```
pub fn total_user_time_ns() -> u64 { TOTAL_USER_TIME_NS.load(Ordering::Relaxed) }
pub fn total_system_time_ns() -> u64 { TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

```
#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct RqStats {
    /// Liczba przełączeń kontekstu na tym CPU.
    pub nr_switches: u64,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 69) - [public]

total_system_time_ns

```
pub fn total_system_time_ns() -> u64 { TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed) }
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

```
#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct RqStats {
    /// Liczba przełączeń kontekstu na tym CPU.
    pub nr_switches: u64,
    /// Liczba dobrowolnych przełączeń.
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 70) - [public]

total_idle_time_ns

```
pub fn total_idle_time_ns() -> u64 { TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed) }
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

```
#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct RqStats {
    /// Liczba przełączeń kontekstu na tym CPU.
    pub nr_switches: u64,
    /// Liczba dobrowolnych przełączeń.
    pub nr_voluntary_switches: u64,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 71) - [public]

total_iowait_time_ns

```
pub fn total_iowait_time_ns() -> u64 { TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed) }
pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

```
#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct RqStats {
    /// Liczba przełączeń kontekstu na tym CPU.
    pub nr_switches: u64,
    /// Liczba dobrowolnych przełączeń.
    pub nr_voluntary_switches: u64,
    /// Liczba przymusowych przełączeń.
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 72) - [public]
total_running_time_ns
    pub fn total_running_time_ns() -> u64 { TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed) }
```

```
#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct RqStats {
    /// Liczba przełączeń kontekstu na tym CPU.
    pub nr_switches: u64,
    /// Liczba dobrowolnych przełączeń.
    pub nr_voluntary_switches: u64,
    /// Liczba przymusowych przełączeń.
    pub nr_involuntary_switches: u64,
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 77) - [public] RqStats
```

```
pub struct RqStats {
    /// Liczba przełączeń kontekstu na tym CPU.
    pub nr_switches: u64,
    /// Liczba dobrowolnych przełączeń.
    pub nr_voluntary_switches: u64,
    /// Liczba przymusowych przełączeń.
    pub nr_involuntary_switches: u64,
    /// Liczba migracji zadań z/do tego CPU.
    pub nr_migrations: u64,
    /// Czas spędzony w trybie użytkownika (ns).
    pub user_time_ns: u64,
    /// Czas spędzony w trybie jądra (ns).
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 126) - [public]
```

```
record_switch
    pub fn record_switch(&mut self, voluntary: bool) {
        self.nr_switches += 1;
        if voluntary {
            self.nr_voluntary_switches += 1;
        } else {
            self.nr_involuntary_switches += 1;
        }
        // Aktualizuj globalne liczniki
        TOTAL_SWITCHES.fetch_add(1, Ordering::Relaxed);
        if voluntary {
            TOTAL_VOLUNTARY_SWITCHES.fetch_add(1, Ordering::Relaxed);
        } else {
            // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 143) - [public]
```

```
record_migration
    pub fn record_migration(&mut self) {
        self.nr_migrations += 1;
        TOTAL_MIGRATIONS.fetch_add(1, Ordering::Relaxed);
    }

#[inline]
pub fn add_user_time(&mut self, delta_ns: u64) {
    self.user_time_ns += delta_ns;
    self.running_time_ns += delta_ns;
    TOTAL_USER_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    TOTAL_RUNNING_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}
// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 149) - [public]

add_user_time

```
pub fn add_user_time(&mut self, delta_ns: u64) {
    self.user_time_ns += delta_ns;
    self.running_time_ns += delta_ns;
    TOTAL_USER_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    TOTAL_RUNNING_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

#[inline]
pub fn add_system_time(&mut self, delta_ns: u64) {
    self.system_time_ns += delta_ns;
    self.running_time_ns += delta_ns;
    TOTAL_SYSTEM_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 157) - [public]

add_system_time

```
pub fn add_system_time(&mut self, delta_ns: u64) {
    self.system_time_ns += delta_ns;
    self.running_time_ns += delta_ns;
    TOTAL_SYSTEM_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
    TOTAL_RUNNING_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

#[inline]
pub fn add_idle_time(&mut self, delta_ns: u64) {
    self.idle_time_ns += delta_ns;
    TOTAL_IDLE_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 165) - [public]

add_idle_time

```
pub fn add_idle_time(&mut self, delta_ns: u64) {
    self.idle_time_ns += delta_ns;
    TOTAL_IDLE_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

#[inline]
pub fn add_iowait_time(&mut self, delta_ns: u64) {
    self.iowait_time_ns += delta_ns;
    TOTAL_IOWAIT_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

#[inline]
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 171) - [public]

add_iowait_time

```
pub fn add_iowait_time(&mut self, delta_ns: u64) {
    self.iowait_time_ns += delta_ns;
    TOTAL_IOWAIT_TIME_NS.fetch_add(delta_ns, Ordering::Relaxed);
}

#[inline]
pub fn set_nr_running(&mut self, nr: u32) {
    self.nr_running = nr;
}

#[inline]
pub fn set_nr_uninterruptible(&mut self, nr: u32) {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 177) - [public]

set_nr_running

```
pub fn set_nr_running(&mut self, nr: u32) {
    self.nr_running = nr;
}

#[inline]
pub fn set_nr_uninterruptible(&mut self, nr: u32) {
    self.nr_uninterruptible = nr;
}

#[inline]
pub fn update_load_stats(&mut self, sum_vruntime: u64, sum_weight: u64) {
    self.sum_vruntime = sum_vruntime;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 182) - [public]

set_nr_uninterruptible

```
pub fn set_nr_uninterruptible(&mut self, nr: u32) {
    self.nr_uninterruptible = nr;
}

#[inline]
pub fn update_load_stats(&mut self, sum_vruntime: u64, sum_weight: u64) {
    self.sum_vruntime = sum_vruntime;
    self.sum_weight = sum_weight;
}

/// Oblicza przybliżone obciążenie CPU (0-100).
pub fn load_percent(&self) -> u32 {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 187) - [public]

update_load_stats

```
pub fn update_load_stats(&mut self, sum_vruntime: u64, sum_weight: u64) {
    self.sum_vruntime = sum_vruntime;
    self.sum_weight = sum_weight;
}

/// Oblicza przybliżone obciążenie CPU (0-100).
pub fn load_percent(&self) -> u32 {
    if self.running_time_ns == 0 {
        return 0;
    }
    let total = self.running_time_ns + self.idle_time_ns;
    if total == 0 {
        return 0;
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 193) - [public] load_percent

/// Oblicza przybliżone obciążenie CPU (0-100).

```
pub fn load_percent(&self) -> u32 {
    if self.running_time_ns == 0 {
        return 0;
    }
    let total = self.running_time_ns + self.idle_time_ns;
    if total == 0 {
        return 0;
    }
    ((self.running_time_ns * 100) / total) as u32
}

pub fn load_avg_1min(&self) -> f32 {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 204) - [public]

```
load_avg_1min
    pub fn load_avg_1min(&self) -> f32 {
        self.load_percent() as f32 / 100.0
    }
}

// -----

/// Prosty histogram czasów wykonania (w ns) w skali logarytmicznej.
#[derive(Debug, Clone, Copy)]
pub struct ExecTimeHistogram {
    // 16 przedziałów: 0-1us, 1-2us, 2-4us, 4-8us, ... >8ms
    buckets: [u64; 16],
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 213) - [public]

```
ExecTimeHistogram
    pub struct ExecTimeHistogram {
        // 16 przedziałów: 0-1us, 1-2us, 2-4us, 4-8us, ... >8ms
        buckets: [u64; 16],
        total_samples: u64,
        total_time_ns: u64,
    }

    impl ExecTimeHistogram {
        pub const fn new() -> Self {
            Self {
                buckets: [0; 16],
                total_samples: 0,
                // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 229) - [public] add_sample

```
    pub fn add_sample(&mut self, exec_ns: u64) {
        self.total_samples += 1;
        self.total_time_ns += exec_ns;

        // Wyznacz indeks na podstawie log2, z dolnym progiem 1024 ns
        let mut idx = 0;
        let mut threshold = 1024u64; // 1us
        let mut upper = 2048u64;
        for i in 0..15 {
            if exec_ns < upper {
                idx = i;
                break;
            }
        }
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 251) - [public] average_ns

```
    pub fn average_ns(&self) -> f32 {
        if self.total_samples == 0 {
            0.0
        } else {
            self.total_time_ns as f32 / self.total_samples as f32
        }
    }

    pub fn total_samples(&self) -> u64 {
        self.total_samples
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 259) - [public]

```
total_samples
    pub fn total_samples(&self) -> u64 {
        self.total_samples
    }
}

impl Default for ExecTimeHistogram {
    fn default() -> Self {
        Self::new()
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 265) - [internal] default

```
fn default() -> Self {
    Self::new()
}

/// Zbiorcza struktura do raportowania wszystkich statystyk systemu.
#[derive(Debug, Default, Clone)]
pub struct SystemStats {
    pub total_switches: u64,
    pub total_voluntary_switches: u64,
    pub total_involuntary_switches: u64,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 273) - [public] SystemStats

```
pub struct SystemStats {
    pub total_switches: u64,
    pub total_voluntary_switches: u64,
    pub total_involuntary_switches: u64,
    pub total_migrations: u64,
    pub total_user_time_ns: u64,
    pub total_system_time_ns: u64,
    pub total_idle_time_ns: u64,
    pub total_iowait_time_ns: u64,
    pub total_running_time_ns: u64,
    pub per_cpu: Vec<CpuStatsSnapshot>,
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 287) - [public]

```
CpuStatsSnapshot

pub struct CpuStatsSnapshot {
    pub cpu: u32,
    pub nr_switches: u64,
    pub user_time_ns: u64,
    pub system_time_ns: u64,
    pub idle_time_ns: u64,
    pub iowait_time_ns: u64,
    pub running_time_ns: u64,
    pub load_percent: u32,
}

pub fn collect_system_stats() -> SystemStats {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 298) - [public]

collect_system_stats

```
pub fn collect_system_stats() -> SystemStats {
    SystemStats {
        total_switches: TOTAL_SWITCHES.load(Ordering::Relaxed),
        total_voluntary_switches: TOTAL_VOLUNTARY_SWITCHES.load(Ordering::Relaxed),
        total_involuntary_switches: TOTAL_INVOLUNTARY_SWITCHES.load(Ordering::Relaxed),
        total_migrations: TOTAL_MIGRATIONS.load(Ordering::Relaxed),
        total_user_time_ns: TOTAL_USER_TIME_NS.load(Ordering::Relaxed),
        total_system_time_ns: TOTAL_SYSTEM_TIME_NS.load(Ordering::Relaxed),
        total_idle_time_ns: TOTAL_IDLE_TIME_NS.load(Ordering::Relaxed),
        total_iowait_time_ns: TOTAL_IOWAIT_TIME_NS.load(Ordering::Relaxed),
        total_running_time_ns: TOTAL_RUNNING_TIME_NS.load(Ordering::Relaxed),
        per_cpu: Vec::new(), // W praktyce wypelnia z tablicy CPU
    } // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 315) - [internal] fmt

```
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    writeln!(f, "CPU stats:");
    writeln!(f, "  switches: {} (voluntary {}, involuntary {})",
        self.nr_switches, self.nr_voluntary_switches, self.nr_involuntary_switches)?;
    writeln!(f, "  migrations: {}", self.nr_migrations)?;
    writeln!(f, "  user time: {} ns", self.user_time_ns)?;
    writeln!(f, "  system time: {} ns", self.system_time_ns)?;
    writeln!(f, "  idle time: {} ns", self.idle_time_ns)?;
    writeln!(f, "  iowait time: {} ns", self.iowait_time_ns)?;
    writeln!(f, "  running time: {} ns", self.running_time_ns)?;
    writeln!(f, "  load: {}%", self.load_percent());
    writeln!(f, "  nr_running: {}, nr_uninterruptible: {}",
        self.nr_running, self.nr_uninterruptible);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/stats.rs (line 332) - [internal] fmt

```
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    writeln!(f, "System-wide scheduler stats:");
    writeln!(f, "  total switches: {}", self.total_switches)?;
    writeln!(f, "  voluntary: {}", self.total_voluntary_switches)?;
    writeln!(f, "  involuntary: {}", self.total_involuntary_switches)?;
    writeln!(f, "  migrations: {}", self.total_migrations)?;
    writeln!(f, "  user time: {} ns", self.total_user_time_ns)?;
    writeln!(f, "  system time: {} ns", self.total_system_time_ns)?;
    writeln!(f, "  idle time: {} ns", self.total_idle_time_ns)?;
    writeln!(f, "  iowait time: {} ns", self.total_iowait_time_ns)?;
    writeln!(f, "  running time: {} ns", self.total_running_time_ns)?;
    if !self.per_cpu.is_empty() {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 44) - [public] TaskFlags

```
pub struct TaskFlags: u32 {
    const PF_KTHREAD          = 1 << 0;
    const PF_EXITING          = 1 << 1;
    const PF_EXITPIDONE       = 1 << 2;
    const PF_FORKNOEXEC       = 1 << 3;
    const PF_WQ_WORKER        = 1 << 4;
    const PF_NO_SETAFFINITY   = 1 << 5;
    const PF_IDLE              = 1 << 6;
    const PF_MEMALLOC         = 1 << 7;
    const PF_FROZEN            = 1 << 8;
    const PF_SUPERPRIV        = 1 << 9;
    const PF_DUMPCORE          = 1 << 10;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 73) - [public]

AtomicTaskFlags

```
pub struct AtomicTaskFlags(AtomicU32);

impl AtomicTaskFlags {
    pub const fn new(flags: TaskFlags) -> Self {
        Self(AtomicU32::new(flags.bits()))
    }

    pub const fn empty() -> Self {
        Self::new(TaskFlags::empty())
    }

    #[inline]
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 85) - [public] load

```
pub fn load(&self, order: Ordering) -> TaskFlags {
    TaskFlags::from_bits_truncate(self.0.load(order))
}

#[inline]
pub fn store(&self, flags: TaskFlags, order: Ordering) {
    self.0.store(flags.bits(), order);
}

#[inline]
pub fn fetch_insert(&self, flags: TaskFlags) -> TaskFlags {
    let prev = self.0.fetch_or(flags.bits(), Ordering::AcqRel);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 90) - [public] store

```
pub fn store(&self, flags: TaskFlags, order: Ordering) {
    self.0.store(flags.bits(), order);
}

#[inline]
pub fn fetch_insert(&self, flags: TaskFlags) -> TaskFlags {
    let prev = self.0.fetch_or(flags.bits(), Ordering::AcqRel);
    TaskFlags::from_bits_truncate(prev)
}

/// Czy❏ci bity `flags`, zwraca poprzedni❏ warto❏.
#[inline]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 95) - [public] fetch_insert

```
pub fn fetch_insert(&self, flags: TaskFlags) -> TaskFlags {
    let prev = self.0.fetch_or(flags.bits(), Ordering::AcqRel);
    TaskFlags::from_bits_truncate(prev)
}

/// Czy❏ci bity `flags`, zwraca poprzedni❏ warto❏.
#[inline]
pub fn fetch_remove(&self, flags: TaskFlags) -> TaskFlags {
    let prev = self.0.fetch_and(!flags.bits(), Ordering::AcqRel);
    TaskFlags::from_bits_truncate(prev)
}

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 102) - [public] fetch_remove
    pub fn fetch_remove(&self, flags: TaskFlags) -> TaskFlags {
        let prev = self.0.fetch_and(!flags.bits(), Ordering::AcqRel);
        TaskFlags::from_bits_truncate(prev)
    }

    #[inline]
    pub fn contains(&self, flags: TaskFlags) -> bool {
        self.load(Ordering::Acquire).contains(flags)
    }

    #[inline]
    pub fn test_and_set(&self, flags: TaskFlags) -> bool {
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 108) - [public] contains
    pub fn contains(&self, flags: TaskFlags) -> bool {
        self.load(Ordering::Acquire).contains(flags)
    }

    #[inline]
    pub fn test_and_set(&self, flags: TaskFlags) -> bool {
        let prev = self.fetch_insert(flags);
        !prev.contains(flags)
    }
}

impl Clone for AtomicTaskFlags {
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 113) - [public] test_and_set
    pub fn test_and_set(&self, flags: TaskFlags) -> bool {
        let prev = self.fetch_insert(flags);
        !prev.contains(flags)
    }
}

impl Clone for AtomicTaskFlags {
    fn clone(&self) -> Self {
        Self::new(self.load(Ordering::Acquire))
    }
}

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 120) - [internal] clone
    fn clone(&self) -> Self {
        Self::new(self.load(Ordering::Acquire))
    }
}

#[repr(u8)]
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum TaskState {
    Runnable = 0,
    Interruptible = 1,
    Uninterruptible = 2,
    Stopped = 3,
    // ... (truncated, see source file for more)
}

```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 127) - [public] TaskState

```
pub enum TaskState {
    Runnable = 0,
    Interruptible = 1,
    Uninterruptible = 2,
    Stopped = 3,
    Traced = 4,
    Zombie = 5,
    Dead = 6,
    Idle = 7,
}

impl TaskState {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 200) - [internal] default

```
fn default() -> Self {
    TaskState::Dead
}

#[repr(u8)]
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum SchedPolicy {
    Normal = 0,
    Batch = 1,
    Idle = 2,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 208) - [public] SchedPolicy

```
pub enum SchedPolicy {
    Normal = 0,
    Batch = 1,
    Idle = 2,
    Fifo = 3,
    RoundRobin = 4,
    Deadline = 5,
    Stop = 6,
}

impl SchedPolicy {
    pub const fn is_realtime(self) -> bool {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 230) - [public] SchedClass

```
pub enum SchedClass {
    Idle = 0,
    Fair = 1,
    RealTime = 2,
    Deadline = 3,
    Stop = 4,
}

impl From<SchedPolicy> for SchedClass {
    fn from(policy: SchedPolicy) -> Self {
        match policy {
            SchedPolicy::Idle => SchedClass::Idle,
            // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 239) - [internal] from

```
fn from(policy: SchedPolicy) -> Self {
    match policy {
        SchedPolicy::Idle => SchedClass::Idle,
        SchedPolicy::Normal | SchedPolicy::Batch => SchedClass::Fair,
        SchedPolicy::Fifo | SchedPolicy::RoundRobin => SchedClass::RealTime,
        SchedPolicy::Deadline => SchedClass::Deadline,
        SchedPolicy::Stop => SchedClass::Stop,
    }
}
}
```

// ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 253) - [public] CpuMask

```
pub struct CpuMask {
    bits: [u64; CPUMASK_WORDS],
}

impl CpuMask {
    pub const fn empty() -> Self {
        Self { bits: [0; CPUMASK_WORDS] }
    }

    pub const fn all() -> Self {
        Self { bits: [u64::MAX; CPUMASK_WORDS] }
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 266) - [public] single

```
pub fn single(cpu: u32) -> Self {
    let mut m = Self::empty();
    m.set(cpu);
    m
}

pub fn first_n(n: u32) -> Self {
    let mut m = Self::empty();
    let n = cmp::min(n as usize, MAX_CPUS);
    for cpu in 0..n {
        m.set(cpu as u32);
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 272) - [public] first_n

```
pub fn first_n(n: u32) -> Self {
    let mut m = Self::empty();
    let n = cmp::min(n as usize, MAX_CPUS);
    for cpu in 0..n {
        m.set(cpu as u32);
    }
    m
}

pub fn set(&mut self, cpu: u32) {
    let cpu = cpu as usize;
    if cpu < MAX_CPUS {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 281) - [public] set

```
pub fn set(&mut self, cpu: u32) {
    let cpu = cpu as usize;
    if cpu < MAX_CPUS {
        self.bits[cpu / 64] |= 1u64 << (cpu % 64);
    }
}

pub fn clear(&mut self, cpu: u32) {
    let cpu = cpu as usize;
    if cpu < MAX_CPUS {
        self.bits[cpu / 64] &= !(1u64 << (cpu % 64));
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 288) - [public] clear

```
pub fn clear(&mut self, cpu: u32) {
    let cpu = cpu as usize;
    if cpu < MAX_CPUS {
        self.bits[cpu / 64] &= !(1u64 << (cpu % 64));
    }
}

pub fn is_set(&self, cpu: u32) -> bool {
    let cpu = cpu as usize;
    cpu < MAX_CPUS && (self.bits[cpu / 64] & (1u64 << (cpu % 64))) != 0
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 295) - [public] is_set

```
pub fn is_set(&self, cpu: u32) -> bool {
    let cpu = cpu as usize;
    cpu < MAX_CPUS && (self.bits[cpu / 64] & (1u64 << (cpu % 64))) != 0
}

pub fn is_empty(&self) -> bool {
    self.bits.iter().all(|w| *w == 0)
}

pub fn count(&self) -> u32 {
    self.bits.iter().map(|w| w.count_ones()).sum()
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 300) - [public] is_empty

```
pub fn is_empty(&self) -> bool {
    self.bits.iter().all(|w| *w == 0)
}

pub fn count(&self) -> u32 {
    self.bits.iter().map(|w| w.count_ones()).sum()
}

pub fn intersects(&self, other: &CpuMask) -> bool {
    self.bits.iter().zip(other.bits.iter()).any(|(a, b)| a & b != 0)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 304) - [public] count

```
pub fn count(&self) -> u32 {
    self.bits.iter().map(|w| w.count_ones()).sum()
}

pub fn intersects(&self, other: &CpuMask) -> bool {
    self.bits.iter().zip(other.bits.iter()).any(|(a, b)| a & b != 0)
}

pub fn and(&self, other: &CpuMask) -> CpuMask {
    let mut out = CpuMask::empty();
    for i in 0..CPUMASK_WORDS {
        out.bits[i] = self.bits[i] & other.bits[i];
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 308) - [public] intersects

```
pub fn intersects(&self, other: &CpuMask) -> bool {
    self.bits.iter().zip(other.bits.iter()).any(|(a, b)| a & b != 0)
}

pub fn and(&self, other: &CpuMask) -> CpuMask {
    let mut out = CpuMask::empty();
    for i in 0..CPUMASK_WORDS {
        out.bits[i] = self.bits[i] & other.bits[i];
    }
    out
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 312) - [public] and

```
pub fn and(&self, other: &CpuMask) -> CpuMask {
    let mut out = CpuMask::empty();
    for i in 0..CPUMASK_WORDS {
        out.bits[i] = self.bits[i] & other.bits[i];
    }
    out
}

pub fn or(&self, other: &CpuMask) -> CpuMask {
    let mut out = CpuMask::empty();
    for i in 0..CPUMASK_WORDS {
        out.bits[i] = self.bits[i] | other.bits[i];
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 320) - [public] or

```
pub fn or(&self, other: &CpuMask) -> CpuMask {
    let mut out = CpuMask::empty();
    for i in 0..CPUMASK_WORDS {
        out.bits[i] = self.bits[i] | other.bits[i];
    }
    out
}

pub fn first(&self) -> Option<u32> {
    for (word_idx, word) in self.bits.iter().enumerate() {
        if *word != 0 {
            return Some((word_idx * 64 + word.trailing_zeros() as usize) as u32);
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 328) - [public] first

```
pub fn first(&self) -> Option<u32> {
    for (word_idx, word) in self.bits.iter().enumerate() {
        if *word != 0 {
            return Some((word_idx * 64 + word.trailing_zeros() as usize) as u32);
        }
    }
    None
}

pub fn next_after(&self, after: u32) -> Option<u32> {
    let start = after.wrapping_add(1);
    for cpu in start..MAX_CPUS as u32 {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 337) - [public] next_after

```
pub fn next_after(&self, after: u32) -> Option<u32> {
    let start = after.wrapping_add(1);
    for cpu in start..MAX_CPUS as u32 {
        if self.is_set(cpu) {
            return Some(cpu);
        }
    }
    for cpu in 0..=after.min(MAX_CPUS as u32 - 1) {
        if self.is_set(cpu) {
            return Some(cpu);
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 352) - [public] iter

```
pub fn iter(&self) -> CpuMaskIter {
    CpuMaskIter { mask: *self, next_cpu: 0 }
}

impl Default for CpuMask {
    fn default() -> Self {
        CpuMask::empty()
    }
}

pub struct CpuMaskIter {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 358) - [internal] default

```
fn default() -> Self {
    CpuMask::empty()
}

pub struct CpuMaskIter {
    mask: CpuMask,
    next_cpu: usize,
}

impl Iterator for CpuMaskIter {
    type Item = u32;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 363) - [public] CpuMaskIter

```
pub struct CpuMaskIter {
    mask: CpuMask,
    next_cpu: usize,
}

impl Iterator for CpuMaskIter {
    type Item = u32;

    fn next(&mut self) -> Option<u32> {
        while self.next_cpu < MAX_CPUS {
            let cpu = self.next_cpu as u32;
            self.next_cpu += 1;
        }
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 371) - [internal] next

```
fn next(&mut self) -> Option<u32> {
    while self.next_cpu < MAX_CPUS {
        let cpu = self.next_cpu as u32;
        self.next_cpu += 1;
        if self.mask.is_set(cpu) {
            return Some(cpu);
        }
    }
    None
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 406) - [public]

nice_to_weight

```
pub fn nice_to_weight(nice: i8) -> u64 {
    let clamped = nice.clamp(NICE_MIN, NICE_MAX);
    NICE_TO_WEIGHT_TABLE[(clamped - NICE_MIN) as usize]
}

pub fn nice_to_wmult(nice: i8) -> u32 {
    let clamped = nice.clamp(NICE_MIN, NICE_MAX);
    NICE_TO_WMULT_TABLE[(clamped - NICE_MIN) as usize]
}

pub fn weight_to_nice(weight: u64) -> i8 {
    let mut best_idx = 0usize;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 411) - [public] nice_to_wmult

```
pub fn nice_to_wmult(nice: i8) -> u32 {
    let clamped = nice.clamp(NICE_MIN, NICE_MAX);
    NICE_TO_WMULT_TABLE[(clamped - NICE_MIN) as usize]
}

pub fn weight_to_nice(weight: u64) -> i8 {
    let mut best_idx = 0usize;
    let mut best_diff = u64::MAX;
    for (idx, w) in NICE_TO_WEIGHT_TABLE.iter().enumerate() {
        let diff = if *w > weight { *w - weight } else { weight - *w };
        if diff < best_diff {
            best_diff = diff;
        }
    }
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 416) - [public]

weight_to_nice

```
pub fn weight_to_nice(weight: u64) -> i8 {
    let mut best_idx = 0usize;
    let mut best_diff = u64::MAX;
    for (idx, w) in NICE_TO_WEIGHT_TABLE.iter().enumerate() {
        let diff = if *w > weight { *w - weight } else { weight - *w };
        if diff < best_diff {
            best_diff = diff;
            best_idx = idx;
        }
    }
    NICE_MIN + best_idx as i8
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 431) - [public]

calc_delta_fair

```
pub fn calc_delta_fair(delta_exec: u64, weight: u64, inv_weight: u32) -> u64 {
    if weight == NICE_0_LOAD {
        return delta_exec;
    }
    let product = (delta_exec as u128) * (inv_weight as u128);
    (product >> WMULT_SHIFT) as u64
}
```

```
#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct InterruptFrame {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 442) - [public]

InterruptFrame

```
pub struct InterruptFrame {
    pub rip: u64,
    pub cs: u64,
    pub rflags: u64,
    pub rsp: u64,
    pub ss: u64,
}

#[repr(C, align(16))]
#[derive(Clone, Copy)]
pub struct FxSaveArea {
    pub bytes: [u8; 512],
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 452) - [public] FxSaveArea

```
pub struct FxSaveArea {
    pub bytes: [u8; 512],
}

impl FxSaveArea {
    pub const fn zeroed() -> Self {
        Self { bytes: [0u8; 512] }
    }
}

impl Default for FxSaveArea {
    fn default() -> Self {
        // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 463) - [internal] default
    fn default() -> Self {
        Self::zeroed()
    }
}

impl fmt::Debug for FxSaveArea {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "FxSaveArea(512 B @ {:p})", self.bytes.as_ptr())
    }
}

#[repr(C)]
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 469) - [internal] fmt
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        write!(f, "FxSaveArea(512 B @ {:p})", self.bytes.as_ptr())
    }
}

#[repr(C)]
#[derive(Debug, Clone, Copy)]
pub struct CpuContext {
    pub rsp: u64,
    pub rdi: u64,
    pub rbx: u64,
    pub rbp: u64,
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 476) - [public] CpuContext
pub struct CpuContext {
    pub rsp: u64,
    pub rdi: u64,
    pub rbx: u64,
    pub rbp: u64,
    pub r12: u64,
    pub r13: u64,
    pub r14: u64,
    pub r15: u64,
    pub rip: u64,
    pub fs_base: u64,
    pub gs_base: u64,
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 493) - [internal] default
    fn default() -> Self {
        Self {
            rsp: 0,
            rdi: 0,
            rbx: 0,
            rbp: 0,
            r12: 0,
            r13: 0,
            r14: 0,
            r15: 0,
            rip: 0,
            fs_base: 0,
            // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 548) - [public]

xsave_area_size

```
pub fn xsave_area_size() -> usize {
    core::mem::size_of::<FxSaveArea>()
}

pub const RB_RED: usize = 0;
pub const RB_BLACK: usize = 1;

#[inline]
pub fn rb_parent(pc: usize) -> *mut TaskStruct {
    (pc & !1) as *mut TaskStruct
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 557) - [public] rb_parent

```
pub fn rb_parent(pc: usize) -> *mut TaskStruct {
    (pc & !1) as *mut TaskStruct
}

#[inline]
pub fn rb_color(pc: usize) -> usize {
    pc & 1
}

#[inline]
pub fn rb_is_red(pc: usize) -> bool {
    rb_color(pc) == RB_RED
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 562) - [public] rb_color

```
pub fn rb_color(pc: usize) -> usize {
    pc & 1
}

#[inline]
pub fn rb_is_red(pc: usize) -> bool {
    rb_color(pc) == RB_RED
}

#[inline]
pub fn rb_is_black(pc: usize) -> bool {
    rb_color(pc) == RB_BLACK
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 567) - [public] rb_is_red

```
pub fn rb_is_red(pc: usize) -> bool {
    rb_color(pc) == RB_RED
}

#[inline]
pub fn rb_is_black(pc: usize) -> bool {
    rb_color(pc) == RB_BLACK
}

#[inline]
pub fn rb_make_parent_color(parent: *mut TaskStruct, color: usize) -> usize {
    (parent as usize & !1) | (color & 1)
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 572) - [public] rb_is_black

```
pub fn rb_is_black(pc: usize) -> bool {
    rb_color(pc) == RB_BLACK
}

#[inline]
pub fn rb_make_parent_color(parent: *mut TaskStruct, color: usize) -> usize {
    (parent as usize & !1) | (color & 1)
}

#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct LoadAvg {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 577) - [public] rb_make_parent_color

```
pub fn rb_make_parent_color(parent: *mut TaskStruct, color: usize) -> usize {
    (parent as usize & !1) | (color & 1)
}

#[repr(C)]
#[derive(Debug, Default, Clone, Copy)]
pub struct LoadAvg {
    pub last_update_time: u64,
    pub load_sum: u64,
    pub load_avg: u32,
    pub util_sum: u64,
    pub util_avg: u32,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 583) - [public] LoadAvg

```
pub struct LoadAvg {
    pub last_update_time: u64,
    pub load_sum: u64,
    pub load_avg: u32,
    pub util_sum: u64,
    pub util_avg: u32,
    pub period_contrib: u32,
}

const PELT_DECAY_SHIFT: u32 = 10;

impl LoadAvg {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 595) - [public] accumulate

```
pub fn accumulate(&mut self, now: u64, delta_ns: u64, weight: u64, running: bool) {
    if delta_ns == 0 {
        self.last_update_time = now;
        return;
    }
    // Wygaszenie starego wkładu: avg -= avg >> PELT_DECAY_SHIFT
    let load_decay = self.load_avg >> PELT_DECAY_SHIFT.min(31);
    self.load_avg = self.load_avg.saturating_sub(load_decay);
    let util_decay = self.util_avg >> PELT_DECAY_SHIFT.min(31);
    self.util_avg = self.util_avg.saturating_sub(util_decay);

    // Nowy wkład, przeskalowany przez czas trwania i wagę encji.
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 624) - [public] SchedEntity

```
pub struct SchedEntity {
    pub rb_left: *mut TaskStruct,
    pub rb_right: *mut TaskStruct,
    pub rb_parent_color: usize,

    pub run_list: *mut TaskStruct,

    pub vruntime: u64,
    pub vlag: i64,
    pub slice: u64,
    pub sum_exec_runtime: u64,
    pub prev_sum_exec_runtime: u64,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 647) - [internal] default

```
fn default() -> Self {
    Self {
        rb_left: ptr::null_mut(),
        rb_right: ptr::null_mut(),
        rb_parent_color: RB_RED, // parent = null, kolor = czerwony (wzrost nowego, poza drzewem)
        run_list: ptr::null_mut(),
        vruntime: 0,
        vlag: 0,
        slice: 0,
        sum_exec_runtime: 0,
        prev_sum_exec_runtime: 0,
        weight: NICE_0_LOAD,
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 671) - [public] RtSchedEntity

```
pub struct RtSchedEntity {
    pub rt_list: ListHead,
    pub rt_priority: u8,
    pub time_slice: u32,
    pub watchdog_stamp: u64,
    pub nr_cpus_allowed: u32,
    pub queued_prio: u32,
    pub owner: *mut TaskStruct,
}
```

```
impl Default for RtSchedEntity {
    fn default() -> Self {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 682) - [internal] default

```
fn default() -> Self {
    Self {
        rt_list: ListHead::new(),
        rt_priority: 0,
        time_slice: default_time_slice(SchedPolicy::RoundRobin),
        watchdog_stamp: 0,
        nr_cpus_allowed: MAX_CPUS as u32,
        queued_prio: MAX_RT_PRIO as u32,
        owner: ptr::null_mut(),
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 703) - [public] PlistNode

```
pub struct PlistNode {
    pub node: ListHead,
    pub same_prio: ListHead,
    pub priority: i32,
    pub owner: *mut TaskStruct,
}

impl PlistNode {
    pub fn init(&mut self, owner: *mut TaskStruct) {
        self.node.init();
        self.same_prio.init();
        self.priority = 0;
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 711) - [public] init

```
pub fn init(&mut self, owner: *mut TaskStruct) {
    self.node.init();
    self.same_prio.init();
    self.priority = 0;
    self.owner = owner;
}

impl Default for PlistNode {
    fn default() -> Self {
        Self {
            node: ListHead::new(),
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 720) - [internal] default

```
fn default() -> Self {
    Self {
        node: ListHead::new(),
        same_prio: ListHead::new(),
        priority: 0,
        owner: ptr::null_mut(),
    }
}

#[repr(C)]
pub struct DLSchedEntity {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 731) - [public] DLSchedEntity

```
pub struct DLSchedEntity {
    pub dl_runtime: u64,
    pub dl_deadline: u64,
    pub dl_period: u64,
    pub runtime: i64,
    pub deadline: u64,
    pub throttled: bool,
    pub yielded: bool,
    pub dl_bw: u64,
    pub replenish_at: u64,
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 744) - [internal] default

```
fn default() -> Self {
    Self {
        dl_runtime: 0,
        dl_deadline: 0,
        dl_period: 0,
        runtime: 0,
        deadline: 0,
        throttled: false,
        yielded: false,
        dl_bw: 0,
        replenish_at: 0,
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 762) - [public] TaskStats

```
pub struct TaskStats {
    pub start_time: u64,
    pub utime: u64,
    pub stime: u64,
    pub guest_time: u64,
    pub nr_voluntary_switches: u64,
    pub nr_involuntary_switches: u64,
    pub nr_migrations: u64,
    pub minflt: u64,
    pub majflt: u64,
    pub wait_sum: u64,
    pub sleep_sum: u64,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 781) - [public] Credentials

```
pub struct Credentials {
    pub uid: u32,
    pub gid: u32,
    pub euid: u32,
    pub egid: u32,
    pub suid: u32,
    pub sgid: u32,
    pub fsuid: u32,
    pub fsgid: u32,
    pub cap_effective: u64,
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 824) - [internal] default

```
fn default() -> Self {
    Credentials::kernel()
}

#[repr(C)]
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub struct RLimit {
    pub cur: u64,
    pub max: u64,
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 831) - [public] RLimit

```
pub struct RLimit {
    pub cur: u64,
    pub max: u64,
}

impl RLimit {
    pub const fn unlimited() -> Self {
        Self { cur: RLIM_INFINITY, max: RLIM_INFINITY }
    }

    pub const fn bounded(cur: u64, max: u64) -> Self {
        Self { cur, max }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 848) - [public]
RlimitResource

```
pub enum RlimitResource {
    Cpu = 0,
    Fsize = 1,
    Data = 2,
    Stack = 3,
    Core = 4,
    Rss = 5,
    Nofile = 6,
    As = 7,
    Nproc = 8,
    Memlock = 9,
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 872) - [public] SignalState

```
pub struct SignalState {
    pub pending: u64,
    pub blocked: u64,
}

impl SignalState {
    pub fn has_pending(&self) -> bool {
        (self.pending & !self.blocked) != 0
    }

    pub fn raise(&mut self, signum: u8) {
        if signum < 64 {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 878) - [public] has_pending

```
pub fn has_pending(&self) -> bool {
    (self.pending & !self.blocked) != 0
}

pub fn raise(&mut self, signum: u8) {
    if signum < 64 {
        self.pending |= 1u64 << signum;
    }
}

pub fn clear(&mut self, signum: u8) {
    if signum < 64 {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 882) - [public] raise

```
pub fn raise(&mut self, signum: u8) {
    if signum < 64 {
        self.pending |= 1u64 << signum;
    }
}

pub fn clear(&mut self, signum: u8) {
    if signum < 64 {
        self.pending &= !(1u64 << signum);
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 888) - [public] clear

```
pub fn clear(&mut self, signum: u8) {
    if signum < 64 {
        self.pending &= !(1u64 << signum);
    }
}
```

```
#[repr(C)]
#[derive(Debug)]
pub struct ListHead {
    pub prev: *mut ListHead,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 898) - [public] ListHead

```
pub struct ListHead {
    pub prev: *mut ListHead,
    pub next: *mut ListHead,
}

impl ListHead {
    pub const fn new() -> Self {
        Self { prev: ptr::null_mut(), next: ptr::null_mut() }
    }

    pub fn init(&mut self) {
        let self_ptr = self as *mut ListHead;
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 908) - [public] init

```
pub fn init(&mut self) {
    let self_ptr = self as *mut ListHead;
    self.prev = self_ptr;
    self.next = self_ptr;
}

pub fn is_empty(&self) -> bool {
    core::ptr::eq(self.next, self as *const ListHead as *mut ListHead)
    || self.next.is_null()
}

pub unsafe fn insert_after(&mut self, node: *mut ListHead) {
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 914) - [public] is_empty
    pub fn is_empty(&self) -> bool {
        core::ptr::eq(self.next, self as *const ListHead as *mut ListHead)
        || self.next.is_null()
    }

    pub unsafe fn insert_after(&mut self, node: *mut ListHead) {
        let self_ptr = self as *mut ListHead;
        let next = self.next;
        (*node).prev = self_ptr;
        (*node).next = next;
        (*next).prev = node;
        self.next = node;
        // ... (truncated, see source file for more)
    }

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 949) - [public] is_linked
    pub fn is_linked(&self) -> bool {
        !self.next.is_null() && !self.prev.is_null()
    }
}

impl Default for ListHead {
    fn default() -> Self {
        ListHead::new()
    }
}

#[cfg(all(target_arch = "x86_64", not(test)))]
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 955) - [internal] default
    fn default() -> Self {
        ListHead::new()
    }
}

#[cfg(all(target_arch = "x86_64", not(test)))]
#[inline(always)]
unsafe fn arch_local_irq_save() -> u64 {
    let flags: u64;
    core::arch::asm!(
        "pushfq",
        "cli",
        // ... (truncated, see source file for more)
    )
}

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 997) - [public] SpinLock
    pub struct SpinLock {
        locked: AtomicBool,
        #[cfg(debug_assertions)]
        holder_cpu: AtomicU32,
    }

    impl SpinLock {
        pub const fn new() -> Self {
            Self {
                locked: AtomicBool::new(false),
                #[cfg(debug_assertions)]
                holder_cpu: AtomicU32::new(CPU_NONE),
                // ... (truncated, see source file for more)
            }
        }
    }

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1012) - [public] lock

```
pub fn lock(&self) {
    while self
        .locked
        .compare_exchange_weak(false, true, Ordering::Acquire, Ordering::Relaxed)
        .is_err()
    {
        while self.locked.load(Ordering::Relaxed) {
            core::hint::spin_loop();
        }
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1024) - [public] unlock

```
pub fn unlock(&self) {
    self.locked.store(false, Ordering::Release);
}

pub fn try_lock(&self) -> bool {
    self.locked
        .compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed)
        .is_ok()
}

pub fn is_locked(&self) -> bool {
    self.locked.load(Ordering::Relaxed)
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1028) - [public] try_lock

```
pub fn try_lock(&self) -> bool {
    self.locked
        .compare_exchange(false, true, Ordering::Acquire, Ordering::Relaxed)
        .is_ok()
}

pub fn is_locked(&self) -> bool {
    self.locked.load(Ordering::Relaxed)
}

pub fn lock_irqsave(&self) -> usize {
    let flags = unsafe { arch_local_irq_save() };
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1034) - [public] is_locked

```
pub fn is_locked(&self) -> bool {
    self.locked.load(Ordering::Relaxed)
}

pub fn lock_irqsave(&self) -> usize {
    let flags = unsafe { arch_local_irq_save() };
    self.lock();
    flags as usize
}

pub fn unlock_irqrestore(&self, flags: usize) {
    self.unlock();
    unsafe { arch_local_irq_restore(flags as u64) };
}
// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1038) - [public] lock_irqsave

```
pub fn lock_irqsave(&self) -> usize {
    let flags = unsafe { arch_local_irq_save() };
    self.lock();
    flags as usize
}
pub fn unlock_irqrestore(&self, flags: usize) {
    self.unlock();
    unsafe { arch_local_irq_restore(flags as u64) };
}

pub fn lock_guard(&self) -> SpinLockGuard<'_> {
    self.lock();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1043) - [public] unlock_irqrestore

```
pub fn unlock_irqrestore(&self, flags: usize) {
    self.unlock();
    unsafe { arch_local_irq_restore(flags as u64) };
}

pub fn lock_guard(&self) -> SpinLockGuard<'_> {
    self.lock();
    SpinLockGuard { lock: self, saved_flags: None }
}

pub fn lock_irqsave_guard(&self) -> SpinLockGuard<'_> {
    let flags = self.lock_irqsave();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1048) - [public] lock_guard

```
pub fn lock_guard(&self) -> SpinLockGuard<'_> {
    self.lock();
    SpinLockGuard { lock: self, saved_flags: None }
}

pub fn lock_irqsave_guard(&self) -> SpinLockGuard<'_> {
    let flags = self.lock_irqsave();
    SpinLockGuard { lock: self, saved_flags: Some(flags) }
}

#[must_use = "SpinLockGuard zwalnia blokad przy Drop – porzucenie natychmiast j odblokuje"]
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1053) - [public] lock_irqsave_guard

```
pub fn lock_irqsave_guard(&self) -> SpinLockGuard<'_> {
    let flags = self.lock_irqsave();
    SpinLockGuard { lock: self, saved_flags: Some(flags) }
}

#[must_use = "SpinLockGuard zwalnia blokad przy Drop – porzucenie natychmiast j odblokuje"]
pub struct SpinLockGuard<'a> {
    lock: &'a SpinLock,
    saved_flags: Option<usize>,
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1060) - [public]

SpinLockGuard

```
pub struct SpinLockGuard<'a> {
    lock: &'a SpinLock,
    saved_flags: Option<usize>,
}

impl<'a> Drop for SpinLockGuard<'a> {
    fn drop(&mut self) {
        match self.saved_flags {
            Some(flags) => self.lock.unlock_irqrestore(flags),
            None => self.lock.unlock(),
        }
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1066) - [internal] drop

```
fn drop(&mut self) {
    match self.saved_flags {
        Some(flags) => self.lock.unlock_irqrestore(flags),
        None => self.lock.unlock(),
    }
}
```

```
#[derive(Debug, Clone, Copy, PartialEq, Eq)]
pub enum TaskError {
    OutOfMemory,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1076) - [public] TaskError

```
pub enum TaskError {
    OutOfMemory,
    InvalidStateTransition { from: TaskState, to: TaskState },
    InvalidRtPriority,
    InvalidNice,
    AffinityDenied,
    AffinityEmpty,
    TaskTerminal,
    ConcurrentStateChange,
    ResourceExhausted,
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1089) - [internal] fmt

```
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    match self {
        TaskError::OutOfMemory => write!(f, "brak pamieci na stos jdra"),
        TaskError::InvalidStateTransition { from, to } => {
            write!(f, "niedozwolone przejcie stanu: {:?} -> {:?}", from, to)
        }
        TaskError::InvalidRtPriority => write!(f, "priorytet RT poza zakresem 0..99"),
        TaskError::InvalidNice => write!(f, "nice poza zakresem -20..=19"),
        TaskError::AffinityDenied => write!(f, "PF_NO_SETAFFINITY blokuje zmian affinity"),
        TaskError::AffinityEmpty => write!(f, "pusta maska affinity"),
        TaskError::TaskTerminal => write!(f, "operacja niedozwolona na zadaniu terminalnym"),
        TaskError::ConcurrentStateChange => {
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1112) - [public] TaskStruct

```
pub struct TaskStruct {
    pub pid: TaskId,
    pub tgid: TaskId,
    pub ppid: TaskId,
    pub comm: [u8; TASK_COMM_LEN],
    pub flags: AtomicTaskFlags,
    state: AtomicU8,

    pub policy: SchedPolicy,
    pub sched_class: SchedClass,
    pub static_prio: i32,
    pub normal_prio: i32,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1403) - [public] state

```
pub fn state(&self) -> TaskState {
    TaskState::from_u8(self.state.load(Ordering::Acquire))
}

fn set_state_unchecked(&self, new_state: TaskState) {
    self.state.store(new_state as u8, Ordering::Release);
}

pub fn set_state(&self, new_state: TaskState) -> Result<(), TaskError> {
    loop {
        let current = self.state();
        if !current.can_transition_to(new_state) {
            return Err(TaskError::InvalidStateTransition { from: current, to: new_state });
        }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1407) - [internal]

set_state_unchecked

```
fn set_state_unchecked(&self, new_state: TaskState) {
    self.state.store(new_state as u8, Ordering::Release);
}

pub fn set_state(&self, new_state: TaskState) -> Result<(), TaskError> {
    loop {
        let current = self.state();
        if !current.can_transition_to(new_state) {
            return Err(TaskError::InvalidStateTransition { from: current, to: new_state });
        }
    }
    match self.state.compare_exchange_weak(
        current as u8,
        new_state as u8,
    ) {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1410) - [public] set_state

```
pub fn set_state(&self, new_state: TaskState) -> Result<(), TaskError> {
    loop {
        let current = self.state();
        if !current.can_transition_to(new_state) {
            return Err(TaskError::InvalidStateTransition { from: current, to: new_state });
        }
    }
    match self.state.compare_exchange_weak(
        current as u8,
        new_state as u8,
        Ordering::AcqRel,
        Ordering::Acquire,
    ) {
        // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1429) - [internal]

set_state_owned

```
fn set_state_owned(&mut self, new_state: TaskState) -> Result<(), TaskError> {
    self.set_state(new_state)
}

pub fn wake_up(&self) -> Result<(), TaskError> {
    match self.state() {
        TaskState::Interruptible | TaskState::Uninterruptible => self.set_state(TaskState::Runnable),
        TaskState::Runnable => Ok(()),
        other => Err(TaskError::InvalidStateTransition { from: other, to: TaskState::Runnable }),
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1433) - [public] wake_up

```
pub fn wake_up(&self) -> Result<(), TaskError> {
    match self.state() {
        TaskState::Interruptible | TaskState::Uninterruptible => self.set_state(TaskState::Runnable),
        TaskState::Runnable => Ok(()),
        other => Err(TaskError::InvalidStateTransition { from: other, to: TaskState::Runnable }),
    }
}

pub fn sleep(&self, interruptible: bool) -> Result<(), TaskError> {
    let target = if interruptible { TaskState::Interruptible } else { TaskState::Uninterruptible };
    self.set_state(target)
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1441) - [public] sleep

```
pub fn sleep(&self, interruptible: bool) -> Result<(), TaskError> {
    let target = if interruptible { TaskState::Interruptible } else { TaskState::Uninterruptible };
    self.set_state(target)
}

pub fn is_kernel_thread(&self) -> bool {
    self.flags.contains(TaskFlags::PF_KTHREAD)
}

pub fn is_idle_task(&self) -> bool {
    self.flags.contains(TaskFlags::PF_IDLE) || self.state() == TaskState::Idle
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1446) - [public]

is_kernel_thread

```
pub fn is_kernel_thread(&self) -> bool {
    self.flags.contains(TaskFlags::PF_KTHREAD)
}

pub fn is_idle_task(&self) -> bool {
    self.flags.contains(TaskFlags::PF_IDLE) || self.state() == TaskState::Idle
}

pub fn is_zombie(&self) -> bool {
    self.state() == TaskState::Zombie
}

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1450) - [public] is_idle_task
    pub fn is_idle_task(&self) -> bool {
        self.flags.contains(TaskFlags::PF_IDLE) || self.state() == TaskState::Idle
    }

    pub fn is_zombie(&self) -> bool {
        self.state() == TaskState::Zombie
    }

    pub fn is_runnable(&self) -> bool {
        self.state().is_runnable()
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1454) - [public] is_zombie
    pub fn is_zombie(&self) -> bool {
        self.state() == TaskState::Zombie
    }

    pub fn is_runnable(&self) -> bool {
        self.state().is_runnable()
    }

    pub fn needs_resched(&self) -> bool {
        self.flags.contains(TaskFlags::PF_NEED_RESCHED)
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1458) - [public] is_runnable
    pub fn is_runnable(&self) -> bool {
        self.state().is_runnable()
    }

    pub fn needs_resched(&self) -> bool {
        self.flags.contains(TaskFlags::PF_NEED_RESCHED)
    }

    pub fn set_need_resched(&self) {
        self.flags.fetch_insert(TaskFlags::PF_NEED_RESCHED);
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1462) - [public]
needs_resched
    pub fn needs_resched(&self) -> bool {
        self.flags.contains(TaskFlags::PF_NEED_RESCHED)
    }

    pub fn set_need_resched(&self) {
        self.flags.fetch_insert(TaskFlags::PF_NEED_RESCHED);
    }

    pub fn clear_need_resched(&self) {
        self.flags.fetch_remove(TaskFlags::PF_NEED_RESCHED);
    }

    // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1466) - [public]

set_need_resched

```
pub fn set_need_resched(&self) {
    self.flags.fetch_insert(TaskFlags::PF_NEED_RESCHED);
}

pub fn clear_need_resched(&self) {
    self.flags.fetch_remove(TaskFlags::PF_NEED_RESCHED);
}

pub fn set_nice(&mut self, nice: i8) -> Result<(), TaskError> {
    if !self.policy.is_fair() {
        return Err(TaskError::InvalidNice);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1470) - [public]

clear_need_resched

```
pub fn clear_need_resched(&self) {
    self.flags.fetch_remove(TaskFlags::PF_NEED_RESCHED);
}

pub fn set_nice(&mut self, nice: i8) -> Result<(), TaskError> {
    if !self.policy.is_fair() {
        return Err(TaskError::InvalidNice);
    }
    let clamped = nice.clamp(NICE_MIN, NICE_MAX);
    self.static_prio = DEFAULT_PRIO + clamped as i32;
    self.normal_prio = self.static_prio;
    self.prio = self.normal_prio;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1474) - [public] set_nice

```
pub fn set_nice(&mut self, nice: i8) -> Result<(), TaskError> {
    if !self.policy.is_fair() {
        return Err(TaskError::InvalidNice);
    }
    let clamped = nice.clamp(NICE_MIN, NICE_MAX);
    self.static_prio = DEFAULT_PRIO + clamped as i32;
    self.normal_prio = self.static_prio;
    self.prio = self.normal_prio;
    self.se.weight = nice_to_weight(clamped);
    self.se.inv_weight = nice_to_wmult(clamped);
    Ok(())
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1487) - [public]

set_rt_priority

```
pub fn set_rt_priority(&mut self, rt_priority: u8) -> Result<(), TaskError> {
    if !self.policy.is_realtime() {
        return Err(TaskError::InvalidRtPriority);
    }
    if rt_priority as i32 >= MAX_RT_PRIO {
        return Err(TaskError::InvalidRtPriority);
    }
    self.rt.rt_priority = rt_priority;
    self.static_prio = rt_priority as i32;
    self.normal_prio = self.static_prio;
    self.prio = self.normal_prio;
    Ok(())
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1501) - [public]

effective_prio

```
pub fn effective_prio(&self) -> i32 {
    self.prio
}

pub fn set_affinity(&mut self, mask: CpuMask) -> Result<(), TaskError> {
    if self.flags.contains(TaskFlags::PF_NO_SETAFFINITY) {
        return Err(TaskError::AffinityDenied);
    }
    if mask.is_empty() {
        return Err(TaskError::AffinityEmpty);
    }
    self.se.cpus_allowed = mask;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1505) - [public] set_affinity

```
pub fn set_affinity(&mut self, mask: CpuMask) -> Result<(), TaskError> {
    if self.flags.contains(TaskFlags::PF_NO_SETAFFINITY) {
        return Err(TaskError::AffinityDenied);
    }
    if mask.is_empty() {
        return Err(TaskError::AffinityEmpty);
    }
    self.se.cpus_allowed = mask;
    self.rt.nr_cpus_allowed = mask.count();
    Ok(())
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1517) - [public] can_run_on

```
pub fn can_run_on(&self, cpu: u32) -> bool {
    self.se.cpus_allowed.is_set(cpu)
}

/// nigdzie.
pub fn rq_ptr(&self) -> *mut core::ffi::c_void {
    self.rq.load(Ordering::Acquire)
}

pub fn set_rq_ptr(&self, rq: *mut core::ffi::c_void) {
    self.rq.store(rq, Ordering::Release);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1522) - [public] rq_ptr

/// nigdzie.

```
pub fn rq_ptr(&self) -> *mut core::ffi::c_void {
    self.rq.load(Ordering::Acquire)
}

pub fn set_rq_ptr(&self, rq: *mut core::ffi::c_void) {
    self.rq.store(rq, Ordering::Release);
}

pub fn charge_cputime(&mut self, delta_ns: u64) {
    self.se.prev_sum_exec_runtime = self.se.sum_exec_runtime;
    self.se.sum_exec_runtime = self.se.sum_exec_runtime.saturating_add(delta_ns);
    self.stats.utime = self.stats.utime.saturating_add(delta_ns);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1526) - [public] set_rq_ptr

```
pub fn set_rq_ptr(&self, rq: *mut core::ffi::c_void) {
    self.rq.store(rq, Ordering::Release);
}

pub fn charge_cputime(&mut self, delta_ns: u64) {
    self.se.prev_sum_exec_runtime = self.se.sum_exec_runtime;
    self.se.sum_exec_runtime = self.se.sum_exec_runtime.saturating_add(delta_ns);
    self.stats.utime = self.stats.utime.saturating_add(delta_ns);

    if self.policy.is_realtime() && self.rt.time_slice > 0 {
        let delta_ticks = (delta_ns / 1_000_000) as u32;
        self.rt.time_slice = self.rt.time_slice.saturating_sub(delta_ticks);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1530) - [public] charge_cputime

```
pub fn charge_cputime(&mut self, delta_ns: u64) {
    self.se.prev_sum_exec_runtime = self.se.sum_exec_runtime;
    self.se.sum_exec_runtime = self.se.sum_exec_runtime.saturating_add(delta_ns);
    self.stats.utime = self.stats.utime.saturating_add(delta_ns);

    if self.policy.is_realtime() && self.rt.time_slice > 0 {
        let delta_ticks = (delta_ns / 1_000_000) as u32;
        self.rt.time_slice = self.rt.time_slice.saturating_sub(delta_ticks);
    }

    if matches!(self.policy, SchedPolicy::Deadline) {
        self.dl.runtime -= delta_ns as i64;
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1549) - [public] record_voluntary_switch

```
pub fn record_voluntary_switch(&mut self) {
    self.stats.nr_voluntary_switches = self.stats.nr_voluntary_switches.saturating_add(1);
}

pub fn record_involuntary_switch(&mut self) {
    self.stats.nr_involuntary_switches = self.stats.nr_involuntary_switches.saturating_add(1);
}

pub fn record_migration(&mut self, new_cpu: u32) {
    self.stats.nr_migrations = self.stats.nr_migrations.saturating_add(1);
    self.se.nr_migrations = self.se.nr_migrations.saturating_add(1);
    self.se.last_cpu = new_cpu;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1553) - [public] record_involuntary_switch

```
pub fn record_involuntary_switch(&mut self) {
    self.stats.nr_involuntary_switches = self.stats.nr_involuntary_switches.saturating_add(1);
}

pub fn record_migration(&mut self, new_cpu: u32) {
    self.stats.nr_migrations = self.stats.nr_migrations.saturating_add(1);
    self.se.nr_migrations = self.se.nr_migrations.saturating_add(1);
    self.se.last_cpu = new_cpu;
    self.cpu.store(new_cpu, Ordering::Release);
}

pub fn set_comm(&mut self, name: &str) {
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1557) - [public]

record_migration

```
pub fn record_migration(&mut self, new_cpu: u32) {
    self.stats.nr_migrations = self.stats.nr_migrations.saturating_add(1);
    self.se.nr_migrations = self.se.nr_migrations.saturating_add(1);
    self.se.last_cpu = new_cpu;
    self.cpu.store(new_cpu, Ordering::Release);
}

pub fn set_comm(&mut self, name: &str) {
    self.comm = [0; TASK_COMM_LEN];
    let name_bytes = name.as_bytes();
    let copy_len = cmp::min(name_bytes.len(), TASK_COMM_LEN - 1);
    self.comm[..copy_len].copy_from_slice(&name_bytes[..copy_len]);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1564) - [public] set_comm

```
pub fn set_comm(&mut self, name: &str) {
    self.comm = [0; TASK_COMM_LEN];
    let name_bytes = name.as_bytes();
    let copy_len = cmp::min(name_bytes.len(), TASK_COMM_LEN - 1);
    self.comm[..copy_len].copy_from_slice(&name_bytes[..copy_len]);
}

pub fn comm_str(&self) -> &str {
    let end = self.comm.iter().position(|&b| b == 0).unwrap_or(TASK_COMM_LEN);
    str::from_utf8(&self.comm[..end]).unwrap_or("")
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1571) - [public] comm_str

```
pub fn comm_str(&self) -> &str {
    let end = self.comm.iter().position(|&b| b == 0).unwrap_or(TASK_COMM_LEN);
    str::from_utf8(&self.comm[..end]).unwrap_or("")
}

impl fmt::Debug for TaskStruct {
    fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
        f.debug_struct("TaskStruct")
            .field("pid", &self.pid)
            .field("tgid", &self.tgid)
            .field("comm", &self.comm_str())
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1578) - [internal] fmt

```
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    f.debug_struct("TaskStruct")
        .field("pid", &self.pid)
        .field("tgid", &self.tgid)
        .field("comm", &self.comm_str())
        .field("state", &self.state())
        .field("policy", &self.policy)
        .field("prio", &self.prio)
        .field("cpu", &self.cpu.load(Ordering::Relaxed))
        .finish()
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1592) - [internal] fmt

```
fn fmt(&self, f: &mut fmt::Formatter<'_>) -> fmt::Result {
    write!(
        f,
        "[{}] {} state={:?} prio={} cpu={}",
        self.pid,
        self.comm_str(),
        self.state(),
        self.prio,
        self.cpu.load(Ordering::Relaxed)
    )
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1608) - [public]

ThreadInfoView

```
pub struct ThreadInfoView<'a> {
    pub stack_base: *mut u8,
    pub stack_size: usize,
    pub context: &'a mut CpuContext,
    pub cpu: &'a AtomicU32,
    pub on_cpu: &'a AtomicBool,
    pub preempt_count: &'a AtomicI32,
}
```

```
impl TaskStruct {
    pub fn thread_info(&mut self) -> ThreadInfoView<'_> {
        ThreadInfoView {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1618) - [public] thread_info

```
pub fn thread_info(&mut self) -> ThreadInfoView<'_> {
    ThreadInfoView {
        stack_base: self.stack_base,
        stack_size: self.stack_size,
        context: &mut self.context,
        cpu: &self.cpu,
        on_cpu: &self.on_cpu,
        preempt_count: &self.preempt_count,
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1641) - [public]

fair_has_priority

```
pub fn fair_has_priority(a: &TaskStruct, b: &TaskStruct) -> bool {
    a.se.vruntime < b.se.vruntime
}

pub fn deadline_has_priority(a: &TaskStruct, b: &TaskStruct) -> bool {
    a.dl.deadline < b.dl.deadline
}
```

```
#[cfg(test)]
impl TaskStruct {
    pub fn blank() -> TaskStruct {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1645) - [public]

deadline_has_priority

```
pub fn deadline_has_priority(a: &TaskStruct, b: &TaskStruct) -> bool {
    a.dl.deadline < b.dl.deadline
}
```

#[cfg(test)]

```
impl TaskStruct {
    pub fn blank() -> TaskStruct {
        TaskStruct {
            pid: 0,
            tgid: 0,
            ppid: 0,
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1652) - [public] blank

```
pub fn blank() -> TaskStruct {
    TaskStruct {
        pid: 0,
        tgid: 0,
        ppid: 0,
        comm: [0; TASK_COMM_LEN],
        flags: AtomicTaskFlags::empty(),
        state: AtomicU8::new(TaskState::Dead as u8),
        policy: SchedPolicy::Normal,
        sched_class: SchedClass::Fair,
        static_prio: DEFAULT_PRIO,
        normal_prio: DEFAULT_PRIO,
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1696) - [public]

init_test_stub

```
pub fn init_test_stub(&mut self, pid: TaskId, policy: SchedPolicy, nice: i8) {
    self.pid = pid;
    self.tgid = pid;
    self.state = AtomicU8::new(TaskState::Runnable as u8);
    self.policy = policy;
    self.sched_class = SchedClass::from(policy);

    self.se = SchedEntity::default();
    self.se.weight = nice_to_weight(nice);
    self.se.inv_weight = nice_to_wmult(nice);

    self.rt = RtSchedEntity::default();
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1760) - [internal]

nice_to_weight_is_monotonically_decreasing

```
fn nice_to_weight_is_monotonically_decreasing() {
    let mut prev = u64::MAX;
    for nice in NICE_MIN..=NICE_MAX {
        let w = nice_to_weight(nice);
        assert!(w < prev);
        prev = w;
    }
}
```

#[test]

```
fn nice_zero_has_default_weight() {
    assert_eq!(nice_to_weight(0), NICE_0_LOAD);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1770) - [internal]

nice_zero_has_default_weight

```
fn nice_zero_has_default_weight() {
    assert_eq!(nice_to_weight(0), NICE_0_LOAD);
}

#[test]
fn nice_to_weight_clamps_out_of_range() {
    assert_eq!(nice_to_weight(-100), nice_to_weight(NICE_MIN));
    assert_eq!(nice_to_weight(100), nice_to_weight(NICE_MAX));
}

#[test]
fn weight_to_nice_round_trip_is_close() {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1775) - [internal]

nice_to_weight_clamps_out_of_range

```
fn nice_to_weight_clamps_out_of_range() {
    assert_eq!(nice_to_weight(-100), nice_to_weight(NICE_MIN));
    assert_eq!(nice_to_weight(100), nice_to_weight(NICE_MAX));
}

#[test]
fn weight_to_nice_round_trip_is_close() {
    for nice in NICE_MIN..=NICE_MAX {
        let w = nice_to_weight(nice);
        assert_eq!(weight_to_nice(w), nice);
    }
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1781) - [internal]

weight_to_nice_round_trip_is_close

```
fn weight_to_nice_round_trip_is_close() {
    for nice in NICE_MIN..=NICE_MAX {
        let w = nice_to_weight(nice);
        assert_eq!(weight_to_nice(w), nice);
    }
}

#[test]
fn calc_delta_fair_is_identity_at_nice_zero() {
    let d = calc_delta_fair(1_000_000, NICE_0_LOAD, nice_to_wmult(0));
    assert_eq!(d, 1_000_000);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1789) - [internal]

calc_delta_fair_is_identity_at_nice_zero

```
fn calc_delta_fair_is_identity_at_nice_zero() {
    let d = calc_delta_fair(1_000_000, NICE_0_LOAD, nice_to_wmult(0));
    assert_eq!(d, 1_000_000);
}

#[test]
fn calc_delta_fair_grows_for_lower_weight() {
    let heavy = calc_delta_fair(1_000_000, nice_to_weight(-5), nice_to_wmult(-5));
    let light = calc_delta_fair(1_000_000, nice_to_weight(5), nice_to_wmult(5));
    assert!(light > heavy);
}

// ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1795) - [internal]
calc_delta_fair_grows_for_lower_weight

fn calc_delta_fair_grows_for_lower_weight() {
    let heavy = calc_delta_fair(1_000_000, nice_to_weight(-5), nice_to_wmult(-5));
    let light = calc_delta_fair(1_000_000, nice_to_weight(5), nice_to_wmult(5));
    assert!(light > heavy);
}

#[test]
fn cpumask_basic_set_clear() {
    let mut mask = CpuMask::empty();
    assert!(mask.is_empty());
    mask.set(3);
    mask.set(70);
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1802) - [internal]
cpumask_basic_set_clear

fn cpumask_basic_set_clear() {
    let mut mask = CpuMask::empty();
    assert!(mask.is_empty());
    mask.set(3);
    mask.set(70);
    assert!(mask.is_set(3));
    assert!(mask.is_set(70));
    assert!(!mask.is_set(4));
    assert_eq!(mask.count(), 2);
    mask.clear(3);
    assert!(!mask.is_set(3));
    assert_eq!(mask.count(), 1);
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1817) - [internal]
cpumask_all_contains_every_cpu

fn cpumask_all_contains_every_cpu() {
    let mask = CpuMask::all();
    assert_eq!(mask.count(), MAX_CPUS as u32);
    assert!(mask.is_set(0));
    assert!(mask.is_set((MAX_CPUS - 1) as u32));
}

#[test]
fn cpumask_intersects() {
    let mut a = CpuMask::empty();
    let mut b = CpuMask::empty();
    a.set(5);
    // ... (truncated, see source file for more)
```

```
kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1825) - [internal]
cpumask_intersects

fn cpumask_intersects() {
    let mut a = CpuMask::empty();
    let mut b = CpuMask::empty();
    a.set(5);
    b.set(6);
    assert!(!a.intersects(&b));
    b.set(5);
    assert!(a.intersects(&b));
}

#[test]
fn cpumask_and_or() {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1836) - [internal]
cpumask_and_or

```
fn cpumask_and_or() {  
    let mut a = CpuMask::empty();  
    let mut b = CpuMask::empty();  
    a.set(1);  
    a.set(2);  
    b.set(2);  
    b.set(3);  
    assert_eq!(a.and(&b).count(), 1);  
    assert!(a.and(&b).is_set(2));  
    assert_eq!(a.or(&b).count(), 3);  
}  
  
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1849) - [internal]
cpumask_next_after_wraps

```
fn cpumask_next_after_wraps() {  
    let mut m = CpuMask::empty();  
    m.set(2);  
    m.set(5);  
    assert_eq!(m.next_after(2), Some(5));  
    assert_eq!(m.next_after(5), Some(2)); // zawija  
}  
  
#[test]  
fn cpumask_first_n() {  
    let m = CpuMask::first_n(4);  
    assert_eq!(m.count(), 4);  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1858) - [internal]
cpumask_first_n

```
fn cpumask_first_n() {  
    let m = CpuMask::first_n(4);  
    assert_eq!(m.count(), 4);  
    assert!(m.is_set(3));  
    assert!(!m.is_set(4));  
}  
  
#[test]  
fn task_state_valid_transitions() {  
    assert!(TaskState::Runnable.can_transition_to(TaskState::Interruptible));  
    assert!(TaskState::Interruptible.can_transition_to(TaskState::Runnable));  
    assert!(TaskState::Runnable.can_transition_to(TaskState::Zombie));  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1866) - [internal]
task_state_valid_transitions

```
fn task_state_valid_transitions() {  
    assert!(TaskState::Runnable.can_transition_to(TaskState::Interruptible));  
    assert!(TaskState::Interruptible.can_transition_to(TaskState::Runnable));  
    assert!(TaskState::Runnable.can_transition_to(TaskState::Zombie));  
    assert!(TaskState::Zombie.can_transition_to(TaskState::Dead));  
}  
  
#[test]  
fn task_state_invalid_transitions_are_rejected() {  
    assert!(!TaskState::Dead.can_transition_to(TaskState::Runnable));  
    assert!(!TaskState::Zombie.can_transition_to(TaskState::Runnable));  
    assert!(!TaskState::Idle.can_transition_to(TaskState::Runnable));  
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1874) - [internal]
task_state_invalid_transitions_are_rejected

    fn task_state_invalid_transitions_are_rejected() {
        assert!(TaskState::Dead.can_transition_to(TaskState::Runnable));
        assert!(TaskState::Zombie.can_transition_to(TaskState::Runnable));
        assert!(TaskState::Idle.can_transition_to(TaskState::Runnable));
    }

#[test]
fn default_rlimits_have_sane_stack_and_nofile() {
    let limits = default_rlimits();
    assert_eq!(limits[RlimitResource::Stack as usize].cur, 8 * 1024 * 1024);
    assert_eq!(limits[RlimitResource::Nofile as usize].cur, 1024);
    assert_eq!(limits[RlimitResource::Cpu as usize].cur, RLIM_INFINITY);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1881) - [internal]
default_rlimits_have_sane_stack_and_nofile

    fn default_rlimits_have_sane_stack_and_nofile() {
        let limits = default_rlimits();
        assert_eq!(limits[RlimitResource::Stack as usize].cur, 8 * 1024 * 1024);
        assert_eq!(limits[RlimitResource::Nofile as usize].cur, 1024);
        assert_eq!(limits[RlimitResource::Cpu as usize].cur, RLIM_INFINITY);
    }

#[test]
fn credentials_kernel_has_full_capabilities() {
    let cred = Credentials::kernel();
    assert_eq!(cred.uid, 0);
    assert_eq!(cred.cap_effective, u64::MAX);
    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1889) - [internal]
credentials_kernel_has_full_capabilities

    fn credentials_kernel_has_full_capabilities() {
        let cred = Credentials::kernel();
        assert_eq!(cred.uid, 0);
        assert_eq!(cred.cap_effective, u64::MAX);
    }

#[test]
fn credentials_user_has_no_extra_capabilities() {
    let cred = Credentials::user(1000, 1000);
    assert_eq!(cred.uid, 1000);
    assert_eq!(cred.cap_effective, 0);
}
// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1896) - [internal]
credentials_user_has_no_extra_capabilities

    fn credentials_user_has_no_extra_capabilities() {
        let cred = Credentials::user(1000, 1000);
        assert_eq!(cred.uid, 1000);
        assert_eq!(cred.cap_effective, 0);
    }

#[test]
fn signal_state_pending_respects_blocked_mask() {
    let mut sig = SignalState::default();
    sig.raise(9);
    assert!(sig.has_pending());
    sig.blocked |= 1 << 9;
    // ... (truncated, see source file for more)

```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1903) - [internal]
signal_state_pending_respects_blocked_mask

```
fn signal_state_pending_respects_blocked_mask() {  
    let mut sig = SignalState::default();  
    sig.raise(9);  
    assert!(sig.has_pending());  
    sig.blocked |= 1 << 9;  
    assert!(!sig.has_pending());  
    sig.clear(9);  
    assert!(!sig.has_pending());  
}  
  
#[test]  
fn list_head_insert_and_remove_roundtrip() {  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1914) - [internal]
list_head_insert_and_remove_roundtrip

```
fn list_head_insert_and_remove_roundtrip() {  
    let mut head = ListHead::new();  
    head.init();  
  
    let mut a = ListHead::new();  
    a.init();  
    let mut b = ListHead::new();  
    b.init();  
  
    unsafe {  
        head.insert_after(&mut a as *mut ListHead);  
        head.insert_after(&mut b as *mut ListHead);  
    }  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1942) - [internal]
list_head_remove_never_creates_a_cycle_with_three_nodes

```
fn list_head_remove_never_creates_a_cycle_with_three_nodes() {  
    let mut head = ListHead::new();  
    head.init();  
    let mut nodes: [ListHead; 3] = [ListHead::new(), ListHead::new(), ListHead::new()];  
    for n in nodes.iter_mut() {  
        n.init();  
    }  
    unsafe {  
        for n in nodes.iter_mut() {  
            head.insert_before(n as *mut ListHead);  
        }  
    }  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1966) - [internal]
spinlock_lock_unlock_cycle

```
fn spinlock_lock_unlock_cycle() {  
    let lock = SpinLock::new();  
    assert!(!lock.is_locked());  
    lock.lock();  
    assert!(lock.is_locked());  
    lock.unlock();  
    assert!(!lock.is_locked());  
}  
  
#[test]  
fn spinlock_try_lock_fails_when_held() {  
    let lock = SpinLock::new();  
    // ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1976) - [internal]

spinlock_try_lock_fails_when_held

```
fn spinlock_try_lock_fails_when_held() {
    let lock = SpinLock::new();
    assert!(lock.try_lock());
    assert!(!lock.try_lock());
    lock.unlock();
    assert!(lock.try_lock());
    lock.unlock();
}

#[test]
fn spinlock_irqsave_guard_unlocks_on_drop() {
    let lock = SpinLock::new();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1986) - [internal]

spinlock_irqsave_guard_unlocks_on_drop

```
fn spinlock_irqsave_guard_unlocks_on_drop() {
    let lock = SpinLock::new();
    {
        let _guard = lock.lock_irqsave_guard();
        assert!(lock.is_locked());
    }
    assert!(!lock.is_locked());
}

#[test]
fn spinlock_plain_guard_unlocks_on_drop() {
    let lock = SpinLock::new();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 1996) - [internal]

spinlock_plain_guard_unlocks_on_drop

```
fn spinlock_plain_guard_unlocks_on_drop() {
    let lock = SpinLock::new();
    {
        let _guard = lock.lock_guard();
        assert!(lock.is_locked());
    }
    assert!(!lock.is_locked());
}

#[test]
fn atomic_task_flags_insert_remove_are_bitwise() {
    let flags = AtomicTaskFlags::empty();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2006) - [internal]

atomic_task_flags_insert_remove_are_bitwise

```
fn atomic_task_flags_insert_remove_are_bitwise() {
    let flags = AtomicTaskFlags::empty();
    flags.fetch_insert(TaskFlags::PF_NEED_RESCHED);
    assert!(flags.contains(TaskFlags::PF_NEED_RESCHED));
    flags.fetch_insert(TaskFlags::PF_KTHREAD);
    assert!(flags.contains(TaskFlags::PF_NEED_RESCHED));
    assert!(flags.contains(TaskFlags::PF_KTHREAD));
    flags.fetch_remove(TaskFlags::PF_NEED_RESCHED);
    assert!(!flags.contains(TaskFlags::PF_NEED_RESCHED));
    assert!(flags.contains(TaskFlags::PF_KTHREAD));
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2019) - [internal]
atomic_task_flags_test_and_set_is_edge_triggered

```
fn atomic_task_flags_test_and_set_is_edge_triggered() {
    let flags = AtomicTaskFlags::empty();
    assert!(flags.test_and_set(TaskFlags::PF_WAKING));
    assert!(!flags.test_and_set(TaskFlags::PF_WAKING));
}

#[test]
fn rb_parent_color_encoding_roundtrips() {
    let fake_parent = 0x1000usize as *mut TaskStruct;
    let pc_red = rb_make_parent_color(fake_parent, RB_RED);
    let pc_black = rb_make_parent_color(fake_parent, RB_BLACK);
    assert_eq!(rb_parent(pc_red), fake_parent);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2026) - [internal]
rb_parent_color_encoding_roundtrips

```
fn rb_parent_color_encoding_roundtrips() {
    let fake_parent = 0x1000usize as *mut TaskStruct;
    let pc_red = rb_make_parent_color(fake_parent, RB_RED);
    let pc_black = rb_make_parent_color(fake_parent, RB_BLACK);
    assert_eq!(rb_parent(pc_red), fake_parent);
    assert_eq!(rb_parent(pc_black), fake_parent);
    assert!(rb_is_red(pc_red));
    assert!(rb_is_black(pc_black));
}

#[test]
fn fx_save_area_default_is_zeroed() {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2037) - [internal]
fx_save_area_default_is_zeroed

```
fn fx_save_area_default_is_zeroed() {
    let area = FxSaveArea::default();
    assert!(area.bytes.iter().all(|&b| b == 0));
    assert_eq!(core::mem::size_of::<FxSaveArea>(), 512);
    assert_eq!(core::mem::align_of::<FxSaveArea>(), 16);
}

#[test]
fn cpu_context_default_has_no_valid_fpu_state() {
    let ctx = CpuContext::default();
    assert!(!ctx.fpu_valid);
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2045) - [internal]
cpu_context_default_has_no_valid_fpu_state

```
fn cpu_context_default_has_no_valid_fpu_state() {
    let ctx = CpuContext::default();
    assert!(!ctx.fpu_valid);
}

#[test]
fn sched_policy_maps_to_expected_class() {
    assert_eq!(SchedClass::from(SchedPolicy::Normal), SchedClass::Fair);
    assert_eq!(SchedClass::from(SchedPolicy::Batch), SchedClass::Fair);
    assert_eq!(SchedClass::from(SchedPolicy::Idle), SchedClass::Idle);
    assert_eq!(SchedClass::from(SchedPolicy::Fifo), SchedClass::RealTime);
    assert_eq!(SchedClass::from(SchedPolicy::RoundRobin), SchedClass::RealTime);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2051) - [internal]

sched_policy_maps_to_expected_class

```
fn sched_policy_maps_to_expected_class() {
    assert_eq!(SchedClass::from(SchedPolicy::Normal), SchedClass::Fair);
    assert_eq!(SchedClass::from(SchedPolicy::Batch), SchedClass::Fair);
    assert_eq!(SchedClass::from(SchedPolicy::Idle), SchedClass::Idle);
    assert_eq!(SchedClass::from(SchedPolicy::Fifo), SchedClass::RealTime);
    assert_eq!(SchedClass::from(SchedPolicy::RoundRobin), SchedClass::RealTime);
    assert_eq!(SchedClass::from(SchedPolicy::Deadline), SchedClass::Deadline);
    assert_eq!(SchedClass::from(SchedPolicy::Stop), SchedClass::Stop);
}

#[test]
fn sched_class_ordering_matches_pick_next_priority() {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2062) - [internal]

sched_class_ordering_matches_pick_next_priority

```
fn sched_class_ordering_matches_pick_next_priority() {
    assert!(SchedClass::Stop > SchedClass::Deadline);
    assert!(SchedClass::Deadline > SchedClass::RealTime);
    assert!(SchedClass::RealTime > SchedClass::Fair);
    assert!(SchedClass::Fair > SchedClass::Idle);
}

#[test]
fn default_time_slice_matches_policy_expectations() {
    assert_eq!(default_time_slice(SchedPolicy::Fifo), 0);
    assert!(default_time_slice(SchedPolicy::RoundRobin) > 0);
    assert_eq!(default_time_slice(SchedPolicy::Normal), 0);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2070) - [internal]

default_time_slice_matches_policy_expectations

```
fn default_time_slice_matches_policy_expectations() {
    assert_eq!(default_time_slice(SchedPolicy::Fifo), 0);
    assert!(default_time_slice(SchedPolicy::RoundRobin) > 0);
    assert_eq!(default_time_slice(SchedPolicy::Normal), 0);
}

#[test]
fn load_avg_accumulate_grows_then_decays() {
    let mut la = LoadAvg::default();
    la.accumulate(1_000_000_000, 1_000_000_000, NICE_0_LOAD, true);
    let after_first = la.load_avg;
    assert!(after_first > 0);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2077) - [internal]

load_avg_accumulate_grows_then_decays

```
fn load_avg_accumulate_grows_then_decays() {
    let mut la = LoadAvg::default();
    la.accumulate(1_000_000_000, 1_000_000_000, NICE_0_LOAD, true);
    let after_first = la.load_avg;
    assert!(after_first > 0);
    la.accumulate(2_000_000_000, 1_000_000_000, 0, false);
    assert!(la.load_avg <= after_first);
}

fn bare_task_for_state_tests() -> TaskStruct {
    TaskStruct {
        pid: 1,
    }
    // ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2086) - [internal]
bare_task_for_state_tests
    fn bare_task_for_state_tests() -> TaskStruct {
        TaskStruct {
            pid: 1,
            tgid: 1,
            ppid: 0,
            comm: [0; TASK_COMM_LEN],
            flags: AtomicTaskFlags::empty(),
            state: AtomicU8::new(TaskState::Runnable as u8),
            policy: SchedPolicy::Normal,
            sched_class: SchedClass::Fair,
            static_prio: DEFAULT_PRIO,
            normal_prio: DEFAULT_PRIO,
            // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2131) - [internal]
set_state_via_shared_reference_does_not_need_mut
    fn set_state_via_shared_reference_does_not_need_mut() {
        let task = bare_task_for_state_tests();
        let task_ref: &TaskStruct = &task;
        assert!(task_ref.set_state(TaskState::Interruptible).is_ok());
        assert_eq!(task_ref.state(), TaskState::Interruptible);
        assert!(task_ref.wake_up().is_ok());
        assert_eq!(task_ref.state(), TaskState::Runnable);
    }

    #[test]
    fn set_state_rejects_illegal_transition_via_shared_reference() {
        let task = bare_task_for_state_tests();
        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2141) - [internal]
set_state_rejects_illegal_transition_via_shared_reference
    fn set_state_rejects_illegal_transition_via_shared_reference() {
        let task = bare_task_for_state_tests();
        assert!(task.set_state(TaskState::Zombie).is_ok());
        assert!(task.set_state(TaskState::Runnable).is_err());
        assert_eq!(task.state(), TaskState::Zombie);
    }

    #[test]
    fn wake_up_process_pattern_from_remote_cpu_context() {
        let task = bare_task_for_state_tests();
        task.set_state(TaskState::Uninterruptible).unwrap();

        // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2149) - [internal]
wake_up_process_pattern_from_remote_cpu_context
    fn wake_up_process_pattern_from_remote_cpu_context() {
        let task = bare_task_for_state_tests();
        task.set_state(TaskState::Uninterruptible).unwrap();

        fn remote_wake(t: &TaskStruct) -> Result<(), TaskError> {
            t.wake_up()
        }

        assert!(remote_wake(&task).is_ok());
        assert_eq!(task.state(), TaskState::Runnable);
    }

    // ... (truncated, see source file for more)

```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2153) - [internal]
remote_wake

```
fn remote_wake(t: &TaskStruct) -> Result<(), TaskError> {
    t.wake_up()
}

assert!(remote_wake(&task).is_ok());
assert_eq!(task.state(), TaskState::Runnable);
}

#[test]
fn rq_backpointer_roundtrips() {
    let task = bare_task_for_state_tests();
    assert!(task.rq_ptr().is_null());
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2162) - [internal]
rq_backpointer_roundtrips

```
fn rq_backpointer_roundtrips() {
    let task = bare_task_for_state_tests();
    assert!(task.rq_ptr().is_null());
    let fake: usize = 0xdead_beef;
    task.set_rq_ptr(fake as *mut core::ffi::c_void);
    assert_eq!(task.rq_ptr() as usize, fake);
}

#[test]
fn need_resched_flag_is_settable_from_shared_reference() {
    let task = bare_task_for_state_tests();
    assert!(task.needs_resched());
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2171) - [internal]
need_resched_flag_is_settable_from_shared_reference

```
fn need_resched_flag_is_settable_from_shared_reference() {
    let task = bare_task_for_state_tests();
    assert!(task.needs_resched());
    task.set_need_resched();
    assert!(task.needs_resched());
    task.clear_need_resched();
    assert!(task.needs_resched());
}

#[test]
fn rt_sched_entity_starts_with_initialized_list_and_no_queued_prio() {
    let rt = RtSchedEntity::default();
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/entities/task.rs (line 2181) - [internal]
rt_sched_entity_starts_with_initialized_list_and_no_queued_prio

```
fn rt_sched_entity_starts_with_initialized_list_and_no_queued_prio() {
    let rt = RtSchedEntity::default();
    assert_eq!(rt.queued_prio, MAX_RT_PRIO as u32);
}

impl TaskStruct {
    pub const PLIST_PRIO_OFFSET: usize = core::mem::offset_of!(TaskStruct, prio);
    pub const PLIST_SAME_PRIO_OFFSET: usize = core::mem::offset_of!(TaskStruct, thread_group);
    pub const PLIST_NODE_OFFSET: usize = core::mem::offset_of!(TaskStruct, tasks);
    pub const RB_LEFT_OFFSET: usize = core::mem::offset_of!(TaskStruct, se.rb_left);
    pub const RB_RIGHT_OFFSET: usize = core::mem::offset_of!(TaskStruct, se.rb_right);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/mod.rs (line 57) - [public] self_test

```
pub fn self_test() -> Result<'static str, &'static str> {
    Ok("scheduler: stub (no runqueues online)")
}

pub unsafe fn get_rq(cpu: u32) -> *mut RunQueue {
    if cpu as usize >= MAX_CPUS {
        return core::ptr::null_mut();
    }
    RUN_QUEUES[cpu as usize]
}

pub fn current_cpu_id() -> u32 {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/mod.rs (line 68) - [public] current_cpu_id

```
pub fn current_cpu_id() -> u32 {
    0
}
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 16) - [public] CapacityState

```
pub struct CapacityState {
    pub frequency: u32,
    pub power: u32,
    pub cost: u32,
    pub performance: u32,
    pub capacity: u32,
    pub voltage: u32,
    pub transition_latency_ns: u32,
    pub flags: u32,
}
```

```
impl CapacityState {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 54) - [public] compute_cost

```
pub fn compute_cost(&mut self, max_cap: u32) {
    if self.capacity == 0 {
        self.cost = u32::MAX;
        return;
    }
    let inverted_cap = max_cap - self.capacity;
    let power_cost = (self.power as u64 * (inverted_cap as u64 + 1)) / EM_CAPACITY_SCALE;
    self.cost = power_cost.min(u32::MAX as u64) as u32;
}

pub fn is_valid(&self) -> bool {
    self.frequency > 0 && self.capacity > 0 && self.power > 0
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 64) - [public] is_valid

```
pub fn is_valid(&self) -> bool {
    self.frequency > 0 && self.capacity > 0 && self.power > 0
}
}
```

```
#[repr(C)]
```

```
pub struct PerformanceDomain {
    pub states: [CapacityState; MAX_OPP_COUNT],
    pub nr_states: u32,
    pub cpus: CpuMask,
    pub td_lock: crate::cpu::scheduler::entities::task::SpinLock,
    pub max_capacity: u32,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 70) - [public] PerformanceDomain

```
pub struct PerformanceDomain {
    pub states: [CapacityState; MAX_OPP_COUNT],
    pub nr_states: u32,
    pub cpus: CpuMask,
    pub td_lock: crate::cpu::scheduler::entities::task::SpinLock,
    pub max_capacity: u32,
    pub min_capacity: u32,
    pub flags: u32,
    pub id: u32,
    pub thermal_pressure: AtomicU32,
    pub effective_capacity: AtomicU32,
    pub active_state_idx: AtomicU32,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 120) - [public] init

```
pub fn init(&mut self, id: u32, cpus: CpuMask) {
    self.id = id;
    self.cpus = cpus;
    self.nr_states = 0;
    self.max_capacity = 0;
    self.min_capacity = u32::MAX;
    self.thermal_pressure.store(0, Ordering::Relaxed);
    self.effective_capacity.store(0, Ordering::Relaxed);
    self.active_state_idx.store(0, Ordering::Relaxed);
    self.current_temp_millideg.store(self.ambient_temp_celsius * 1000, Ordering::Relaxed);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 132) - [public] add_state

```
pub fn add_state(&mut self, state: CapacityState) -> bool {
    if self.nr_states as usize >= MAX_OPP_COUNT {
        return false;
    }
    if !state.is_valid() {
        return false;
    }
    let idx = self.nr_states as usize;
    self.states[idx] = state;
    self.nr_states += 1;
    if state.capacity > self.max_capacity {
        self.max_capacity = state.capacity;
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 151) - [public] finalize

```
pub fn finalize(&mut self) {
    let max_cap = self.max_capacity;
    for i in 0..self.nr_states as usize {
        self.states[i].compute_cost(max_cap);
    }
    self.sort_states();
    self.effective_capacity.store(self.max_capacity, Ordering::Release);
}

fn sort_states(&mut self) {
    let n = self.nr_states as usize;
    for i in 0..n {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 160) - [internal] sort_states

```
fn sort_states(&mut self) {
    let n = self.nr_states as usize;
    for i in 0..n {
        for j in (i + 1)..n {
            if self.states[j].capacity < self.states[i].capacity {
                let tmp = self.states[i];
                self.states[i] = self.states[j];
                self.states[j] = tmp;
            }
        }
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 173) - [public]

find_state_for_capacity

```
pub fn find_state_for_capacity(&self, target_cap: u32) -> Option<&CapacityState> {
    for i in 0..self.nr_states as usize {
        if self.states[i].capacity >= target_cap {
            return Some(&self.states[i]);
        }
    }
    if self.nr_states > 0 {
        Some(&self.states[self.nr_states as usize - 1])
    } else {
        None
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 186) - [public]

find_state_for_frequency

```
pub fn find_state_for_frequency(&self, target_freq: u32) -> Option<&CapacityState> {
    let mut best: Option<&CapacityState> = None;
    let mut best_diff = u32::MAX;
    for i in 0..self.nr_states as usize {
        let diff = if self.states[i].frequency > target_freq {
            self.states[i].frequency - target_freq
        } else {
            target_freq - self.states[i].frequency
        };
        if diff < best_diff {
            best_diff = diff;
            best = Some(&self.states[i]);
        }
    }
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 203) - [public]

get_effective_capacity

```
pub fn get_effective_capacity(&self) -> u32 {
    self.effective_capacity.load(Ordering::Acquire)
}

pub fn update_thermal_pressure(&self, pressure: u32) {
    self.thermal_pressure.store(pressure, Ordering::Relaxed);
    let max_cap = self.max_capacity;
    let effective = if pressure >= max_cap {
        self.min_capacity
    } else {
        max_cap - pressure
    };
}
// ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 207) - [public]

update_thermal_pressure

```
pub fn update_thermal_pressure(&self, pressure: u32) {
    self.thermal_pressure.store(pressure, Ordering::Relaxed);
    let max_cap = self.max_capacity;
    let effective = if pressure >= max_cap {
        self.min_capacity
    } else {
        max_cap - pressure
    };
    self.effective_capacity.store(effective, Ordering::Release);
}

pub fn compute_power_at_state(&self, state_idx: usize, temp_millideg: u32) -> u32 {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 218) - [public]

compute_power_at_state

```
pub fn compute_power_at_state(&self, state_idx: usize, temp_millideg: u32) -> u32 {
    if state_idx >= self.nr_states as usize {
        return 0;
    }
    let state = &self.states[state_idx];
    let freq_mhz = state.frequency as u64;
    let volt_mv = state.voltage as u64;
    let dyn_power = (self.dynamic_coefficient as u64 * freq_mhz * volt_mv * volt_mv) / (EM_POWER_SCALE as u64);
    let temp_celsius = temp_millideg / 1000;
    let temp_factor = if temp_celsius > 25 {
        1024 + ((temp_celsius - 25) * 10)
    } else {
        1024
    };
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 237) - [public] update_temperature

```
pub fn update_temperature(&self, power_mw: u32, delta_time_ms: u32) {
    let current_temp = self.current_temp_millideg.load(Ordering::Relaxed);
    let ambient = self.ambient_temp_celsius * 1000;
    let thermal_r = self.thermal_resistance as u64;
    let thermal_c = self.thermal_capacitance as u64;
    let dt = delta_time_ms as u64;
    let power = power_mw as u64;
    let temp_diff = current_temp as i64 - ambient as i64;
    let heat_flow = (temp_diff * 1000) / (thermal_r.max(1) as i64);
    let net_heat = ((power as i64) * (dt as i64)) - heat_flow;
    let temp_delta = (net_heat * 1000) / (thermal_c.max(1) as i64);
    let new_temp = (current_temp as i64 + temp_delta).max(ambient as i64).min(self.max_temp_millideg as i64);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 252) - [public] contains_cpu

```
pub fn contains_cpu(&self, cpu: u32) -> bool {
    self.cpus.is_set(cpu)
}

pub fn get_active_state_idx(&self) -> u32 {
    self.active_state_idx.load(Ordering::Acquire)
}

pub fn set_active_state_idx(&self, idx: u32) {
    if idx < self.nr_states {
        self.active_state_idx.store(idx, Ordering::Release);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 256) - [public]

get_active_state_idx

```
pub fn get_active_state_idx(&self) -> u32 {
    self.active_state_idx.load(Ordering::Acquire)
}

pub fn set_active_state_idx(&self, idx: u32) {
    if idx < self.nr_states {
        self.active_state_idx.store(idx, Ordering::Release);
    }
}

#[repr(C)]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 260) - [public]

set_active_state_idx

```
pub fn set_active_state_idx(&self, idx: u32) {
    if idx < self.nr_states {
        self.active_state_idx.store(idx, Ordering::Release);
    }
}

#[repr(C)]
pub struct EnergyModel {
    pub domains: [PerformanceDomain; MAX_PD_COUNT],
    pub nr_domains: u32,
    pub cpu_to_pd: [u32; MAX_CPUS],
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 268) - [public] EnergyModel

```
pub struct EnergyModel {
    pub domains: [PerformanceDomain; MAX_PD_COUNT],
    pub nr_domains: u32,
    pub cpu_to_pd: [u32; MAX_CPUS],
    pub lock: crate::cpu::scheduler::entities::task::SpinLock,
    pub flags: u32,
    pub origin: u32,
    pub global_max_capacity: u32,
    pub global_min_capacity: u32,
    pub total_system_power_mw: AtomicU32,
    pub last_update_time_ns: AtomicU64,
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 298) - [public] init

```
pub fn init(&mut self) {
    self.nr_domains = 0;
    self.global_max_capacity = 0;
    self.global_min_capacity = u32::MAX;
    for i in 0..MAX_CPUS {
        self.cpu_to_pd[i] = u32::MAX;
    }
}

pub fn register_domain(&mut self, pd: PerformanceDomain) -> bool {
    if self.nr_domains as usize >= MAX_PD_COUNT { return false; }
    let idx = self.nr_domains as usize;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 307) - [public] register_domain

```
pub fn register_domain(&mut self, pd: PerformanceDomain) -> bool {
    if self.nr_domains as usize >= MAX_PD_COUNT { return false; }
    let idx = self.nr_domains as usize;

    if pd.max_capacity > self.global_max_capacity {
        self.global_max_capacity = pd.max_capacity;
    }
    if pd.min_capacity < self.global_min_capacity {
        self.global_min_capacity = pd.min_capacity;
    }

    self.domains[idx] = pd; // Przeniesienie następuje tutaj, po odczytaniu wartości
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 329) - [public] get_pd_for_cpu

```
pub fn get_pd_for_cpu(&self, cpu: u32) -> Option<&PerformanceDomain> {
    if cpu as usize >= MAX_CPUS {
        return None;
    }
    let pd_idx = self.cpu_to_pd[cpu as usize];
    if pd_idx == u32::MAX || pd_idx >= self.nr_domains {
        return None;
    }
    Some(&self.domains[pd_idx as usize])
}

pub fn get_pd_for_cpu_mut(&mut self, cpu: u32) -> Option<&mut PerformanceDomain> {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 340) - [public] get_pd_for_cpu_mut

```
pub fn get_pd_for_cpu_mut(&mut self, cpu: u32) -> Option<&mut PerformanceDomain> {
    if cpu as usize >= MAX_CPUS {
        return None;
    }
    let pd_idx = self.cpu_to_pd[cpu as usize];
    if pd_idx == u32::MAX || pd_idx >= self.nr_domains {
        return None;
    }
    Some(&mut self.domains[pd_idx as usize])
}

pub fn compute_system_energy(&self, delta_time_ms: u32) -> u32 {
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 351) - [public] compute_system_energy

```
pub fn compute_system_energy(&self, delta_time_ms: u32) -> u32 {
    let mut total_power = 0u32;
    for i in 0..self.nr_domains as usize {
        let pd = &self.domains[i];
        let active_idx = pd.get_active_state_idx() as usize;
        let temp = pd.current_temp_millideg.load(Ordering::Relaxed);
        let power = pd.compute_power_at_state(active_idx, temp);
        total_power = total_power.saturating_add(power);
        pd.update_temperature(power, delta_time_ms);
    }
    self.total_system_power_mw.store(total_power, Ordering::Relaxed);
    total_power
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 365) - [public]

compute_task_energy_on_cpu

```
pub fn compute_task_energy_on_cpu(&self, task_util: u32, cpu: u32) -> u32 {
    let pd = match self.get_pd_for_cpu(cpu) {
        Some(p) => p,
        None => return u32::MAX,
    };
    let target_cap = (task_util as u64 * pd.max_capacity as u64 / 1024) as u32;
    let state = match pd.find_state_for_capacity(target_cap) {
        Some(s) => s,
        None => return u32::MAX,
    };
    let temp = pd.current_temp_millideg.load(Ordering::Relaxed);
    let power = pd.compute_power_at_state(pd.states.iter().position(|s| s.capacity == state.capacity)
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 381) - [public]

find_best_state_for_domain

```
pub fn find_best_state_for_domain(&self, pd_idx: u32, target_util: u32) -> Option<usize> {
    if pd_idx >= self.nr_domains {
        return None;
    }
    let pd = &self.domains[pd_idx as usize];
    let eff_cap = pd.get_effective_capacity();
    let target_cap = (target_util as u64 * eff_cap as u64 / 1024) as u32;
    let mut best_idx = None;
    let mut best_cost = u32::MAX;
    for i in 0..pd.nr_states as usize {
        if pd.states[i].capacity >= target_cap {
            if pd.states[i].cost < best_cost {
                best_idx = Some(i);
                best_cost = pd.states[i].cost;
            }
        }
    }
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 404) - [public]

invert_capacity_for_thermal

```
pub fn invert_capacity_for_thermal(&mut self, cpu: u32, thermal_pressure: u32) {
    if let Some(pd) = self.get_pd_for_cpu_mut(cpu) {
        pd.update_thermal_pressure(thermal_pressure);
    }
}

pub fn clear_thermal_pressure(&mut self) {
    for i in 0..self.nr_domains as usize {
        self.domains[i].update_thermal_pressure(0);
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 410) - [public]

clear_thermal_pressure

```
pub fn clear_thermal_pressure(&mut self) {
    for i in 0..self.nr_domains as usize {
        self.domains[i].update_thermal_pressure(0);
    }
}

pub fn get_max_capacity(&self) -> u32 {
    self.global_max_capacity
}

pub fn get_min_capacity(&self) -> u32 {
    self.global_min_capacity
}

// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 416) - [public] get_max_capacity
    pub fn get_max_capacity(&self) -> u32 {
        self.global_max_capacity
    }

    pub fn get_min_capacity(&self) -> u32 {
        self.global_min_capacity
    }

    pub fn is_cpu_big(&self, cpu: u32) -> bool {
        if let Some(pd) = self.get_pd_for_cpu(cpu) {
            pd.max_capacity > (self.global_max_capacity / 2)
        } else {
            // ... (truncated, see source file for more)
        }
    }

kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 420) - [public] get_min_capacity
    pub fn get_min_capacity(&self) -> u32 {
        self.global_min_capacity
    }

    pub fn is_cpu_big(&self, cpu: u32) -> bool {
        if let Some(pd) = self.get_pd_for_cpu(cpu) {
            pd.max_capacity > (self.global_max_capacity / 2)
        } else {
            false
        }
    }

    // ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 424) - [public] is_cpu_big
    pub fn is_cpu_big(&self, cpu: u32) -> bool {
        if let Some(pd) = self.get_pd_for_cpu(cpu) {
            pd.max_capacity > (self.global_max_capacity / 2)
        } else {
            false
        }
    }

    pub fn is_cpu_little(&self, cpu: u32) -> bool {
        if let Some(pd) = self.get_pd_for_cpu(cpu) {
            pd.max_capacity <= (self.global_max_capacity / 2)
        } else {
            // ... (truncated, see source file for more)
        }
    }

kernel_Workspace/kernel/src/cpu/scheduler/power/em.rs (line 432) - [public] is_cpu_little
    pub fn is_cpu_little(&self, cpu: u32) -> bool {
        if let Some(pd) = self.get_pd_for_cpu(cpu) {
            pd.max_capacity <= (self.global_max_capacity / 2)
        } else {
            false
        }
    }

    pub fn dump_state(&self) {
        let _ = self.nr_domains;
        let _ = self.global_max_capacity;
        let _ = self.total_system_power_mw.load(Ordering::Relaxed);
        // ... (truncated, see source file for more)
    }

```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 440) - [public] dump_state

```
pub fn dump_state(&self) {
    let _ = self.nr_domains;
    let _ = self.global_max_capacity;
    let _ = self.total_system_power_mw.load(Ordering::Relaxed);
}

pub static mut GLOBAL_ENERGY_MODEL: EnergyModel = EnergyModel::empty();

pub unsafe fn em_init() {
    GLOBAL_ENERGY_MODEL.init();
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 484) - [public]
em_build_synthetic_little_pd

```
pub fn em_build_synthetic_little_pd(cpus: CpuMask) -> PerformanceDomain {
    let mut pd = PerformanceDomain::empty();
    pd.init(0, cpus);
    pd.dynamic_coefficient = 200;
    pd.leakage_coefficient = 50;
    pd.thermal_resistance = 15;
    pd.thermal_capacitance = 120;
    pd.add_state(CapacityState::new(500, 100, 150, 800, 10));
    pd.add_state(CapacityState::new(700, 150, 250, 850, 10));
    pd.add_state(CapacityState::new(1000, 250, 382, 900, 10));
    pd.add_state(CapacityState::new(1200, 350, 480, 950, 10));
    pd.add_state(CapacityState::new(1500, 500, 600, 1000, 10));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 502) - [public]
em_build_synthetic_big_pd

```
pub fn em_build_synthetic_big_pd(cpus: CpuMask) -> PerformanceDomain {
    let mut pd = PerformanceDomain::empty();
    pd.init(1, cpus);
    pd.dynamic_coefficient = 800;
    pd.leakage_coefficient = 200;
    pd.thermal_resistance = 8;
    pd.thermal_capacitance = 80;
    pd.add_state(CapacityState::new(600, 300, 400, 850, 20));
    pd.add_state(CapacityState::new(900, 500, 650, 900, 20));
    pd.add_state(CapacityState::new(1200, 800, 900, 950, 20));
    pd.add_state(CapacityState::new(1500, 1200, 1100, 1000, 20));
    pd.add_state(CapacityState::new(1800, 1800, 1400, 1050, 20));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/em.rs (line 521) - [public]
em_build_synthetic_huge_pd

```
pub fn em_build_synthetic_huge_pd(cpus: CpuMask) -> PerformanceDomain {
    let mut pd = PerformanceDomain::empty();
    pd.init(2, cpus);
    pd.dynamic_coefficient = 1500;
    pd.leakage_coefficient = 400;
    pd.thermal_resistance = 5;
    pd.thermal_capacitance = 50;
    pd.add_state(CapacityState::new(800, 800, 800, 900, 30));
    pd.add_state(CapacityState::new(1200, 1500, 1300, 950, 30));
    pd.add_state(CapacityState::new(1600, 2500, 1800, 1000, 30));
    pd.add_state(CapacityState::new(2000, 4000, 2400, 1050, 30));
    pd.add_state(CapacityState::new(2400, 6000, 3000, 1100, 30));
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 28) - [public] PidState

```
pub struct PidState {
    pub setpoint: i64,
    pub integral: i64,
    pub prev_error: i64,
    pub output: i64,
}

impl PidState {
    pub const fn new(setpoint: i64) -> Self {
        Self {
            setpoint,
            integral: 0,
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 45) - [public] update

```
pub fn update(&mut self, measurement: i64) -> i64 {
    let error = self.setpoint - measurement;
    self.integral = (self.integral + error).clamp(PID_INTEGRAL_MIN, PID_INTEGRAL_MAX);
    let derivative = error - self.prev_error;
    self.prev_error = error;
    self.output = (PID_KP * error + PID_KI * self.integral + PID_KD * derivative) / 100;
    self.output
}

pub fn reset(&mut self) {
    self.integral = 0;
    self.prev_error = 0;
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 54) - [public] reset

```
pub fn reset(&mut self) {
    self.integral = 0;
    self.prev_error = 0;
    self.output = 0;
}

#[repr(C)]
pub struct ThermalZone {
    pub pd_idx: u32,
    pub pid: PidState,
    pub current_temp: AtomicU32,
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 62) - [public] ThermalZone

```
pub struct ThermalZone {
    pub pd_idx: u32,
    pub pid: PidState,
    pub current_temp: AtomicU32,
    pub current_level: AtomicU32,
    pub trip_points: [u32; 5],
    pub hysteresis: u32,
    pub mitigation_active: AtomicBool,
    pub cooldown_time_ms: AtomicU32,
    pub last_mitigation_time_ms: AtomicU64,
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 89) - [public] init

```
pub fn init(&mut self, pd_idx: u32) {
    self.pd_idx = pd_idx;
    self.pid = PidState::new(TEMP_LIGHT_MAX as i64);
    self.current_temp.store(25000, Ordering::Relaxed);
    self.current_level.store(THERMAL_NORMAL, Ordering::Relaxed);
    self.mitigation_active.store(false, Ordering::Relaxed);
    self.cooldown_time_ms.store(100, Ordering::Relaxed);
    self.last_mitigation_time_ms.store(0, Ordering::Relaxed);
}

pub fn update_temp(&self, temp_millideg: u32) {
    self.current_temp.store(temp_millideg, Ordering::Relaxed);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 99) - [public] update_temp

```
pub fn update_temp(&self, temp_millideg: u32) {
    self.current_temp.store(temp_millideg, Ordering::Relaxed);
}

pub fn get_temp(&self) -> u32 {
    self.current_temp.load(Ordering::Acquire)
}

pub fn get_level(&self) -> u32 {
    self.current_level.load(Ordering::Acquire)
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 103) - [public] get_temp

```
pub fn get_temp(&self) -> u32 {
    self.current_temp.load(Ordering::Acquire)
}

pub fn get_level(&self) -> u32 {
    self.current_level.load(Ordering::Acquire)
}

pub fn evaluate_level(&self) -> u32 {
    let temp = self.get_temp();
    let current_level = self.get_level();
    let hyst = self.hysteresis;
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 107) - [public] get_level

```
pub fn get_level(&self) -> u32 {
    self.current_level.load(Ordering::Acquire)
}

pub fn evaluate_level(&self) -> u32 {
    let temp = self.get_temp();
    let current_level = self.get_level();
    let hyst = self.hysteresis;

    let new_level = if temp >= self.trip_points[4] {
        THERMAL_CRITICAL
    } else if temp >= self.trip_points[3] {
        // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 111) - [public] evaluate_level

```
pub fn evaluate_level(&self) -> u32 {
    let temp = self.get_temp();
    let current_level = self.get_level();
    let hyst = self.hysteresis;

    let new_level = if temp >= self.trip_points[4] {
        THERMAL_CRITICAL
    } else if temp >= self.trip_points[3] {
        THERMAL_SEVERE
    } else if temp >= self.trip_points[2] {
        THERMAL_MEDIUM
    } else if temp >= self.trip_points[1] {
        // ... (truncated, see source file for more)
    }
```

kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 148) - [public] compute_pid_output

```
pub fn compute_pid_output(&mut self) -> i64 {
    let temp = self.get_temp() as i64;
    self.pid.update(temp)
}
```

```
#[repr(C)]
pub struct PowerScheduler {
    pub zones: [ThermalZone; 8],
    pub nr_zones: u32,
    pub lock: SpinLock,
    pub eas_enabled: AtomicBool,
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 155) - [public] PowerScheduler

```
pub struct PowerScheduler {
    pub zones: [ThermalZone; 8],
    pub nr_zones: u32,
    pub lock: SpinLock,
    pub eas_enabled: AtomicBool,
    pub thermal_mitigation_enabled: AtomicBool,
    pub global_thermal_level: AtomicU32,
    pub tick_count: AtomicU64,
    pub last_tick_time_ns: AtomicU64,
    pub emergency_shutdown_triggered: AtomicBool,
    pub max_allowed_capacity: [AtomicU32; MAX_CPUS],
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 186) - [public] init

```
pub fn init(&mut self) {
    self.nr_zones = 0;
    self.eas_enabled.store(true, Ordering::Relaxed);
    self.thermal_mitigation_enabled.store(true, Ordering::Relaxed);
    self.global_thermal_level.store(THERMAL_NORMAL, Ordering::Relaxed);
    self.tick_count.store(0, Ordering::Relaxed);
    self.last_tick_time_ns.store(0, Ordering::Relaxed);
    self.emergency_shutdown_triggered.store(false, Ordering::Relaxed);
    for i in 0..MAX_CPUS {
        self.max_allowed_capacity[i].store(u32::MAX, Ordering::Relaxed);
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 199) - [public] register_zone

```
pub fn register_zone(&mut self, pd_idx: u32) -> bool {
    if self.nr_zones as usize >= 8 {
        return false;
    }
    let idx = self.nr_zones as usize;
    self.zones[idx].init(pd_idx);
    self.nr_zones += 1;
    true
}

pub fn get_zone_for_pd(&self, pd_idx: u32) -> Option<&ThermalZone> {
    for i in 0..self.nr_zones as usize {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 209) - [public] get_zone_for_pd

```
pub fn get_zone_for_pd(&self, pd_idx: u32) -> Option<&ThermalZone> {
    for i in 0..self.nr_zones as usize {
        if self.zones[i].pd_idx == pd_idx {
            return Some(&self.zones[i]);
        }
    }
    None
}

pub fn get_zone_for_pd_mut(&mut self, pd_idx: u32) -> Option<&mut ThermalZone> {
    for i in 0..self.nr_zones as usize {
        if self.zones[i].pd_idx == pd_idx {
            // ... (truncated, see source file for more)
        }
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 218) - [public]

get_zone_for_pd_mut

```
pub fn get_zone_for_pd_mut(&mut self, pd_idx: u32) -> Option<&mut ThermalZone> {
    for i in 0..self.nr_zones as usize {
        if self.zones[i].pd_idx == pd_idx {
            return Some(&mut self.zones[i]);
        }
    }
    None
}

pub fn update_tick(&mut self, current_time_ns: u64) {
    let last_time = self.last_tick_time_ns.load(Ordering::Relaxed);
    let delta_ns = current_time_ns.saturating_sub(last_time);
    // ... (truncated, see source file for more)
}
```

kernel_Workspace/kernel/src/cpu/scheduler/power/mod.rs (line 227) - [public] update_tick

```
pub fn update_tick(&mut self, current_time_ns: u64) {
    let last_time = self.last_tick_time_ns.load(Ordering::Relaxed);
    let delta_ns = current_time_ns.saturating_sub(last_time);
    self.last_tick_time_ns.store(current_time_ns, Ordering::Relaxed);
    self.tick_count.fetch_add(1, Ordering::Relaxed);

    if delta_ns < 10_000_000 {
        return;
    }

    let delta_ms = (delta_ns / 1_000_000) as u32;
    let _total_power = unsafe { GLOBAL_ENERGY_MODEL.compute_system_energy(delta_ms) };
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 258) - [public] apply_mitigation

```
pub fn apply_mitigation(&mut self) {
    if !self.thermal_mitigation_enabled.load(Ordering::Relaxed) {
        return;
    }

    for i in 0..self.nr_zones as usize {
        let zone = &mut self.zones[i];
        let level = zone.get_level();
        let pd_idx = zone.pd_idx;

        let pd = unsafe { &GLOBAL_ENERGY_MODEL.domains[pd_idx as usize] };
        let max_cap = pd.max_capacity;
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 308) - [public] get_max_allowed_capacity

```
pub fn get_max_allowed_capacity(&self, cpu: u32) -> u32 {
    if cpu as usize >= MAX_CPUS {
        return 0;
    }
    self.max_allowed_capacity[cpu as usize].load(Ordering::Acquire)
}

pub fn is_eas_enabled(&self) -> bool {
    self.eas_enabled.load(Ordering::Acquire)
}

pub fn enable_eas(&self) {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 315) - [public] is_eas_enabled

```
pub fn is_eas_enabled(&self) -> bool {
    self.eas_enabled.load(Ordering::Acquire)
}

pub fn enable_eas(&self) {
    self.eas_enabled.store(true, Ordering::Release);
}

pub fn disable_eas(&self) {
    self.eas_enabled.store(false, Ordering::Release);
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 319) - [public] enable_eas

```
pub fn enable_eas(&self) {
    self.eas_enabled.store(true, Ordering::Release);
}

pub fn disable_eas(&self) {
    self.eas_enabled.store(false, Ordering::Release);
}

pub fn is_emergency_shutdown(&self) -> bool {
    self.emergency_shutdown_triggered.load(Ordering::Acquire)
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 323) - [public] disable_eas

```
pub fn disable_eas(&self) {
    self.eas_enabled.store(false, Ordering::Release);
}

pub fn is_emergency_shutdown(&self) -> bool {
    self.emergency_shutdown_triggered.load(Ordering::Acquire)
}

pub fn dump_thermal_state(&self) {
    let _ = self.global_thermal_level.load(Ordering::Relaxed);
    let _ = self.nr_zones;
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 327) - [public] is_emergency_shutdown

```
pub fn is_emergency_shutdown(&self) -> bool {
    self.emergency_shutdown_triggered.load(Ordering::Acquire)
}

pub fn dump_thermal_state(&self) {
    let _ = self.global_thermal_level.load(Ordering::Relaxed);
    let _ = self.nr_zones;
}

pub static mut GLOBAL_POWER_SCHEDULER: PowerScheduler = PowerScheduler::empty();

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/mod.rs (line 331) - [public] dump_thermal_state

```
pub fn dump_thermal_state(&self) {
    let _ = self.global_thermal_level.load(Ordering::Relaxed);
    let _ = self.nr_zones;
}

pub static mut GLOBAL_POWER_SCHEDULER: PowerScheduler = PowerScheduler::empty();

pub unsafe fn power_init() {
    GLOBAL_POWER_SCHEDULER.init();
    crate::cpu::scheduler::power::em::em_init();
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/placement.rs (line 14) - [public] CpuSnapshot

```
pub struct CpuSnapshot {
    pub cpu: u32,
    pub pd_idx: u32,
    pub eff_capacity: u32,
    pub max_capacity: u32,
    pub util: u32,
    pub nr_running: u32,
    pub is_big: bool,
    pub is_little: bool,
    pub shares_cache_with_prev: bool,
    pub thermal_pressure: u32,
}
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/placement.rs (line 35) - [public]

DomainSnapshot

```
pub struct DomainSnapshot {
    pub pd_idx: u32,
    pub cpus: [CpuSnapshot; MAX_CPUS],
    pub nr_cpus: u32,
    pub total_util: u32,
    pub max_eff_capacity: u32,
    pub min_energy: u32,
    pub best_cpu: u32,
    pub is_overutilized: bool,
}

impl DomainSnapshot {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/placement.rs (line 54) - [public] EnergyCtx

```
pub struct EnergyCtx {
    pub task: *mut TaskStruct,
    pub task_util: u32,
    pub task_util_est: u32,
    pub target_cpu: u32,
    pub prev_cpu: u32,
    pub env: [DomainSnapshot; 8],
    pub nr_domains: u32,
    pub skip_greedy: bool,
    pub sync_wake: bool,
    pub uclamp_min: u32,
    pub uclamp_max: u32,
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/power/placement.rs (line 77) - [public]

PlacementResult

```
pub struct PlacementResult {
    pub best_cpu: u32,
    pub best_energy: u32,
    pub best_pd_idx: u32,
    pub fallback_cpu: u32,
    pub reason: u32,
}

impl PlacementResult {
    pub const fn empty() -> Self {
        Self { best_cpu: CPU_NONE, best_energy: u32::MAX, best_pd_idx: u32::MAX, fallback_cpu: CPU_NONE, }
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/active.rs (line 13) - [internal]

ActiveBalanceArg

```
struct ActiveBalanceArg {
    src_cpu: u32,
    dst_cpu: u32,
    sd_flags: u32,
}

/// Przenosi (przez `stop_one_cpu`, czyli bezpiecznie wzgl┐dem wsp┐bie┐nego
/// `schedule()` na CPU ┐r┐d┐owym) bie┐ce zadanie najbardziej obci┐onego
/// CPU na `cpu`, je┐li `calc` wskazuje na realn┐ nier┐wnowag┐. Wcze┐niej ta
/// funkcja ┐y┐a (razem z `ActiveBalanceArg`/`active_balance_fn`) w
/// `smp/mod.rs`, mimo ┐e logicznie nale┐y do `balancing` – przeniesione tu,
/// ┐eby `mod active;` w `balancing/mod.rs` w og┐le co┐ zawiera┐.
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/active.rs (line 84) - [internal]

active_balance_is_noop_with_zero_imbalance

```
fn active_balance_is_noop_with_zero_imbalance() {
    let mut sg = SchedGroup::empty();
    sg.group_mask.set(3);
    let mut sd = crate::cpu::scheduler::smp::topology::SchedDomain::empty();
    let calc = LoadCalculation::empty();
    unsafe {
        active_balance(0, &mut sd, &mut sg, ptr::null_mut(), &calc);
    }
}

#[test]
fn active_balance_is_noop_when_target_equals_busiest() {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/active.rs (line 95) - [internal]

active_balance_is_noop_when_target_equals_busiest

```
fn active_balance_is_noop_when_target_equals_busiest() {
    let mut sg = SchedGroup::empty();
    sg.group_mask.set(2);
    let mut sd = crate::cpu::scheduler::smp::topology::SchedDomain::empty();
    let mut calc = LoadCalculation::empty();
    calc.imbalance = 10;
    unsafe {
        // busiest == cpu -> nie ma sensu "aktywnie" ■ci■ga■ zadania z
        // samego siebie na siebie.
        active_balance(2, &mut sd, &mut sg, ptr::null_mut(), &calc);
    }
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 17) - [public]

GroupType

```
pub enum GroupType {
    FullyIdle,
    HasIdle,
    GroupAsym,
    GroupMisfit,
    GroupOverloaded,
    GroupSmt,
    GroupImbalanced,
}

#[repr(C)]
pub struct LoadCalculation {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 28) - [public]

LoadCalculation

```
pub struct LoadCalculation {
    pub src_load: u64,
    pub dst_load: u64,
    pub src_util: u64,
    pub dst_util: u64,
    pub src_capacity: u64,
    pub dst_capacity: u64,
    pub imbalance: u64,
    pub nr_moving: u32,
    pub env_task_util: u32,
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 244) - [internal]
load_calculation_defaults

```
fn load_calculation_defaults() {
    let calc = LoadCalculation::empty();
    assert_eq!(calc.imbalance, 0);
    assert_eq!(calc.src_util, 0);
}

#[test]
fn group_classification_idle() {
    let mut sg = SchedGroup::empty();
    sg.group_util = 0;
    unsafe {
        assert_eq!(classify_group(&mut sg), GroupType::FullyIdle);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 251) - [internal]
group_classification_idle

```
fn group_classification_idle() {
    let mut sg = SchedGroup::empty();
    sg.group_util = 0;
    unsafe {
        assert_eq!(classify_group(&mut sg), GroupType::FullyIdle);
    }
}

#[test]
fn group_classification_has_idle_beats_overloaded() {
    let mut sg = SchedGroup::empty();
    sg.group_util = 500;
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 260) - [internal]
group_classification_has_idle_beats_overloaded

```
fn group_classification_has_idle_beats_overloaded() {
    let mut sg = SchedGroup::empty();
    sg.group_util = 500;
    sg.idle_cpus.set(3);
    sg.group_overloaded = true;
    unsafe {
        assert_eq!(classify_group(&mut sg), GroupType::HasIdle);
    }
}

#[test]
fn group_classification_overloaded_when_busy_and_full() {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 271) - [internal]
group_classification_overloaded_when_busy_and_full

```
fn group_classification_overloaded_when_busy_and_full() {
    let mut sg = SchedGroup::empty();
    sg.group_util = 900;
    sg.group_overloaded = true;
    unsafe {
        assert_eq!(classify_group(&mut sg), GroupType::GroupOverloaded);
    }
}

#[test]
fn calculate_imbalance_moves_load_toward_equilibrium() {
    let mut busiest = SchedGroup::empty();
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 281) - [internal]
calculate_imbalance_moves_load_toward_equilibrium

```
fn calculate_imbalance_moves_load_toward_equilibrium() {
    let mut busiest = SchedGroup::empty();
    busiest.group_capacity = 1024;
    busiest.group_util = 900;

    let mut local = SchedGroup::empty();
    local.group_capacity = 1024;
    local.group_util = 100;

    let mut calc = LoadCalculation::empty();
    unsafe {
        calculate_imbalance(&mut calc, &mut busiest, &mut local);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/calculate.rs (line 304) - [internal]
calculate_imbalance_is_zero_for_balanced_groups

```
fn calculate_imbalance_is_zero_for_balanced_groups() {
    let mut busiest = SchedGroup::empty();
    busiest.group_capacity = 1024;
    busiest.group_util = 500;

    let mut local = SchedGroup::empty();
    local.group_capacity = 1024;
    local.group_util = 500;

    let mut calc = LoadCalculation::empty();
    unsafe {
        calculate_imbalance(&mut calc, &mut busiest, &mut local);
    }
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 18) - [public]
LoadBalanceEnv

```
pub struct LoadBalanceEnv {
    pub sd: *mut SchedDomain,
    pub src_cpu: u32,
    pub dst_cpu: u32,
    pub flags: u32,
    pub loop_max: u32,
    pub loop_done: u32,
    pub failed: u32,
}

impl LoadBalanceEnv {
    pub const fn new(sd: *mut SchedDomain, src_cpu: u32, dst_cpu: u32) -> Self {
        // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 41) - [public] set_flag

```
pub fn set_flag(&mut self, flag: u32) {
    self.flags |= flag;
}

pub fn clear_flag(&mut self, flag: u32) {
    self.flags &= !flag;
}

pub fn has_flag(&self, flag: u32) -> bool {
    self.flags & flag != 0
}

// ... (truncated, see source file for more)
```


kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 45) - [public] clear_flag

```
pub fn clear_flag(&mut self, flag: u32) {
    self.flags &= !flag;
}

pub fn has_flag(&self, flag: u32) -> bool {
    self.flags & flag != 0
}

/// Czy przebieg powinien si■ zatrzyma■ (osi■gni■to limit prób
/// przeniesienia zada■ w tej iteracji `load_balance`).
pub fn should_stop(&self) -> bool {
    self.loop_done >= self.loop_max
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 49) - [public] has_flag

```
pub fn has_flag(&self, flag: u32) -> bool {
    self.flags & flag != 0
}

/// Czy przebieg powinien si■ zatrzyma■ (osi■gni■to limit prób
/// przeniesienia zada■ w tej iteracji `load_balance`).
pub fn should_stop(&self) -> bool {
    self.loop_done >= self.loop_max
}

pub fn record_attempt(&mut self) {
    self.loop_done += 1;
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 55) - [public] should_stop

*/// Czy przebieg powinien si■ zatrzyma■ (osi■gni■to limit prób
/// przeniesienia zada■ w tej iteracji `load\_balance`).*

```
pub fn should_stop(&self) -> bool {
    self.loop_done >= self.loop_max
}

pub fn record_attempt(&mut self) {
    self.loop_done += 1;
}

pub fn record_failure(&mut self) {
    self.failed += 1;
    self.set_flag(LBF_SOME_PINNED);
}
// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 59) - [public] record_attempt

```
pub fn record_attempt(&mut self) {
    self.loop_done += 1;
}

pub fn record_failure(&mut self) {
    self.failed += 1;
    self.set_flag(LBF_SOME_PINNED);
}

pub fn reset_for_next_iteration(&mut self) {
    self.loop_done = 0;
    self.clear_flag(LBF_NEED_BREAK);
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 63) - [public]

record_failure

```
pub fn record_failure(&mut self) {
    self.failed += 1;
    self.set_flag(LBF_SOME_PINNED);
}

pub fn reset_for_next_iteration(&mut self) {
    self.loop_done = 0;
    self.clear_flag(LBF_NEED_BREAK);
}

#[cfg(test)]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 68) - [public]

reset_for_next_iteration

```
pub fn reset_for_next_iteration(&mut self) {
    self.loop_done = 0;
    self.clear_flag(LBF_NEED_BREAK);
}

#[cfg(test)]
mod tests {
    use super::*;
    use core::ptr;

    fn env() -> LoadBalanceEnv {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 79) - [internal] env

```
fn env() -> LoadBalanceEnv {
    LoadBalanceEnv::new(ptr::null_mut(), 0, 1)
}

#[test]
fn flags_set_clear_roundtrip() {
    let mut e = env();
    assert!(!e.has_flag(LBF_ALL_PINNED));
    e.set_flag(LBF_ALL_PINNED);
    assert!(e.has_flag(LBF_ALL_PINNED));
    e.clear_flag(LBF_ALL_PINNED);
    assert!(!e.has_flag(LBF_ALL_PINNED));
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 84) - [internal]

flags_set_clear_roundtrip

```
fn flags_set_clear_roundtrip() {
    let mut e = env();
    assert!(!e.has_flag(LBF_ALL_PINNED));
    e.set_flag(LBF_ALL_PINNED);
    assert!(e.has_flag(LBF_ALL_PINNED));
    e.clear_flag(LBF_ALL_PINNED);
    assert!(!e.has_flag(LBF_ALL_PINNED));
}

#[test]
fn should_stop_once_loop_max_reached() {
    let mut e = env();
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 94) - [internal]
should_stop_once_loop_max_reached

```
fn should_stop_once_loop_max_reached() {
    let mut e = env();
    e.loop_max = 3;
    assert!(!e.should_stop());
    e.record_attempt();
    e.record_attempt();
    assert!(!e.should_stop());
    e.record_attempt();
    assert!(e.should_stop());
}

#[test]
// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 106) - [internal]
record_failure_sets_some_pinned

```
fn record_failure_sets_some_pinned() {
    let mut e = env();
    assert!(!e.has_flag(LBF_SOME_PINNED));
    e.record_failure();
    assert_eq!(e.failed, 1);
    assert!(e.has_flag(LBF_SOME_PINNED));
}

#[test]
fn reset_clears_loop_counter_and_need_break() {
    let mut e = env();
    e.record_attempt();
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/balancing/env.rs (line 115) - [internal]
reset_clears_loop_counter_and_need_break

```
fn reset_clears_loop_counter_and_need_break() {
    let mut e = env();
    e.record_attempt();
    e.set_flag(LBF_NEED_BREAK);
    e.reset_for_next_iteration();
    assert_eq!(e.loop_done, 0);
    assert!(!e.has_flag(LBF_NEED_BREAK));
}
}
```

kernel_workspace/kernel/src/cpu/scheduler/smp/ipi.rs (line 8) - [public] IpiType

```
pub enum IpiType {
    Resched = 0,
    CallFunction = 1,
    MigrationStop = 2,
    TlbShootdown = 3,
    NmiBacktrace = 4,
}

pub const IPI_COUNTER_INIT: AtomicU64 = AtomicU64::new(0);
pub static IPI_RESCHED_COUNT: [AtomicU64; MAX_CPUS] = [IPI_COUNTER_INIT; MAX_CPUS];
pub static IPI_CALL_COUNT: [AtomicU64; MAX_CPUS] = [IPI_COUNTER_INIT; MAX_CPUS];

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/ipi.rs (line 75) - [internal]

migration_stop_ipi_increments_dedicated_counter

```
fn migration_stop_ipi_increments_dedicated_counter() {
    let cpu = 60usize;
    let before = IPI_CALL_COUNT[cpu].load(Ordering::Relaxed);
    unsafe {
        send_migration_stop_ipi(cpu as u32);
    }
    assert_eq!(IPI_CALL_COUNT[cpu].load(Ordering::Relaxed), before + 1);
}

#[test]
fn out_of_range_cpu_is_a_safe_noop() {
    unsafe {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/smp/ipi.rs (line 85) - [internal]

out_of_range_cpu_is_a_safe_noop

```
fn out_of_range_cpu_is_a_safe_noop() {
    unsafe {
        send_migration_stop_ipi(MAX_CPUS as u32 + 5);
    }
}

#[test]
fn handle_ipi_entry_ignores_unimplemented_types_without_panicking() {
    unsafe {
        handle_ipi_entry(0, IpiType::Resched);
        handle_ipi_entry(0, IpiType::CallFunction);
        handle_ipi_entry(0, IpiType::TlbShootdown);
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/smp/ipi.rs (line 92) - [internal]

handle_ipi_entry_ignores_unimplemented_types_without_panicking

```
fn handle_ipi_entry_ignores_unimplemented_types_without_panicking() {
    unsafe {
        handle_ipi_entry(0, IpiType::Resched);
        handle_ipi_entry(0, IpiType::CallFunction);
        handle_ipi_entry(0, IpiType::TlbShootdown);
        handle_ipi_entry(0, IpiType::NmiBacktrace);
    }
}

}
```

kernel_workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 10) - [public]

StopperState

```
pub enum StopperState {
    Idle = 0,
    Preparing = 1,
    Running = 2,
    Done = 3,
    Aborted = 4,
}

#[repr(C)]
pub struct StopperWork {
    pub state: AtomicU32,
    pub fn_ptr: AtomicPtr<core::ffi::c_void>, // ZMIENIONE
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 19) - [public]

StopperWork

```
pub struct StopperWork {
    pub state: AtomicU32,
    pub fn_ptr: AtomicPtr<core::ffi::c_void>, // ZMIENIONE
    pub arg: AtomicPtr<core::ffi::c_void>,    // ZMIENIONE
    pub ret: AtomicU32,
    pub started: AtomicBool,
    pub completed: AtomicBool,
}
```

```
impl StopperWork {
    pub const fn empty() -> Self {
        // ... (truncated, see source file for more)
    }
}
```

kernel_workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 41) - [public]

get_state

```
pub fn get_state(&self) -> StopperState {
    match self.state.load(Ordering::Acquire) {
        0 => StopperState::Idle,
        1 => StopperState::Preparing,
        2 => StopperState::Running,
        3 => StopperState::Done,
        4 => StopperState::Aborted,
        _ => StopperState::Idle,
    }
}
```

```
pub fn set_state(&self, new_state: StopperState) {
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 52) - [public]

set_state

```
pub fn set_state(&self, new_state: StopperState) {
    self.state.store(new_state as u32, Ordering::Release);
}
```

```
pub type StopperFn = unsafe extern "C" fn(*mut core::ffi::c_void) -> u32;
```

```
#[repr(C)]
pub struct CpuStopper {
    pub cpu: u32,
    pub lock: SpinLock,
    pub work: StopperWork,
    // ... (truncated, see source file for more)
}
```

kernel_workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 60) - [public]

CpuStopper

```
pub struct CpuStopper {
    pub cpu: u32,
    pub lock: SpinLock,
    pub work: StopperWork,
    pub stopper_task: AtomicPtr<TaskStruct>,
    pub enabled: AtomicBool,
    pub park_task: AtomicBool,
    pub nr_stops: AtomicU32,
}

impl CpuStopper {
    pub const fn empty() -> Self {
        // ... (truncated, see source file for more)
    }
}
```

kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 83) - [public] init

```
pub fn init(&mut self, cpu: u32) {
    self.cpu = cpu;
    self.enabled.store(true, Ordering::Release);
}

pub fn queue_work(&self, func: StopperFn, arg: *mut core::ffi::c_void) -> bool {
    let _guard = self.lock.lock_irqsave_guard();

    if self.work.get_state() != StopperState::Idle {
        return false;
    }

    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 88) - [public] queue_work

```
pub fn queue_work(&self, func: StopperFn, arg: *mut core::ffi::c_void) -> bool {
    let _guard = self.lock.lock_irqsave_guard();

    if self.work.get_state() != StopperState::Idle {
        return false;
    }

    self.work.fn_ptr.store(func as *mut core::ffi::c_void, Ordering::Relaxed);
    self.work.arg.store(arg, Ordering::Relaxed);
    self.work.ret.store(0, Ordering::Relaxed);
    self.work.started.store(false, Ordering::Relaxed);
    self.work.completed.store(false, Ordering::Relaxed);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 108) - [internal] wake_stopper_task

```
fn wake_stopper_task(&self) {
    let task_ptr = self.stopper_task.load(Ordering::Acquire);
    if task_ptr.is_null() { return; }

    unsafe {
        let task = &*task_ptr;
        if task.state() == TaskState::Interruptible || task.state() == TaskState::Uninterruptible {
            let _ = task.wake_up();
        }
    }
}

// ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 120) - [public] execute_work

```
pub fn execute_work(&self) {
    let _guard = self.lock.lock_irqsave_guard();

    if self.work.get_state() != StopperState::Preparing {
        return;
    }

    self.work.set_state(StopperState::Running);
    self.work.started.store(true, Ordering::Release);

    let func_ptr = self.work.fn_ptr.load(Ordering::Relaxed);
    let arg_ptr = self.work.arg.load(Ordering::Relaxed);
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 146) - [public]
wait_for_completion

```
pub fn wait_for_completion(&self) -> u32 {  
    loop {  
        let state = self.work.get_state();  
        if state == StopperState::Done || state == StopperState::Aborted {  
            break;  
        }  
        core::hint::spin_loop();  
    }  
  
    let ret = self.work.ret.load(Ordering::Acquire);  
  
    let _guard = self.lock.lock_irqsave_guard();  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 166) - [public]
StopperRegistry

```
pub struct StopperRegistry {  
    pub stoppers: [CpuStopper; MAX_CPUS],  
}  
  
impl StopperRegistry {  
    pub const fn empty() -> Self {  
        const EMPTY_STOPPER: CpuStopper = CpuStopper::empty();  
        Self {  
            stoppers: [EMPTY_STOPPER; MAX_CPUS],  
        }  
    }  
  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 178) - [public]
init_cpu

```
pub fn init_cpu(&mut self, cpu: u32) {  
    if (cpu as usize) < MAX_CPUS {  
        self.stoppers[cpu as usize].init(cpu);  
    }  
}  
  
pub fn get(&self, cpu: u32) -> Option<&CpuStopper> {  
    if (cpu as usize) < MAX_CPUS {  
        Some(&self.stoppers[cpu as usize])  
    } else {  
        None  
    }  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 184) - [public] get

```
pub fn get(&self, cpu: u32) -> Option<&CpuStopper> {  
    if (cpu as usize) < MAX_CPUS {  
        Some(&self.stoppers[cpu as usize])  
    } else {  
        None  
    }  
}  
}  
  
pub static mut GLOBAL_STOPPERS: StopperRegistry = StopperRegistry::empty();  
  
pub unsafe fn stopper_init() {  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 280) - [internal]
stopper_state_transitions

```
fn stopper_state_transitions() {
    let work = StopperWork::empty();
    assert_eq!(work.get_state(), StopperState::Idle);

    work.set_state(StopperState::Preparing);
    assert_eq!(work.get_state(), StopperState::Preparing);

    work.set_state(StopperState::Running);
    assert_eq!(work.get_state(), StopperState::Running);

    work.set_state(StopperState::Done);
    assert_eq!(work.get_state(), StopperState::Done);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 295) - [internal]
stopper_registry_bounds_check

```
fn stopper_registry_bounds_check() {
    let reg = StopperRegistry::empty();
    assert!(reg.get(0).is_some());
    assert!(reg.get(MAX_CPUS as u32).is_none());
}

#[test]
fn queue_execute_and_wait_roundtrip_returns_value_from_function() {
    let mut stopper = CpuStopper::empty();
    stopper.init(7);
    assert!(stopper.queue_work(dummy_stopper_fn, ptr::null_mut()));
    assert_eq!(stopper.work.get_state(), StopperState::Preparing);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 302) - [internal]
queue_execute_and_wait_roundtrip_returns_value_from_function

```
fn queue_execute_and_wait_roundtrip_returns_value_from_function() {
    let mut stopper = CpuStopper::empty();
    stopper.init(7);
    assert!(stopper.queue_work(dummy_stopper_fn, ptr::null_mut()));
    assert_eq!(stopper.work.get_state(), StopperState::Preparing);

    stopper.execute_work();
    assert_eq!(stopper.work.get_state(), StopperState::Done);

    let ret = stopper.wait_for_completion();
    assert_eq!(ret, 0);
    assert_eq!(stopper.work.get_state(), StopperState::Idle);
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 317) - [internal]
queue_work_rejects_a_second_job_while_busy

```
fn queue_work_rejects_a_second_job_while_busy() {
    let mut stopper = CpuStopper::empty();
    stopper.init(7);
    assert!(stopper.queue_work(dummy_stopper_fn, ptr::null_mut()));
    assert!(!stopper.queue_work(dummy_stopper_fn, ptr::null_mut()));
}

#[test]
fn execute_work_without_queued_job_is_a_noop() {
    let mut stopper = CpuStopper::empty();
    stopper.init(7);
    stopper.execute_work();
    // ... (truncated, see source file for more)
```


kernel_workspace/kernel/src/cpu/scheduler/smp/migration/stopper.rs (line 325) - [internal]
execute_work_without_queued_job_is_a_noop

```
fn execute_work_without_queued_job_is_a_noop() {  
    let mut stopper = CpuStopper::empty();  
    stopper.init(7);  
    stopper.execute_work();  
    assert_eq!(stopper.work.get_state(), StopperState::Idle);  
}  
}
```

kernel_workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 24) - [public] CacheLevel

```
pub enum CacheLevel {  
    L1Data = 1,  
    L1Instruction = 2,  
    L2Unified = 3,  
    L3Unified = 4,  
}  
  
#[repr(C)]  
#[derive(Debug, Clone, Copy)]  
pub struct CacheTopology {  
    pub level: CacheLevel,  
    pub size_kb: u32,  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 33) - [public] CacheTopology

```
pub struct CacheTopology {  
    pub level: CacheLevel,  
    pub size_kb: u32,  
    pub line_size: u32,  
    pub shared_cpus: CpuMask,  
}  
  
#[repr(C)]  
pub struct NumaNode {  
    pub node_id: u32,  
    pub cpus: CpuMask,  
    pub memory_start: u64,  
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 41) - [public] NumaNode

```
pub struct NumaNode {  
    pub node_id: u32,  
    pub cpus: CpuMask,  
    pub memory_start: u64,  
    pub memory_size: u64,  
    pub distance: [u8; MAX_NUMA_NODES],  
}  
  
impl NumaNode {  
    pub const fn empty() -> Self {  
        Self {  
            node_id: u32::MAX,  
            // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 60) - [public] distance_to

```
pub fn distance_to(&self, other_node: u32) -> u8 {  
    if other_node as usize >= MAX_NUMA_NODES { return 255; }  
    self.distance[other_node as usize]  
}  
  
#[repr(C)]  
pub struct SchedGroup {  
    pub group_mask: CpuMask,  
    pub group_weight: u32,  
    pub group_capacity: u32,  
    pub group_util: u32,  
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 67) - [public] SchedGroup

```
pub struct SchedGroup {
    pub group_mask: CpuMask,
    pub group_weight: u32,
    pub group_capacity: u32,
    pub group_util: u32,
    pub group_misfit_task: bool,
    pub group_overloaded: bool,
    pub next: *mut SchedGroup,
    pub idle_cpus: CpuMask,
    pub rq_avg_load: u64,
}
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 96) - [public] SchedDomain

```
pub struct SchedDomain {
    pub level: u32,
    pub flags: u32,
    pub span: CpuMask,
    pub child: *mut SchedDomain,
    pub parent: *mut SchedDomain,
    pub groups: *mut SchedGroup,
    pub nr_groups: u32,
    pub cache_nice_tries: u32,
    pub busy_idx: u32,
    pub idle_idx: u32,
    pub newidle_idx: u32,
    // ... (truncated, see source file for more)
```

kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 137) - [public] has_flag

```
pub fn has_flag(&self, flag: u32) -> bool {
    (self.flags & flag) != 0
}
```

```
pub fn is_numa(&self) -> bool {
    self.has_flag(SD_NUMA)
}
```

```
pub fn shares_cache(&self) -> bool {
    self.has_flag(SD_SHARE_PKG_RESOURCES)
}
```

// ... (truncated, see source file for more)

kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 141) - [public] is_numa

```
pub fn is_numa(&self) -> bool {
    self.has_flag(SD_NUMA)
}
```

```
pub fn shares_cache(&self) -> bool {
    self.has_flag(SD_SHARE_PKG_RESOURCES)
}
```

```
pub fn is_smt(&self) -> bool {
    self.has_flag(SD_SHARE_CPUCAPACITY)
}
```

```
}
// ... (truncated, see source file for more)
```

```

kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 145) - [public] shares_cache
    pub fn shares_cache(&self) -> bool {
        self.has_flag(SD_SHARE_PKG_RESOURCES)
    }

    pub fn is_smt(&self) -> bool {
        self.has_flag(SD_SHARE_CPUCAPACITY)
    }
}

#[repr(C)]
pub struct CpuTopology {
    pub cpu_id: u32,
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 149) - [public] is_smt
    pub fn is_smt(&self) -> bool {
        self.has_flag(SD_SHARE_CPUCAPACITY)
    }
}

#[repr(C)]
pub struct CpuTopology {
    pub cpu_id: u32,
    pub package_id: u32,
    pub core_id: u32,
    pub smt_id: u32,
    pub numa_node_id: u32,
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 155) - [public] CpuTopology
pub struct CpuTopology {
    pub cpu_id: u32,
    pub package_id: u32,
    pub core_id: u32,
    pub smt_id: u32,
    pub numa_node_id: u32,
    pub l1d_cache: CacheTopology,
    pub l2_cache: CacheTopology,
    pub l3_cache: CacheTopology,
    pub thread_siblings: CpuMask,
    pub core_siblings: CpuMask,
    pub book_siblings: CpuMask,
    // ... (truncated, see source file for more)
}

kernel_Workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 192) - [public] TopologyState
pub struct TopologyState {
    pub cpus: [CpuTopology; MAX_CPUS],
    pub nodes: [NumaNode; MAX_NUMA_NODES],
    pub nr_nodes: u32,
    pub sd_pool: [SchedDomain; MAX_CPUS * MAX_SCHED_DOMAIN_LEVELS],
    pub sg_pool: [SchedGroup; MAX_CPUS * MAX_SCHED_DOMAIN_LEVELS],
    pub sd_used: AtomicU32,
    pub sg_used: AtomicU32,
    pub per_cpu_domains: [*mut SchedDomain; MAX_CPUS],
    pub lock: SpinLock,
    pub initialized: AtomicBool,
}

// ... (truncated, see source file for more)

```

```

kernel_workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 226) - [public] alloc_domain
    pub fn alloc_domain(&mut self) -> Option<&mut SchedDomain> {
        let idx = self.sd_used.load(Ordering::Relaxed) as usize;
        if idx >= self.sd_pool.len() { return None; }
        self.sd_used.fetch_add(1, Ordering::Relaxed);
        self.sd_pool[idx] = SchedDomain::empty();
        Some(&mut self.sd_pool[idx])
    }

    pub fn alloc_group(&mut self) -> Option<&mut SchedGroup> {
        let idx = self.sg_used.load(Ordering::Relaxed) as usize;
        if idx >= self.sg_pool.len() { return None; }
        self.sg_used.fetch_add(1, Ordering::Relaxed);
        // ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 234) - [public] alloc_group
    pub fn alloc_group(&mut self) -> Option<&mut SchedGroup> {
        let idx = self.sg_used.load(Ordering::Relaxed) as usize;
        if idx >= self.sg_pool.len() { return None; }
        self.sg_used.fetch_add(1, Ordering::Relaxed);
        self.sg_pool[idx] = SchedGroup::empty();
        Some(&mut self.sg_pool[idx])
    }
}

pub static mut GLOBAL_TOPOLOGY: TopologyState = TopologyState::empty();

pub unsafe fn topology_init() {
    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 393) - [internal]
topology_alloc_domains_respects_limits
    fn topology_alloc_domains_respects_limits() {
        unsafe {
            let mut state = TopologyState::empty();
            let mut count = 0;
            while state.alloc_domain().is_some() {
                count += 1;
            }
            assert_eq!(count, MAX_CPUS * MAX_SCHED_DOMAIN_LEVELS);
            assert!(state.alloc_domain().is_none());
        }
    }

    // ... (truncated, see source file for more)

kernel_workspace/kernel/src/cpu/scheduler/smp/topology.rs (line 406) - [internal]
sched_domain_flags_logic
    fn sched_domain_flags_logic() {
        let mut sd = SchedDomain::empty();
        sd.flags = SD_LOAD_BALANCE | SD_SHARE_CPUCAPACITY;
        assert!(sd.has_flag(SD_LOAD_BALANCE));
        assert!(sd.is_smt());
        assert!(!sd.is_numa());
        assert!(!sd.shares_cache());
    }
}

```

kernel_workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 5) - [internal]
make_idle

```
fn make_idle(pid: u64, cpu: u32) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, SchedPolicy::Idle, 0);
    let _ = cpu;
    t
}

fn make_task(pid: u64, policy: SchedPolicy, nice: i8) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, policy, nice);
    t
}

// ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 12) - [internal]
make_task

```
fn make_task(pid: u64, policy: SchedPolicy, nice: i8) -> TaskStruct {
    let mut t = TaskStruct::blank();
    t.init_test_stub(pid, policy, nice);
    t
}

fn ptr_of(t: &mut TaskStruct) -> *mut TaskStruct {
    t as *mut TaskStruct
}

#[test]
fn select_task_rq_returns_preferred_cpu_when_idle() {
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 18) - [internal]
ptr_of

```
fn ptr_of(t: &mut TaskStruct) -> *mut TaskStruct {
    t as *mut TaskStruct
}

#[test]
fn select_task_rq_returns_preferred_cpu_when_idle() {
    let mut idle0 = make_idle(100, 0);
    let mut idle1 = make_idle(101, 1);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));
    rq0.bind_idle_task();
    rq1.bind_idle_task();
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 23) - [internal]
select_task_rq_returns_preferred_cpu_when_idle

```
fn select_task_rq_returns_preferred_cpu_when_idle() {
    let mut idle0 = make_idle(100, 0);
    let mut idle1 = make_idle(101, 1);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));
    rq0.bind_idle_task();
    rq1.bind_idle_task();

    let mut t = make_task(1, SchedPolicy::Normal, 0);
    t.se.last_cpu = 1;

    let registry: [*mut RunQueue; 2] = [&mut rq0, &mut rq1];
    // ... (truncated, see source file for more)
```

kernel_workspace/kernel/src/cpu/scheduler/tests/test_balance_math.rs (line 42) - [internal]

select_task_rq_falls_back_to_least_loaded

```
fn select_task_rq_falls_back_to_least_loaded() {
    let mut idle0 = make_idle(100, 0);
    let mut idle1 = make_idle(101, 1);
    let mut rq0 = RunQueue::new(0, ptr_of(&mut idle0));
    let mut rq1 = RunQueue::new(1, ptr_of(&mut idle1));
    rq0.bind_idle_task();
    rq1.bind_idle_task();

    let mut filler = make_task(2, SchedPolicy::Normal, 0);
    unsafe {
        let f0 = rq0.lock.lock_irqsave();
        rq0.enqueue_task(ptr_of(&mut filler), EnqueueFlags::ENQUEUE_NEW);
        // ... (truncated, see source file for more)
    }
}
```

(List truncated to 4000 entries to keep the document size manageable.)